ARCH LINUX 分册教材

第三册

系统管理与内核

文件系统、存储、内核、虚拟化与系统维护

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

装好系统、能开机、能装包，只是把 Arch“立起来”。真正消耗时间的是之后那些问题：根分区快满了怎么腾挪，内核升级后网卡失灵怎么办，笔记本续航为什么只有两小时，硬盘读不出来时怎样先把数据抢回来。这些都没有一条命令能包办，靠的是对“系统里到底在发生什么”的心智模型。

这一册要解决的问题

本册处理一台机器从“能跑”到“跑得住”的中间层：磁盘怎么分区与挂载、选哪种文件系统；内核与它的模块、参数、initramfs 是什么关系；服务与设备文件由谁管；断电、误删之后如何恢复；以及在同一台机器上再跑几个系统。

读完应当能做到：读懂分区表与挂载关系，为不同用途选文件系统；用逻辑卷与软 RAID 组织存储；独立安装、升级、回退内核，并为外置驱动打包模块；判断故障出在硬件、驱动还是配置；系统起不来时进救援环境修回；在 KVM/QEMU 或容器里开出第二套环境；做出一份真能恢复的备份。

在九册中的位置

第一册回答“Arch 是什么、为什么这样设计”，第二册把系统装起来，并教会你用 pacman 与打包体系补齐软件。本册接手的是已经在自己跑的机器——第二册交付一个起点，这里交付维护它的能力。

往后看，第四册的网络与服务建立在本册的服务管理、日志与存储之上；第五册的硬件排障与第六册的图形栈，都要从内核参数、模块与设备文件出发；第九册的加密存储与沙箱，底层是这里的块设备、命名空间与 cgroups。本册既承上，也是后几册共同踩的地基。

推荐的阅读顺序

先读系统管理那一章，把设备、日志与调试手段立起来；再进文件系统与存储，这两块是日常维护的高频区；然后才是内核，它更抽象，也更容易一动手就让系统起不来。系统恢复与电源管理适合按问题驱动去读，真出了事再翻。

虚拟化的几章自成一组，可以整块后置。压缩归档、FUSE、光盘、备份、同步这些专题篇幅很短，当手册随用随查；末尾几章各只有一篇，跳读不影响理解。

前置知识与常见误区

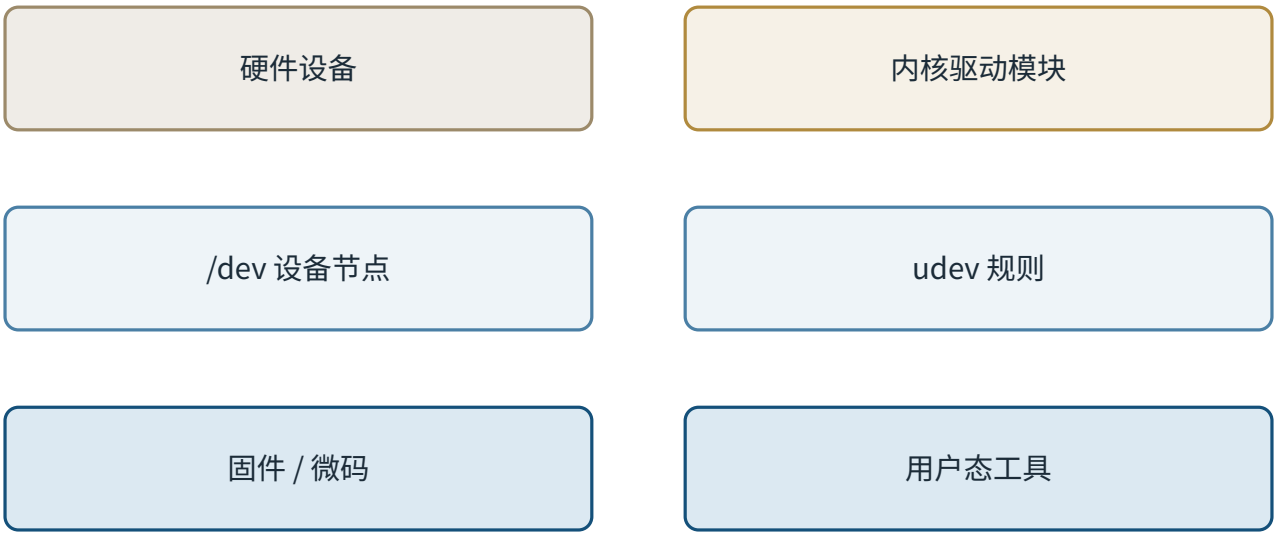
前提是能用 shell 干活、会改配置文件，知道分区与挂载点是什么、`/etc/fstab` 管什么，也大致知道 systemd 用单元文件管服务。手边最好有一台可以随便折腾的虚拟机或备用机。

几个常见的误区值得先说：快照不等于备份，同盘上的快照会跟着盘一起没；内核与 initramfs 是两件事，改了模块却忘了重新生成后者，重启就进不了系统；chroot 只对“系统还在盘上”的情况有用，分区表或文件系统本身崩了，得先修那一层；逻辑卷、RAID 与快照不是同一层的三个兄弟；虚拟机跑完整内核，容器共享宿主内核，只靠命名空间与 cgroups 隔离。

最后一条不是误区而是提醒：分区、内核、启动这三样，动手之前先想好怎么退回去。

硬件与驱动

从固件到用户态



硬件不工作时按「编译进内核 → 模块 → 固件 → udev → 用户态」逐层排查

硬件不工作时逐层定位的五步路径

目　录

第 1 章　文件系统（21 节）

1.1 Partclone　1.2 Cfdisk　1.3 Overlay 文件系统　1.4 Yabsnap　1.5 Timeshift　1.6 NTFS　1.7 FAT　1.8 Tmpfs　1.9 Fsck　1.10 F2FS　1.11 Swap　1.12 ZFS/虚拟磁盘　1.13 Bcachefs　1.14 Ext4　1.15 Autofs　1.16 Bcache　1.17 XFS　1.18 Snapper　1.19 文件系统　1.20 Btrfs　1.21 ZFS

第 2 章　内核（15 节）

2.1 内核热补丁　2.2 Linux-ck　2.3 DKMS　2.4 内核模块打包指南　2.5 SystemTap　2.6 Zswap　2.7 内核/Arch 构建系统　2.8 Zram　2.9 PAM　2.10 内核模块　2.11 内核参数　2.12 内核级显示模式设置　2.13 内核/传统编译　2.14 内核　2.15 Sysctl

第 3 章　系统管理（15 节）

3.1 IPMI　3.2 D-Bus　3.3 Preload　3.4 自动启动　3.5 Scx-scheds　3.6 调试　3.7 Anything-sync-daemon　3.8 设备文件　3.9 Windows PE　3.10 Systemd/定时器　3.11 默认应用程序　3.12 环境变量　3.13 系统维护　3.14 Cron　3.15 Systemd/用户

第 4 章　电源管理（11 节）

4.1 ACPI 模块　4.2 Powertop　4.3 Display Power Management Signaling　4.4 TLP　4.5 电源管理/唤醒触发器　4.6 Acpid　4.7 DSDT　4.8 Pm-utils　4.9 Wake-on-LAN　4.10 电源管理　4.11 电源管理/挂起与休眠

第 5 章　系统恢复（11 节）

5.1 System-tar-and-restore　5.2 重置 root 密码　5.3 使用 LVM 为根文件系统创建快照　5.4 Chroot　5.5 迁移到新硬件　5.6 Fdisk　5.7 文件恢复　5.8 Parted　5.9 GPT fdisk　5.10 常规故障排除　5.11 分区

第 6 章　虚拟化（11 节）

6.1 User-mode Linux　6.2 VPS 上的 Arch Linux　6.3 适用于 AWS 的 Arch Linux AMI　6.4 Vagrant　6.5 Cloud-init　6.6 Incus　6.7 控制组　6.8 Intel GVT-g　6.9 Kubernetes　6.10 Libvirt　6.11 使用 OVMF 直通 PCI

第 7 章　存储（9 节）

7.1 ISCSI　7.2 ISCSI/LIO　7.3 USB 存储设备　7.4 USB/IP　7.5 Open-iSCSI　7.6 Hdparm　7.7 固态硬盘/存储单元擦除　7.8 先进格式化　7.9 固态硬盘

第 8 章　虚拟机管理（6 节）

8.1 KVM　8.2 在 WSL 上安装 Arch Linux　8.3 VirtualBox/在虚拟机中安装 Arch Linux　8.4 VMware　8.5 Xen　8.6 QEMU

第 9 章　数据压缩与归档（4 节）

9.1 7-Zip　9.2 Rar　9.3 硬盘克隆　9.4 归档与压缩

第 10 章　FUSE（3 节）

10.1 Gitfs　10.2 FUSE　10.3 NTFS-3G

第 11 章　光盘（3 节）

11.1 CDemu　11.2 蓝光　11.3 光盘驱动器

第 12 章　备份（3 节）

12.1 Back In Time　12.2 使用 tar 备份整个系统　12.3 系统备份

第 13 章　系统监视器（3 节）

13.1 Hddtemp　13.2 Conky　13.3 Lm sensors

第 14 章　同步（2 节）

14.1 同步和备份程序　14.2 Rsync

第 15 章　存储虚拟化（2 节）

15.1 LVM　15.2 RAID

第 16 章　集群文件系统（2 节）

16.1 Glusterfs　16.2 Ceph

第 17 章　文件管理（1 节）

17.1 Dolphin

第 18 章　日志（1 节）

18.1 Systemd/Journal

第 19 章　模块（1 节）

19.1 Capi4hylafax

第 20 章　监控（1 节）

20.1 Motion

文件系统

文件系统是系统管理的底子：怎样认出分区是什么格式、怎样创建、怎样挂载，出了问题又怎样查。本章先用《文件系统》通讲类型、查看现有文件系统、创建与挂载，再逐节落到具体格式：Ext4、XFS、Btrfs、ZFS、F2FS、Bcachefs、FAT、NTFS、Tmpfs、Overlay，以及 Swap、Autofs、Bcache 这些配套组件和 Partclone、Cfdisk、Fsck 等维护工具。

《文件系统》必读，读完再按需挑一种格式深入，Ext4 与 Btrfs 篇幅最足、值得通读；只在意快照与回滚，读 Snapper、Timeshift、Yabsnap 就够；ZFS/虚拟磁盘、Bcache 与 Bcachefs 场景特殊，可以先跳过。最容易误解的是“装上就能用”：FAT 需要内核配置，普通用户写入还要额外设置；NTFS、Btrfs、F2FS、XFS 都各带已知问题与排障小节，动手前先看一遍，比事后修省力得多。

1.1　Partclone

难度 入门 | **篇幅** 1.5 KB | **所属** 文件系统

本节将讲到

- 安装
- 对一个 ext4 格式的分区使用 Partclone

Partclone 与众所周知的 Partimage 一样可用于备份和还原分区，同时仅考虑已使用的块。

安装

安装 partclone包 包。

对一个 ext4 格式的分区使用 Partclone

不使用压缩

要备份而不使用压缩：

```
# partclone.ext4 -c -s /dev/sda1 -o ~/image_sda1.pcl
```

还原它：

```
# partclone.ext4 -r -s ~/image_sda1.pcl -o /dev/sda1
```

使用压缩

要**进行**压缩备份：

```
# partclone.ext4 -c -s /dev/sda1 | gzip -c > ~/image_sda1.pcl.gz
```

注意：为获得最大压缩效果，请使用 `gzip -c9`，或者尝试使用其他压缩工具

还原它：

```
# zcat ~/image_sda1.pcl.gz | partclone.ext4 -r -o /dev/sda1
```

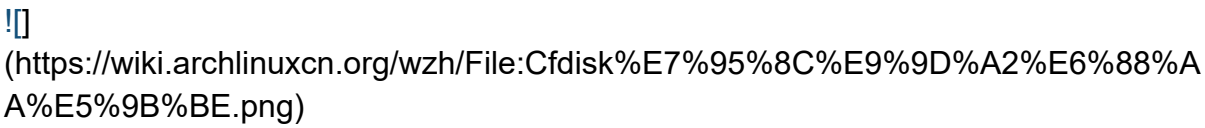
1.2 Cfdisk

难度 入门 | **篇幅** 2.5 KB | **所属** 文件系统

本节将讲到

- 安装
- 使用
- 技巧与提示
- 参见

警告：在对磁盘进行任何操作之前一定要慎重！


!
(https://wiki.archlinuxcn.org/wzh/File:Cfdisk%E7%95%8C%E9%9D%A2%E6%88%AA%E5%9B%BE.png)

简体中文环境下 cfdisk 用户界面

cfdisk 是 fdisk 的 TUI 版本，提供了基本的分区功能和基于 curses 库的用户界面。

与 fdisk 不同的是，cfdisk 可能对新手会更友好，因为它为之提供了一个可视化的界面。

安装

cfdisk 由 util-linux包 软件包提供，而 util-linux包 软件包是 base包 元软件包的依赖软件包。所以，一般您无需显式安装 cfdisk。

使用

要使用 cfdisk 管理某块设备，输入：

```
# cfdisk /dev/sda
```

注意：将 */dev/sda* 改为您希望进行操作的块设备，而**不是**分区。

关于用户界面的使用，参见 cfdisk(8) § COMMANDS。

关于 cfdisk 命令行选项，参见 cfdisk(8) § OPTIONS。

技巧与提示

输出着色

使用配置文件

输出着色由 terminal-colors.d(5) 功能实现。可以通过空文件 `/etc/terminal-colors.d/cfdisk.disable` 禁用针对 cfdisk 的隐式着色。

用户指定的 `$XDG_CONFIG_HOME/terminal-colors.d` 或 `$HOME/.config/terminal-colors.d` 会覆盖全局设置。

注意：输出着色可能默认启用，在这种情况下，terminal-colors.d 目录不必存在。

cfdisk 不支持使用 color-scheme 文件自定义颜色。

使用命令行选项

可以使用命令行选项 `-L`，`--color[=*when*]` 为输出结果着色。可选参数 *when* 可以是 **auto**、**never** 或 **always**。如果省略 *when* 参数，则默认为 **auto**。可以禁用颜色，有关当前内置默认值，请参阅 `--help` 输出。

参见

cfdisk(8)

1.3 Overlay 文件系统

难度 入门 | **篇幅** 3.8 KB | **所属** 文件系统

本节将讲到

- 安装
- 使用方法
- 参考资料

引入 overlayfs 的内核 commit有云：

: Overlayfs 允许通常需要读写的目录树覆盖到另一个只读的目录树上。所有的修改将作用于可写的上层目录。这种机制经常被用于 live CD，不过也有许多其他用途。

: 该实现与其他“联合文件系统”实现的差异是，文件打开后的所有操作都直接作用于底层（上层或下层）的文件系统。这简化了实现方式，并允许其能获得原生级别的性能。

Linux 内核于 3.18 加入 overlayfs。

安装

overlayfs 在默认内核中就已开启，并在运行 mount 命令时会自动加载 `overlay` 内核模块。

使用方法

可以使用以下 `mount` 选项来挂载 overlay 文件系统：

```
# mount -t overlay overlay -o lowerdir=/lower,upperdir=/upper,workdir=/work /merged
```

注意：工作目录 (`workdir`) 必须是和上层目录处于同一文件系统中的空目录。

- 下层目录可以是只读目录，也可以同样是 overlay。
- 上层目录一般来说是可写的。
- 工作目录用于准备层与层之间交换的文件。

`lower` 目录可以是由 `:` 分隔的一系列目录，所有在 `merged` 目录中的更改会反映到 `upper` 目录中。

示例：

```
# mount -t overlay overlay -o
lowerdir=/lower1:/lower2:/lower3,upperdir=/upper,workdir=/work /merged
```

注意：填写下层目录时，最右侧的目录层级最低；上层目录则被放在从左向右排列的下层目录整体的左侧，而**不是**下层目录最右侧一项的左侧，尽管看起来好像是这样的。

上述示例会产生下方的堆叠顺序：

```
/upper
/lower1
/lower2
/lower3
```

可以使用下面的格式添加一项 overlay 挂载到 `/etc/fstab` 中。

```
/etc/fstab
overlay /merged overlay noauto,x-
systemd.automount,lowerdir=/lower,upperdir=/upper,workdir=/work 0 0
```

有必要添加 `noauto` 和 `x-systemd.automount` mount 选项以避免 systemd 在挂载 overlayfs 失败时阻塞启动。只有当挂载的 overlayfs 被第一次访问时，它才会被真正挂载。挂载就绪前，内核会缓冲访问请求。另请参阅 fstab#通过 systemd 自动挂载。

只读 overlay

有时候，您可能只是想创建多个目录的组合只读视图。此时可以使用以下方式轻松创建视图，而**不需要**指定 `upper` 目录和 `work` 目录：

```
# mount -t overlay overlay -o lowerdir=/lower1:/lower2 /merged
```

当 `upper` 目录未指定时，overlay 文件系统将自动被挂载为只读。

参考资料

- Overlay Filesystem documentation
- Explaining overlayfs - What it does and how it works
- Wikipedia:OverlayFS

1.4 Yabsnap

难度 入门 | **篇幅** 3.9 KB | **所属** 文件系统

本节将讲到

- 安装

- 配置
- 主要命令
- 清理或删除快照
- 回滚

Yabsnap 是一个为 Arch 编写的、用于 Btrfs 分区的定时快照管理器。

安装

安装 yabsnapAUR 软件包，开发版本可以通过 yabsnap-gitAUR 安装。

配置

运行以下命令将会创建一个配置文件：

```
# yabsnap create-config configname
```

此命令会在 `/etc/yabsnap/configs/*configname*.conf` 生成配置文件。编辑该文件，修改以下内容：

- `source` - 子卷挂载点。例如 `/home`。
- `dest_prefix` - 快照名称的完整路径及前缀。例如，`/.snapshot/@home-` 会创建如下格式的备份快照：`/.snapshot/@home-20230525120000`。

您还可以编辑其他参数，以指定何时触发或清理备份。

主要命令

查看快照

要查看现有的快照，执行：

```
$ yabsnap list
```

创建快照

要为所有已有配置创建快照，执行:

```
# yabsnap create --comment 'COMMENT'
```

您也可以单独地为已挂载子卷创建快照， Yabsnap 会自动地找到相应的配置。执行：

```
# yabsnap --source '/home' create --comment 'COMMENT'
```

清理或删除快照

自动清理

根据 `/etc/yabsnap/configs/*.conf` 中的配置，Yabsnap 会自动删除超出上限的旧快照。

删除快照

也可以手动删除快照。

指定完整路径以删除特定的快照：

```
# yabsnap delete /.snapshots/@home-20230525120000
```

指定时间戳可删除所有匹配的同时拍摄的快照：

```
# yabsnap delete 20230525120000
```

回滚

回滚注意事项：使用 subvol 而不是 subvolid 挂载

建议使用 subvol 而不是 subvolid 来挂载所有的 btrfs 子卷。

例如使用下列 fstab 条目：

```
UUID=[YOUR_UUID] / btrfs rw,noatime,ssd,space_cache=v2,compress=zstd,subvol=/@ 0 0
```

注意：使用的挂载选项是 `subvol=/@`。**不要** 使用 `subvolid=`。

这样做的原因是该回滚机制不会修改您的 fstab 文件，它只会确保正确的快照挂载在相应的位置。（如果使用`subvolid`，那么会挂载原来的子卷）

回滚操作

回滚命令是安全的（因为并未执行任何修改操作），直到执行其生成的脚本。

此命令将生成一个回滚脚本，请将时间戳更改为现有的某个快照：

```
$ yabsnap rollback-gen 20230525081049 | tee ~/rollback.sh
```

只有运行该脚本才会执行真正的回滚操作。因此建议在回滚之前先检查生成的命令。

检查完生成的回滚脚本后，将其设为可执行，然后运行：

```
# ~/rollback.sh
```

此操作将回滚前述指定时间戳的所有快照。

与 Snapper 的比较

Yabsnap 是为了克服 Snapper 的某些不足而创立。具体而言，它可以做一些 Snapper 难以或无法完成的任务（截至撰写本文时）：

- 自定义备份目的路径 (issue),
- 对所有已配置的快照进行回滚（而不仅仅是默认子卷）
- 集成了 pacman hook 支持。

参见

- Yabsnap github 页面

1.5 Timeshift

难度 入门 | **篇幅** 3.9 KB | **所属** 文件系统

本节将讲到

- 安装
- btrfs 快照的 GRUB 条目
- 问题解决

Timeshift 最初是 Tony George 创建的一个工具，现在是 Xapp project 项目的一部分。

Timeshift 可帮助定期创建文件系统的增量快照，然后在以后恢复到这些快照，以撤销对系统的所有更改。

它支持针对所有文件系统的 rsync 快照，也可使用 Btrfs 驱动器的内置快照功能，这需要驱动器根目录和 home 目录分别使用 `@` 和 `@home` 子卷布局配置。

安装

安装 timeshift包 软件包，并安装/启用您选择的 cron 计划任务程序（请参阅 cron#配置，cronie 已作为 Timeshift 的依赖而被安装）。这将保证 Timeshift 应用程序中的快照有计划地进行。

另外，也可以安装 timeshift-systemd-timerAUR 来代替 cron 计划任务程序。

btrfs 快照的 GRUB 条目

要在每次生成 GRUB 配置时向 GRUB 菜单添加快照，请安装 grub-btrfs包 软件包。该软件包附带 `grub-btrfsd.service`，启用后可在创建新快照时自动更新 GRUB 配置。

要使 grub-btrfsd 与 Timeshift 一起工作，请运行以下命令编辑服务：

```
# systemctl edit --full grub-btrfsd
```

并将 `grub-btrfsd --syslog /.snapshots` 替换为 `grub-btrfsd --syslog -t`。

问题解决

Timeshift 图形用户界面无法在 Wayland 上启动

XWayland 只允许启动 X 服务器的用户连接客户端（请参阅以根用户权限启动图形用户界面（英语：Running GUI applications as root））。

由于 Timeshift 需要 root 权限，试图通过应用程序启动器或终端使用命令 `timeshift-launcher` 启动 Timeshift GUI 时，会出现包含 `xhost： command not found` 的错误。

遇到此错误的用户还可能会看到他们的身份验证组件提示输入密码，但输入密码后却发现 Timeshift GUI 无法启动。

这是因为命令 `timeshift-launcher` 需要 xorg-xhost包 软件包，请安装它。

警告：该命令可授予任何本地用户访问 X 屏幕的权限。建议不要在多用户系统上运行 `xhost`。详情请参见 xhost 和以根用户权限启动图形用户界面（英语：Running GUI applications as root）。

1.6 NTFS

难度 入门 | **篇幅** 4.8 KB | **所属** 文件系统

本节将讲到

- 提示与技巧
- 已知问题
- 排障
- 参见

翻译状态：

- 本文（或部分内容）译自 NTFS，最近一次同步于 2026-7-21，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

来自维基百科：

: **NTFS (New Technology File System)** 是一个由微软开发的专有日志文件系统。从 Windows NT 3.1 开始是 Windows NT 系列的默认文件系统。

Linux有两个内核驱动提供了对NTFS读写支持。

- ntfs3—最初被 Paragon 提交到Linux内核中。ntfs3 内核驱动并没有配套的用户空间工具。安装 来自 NTFS-3G 的 ntfsprogs包 或使用 Windows 设备以格式化分区和执行维护。
- ntfs— 从 Linux 7.1 其可用，基于在 Linux 6.9 被移除的老旧只读驱动，用户空间工具在 ntfsprogs-plusAUR 中可用。

目前来看，没有哪个驱动系统明显更好，两者各有优缺点，操作上也存在一些细微差别。值得注意的是：

- *ntfs* 的整体性能应该略好一些。
- *ntfs* 不支持 Windows 原生符号链接（计划 7.2 支持 ）
- 在 TRIM 实现上的差异

提示与技巧

防止创建 Windows 不支持的命名

NTFS 本身对字符和名称没有限制，但 Windows 对此有限制。

内核驱动支持 `windows_names` mount(8) 选项。使用该选项可严格保证兼容性。

在 GUI 文件管理器中使用 ntfs 驱动

大多 GUI 文件管理器使用 udisks 挂载，它默认偏向使用 *ntfs3* 驱动。参见Udisks#默认挂载选项以使用 *ntfs* 驱动替代。

已知问题

挂载时需要明确的文件系统类型

在7.1版本之前的内核上，NTFS 分区默认可能以只读方式挂载或完全无法挂载。

这种情况下，需要明确指定`ntfs3`为文件系统类型。例如，使用 mount(8) 的 `-t`/`--types` 选项：

```
# mount -t ntfs3 /dev/sdxY /mnt
```

或者使用 fstab：

```
/etc/fstab
/dev/sdxY /mnt ntfs3 defaults 0 0
```

排障

文件系统挂载为只读

如果使用版本旧于 7.1 的内核，使用`ntfs3`挂载文件系统以读写。详见#挂载时需要明确的文件系统类型。

unknown filesystem type 'ntfs'

挂载 NTFS 时，可能会遇到如下错误：

```
mount: /mnt: unknown filesystem type 'ntfs'
```

请检查是否已安装 ntfs-3g：

```
$ pacman -Qi ntfs-3g
```

如果尚未安装，可安装之。也可以使用 `-t ntfs3` 参数来指定使用内核的 ntfs3 驱动。

无法用 ntfs3 挂载被标记为脏的分区

尝试挂载一个正常的 NTFS 分区时（成功地使用 NTFS-3G 挂载，执行 `ntfsfix --no-action` 不汇报任何错误时），你可能会获得下面的错误：

```
mount: /mnt: wrong fs type, bad option, bad superblock on /dev/sdb1, missing codepage or helper program, or other error.
       dmesg(1) may have more information after failed mount system call.
```

当分区被标记为“脏的”时，ntfs3 将不会挂载它。`dmesg` 会帮助你判断这个情况，会说：

```
sdb1: volume is dirty and "force" flag is not set!
```

你可以尝试用 `--clear-dirty` 参数来使用 {ntfsfix(8) 来清理它。

参见

- NTFS3 内核文档
- NTFS3 驱动 FAQ – Paragon Software Group
- NTFS3 性能比较

1.7 FAT

难度 进阶 | **篇幅** 6.0 KB | **所属** 文件系统

本节将讲到

- 创建文件系统
- 内核配置
- 以普通用户身份写入 FAT32
- 查询 FAT 文件系统类型
- 参见

翻译状态：

- 本文（或部分内容）译自 FAT，最近一次同步于 2024-7-4，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

摘自 Wikipedia:File Allocation Table:

: File Allocation Table（FAT）是一种计算机文件系统结构，一系列行业标准文件系统都使用它。FAT 文件系统是一种传统的文件系统，简单而强大。即使在轻量级实现中，它也能提供良好的性能，但无法提供与某些现代文件系统相同的性能、可靠性和可扩展性。不过，出于兼容性的考虑，目前几乎所有已开发的个人计算机操作系统以及许多移动设备和嵌入式系统都支持 FAT 文件系统，因此它是一种非常适合在 1981 年至今几乎所有类型和年代的计算机和设备之间进行数据交换的格式。

创建文件系统

要创建 FAT 文件系统，请安装 dosfstools包。

`mkfs.fat` 支持创建 FAT12、FAT16 和 FAT32，有关它们差异的解释，请参阅 Wikipedia:File Allocation Table#Types 。 `mkfs.fat` 将根据分区大小选择 FAT 类型，要明确地创建特定类型的 FAT 文件系统，请使用 `-F` 选项。参见 mkfs.fat(8) 以了解更多信息。

提示：在大多数情况下，你都希望使用 FAT32。为确保分区能格式化为 FAT32，在逻辑扇区大小为 512 字节的硬盘上，分区大小至少应为 36 MiB；在逻辑扇区大小为 4096 字节的驱动器上，分区大小至少应为 260 MiB。

将分区格式化为 FAT32：

```
# mkfs.fat -F 32 /dev/partition
```

注意：`mkfs.vfat` 和 `mkfs.msdos` 都是`mkfs.fat`符号链接，它们是相同的工具。

内核配置

以下是内核中默认*挂载*配置的示例：

```
$ zgrep -e FAT -e DOS /proc/config.gz | sort -r
# DOS/FAT/NT Filesystems
CONFIG_FAT_FS=m
CONFIG_MSDOS_PARTITION=y
CONFIG_FAT_FS=m
CONFIG_MSDOS_FS=m
CONFIG_VFAT_FS=m
CONFIG_FAT_DEFAULT_CODEPAGE=437
CONFIG_FAT_DEFAULT_IOCHARSET="iso8859-1"
CONFIG_NCPFS_SMALLDOS=y
```

选项的简短说明：

- 语言设置： `CONFIG_FAT_DEFAULT_CODEPAGE`, `CONFIG_FAT_DEFAULT_IOCHARSET`
- 如果启用，则在 FAT 分区上把所有文件名转为小写字母：`CONFIG_NCPFS_SMALLDOS`
- 启用对 FAT 文件系统的支持： `CONFIG_FAT_FS`, `CONFIG_MSDOS_FS`, `CONFIG_VFAT_FS`
- 在 x86 PC 上启用对 FAT 分区驱动器的支持：`CONFIG_MSDOS_PARTITION`

如果 mount 检测到的分区类型是 VFAT，那么它将运行 `/usr/bin/mount.vfat` 脚本。

```
/usr/bin/mount.vfat
#!/bin/bash
#mount VFAT with full rw (read-write) permissions for all users
#/usr/bin/mount -i -t vfat -oumask=0000,iocharset=utf8 "$@"
#The above is the same as
mount -i -t vfat -oiocharset=utf8,fmask=0000,dmask=0000 "$@"
```

以普通用户身份写入 FAT32

要在 FAT32 分区上写入，必须对 fstab 文件进行一些更改。

```
/etc/fstab
/dev/sdxY    /mnt/some_folder  vfat    user,rw
```

`user` 选项意味着任何用户（即使是非root用户）都可以挂载和卸载分区 `/dev/sd*xY*` (mount(8) § Non-superuser mounts)。 `rw` 提供读写访问权限。

例如，如果您的 FAT32 分区位于 `/dev/sda9`，而您希望将其挂载到 `/mnt/fat32`，则应使用：

```
/etc/fstab
/dev/sda9    /mnt/fat32        vfat    user,rw
```

现在，任何用户都可以使用以下方式挂载它：

```
$ mount /mnt/fat32
```

并使用以下命令卸载它：

```
$ umount /mnt/fat32
```

请注意，FAT 不支持 Linux 文件权限。每个文件也将显示为可执行文件。您可能希望使用 `showexec` 选项仅将 Windows 可执行文件（com、exe、bat）标记为可执行文件。参见 mount(8) § Mount options for fat 来获得更多选项。

查询 FAT 文件系统类型

如果需要知道分区使用的是哪种类型的FAT文件系统，请使用 *file* 命令：

```
# file -s /dev/partition
/dev/partition: DOS/MBR boot sector, code offset 0x3c+2, OEM-ID "mkfs.fat",
sectors/cluster 4, root entries 512, sectors 4096 (volumes <=32 MB), Media descriptor
0xf8, sectors/FAT 3, sectors/track 32, heads 64, serial number 0x5bc09c21, unlabeled,
FAT (12 bit)
```

或者，您可以使用 mtools包 包中的 *minfo*：

```
# minfo -i /dev/partition ::
device information:
===================
filename="/dev/partition"
sectors per track: 32
heads: 64
cylinders: 2

media byte: f8

mformat command line: mformat -t 2 -h 64 -s 32 -i "/dev/partition" ::

bootsector information
======================
banner:"mkfs.fat"
sector size: 512 bytes
cluster size: 4 sectors
reserved (boot) sectors: 1
fats: 2
max available root directory slots: 512
small size: 4096 sectors
media descriptor byte: 0xf8
sectors per fat: 3
```

```
sectors per track: 32
heads: 64
hidden sectors: 0
big size: 0 sectors
physical drive id: 0x80
reserved=0x0
dos4=0x29
serial number: 5BC09C21
disk label="NO NAME    "
disk type="FAT12   "
```

参见

- MountFATFileSystems

1.8 Tmpfs

难度 进阶 | **篇幅** 6.9 KB | **所属** 文件系统

本节将讲到

- 用法
- 例子
- 禁用自动挂载
- 故障排除
- 提示与技巧

tmpfs 是一个停留在内存与/或交换分区的临时文件系统。把目录挂载为 tmpfs 是一个加速文件访问或是确保内容重启自动清除的有效手段。

提示：tmpfs 目录中的临时文件可以使用 systemd-tmpfiles 在计算机启动时重新创建。

用法

tmpfs 经常会被使用在/tmp、/var/lock 以及 /var/run。请**不要**把它用在 /var/tmp 上，这个目录是用于跨重启保存的临时文件的。

Arch 对 `/run` 目录使用 tmpfs，同时存在的 `/var/run` 和 `/var/lock` 只是单纯为了维持兼容性的符号链接。在默认的 systemd 配置下，tmpfs 也被用于 `/tmp`，如果没有特殊需求不需要专门在 fstab 里添加条目。

glibc包 2.2 及以上为了 POSIX 共享内存会假定有 tmpfs 被挂载于`/dev/shm`。这一挂载会由 systemd 自动处理，无需手动配置 fstab。

一般来说，需要经常进行读写操作的程序与任务可以从使用 tmpfs 的目录中获益。某些应用甚至可以通过将部分或是全部数据放至共享内存以获得巨大的性能提升，例如可以把 Firefox 的用户资料放在内存中来大幅提升性能。

例子

注意：真正的内存/交换区消耗依实际使用而定，tmpfs 分区在真正需要内存之前不会使用任何内存。

tmpfs 分区的默认最大容量是可用内存的一半，不过这是可以修改的。若需要显式地设定最大容量，使用 `size` 挂载选项（以下的例子会覆盖默认的 `/tmp` 挂载）：

```
/etc/fstab
tmpfs   /tmp         tmpfs   rw,nodev,nosuid,size=2G          0  0
```

若想让挂载更安全，可以使用以下选项：

```
/etc/fstab
tmpfs   /www/cache    tmpfs
rw,size=1G,nr_inodes=5k,noexec,nodev,nosuid,uid=user,gid=group,mode=1700 0 0
```

请查看 tmpfs(5) 手册页和安全#文件系统来获取更多信息。

重启以使修改起效。注意虽然直接运行 `mount -a` 让修改立刻起效很有吸引力，但这一操作会让当前目录的所有文件都无法被访问（这对正在运行的持有锁文件的程序可能是个大问题）。不过，如果它们都是空的，使用 `mount -a` 而非重启应该是安全的（也可以通过单独挂载各个部分来绕开这个问题）。

在应用更改之后，通过查看 `/proc/mounts` 或是使用 `findmnt` 来验证是否起效。

```
$ findmnt /tmp
TARGET SOURCE FSTYPE OPTIONS
/tmp   tmpfs  tmpfs  rw,nosuid,nodev,relatime
```

tmpfs 也可以临时改变大小而无需重启。如果有一个大型的编译任务即将运行，可以直接：

```
# mount -o remount,size=4G /tmp
```

或基于 RAM 重新调整大小：

```
# mount -o remount,size=80% /tmp
```

禁用自动挂载

在 systemd 下，`/tmp` 会被自动挂载为 tmpfs，除非 `/tmp` 是 `/etc/fstab` 中一个已经存在的挂载点（不论这个挂载点是 tmpfs 还是储存设备）。如果想禁用自动挂载，mask 掉 `tmp.mount` 这一 systemd 单元。

文件将不会被储存在 tmpfs 中，而是直接到块设备上。`/tmp` 中的内容现在会在每次重启间保存，这可能不是所期望的行为（虽说文件还是会在 10 天后被清理）。若想恢复之前的行为，在重启时自动清理 `/tmp` 目录的话，可以考虑使用 tmpfiles.d(5)：

```
/etc/tmpfiles.d/tmp.conf
# see tmpfiles.d(5)
# always enable /tmp directory cleaning
D! /tmp 1777 root root 0

# remove files in /var/tmp older than 10 days
D /var/tmp 1777 root root 10d

# namespace mountpoints (PrivateTmp=yes) are excluded from removal
x /tmp/systemd-private-*
x /var/tmp/systemd-private-*
X /tmp/systemd-private-*/tmp
X /var/tmp/systemd-private-*/tmp
```

故障排除

作为 root 在 tmpfs 中打开符号链接失败

假设 `/tmp` 正在使用 tmpfs，首先更改当前工作目录到 `/tmp`，然后在这个 `/tmp` 目录下创建一个文件以及指向这个文件的符号连接。root 在读取这个符号链接时会遇到“权限不够”的错误，这是因为 `/tmp` 设置了黏滞位。

这个行为可以通过 `/proc/sys/fs/protected_symlinks` 控制，或是直接使用 sysctl：`sysctl -w fs.protected_symlinks=0`。参阅 Sysctl#配置来永久更改这个行为。

警告：更改这一行为可能会导致安全问题！

write() 出现 EDQUOT 失败: 超出磁盘配额

自 systemd v258 起，systemd 默认为 `/tmp` 和 `/dev/shm` 启用磁盘配额并为每个用户分配相当于 tmpfs 大小 80% 的配额。使用更大的大小重新挂载 tmpfs 不会自动更新配额，因此需要安装quota-tools包并手动进行修改：

```
# setquota -u username-or-uid block-softlimit block-hardlimit inode-softlimit inode-hardlimit /tmp
```

举例：

```
# setquota -u user 4G 4G 0 0 /tmp
```

提示与技巧

为 /run/user/xxxx 分配更多内存

控制 `/run/user/` 中 tmpfs 大小的标准方法是通过修改 `/etc/systemd/logind.conf` 中的 `RuntimeDirectorySize` （详见 logind.conf(5)）。默认大小为物理内存的 10%，但可以安全地增加此值。需要记住的是，tmpfs 仅占用已实际使用的空间，此处指定的数值只是允许的最大值。

另请参阅

- Linux 内核文档

1.9 Fsck

难度 进阶 | **篇幅** 7.2 KB | **所属** 文件系统

本节将讲到

- 启动时检查
- 提示和技巧
- 疑难解答

翻译状态：

- 本文（或部分内容）译自 Fsck，最近一次同步于 2026-03-25，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

fsck 是*"file system check"*的缩写，意为"文件系统检查"，用于检查 Linux 上的文件系统并尝试修复发现的错误。通常，fsck 会并行检查不同物理磁盘上的文件系统来节省时间（请参阅 fsck(8)）。

在 Arch 的启动流程中，*fsck* 会在您每次开机时自动检查相关的驱动器，因此您无需使用命令行。

启动时检查

方式

提示：本小节中所有的"检查"（共两处）在英文原文中均为动词 "fsck"，而 fsck 本身则包含"检查并尝试修复"之意，为保证语句通顺译为"检查"，请知悉。

有两种方式：

1. mkinitcpio 为您提供了一个选择，即在挂载您的根文件系统前使用 `fsck` 钩子检查它。若如此做，您应当用 `rw` 内核参数将根文件系统挂载为可读写。
2. systemd 会检查所有 fsck 选项大于 0 的文件系统（使用 fstab 选项或用户提供的单元文件）。对于根文件系统，其必须在开始时被使用 `ro` 内核参数被挂载为只读，然后再通过 fstab 被重新挂载为可读写（请注意 `defaults` 挂载选项包含 `rw`）。

推荐第一种方式，若您遵循了安装指南，则会默认这样。若您想使用第二种方式，您应当从 `mkinitcpio.conf` 中移除 `fsck` 钩子并在内核命令行中使用 `ro`。使用 `fsck.mode=skip` 内核参数可在两种做法中完全禁用 ***fsck***。

强制检查

如果您使用 `base` 的 mkinitcpio 钩子，您可以向内核传递 `fsck.mode=force` 内核参数以在启动时强制执行 ***fsck***，这会检查您的设备上的每一个文件系统。

此外，systemd 提供了 systemd-fsck@.service(8)，用于检查已配置的文件系统，且不在 initramfs 中执行。然而，用这种方法检测根文件系统会推迟启动流程，因为要重新挂载根文件系统。

注意：习惯使用其他 GNU/Linux 发行版的用户可能会习惯向每个文件系统写入 `forcefsck` 文件或使用 `shutdown -F` 关机命令等旧方法，但它们仅适用于 SysVinit 和早期版本的 Upstart，但不适用于 systemd。因此上述方法是 Arch Linux 上唯一的解决方法。

在修复询问中自动回答 yes

启动时的 *fsck* 检查可能会停止并返回 `"UNEXPECTED INCONSISTENCY; RUN fsck MANUALLY."`。

当您需要允许某些更改以修复系统，而这些更改并不被认为是完全安全的时候，它就会发生。因此需要手动执行 *fsck*。

您可以通过将 `fsck.repair` 内核命令行选项设为 `yes` 来让 `fsck` 自动接受所有建议的更改（即对所有问题回答 yes）。

提示：其他可能的值为 `no` 和 `preen`，参阅 systemd-fsck@.service(8) 以获取这些选项的意义。

提示和技巧

尝试修复坏块

欲自动修复 ext2/ext3/ext4 或 FAT 文件系统上损坏的部分，请执行：

警告：这条命令不会询问您是否修复，当您运行它时，回答就已经是 yes。

```
# fsck -a
```

交互式地修复坏块

这在 boot 分区上的文件被改动而日志未能正确更新时会很有用。在这种情况下，请取消挂载 boot 分区，然后执行以下命令以修复损坏的部分：

```
# fsck -r 设备
```

更改检查频率

注意：本小节的 *tune2fs* 命令和 *dumpe2fs* 命令仅在 ext2/ext3/ext4 文件系统上工作。

默认情况下，*fsck* 每 30 次启动检查一次文件系统（不同分区独立计数）。要更改检查频率，请执行：

```
# tune2fs -c 20 /dev/sda1
```

在本例中，`20` 是相邻两次检查之间的启动次数。

请注意 `1` 会在每次启动时都执行检查，而 `0` 会完全停止检查。

欲查看某分区的检查频率和挂载次数，请执行：

```
# dumpe2fs -h /dev/sda1 | grep -i 'mount count'
```

fstab 选项

fstab 是一个系统配置文件，用于告诉 Linux 内核将哪些分区（文件系统）挂载到文件系统树的哪里。

一个典型的 `/etc/fstab` 条目可能看起来像这样：

```
/dev/sda1   /          ext4      defaults        0  1
/dev/sda2   /other     ext4      defaults        0  2
/dev/sda3   /win       ntfs-3g   defaults        0  0
```

第 6 列（粗体文字）是 fsck 选项。

- `0`——不检查
- `1`——要检查的第一个文件系统（分区），`/`（根分区）应当被设为 `1`。
- `2`——所有要检查的其他文件系统。

疑难解答

无法在独立的 /usr 分区上运行 fsck

1. 确保您的 `/etc/mkinitcpio.conf` 文件中配置了需要的钩子，并在编辑该文件后重新生成了 initramfs。
2. 检查您的 fstab！只有根分区的最后一项需要是 `1`，其他的都是 `2` 或 `0`。检查是否存在其他拼写错误。

“ext2fs: no external journal” 报错

有时候 ext3/ext4 文件系统可能会损坏（由于电源故障）。通常，fsck 会提示 `no external journal`（无外部日志），在这种情况下，请运行以下命令：

将该分区从其挂载点取消挂载：

```
# umount directory
```

向该分区写入新日志：

```
# tune2fs -j /dev/partition
```

运行 fsck 以修复该分区：

```
# fsck -p /dev/partition
```

1.10 F2FS

难度 进阶 | **篇幅** 7.3 KB | **所属** 文件系统

本节将讲到

- 已知问题
- 创建 F2FS 文件系统
- 挂载F2FS文件系统
- 检查与修复

- 扩容 F2FS 文件系统

F2FS (Flash-Friendly File System) 是一个为配备了 Flash Transition Layer 的 NAND 闪存开发的文件系统，与 JFFS 或 UBIFS 不同，它依靠 FTL 来处理写入分发。 Linux 从内核3.8开始支持 F2FS 。

警告：若运行的内核版本比创建F2FS文件系统的内核版本低，则文件系统可能无法使用。例如，使用linux包提供的内核创建文件系统，当系统需要降级到linux-lts包提供的内核时，就可能出现问题。详见FS#69363。

已知问题

fsck错误

F2FS的fsck不完善，可能在系统突然断电后导致数据丢失。

如果经常遭遇突然断电，建议使用其它文件系统。

GRUB 支持

尽管GRUB从2.0.4版本开始就支持F2FS，但它无法从启用了`extra_attr`flag的F2FS分区中正确读取启动文件。（详见GRUB#不支持的文件系统）

创建 F2FS 文件系统

本文假定设备已分区。

首先，安装f2fs-tools包软件包。

用`mkfs.f2fs`命令创建一个文件系统（`*/dev/sdxY*` 是想要设置成 F2FS 的分区。）：

```
# mkfs.f2fs -l mylabel -O extra_attr,inode_checksum,sb_checksum /dev/sdxY
```

注意：推荐至少要设置上例中的选项，这些选项能帮助`f2fs.fsck`检测并修复某些文件系统损坏。对于所有可用的选项，见mkfs.f2fs(8)。

压缩

注意：与其它支持inline compression的文件系统不同，F2FS的压缩默认情况下并不会提供额外的可用空间。不论是否启用压缩，F2FS都会保留相同数目的block，这主要是为了减少写入放大以延长闪存寿命，同时可能带来微小的性能提升。详见内核文档中的Compression Implementation。可以通过iotcl系统调用传递`F2FS_IOC_RELEASE_COMPRESS_BLOCKS`命令将对应文件未使用的空间暴露出来，但这将使该文件不可变。

要使用压缩，添加`compression`选项，例如：

```
# mkfs.f2fs -l mylabel -O extra_attr,inode_checksum,sb_checksum,compression /dev/sdxY
```

挂载文件系统时，指定`compress_algorithm=(lzo|lz4|zstd|lzo-rle)`挂载选项。此外，`compress_extension=txt`挂载选项可自动压缩所有创建的txt文件。

要指定F2FS压缩某一文件或目录，使用：

```
$ chattr -R +c [FOLDER]
```

文件层面的加密

从Linux内核4.2开始，F2FS原生支持文件加密。F2FS的加密是目录层级的，不同的目录可以使用不同的密钥。这不同于 dm-crypt（块设备层级）或eCryptfs（嵌套在已有文件系统上）。要使用F2FS的原生加密支持，用如下命令创建文件系统：

```
 # mkfs.f2fs -l mylabel -O extra_attr,inode_checksum,sb_checksum,encrypt /dev/sdxY
```

对于已有的文件系统，可使用`fsck.f2fs -O encrypt /dev/sdxY`启用加密。关于如何使用加密，见fscrypt。

挂载F2FS文件系统

既可以手动挂载文件系统，也可以使用其它挂载机制：

```
# mount /dev/sdxY /mnt/foo
```

推荐的挂载选项

因为F2FS是设计为在闪存设备上使用的，建议启用压缩。要启用压缩，必须在使用`mkfs.f2fs`创建文件系统时就指定相应选项。 可以使用以下挂载选项略微提升性能：

```
# mount -o
compress_algorithm=zstd:6,compress_chksum,atgc,gc_merge,lazytime /dev/sdxY /mnt/foo
```

- `compress_algorithm=zstd:6`：使用zstd压缩等级6进行压缩, 可提供很好的压缩率。
- `compress_chksum`：使用校验码检查压缩的block，可防止数据损坏。
- `atgc,gc_merge`：启用更好的GC方式，异步执行某些前台GC。
- `lazytime`：不使用同步方式更新文件的访问与修改时间，可提升IO性能与闪存寿命。

discard实现方式

默认情况下，F2FS使用混合型TRIM模式（类似continuous TRIM）。该实现方式使用异步的discard线程来缓解RW IO的高discard延迟。其在内存中维护一个discard操作的等待队列，并在空闲时执行它们（详见）。若要使用periodic TRIM，则需要指定`nodiscard`挂载选项。

检查与修复

检查与修复F2FS文件系统需要使用f2fs-tools包提供的`fsck.f2fs`工具。要检查一个F2FS文件系统，执行：

```
# fsck.f2fs /dev/sdxY
```

根据结果选择不同的选项来修复不一致问题（详见fsck.f2fs(8)），例如：

```
# fsck.f2fs -f /dev/sdxY
```

扩容 F2FS 文件系统

如果文件系统未被挂载，且对应的分区已经被拓展，则可以扩容文件系统。但目前不支持收缩。 首先使用分区工具调整分区大小。例如，假设`parted`控制台中`print`命令的输出如下：

```
Number  Start   End     Size        File system     Name                  Flag
 1      1049kB  106MB   105MB       fat32           EFI system partition  boot, esp
 2      106MB   11,0GB  10,9GB      ext4
 3      11,0GB  12,3GB  1322MB      f2fs
 4      31,0GB  31,3GB  261MB       ext4
```

要调整使得`f2fs`对应分区（3号）占据4号分区前的所有空闲空间，执行`resizepart 3 31GB`。使用`exit`退出`parted`。

然后扩容文件系统来填充新的分区，使用以下命令:

```
# resize.f2fs /dev/sdxY
```

`*/dev/sdxY*`是要扩容的 F2FS 分区。可用的选项见 resize.f2fs(8)。

注意：如果使用 GPT，分区的 GUID (可通过`/dev/disk/by-partuuid/`查看) 可能会改变，但是文件系统的 UUID (可通过`/dev/disk/by-uuid/`查看) 应该保持不变。

1.11 Swap

难度 进阶 | **篇幅** 11.3 KB | **所属** 文件系统

本节将讲到

- swap
- 交换分区
- 交换文件
- 交换加密
- 性能优化

swap也叫交换空间，是用于暂存暂时用不到内存页面的地方，以为更加常用的数据腾出更多空间。本文介绍了GNU/Linux上的swap和分页，教您如何创建和启用交换分区或交换文件。

swap支持由Linux内核和来自 util-linux包 软件包的用户空间实用程序提供。

swap

swap可以是磁盘的一个分区，也可以是一个文件。用户可以在安装时或安装后的任何时候创建swap。swap有两种用途：第一，是暂存暂时用不到内存页面，将内存空间腾出给更常用的数据；第二，是用于 suspend-to-disk 支持。

使用swap来暂时内存数据是否有好处取决于您的物理内存大小。如果物理内存非常充裕，通常有大量未使用空间的话，则根本用不到swap。如果物理内存非常不足，无法存下您当前正在使用的软件的活跃数据（「工作集」）的话，使用swap将导致频繁交换以至于系统运行缓存。而在以上两种极端情况之间，使用swap通常能够交换出不活跃的内存数据、将释放出来的空间用于I/O缓存或者活跃数据，将提升使用体验，或者让更多的软件可以放在后台驻留而不是为了腾出内存空间给当前程序使用而关闭。这对于某些占用内存很多、但又经常有大量不活跃数据的软件很有效，比如网页浏览器和Telegram。

参见《替swap辩护：常见的误解》和《关于swap的一些补充》。

注意：连续的交换文件和交换分区之间没有性能之别，两者的处理方式是一样的。

要检查swap的状态，使用：

```
$ swapon --show
```

或者显示物理内存以及交换使用情况：

```
$ free -h
```

交换分区

交换分区可以用大多数 GNU/Linux 分区工具创建。交换分区在 GPT 上的分区类型 GUID 为 `0657FD6D-A4AB-43C4-84E5-0933C84B4F4F`（gdisk 的 `8200` 类型，fdisk 的 `swap` 类型），在 MBR 上的类型 ID 为 `82`。

要将分区设置为 Linux 交换分区，请使用 mkswap(8) 命令。例如：

```
# mkswap /dev/sdxy
```

警告：指定分区上的所有数据会丢失。

想要启用一个设备作为交换分区：

```
# swapon /dev/sdxy
```

参见 swapon(8) 了解选项语法。

启动时启用

要在启动时启用交换分区，可以：

- 使用 systemd#GPT分区自动挂载
- 或者添加一个条目到 `/etc/fstab`。例如：

```
    UUID=device_UUID none swap defaults 0 0
```

: 其中 `*device_UUID*` 是交换分区的 UUID。

参见 fstab 了解文件语法，以及 systemd#systemd.mount - mounting。

关闭交换分区

使用下面的命令关闭交换分区：

```
# swapoff /dev/sdxy
```

也可以使用 `-a` 参数来关闭所有的交换分区。

因为 swap 通过 systemd 管理，因此会在下一次系统启动时再次激活。要永久禁用该特性，运行 `systemctl --type swap` 来查找 *.swap* 单元，然后 mask 它。

交换文件

相比于使用一个磁盘分区作为swap，使用交换文件可以更方便地随时调整大小或者移除。当磁盘空间有限（例如常规大小的SSD）时，使用交换文件更加理想。

文件系统	是否支持交换文件
Bcachefs	否
Btrfs	是
F2FS	是
ext4	是
JFS	是
NILFS2	否
NTFS3	是
ReiserFS	是
XFS	是
ZFS	否

建立交换文件

注意：如果您使用Btrfs，请先参阅 Btrfs#交换文件。

使用 mkswap(8) 创建一个由你自己指定大小的交换文件（参见 Partitioning#Swap 获取建议）。例如，创建一个 4 GiB 的交换文件：

```
# mkswap -U clear --size 4G --file /swapfile
```

启用交换文件：

```
# swapon /swapfile
```

最后，编辑 `/etc/fstab`，在为交换文件添加一个条目：

```
/etc/fstab
/swapfile none swap defaults 0 0
```

有关更多信息，请参见 fstab#Usage.

注意：交换文件必须由文件系统上的位置指定，而不是由UUID或标签指定。

删除交换文件

如果要删除一个交换文件，必须先停用它。

作为root运行：

```
# swapoff /swapfile
```

然后即可删除它：

```
# rm -f /swapfile
```

最后从 `/etc/fstab` 中删除相关条目

交换加密

查看 dm-crypt/交换分区加密。

性能优化

交换操作通常比直接访问RAM中的数据慢得多。然而，完全禁用交换以提高性能有时会导致性能下降。如果没有足够的物理内存来容纳所有内容，完全不进行交换会减少文件系统缓存的可用内存，从而导致更频繁且代价高昂的磁盘使用。

可以调整交换值以提高性能：

swappiness

当内存使用达到某个阈值时，内核会开始查看活动内存并尝试释放一些内容。文件数据可以写入文件系统（如果已更改），卸载并在稍后重新加载；其他数据必须先写入swap才能卸载。

swappiness sysctl 参数代表了内核对于写入swap而不是文件的偏好。它的值可以是0到200之间（在Linux内核版本低于5.8时最大值为100），默认值为60。较低的值会导致内核更倾向于释放打开的文件，较高的值会导致内核尝试使用swap，而值100意味着IO成本被假定为相等。

注意：有一个常见的误解是swappiness会影响内存阈值或阻止使用交换空间，但它只影响释放文件页面而不是交换的偏好。有关更详细的解释，请参阅：这篇文章 或 内核源代码 中它的使用。

要查看当前交换值(Swappiness)，请执行以下操作：

```
$ sysctl vm.swappiness
```

此外，可以读取文件 `/proc/sys/vm/swappiness` 以获得原始整数值。

要临时设置交换值(Swappiness)，请执行以下操作：

```
# sysctl -w vm.swappiness=35
```

要永久设置交换值，请创建sysctl.d(5)配置文件。例如：

```
/etc/sysctl.d/99-swappiness.conf
vm.swappiness=35
```

为了让 boot loader 在加载内核时设置交换值，添加一个 kernel parameter，例如 `sysctl.vm.swappiness=35`.

要测试和了解这可能起作用的原因，请查看此文章。有关最近的反对意见，请参阅这篇文章。

VFS 缓存压力

`vm.vfs_cache_pressure` 是另外一个影响交换性能的 *sysctl* 参数，这个参数控制内核回收 VFS 缓存的程度，增大数值会增加回收 VFS 缓存的频率。更多信息请阅读 Linux kernel documentation。

优先级

如果你有多个交换文件或交换分区，你应该考虑给它们各自分配一个优先级值(0 到 32767)。系统会优先使用较高优先级的交换区域，然后再使用较低优先级的交换区域。例如，如果你有一个较快的磁盘和一个较慢的磁盘，给较快的设备分配一个更高的优先级。优先级可以在 fstab 中通过 `pri` 参数指定：

```
UUID=f9fe0b69-a280-415d-a03a-a32752370dee none swap defaults,pri=100 0 0
UUID=d7eb6062-01c8-4367-ac1d-3bf1167de8bb none swap defaults,pri=10  0 0
```

或者通过 swapon 的 `--priority` 参数：

```
# swapon --priority 100 /dev/sda1
```

如果两个或更多的区域有同样的优先级，并且它们都是可用的最高优先级，页面会按照循环的方式在它们之间分配。

分片

不需要使用 RAID 提高交换的性能，只要在 `/etc/fstab` 中给交换设备设置相同的优先级，内核会将交换分片到多个设备。详情请参考 Software-RAID 指南。

丢弃（即 trim）

参见 Solid state drive#swap。

在RAM中使用压缩块设备

参见 Improving performance#zram or zswap。

仅将swap用于休眠

如果你只需要swap作为休眠映像存储空间，那么你可以使用 zswap 并禁用其回写，以便在常规交换使用中没有磁盘写入。参见 Power management/Suspend and hibernate#Disable zswap writeback to use the swap space only for hibernation。

参见

- 替swap辩护：常见的误解
- 关于swap的一些补充

1.12 ZFS/虚拟磁盘

难度 进阶 | **篇幅** 15.4 KB | **所属** 文件系统

本节将讲到

- 创建和删除 Zpools
- 创建与删除数据集
- 显示并设置属性
- Add content to the Zpool and query compression performance
- Simulate a disk failure and rebuild the Zpool

翻译状态：

- 本文（或部分内容）译自 ZFS/Virtual disks，最近一次同步于 2023-10-28，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面/Virtual_disks_(%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87)?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

该篇文章涵盖了关于 ZFS 的一些基本任务和用例。它与 ZFS 主文章有所不同，此处的案例使用了基于虚拟磁盘搭建的 zpool。因此，只要用户不在创建出的磁盘中存放任何关键数据，他们就无需担心任何的数据丢失问题。

在该文章的案例中使用了多个虚拟磁盘作为 ZFS 的 VDEV。用户可以用现有物理磁盘创建 VDEV，也可以在空闲内存足够时使用 tempfs（内存盘）创建 VDEV。

注意：使用文件作为 VDEV 是测试 ZFS 的好方法，但不建议用于存放实际数据。

由于许可证的差异问题，ZFS 二进制文件和内核模块可以很容易地以源代码进行分发，但不容易以软件包和预编译集形式进行分发。详细信息可参考 ZFS#安装。

创建和删除 Zpools

管理 ZFS 非常简单，只需要用到两个工具：

- `/usr/bin/zpool`
- `/usr/bin/zfs`

镜像

对于仅包含两个磁盘的带冗余 zpool，建议使用类似 RAID1 对数据镜像镜像的*镜像*模式。镜像同时可以作为 Raidz 配置的替代，以获得出人意料的结果。关于 vdev 镜像的详细信息可参考这里。

RAIDZ1

RAIDZ1 的最低硬盘数需求为 3。为了优化存储空间效率和达到性能最佳点，建议盘数遵顼“二次方加验证盘”的规则。对于 RAIDZ-1，建议使用 3（2+1），5（4+1），或者 9（8+1）块盘。下列示例将使用最简单的（2+1）。

创建 3 个 2G 的文件作为虚拟硬盘：

```
$ for i in {1..3}; do truncate -s 2G /scratch/$i.img; done
```

创建 RAIDZ1：

```
# zpool create zpool raidz1 /scratch/1.img /scratch/2.img /scratch/3.img
```

注意已成功创建并挂载了一个 3.91G 的 zpool：

```
# zfs list
 NAME   USED  AVAIL  REFER  MOUNTPOINT
 test   139K  3.91G  38.6K  /zpool
```

可通过如下方式获取设备信息：

```
# zpool status zpool
  pool: zpool
 state: ONLINE
  scan: none requested
config:

        NAME                STATE     READ WRITE CKSUM
           zpool            ONLINE       0     0     0
           raidz1-0         ONLINE       0     0     0
             /scratch/1.img  ONLINE       0     0     0
             /scratch/2.img  ONLINE       0     0     0
             /scratch/3.img  ONLINE       0     0     0

errors: No known data errors
```

要删除 zpool：

```
# zpool destroy zpool
```

RAIDZ2 和 RAIDZ3

高阶 ZRAID 的操作与上述步骤类似，只需调整 for 循环参数来调整创建的文件数、在创建 zpool 时指定“raidz2”或“raidz3”，并补充上额外添加的镜像文件。

总结下 Toponce 的指南：

- RAIDZ2 硬盘数应为 4（2+2），6（4+2），10（8+2）或 18（16+2）。
- RAIDZ3 硬盘数应为 5（2+3），7（4+3），11（8+3）或 19（16+3）。

线性扩展

This setup is for a JBOD, good for 3 or less drives normally, where space is still a concern and you are not ready to move to full features of ZFS yet because of it. RaidZ will be your better bet once you achieve enough space to satisfy, since this setup is NOT taking advantage of the full features of ZFS, but has its roots safely set in a beginning array that will suffice for years until you build up your hard drive collection.

Assemble the Linear Span:

```
# zpool create zpool san /dev/sdd /dev/sde /dev/sdf
# zpool status zpool
  pool: zpool
 state: ONLINE
  scan: scrub repaired 0 in 4h22m with 0 errors on Fri Aug 28 23:52:55 2015
config:
```

```
        NAME        STATE     READ WRITE CKSUM
        zpool       ONLINE       0     0     0
          sde       ONLINE       0     0     0
          sdd       ONLINE       0     0     0
          sdf       ONLINE       0     0     0

errors: No known data errors
```

创建与删除数据集

以下演示了如何创建带压缩的子数据集：

- 创建数据集

```
# zfs create -p -o compression=on san/vault/falcon/snapshots
# zfs create -o compression=on san/vault/falcon/version
# zfs create -p -o compression=on san/vault/redtail/c/Users
```

- 接下来列出数据集 (this was a linear span)

```
$ zfs list
```

Note, there is a huge advantage(file deletion) for making a 3 level dataset. 如果你存放了大量的数据，通过使用数据集进行切分，可使得删除文件时只需删除对应的数据集，而无需等待删除递归操作完成。

显示并设置属性

Without specifying them in the creation step, users can set properties of their zpools at any time after its creation using `/usr/bin/zfs`.

显示属性

To see the current properties of a given zpool:

```
# zfs get all zpool
```

修改属性

Disable the recording of access time in the zpool:

```
# zfs set atime=off zpool
```

Verify that the property has been set on the zpool:

```
# zfs get atime
NAME  PROPERTY     VALUE     SOURCE
zpool  atime        off       local
```

提示：This option like many others can be toggled off when creating the zpool as well by appending the following to the creation step: -O atime-off

Add content to the Zpool and query compression performance

Fill the zpool with files. For this example, first enable compression. ZFS uses many compression types, including, lzjb, gzip, gzip-N, zle, and lz4. Using a setting of simply 'on' will call the default algorithm (lzjb) but lz4 is a nice alternative. See zfsprops(7) § compression for more.

```
# zfs set compression=lz4 zpool
```

In this example, the linux source tarball is copied over and since lz4 compression has been enabled on the zpool, the corresponding compression ratio can be queried as well.

```
$ wget https://www.kernel.org/pub/linux/kernel/v3.x/linux-3.11.tar.xz
$ tar xJf linux-3.11.tar.xz -C /zpool
```

To see the compression ratio achieved:

```
# zfs get compressratio
NAME      PROPERTY       VALUE  SOURCE
zpool  compressratio  2.32x  -
```

Simulate a disk failure and rebuild the Zpool

To simulate catastrophic disk failure (i.e. one of the HDDs in the zpool stops functioning), zero out one of the VDEVs.

```
$ dd if=/dev/zero of=/scratch/2.img bs=4M count=1 2>/dev/null
```

Since we used a blocksize (bs) of 4M, the once 2G image file is now a mere 4M:

```
$ ls -lh /scratch
total 317M
-rw-r--r-- 1 facade users 2.0G Oct 20 09:13 1.img
-rw-r--r-- 1 facade users 4.0M Oct 20 09:09 2.img
-rw-r--r-- 1 facade users 2.0G Oct 20 09:13 3.img
```

The zpool remains online despite the corruption. Note that if a physical disc does fail, dmesg and related logs would be full of errors. To detect when damage occurs, users must execute a scrub operation.

```
# zpool scrub zpool
```

Depending on the size and speed of the underlying media as well as the amount of data in the zpool, the scrub may take hours to complete. The status of the scrub can be queried:

```
# zpool status zpool
  pool: zpool
 state: DEGRADED
status: One or more devices could not be used because the label is missing or
       invalid.  Sufficient replicas exist for the pool to continue
       functioning in a degraded state.
action: Replace the device using 'zpool replace'.
   see: http://zfsonlinux.org/msg/ZFS-8000-4J
  scan: scrub repaired 0 in 0h0m with 0 errors on Sun Oct 20 09:13:39 2013
config:

       NAME                STATE     READ WRITE CKSUM
          zpool            DEGRADED     0     0     0
            raidz1-0       DEGRADED     0     0     0
              /scratch/1.img  ONLINE       0     0     0
              /scratch/2.img  UNAVAIL      0     0     0  corrupted data
              /scratch/3.img  ONLINE       0     0     0

errors: No known data errors
```

Since we zeroed out one of our VDEVs, let us simulate adding a new 2G HDD by creating a new image file and adding it to the zpool:

```
$ truncate -s 2G /scratch/new.img
# zpool replace zpool /scratch/2.img /scratch/new.img
```

Upon replacing the VDEV with a new one, zpool rebuilds the data from the data and parity info in the remaining two good VDEVs. Check the status of this process:

```
# zpool status zpool
  pool: zpool
 state: ONLINE
  scan: resilvered 117M in 0h0m with 0 errors on Sun Oct 20 09:21:22 2013
config:

       NAME                    STATE     READ WRITE CKSUM
          zpool                ONLINE       0     0     0
            raidz1-0           ONLINE       0     0     0
              /scratch/1.img   ONLINE       0     0     0
              /scratch/new.img ONLINE       0     0     0
              /scratch/3.img   ONLINE       0     0     0

errors: No known data errors
```

快照和恢复已删除的文件

ZFS 是写时复制文件系统，所有文件在写入那一刻时就已存入。保存文件修改实际上创建了同一文件的拷贝及其修改记录。快照实际上利用了这一特性，使得用户可以访问快照创建时的旧版本文件。

注意：因 ZFS 使用快照方式较为特殊，当使用快照时，像 **df** 一类汇报文件系统空间的 Linux 应用会给出不准确的数据。`/usr/bin/zfs list` 会给出准确的 zpool 可用及剩余空间数据。

为了简化示例，我们将在 zpool 中创建数据集并创建快照，快照可以对整个 zpool 或者池中的单个数据集进行创建，区别只有命名方式：

快照目标	快照名
整个 zpool	zpool@snapshot-name
数据集	zpool/dataset@snapshot-name

创建新数据集并获取所有权：

```
# zfs create zpool/docs
# chown facade:users /zpool/docs
```

注意：创建命令中缺少 / 是有意为之的，并不是缺漏！

Time 0

添加一些文件到新数据集中（/zpool/docs）：

```
$ wget -O /zpool/docs/Moby_Dick.txt  https://www.gutenberg.org/ebooks/2701.txt.utf-8
$ wget -O /zpool/docs/War_and_Peace.txt https://www.gutenberg.org/ebooks/2600.txt.utf-8
$ wget -O /zpool/docs/Beowulf.txt https://www.gutenberg.org/ebooks/16328.txt.utf-8
# zfs list
NAME           USED  AVAIL  REFER  MOUNTPOINT
zpool         5.06M  3.91G  40.0K  /zpool
zpool/docs    4.92M  3.91G  4.92M  /zpool/docs
```

这意味着下载的书籍占用了 /zpool/docs 中 4.92M 的空间。

Time +1

接下来为数据集创建快照：

```
# zfs snapshot zpool/docs@001
```

再运行一次 list 命令：

```
# zfs list
NAME           USED  AVAIL  REFER  MOUNTPOINT
```

```
zpool       5.07M  3.91G  40.0K  /zpool
zpool/docs  4.92M  3.91G  4.92M  /zpool/docs
```

注意，因为这三个文件没有被修改，因此 USED 一列没有变更，意味着快照不占用 zpool 空间。

我们可以通过如下方式列出快照，并确认快照不占用存储空间，而是引用了原始文件，这些文件共占用 4.92M：

```
# zfs list -t snapshot
NAME               USED  AVAIL  REFER  MOUNTPOINT
zpool/docs@001      0      -  4.92M  -
```

Time +2

接下来添加些新文件，并创建新快照：

```
$ wget -O /zpool/docs/Les_Mis.txt https://www.gutenberg.org/ebooks/135.txt.utf-8
# zfs snapshot zpool/docs@002
```

再次使用 list 来查看空间变化：

```
# zfs list -t snapshot
NAME               USED  AVAIL  REFER  MOUNTPOINT
zpool/docs@001  25.3K      -  4.92M  -
zpool/docs@002      0      -  8.17M  -
```

可以看到 001 快照的元数据占用了 25.3K 大小，并依旧引用了之前的 4.92M 数据；新快照不占用任何空间，并引用了总共 8.17M 的数据。

Time +3

接下来我们模拟下意外覆盖文件导致的数据丢失：

```
$ echo "this book sucks" > /zpool/docs/War_and_Peace.txt
```

与之前一样，再创建一个快照：

```
# zfs snapshot zpool/docs@003
```

接下来列出快照，可见被引用的数据减少了 3.1M：

```
# zfs list -t snapshot
NAME               USED  AVAIL  REFER  MOUNTPOINT
zpool/docs@001  25.3K      -  4.92M  -
zpool/docs@002  25.5K      -  8.17M  -
zpool/docs@003      0      -  5.04M  -
```

我们可以在一个或者多个旧快照中找到该文件的未损坏版本。ZFS 将快照存放在 zpool 中一个隐藏文件夹内：`/zpool/files/.zfs/snapshot`：

```
$ ls -l /zpool/docs/.zfs/snapshot
total 0
dr-xr-xr-x 1 root root 0 Oct 20 16:09 001
dr-xr-xr-x 1 root root 0 Oct 20 16:09 002
dr-xr-xr-x 1 root root 0 Oct 20 16:09 003
```

我们可以从任意一个快照中复制一份未损坏版本的书到 zpool 内或外部的任意地址：

```
% cp /zpool/docs/.zfs/snapshot/002/War_and_Peace.txt /zpool/docs
```

注意：<TAB> 自动补全在默认情况下不可用，但可通过修改存储池或数据集的 *snapdir* 属性来启用。

```
# zfs set snapdir=visible zpool/docs
```

接下来进入一个或各个快照文件夹：

```
$ cd /zpool/docs/.zfs/snapshot/001
$ cd /zpool/docs/.zfs/snapshot/002
```

重复 df 命令：

```
$ df -h | grep zpool
zpool           4.0G     0  4.0G   0% /zpool
zpool/docs      4.0G  5.0M  4.0G   1% /zpool/docs
zpool/docs@001  4.0G  4.9M  4.0G   1% /zpool/docs/.zfs/snapshot/001
zpool/docs@002  4.0G  8.2M  4.0G   1% /zpool/docs/.zfs/snapshot/002
```

注意：通过将 zpool 设为离线并重新挂载，或是重启服务器，可以撤销 .zfs 下所有文件夹可见。

例如：

```
# zpool export zpool
# zpool import -d /scratch/ zpool
$ df -h | grep zpool
zpool          4.0G     0  4.0G   0% /zpool
zpool/docs     4.0G  5.0M  4.0G   1% /zpool/docs
```

Time +4

现在一切已恢复正常，我们可以为目前状态再次创建一个快照：

```
# zfs snapshot zpool/docs@004
```

此时的 list 输出为：

```
# zfs list -t snapshot
NAME               USED  AVAIL  REFER  MOUNTPOINT
zpool/docs@001  25.3K      -  4.92M  -
zpool/docs@002  25.5K      -  8.17M  -
zpool/docs@003   155K      -  5.04M  -
zpool/docs@004      0      -  8.17M  -
```

列出快照

需要指出，这一简单但十分重要的命令在同类文章中经常被忽略，因此有必要特别提一下。

使用如下命令列出系统中的所有快照：

```
$ zfs list -t snapshot
```

删除快照

可存放的快照数量最大为 2^64，用户可以通过如下命令删除快照：

```
# zfs destroy zpool/docs@001
# zfs list -t snapshot
NAME               USED  AVAIL  REFER  MOUNTPOINT
zpool/docs@002  3.28M      -  8.17M  -
zpool/docs@003   155K      -  5.04M  -
zpool/docs@004      0      -  8.17M  -
```

Troubleshooting

If your system is not configured to load the zfs pool upon boot, or for whatever reason you want to manually remove and add back the pool, or if you have lost your pool completely, a convenient way is to use import/export.

If your pool was named <zpool>

```
# zpool import -d /scratch zpool
```

If you have any problems accessing your pool at any time, try export and reimport.

```
# zpool export zpool
# zpool import -d /scratch zpool
```

1.13 Bcachefs

难度 进阶 | **篇幅** 15.9 KB | **所属** 文件系统

本节将讲到

- 安装
- 初次设置
- 配置
- 提示和技巧
- 故障排除

Bcachefs 是一种写时复制（CoW）文件系统，支持多设备、RAID、压缩、校验和加密。它旨在提供类似于 Btrfs 和 ZFS 的现代功能。

Bcachefs 基于 Bcache，两者都主要由 Kent Overstreet 开发。

注意： 自 2026-04-15 起, 官方的 常见问题页面 声明 Bcachefs 已经稳定，这一点也在 1.38.6 发布日志 中得到了确认。不过，继续为重要数据保留备份并没有坏处。

安装

注意： Bcachefs 树内内核模块已于 Linux 6.18 被移除。

安装 bcachefs-tools包、bcachefs-dkms包以及内核对应的头文件，以支持DKMS。比如，为默认的 linux包内核安装linux-headers包。

初次设置

单盘

```
# bcachefs format /dev/sdX
# mount -t bcachefs /dev/sdX /mnt
```

多盘

与 RAID0 类似，Bcachefs 默认会将数据条带化存储 (stripe)。数据冗余由 **replicas** 选项进行处理。 对 2 个存储设备使用 `--replicas=2` 相当于 RAID1，对 4 个存储设备使用 `--replicas=2` 相当于 RAID10，以此类推。

```
# bcachefs format /dev/sdX /dev/sdY --replicas=n
# mount -t bcachefs /dev/sdX:/dev/sdY /mnt
```

Bcachefs 支持混合使用不同的储存设备。如果储存设备的大小不同，为了让每个储存设备以相同的速率填满数据，文件系统在某些储存设备上会使用更大的条带。如果储存设备的速度不同，对冗余数据的读取请求会被发往 IO 延迟最低的设备。如果有些

储存设备比其他更可靠（例如硬件 RAID），可以通过 `--durability=2 *device*` 来把上面的每份数据副本算成两份。

加密根文件系统

Bcachefs 使用 自行实现 的全文件系统加密，采用 ChaCha20-Poly1305。要格式化加密的文件系统：

```
# bcachefs format --encrypted /dev/sdX
```

如果用于根文件系统，请在 `/etc/mkinitcpio.conf` 的 HOOKS 中添加 `bcachefs`，以便在启动或从休眠恢复时提示文件系统解锁。

注意： 当前只有基于 BusyBox 的初始内存盘（initramfs）支持`bcachefs` hook。

SSD 缓存

Bcachefs 有四个储存目标（storage targets）: metadata、background、foreground 和 promote。对文件系统进行的写入优先使用 foreground，然后随着时间推移移动至 background。对文件系统的读取会缓存在 promote 上。metadata 目标通常与超低读取延迟的 NVMe 驱动器一起使用，例如 Intel Optane。

注意：这只是对一个单一储存池而言的优先级。如果 foreground 满了，则系统会直接向 background 写入，或者如果两个都满了则会试着写入 promote。文件系统的元数据会优先写入 foreground，但也有可能会写入储存目标中的任意一个。在移除缓存设备时请小心，因为它们可能仍旧储存了数据。请参见 #移除设备

推荐的一种配置是使用一组 SSD 作为 foreground 和 promote，使用一组 HDD 作为 background（也就是回写缓存）。

```
# bcachefs format \
    --label=ssd.ssd1 /dev/sdA \
    --label=ssd.ssd2 /dev/sdB \
    --label=hdd.hdd1 /dev/sdC \
    --label=hdd.hdd2 /dev/sdD \
    --label=hdd.hdd3 /dev/sdE \
    --label=hdd.hdd4 /dev/sdF \
    --replicas=2 \
    --foreground_target=ssd \
    --promote_target=ssd \
    --background_target=hdd
# mount -t bcachefs /dev/sdA:/dev/sdB:/dev/sdC:/dev/sdD:/dev/sdE:/dev/sdF /mnt
```

如果想配置直写缓存，和上面操作基本一致，不过要对每个 SSD 设备设置 `--durability=0 *device*`。 如果想配置绕写缓存，只需把 HDD 设为 foreground，把 SSD 设为 promote。

挂载

默认的挂载方式是在 mount 指令中指定每个设备。

```
# mount -t bcachefs /dev/sdA:/dev/sdB:/dev/sdC:/dev/sdD:/
```

`mount.bcachefs` 支持通过 UUID 挂载文件系统，UUID 会在使用 `bcachefs format` 创建文件系统时显示。

```
# mount.bcachefs UUID=f66d108f-83d2-4679-b50b-7d5e710f6
```

若需在启动时挂载单个或多个设备，将 `bcachefs` 加入到 `/etc/mkinitcpio.conf` 的`HOOKS` 中，并重新生成 initramfs。

配置

有关文件系统选项可以阅读bcachefs 文档。

大多数选项可以通过以下方式设置：

- 在 `bcachefs format` 期间
- 在格式化后使用 `bcachefs set-fs-option`
- 在挂载时通过 `mount -o *option=value*`
- 或通过 sysfs，例如 `echo *X* > /sys/fs/bcachefs/*UUID*/options/*option*`

挂载选项会覆盖其他方法设置的选项，后者会将选项保存到文件系统的超级块中。

注意：要使用 sysfs 必须先挂载文件系统。除了 fsck 以外，所有的操作都可以在文件系统挂载状态下进行。

部分选项举例如下：

<table class="wikitable"><caption>Bcachefs 选项</caption><tbody><tr><th>选项</th><th>默认值</th><th>描述</th></tr><tr><td>metadata_checksum</td><td><code>crc32c</code></td><td>指定用于元数据写入的校验和算法。可选：<code>none</code>、<code>crc32c</code>、<code>crc64</code>、<code>xxhash</code>。</td></tr><tr><td>data_checksum</td><td><code>crc32c</code></td><td>指定用于数据写入的校验和算法，可选算法与 <code>metadata_checksum</code> 相同。</td></tr><tr><td>compression</td><td><code>none</code></td><td>指定用于前台压缩的算法。可选：<code>none</code>、<code>lz4</code>、<code>gzip</code>、<code>zstd</code>。</td></tr><tr><td>background_compression</td><td><code>none</code></td><td>指定用于后台压缩的算法，可选算法与 <code>compression</code> 相同。可以使用与前台压缩不同的算法。</td></tr><tr><td>str_hash</td><td><code>siphash</code></td><td>指定用于directory entries和 xattrs 的哈希函数。可选：<code>crc32c</code>、

<code>crc64</code>、<code>siphash</code>。</td></tr><tr><td>nocow</td><td>-</td><td>在可能的情况下，所有写入都将就地完成。快照 (snapshot) 和 reflink 仍会导致 COW 写入。该选项会隐式禁用数据校验、压缩和加密。</td></tr><tr><td>encrypted</td><td>-</td><td>启用文件系统加密（chacha20/poly1305）；将提示输入密码。</td></tr></tbody></table>

以下内容可以通过 `bcachefs setattr *file* --option=value` 按每个文件或目录进行设置。如果在一个目录中设置，它会进行递归传递。

注意：重新平衡线程目前为止不会在后台调整副本。也就是说，如果更改文件的副本选项，则必须手动运行 rereplicate 命令以确保旧文件遵循新规则。

- 数据冗余数 (data_replicas)
- 数据校验和 (data_checksum)
- 前台压缩 (compression)、后台压缩 (background_compression)
- 存储组配置：foreground_target、background_target、promote_target

要检查哪些选项处于活动状态，可以执行 `getfattr -d -m 'bcachefs_effective\.' *directory/file*`

注意：目前储存设备用量显示的是未压缩时的大小。除此之外，压缩功能一切正常。

更改设备所属储存组

可以通过 sysfs 更改所属储存组：

```
# echo group.drive_name > /sys/fs/bcachefs/filesystem_uuid/dev-X/label
```

注意：需要重新挂载才能生效。

添加设备

```
# bcachefs device add --label=group.drive_name /mnt /dev/device
```

若这是某储存组中的第一个设备，则需要修改储存目标的设置才能使用。以下例子添加了一个缓存设备。

```
# echo new_group > /sys/fs/bcachefs/filesystem_uuid/options/promote_target
# echo new_group > /sys/fs/bcachefs/filesystem_uuid/options/foreground_target
# echo old_group > /sys/fs/bcachefs/filesystem_uuid/options/background_target
```

注意：只有新的写入会被分散到新添加的设备中去。已经写入的内容不会改变，除非储存设备的占用达到一个阈值，从而触发重新平衡（rebalance/restripe）。目前无法手动触发重新平衡。

移除设备

首先请确保至少设备上有两份元数据（设备移除（evacuate）功能似乎对元数据不奏效）。如果数据和元数据都已经有冗余了，可以跳过这一步。

```
# echo 2 > /sys/fs/bcachefs/UUID/options/metadata_replicas
# bcachefs data rereplicate /mnt
```

```
# bcachefs device set-state ro device
# bcachefs device evacuate device
```

将状态设置为 ro，表示只读。

若想移除设备：

```
# bcachefs device remove device
# bcachefs data rereplicate /mnt
```

冗余

Bcachefs 中的元数据和数据的冗余级别可以独立配置，以控制数据冗余和持久性（durability）。 以下选项既定义了同步（即时）冗余，它会在写入时原子执行以确保数据完整；同时也定义了最终冗余目标（eventual replication target），其会在在后台异步完成以提高冗余和故障容忍度。

- `--replicas=X` 设置元数据和数据副本的**目标**数量。
- `--metadata_replicas=X` 设置在正常运行和重新平衡期间后台维护的元数据副本的**目标**数量。
- `--data_replicas=X` 设置**最终**要维持的数据副本的**目标**数量。
- `--metadata_replicas_required=X` 设置在元数据被视为已提交之前，必须**同步**写入的元数据副本的**最小**数量。
- `--data_replicas_required=X` 设置在数据被视为已提交之前，必须**同步**写入的数据副本的**最小**数量。

注意： 区分 `--[meta]data_replicas_required` 与 `--[meta]data_replicas` 非常重要，因为 `_replicas_required` 值设定了将被立即写入的副本数量的下限，而 `_replicas` 值设定了最终会写入的副本数量。可以将 `_replicas_required` 解释为*如果我们只能完成这么多写入，文件系统依然能以降级模式（degraded）运行*。

压缩

压缩通过 `--compression=` 选项设置，支持设置压缩级别。比如要设置 zstd 压缩并使用级别 5，可以使用 `--compression=zstd:5`。

注意： 目前尚未实现多线程压缩。

子卷

Bcachefs 支持使用与 Btrfs 类似的用户空间接口来管理子卷和快照。新子卷可以创建为空子卷，也可以是另一个子卷的快照。快照是可写的，并且可以多次快照，从而形成快照树。

创建快照的成本非常低：它们不像 Btrfs 那样基于 COW btree 的克隆，而是基于 btree 中单个键的版本控制。你可以创建成千上万个快照，唯一的限制是磁盘空间。

创建子卷

创建一个新的空子卷：

```
# bcachefs subvolume create /path/to/subvolume
```

删除子卷

删除已有的子卷或快照：

```
# bcachefs subvolume delete /path/to/subvolume
```

为现有子卷创建快照

为现有子卷创建快照：

```
# bcachefs subvolume snapshot /path/to/source /path/to/dest
```

子卷在删除其中所有内容后也可以使用常规的 rmdir 删除，比如使用 `rm -rf`。

递归快照创建和递归列出子卷等功能仍待实现。

提示和技巧

请查看 Systemd/Journal 以获取更多有用的错误信息。

标志顺序

一些 `bcachefs format` 标志是根据其参数顺序设置的，并且仅影响切换标志后出现的驱动器。例如，如果希望 SSD 具有 `--durability=0` 并启用 `--discard` 而 HDD 使用默认值，请确保按以下顺序传递参数：

```
# bcachefs format \
    --label=hdd.hdd1 /dev/sdC \
    --label=hdd.hdd2 /dev/sdD \
    --label=hdd.hdd3 /dev/sdE \
    --label=hdd.hdd4 /dev/sdF \
    --durability=0 --discard \
    --label=ssd.ssd1 /dev/sdA \
    --label=ssd.ssd2 /dev/sdB \
    --replicas=2 \
    --foreground_target=ssd \
    --promote_target=ssd \
    --background_target=hdd
```

格式化后设置冗余

可以使用 `set-fs-option` 在格式化后设置副本（冗余）数量。

```
# bcachefs set-fs-option --metadata_replicas=2 --data_replicas=2 /dev/sdX
```

之后需要让 bcachefs 确保所有文件都拥有副本，命令为：

```
# bcachefs data rereplicate /mnt
```

故障排除

32 位程序无法查看目录内容

部分 32 位程序可能无法获取 Bcachefs 目录中的内容，这是由于执行 readdir(3) syscall 时文件系统返回的数据不兼容所导致。

要解决这个问题，可以暂时使用不同的文件系统供程序读写，比如 tmpfs。

swapfile 包含空洞或其他不支持的扩展

Bcachefs 目前不支持交换文件。

多设备 fstab

目前 systemd 存在一个 bug，导致无法在启动时通过在 fstab 中使用以冒号分隔的设备来挂载多设备 bcachefs 文件系统。使用 `mount -a` 时可以正常挂载，但在开机启动时不会挂载。不过自从 bcachefs-tools包 版本 1.7.0 起，可以通过单个设备节点来挂载多设备阵列，从而允许使用正常的 UUID 指定方法。

```
# UUID=10176fc9-c4fa-4a30-9fd0-a756d861c4cd     /mnt   bcachefs defaults,nofail 0 0
```

可以通过以下任一命令找到文件系统 UUID / 外部 UUID：

```
# bcachefs fs usage /mnt
# bcachefs show-super /dev/sdXY
```

挂载加密设备错误

对于使用 `--encrypted` 选项创建的设备，当 `bcachefs unlock /dev/sd*XY*` 后挂载失败时，出现

```
ERROR - bcachefs::commands::cmd_mount: Fatal error: Required key not available
```

可以通过手动将密钥链接到会话来解决这个问题：

```
# keyctl link @u @s
# mount /dev/sdXY /mnt
Enter passphrase:
```

不需要输入 *mount* 要求的密码（按回车键即可）。

另请参阅

- 官方手册
- Kent Overstreet 的 Patreon 页面
- Wikipedia:Bcachefs
- bcachefs on ReadTheDocs

1.14 Ext4

难度 进阶 | **篇幅** 17.1 KB | **所属** 文件系统

本节将讲到

- 创建 ext4 文件系统
- 从 ext2/ext3 迁移到 ext4
- 提升性能
- 技巧与窍门
- 参见

来自 Ext4 - Linux Kernel Newbies：

: Ext4 是最常用的 Linux 文件系统 Ext3 的进化。在许多方面，Ext4对于 Ext3 有着比 Ext3 对于 Ext2 更多更深的改变。Ext3 主要是向 Ext2 添加了日志系统，而 Ext4 修改了重要的文件系统的数据结构，比如用来存储文件数据的那部分。当然结果就是文件系统有更好的设计，更好的性能，稳定性还有更多的功能。

创建 ext4 文件系统

安装 e2fsprogs包。

要格式化分区，使用：

```
# mkfs.ext4 /dev/分区
```

提示：

- 更多选项参见 mke2fs(8)；编辑 `/etc/mke2fs.conf` 可以查看/修改默认配置。
- 如果支持，可以启用元数据校验和。

Bytes-per-inode 比例

来自 mke2fs(8):

: ***mke2fs*** *在硬盘上为每个bytes-per-inode大小的空间创建一个 inode. B*ytes-per-inode *的比例越大, 创建的 inode 就越少.*

创建新文件、目录、符号链接等都需要至少一个空闲的 inode。如果 inode 数过低，即使文件系统中仍有空间也无法创建文件。

由于无法在创建文件系统后修改 bytes-per-inode 比例以及 inode 数量, `mkfs.ext4` 默认使用更低的 inode 比例，即每 16384 bytes (16 KiB) 一个 inode 以避免这个问题.

然而，对于一个具有对于大小为数百或数千GB、平均文件大小为兆字节的分区，这通常会导致 inode 过大，因为创建的文件数永远不会达到 inode 数。

这样的结果是浪费硬盘空间, 因为这些未使用的 inode 每个都会占用文件系统 256 bytes 的空间 (这是在 `/etc/mke2fs.conf` 中设置的，但是不应该改变). 256 * 数百万 = 许多GB在未使用的节点中被浪费了.

这种情况可以通过比较以下几点进行评价， `Use%` 以及 `IUse%` 由 `df` 和 `df -i` 提供:

```
$ df -h /home
Filesystem              Size    Used   Avail  Use%   Mounted on
/dev/mapper/lvm-home    115G    56G    59G    49%    /home
$ df -hi /home
Filesystem              Inodes  IUsed  IFree  IUse%  Mounted on
/dev/mapper/lvm-home    1.8M    1.1K   1.8M   1%     /home
```

要指定一个不同的 Bytes-per-inode 比率，你可以使用 `-T *usage-type*` 选项，将会使用`/etc/mke2fs.conf`中定义的类型，表示文件系统预期的用法。 在这些类型中较大的 `largefile` 和 `largefile4` 将提供更相关联的比例，即每 1 MiB 或 4 MiB 一个 inode。

```
# mkfs.ext4 -T largefile /dev/device
```

Bytes-per-inode 比例可以直接通过 `-i` 选项设定: *e.g.* 使用 `-i 2097152` 对应 2 MiB 的比例以及 `-i 6291456` 对应 6 MiB 的比例.

提示：相反地，如果你需要建立一个分区用于存放数百万计的小文件（例如邮件或新闻组项目）你可以使用更小的 *usage-type* 值，例如 `news` (每 4096 bytes 一个 节点) 或者 `small` (相同但更小的 inode 和 block 大小).

警告：如果你大量使用符号链接，请确保保持足够高的 inode 数和较低的 bytes-per-inode，因为符号链接虽然不会占用更多的空间，但每一个新的符号链接都会消耗一个新的 inode，因此文件系统中的 inode 可能会被很快用完。

保留块

默认情况下，5% 的文件系统块会被预留给超级用户，以避免碎片化并“*允许由 root 拥有的守护程序在非特权进程被阻止向文件系统写入后继续正常运行*”（来自 mke2fs(8)）。

对于现在的大容量磁盘，如果分区被用于长期存储或对系统运作并非至关重要（例如 `/home`），这将比必要的大小更大。查看这封邮件，可以了解ext4 的开发者 Ted Ts'o 对保留块的看法 ，以及这个 superuser 的回答，为这个话题提供了一般的背景信息。

如果分区满足以下条件之一，则通常可以放心缩小保留块的比例：

- 非常大（例如 > 50G）
- 用于长期存储，即文件不会被非常频繁地创建和删除

在 ext4 相关的实用程序中可以使用 `-m` 选项指定保留块的比例。

要在创建文件系统时不创建保留块，使用：

```
# mkfs.ext4 -m 0 /dev/设备
```

要在之后将比例改为 1%，使用：

```
# tune2fs -m 1 /dev/设备
```

要将保留块空间设置为以千兆字节为单位的绝对大小，使用 `-r`:

```
# tune2fs -r $((ngigs * 1024**3 / blocksize)) /dev/设备
```

blocksize 是文件系统中的块大小，以字节为单位。它几乎总是4096，但你可以检查以确保无误：

```
# tune2fs -l /dev/设备 | grep 'Block size:'
Block size:               4096
```

$(()) 语法用于数学扩展。这个语法在 bash包 和 zsh包中有效，但在 fish包中不适用。对于 fish，应使用以下语法：

```
# tune2fs -r (math 'ngigs * 1024^3 / blocksize') /dev/设备
```

这些命令可以应用于已挂载的文件系统，改变将立即生效。可以使用 findmnt(8) 查找设备名：

```
# tune2fs -m 1 "$(findmnt -no SOURCE 挂载点路径)"
```

查询当前保留块的数量：

```
# tune2fs -l /dev/mapper/proxima-root | grep 'Reserved block count:'
Reserved block count:     2975334
```

这是块的数量，所以需要乘上文件系统的块大小才能得到字节数：`2975334 * 4096 / 1024**3 = 11.34 GiB`。

从 ext2/ext3 迁移到 ext4

不转换直接把 ext2/ext3 分区挂载成 ext4 分区格式

基本原理

转换到ext4和继续使用 ext2/ext3 格式的折中的办法就是把分区当作 ext4 分区来挂载。

优点:

- 兼容性（分区的文件系统可以继续挂载为 ext3） – 这允许用户继续在不支持 ext4 文件格式的操作系统中读取分区。（例如：带 ext2/ext3 驱动的 Windows 系统）
- 提高性能（不过没有完全转换成 ext4 分区后好）。

缺点:

- 仅能使用少部分 ext4 特性。（只有那些不改变分区格式的功能能被使用，例如 multiblock allocation 和 delayed allocation）

注意：除了由 ext4 格式带来的相对新的不一样的特性（可以看作一种潜在风险）之外，**这种技术没有大的缺点**

步骤

1. 修改 `/etc/fstab`，把你想要挂载成 ext4 的分区的“type”栏的内容从 ext2/ext3 改为 ext4。
2. 重新挂载分区。

将 ext2/ext3 分区转换为 ext4 格式

基本原理

为了能够使用 ext4 的全部特性，必须完成一个不可逆的转换过程。

优点：

- 更好的性能以及新功能。

缺点：

- 对于主要包含静态文件的分区，例如 /boot 分区，新功能可能并没有什么好处。此外，添加日志功能（将ext2分区转换为ext3/ext4分区会自动进行）总是会带来额外的性能开销。
- 不可逆（ext4 分区不能“降级”到 ext2/ext3。不过，在启用 extent)和其他特有功能前，是可以向下兼容的)

步骤

这些指令改编自内核文档和 BBS thread。

警告：

- 如果你要转换系统的根文件系统，请确保在重启时可以使用'fallback' initramfs。或者，根据 Mkinitcpio#模块（MODULES） 添加ext4，并在启动前重新创建initramfs 。
- 如果你决定转换一个 `/boot` 分区, 请确保 引导加载程序 支持从ext4启动.

在下面的步骤中，`/dev/sdxX` 代表要转换的分区的路径，例如 `/dev/sda1`。

1. 在将任何ext3分区转换为ext4之前，备份所有数据。clonezilla包是一个有用的工具，尤其对于根分区来说。
2. 编辑 `/etc/fstab` ，将要转换为ext4分区的'type'栏的内容从ext3改为ext4。
3. 启用实时介质（如果需要）。使用 e2fsprogs包 进行转换时分区必须未挂载。如果要转换根分区，最简单的方法就是从其他实时介质启动。
4. 确保分区未被挂载
5. 如果你想转换一个ext2分区，第一步就是以root身份运行 `tune2fs -j /dev/sdxX` 来添加日志，使其成为ext3分区。

6. 以root身份运行`tune2fs -O extent,uninit_bg,dir_index /dev/sdxX`。这个命令将ext3分区转换为ext4（不可逆）。
7. 以 root 身份运行 `fsck -f /dev/sdxX`。
8. 这一步是必须的，否则文件系统**将无法读取**。这次运行 *fsck* 是为了将文件系统恢复到一致状态。它将发现组描述中的校验和错误。`-f` 选项要求 *fsck* 强制检查，即使文件系统看起来是干净的。可以在上面使用 `-p` 选项来“自动修复”（否则，每个错误都要求用户输入）。
9. 推荐操作：挂载分区并以root身份运行 `e4defrag -c -v /dev/sdxX`。
10. 即使现在文件系统已经转换为ext4了，但以前写入的文件还没有利用ext4的extent)的优势，该操作将提高大文件性能并减少碎片和文件系统检查时间。为了充分利用ext4的优势，所有文件都需要在磁盘上重写。使用 `e4defrag`来解决这个问题。
11. 重新启动

提升性能

E4rat

E4rat 是为 ext4 文件系统设计的预加载应用程序。它监视启动时打开的文件，并通过优化它们在分区上所处的位置来提升访问效率，并在启动过程之初就预加载它们。与机械硬盘不同的是 *E4rat* 不会提升固态硬盘的性能，因为后者的访问时间与前者相比可以忽略不计。

禁用访问时间更新

ext4 文件系统会记录于文件上次被访问的时间相关的信息，而记录这些信息会导致开销。使用 `noatime` 选项可防止更新访问时间戳。

```
/etc/fstab
/dev/sda5    /    ext4    defaults,noatime    0    1
```

这样做会破坏访问依赖时间的应用程序，请查看 fstab#atime 参数获取可能的解决方案。

增加提交时间间隔

通过为 提交 选项提供更长的延迟时间，可以增加数据和元数据的同步间隔。

默认的5秒意味着如果断电，最多只会丢失5秒的工作。它每5秒强制对所有数据/日志进行一次完整的同步到物理介质。由于有了日志功能，文件系统不会受损。以下的 fstab 示例展示了`commit`选项的使用：

```
/etc/fstab
/dev/sda5    /    ext4    defaults,noatime,commit=60    0    1
```

关闭屏障

警告：如果磁盘无法保证在电源掉电时缓存正确写入，禁用屏障可能会导致严重的文件系统损坏和数据丢失。

Ext4 默认启用写屏障。它确保文件系统元数据磁盘上被正确地写入和排序，即使在写缓存掉电时也是如此。这会带来性能成本，特别是对于大量使用 *fsync* 或创建和删除许多小文件的应用程序。对于写缓存由电池供电的磁盘，禁用障碍可以会安全地提高性能。

要关闭屏障，将 `barrier=0` 选项添加到文件系统中。例如：

```
/etc/fstab
/dev/sda5    /    ext4    noatime,barrier=0    0    1
```

禁用日志

警告：使用没有日志的文件系统，在突发卸载的情况下，例如断电或者内核锁定，将导致数据丢失。

ext4 中禁用日志，可以对已卸载的硬盘使用下列指令完成操作:

```
# tune2fs -O "^has_journal" /dev/sdXN
```

技巧与窍门

使用基于文件的加密

自从Linux 4.1以来，ext4原生支持文件加密，请参阅 fscrypt 文章。加密是在目录级别应用的，在不同的目录中可以使用不同的加密密钥。这与 dm-crypt（块级别的加密）和 eCryptfs（英语：eCryptfs）（堆叠加密文件系统）都不同

在现有的文件系统中启用元数据校验和

当一个文件系统是使用 e2fsprogs包 1.43 (2016) 或更高的版本创建时，默认情况下会启用元数据校验和。可以转换现有文件系统以启用元数据校验和支持。

如果CPU支持 SSE 4.2，请确保加载 `crc32c_intel` 内核模块以启用硬件加速的 CRC32C 算法。如果不支持，则改为加载 `crc32c_generic` 模块。。

要了解有关元数据校验和的更多信息，请参阅 ext4 wiki。

提示：使用`dumpe2fs`检查文件系统上启用的功能：

```
# dumpe2fs -h /dev/path/to/disk
```

注意：必须卸载文件系统。

首先需要使用 `e2fsck`来检查和优化分区：

```
# e2fsck -Df /dev/path/to/disk
```

将文件系统转换为 64 位：

```
# resize2fs -b /dev/path/to/disk
```

最后启用校验和支持：

```
# tune2fs -O metadata_csum /dev/path/to/disk
```

验证：

```
# dumpe2fs -h /dev/path/to/disk | grep features:
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype extent
64bit flex_bg sparse_super large_file huge_file dir_nlink extra_isize metadata_csum
```

在现有的文件系统中启用 fast_commit

从 5.10 内核开始，可以启用`fast_commit` 选项来提高ext4的性能：

```
# tune2fs -O fast_commit /dev/drivepartition
```

阐述当前配置或更改：

```
# tune2fs -l /dev/drivepartition | grep features
```

启用不区分大小写模式

警告：GRUB目前不支持Ext4的 `casefold`功能；请参见GRUB bug #56897。为GRUB需要读取的文件系统启用该功能将会导致系统无法启动，并出现未知文件系统 错误，即使没有目录实际使用该功能。

ext4 可以在不区分大小写的模式下使用，这可以提高在 Wine 中运行应用程序和游戏的性能。此功能不影响整个文件系统，只影响了启用不区分大小写属性的目录。

首先，在文件系统中启用该功能：

```
# tune2fs -O casefold /dev/path/to/disk
```

现在，你可以在任何目录中启用不区分大小写：

```
$ chattr +F /mnt/partition/case-insensitive-directory
```

请注意，目录必须为空，且从其他地方移动子目录不会继承他们的属性，因此，请相应地提前规划。

参见

- Ext4 官方 wiki
- Ext4 Disk Layout described in its wiki
- Ext4 加密 LWN 文章
- Kernel commits for ext4 encryption
- e2fsprogs Changelog
- Ext4 元数据校验和
- Ext4 fast commits

1.15 Autofs

难度 进阶 | **篇幅** 17.6 KB | **所属** 文件系统

本节将讲到

- 安装

- 配置
- MTP
- Troubleshooting and tweaks
- Alternatives to AutoFS

本文将概述AutoFS的配置方法,当未被挂载的可移除文件系统或是网络共享的文件系统被用户访问时，这个软件包可以提供的自动挂载的支持。

安装

安装软件包 autofsAUR。

提示：你将不再需要载入`autofs4` 模式.

配置

AutoFS uses template files for configuration which are located in `/etc/autofs` The main template is called `auto.master`, which can point to one or more other templates for specific media types.

- Open the file `/etc/autofs/auto.master` with your favorite editor, you will see something similar to this:

```
/etc/autofs/auto.master
#/media /etc/autofs/auto.media
```

The first value on each line determines the base directory under which all the media in a template are mounted, the second value is which template to use. The default base path is `/media`, but you can change this to any other location you prefer. For instance:

```
/etc/autofs/auto.master
/media/misc     /etc/autofs/auto.misc     --timeout=5
/media/net      /etc/autofs/auto.net      --timeout=60
```

注意：Make sure there is an empty line on the end of template files (press `ENTER` after last word). If there is no correct EOF (end of file) line, the AutoFS daemon will not properly load.

The optional parameter `timeout` sets the amount of seconds after which to unmount directories.

The base directory will be created if it does not exist on your system. The base directory will be mounted on to load the dynamically loaded media, which means any content in the base directory will not be accessible while autofs is on. This procedure is however non-destructive, so if you accidentally automount into a live directory you can just change the location in `auto.master` and restart AutoFS to regain the original contents.

If you still want to automount to a target non-empty directory and want to have the original files available even after the dynamically loaded directories are mounted, you can use autofs to mount them to another directory (e.g. /var/autofs/net) and create soft links.

```
# ln -s /var/autofs/net/share_name /media/share_name
```
Alternatively, you can have autofs mount your media to a specific folder, rather than inside a common folder.

```
/etc/autofs/auto.master
/-      /etc/autofs/auto.template
/etc/autofs/auto.template
/path/to/folder      -options :/device/path
/home/user/usbstick  -fstype=auto,async,nodev,nosuid,umask=000  :/dev/sdb1
```
注意：This can cause problems with resources getting locked if the connection to the share is lost. When trying to access the folder, programs will get locked into waiting for a response, and either the connection has to be restored or the process has to be forcibly killed before unmounting is possible. To mitigate this, only use if you will always be connected to the share, and do not use your home folder or other commonly used folders lest your file browser reads ahead into the disconnected folder

- Open the file `/etc/nsswitch.conf` and add an entry for automount:

```
automount: files
```
- When you are done configuring your templates (see below), launch the AutoFS daemon as root by enabling and starting the `autofs.service`.

Devices are now automatically mounted when they are accessed, they will remain mounted as long as you access them.

Removable media

- Open `/etc/autofs/auto.misc` to add, remove or edit miscellaneous devices. For instance:

```
/etc/autofs/auto.misc
#kernel    -ro                                          ftp.kernel.org:/pub/linux
#boot      -fstype=ext2                                 :/dev/hda1
usbstick   -fstype=auto,async,nodev,nosuid,umask=000    :/dev/sdb1
cdrom      -fstype=iso9660,ro                           :/dev/cdrom
#floppy    -fstype=auto                                 :/dev/fd0
```
If you have a CD/DVD combo-drive you can change the `cdrom` line with `-fstype=auto` to have the media type autodetected.

NFS network mounts

AutoFS provides a new way of automatically discovering and mounting NFS\-shares on remote servers (the AutoFS network template in `/etc/autofs/auto.net` has been removed in autofs5). To enable automatic discovery and mounting of network shares from all accessible servers without any further configuration, you will need to add the following to the `/etc/autofs/auto.master` file:

```
/net -hosts --timeout=60
```
Each host name needs to be resolveable, e.g. the name an IP address in `/etc/hosts` or via DNS and please make sure you have nfs-utils包 installed and working. You also have to enable RPC (systemctl start|enable rpcbind) to browse shared Folders.

For instance, if you have a remote server *fileserver* (the name of the directory is the hostname of the server) with an NFS share named */home/share*, you can just access the share by typing:

```
# cd /net/fileserver/home/share
```

注意：Please note that *ghosting*, i.e. automatically creating directory placeholders before mounting shares is enabled by default, although AutoFS installation notes claim to remove that option from `/etc/conf.d/autofs` in order to start the AutoFS daemon.

The `-hosts` option uses a similar mechanism as the `showmount` command to detect remote shares. You can see the exported shares by typing:

```
# showmount <servername> -e
```

Replacing *<servername>* with the name of your own server.

An alternative Way is to use the automount-service from Systemd, see NFS with systemd-automount

Manual NFS configuration

To mount a NFS share on server_name called /srv/shared_dir to another computer named client_pc at location /mnt/foo, edit auto.master and create a config file for the share (auto.server_name):

```
/etc/autofs/auto.master
/mnt   /etc/autofs/auto.server_name --timeout 60
/etc/autofs/auto.server_name
foo  -rw,soft,intr,rsize=8192,wsize=8192 server_name:/srv/shared_dir
```

Samba

The Arch package does not provide any Samba or CIFS templates/scripts (23.07.2009), but the following should work for single shares:

Add the following to `/etc/autofs/auto.master`:

```
/media/[my_server] /etc/autofs/auto.[my_server] --timeout=60 --ghost
```

where `--timeout` defines how many seconds to wait before the file system is unmounted. The `--ghost` option creates empty folders for each mount-point in the file in order to prevent timeouts, if a network share cannot be contacted.

Next create a file `/etc/autofs/auto.[my_server]`

```
[any_name] -fstype=cifs,[other_options] ://[remote_server]/[remote_share_name]
```

You can specify a user name and password to use with the share in the `other_options` section:

```
[any_name] -fstype=cifs,username=[username],password=[password],[other_options] ://
[remote_server]/[remote_share_name]
```

注意：Escape $, and other characters, with a backslash when neccessary.

FTP and SSH (with FUSE)

Remote FTP and SSH servers can be accessed seamlessly with AutoFS using FUSE, a virtual file system layer.

Remote FTP

First, install the curlftpfs包 package. Load the **fuse** module:

```
# modprobe fuse
```

Create a `/etc/modules-load.d/fuse.conf` file containg `fuse` to load it on each system boot.

Next, add a new entry for FTP servers in `/etc/autofs/auto.master`:

```
/media/ftp        /etc/autofs/auto.ftp    --timeout=60
```

Create the file `/etc/autofs/auto.ftp` and add a server using the `[ftp://myuser:mypassword@host:port/path]` `(ftp://myuser:mypassword@host:port/path)` format:

```
servername -
fstype=curl,rw,allow_other,nodev,nonempty,noatime    :ftp\://myuser\:mypassword\@remote
server
```

注意： Your passwords are plainly visible for anyone that can run `df` (only for mounted servers) or view the file `/etc/autofs/auto.ftp`.

If you want slightly more security you can create the file `~root/.netrc` and add the passwords there. Passwords are still plain text, but you can have mode 600, and `df` command will not show them (mounted or not). This method is also less sensitive to special characters (that else must be escaped) in the passwords. The format is:

```
machine remoteserver
login myuser
password mypassword
```

The line in `/etc/autofs/auto.ftp` looks like this without user and password:

```
servername -fstype=curl,allow_other    :ftp\://remoteserver
```

Create the file `/sbin/mount.curl` with this code:

```
/sbin/mount.curl
 #! /bin/sh
 curlftpfs $1 $2 -o $4,disable_eprt
```

Create the file `/sbin/umount.curl` with this code:

```
/sbin/umount.curl
 #! /bin/sh
 fusermount -u $1
```

Set the permissions for both files:

```
# chmod 755 /sbin/mount.curl
# chmod 755 /sbin/umount.curl
```

After a restart your new FTP server should be accessible through `/media/ftp/servername`.

Remote SSH

**本文或本章节的事实准确性存在争议。**

原因： All the *ssh** commands should be executed as the same user, as before this edit. It should not matter if it is root or unprivileged.（在 Talk:Autofs 中讨论）

These are basic instructions to access a remote filesystem over SSH with AutoFS.

注意：Password-less authentication may be convenient but also has security implications. See SSH keypair for more details

Install the sshfs包 package.

Load the `fuse` module:

```
# modprobe fuse
```

Create a `/etc/modules-load.d/fuse.conf` file containg `fuse` to load it on each system boot if you have not one yet.

Install openssh包.

Generate an SSH keypair:

```
$ ssh-keygen
```

When the generator ask for a passphrase, just press `ENTER`. Using SSH keys without a passphrase is less secure, yet running AutoFS together with passphrases poses some additional difficulties which are not (yet) covered in this article.

Next, copy the public key to the remote SSH server:

```
$ ssh-copy-id username@remotehost
```

As root, see that you can login to the remote server without entering a password:

```
# ssh username@remotehost
```

注意：This will add the remote server to root's list of `known_hosts`. Hosts can be also be manually added to `/etc/ssh/ssh_known_hosts`.

Create a new entry for SSH servers in `/etc/autofs/auto.master`:

```
/media/ssh           /etc/autofs/auto.ssh --timeout=60
```

Create the file `/etc/autofs/auto.ssh` and add an SSH server:

```
/etc/autofs/auto.ssh
servername     -
fstype=fuse,rw,allow_other,IdentityFile=/home/username/.ssh/id_rsa :sshfs\#username@hos
t\:/
```

After a restart your SSH server should be accessible through `/media/ssh/servername`.

MTP

Media Transfer Protocol (MTP) is used in some Android devices.

Install the mtpfs包 package.

Create a new entry for MTP Device in `/etc/autofs/auto.misc`:

```
android -fstype=fuse,allow_other,umask=000     :mtpfs
```

Troubleshooting and tweaks

This section contains a few solutions for common issues with AutoFS.

Using NIS

Version 5.0.5 of AutoFS has more advanced support for NIS. To use AutoFS together with NIS, add `yp:` in front of the template names in `/etc/autofs/auto.master`:

```
/home   yp:auto_home    --timeout=60
/sbtn   yp:auto_sbtn    --timeout=60
+auto.master
```

On earlier versions of NIS (before 5.0.4), you should add `nis` to `/etc/nsswitch.conf`:

```
automount: files nis
```

Optional parameters

You can set parameters like `timeout` systemwide for all AutoFS media in `/etc/default/autofs`:

- Open the `/etc/default/autofs` file and edit the `OPTIONS` line:

```
OPTIONS='--timeout=5'
```

- To enable logging (default is no logging at all), uncomment and add `--verbose` to the `OPTIONS` line in `/etc/default/autofs` e.g.:

```
OPTIONS='--verbose --timeout=5'
```

After restarting the `autofs` daemon, verbose output is visible in `systemctl status` or in `journalctl`.

Identify multiple devices

If you use multiple USB drives/sticks and want to easily tell them apart, you can use AutoFS to set up the mount points and Udev to create distinct names for your USB drives. See udev#Setting static device names for instructions on setting up Udev rules.

AutoFS permissions

If AutoFS is not working for you, make sure that the permissions of the templates files are correct, otherwise AutoFS will not start. This may happen if you backed up your configuration files in a manner which did not preserve file modes. Here are what the modes should be on the configuration files:

- 0644 - /etc/autofs/auto.master
- 0644 - /etc/autofs/auto.media
- 0644 - /etc/autofs/auto.misc
- 0644 - /etc/conf.d/autofs

In general, scripts (like previous `auto.net`) should have executable (`chmod a+x filename`) bits set and lists of mounts should not.

If you are getting errors in `/var/log/daemon.log` similar to this, you have a permissions problem:

```
May  7 19:44:16 peterix automount: lookup(program): lookup for petr failed
May  7 19:44:16 peterix automount: failed to mount /media/cifs/petr
```

fusermount problems

With certain versions of util-linux, you may not be able to unmount a fuse file system drive mounted by autofs, even if you use the "user=" option. See the discussion here: http://fuse.996288.n3.nabble.com/Cannot-umount-as-non-root-user-anymore-tp689p697.html

Debugging auto mount issues

For better debugging you might try running automount in foreground.

```
# systemctl stop autofs.service
# automount -f -v
```

Of if you want more debug info than try:

```
# automount -f --debug
```

Alternatives to AutoFS

- Systemd can automount filesystems upon demand; see here for the description and the article on sshfs for an example.
- Thunar Volume Manager is an automount system for users of the Thunar file manager.
- PCManFM is a lightweight file manager with built-in support for accessing remote shares
- Udisks is a minimalistic automatic disk mounting service

See also

- FTP and SFTP usage with AutoFS is based on this Gentoo Wiki article: https://web.archive.org/web/20130414074212/http://en.gentoo-wiki.com/wiki/Mounting_SFTP_and_FTP_shares
- More information on SSH can be found on the SSH and Using SSH Keys pages of this wiki.
- Ubuntu's Autofs help wiki is at https://help.ubuntu.com/community/Autofs
- mount(8) § FILESYSTEM-INDEPENDENT MOUNT OPTIONS
- For fuse specific mount options check https://manpages.ubuntu.com/manpages/precise/man8/mount.fuse.8.html

1.16 Bcache

难度 进阶 | **篇幅** 19.4 KB | **所属** 文件系统

本节将讲到

- 在现存的系统上配置bcache设备
- 将系统安装到一个 bcache 设备
- 从安装盘访问bcache分区
- 配置
- 高级操作

Bcache (block 缓存) 允许使用固态硬盘作为读写缓存（writeback模式）或者读缓存（writethrough 或者 writearound模式）来为另一个 block 设备（通常是机械硬盘或硬盘阵列）加速。这篇文章会展示如何把 Bcache 作为根分区安装 Arch Linux。如果要看对于 bcache 本身的介绍，请浏览 bcache 首页。请一定要阅读并参考 bcache 手

册。Bcache 从 Linux 内核 3.10 版本开始就已经并入 mainline，而 Arch Linux 的安装盘自从 2013.08.01 版本开始就支持 bcache。

类似 Bcache 的还有 Facebook 的 Flashcache。

**本文或本章节的事实准确性存在争议。**

原因： Backwards incompatible changes to on-disk format for linux 3.18 appears to be false. bcache on-disk format changes have not yet landed upstream yet as of 3.19（在 Talk:Bcache 中讨论）

Bcache 需要把后端设备格式化成 bcache block 设备。多数情况下，blocks to-bcache 可以用来做转换。

警告：* 一定要先备份数据！

- bcache-dev 分支正处于非常不成熟的开发阶段。The on-disk format has undergone changes in 3.18 which are not backwards compatible with previous formats. Note: This only applies to users who compile-in bcache-dev. The version built-in to the upstream Linux kernel is unaffected.
- Bcache and btrfs could leave you with a corrupted filesystem. Please visit this post for more information. Btrfs wiki reports that it was fixed in kernels 3.19+ .

在现存的系统上配置bcache设备

警告：make-bcache **不会** 导入(import)一个现存的设备或分区——它会格式化它。请务必做好数据备份。

1\. 从 AUR 安装 bcache-toolsAUR。

2\. 创建一个后端设备（通常来说这是你的机械硬盘）。后端设备可以是整个设备、一个分区或者其他任何的block设备。这将会创建/dev/bcache0：

```
# make-bcache -B /dev/sdx1
```

3\. 创建一个缓存设备（这通常是你的固态硬盘）。缓存设备可以是整个设备、一个分区或者其他任何的block设备：

```
# make-bcache -C /dev/sdy2
```

在这个例子里，默认的block大小是512B、bucket大小是128kB。block的大小应该与后端设备的sector大小匹配（通常是512或者4k）。bucket的大小应该与缓存设备的擦除block大小匹配（以减少写入放大）。例如，如果是一个4k sector的HDD和一个擦除block大小是2MB的SSD搭配，命令就应该是这样的：

```
# make-bcache --block 4k --bucket 2M -C /dev/sdy2
```

4\. 把缓存设备注册到后端设备。为了找到它的*cache set UUID*，运行`# bcache-super-show /dev/sdy2 | grep cset.uuid`然后把它添加到bcache设备。Udev规则会在以后的重启阶段自动搞定这个，因此你只需要执行一次这个命令：

```
# echo cset.uuid > /sys/block/bcache0/bcache/attach
```

5\. 更改你的缓存模式（如果你想要同时缓存读和写）：

```
# echo writeback > /sys/block/bcache0/bcache/cache_mode
```

6\. 如果你需要让这个分区在initcpio的时候就已经可以使用（即，在系统启动过程中需要用到它），你需要把'bcache'添加到`/etc/mkinitcpio.conf`文件里：

```
/etc/mkinitcpio.conf
...
MODULES=(... bcache)
...
HOOKS=(... block ... bcache ... filesystems ...)
...
```

然后请重新生成initramfs镜像：

```
# mkinitcpio -p linux
```

管理 Bcache

1\. 确认所有的东西都已经正确地配置了：

```
# cat /sys/block/bcache0/bcache/state
```

输出的内容有以下可能：

- **no cache**: 这代表你还没有绑定缓存设备到你的后端设备上
- **clean**: 这代表一切正常，缓存是clean的
- **dirty**: 这代表一切正常，缓存模式被设置成了*writeback*，缓存是dirty的
- **inconsistent**: 这代表问题很大，后端设备与缓存设备没有同步

使用一个没有缓存设备的 `/dev/bcache0` 的话所有的IO都会直接在后端设备上执行，等于pass-through模式。

2\. 查看正在使用的缓存模式：

```
# cat /sys/block/bcache0/bcache/cache_mode
[writethrough] writeback writearound none
```

在这个示例中，输出显示当前使用的是*writethrough*模式。

3\. 显示bcached设备的信息：

```
# bcache-super-show /dev/sdXY
```

4\. 停止后端设备：

```
# echo 1 > /sys/block/sdX/sdX[Y]/bcache/stop
```

5\. 让缓存设备脱机：

```
# echo 1 > /sys/block/sdX/sdX[Y]/bcache/detach
```

6\. 安全移除缓存设备：

```
# echo <cache-set-uuid> > /sys/block/bcache0/bcache/detach
```

7\. 释放已连接的设备：

```
# echo 1 > /sys/fs/bcache/<cache-set-uuid>/stop
```

将系统安装到一个 bcache 设备

1\. 从 Arch Linux 的USB安装盘启动 (至少是 2013.08.01 版本的)。

2\. 从 AUR 安装 bcache-toolsAUR。

3\. 对硬盘进行分区

注意：虽然 Grub2 理论上不提供对 bcache 的支持，但是因为它完整支持 UEFI ，所以是有办法启动系统的。只要用来处理 boot 设备的模块被正确编译进内核或者包含在 initramfs 里面，那么就没必要按照下面的例子单独分区一个 boot 分区（第二行，名字是 arch_boot 的）。详情参考 GRUB 和 UEFI。

grub 无法处理 bcache, 所以你需要至少两个分区 (一个 boot 和一个作为 bcache 后端设备的分区). 如果你用的是 UEFI, 则额外需要 EFI system partition (ESP)。比如:

```
     1            2048           22527   10.0 MiB    EF00  EFI System
     2           22528          432127   200.0 MiB   8300  arch_boot
     3          432128       625142414   297.9 GiB   8300  bcache_backing
```

注意：例子里是没有 swap 文件或分区的，如果要让 swap 也受到 cache 设备缓存，使用步骤7里面的 LVM 来实现。如果不需要 swap 被缓存，请在这一步就创建 swap 分区。

4\. 配置你的机械硬盘作为 bcache 后端设备。

```
# make-bcache -B /dev/sda3
```

注意：

- 在配置启动盘的时候，一定要明白自己在做什么——多看几次你选择的 boot-loader/boot-manager 的文档，想想它和 bcache 一起用会不会有兼容性问题。
- 如果所有相关的设备都通过下面这样的命令连接起来的话，bcache 会自动把缓存设备连接到后端设备，步骤5就可以跳过。`# make-bcache -B /dev/sd? /dev/sd? -C /dev/sd?`

现在你有了一个 `/dev/bcache0`。

5\. 配置固态硬盘。

下面这个例子是把整个固态硬盘（`/dev/sdb`）作为缓存设备格式化，并链接到后端设备：

```
# make-bcache -C /dev/sdb
# echo /dev/sdb > /sys/fs/bcache/register
# echo 前面的命令输出的UUID > /sys/block/bcache0/bcache/attach
```

注意：如果忘了 UUID 是什么，可以用 `ls /sys/fs/bcache/` 命令找到（如果缓存设备成功链接），最长的那一串东西就是 UUID。

6\. 格式化 bcache 设备。如果你要在 `/dev/bcache0` 里面分区的话，用 LVM 或者 btrfs 的 子卷来实现，比如下面的例子用 btrfs 的子卷功能分了 `/` 和 `/home` 两个区：

```
# mkfs.btrfs /dev/bcache0
# mount /dev/bcache0 /mnt/
# btrfs subvolume create /mnt/root
# btrfs subvolume create /mnt/home
# umount /mnt
```

你甚至可以配置 LUKS 如果你想要用类似 cryptsetup 的东西。把内核 option 'cryptdevice' 指向 bcache 设备是没有问题的。

7\. 配置安装挂载点（这里用的是步骤3的分区方案和步骤6的btrfs子卷分区方案）：

```
# mkfs.ext4 /dev/sda2 （arch_boot 分区）
# mkfs.msdos /dev/sda1 （如果你的 ESP 分区有至少 500MB，请用 mkfs.vfat 命令来创建一个 FAT32 分区）
# pacman -S arch-install-scripts
# mount /dev/bcache0 -o subvol=root,compress=lzo /mnt/ （挂载btrfs的root子卷作为根目录）
# mkdir /mnt/boot /mnt/home
# mount /dev/bcache0 -o subvol=home,compress=lzo /mnt/home （挂载btrfs的home子卷作为home目录）
# mount /dev/sda2 /mnt/boot
# mkdir /boot/efi
# mount /dev/sda1 /mnt/boot/efi/
```

8\. 按照 Installation Guide 的步骤安装系统，除了下面这个：

在修改 `/etc/mkinitcpio.conf` 并运行 `mkinitcpio -p linux` 之前：

- 在新系统上，从 AUR 安装 bcache-toolsAUR。
- 修改 `/etc/mkinitcpio.conf`:
- 添加 bcache 到 MODULES=() 里面
- 添加 bcache 到 HOOKS=() 的 block 和 filesystems 之间

注意：因为安装介质不会储存任何更改，所以如果你想在重启之后再在安装介质里打开bcache的后端设备，你需要手动注册它。首先确认 bcache 模块已经加载，然后把相关的设备都 echo 到 `/sys/bcache/register`。可以通过 `dmesg` 来检查是否已经成功注册。

从安装盘访问bcache分区

要访问一个在安装盘启动之前就已经存在的 bcache 分区，首先从 AUR 安装 bcache-toolsAUR，然后加载 bcache 模块：

```
# modprobe bcache
```

然后强制内核重新加载分区表：

```
# partprobe
```

现在 `/dev/bcache*` 应该会存在了，挂载、格式化之类的操作也可以进行了。

配置

可以配置的选项有很多，比如缓存模式、缓存写入时间间隔、启发式顺序写入等。通过写入 `/sys/block/bcache[0-9]/bcache` 目录中的对应文件可以进行相应的配置，详情查看 bcache 用户文档。

例如，改变缓存模式通过以下命令实现：

```
echo 'writethrough' > /sys/block/bcache[0-9]/bcache/cache_mode
```

可以把 'writethrough' 换成 'writeback', 'writearound' 或者 'none' 中的任意一个。

注意：写入 /sys 的设置有些是临时的，重启之后就会失效（缓存模式的更改不会失效）。如果要“模拟”永久更改，在 `/etc/tmpfile.d` 目录中创建包含类似下面的内容的 *.conf* 文件：

```
/etc/tmpfile.d/my-bcache.conf
w /sys/block/bcache0/bcache/sequential_cutoff - - - - 1M
w /sys/block/bcache0/bcache/cache_mode        - - - - writeback
```

在这个例子中该文件会在系统启动时把 sequential cutoff 设置为1M、cache mode 设置为 writeback。

高级操作

调整后端设备的容量

It is possible to resize the backing device so long as you do not move the partition start. This process is described on the mailing list. Here is an example using btrfs volume directly on bcache0. For LVM containers or for other filesystems, procedure will differ.

扩容举例

In this example, I grow the filesystem by 4GB.

1\. Reboot to a live CD/USB Drive (need not be bcache enabled) and use fdisk, gdisk, parted, or your other favorite tool to delete the backing partition and recreate it with the same start and a total size 4G larger.

警告：Do not use a tool like GParted that might perform filesystem operations! It will not recognize the bcache partition and might overwrite part of it!!

2\. Reboot to your normal install. Your filesystem will be currently mounted. That is fine. Issue the command to resize the partition to its maximum. For btrfs, that is

```
# btrfs filesystem resize max /
```

For ext3/4, that is:

```
# resize2fs /dev/bcache0
```

缩容举例

这里举例的是缩小4G容量。

1\. 切换缓存模式，从 writeback 改成 writethrough，并等待缓存写入后端设备。

```
# echo writethrough > /sys/block/bcache0/bcache/cache_mode
$ watch cat /sys/block/bcache0/bcache/state
```

请等到第二个命令输出 "clean"。

强制把缓存写入后端设备

```
 # echo 0 > /sys/block/bcache0/bcache/writeback_percent
```

这会把缓存比例改成0，从而强制把缓存的内容写入到后端设备。

然后把缓存比例改回来：

```
 # echo 10 > /sys/block/bcache0/bcache/writeback_percent
```

2\. 为了避免之后缩小分区的时候不会破坏文件系统，请把文件系统缩小比想要缩小的大小更多的量。比如例子里要把分区缩小4G，那就要把文件系统缩小5G以避免破坏了文件系统。如果用的是 btrfs，用下面的命令来缩小文件系统：

```
# btrfs filesystem resize -5G /
```

对于 ext3/4，用的命令是 *resize2fs*，但是需要目标文件系统没有处于挂载：

```
$ df -h /home
/dev/bcache0    290G   20G   270G   1% /home
# umount /home
# resize2fs /dev/bcache0 283G
```

3\. 重启到 LiveCD/USB盘 (并不需要支持 bcache) 并使用 fdisk、gdisk、parted或者其他你用得顺手的工具去把后端分区删掉并重新创建一个 start 位置一样的、但是总大小少4G的分区。

警告：不要使用类似 Gparted 的工具，因为它们会进行文件系统操作！它们可能识别不出来 bcache 分区并破坏它！

4\. 重启到原来的系统，现在你的文件系统会挂载好，并且分区已经缩小了4G。现在要做的是把文件系统扩展到分区的大小。对于 btrfs，命令是：

```
# btrfs filesystem resize max /
```

对于 ext3/4：

```
# resize2fs /dev/bcache0
```

5\. 把缓存模式重新改成 writeback （如果你想用 writeback 的话）：

```
# echo writeback > /sys/block/bcache0/bcache/cache_mode
```

注意：如果你够小心，第二步里面文件系统可以刚好缩小4G而不预留更多的空间，但是很多分区工具并不是按照字面意思来工作的，它们会把分区的起始和终止位置对齐到底层硬盘的 sector 边界，所以就会出现意料之外的文件系统破坏事故。小心为妙。

疑难解答

启动时 /dev/bcache 不存在

如果你在启动时见到下面的提示信息并进入了busybox shell:

```
ERROR: Unable to find root device 'UUID=b6b2d82b-f87e-44d5-bbc5-c51dd7aace15'.
You are being dropped to a recovery shell
    Type 'exit' to try and continue booting
```

可能是因为后端设备设置成了"writeback"模式 (默认是 writearound)。在"writeback"模式下 /dev/bcache0 在缓存设备注册并连接之前是不存在的。注册(register)是每次启动时需要进行的操作，但是连接(attach)只需要进行一次。

To continue booting, try one of the following:

- Register both the backing device and the caching device

```
# echo /dev/sda3 > /sys/fs/bcache/register
# echo /dev/sdb > /sys/fs/bcache/register
```

If the /dev/bcache0 device now exists, type exit and continue booting. You will need to fix your initcpio to ensure devices are registered before mounting the root device.

注意：

- An error of "sh: echo: write error: Invalid argument" means the device was already registered or is not recognized as either a bcache backing device or cache. If using the udev rule on boot it should only attempt to register a device if it finds a bcache superblock
- This can also happen if using udev's 69-bcache.rules in Installation's step 7 and blkid and bcache-probe "disagree" due to rogue superblocks. See bcache's wiki for a possible explanation/resolution.
- Re-attach the cache to the backing device:

If the cache device was registered, a folder with the UUID of the cache should exist in `/sys/fs/bcache`. Use that UUID when following the example below:

```
# ls /sys/fs/bcache/
b6b2d82b-f87e-44d5-bbc5-c51dd7aace15     register     register_quiet
# echo b6b2d82b-f87e-44d5-bbc5-c51dd7aace15 > /sys/block/sda/sda3/bcache/attach
```

If the `/dev/bcache0` device now exists, type exit and continue booting. You should not have to do this again. If it persists, ask on the bcache mailing list.

注意：An error of `sh: echo: write error: Invalid argument` means the device was already attached. An error of `sh: echo: write error: No such file or directory` means the UUID is not a valid cache (make sure you typed it correctly).

- Invalidate the cache and force the backing device to run without it. You might want to check some stats, such as "dirty_data" so you have some idea of how much data will be lost.

```
# cat /sys/block/sda/sda3/bcache/dirty_data
-3.9M
```

dirty data is data in the cache that has not been written to the backing device. If you force the backing device to run, this data will be lost, even if you later re-attach the cache.

```
# cat /sys/block/sda/sda3/bcache/running
0
# echo 1 > /sys/block/sda/sda3/bcache/running
```

The `/dev/bcache0` device will now exist. Type exit and continue booting. You might want to unregister the cache device and run make-bcache again. An fsck on `/dev/bcache0` would also be wise. See the bcache documentation.

警告：Only invalidate the cache if one of the two options above did not work.

/sys/fs/bcache/ 不存在

当前尝试启动的内核不支持bcache或者bcache内核模块未加载。

参考资料

- Bcache Homepage
- Bcache Manual

1.17 XFS

难度 高级 | **篇幅** 25.1 KB | **所属** 文件系统

本节将讲到

- 预备
- 创建
- 性能
- 管理
- 故障排除

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 未完成（在 Talk:XFS# 中讨论）

XFS 是由硅谷图形公司（Silicon Graphics, Inc.）开发的高性能日志式文件系统。XFS 因其基于分配组 （allocation group）的设计而特别擅长并行 IO。当该文件系统跨越多个存储设备时，这种设计使得 IO 线程数、文件系统带宽、文件和文件系统大小都具有极大的可伸缩性。

预备

为了使用 XFS 用户空间实用程序，请安装 xfsprogs包 软件包。它包含了管理 XFS 文件系统所需的必要工具。

创建

可以使用如下命令在 *device* 上创建新文件系统：

```
# mkfs.xfs device
```

一般来说，默认选项是最适合正常使用的。

输出示例：

```
meta-data=/dev/device             isize=256    agcount=4, agsize=3277258 blks
         =                        sectsz=512   attr=2
data     =                        bsize=4096   blocks=13109032, imaxpct=25
         =                        sunit=0      swidth=0 blks
naming   =version 2               bsize=4096   ascii-ci=0
log      =internal log            bsize=4096   blocks=6400, version=2
         =                        sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                    extsz=4096   blocks=0, rtextents=0
```

提示：

- 可以使用 `-L *label*` 选项来为文件系统分配标签。
- 在对已包含文件系统的块设备使用 *mkfs.xfs* 时，需使用 `-f` 选项来覆盖掉原有的文件系统。***这会完全清除旧文件系统中的所有数据！***

注意：“创建 XFS 文件系统后就无法缩容，但可以通过 xfs_growfs 进行扩容。”具体请参考#调整大小。

数据完整性

xfsprogs包 3.2.0 引入了一种新型磁盘格式（v5），其包含了称为自描述元数据（Self-Describing Metadata）的元数据校验方案。 基于 CRC32，它提供的额外保护措施可以防止元数据损坏（例如在意外断电时）。当使用 xfsprogs包 3.2.3 或更高版本时，这种校验默认是打开的。如果需要在旧版内核中挂载 XFS 为可读写，可以在调用 mkfs.xfs(8) 时加上 `-m crc=0` 来关闭校验特性：

```
# mkfs.xfs -m crc=0 /dev/target_partition
```

注意：禁用元数据校验同时会禁用下面提到的 #Free inode btree、#Reverse mapping btree 和 #大时间戳（Big timestamps）功能，以及“reference count btrees”（具体信息请参考 mkfs.xfs(8) § OPTIONS）。

自 Linux 内核版本 3.15 起，XFS v5 磁盘格式被视作稳定特性，可用于生产环境。

注意：与 Btrfs 和 ZFS 不同，XFS 中的 CRC32 校验仅用于元数据而非实际数据。

空闲 inode B+ 树

自 Linux 3.16 起，XFS 增加了一个 B+ 树用于索引未被使用的 inode。它等同于现有的索引已使用 inode 的 B+ 树，不同之处在于索引未用 inode 的 B+ 树会跟踪至少包含一个未用 inode 的 inode 块。这一设计的目的是改进分配 inode 时寻找未用 inode 簇的性能。它可以提高长期使用后的文件系统性能，比如你在数月或数年之间已经向文件系统写入或删除了数百万的文件。使用这个功能不会影响整个文件系统的可靠性程度或恢复能力。

这个功能依赖于新的 v5 磁盘格式，自 Linux 内核 3.15 版本起它被视作为可用于生产环境的稳定特性。它没有改变磁盘上原本的数据结构，但会添加了一个与分配 inode 的 B+ 树保持一致的结构；因此，旧版本的内核只能将带有 B+ 树功能的文件系统挂载为只读模式。

当使用 xfsprogs 3.2.3 或更高版本时这个功能默认是开启的。如果你需要一个旧版本内核可写入的文件系统，这个功能可以在格式化 XFS 分区时用 `finobt=0` 开关来关闭。你还需要把它和 `crc=0` 一起使用：

```
# mkfs.xfs -m crc=0,finobt=0 /dev/target_partition
```

也可以简写为（`finobt` 依赖于 `crc`）：

```
# mkfs.xfs -m crc=0 /dev/target_partition
```

Reverse mapping btree

The reverse mapping btree is at its core:

: a secondary index of storage space usage that effectively provides a redundant copy of primary space usage metadata. This adds some overhead to filesystem operations, but its inclusion in a filesystem makes cross-referencing very fast. It is an

essential feature for repairing filesystems online because we can rebuild damaged primary metadata from the secondary copy. : The feature graduated from EXPERIMENTAL status in Linux 4.16 and is production ready. However, online filesystem checking and repair is (so far) the only use case for this feature, so it will remain opt-in at least until online checking graduates to production readiness.

From mkfs.xfs(8) § OPTIONS:

: The reverse mapping btree maps filesystem blocks to the owner of the filesystem block. Most of the mappings will be to an inode number and an offset, though there will also be mappings to filesystem metadata. This secondary metadata can be used to validate the primary metadata or to pinpoint exactly which data has been lost when a disk error occurs.

See also and for more information.

This feature is enabled by default for new filesystems as of xfsprogs 6.5.0.

大时间戳（Big timestamps）

Starting in Linux 5.10, XFS supports using refactored "timestamp and inode encoding functions to handle timestamps as a 64-bit nanosecond counter and bit shifting to increase the effective size. This now allows XFS to run well past the Year 2038 problem to now the Year 2486. Making a new XFS file-system with *bigtime* enabled allows a timestamp range from December 1901 to July 2486 rather than December 1901 to January 2038." The feature will also allow quota timer expirations from January 1970 to July 2486 rather than January 1970 to February 2106.

Big timestamps are enabled by default for new filesystems as of xfsprogs 5.15.

升级

可以通过 xfs_info(8) 检查现有文件系统是否已启用大时间戳：

```
# xfs_info / | grep bigtime
... bigtime=0 ...
```

在 xfsprogs包 5.11 及更新版本上，可以使用 xfs_admin(8) 来升级未挂载的现有文件系统：

```
# xfs_admin -O bigtime=1 device
```

也可以使用 xfs_repair(8)：

```
# xfs_repair -c bigtime=1 device
```

另外可以考虑顺便启用 `inobtcount`（这是另一个新的默认配置项）。

性能

摘自 XFS FAQ：

: 默认参数已针对性能进行过优化。mkfs.xfs 可以检测到单盘与 MD/DM RAID 配置环境间的区别，并根据环境自动修改文件系统的默认参数。 : 基本上你只需在使用对硬件 RAID 时为 `mkfs.xfs` 指定条带单元和宽度。

（详细内容请参考 #带区大小和宽度）

提示：当在 RAID 设备上使用 XFS 文件系统时，可通过使用 `largeio` 和 `swalloc` 值，以及比默认情况更大的 `logbsize` 和 `allocsize` 值等来提高性能。下列文章能提供更多有关详情：

- https://www.beegfs.io/wiki/StorageServerTuning
- https://help.marklogic.com/Knowledgebase/Article/View/505/0/recommended-xfs-settings-for-marklogic-server

: 对于挂载选项，只有 `logbsize` 会对元数据性能产生可观影响。增加 `logbsize` 可以降低特定工作负载下日志 IO 的数量，但如果系统在进行大量修改时崩溃，那么恢复后可能会丢失更多的修改操作。

提示：所有可用挂载选项请参考 xfs(5)。

: 从内核 3.2.12 版本开始，默认的 I/O 调度器 CFQ 将使 XFS 的并行化大打折扣。

注意：在检测到 SATA 或 NVMe SSD 时，Arch 会自动配置为不使用 I/O 调度器，该行为可通过读取 `/sys/block/nvme*n*/queue/scheduler` 中的内容进行验证。

因此基本上参照#创建即可获得最佳性能。

带区大小和宽度

如果这个文件系统位于条带化的 RAID 上，可以在 mkfs.xfs(8) 命令中指定带区大小来获得显著的性能提升。

XFS 有时可以检测到软 RAID 下的几何形 (geometry), 但万一您要重塑其或正在使用硬 RAID, 请参阅如何计算出正确的 sunit 和 swidth 值以获得最佳性能

访问时间记录

某些文件系统可以通过在 `/etc/fstab` 文件中添加 `noatime` 挂载选项来增强性能。对于 XFS 文件系统来说，默认的访问时间记录行为是 `relatime`，与 noatime 相比这几乎没有额外开销，且仍然可以记录正确的访问时间。所有 Linux 文件系统现在都以这个选项为默认值（从大约 2.6.30 版本开始），但是 XFS 从 2006 年开始就采用了类似 relatime 的特性，因此不需要出于性能考虑而在 XFS 上使用 noatime。

更多信息请参考 Fstab#atime 参数。

Discard

Despite XFS supporting async discard since kernel 4.7, xfs(5) still recommends "that you use the fstrim application to discard unused blocks rather than the discard mount option because the performance impact of this option is quite severe."

See 固态硬盘#定期 TRIM.

磁盘碎片整理

尽管 XFS 本质上基于区段 (Extent) 并且延迟分配策略很大程度上增强了它对磁盘碎片的抗性，XFS 仍然提供了磁盘碎片整理程序（*xfs_fsr*，XFS filesystem reorganizer 的缩写），它可以在已挂载且活动的 XFS 文件系统上整理碎片。定期查看 XFS 碎片也很有用。

xfs_fsr(8) 可以改进已挂载文件系统的文件组织。该重组织算法一次操作一份文件，对文件进行压实或改进文件区段布局（改成连续数据块）。

检查碎片程度

查看当前文件系统中有多少磁盘碎片：

```
# xfs_db -c frag -r /dev/partition
```

进行碎片整理

要启动碎片整理，使用 xfs_fsr(8) 命令：

```
# xfs_fsr /dev/partition
```

去重

The *reflink* feature, available since kernel version 4.9 and enabled by default since `mkfs.xfs` version 5.1.0, allows creating fast reflink'ed copies of files as well as deduplication after the fact, in the same way as btrfs:

Reflink copies

Reflink copies initially use no additional space:

```
$ cp --reflink bigfile1 bigfile2
```

Until either file is edited, and a copy-on-write takes place. This can be very useful to create snapshots of (large) files.

去重

现有文件系统可使用 duperemove包 或 util-linux包 的 hardlink(1) 工具进行去重。

外部 XFS 日志

使用外部日志（元数据日志）可能对提高性能很有帮助 (例如在 SSD 上)。请参阅 mkfs.xfs(8) 获取有关 `logdev` 参数的更多详情.

警告：当心：使用闪存的情况下可能会减损硬盘寿命。请参阅性能优化#减少磁盘读写获取有关 SSD 寿命减损的详情。

要在创建 XFS 文件系统时保留指定大小的外部日志，请为 `mkfs.xfs` 命令指定 `-l logdev=*device*,size=*size*` 选项。如果省略 `size` 参数，则会使用基于文件系统大小的日志大小。要在挂载 XFS 文件系统时让其使用外部日志，请为 mount 命令指定 `-o logdev=*device*` 选项。

同步间隔

XFS 有其专有的 sysctl 变量来设置“回写间隔”,默认为 3000.

警告：提高该值可能会提高性能，但同时会提高断电时数据丢失的严重程度。

```
/etc/sysctl.d/20-xfs-sync-interval.conf
fs.xfs.xfssyncd_centisecs = 10000
```

管理

调整大小

XFS 支持通过 xfs_growfs(8) 在线调整大小：

```
# xfs_growfs -D size /path/to/mnt/point
```

如果缺省 `-D size` 参数，那文件系统会自动扩大到可能的最大大小（即分区大小）。

注意：根据 xfs_growfs(8)，xfs 尚未完全支持缩容：

“只有 1 AG 大小的文件系统无法缩容，且无法将文件系统缩到 1 AG 大小，其中 *AG* 指的是 *分配组*。”

在线元数据检查（scrub）

警告：该程序目前是**实验性**的，这意味着它的行为和接口可能随时发生变化。详情请见 xfs_scrub(8)。

`xfs_scrub` 请求内核检查 XFS 文件系统中的所有元数据对象。内核会扫描元数据记录以查找明显错误的值，然后与其它元数据进行交叉引用。其目的是通过检查单个元数据记录与文件系统中其它元数据的一致性，建立对整个文件系统一致性的合理置信度。如果存在完整的冗余数据结构，则可以根据其它元数据重建损坏的元数据。

启用/启动 `xfs_scrub_all.timer` 以定期在线检查所有 XFS 文件系统的元数据。

注意：有时可能需要编辑 `xfs_scrub_all.timer`，且如果错过了上一次执行时间（例如因为系统关机/断电），计时器将会立即激发。

修复

注意："Unlike other Linux file systems, *xfs_repair* does not run at boot time, even when an XFS file system was not cleanly unmounted. In the event of an unclean unmount, *xfs_repair* simply replays the log at mount time, ensuring a consistent file system."

From Checking and Repairing an XFS File System (emphasis ours):

: If you can't mount an XFS file system, you can use the `xfs_repair -n` command to check its consistency. Typically, you would **only run this command on the device file of an unmounted file system** that you believe has a problem. The `xfs_repair -n` command displays output to indicates changes that would be made to the file system in the case where it would need to complete a repair operation, but doesn't modify the file system directly. : If you can mount the file system and you don't have a suitable backup, you can use the *xfsdump* command to back up the existing file system data. However, note that the command might fail if the file system's metadata has become corrupted. : You can use the *xfs_repair* command to attempt to repair an XFS file system specified by its device file. The command replays the journal log to fix any inconsistencies that might have resulted from the file system not being cleanly unmounted. Unless the file system has an inconsistency, you typically don't need to use the follwoing command, as the journal is replayed every time that you mount an XFS file system.

: ```

```
: If the journal log has become corrupted, you can reset the log by specifying the `-L` option to *xfs_repair*.

: > **警告：**
>
> - The *xfs_repair* utility cannot repair an XFS file system with a dirty log. To clear the log, mount and unmount the XFS file system. If the log is corrupt and cannot be replayed, use the `-L` option ("force log zeroing") to clear the log, that is, `xfs_repair -L /dev/*device*`. Be aware that this may result in further corruption or data loss.
> - Resetting the log can leave the file system in an inconsistent state, resulting in data loss and data corruption. Unless you're experienced with debugging and repairing XFS file systems by using the *xfs_db*, it is recommended that you instead recreate the file system and restore its contents from a backup.

: If you can't mount the file system or you don't have a suitable backup, running *xfs_repair* is the only viable option, unless you're experienced in using the *xfs_db* command.
: *xfs_db* provides an internal command set that allows you to debug and repair an XFS file system manually. The commands enable you to perform scans on the file system, and navigate and display its data structures. If you specify the `-x` option to enable expert mode, you can modify the data structures.

: ```
# xfs_db [-x] device
```

: For more information, see the xfs_db(8) and xfs_repair(8), and the *help* command within *xfs_db*.

See also Which factors influence the memory usage of xfs_repair? and XFS Repair.

数据恢复

Even when being mounted read-only with `mount -o ro` an XFS file system's log will be replayed if it has not been unmounted cleanly.

There may be situations where a compromised XFS file system on a damaged storage device should be mounted read-only, so that files may be copied off it

hopefully without causing further damage, yet it cannot be mounted because it has not been unmounted cleanly and is damaged to such an extent that the log cannot be replayed. Also, consider that replaying the log means writing to the compromised file system, which might be a bad idea in itself.

To mount an XFS file system without writing to it in any way and without replaying the log, use `mount -o ro,norecovery`.

撤销删除

xfs_undelete-gitAUR 可以从未挂载或只读挂载的 XFS 文件系统中恢复被删除的文件，但存在一定限制。更多信息请参考 https://github.com/ianka/xfs_undelete 。

故障排除

根文件系统配额

XFS 配额挂载选项（`uquota`、`gquota`、`prjquota` 等）会在重新挂载文件系统时失效。要对根文件系统启用配额功能，这个挂载选项需要作为内核参数 `rootflags=` 传递给初始化内存盘（initramfs）。在随后的启动过程中，这个选项不需要在 `/etc/fstab` 中挂载根（`/`）文件系统的挂载选项里再次列出。

注意：XFS 配额相较于标准 Linux 磁盘配额有一些区别，可以参考下 https://inai.de/linux/adm_quota 这篇文章。

如果用户“nobody”无法访问挂载点，xfs_scrub_all 会执行失败

当执行 `xfs_scrub_all` 时，它将为每个已挂载的 XFS 文件系统启动 `xfs_scrub@.service` 服务。这项服务以用户 `nobody` 身份运行，所以如果 `nobody` 无法进入目录时，命令执行将会失败，并随附以下错误：

```
xfs_scrub@mountpoint.service: Changing to the requested working directory failed: Permission denied
xfs_scrub@mountpoint.service: Failed at step CHDIR spawning /usr/bin/xfs_scrub: Permission denied
xfs_scrub@mountpoint.service: Main process exited, code=exited, status=200/CHDIR
```

为了能让对应服务运行，请更改挂载点的权限以使用户 `nobody` 拥有执行权限。

fsck.xfs fails in systemd-based initramfs

When using a mkinitcpio\-generated systemd based initramfs without the `base` hook, you will see the following messages in the journal:

```
systemd-fsck: fsck: /usr/bin/fsck.xfs: execute failed: No such file or directory
systemd-fsck: fsck failed with exit status 8.
systemd-fsck: Ignoring error.
```

This is because fsck.xfs(8) is a shell script and requires `/bin/sh` to execute. `/usr/bin/sh` is provided by the `base` hook, so the solution is to prepend it to the HOOKS array in `/etc/mkinitcpio.conf`. E.g.:

```
HOOKS=(base systemd ... )
```

参考资料

- XFS wiki (archive)
- XFS FAQ
- Improving Metadata Performance By Reducing Journal Overhead
- XFS Wikipedia Entry
- XFS User Guide\[失效链接 2024-03-03 ⓘ\] XFS User Guide no longer exists but has a link to the git repository

1.18 Snapper

难度 高级 | **篇幅** 29.6 KB | **所属** 文件系统

本节将讲到

- 安装
- 建立一个新的配置
- 创建快照
- 管理快照
- 提示与技巧

Snapper 是一个由 openSUSE 的 Arvin Schnell 开发的工具，用于管理 Btrfs 子卷和 LVM 精简配置(thin-provisioned)卷。它可以创建和比较快照，在快照间回滚，并支持自动按时间序列创建快照。

安装

安装 snapper包 包。或者安装开发版本 snapper-gitAUR。

此外，还可以安装 GUI 前端 snapper-gui-gitAUR、btrfs-assistant包 或 snapper-toolsAUR。

建立一个新的配置

在为 btrfs 子卷建立一个 snapper 配置前，这个子卷必须已经存在。否则，你应该在建立 snapper 配置前创建它。

要为位置为 `*/path/to/subvolume*` 的 btrfs 子卷创建一个新的 snapper 配置文件，并命名为 `*config*`：

```
# snapper -c config create-config /path/to/subvolume
```

这将会：

- 根据 `/usr/share/snapper/config-templates/default` 处的默认配置模板创建一个配置文件 `/etc/snapper/configs/*config*`。

- 在 */path/to/subvolume*/.snapshots 处创建一个子卷，用于存储未来该配置文件产生的子卷。子卷的路径将会是 */path/to/subvolume*/.snapshots/*#*/snapshot，*#* 是子卷序号。
- 将 *config* 加入到 /etc/conf.d/snapper 的 SNAPPER_CONFIGS 中。

例如，要为挂载在 / 的子卷创建一个配置文件：

```
# snapper -c root create-config /
```

注意：如果你正在使用 archinstall 推荐的 btrfs 分区布局，则 @.snapshots 子卷已经被挂载到 /.snapshots，这会导致 snapper create-config 无法正常创建配置 。要让 Snapper 使用 @.snapshots 来创建备份，需要进行如下操作：

- 卸载 @.snapshots 子卷并删除已存在的挂载点。
- 创建 Snapper 配置。
- 删除 Snapper 创建的子卷。
- 重新创建 /.snapshots 挂载点，重新挂载 @.snapshots 子卷。

此时，配置文件已经激活。如果你的 cron 守护进程已经运行， snapper 将会使用 #自动按时创建快照。否则，你需要使用 systemd 单元文件和定时器。参阅 #启用/停用。

参阅 snapper-configs(5)。

创建快照

自动按时创建快照

一个快照时间线(timeline)由可配置数目的每小时/日/月/年快照组成。当自动按时创建启用时，默认每小时创建一个快照。每天由时间线清理算法清理多余快照。详情请参照 snapper-configs(5) 中的 TIMELINE_* 变量。

启用/停用

如果你拥有一个 cron 守护进程，该特性应该已经自动启用。要停用，编辑你想禁用该特性的子卷对应配置文件为：

```
TIMELINE_CREATE="no"
```

如果你没有 cron 守护进程，你可以使用提供的 systemd 单元文件。启动并启用 snapper-timeline.timer 来启用自动按时创建快照。另外，启用并启动 snapper-cleanup.timer 来定期清理老旧快照。

注意：如果你有一个 cron 守护进程并同时启用了 systemd 单元，这可能会导致重复创建快照。如果你希望使用 systemd 单元并禁用 cron 集成，一中可行的方法是使用 pacman 的 NoExtract 配置选项。参见。

设置快照限制

默认配置将保留 10 个每小时快照，10 个每日快照，10 个每月快照和 10 个每年快照。你可以在配置文件中更改这些限制，特别是在繁忙的子卷，例如 / 上。参阅 #避免影响性能。

这是一份名为 `*config*` 的配置文件的示例片段，它将保留 5 个每小时快照，7 个每日快照，不保留每月和每年快照：

```
/etc/snapper/configs/config
TIMELINE_MIN_AGE="1800"
TIMELINE_LIMIT_HOURLY="5"
TIMELINE_LIMIT_DAILY="7"
TIMELINE_LIMIT_WEEKLY="0"
TIMELINE_LIMIT_MONTHLY="0"
TIMELINE_LIMIT_YEARLY="0"
```

更改创建和清理频率

如果你使用提供的 systemd 定时器，你可以修改它们来更改创建和清理频率。

例如，编辑 `snapper-timeline.timer`，加入下列配置来设定快照频率为间隔五分钟，而不是一小时：

```
[Timer]
OnCalendar=
OnCalendar=*:0/5
```

在编辑 `snapper-cleanup.timer` 来每小时运行清理，而不是每天的时候, 你需要更改 `OnUnitActiveSec`。 加入:

```
[Timer]
OnUnitActiveSec=1h
```

参阅 Systemd/定时器和 Systemd#附加配置片段。

手动创建快照

Single 快照

默认情况下 snapper 创建 *single* 类型的快照，它们与其他快照没有特别关系。

要为一个子卷创建快照：

```
# snapper -c config create --description desc
```

以上命令没有对应的清理算法，因此该快照将会一直存储直到删除它。

要设置一个清理算法，在 `create` 后使用 `-c` 选项，并在 `number`, `timeline`, `pre` 或 `post` 中选择一个参数。 `number` 使 snapper 定期清理超出配置文件中设置的数量限制的快照。例如，要创建一个使用 `number` 清理算法的快照：

```
# snapper -c config create -c number
```

参阅 #自动按时创建快照查看 `timeline` 是如何工作的。参阅 #pre/post 快照查看 `pre` `post` 如何工作。

pre/post 快照

另一种快照类型是 *pre/post* 快照。两个快照会在进行重大更改时先后成对创建（例如进行系统更新时）。

如果这种重大更改是/可被单个命令调用，那么 `snapper create --command` 可以在调用这个命令的同时创建 pre/post 快照。

```
# snapper -c config create --command cmd
```

提示：要在创建 *pre/post* 快照时执行任何 shell 命令，可以考虑使用 snpAUR 脚本。这个脚本相比 Snapper 自身的 `--command` 选项能更好的处理输出重定向。

*pre/post*快照也可以被手动创建。

首先创建一个*pre*快照：

```
# snapper -c config create -t pre -p
```

记下被创建快照的序号（在创建*post*快照时需要用到）。

然后进行想要进行的操作（比如安装一个程序，或是进行升级等）。

最后创建*post*快照，将下文的 `*N*` 替换为*pre*快照的序号：

```
# snapper -c config create -t post --pre-number N
```

参阅 #在进行 pacman 操作时创建快照。

启动时快照

要让 snapper 为 `root` 配置文件创建一个快照，启用 `snapper-boot.timer`。通过这种方式创建的快照是 *Single* 快照。

管理快照

列出配置

列出所有你已经创建的配置:

```
 # snapper list-configs
```

列出快照

要列出名为 *config* 的配置文件对应的快照:

```
 # snapper -c config list
```

还原快照

在还原快照时，部分文件可能会保持原样，原因可能是快照中不包含该文件（例如，该文件位于另一个子卷上），也可能是过滤配置排除了该文件。

过滤配置

**本文或本章节的事实准确性存在争议。**

原因： `/etc/mtab` is a symlink to `/proc/self/mounts`, so reverting it has no effect on the system.（在 Talk:Snapper 中讨论）

部分文件保存着系统的状态信息，例如 `/etc/mtab`。这些文件不应该被还原。Arch Linux 提供的 Snapper 的默认配置已经排除了这些文件。Snapper 支持按照用户排除某些文件或文件夹。`/etc/snapper/filters/*.txt` 和 `/usr/share/snapper/filters/*.txt` 中的每行指定了要排除的项目。当 Snapper 计算快照之间的差别时，会排除这些文件。注意，这些文件仍会包含在快照中，如果不想让这些文件包含在快照中，使用单独的子卷或者挂载点。

**本文或本章节的事实准确性存在争议。**

原因： How is the list from SLES documentation relevant for Arch Linux?（在 Talk:Snapper 中讨论）

参见 SUSE 的文档Directories That Are Excluded from Snapshots\[失效链接 2024-03-03 ⓘ\]。

还原快照

**本文或本章节的事实准确性存在争议。**

原因： What is the "default layout"? What is the alternative?（在 Talk:Snapper 中讨论）

如果你使用 Snapper 的默认布局，每个快照是在所指定子卷下的 `.snapshots` 目录的下层子卷，例如 `@home` 的快照位于 `@home/.snapshots`下。

**本文或本章节的事实准确性存在争议。**

原因： Subvolumes that are not used for `/` can be restored from the system itself. Just log in as root, make sure that the subvolume is not used, and unmount it.（在 Talk:Snapper 中讨论）

例如要将某个快照恢复到 `home`，首先启动到 Arch Linux 的 LiveCD。 使用UUID将 btrfs 的顶层子卷挂载到`/mnt`。

```
# mount -t btrfs -o subvol=/ /dev/disk/by-uuid/UUID_of_root_volume /mnt
# cd /mnt
```

将目前的子卷移动到其他地方，例如将 `@home` 移动到`@home-backup`：

```
# mv @home @home-backup
```

找到你想恢复快照的序号（每个快照都单独显示为一行，所以能很方便的按照时间查找序号）：

```
# grep -r '<date>'  /mnt/@home-backup/.snapshots/*/info.xml
...
/mnt/@home-backup/.snapshots/number/info.xml:  <date>2021-07-26 22:00:00</date>
...
```

注意：在`info.xml`记录的时间为 UTC 时间，请务必将其和本地时间的时差计算在内。

记住 `*number*`。

使用 *number* 快照来创建新快照。

```
# btrfs subvolume snapshot @home-backup/.snapshots/number/snapshot @home
```

将 .snapshots 移回恢复完成的子卷下，例如@home。

```
# mv @home-backup/.snapshots @home/
```

注意：如果在 fstab中指定的是 subvolid 而不是子卷路径，则需要更改 /mnt/@/etc/fstab 中的 subvolid（假定把@子卷挂载到了/）。 可以使用 btrfs subvolume list /mnt | grep @home$ 来查询新的 subvolid。

重新启动计算机。

检查计算机运行是否正常。这时可以删除在还原之前创建的快照了——例如@home-backup。在删除前记得检查是否有需要保留的数据。

删除快照

要删除序号为 *N* 的快照:

```
 # snapper -c config delete N
```

可以一次删除多个快照。例如，要删除 root 配置文件的 65 和 70 号快照：

```
 # snapper -c root delete 65 70
```

要同时删除多个连续的快照，例如删除 65 到 70 号所有快照：

```
# snapper -c root delete 65-70
```

要立即释放快照占用的磁盘空间，使用 --sync 选项：

```
# snapper -c root delete --sync 65
```

要删除一个 Snapper 配置和其所有快照，包括 .snapshots 子卷：

```
# snapper -c config delete-config
```

注意：删除 pre 快照时, 你应该总是删除对应的 post 快照。反之亦然。

允许非 root 用户访问

所有配置文件都是由 root 用户创建的，默认情况下，也只有 root 用户可以查看并访问它们。

要让指定用户可以列出某一配置文件的快照，修改 /etc/snapper/configs/*config* 中的 ALLOW_USERS 项。现在你应该可以以普通用户权限运行 snapper -c *config*list

最后，如果你想允许某一用户浏览 .snapshots 目录，但是该目录的拥有者必须为 root。因此，你应该修改包括该用户的组为组拥有者，以 users 为例：

```
# chmod a+rx .snapshots
# chown :users .snapshots
```

提示与技巧

在进行 pacman 操作时创建快照

下列软件包可以依据 pacman 进行的事务自动创建快照。

- **snap-pac** — 使用 pacman 钩子使 pacman 自动使用 Snapper 创建#pre/post 快照。类似与 openSUSE 的 YaST 的行为。

: https://github.com/wesbarnett/snap-pac || snap-pac包

- **grub-btrfs** — 包含了一个名为 *grub-btrfsd* 的 systemd 单元。可自动查找快照并将其添加到 GRUB 菜单中。

: https://github.com/Antynea/grub-btrfs || grub-btrfs包

- **snap-pac-grub** — 在 snap-pac包 创建快照后自动使用 grub-btrfs包 更新 GRUB 启动项。同样使用pacman 钩子。

: https://github.com/maximbaz/snap-pac-grub || snap-pac-grubAUR

- **refind-btrfs** — 在 snap-pac包 创建快照后自动在 rEFInd 中创建启动项。

: https://github.com/Venom1991/refind-btrfs || refind-btrfsAUR

- **snp** — 包装任何 shell 命令以供创建 pre/post 快照（例如`snp pacman -Syu`）相比 Snapper 自身的 `--command` 选项能更好的处理输出重定向（参见 #pre/post 快照）。

: https://gist.github.com/erikw/5229436 || snpAUR

- **limine-snapper-sync** — 在 snap-pac包 创建快照后自动在 Limine 中创建启动项。（参见Limine#Snapper snapshot integration for Btrfs）

: https://gitlab.com/Zesko/limine-snapper-sync || limine-snapper-syncAUR

启动到只读快照中

使用 grub-btrfs包、snap-pac-grubAUR或limine-snapper-syncAUR的用户应该注意：在默认状态下，Snapper 创建的快照是只读的。而在启动到只读快照时会有一些困难。许多服务，例如显示管理器，都需要一个可写的 `/var` 目录。在启动到只读快照时，这些服务将无法正常工作。

为了避免这个问题，可以使快照设为可写，或者使用 overlayfs 启动到快照中，这也是开发者认可的方法。这种情况下快照像是一个 Live CD 环境。

注意：任何对快照中文件的更改都不会被保存，因为文件系统只是存在于内存上。

要使用 overlayfs 启动快照：

- 确保 grub-btrfs包 已经安装。
- 添加 `grub-btrfs-overlayfs` 到 `/etc/mkinitcpio.conf` `HOOKS` 的末尾。例如：

```
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block filesystems fsck grub-btrfs-overlayfs)
```

注意：因为 *grub-btrfs-overlayfs*只提供了 运行时钩子，没有提供 systemd 单元，所以并不支持基于 systemd 的 initramfs。确保你使用的是 基于 Busybox 的 initramfs。参见GitHub issue。

- 重新生成 initramfs。

进一步了解：

- grub-btrfs README （包含了使用 dracut包 而不是 mkinitcpio包 的配置方法）
- Github 上的讨论

提示：你可以使用 Limine 启动到快照中。Limine 和 Snaper 间的集成由 limine-snapper-syncAUR 提供。参见Limine#Snapper snapshot integration for Btrfs。

在进行 pacman 事务时备份非 btrfs /boot 分区

如果你的 `/boot` 分区是在一个非 btrfs 文件系统上（例如一个 ESP 分区），Snapper 无法备份这些分区。 在系统备份#快照与_/boot_分区中提供了一个自动在内核更新时将 `/boot`复制到你的 btrfs 根目录下的方法。这种方法也能和 snap-pac包 一起使用。

增量备份到外部驱动器

一些工具可以使用 Snapper 进行自动备份。参见 Btrfs#增量备份到外置设备。

建议的文件系统布局

注意：下列布局不是为了与 `snapper rollback` 搭配使用。而是为了缓解#Restoring / to its previous snapshot 中的某些问题。参见 论坛帖子。

这是一种建议的文件系统布局，可以方便的将挂载到 `/` 的 `@` 子卷还原到之前的状态：

<table class="wikitable"><caption>文件系统布局</caption><tbody><tr><th>子卷</th><th>挂载点</th></tr><tr><td>@</td><td>/</td></tr><tr><td>@home</td><td>/home</td></tr><tr><td>@snapshots</td><td>/.snapshots</td></tr><tr><td>@var_log</td><td>/var/log</td></tr></tbody></table>

```
subvolid=5
  |
  ├── @ -|
  |      contained directories:
  |        ├── /usr
  |        ├── /bin
  |        ├── /.snapshots
  |        ├── ...
```

```
/
├── @home
├── @snapshots
├── @var_log
└── @...
```

子卷`@...`应当挂载到任何应有自己子卷的目录。

注意：

- 当对 `@` 子卷（挂载到`/`）创建快照时，其余子卷的将不会被包含在快照中。即使这些子卷位于 `@` 之下（子卷不是递归包含的）。如果需要为其他子卷创建快照，需要单独为这些子卷创建 Snapper 配置。
- 由于Btrfs 的限制，需要创建快照的子卷中不能包含swap 文件。将 swap 文件放到另外的子卷或者使用 swap 分区。

如果你想将`@`子卷恢复到之前的状态，其他的子卷将不会被影响。例如，当你在将`@`恢复到之前的快照时，`/home`将不会受到影响，因为`/home`挂载到了单独的子卷下。

这个布局能使 Snapper 定期为 `/`创建快照，并能让在出现启动问题时更容易还原快照。

在这种情况下，进行初始设置之后，Snapper不需要更多配置。

提示：

- 考虑为其他不想包含在`@`快照中的目录创建子卷。例如 `/var/cache`, `/var/spool`, `/var/tmp`, `/var/lib/machines` (systemd-nspawn), `/var/lib/docker` (Docker), `/var/lib/postgres` (PostgreSQL)，或是其他 `/var/lib/` 下的目录。你可以使用类似上面的平行子卷结构，或是创建下层子卷。但是，pacman 数据库 `/var/lib/pacman` 必须位于挂载到`/`的子卷下。
- 你可以为`@home`或是其他子卷单独创建快照或者还原快照。

配置 Snapper 和挂载点

假定`@`挂载到了`/`下，同时 `/.snapshots` 未被挂载，同时也不存在同名目录。可以使用如下命令确认：

```
# umount /.snapshots
# rm -r /.snapshots
```

接下来为/#建立一个新的配置。Snapper 会自动在`@`下创建 `.snapshots` 子卷。这在建议的布局下是不必要的，可以删除。

```
# btrfs subvolume delete /.snapshots
```

删除这个子卷之后，重新创建 `/.snapshots` 目录。

```
# mkdir /.snapshots
```

现在将 `@snapshots` 子卷挂载到 `/.snapshots` 目录。例如，当文件系统位于 `/dev/sda1` 时：

```
# mount -o subvol=@snapshots /dev/sda1 /.snapshots
```

要使此配置永久生效，在 fstab 中添加条目。

若 fstab 中已存在这个条目，重新挂载：

```
# mount -a
```

将此文件夹权限设为 `750` 。

这将使 Snapper 创建的快照储存在 `@` 之外。所以 `@` 被还原或替换时也不会丢失快照。

还原到之前的快照

要将 `/` 还原到某个快照的状态，首先启动到 Arch Linux Live CD。

挂载顶层子卷（subvolid=5）。也就是说，不使用任何 `subvolid` 或 `subvol` 挂载参数。

找到想还原快照的序号：

```
# grep -r '<date>' /mnt/@snapshots/*/info.xml
```

输出类似这样。每个快照都单独显示为一行，所以能很方便的按照时间查找序号。

```
/mnt/@snapshots/number/info.xml:  <date>2021-07-26 22:00:00</date>
```

注意：在`info.xml`记录的时间为 UTC 时间，请务必将其和本地时间的时差计算在内。

记住`*number*`。

将`@`移动到其他位置，例如 `/@.broken`来备份当前的系统状态。或是直接删除`@`子卷：`btrfs subvolume delete /mnt/@`。

为 Snapper 创建的快照创建一个可读写快照：

```
# btrfs subvolume snapshot /mnt/@snapshots/number/snapshot /mnt/@
```

将 `*number*` 替换为你想还原的快照序号。

注意：如果在 fstab中指定的是 `subvolid` 而不是子卷路径，则需要更改 `/mnt/@/etc/fstab` 中的 `subvolid`（假定把@子卷挂载到了`/`）。 可以使用 `btrfs subvolume list /mnt | grep @$` 来查询新的 `subvolid`。另外若在启动加载器中使用了 `subvolid`，也需要一并更改。例如 `refind_linux.conf`。

最后，卸载顶层子卷（subvolid=5），挂载`@`到`/mnt`以及 ESP分区。chroot到完成还原的根目录下，并重新生成 initramfs。

现在`/`已经恢复到了之前快照的状态。重新启动计算机。

提示：你也可以使用为这种布局设计的自动回滚工具：snapper-rollbackAUR。编辑 `/etc/snapper-rollback.conf` 来匹配系统的实际布局情况。

还原其他子卷到之前的状态

参见#还原快照。

从快照中删除文件

若你想删除指定的文件或文件夹而不想删除整个快照，可以使用 snappersAUR 实现。这个脚本还可用于以 Snapper 目前不支持的其他方式操作过去的快照。

若你不想使用其他脚本，你可以使快照变为可读写：

```
# btrfs property set /path/to/.snapshots/<snapshot_num>/snapshot ro false
```

验证 `ro=false`：

```
# btrfs property get /path/to/.snapshots/<snapshot_num>/snapshot
ro=false
```

现在可以像平常一样修改 `/path/to/.snapshots/<snapshot_num>/snapshot` 了。你也可以使用 shell 循环批量处理快照。

避免影响性能

在/这种繁忙的文件系统中，长时间保留大量快照会占用大量硬盘空间，可能影响文件系统的性能。你可以通过以下方式降低影响：

- 为不值得被快照的目录创建单独的子卷，例如`/var/cache/pacman/pkg`，`/var/abs`，`/var/tmp`，`/srv`。
- 编辑默认创建快照的频率，参见#自动按时创建快照。

updatedb

默认情况下，`updatedb`（参见 locate）也会索引由 snapper 创建的 `.snapshots` 目录，如果快照较多，这可能会导致运行速度严重的性能影响和大量内存占用。你可以编辑`updatedb`，阻止`.snapshots`索引该目录：

```
/etc/updatedb.conf
PRUNENAMES = ".snapshots"
```

禁用配额

Btrfs 的配额功能（不是后来引入的「简单配额」（simple quota））可能会导致删除快照时占用大量 CPU 来维护配额信息。

要确定配额是否已启用，使用如下命令：

```
# btrfs qgroup show /
```

可以使用如下命令禁用配额：

```
# btrfs quota disable /
```

计算快照数量

如果禁用配额没有减缓性能问题，那么计算快照数量可能会有用：

```
# btrfs subvolume list -s / | wc -l
```

为用户数据和日志创建子卷

如果目录中包含用户数据（如电子邮件或日志），建议将其存储在自己的子卷而不是根子卷 `/` 中。这样，如果恢复 `/` 的快照，用户数据和日志不会受到影响。可以为用户数据使用单独的 timeline。不建议为 `/var/log` 创建日志快照。这可能会导致难以排除故障。

还可以使用#过滤配置在还原过程中跳过某些目录。有关跳过某些目录的示例和原因，请参阅 SLES 文档。

依据磁盘使用量清理快照

疑难解答

日志

Snapper 将所有活动写入到 `/var/log/snapper.log` 中——在你认为出错时，首先检查该文件。

当你遇到关于每小时/每日/每周快照的问题时，最常见的原因是由于 cronie 服务（或者你使用的其他 cron 守护进程）没有运行。

IO 错误

如果你在试图创建快照时遇到 'IO Error'，请确认你试图创建快照的子卷对应的 .snapshots 目录是一个子卷。

另一个可能的原因是 .snapshots 目录的拥有者不是 root。你会在在 `/var/log/snapper.log` 中找到 `Btrfs.cc(openInfosDir):219 - .snapshots must have owner root`。

未被管理的快照导致空间浪费

快照有可能“丢失”，它们仍然存在于磁盘上，但不会被 snapper 管理。这会造成大量磁盘空间浪费。要检查这种情况，请比较：

```
# snapper -c <config> list
```

与

```
# btrfs subvolume list -o <parent subvolume>/.snapshots
```

第二个命令列出的子卷但未出现在第一个命令的输出中的都是未被 snapper 管理的快照。可以手动删除。

相关资源

- Snapper homepage

- openSUSE Snapper portal
- Btrfs homepage
- Linux.com: Snapper: SUSE's Ultimate Btrfs Snapshot Manager

1.19 文件系统

难度 高级 | **篇幅** 35.6 KB | **所属** 文件系统

本节将讲到

- 文件系统类型
- 查看现有文件系统
- 创建文件系统
- 挂载文件系统
- 参阅

根据维基百科：

: 在计算机体系中，文件系统负责控制如何存放和读取数据。如果没有文件系统，存储介质中的数据就会完全混在一起，无法进行区分。通过把数据分块和命名，不同的信息就可以被隔离和区分。将各组数据命名为“文件”是取自纸质信息系统的命名方式，而“文件系统”是指用于管理信息的分组和命名的结构和逻辑规则。

文件系统有很多种，每个硬盘分区可以且仅可使用其中的一种。每种文件系统有自己的优缺点和独有特性。以下内容是目前所支持的文件系统类型的概述，相应的链接会提供更加详细的信息。

文件系统类型

filesystems(5) 包含一个简单介绍，详细比较请参考 zhwp:文件系统的对比。内核已经加载和内置的文件系统可以通过 `/proc/filesystems` 查看。通过 `ls /lib/modules/$(uname -r)/kernel/fs` 可以查看所有已安装的内核模块。

<table class="wikitable sortable"><caption>内核树内和 FUSE 文件系统
</caption><tbody><tr><th>文件系统</th><th>创建命令</th><th>用户空间工具
</th><th>Archiso <a rel="nofollow"
class="external autonumber"
href="https://gitlab.archlinux.org/archlinux/archiso/-/blob/master/configs/releng/packages.x86_64"></th><th>内核文档 <a rel="nofollow" class="external autonumber"
href="https://docs.kernel.org/filesystems/index.html"></th><th>说明
</th></tr><tr><td><a href="/wiki/Bcachefs"
title="Bcachefs">Bcachefs</td><td><a rel="nofollow" class="external text"
href="https://man.archlinux.org/man/bcachefs.8">bcachefs(8)</td><td><

span class="nobreak">bcachefs-tools^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td></td><td>Linux 6.7+ 版本可用，实验性</td></tr><tr><td>Btrfs</td><td>mkfs.btrfs(8)</td><td>btrfs-progs^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>btrfs.html</td><td>可靠性</td></tr><tr><td>VFAT</td><td>mkfs.fat(8)</td><td>dosfstools^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>vfat.html</td><td>Windows 9x 的文件系统，常用于 U 盘和 SD 卡。</td></tr><tr><td rowspan="2">exFAT</td><td>mkfs.exfat(8)</td><td><a rel="nofollow" class="external text" href="https://archlinux.org/packages/?

name=exfatprogs">exfatprogs<sup><small>包
</small></sup></td><td data-sort-value="5" class="archwiki-table-cell-
background-green" style="vertical-align: middle; text-align: center;">是
</td><td></td><td>从 Linux 内核 5.4 <a rel="nofollow" class="external autonumber"
href="https://itsfoss.com/linux-kernel-5-4/"> 开始，内核原生支持该文件系统。常
用于 U 盘和 SD 卡。</td></tr><tr><td><span class="plainlinks archwiki-template-
man" title="$ man 8 mkexfatfs"><a rel="nofollow" class="external text"
href="https://man.archlinux.org/man/mkexfatfs.8">mkexfatfs(8)</td><td>
<a
rel="nofollow" class="external text" href="https://archlinux.org/packages/?
name=exfat-utils">exfat-utils<sup><small>包
</small></sup></td><td data-sort-value="1" class="archwiki-table-cell-
background-red" style="vertical-align: middle; text-align: center;">否
</td><td>N/A（基于 FUSE）</td><td></td></tr><tr><td><a href="/wiki/F2FS"
title="F2FS">F2FS</td><td><span class="plainlinks archwiki-template-man"
title="$ man 8 mkfs.f2fs"><a rel="nofollow" class="external text"
href="https://man.archlinux.org/man/mkfs.f2fs.8">mkfs.f2fs(8)</td><td><
span class="nobreak"><a
rel="nofollow" class="external text" href="https://archlinux.org/packages/?name=f2fs-
tools">f2fs-tools^{<small>包</small>}</td><td data-
sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align:
middle; text-align: center;">是</td><td><a rel="nofollow" class="external text"
href="https://docs.kernel.org/filesystems/f2fs.html">f2fs.html</td><td>基于闪存
的设备，无法缩容。</td></tr><tr><td><a href="/wzh/index.php?
title=Ext3&action=edit&redlink=1" class="new" title="Ext3（页面不存
在）">ext3</td><td><span class="plainlinks archwiki-template-man" title="$
man 8 mkfs.ext3"><a rel="nofollow" class="external text"
href="https://man.archlinux.org/man/mkfs.ext3.8">mkfs.ext3(8)</td><td>
<a
rel="nofollow" class="external text" href="https://archlinux.org/packages/?
name=e2fsprogs">e2fsprogs<sup><small>包
</small></sup></td><td data-sort-value="5" class="archwiki-table-cell-
background-green" style="vertical-align: middle; text-align: center;">是</td><td><a
rel="nofollow" class="external text"
href="https://docs.kernel.org/filesystems/ext3.html">ext3.html</td><td></td></tr
><tr><td>ext4</td><td><span class="plainlinks
archwiki-template-man" title="$ man 8 mkfs.ext4"><a rel="nofollow" class="external
text"
href="https://man.archlinux.org/man/mkfs.ext4.8">mkfs.ext4(8)</td><td>

e2fsprogs^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>ext4.html</td><td></td></tr><tr><td>HFS</td><td>mkfs.hfsplus(8)</td><td>hfsprogs^{<small>AUR</small>}</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td>hfs.html</td><td>Classic Mac OS 文件系统</td></tr><tr><td>HFS+</td><td>mkfs.hfsplus(8)</td><td>hfsprogs^{<small>AUR</small>}</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td>hfsplus.html</td><td>macOS（8-10.12）文件系统</td></tr><tr><td>JFS</td><td>mkfs.jfs(8)</td><td><a rel="nofollow" class="external text" href="https://archlinux.org/packages/?

name=jfsutils">jfsutils^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>jfs.html</td><td></td></tr><tr><td>NILFS2</td><td>mkfs.nilfs2(8)</td><td>nilfs-utils^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>nilfs2.html</td><td>设计用在基于闪存的设备上，不支持 xattrs 和 ACL。</td></tr><tr><td rowspan="2">NTFS</td><td></td><td></td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>ntfs3.html</td><td>Windows NT 文件系统。是从 Linux 5.15 起可用的新驱动。</td></tr><tr><td>mkfs.ntfs(8)</td><td>ntfs-3g^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>N/A（基于 FUSE）</td><td>有较多功能的 FUSE 驱动。</td></tr><tr><td>ReiserFS</td><td><a rel="nofollow" class="external text"

href="https://manpages.debian.org/unstable/reiserfsprogs/mkfs.reiserfs.8.en.html">mkfs.reiserfs(8)</td><td>reiserfsprogs^{<small>AUR</small>}</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td><td></td><td>已在Linux 5.18 版本内核中弃用，并已在 Linux 6.13 版本中移除。</td></tr><tr><td>UDF</td><td>mkfs.udf(8)</td><td>udftools^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td>udf.html</td><td>用于光盘映像，DVD 和蓝光盘的 ISO/IEC 13346 文件系统。</td></tr><tr><td>XFS</td><td>mkfs.xfs(8)</td><td>xfsprogs^{<small>包</small>}</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td><td> xfs.html
xfs-delayed-logging-design.html
xfs-self-describing-metadata.html </td><td><a rel="nofollow" class="external text"

href="https://web.archive.org/web/20220902080136/https://xfs.org/index.php/XFS_FAQ#Q:_Is_there_a_way_to_make_a_XFS_filesystem_larger_or_smaller.3F">无法缩容</td></tr></tbody></table>

<table class="wikitable sortable"><caption>代码在单独仓库的文件系统</caption><tbody><tr><th>文件系统</th><th>创建命令</th><th>内核补丁</th><th>用户空间工具</th><th>说明</th></tr><tr><td>APFS</td><td>mkapfs(8)</td><td>linux-apfs-rw-dkms^{<small>包</small>}</td><td>apfsprogs^{<small>包</small>}</td><td>macOS（10.13 及更新版本）的文件系统。只读，并带有实验性的写入支持。另外也请参考 FUSE 版的 apfs-fuse-git^{<small>AUR</small>}。</td></tr><tr><td>Reiser4</td><td>mkfs.reiser4(8)</td><td></td><td>reiser4progs^{<small>AUR</small>}</td><td></td></tr><tr><td>ZFS</td><td></td><td>zfs-linux^{<small>AUR</small>}, zfs-dkms^{<small>AUR</small>}</td><td><a rel="nofollow"

class="external text" href="https://aur.archlinux.org/packages/zfs-utils/">zfs-utils^{<small>AUR</small>}</td><td>OpenZFS 移植</td></tr></tbody></table>

文件系统日志

ext3/4，HFS+，JFS，NTFS，ReiserFS 和 XFS 文件系统都使用文件系统日志。文件系统日志通过在数据更改写入储存设备前，把变更写入日志来提供故障恢复能力。当出现系统崩溃或电源故障的时候，这些文件系统能够更快的恢复到可用状态，并且在恢复过程中更不容易出现错误。日志活动在文件系统中专用的一部分空间里进行。

ext3/4 还可以记录数据日志（data-mode journaling），可以除元数据外额外记录数据本身。由于需要进行两次写入，该模式的速度损耗较大：首先需要写入日志，然后再将数据写入硬盘。因此，该模式默认不启用。在选择文件系统和功能时，必须考虑系统速度和数据安全性之间的平衡。

与之类似，Reiser4 提供了可配置的“事务模型”（transaction models）：一种被称为 wandering logs，使得无需两次写入硬盘；另一种被称为 **write-anywhere**，纯粹是是写时复制（copy-on-write）的实现；还有一种被称为 **hybrid**，可以智能地在前两种间进行切换。

类似 Reiser4，Btrfs，Bcachefs 和 ZFS 等基于写时复制（copy-on-write，又被称作 write-anywhere）的文件系统，其运行方式在设计上是完全原子化的，并为元数据和内联数据都提供了校验和（效果为操作只存在“完成”和“未发生”两种状态，在正常硬件上不会因操作卡在一半而导致数据损坏）。因此，这一类文件系统在设计上就相对不易发生数据丢失，且由于数据永远不会被就地更新，它们也不需要使用传统的日志来保护元数据。虽然 Btrfs 还在使用日志树，但这个树仅仅是为了加快 fdatasync/fsync 的速度。

FAT，exFAT，ext2 和 HFS 既不提供日志，也不具备原子性，因此只建议用于临时或是旧设备兼容上，不建议用于需要存储可靠性的情况。

基于 FUSE 的文件系统

参考 FUSE。

堆栈式文件系统

- **eCryptfs** — 企业级加密文件系统，这是一个 Linux 平台上的硬盘加密软件包。它被设计为与 POSIX 兼容的文件系统级加密层，目标是在操作系统级别提供类似 GnuPG 的功能。

: https://ecryptfs.org || ecryptfs-utils包

- **mergerfs** — 基于 FUSE 的联合文件系统。

: https://github.com/trapexit/mergerfs || mergerfsAUR

- **mhddfs** — 多机械硬盘 FUSE 文件系统，是基于 FUSE 的联合文件系统。

: http://mhddfs.uvw.ru || mhddfsAUR

- **overlayfs** — OverlayFS 是为 Linux 设计的文件系统服务，为其它文件系统实现了联合挂载的功能。

: https://docs.kernel.org/filesystems/overlayfs.html || linux包

- **unionfs-fuse** — Unionfs 的用户空间实现。

: https://github.com/rpodgorny/unionfs-fuse || unionfs-fuseAUR

只读文件系统

- **DwarFS** — DwarFS is a fast high compression read-only file system for Linux and Windows. DwarFS more or less supports the same features as SquashFS, but is able to compress better, and has configurable hash algorithm.

: https://github.com/mhx/dwarfs || dwarfsAUR

- **EROFS** — Enhanced Read-Only File System is a lightweight read-only file system, it aims to improve performance and compress storage capacity.

: https://erofs.docs.kernel.org/ || erofs-utils包

- **SquashFS** — SquashFS 是一个压缩的只读文件系统，它会压缩文件、inodes 和目录，而且最高支持 1 MiB 的块大小来实现更强力的压缩。

: https://github.com/plougher/squashfs-tools || squashfs-tools包

集群文件系统

- **BeeGFS** — A parallel file system, developed and optimized for high-performance computing.

: https://www.beegfs.io/c/ || beegfs-clientAUR

- **Ceph** — 统一的分布式存储系统，旨在实现出色的性能，可靠性和可扩展性。

: https://ceph.com/ || cephAUR

- **Glusterfs** — 支持扩展到几 PB 的集群文件系统。

: https://www.gluster.org/ || glusterfs包

- **IPFS** — 一种点对点超媒体协议，使网络更快，更安全，更开放。IPFS 旨在取代 HTTP 并为所有人构建更好的互联网。它使用块来存储文件的各个部分，每个网络节点仅存储它感兴趣的内容，并提供了仅受用户限制的去重、分布式的可扩展文件系统。（现为 alpha 阶段）

: https://ipfs.io/ || kubo包

- **MinIO** — MinIO offers high-performance, S3 compatible object storage.

: https://min.io || minio包

- **MooseFS** — MooseFS 是一个容错、高可用性和高性能的向外扩展的网络分布式文件系统。

: https://moosefs.com || moosefs包

- **OpenAFS** — AFS 分布式文件系统的开源实现

: https://www.openafs.org || openafsAUR

- **OrangeFS** — OrangeFS 是一个向外扩展的网络文件系统，为透明、并行地访问基于多个服务器的硬盘储存而设计。它为并行分布式应用提供了优化的 MPI-IO。它使 Linux 客户端、Windows、Hadoop 和 WebDAV 使用存储集群变得更为简便。它与 POSIX 兼容，而且是 Linux 内核（4.6+）的一部分。

: https://www.orangefs.org/ || 未被打包？在 AUR 里搜索

- **Sheepdog** — 用于卷和容器服务的分布式对象存储系统，能智能地管理硬盘和节点。

: https://sheepdog.github.io/sheepdog/ || sheepdogAUR

共享硬盘文件系统

- **GFS2** — GFS2 allows all members of a cluster to have direct concurrent access to the same shared block storage

: https://pagure.io/gfs2-utils || gfs2-utilsAUR

- **OCFS2** — The Oracle Cluster File System (version 2) is a shared disk file system developed by Oracle Corporation and released under the GNU General Public License

: https://oss.oracle.com/projects/ocfs2/ || ocfs2-toolsAUR

- **VMware VMFS** — VMware's VMFS (Virtual Machine File System) is used by the company's flagship server virtualization suite, vSphere.

: https://core.vmware.com/resource/vmware-vsphere-vmfs || vmfs-toolsAUR

查看现有文件系统

用 lsblk 查看现有的文件系统：

```
$ lsblk -f
NAME   FSTYPE LABEL     UUID                                 MOUNTPOINT
sdb
└─sdb1 vfat   Transcend 4A3C-A9E9
```

如果文件系统存在，那 FSTYPE 列会显示它的名字；如果它被挂载了，那么挂载点会显示在 MOUNTPOINT 那一列。

创建文件系统

文件系统可以被创建在一个分区上，逻辑容器（如 LVM，RAID，或是 dm-crypt）上，或是普通文件上（参考 Loop 设备）。下文将描述创建在分区上的情况。

注意：文件系统可以直接被写入到硬盘上，这样的设备叫做 *superfloppy* 或是无分区硬盘。这样做会带来一些约束，特别是用它来启动系统的时候。具体示例可参考 Btrfs#无分区 Btrfs 磁盘。

警告：

- 创建新文件系统之后，之前存放在该分区的数据会丢失且通常无法找回。**请备份所有要保留的数据**。
- 分区的用途通常会限制文件系统上的选择，比如 EFI 系统分区就只能用 FAT32，而包含 /boot 目录的文件系统要考虑引导加载程序是否支持。

在进行下一步前，需识别文件系统目标分区所在的设备，以及其是否已被挂载。例如：

```
$ lsblk -f
NAME   FSTYPE   LABEL       UUID                                 MOUNTPOINT
sda
├─sda1                      C4DA-2C4D
├─sda2 ext4                 5b1564b2-2e2c-452c-bcfa-d1f572ae99f2 /mnt
└─sda3                      56adc99b-a61e-46af-aab7-a6d07e504652
```

如果分区已被挂载，则在进行下一步前**必须**先将其卸载。在该示例中，现有文件系统 /dev/sda2 被挂载到了 /mnt，这时可以使用以下命令进行卸载：

```
# umount /dev/sda2
```

查看挂载了的所有文件系统，参考#列出挂载的文件系统。

要创建新的文件系统，需要使用 mkfs(8)。具体文件系统类型和对应需要安装的用户空间工具请参考#文件系统类型。

例如，如果要在 /dev/sda1 上创建新的 ext4 类型（常用于 Linux 数据分区）文件系统，可以执行：

```
# mkfs.ext4 /dev/sda1
```

提示：

- 使用 *mkfs.ext4* 的时候可以用 -L 选项来给文件系统设置一个 文件系统标签。*e2label* 可用来修改现有文件系统的标签。
- 文件系统在创建之后是有条件地 *改变大小*（resized）的。比如 XFS 可以扩容，但是不能缩小。参考 Resize capabilities。
- 在创建文件系统后或许可以*调整大小*，但会有一定限制。例如 XFS 可以扩容，但是不能缩小。具体请参考 Wikipedia:Comparison of file systems#Resize capabilities 和对应文件系统的文档。

创建完成后就可以将新文件系统挂载到目录了。

挂载文件系统

如果要手动挂载一个设备（例如分区）上的文件系统到某个目录，需使用 mount(8)。下面这个例子将把 /dev/sda1 挂载到 /mnt：

```
# mount /dev/sda1 /mnt
```

挂载之后，在 /mnt 下面就能看见 /dev/sda1 里的内容。在卸载设备前，/mnt 文件夹里挂载前的内容将会变得不可见。

fstab 描述了如果某个设备存在的话系统应该怎么自动挂载它，相关修改方法请参考 fstab。

如果 /etc/fstab 里描述了一个设备的挂载参数，而用 mount 时只给了设备名字或者挂载点，这时就会用 *fstab* 里的参数自动补全。比如，/etc/fstab 里指示 /dev/sda1 应该挂载到 /mnt，那下面的命令就会把 /dev/sda1 挂载到 /mnt，即使你没有显式指明：

```
# mount /dev/sda1
```

或者

```
# mount /mnt
```

mount 接受很多参数，这些参数依赖于文件系统是否支持。要改变这些参数的话，可以通过以下形式：

- mount ... ... **-o** *options*
- 修改 fstab
- 创建 udev 规则
- 自己编译内核
- 使用针对不同文件系统的 mount 脚本（/usr/bin/mount.*）

更多信息请参考相关文件系统的页面。

提示：

- 除了 *mount* 外，还可以使用 *systemd-mount* 进行挂载。如果没有指定挂载点，那么文件系统会被挂载到 `/run/media/system/*device_identifier*/`，这样就不用考虑具体要挂载到哪了。具体用法等信息请参考 systemd-mount(1)。
- 如果需要以普通用户身份挂载文件系统，请参考 udisks#使用。这将使得无需图形环境或调用 udisk 的文件管理器就能免根用户权限进行挂载。

列出挂载的文件系统

使用 findmnt(8)：

```
$ findmnt
```

findmnt 支持很多参数，可以利用它们来筛选输出或者获得更多信息。比如把某个挂载点或者设备作为参数，它就会只输出这个下面挂载了什么：

```
$ findmnt /dev/sda1
```

findmnt 会从 `/etc/fstab`、`/etc/mtab` 和 `/proc/self/mounts` 文件里收集信息。

卸载文件系统

可以使用 umount(8) 来卸载文件系统。参数可以是包含文件系统的设备（比如 `/dev/sda1`），也可以是挂载点（比如 `/mnt`）：

```
# umount /dev/sda1
```

也可以使用

```
# umount /mnt
```

"linux Structure needs cleaning"

卸载文件系统，然后对出现报错的卷执行 fsck。

参阅

- filesystems(5)
- systemd-mount(1)
- Linux 支持的文件系统
- zhwp:文件系统
- zhwp:mount (Unix))

1.20 Btrfs

难度 高级 | **篇幅** 77.6 KB | **所属** 文件系统

本节将讲到

- 准备

- 创建文件系统
- 配置文件系统
- 使用
- 已知问题

译自 Introduction — BTRFS Documentation：

: Btrfs 是为 Linux 开发的现代写时复制（CoW, Copy-on-Write）文件系统，旨在提供高级功能，同时也注重容错、修复及便于管理。

注意：与其它一些文件系统类似，Btrfs 仍在持续开发中，因此某些特定的功能可能尚未成熟到可以满足日常使用。要检查特定使用场景是否会受到影响，可查阅 Btrfs 官方文档的状态页面（英文）及本文的#已知问题部分。

准备

安装用户空间工具 btrfs-progs包，这是执行基本操作所必需的。

若需要从 Btrfs 文件系统启动（即内核和 initramfs 位于 Btrfs 文件系统），请检查引导加载程序是否支持 Btrfs。

创建文件系统

下文展示如何创建一个新的 Btrfs 文件系统。若要将一个 ext3/4 文件系统转换为 Btrfs，请参考 #从 ext3/4 转换。若要使用无分区的配置，请参考 #无分区 Btrfs 磁盘。

查阅 mkfs.btrfs(8) 以获取更多信息。

Btrfs 文件系统可建立在一个或多个设备上。此处“设备”是指 Linux 块设备，例如整个磁盘、分区、LVM 逻辑卷、回环设备、网络块设备等。

单设备文件系统

要在分区 `/dev/*分区名*` 上创建一个 Btrfs 文件系统，执行：

```
# mkfs.btrfs -L 自定义卷标 /dev/分区名
```

Btrfs 文件系统中，元数据的默认节点大小（nodesize）为 16 KiB 和系统页大小之中较大者，而数据的默认扇区大小等于系统页大小（自动检测）。如需为元数据指定更大的节点大小（必须是扇区大小的整数倍，最大允许 64 KiB），可通过 `-n` 选项来设置，如下例使用 32 KiB 的节点大小：

```
# mkfs.btrfs -L 自定义卷标 -n 32k /dev/分区名
```

注意：根据 mkfs.btrfs(8) § OPTIONS 手册页内容：“较小的节点大小会增加碎片化，但B树变得更高，从而能够减少锁竞争。较大的节点大小则能使数据更加紧凑并降低碎片化程度，但在更新元数据块时需要更多昂贵的内存操作。”

多设备文件系统

警告：

- Btrfs 的 RAID 5 和 RAID 6 模式存在致命缺陷，除非用来做丢失数据无所谓的测试，否则不应当用于任何场景。这里有已知问题和部分绕过方案的列表。 查阅 btrfs(5) § RAID56 STATUS AND RECOMMENDED PRACTICES 以获取最新动态。
- 默认情况下，systemd 会为 `/var/log/journal` 目录禁用写时复制（CoW），这在 RAID 1 上会导致数据损坏（参见 #禁用写时复制）。要避免这种情况，请创建一个空文件 `/etc/tmpfiles.d/journal-nocow.conf`，用来覆盖 `/usr/lib/tmpfiles.d/journal-nocow.conf`（参阅 tmpfiles.d(5) § CONFIGURATION DIRECTORIES AND PRECEDENCE）。

Btrfs 可以在多个设备上创建 RAID。Btrfs 支持的 RAID 级别包括 RAID 0、RAID 1、RAID 10、RAID 5 和 RAID 6（详情参阅下文 #profile）。Btrfs 的 RAID1 profile 仅保存两个副本，内核 5.5 版本引入了 RAID1c3 和 RAID1c4，分别对应带有3和4个副本的 RAID 1 级别。可以使用 `-d` 和 `-m` 参数分别为数据和元数据配置 RAID 级别。默认情况下，数据只有一份（`single`），元数据则镜像存储（`raid1`）。这相当于 JBOD 配置——多个磁盘会被看做成一个文件系统，但文件不冗余。更多有关如何创建一个 Btrfs RAID 卷的信息请参阅 Btrfs Wiki: 在多个设备上使用 Btrfs。

```
# mkfs.btrfs -d single -m raid1 /dev/分区1 /dev/分区2 ...
```

必须在 `/etc/mkinitcpio.conf` 中加入 `udev`、`systemd` 或 `btrfs` 钩子之一才能使用多设备 Btrfs 文件系统。查阅 mkinitcpio#常用钩子以获取更多信息。

创建好文件系统之后，建议使用下面的命令扫描并注册多设备 Btrfs 文件系统，这样在挂载多设备文件系统时就仅需指定其中的一个设备：

```
# btrfs device scan
```

注意：

- 可以之后再将设备添加到多设备文件系统中，详情请参见 Btrfs Wiki 上的这篇文章。
- 多个设备可以大小各异。但如果使用 RAID 配置存储数据，当一个磁盘的大小比其他的加起来都大时，多出的空间并不会被使用。要确定多设备 Btrfs 的可用空间，可以使用 Btrfs 磁盘使用计算器确定。
- 有些引导加载程序不支持多设备文件系统，比如 Syslinux。
- Btrfs 不会自动从速度最快的设备读取，因此混合使用不同类型的磁盘会导致性能表现不稳定。详情请参阅 Stack Overflow 上的这个回答。

#RAID 小节中有关于维护多设备上的 Btrfs 文件系统的建议。

profile

Btrfs 使用 profile 这个概念来表示镜像、奇偶校验和分条，对应 RAID 术语中的“RAID 级别”（若适用）。在同一个 Btrfs 文件系统中，元数据的 profile（mkfs.btrfs(8) 的 `-m` 选项）和数据的 profile（mkfs.btrfs(8) 的 `-d` 选项）可以不一样。

一些重要的 profile：

single：没有镜像、分条和奇偶校验，让多个设备映射到单个文件系统中。在 mdadm 术语中被叫做 `LINEAR`。 **RAID0**：没有镜像、分条和奇偶校验，但允许并行访问设备。不像传统的 mdadm RAID 那样要求设备大小相同。 **RAID1**：镜像，没有分条和奇偶校验，允许在一个设备失效的情况下恢复数据。

更多 profile 请参阅手册 mkfs.btrfs(8) § PROFILES。

配置文件系统

写时复制（CoW）

默认情况下 Btrfs 总是对所有文件使用写时复制（CoW）。写入操作不会原地覆盖数据，而是将修改后的数据块写入新位置，同时更新元数据指向新位置。请参阅 Btrfs Sysadmin 相关章节了解实现细节以及其优缺点。

禁用写时复制

警告：在 Btrfs 中禁用写时复制会禁用数据校验和，禁用数据校验和会使 Btrfs 无法得知这些文件是否损坏。与 RAID 1（包括 profile 中的 `RAID1`、`RAID1c3` 与 `RAID1c4`）结合使用时，断电或其他原因导致的数据损坏可能会造成副本间的数据不同步。

使用 `nodatacow` 选项挂载文件系统可禁用其上新文件的写时复制特性。注意这只会影响新建的文件，而不会影响已有的文件。`nodatacow` 参数会禁用压缩与数据校验和，参阅 btrfs(5) 以了解细节。

注意：

- 根据 btrfs(5) § MOUNT OPTIONS：“在单个文件系统中，无法让一些子卷使用 `nodatacow` 参数挂载，而其他的使用 `datacow` 参数挂载。第一个被挂载子卷的挂载参数将会应用于其他所有子卷。”
- 即使已禁用写时复制，在特定情况下写时复制仍然会被使用，详见下文#仍然会触发写时复制的情况。

要禁用空文件或目录中新文件（同上，不会影响其中已有文件）的写时复制特性，请使用下面的命令：

```
$ chattr +C 目录或文件路径
```

注意：译自 BTRFS documentation: Attributes：“在目录上设置此属性时，所有新建的文件将继承该属性。由于实现限制，此标志只能在空文件上设置或取消。”

提示：为目录中已存在的所有文件禁用写时复制可以使用下面的方法：

```
$ mv /path/to/dir /path/to/dir_old
$ mkdir /path/to/dir
$ chattr +C /path/to/dir
$ cp -a --reflink=never /path/to/dir_old/. /path/to/dir
$ rm -rf /path/to/dir_old
```

需要保证这个过程中未使用涉及到的数据。关于此处 `--reflink=never` 参数的使用，详见下文 #对复制的影响。

仍然会触发写时复制的情况

具有 +C 属性的文件仍然允许有超过一个引用（例如使用 `cp` reflink 的副本与 Btrfs 中的快照），此时写时复制**依然会生效**。

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：子章节嵌套过多。（在en:Talk:Btrfs讨论）

对复制的影响

使用 `cp` 复制文件时，+C 属性**并非**由源文件决定，而取决于目标路径——要么从上级目录继承，要么由文件系统是否使用 `nodatacow` 参数挂载决定。当源文件与目标文件都具有/不具有 +C 属性时，能够通过 reflink 进行复制，否则不能。实际行为依赖于 `--reflink` 参数，有以下三种结果：

- 使用 `--reflink=auto` 参数（默认情况）：若 +C 属性相同，则使用 reflink；否则进行深拷贝。
- 使用 `--reflink`（或 `--reflink=always`）参数：若 +C 属性相同，则使用 reflink；否则操作失败。
- 使用 `--reflink=never` 参数：总是进行深拷贝。

总的来说，如果确实要避免写时复制，必须显式地使用 `--reflink=never` 参数（若目标路径没有 +C 属性，先创建一个有 +C 属性的目录/空文件再覆盖它）。有关 `--reflink` 参数的更多信息请参阅 cp(1)。

对快照的影响

如果为禁用了写时复制的文件创建快照，快照会锁定当前文件的数据块，因此首次写入该文件的块时会触发写时复制。但文件仍保留 +C 属性，故后续对同一文件块的写入仍为原地操作，直至下次创建快照。

频繁快照会减弱 +C 属性的效果，因为在每次快照后的第一次写入都会触发写时复制。要避免这种情况，可以把所有禁用写时复制的文件放在一个单独的子卷中并避免为该子卷创建快照。

压缩

Btrfs 支持自动的透明压缩，这不仅能缩减文件体积，还能通过减少写入放大来显著延长闪存介质的使用寿命，详见 Fedora:Changes/BtrfsByDefault#Compression、 与 。在某些情况下（例如单线程、重文件 I/O 时），压缩可以提升性能，而在其他场景中（如多线程和/或大文件I/O的CPU密集型任务），性能则会明显受影响。采用快速的压缩算法（`zstd` 和 `lzo`）通常性能也就越好。一些基准测试提供了详细的对比数据。

压缩类型

Btrfs 支持的压缩算法有 `zlib`（即 zlib 库提供的 DEFLATE 压缩算法）、`lzo`（即 LZO）和 `zstd`（即 zstd 实现的 Zstandard 压缩算法）。其中 `lzo` 无压缩级别，而 `zlib` 和 `zstd` 可调整压缩级别（`zlib`：1～9；`zstd`：-15～-1、1～15，需为整数），级别越高压缩效果越好而所需时间越长。改变压缩级别对 CPU 和 I/O 吞吐的影响不一样，因此在变更前后最好进行检查和性能测试。关于压缩类型的更多信息，可参阅 btrfs(5) § COMPRESSION。

警告：如果使用 `zstd` 算法（尤其是 -15～-1 级别），使用较旧的内核或 btrfs-progs包 的系统可能不能读取或修复该文件系统。

启用压缩

为文件系统启用

挂载时指定 `compress=*算法[:级别]*` 挂载选项，对于文件系统中每次写入，Btrfs 将自动考虑是否启用压缩。其中的 `*算法*` 处可以填写 `zlib`、`lzo`、`zstd` 或 `no`（即不压缩），可选的 `*级别*` 处填写压缩级别，不填写或填写 0 时取默认值 3（`lzo` 不适用）。使用该选项时，Btrfs 会检测文件写入的数据的**首个数据块**经压缩后体积是否缩小。如果缩小，则压缩本次写入的全部内容，否则文件会被标记为 `NOCOMPRESS`，整个写入过程**及该文件后续的所有写入过程**都不会触发压缩 。这是为了防止等待所有待写入数据完全交付给 Btrfs 并进行尝试压缩后，磁盘才开始执行写入操作。

注意：

- 压缩挂载选项会在同一文件系统的所有挂载点（包括不同子卷与 bind 挂载）之间共享 。
- 若文件的一次写入无法压缩，此后写入均不会进行压缩。

或者，您可以在 Btrfs 子卷上设置压缩属性，并且无论它如何挂载，它都会持久化并应用，使用 `btrfs property set / compression *alg*`，其中 `*alg*` 可以是 `zstd` 或 `zstd:*n*`，其中 `*n*` 是压缩级别。

若改用 `compress-force=*算法[:级别]*` 挂载选项，Btrfs 会检查每次写入的**每个数据块**并分别决定是否压缩。在多个混合用途系统上进行的经验测试表明，使用 `compress-force=zstd` 相比使用 `compress=zstd` 能显著提高空间节省率（从 10% 提升到 20%），但这会导致 CPU 占用率无端（略微）升高且拖长写入所需时间。官方 Btrfs 指导手册不推荐使用该选项。

使用 `compress-force` 也会限制 extent 的最大大小，从而增加碎片化。

只有在添加以上两个挂载选项之一后新建或修改的文件才会被自动压缩。

提示：在新的 Btrfs 上安装 Arch Linux 时，如果需要将其压缩后存储，需要在系统安装前就使用 `compress` 选项挂载，例如：`mount -o compress=zstd /dev/sd*xY* /mnt/`，并注意在其 fstab 中为根目录文件系统指定相同的 `compress` 挂载选项。

为现有文件启用

要为现有文件启用压缩，可使用 `btrfs filesystem defragment -c*算法*` 命令，`*算法*` 处可选填为 `zlib`、`lzo` 或 `zstd`，使用 `-L` 选项指定压缩级别，否则使用默认值 3（`lzo` 不适用）。例如，执行以下命令可用 `zstd` 的 1 压缩级别重新压缩所有文件：

```
# btrfs -v filesystem defragment -r -czstd -L1 /
```

警告：defragment 操作会断开 #写时复制（CoW）数据（包括通过 `cp` 复制的文件、#快照和#去重数据等）之间的 reflink，产生独立的文件，可能显著增加空间使用量 。

使用上面的方式压缩文件不会被持久化，其他写入操作将应用原有压缩设置。

以下两种方式可以持久性启用对单个文件的压缩：

```
$ chattr +c 文件
$ btrfs property set 文件 compression zstd
```

前一条命令使用的是从 ext2 文件系统继承的文件属性旧接口，灵活性不足，只能够使用默认的 `zlib` 压缩算法。后一条命令则可以指定压缩算法，但指定压缩级别尚未实现，因此采用默认值 3（`lzo` 不适用）。

查看压缩类型和压缩率

compsize包 接收一个包含文件或目录（可为整个 Btrfs 文件系统）的列表，输出其使用的压缩类型以及实际压缩率（压缩后大小/压缩前大小）。压缩前大小可能与其他程序（如 du(1)）显示的数值不一致，因为每个 extent 都会被统计且只统计一次——即使有多个 reflink，或其部分内容已不再使用但尚未被垃圾回收。

`-x` 选项可限制在单一文件系统内执行，这在类似于 `compsize -x /` 的场景中非常有用，可以避免程序因尝试访问非 Btrfs 的子目录而导致整个运行的失败。

子卷

“Btrfs 子卷是可独立挂载的 POSIX 文件树，而**不是**也不能看作是**块设备**。大多数其他 POSIX 文件系统只有一个可挂载根，而 Btrfs 不仅整个卷（称作‘顶层子卷’）可以挂载，每个子卷也可独立挂载；一个 Btrfs 卷可以包含不止一个文件树——包含一个‘文件森林’。Btrfs 子卷可以理解为一个 POSIX 文件命名空间。”

每个 Btrfs 文件系统都有一个 ID 为 5 的顶层子卷。这个子卷不能被移除或被其他子卷替代。顶层子卷在文件系统中的路径为 `/`，其他子卷则嵌套在顶层子卷之下。子卷（不包括顶层子卷）可以在文件系统中移动，其路径可以发生变化，但子卷 ID 不可更改。

默认情况下，挂载文件系统时会挂载顶层子卷。可以通过选项来挂载指定子卷。

子卷的主要用途之一是#快照。

更多详细信息请参阅以下链接（英文）：

- Btrfs 文档
- Btrfs Wiki SysadminGuide#Subvolumes
- Btrfs Wiki Trees

创建子卷

要创建子卷，必须先挂载该 Btrfs 文件系统以使其要创建的路径可以访问（挂载根子卷或者上级子卷）。

```
# btrfs subvolume create 路径/子卷名
```

例如，在 `/mnt` 所在子卷下创建子卷 `@sub`（路径为 `/mnt/@sub`）：

```
# btrfs subvolume create /mnt/@sub
```

注意：

- 使用 `-p`（`--parents`）选项可以自动创建不存在的父目录。
- Btrfs 中子卷名可以是任何有效的目录名，以 `@` 符号开头只是timeshift包的命名约定。
- 如果顶层子卷没有挂载，而你想在其内部创建子卷，可以通过指定 `subvolid=5` 来临时挂载它。参见 #挂载子卷。

列出子卷

查看*路径*所属文件系统的所有子卷列表：

```
# btrfs subvolume list -t 路径
```

使用 `-t` 选项会显示更易读的表格视图。

删除子卷

删除子卷：

```
# btrfs subvolume delete 子卷路径
```

如果子卷包含其他子卷，需要添加 `-R`（`--recursive`）选项来递归删除包含的所有子卷。子卷也可以当作常规目录删除（使用 `rm -r`、`rmdir`；但这样会比较慢）。

挂载子卷

使用`subvol=*子卷路径（相对于顶层子卷）*`或`subvolid=*子卷ID*`选项来挂载子卷。通过在文件系统的顶层创建各种子卷，然后将其挂载到适当的挂载点，可以模拟传统的文件系统分区。

例如，以下将顶层子卷挂载到 `/mnt/` 并创建两个名为 `subvol_root` 和 `subvol_home` 的子卷。当顶层子卷被卸载时，新的子卷将被挂载到 `/mnt/` 和 `/mnt/home/`：

```
# mount device /mnt/ -o subvolid=5
# btrfs subvolume create /mnt/subvol_root/
# btrfs subvolume create /mnt/subvol_home/
# umount /mnt/
# mount -o subvol=/subvol_root device /mnt/
# mount -o subvol=/subvol_home --mkdir device /mnt/home/
```

使用 `subvol` 挂载选项引用子卷时，需要使用相对于「顶层子卷」的路径。

提示：

- 子卷 ID 可通过#列出子卷显示。
- 不将顶层子卷作为 `/` 挂载会使更改子卷布局的更加简单。因此，考虑创建一个子卷来作为 `/` 挂载（用以存放你的系统文件）。

注意：译自 btrfs(5) § MOUNT OPTIONS：“大多数挂载选项适用于**整个文件系统**，因而只有第一个挂载的子卷的选项才会生效。这是由于尚未实现分子卷的选项处理，将来情况可能会改变。这意味着（例如）你无法使用挂载选项为每个子卷单独设置 `nodatacow`、`nodatasum` 或 `compress`。这最终应该会被修复，但事实证明在 Linux VFS 框架内正确实现是困难的。”

有关目前及计划可以为每个子卷单独应用的挂载选项，请参阅 Btrfs Wiki FAQ。

提示：最好使用 `subvol=*/path/to/subvolume*` 来挂载，而不是使用 subvolid，因为在恢复 #快照 时，subvolid 可能会发生变化，从而需要更改挂载配置。

请参阅 Snapper#建议的文件系统布局、Btrfs SysadminGuide#Managing Snapshots 和 SysadminGuide#Layout，了解使用子卷的文件系统布局的例子。

有关 Btrfs 特有的挂载选项的完整列表，请参阅 btrfs(5) § BTRFS SPECIFIC MOUNT OPTIONS。

将子卷挂载为 /

如果要将子卷挂载为 `/`，要么#更改默认子卷，要么在 `rootflags` 内核参数中指定挂载子卷的选项（一些引导加载程序可以自动添加，例如 GRUB 的 `grub-mkconfig`

命令），并编辑 /etc/fstab。建议使用 subvol=*子卷路径* 而不是 subvolid 来挂载，因为后者可能在恢复#快照之后发生变化，如果不同时更改挂载选项，系统将不能启动。

如果你计划使用其中一种同步和备份程序，建议不要更改默认子卷，因为它会在恢复除默认子卷之外的其他子卷的快照时导致问题。例如，Timeshift会相对于文件系统的默认子卷创建一个路径timeshift-btrfs来存储快照，并且建议为根和家目录使用一个单独的非默认子卷。

更改默认子卷

如果没有提供 subvol 与 subvolid 选项，将挂载默认子卷。要将指定子卷设置为所在文件系统的默认子卷，执行：

```
# btrfs subvolume set-default 子卷路径
```

或者使用：

```
# btrfs subvolume set-default 子卷ID 路径
```

将*路径*所在文件系统的默认子卷设置为 ID 为*子卷ID* 的子卷。

提示：子卷ID可通过#列出子卷显示。

使用 btrfs subvolume set-default 更改默认子卷后，顶层子卷需要使用 subvol=/ 或 subvolid=5 挂载选项挂载 。

注意：

- GRUB 的 grub-mkconfig 脚本不遵守 Btrfs 的默认子卷设置——如果 / 是顶层子卷，始终不会指定 subvol=/ 或 subvolid=5 挂载选项，可能导致无法启动；如果 / 非顶层子卷，始终会指定相应的 subvol 选项，导致无法仅通过设置默认子卷回退（bug 报告）。目前可以通过手动修改 grub.cfg 中 rootflags 内核参数解决（参见 #挂载子卷），或者使用工具自动在 GRUB 中为快照创建启动项（参见 #引导进入快照）。另外，如果 grub.cfg 的实际位置发生改变，例如包含 /boot 的子卷改变，需要再次运行 grub-install。详见这个论坛帖子。
- 如果你打算使用某个同步和备份程序，不推荐更改默认子卷，以免在恢复非默认子卷的快照时出现问题。例如Timeshift会在默认子卷下创建路径timeshift-btrfs来储存快照，为/和/home使用单独的非默认子卷比较好。

配额

磁盘配额的概念在Unix世界有着悠久的传统。传统配额基于文件所有权，通过限制用户拥有的所有文件的总大小来控制空间使用。Btrfs采用了与传统Unix系统不同的配额设计理念，不支持传统的基于用户或用户组的配额功能，这是因为其现代存储特性（如#快照、#写时复制（CoW）、#去重和#压缩）使得传统配额的空间计算变得极其复杂且不准确 。作为替代，Btrfs 采用基于子卷的配额机制，现提供两种实现机制——

传统的#配额组和较新的#简单配额（在2023年底发布的6.7版本内核中引入）来实现类似的空间管理效果。虽然增加了管理复杂性，但提供了更灵活和精确的存储控制能力，且不用分区即可限制目录大小，支持动态调整配额。

注意：挂载Btrfs文件系统（或其中任一子卷）后才可以管理配额。

配额组

警告：

尽管Btrfs 配额组（quota group，简称 qgroup）在常规使用中被认为是稳定的，但启用配额组功能，尤其是在频繁创建大量快照的环境中，仍可能导致性能下降，特别是在删除快照时。配额组统计会引入额外的元数据开销，并可能在高强度快照工作负载下影响系统响应速度。

近期的内核改进（5.15及后续版本）已修复诸多历史问题，但某些限制依然存在，特别是在深度嵌套的子卷或复杂的配额组层级结构等高级应用场景中。2016年就写在这里的建议——启用配额组前需审慎评估具体使用场景，至今仍然适用。请务必查阅最新的 Btrfs 文档获取当前详细信息。值得注意的是，为了解决配额组的性能问题，Btrfs引入了#简单配额功能，提供了灵活性略差但性能更优的替代方案。

Btrfs精确的配额支持是通过子卷之上的配额组实现的。配额组使用配额组ID（qgroupid）表示，格式为*层级*/*ID*。子卷的层级是0，`0/` 可以省略，而子卷对应的 `*ID*` 就是子卷的 ID，例如 `0/5` 表示顶层子卷的配额组，也可以用子卷路径代替 `0/*ID*` 的表示方式。配额组形成“树”状层级结构，但一个配额组可以有多个父配额组 。叶子配额组层级为0，即子卷直接对应的配额组。所有更高层级配额组的 `*ID*` 可以自由指定。高层级配额组包含低层级配额组，相同层级之间不可嵌套，但层级不表示嵌套深度。例如，可以为 `5/100` 配额组分配子配额组 `0/5`。

总引用空间和独占引用空间

每个配额组主要追踪两个数值：**总引用空间**（total referenced space）和**独占引用空间**（exclusively referenced space）。其中**总引用空间**是指所有能够从配额组内访问到的数据占用的存储空间，**独占引用空间**是指只被配额组所属的子卷引用（而不被该配额组以外的子卷引用）的数据所占用的存储空间。

启用/禁用配额组

使用以下命令为*路径*所在文件系统启用配额组：

```
# btrfs quota enable 路径
```

将 `enable` 改为 `disable` 以禁用配额组。

警告：禁用配额组会销毁当前所有配额组并解除其关系。

创建/销毁配额组

子卷对应的0层级配额组始终会自动创建。即便子卷在启用配额前创建，启用配额时也会自动创建相应配额组。同样，#删除子卷时子卷对应的配额组也会被删除，并自动解除与原包含有这一配额组的高层级配额组的关系。

使用以下命令在*路径*所在文件系统创建配额组*层级*/*ID*：

```
# btrfs qgroup create 层级/ID 路径
```

将 `create` 改为 `destroy` 以销毁配额组。销毁配额组类似于使用 `rm -d` 删除目录——如果配额组存在子配额组，必须先解除关系；如果仅存在父配额组，关系会自动解除。

注意：子卷对应的配额组不能直接删除，而应该通过直接删除子卷来自动删除。

分配/删除子配额组

使用以下命令将*路径*所在文件系统的*子配额组*分配到*父配额组*，注意*父配额组*必须比*子配额组*层级更高：

```
# btrfs qgroup assign 子配额组 父配额组 路径
```

将 `assign` 改为 `remove` 以解除这一关系。

注意：分配/解除分配子配额组后可能需要 #配额重扫描，这在当前版本中会自动触发，但可能造成性能消耗，详见#配额重扫描章节。

限制/取消限制配额组

可以限制配额组的总引用空间或独占引用空间，或同时限制两者。例如，使用以下命令在*路径*所在文件系统中限制*配额组*的**总引用空间**不超过 1 GiB：

```
# btrfs qgroup limit 1G 配额组 路径
```

注意：如果已#启用压缩，Btrfs目前仅支持限制压缩后的数据占据的空间。

其中*配额组*若省略，则会尝试将*路径*解释为子卷路径。添加选项 `-e`（到 `limit` 之后）改为限制**独占引用空间**。将*大小限制*改为`none` 以取消限制。

列出配额组

使用以下命令列出*路径*所在文件系统的所有配额组：

```
# btrfs qgroup show 路径
```

注意：由于 `btrfs qgroup show` 使用的是磁盘中的数据，可能需要 `sync` 后才能显示准确的统计数据。为此可以添加 `--sync` 选项（到 `show` 之后），先对*路径*所在文件系统执行强制同步操作。

输出的“Path”列可能显示一些特殊值，含义如下：

- `<toplevel>`：顶层子卷对应的配额组。

- `<under deletion>`：该配额组对应子卷已被删除（其目录已移除），但子卷元数据尚未完全清理。

提示：可以使用 `btrfs subvolume sync *路径* *子卷ID*` 等待*路径*所在文件系统中子卷完全删除（可以指定多个子卷 ID；若一个也没有指定则等待当前文件系统中所有待删除子卷删除完毕）。但请注意先卸载该子卷。

- `<squota space holder>`：（仅适用于 #简单配额）该配额组对应子卷已完全删除但相关统计信息仍保留。这是由于简单配额只将extent归属于首次分配它的子卷，而该子卷已完全删除但extent仍被其他子卷引用。

根据需求，可以添加一些显示选项（到 `show` 之后）。常用的选项有：

- `-p`：显示父配额组。
- `-c`：显示子配额组。
- `-r`：显示总引用空间限制。
- `-e`：显示独占引用空间限制。
- `-f`：仅列出*路径*所在子卷对应的配额组。
- `-F`：仅列出*路径*所在子卷对应的配额组及包含这些配额组的高层级配额组。

更多选项请参阅 btrfs-qgroup(8) § show。

配额重扫描

配额重扫描（quota rescan）会读取文件系统中所有extent的元数据，据其更新各配额组的统计值。

Btrfs的配额组可以处理许多复杂的extent共享和不再共享场景（包括子卷、子配额组的删除），同时保持总引用空间与独占引用空间的准确计数。但当配额组关系发生变化（手动 #分配/解除分配子配额组）时，由于记录的数据不足（仅记录总引用空间与独占引用空间，而未明确记录哪些extent属于共享或独占），通常需要在整个文件系统内配额重扫描，此时配额组会被标记为“inconsistent”（不一致），即需要配额重扫描的状态。

目前实现中唯一的例外是配额组的**总引用空间**等于**独占引用空间**（即所有数据都为独占）时，仅需要在父配额组的总引用空间与独占引用空间上同时加上/减去该配额组的引用空间，不需要重新扫描配额。

自4.19版本内核起，手动#分配/解除分配子配额组时会自动触发配额重扫描（除了以上例外情况）。但考虑到其开销较大、同时只能进行一个操作、无法等待重扫描结束

后再返回，可以在 `assign/remove` 之后添加 `--no-rescan` 选项以避免自动触发重扫描，转而在之后再手动触发。

提示：使用 `--no-rescan` 选项#分配/解除分配子配额组后，若需要配额重扫描，会显示相应警告。也可以通过#列出配额组显示警告。

使用以下命令手动触发重新扫描*路径*所在文件系统的配额：

```
# btrfs quota rescan 路径
```

命令运行后立即返回，添加 `-w`（`--wait`）选项（到 `rescan` 之后）以等待重新扫描结束（即使已开始）再返回。使用 `-W`（`--wait-norescan`）选项仅等待当前正在运行的重新扫描结束。使用 `-s`（`--status`）选项返回当前进度。

简单配额

#配额组可以处理许多复杂的extent共享和不再共享场景（包括子卷、子配额组的删除），同时维护总引用空间与独占引用空间的准确计数，但这种灵活性是有代价的：许多计算操作具有全局性，即当extent引用关系变更后，会影响到引用该extent的所有配额组的统计值。这可能导致事务提交速度下降并引发不可接受的延迟，尤其在快照数量激增的情况下更为明显。

为解决配额组的这一局限，Btrfs还支持另一套配额规则体系：简单配额（simple quota，简称squota）。简单配额复用了配额组的API和层次结构模型，但不追踪共享与独占使用情况。相反，简单配额将所有extent归属于首次分配它的子卷。通过引入少量新的元数据记录，这样所有核算决策都能在涉及数据块本身的分配或释放操作的局部完成，完全避免了复杂且耗时的反向引用解析过程。

简单配额需要在文件系统级别启用，在启用配额组的命令中添加 `-s`（`--simple`）选项（到 `enable` 之后）即可。注意若配额组已经开启，需要先禁用配额组再启用简单配额，否则不会生效 。由于其简单的计算方法，简单配额模式下不需要（也不能）进行#配额重扫描。除了这两点外，简单配额的操作方式与配额组完全相同，但显示与限制使用的所谓“总引用空间”和“独占引用空间”都是通过上文所述方法（将所有extent归属于首次分配它的子卷）计算得到的值。由于简单配额需要记录首次分配extent的子卷，因此仅对启用后写入的extent生效，若在非空文件系统中启用简单配额，刚启用时统计值会是0，因此只有在空子卷启用简单配额限制后再写入数据，才能确保统计准确。

提交间隔

Btrfs 默认设置 30 秒的事务提交（commit）间隔，即每 30 秒将新数据从内存同步到永久存储。可通过 `commit` 挂载参数（单位为秒）更改，例如修改 fstab：

```
LABEL=arch64 / btrfs defaults,noatime,compress=lzo,commit=120 0 0
```

Btrfs 层面的周期性提交并非事务提交的唯一机制，该操作也可通过 `sync` 命令显式触发，或通过影响全局文件系统状态的其他命令间接触发，事务提交的机制还包括基于各类阈值或策略（如控制组）进行刷新的内核机制，这超出了本文的范围，因此不再赘述。

固态硬盘数据块异步丢弃

TRIM 或丢弃（discard）数据块是指文件系统通知固态硬盘哪些数据块不再使用的操作，从而固态硬盘能够在空闲时擦除（回收）这些数据块，避免等到需覆盖时再进行。内核 6.2 版本起默认启用异步丢弃功能：extent在释放事务提交后不会立即触发丢弃其包含的数据块，而是由独立的工作线程分组后统一进行，从而改善提交延迟。

异步丢弃可以与定期 TRIM 一同使用 。

有关 TRIM 的更多信息，请参阅固态硬盘#TRIM。

使用

查看空间使用情况

因为不能同时考虑文件和元数据的使用情况，df(1) 之类的通用用户空间工具在 Btrfs 上并不能准确地报告剩余空间大小。推荐使用 `btrfs filesystem usage` 来查看空间使用情况，例如：

```
# btrfs filesystem usage /
```

或者使用 `btrfs filesystem df` 快速检查已分配空间及其使用情况，无需 root 权限：

```
$ btrfs filesystem df /
```

注意：对于 #多设备文件系统，在 `RAID5` 或 `RAID6` #profile 与其它 profile 共存于同一文件系统时（例如，元数据使用 `DUP` 而数据使用 `RAID5`），`btrfs filesystem usage` 和 `btrfs filesystem df` 显示出的空间使用情况不准确 。

更多信息可参阅 。

因为不考虑 #写时复制（CoW） 和透明 #压缩，du(1) 和 ncdu(1) 之类的文件空间使用情况分析工具的数据也不准确。可考虑改用 btduAUR 或 compsize 等支持 Btrfs 特性的工具。

碎片整理

文件碎片整理是指尝试通过让文件包含的extent分布更加连续，合并小的extent，从而更有效地存储文件的过程。

由于 Btrfs 的 #写时复制（CoW）设计，修改文件时使用的新数据块在物理设备上可能与原数据块相距很远，造成文件的碎片化。碎片化在机械硬盘上是严重的问题，因为移动磁头到较远位置会导致延迟。对于现代没有这种延迟的设备（如固态硬盘）来说，这不再是问题，但进行碎片整理仍有意义，可以减少追踪分散extent所需的元数据大小。

可以安全地碎片整理使用中的文件数据，但更难达到最优分布。

警告：碎片整理会断开#写时复制（CoW）数据（包括通过 `cp` 复制的文件、#快照和#去重数据等）之间的 reflink，产生多份独立的数据，可能显著增加空间使用量 。

手动

使用 `btrfs filesystem defragment` 命令手动进行碎片整理，例如碎片整理根目录下的文件（不进入子卷、挂载点及到目录的软链接）：

```
# btrfs filesystem defragment -r /
```

注意：不使用 `-r` 参数则碎片整理不会递归进行，但会对某些内部树——区段树（extent tree）和子卷树——进行碎片整理。这种设计容易引发混淆，未来版本可能会取消。

提示：可以使用 `-c`、`-L` 参数指定压缩方式，参见压缩章节中 #为现有文件启用部分。

自动

使用挂载参数 `autodefrag` 启用自动碎片整理，参见 btrfs(5) § MOUNT OPTIONS。

scrub

scrub文件系统或设备是指验证其中所有数据和元数据的过程，能够检测数据校验和错误、基本超级块错误、基本元数据块头错误以及磁盘读取错误。

注意：运行中的检修任务会阻止系统挂起，详见该讨论。

手动启动

使用以下命令在后台启动 scrub 任务（此处“设备”指 Linux 块设备）：

```
# btrfs scrub start 路径或设备
```

如果指定路径，则 scrub 路径所在的文件系统；如果指定设备，则 scrub 该设备。

注意：在使用镜像块组 #profile（如 RAID1）的 #多设备文件系统上，scrub 操作还会通过从其他副本复制已验证完好的数据来自动修复任何损坏。

检查该任务的运行状态：

```
# btrfs scrub status /
```

也可以通过启动 `btrfs-scrub@.service` 来手动运行 scrub (使用同样的转移后路径)，相较 (以 root 用户身份运行) `btrfs scrub`，这么做的优点是输出内容会记录在 Systemd 日志中。

自动

btrfs-progs包 软件包带有 `btrfs-scrub@.timer` 系统单元，用来每月运行 scrub 命令。通过添加挂载点的参数来启用它，例如`btrfs-scrub@-.timer` (`/`) 或者 `btrfs-scrub@home.timer` (`/home`)。你可以使用`systemd-escape -p */path/to/mountpoint*`来转义路径，参见systemd-escape(1)。

balance

『balance 将会通过分配器再次传递文件系统中的所有数据。它主要用于在添加或删除设备时跨设备重新平衡文件系统中的数据。如果有设备出现故障，balance 将为指定的冗余 RAID 级别重新生成缺失的副本』。参阅上游的 FAQ。

在单设备文件系统上，balance 对于（临时）减少分配但未使用（元）数据块的数量也是有用的。有时候这对于解决 "filesystem full" 故障来说也是必须的。为减少无法分配出元数据块组的可能，不建议对元数据进行 balance 操作，可指定如 `-dusage=30` 只对数据块组操作。

```
# btrfs balance start --bg /
# btrfs balance status /
```

交换文件

注意：

Btrfs 与 Linux 交换子系统的实现导致一些限制：

- 不支持在跨设备的文件系统上使用交换文件
- 数据 #profile 需为 `single`
- 不能为含有正在使用中的交换文件的子卷创建快照
- 交换文件必须预分配（即无空洞）
- 必须单独为交换文件 #禁用写时复制（即交换文件必须有 `+C` 属性）

其中后两个对交换文件的限制 `btrfs filesystem mkswapfile` 命令会自动处理。

交换文件的介绍参见 Swap#交换文件。

在 Btrfs 中创建交换文件的正确方法是首先创建一个子卷存储交换文件，因为随后无法为该子卷创建快照。例如创建 `/swap` 子卷：

```
# btrfs subvolume create /swap
```

提示：考虑直接在顶层子卷下 #创建子卷，然后 #挂载子卷到 `/swap`（或其他可访问路径）。

在 `/swap/` 中创建一个 4 GiB 的交换文件 `swapfile`：

```
# btrfs filesystem mkswapfile --size 4g --uuid clear /swap/swapfile
```

`--size` 选项指定交换文件大小，默认为 2 GiB，最小 40 KiB。

启用交换文件：

```
# swapon /swap/swapfile
```

最后编辑 fstab，添加交换文件的配置：

```
/etc/fstab
/swap/swapfile none swap defaults 0 0
```

更多信息参见 fstab#用法。

有关删除交换文件等更多信息请参阅 Swap#交换文件。

提示：若要使用休眠功能，除了配置交换文件外，还需要完成电源管理/挂起与休眠中描述的额外步骤。

RAID

Btrfs 提供对 RAID #多设备文件系统的原生支持。相较于使用 mdadm，其显著优势包括自修复冗余阵列和在线数据平衡。更多信息请参见该 Btrfs Wiki 页面。Btrfs SysadminGuide 提供了一些更具技术性的背景信息。

警告：奇偶校验 RAID（RAID 5/6）代码中存在多个严重的数据丢失漏洞。更多信息请参阅 Btrfs 文档的 RAID 5/6 页面（英文）以及 linux-btrfs 邮件列表中关于此问题的错误报告。2020 年 6 月，有用户在上述邮件列表中整理发布了当前问题的完整清单及实用的恢复指南。

快照

"快照是和其它子卷共享数据和元数据的简单子卷, 利用了 btrfs 的写时复制特性。" 详见 Btrfs Wiki SysadminGuide#Snapshots 和 the Btrfs documentation。

要创建一个快照:

```
# btrfs subvolume snapshot source [dest/]name
```

`*source*`为要创建快照的对象，`[*dest*/]*name*`为快照安放路径。

加入 `-r` 参数可以创建一个只读快照. 为只读快照创建一个快照可以获得一个只读快照的可写入版本.

注意：

- 将一个只读快照通过`btrfs property set -f -ts '/path/to/snapshot' ro false`原位转换为可写快照是可能的。但并不推荐这样做，因为之后任何增量发送/接受（send/receive）操作会造成问题。创建一个新的可写快照可以预防这类问题。
- 快照不是递归包含的，这意味着子卷内的子卷在快照里是空目录。

发送和接收

可以通过 `send` 命令发送一个快照,通常会与 btrfs 中的 `receive` 组成管道.例如将快照 `/root_backup` (也许是/的备份) 发送到 `/backup`:

```
 # btrfs send /root_backup | btrfs receive /backup
```

*只能*发送只读快照,上面的命令在将子卷复制到外部设备 (例如备份驱动器) 时会很有用。

接受结束后将创建对应子卷，无需手动再创建。

还有一个例子创建了/mnt/arch-v2/subvolumes/@var子卷：

```
# btrfs send --proto 2 --compressed-data '/mnt/arch/snapshots/@var' | btrfs receive '/mnt/arch-v2/subvolumes/'
```

参数--proto 2和--compressed-data在本例中能够更有效地发送快照（假设数据可压缩）。

也可以只发送两个快照间发生变化的部分,例如如果你已经发送了快照 root_backup ,然后又建立了一个新的只读快照 root_backup_new ,可以这样完成增量发送:

```
 # btrfs send -p /root_backup /root_backup_new | btrfs receive /backup
```

现在你 /backup 的快照会是 root_backup_new。

参阅 Btrfs Wiki 上关于增量备份的页面 (英文) 获得更多信息 (例如使用工具自动化这一过程)。

去重

使用写时复制，Btrfs 能够复制文件或整个子卷而无需实际复制数据。但是，无论何时更改文件，都会创建一个新的“真正的”副本。重复数据删除更进一步，通过主动识别含有相同内容的数据块并将它们通过 CoW 合并到同一个extent内。

专用于 Btrfs 分区去重的工具包括 duperemove包和bees包。也许你还希望基于文件的级别对数据进行重复数据删除，比如 rmlint-git包、rdfind包、jdupesAUR 或者 dduper-gitAUR。有关这些程序的可用功能的概述和其他信息，请参阅上游 Wiki 条目 (英文)。

调整大小

警告：为避免数据丢失，确保在调整大小前备份好你的数据。

你可以增加文件系统的大小到设备的最大可用空间，或到一个指定大小。确保你在尝试增加文件系统大小前设备或逻辑卷有足够空间。 当指定一个设备上的文件系统到指定大小时，不论增加或减少，确保新的大小满足下面的条件：

- 新的大小必须大于已有数据的大小，否则会导致数据损失。
- 新的大小必须等于或小于当前设备的可用空间。

注意：如果你还计划减小文件系统所在的逻辑卷大小，请先减小文件系统大小后再尝试减小逻辑卷的大小。

将文件系统扩展到设备的最大可用大小：

```
# btrfs filesystem resize max /
```

将文件系统扩展到特定大小：

```
# btrfs filesystem resize size /
```

将 *size* 替换为你需要的大小（按字节计算）。也可以为大小指定单位，如K（千字节），M（兆字节）或G（吉字节）。 或者也可以通过在大小前添加前缀指定增加（+）或减少（-）的变化量：

```
# btrfs filesystem resize +size /
# btrfs filesystem resize -size /
```

已知问题

一些在尝试之前应该知道的限制。

加密

Btrfs 暂未内置加密支持，但当前正进行一项基于 Fscrypt 的加密集成工作 。

不过用户仍可在运行 `mkfs.btrfs` 前加密**分区**，具体参见 dm-crypt/加密整个系统。另一种方案是采用栈式文件系统加密。

检查 btrfs 文件系统问题

`btrfs check` 工具目前有一些已知问题，在继续深入阅读了解之前，您不应该直接运行它, 参见 #检查 Btrfs 文件系统小节。

提示和技巧

无分区 Btrfs 磁盘

警告：这种配置不建议用于启动设备，推荐设置一个单独的EFI 系统分区和Btrfs分区。 此外，GRUB 强烈反对安装GRUB到无分区磁盘。

Btrfs 能占用整个存储设备，使用子卷模拟分区表来替代 MBR 或 GPT 分区表。虽然无分区磁盘不需要使用其它方法创建分区，然后在一个分区上创建 Btrfs 文件系统，但在单个磁盘上使用无分区配置存在一些限制：

- 不能在同一磁盘上的不同分区上创建其它的文件系统。
- 受上一条影响，无法在该磁盘上创建EFI 系统分区。需要额外的储存设备用于 UEFI 启动。

运行下面的命令把整个设备的分区表替换成 Btrfs:

```
# mkfs.btrfs /dev/sdX
```

如果设备上存在分区表，则需要使用：

```
# mkfs.btrfs -f /dev/sdX
```

例如， 指定/dev/sda 而不是 /dev/sda1。后一种形式会格式化现有的分区而不是替换掉原有的分区表。由于根分区是 Btrfs 文件系统，请确保已将 btrfs 编译进内核，或者将 btrfs 放入 Mkinitcpio#模块（MODULES）中并且重新生成 initramfs。

像使用普通的 MBR 分区表存储设备一样安装引导加载程序, 参考 Syslinux#手动安装 或 GRUB/技巧和窍门#安装到分区上或者无分区磁盘上。 如果你的内核因为 Failed to mount /sysroot. 错误无法启动, 请在 /etc/default/grub 里添加 GRUB_PRELOAD_MODULES="btrfs" 并重新生成 GRUB 配置文件 。

从 Ext3/4 转换

警告：Btrfs 的邮件列表中报告了多起转换不完整/损坏/失败的案例。在开始之前请确定您有*可用的*备份并且愿意承担丢失数据的风险。 参见Btrfs维基的Convert页。

从安装 CD 启动，然后转化分区：

```
# btrfs-convert /dev/partition
```

挂载转换后的分区并修改 /etc/fstab 文件， 指定分区类型 (**type** 为 btrfs，并且 **fs_passno**\[最后一列\] 要修改为0，因为 Btrfs 在启动时并不进行磁盘检查)。 还要注意的是分区的 UUID 将有改变，所以使用 UUID (指定分区) 时，请更新 fstab 中相应的条目。 chroot 到系统并重建你的引导加载程序（如果对此过程不熟悉，参考从现有 Linux 发行版安装 Arch Linux)。 如果转换了根文件系统，还需要在 chroot 环境中重建初始化内存盘以确保系统正确启动。

注意：

- 如果转换过程中有任何异样，不管是无法挂载新转换的 Btrfs 文件系统或是无法往其中写入数据，只要备份子卷 /ext2_saved 还在，就可以进行回滚。请使用 btrfs-convert -r /dev/*partition* 命令进行回滚，这将会丢弃任何对新转换 Btrfs 文件系统的更改。
- 如果挂载已转换的 ext 分区时出现如下错误：ERROR: dev extent devid 1 physical offset XXX len XXX is beyond device boundary XXX，那么在分区末尾预留一些空闲空间后，转换可能会成功。请先回滚转换，然后使用 resize2fs 将分区缩小 50000 个块，再扩大 5000 个块（从而创建出 45000 个空闲块）。然后再次尝试转换。参见 上游 bug 报告。

确认没有问题后，通过删除 ext2_saved 备份子卷完成转换的最后一步。请注意，如果没了它 (备份子卷)，你将没办法还原回 ext3/4 文件系统。

```
# btrfs subvolume delete /ext2_saved
```

最后通过数据平衡)回收空间。

别忘了先前安装的一些应用需要额外设置来适配Btrfs。

注意：Ext3/4 转换到 Btrfs 的过程很耗时。一个普通机械硬盘上的4TB文件系统转换耗时可高达10小时。

从 NTFS 转换

使用 ntfs2btrfsAUR 或 ntfs2btrfs-gitAUR 转换NTFS分区。 卸载目标文件系统并运行转换，并指定确切的压缩和校验和类型：

```
# ntfs2btrfs -c zstd -h crc32c /dev/partition
```

如果转换后一切正常，你可以删除原本用于恢复原始文件系统的 `image/ntfs.img` 。

转换后

警告：只有在确认不会恢复到原始文件系统后，才执行这些步骤，因为它们会使得恢复变得不可能，即使你保留了恢复文件。

可以在将现有文件系统转换为 Btrfs 后采取一些步骤使文件数据更连续。

```
# btrfs filesystem defrag -v -r -f -t 32M /mnt/filesystem
```

#Balance 使 Btrfs 元数据更加紧凑

损坏恢复

警告：`btrfs check` 工具有一些已知问题，参见 #检查 Btrfs 文件系统 小节。

btrfs-check 不能在一个已挂载的文件系统上工作。为了能够在不从 Live USB 启动的情况下使用 *btrfs-check*，需要将其添加到初始内存盘：

```
/etc/mkinitcpio.conf
BINARIES=(btrfs)
```

然后重新生成 initramfs。

之后如果启动时出现问题，则可以使用该实用程序进行修复。

注意：如果 fsck 进程必须使空间缓存 (和/或其他缓存？) 无效 (invalidate cache)，那么随后的引导会挂起一段时间，这是正常的（进程可能会给出关于 btrfs-transaction 挂起的控制台消息）。系统应该在一段时间后从中恢复正常。

查阅 btrfs-check(8) 以获取更多信息。

引导进入快照

要引导进入快照，因为快照可以像子卷那样被挂载，所以请像挂载子卷为根分区那样进行同样的流程 (已交代于#挂载子卷为根挂载点的一段中)。

- 如果使用 GRUB，则可以在 grub-btrfs包 或 grub-btrfs-gitAUR 的帮助下，在重新生成配置文件时使用 Btrfs 快照自动填充启动菜单。
- 如果使用 rEFInd，则可以在 refind-btrfsAUR 的帮助下，启用 `refind-btrfs.service`后，在重新生成配置文件时使用 Btrfs 快照自动填充启动菜单。

- 如果使用 Limine（英语：Limine），可安装 limine-snapper-syncAUR。该工具在启用 `limine-snapper-sync.service` 后，每当 Snapper 列表变更时，将自动在启动菜单中生成快照条目。详情参见 Lamine#Btrfs 的 Snapper 快照集成（英语：Limine#Snapper snapshot integration for Btrfs）。

搭配 systemd-nspawn 使用 Btrfs 子卷

可查阅 systemd-nspawn#使用Btrfs子卷作为容器的根和 systemd-nspawn#在 systemd-nspawn 中运行 docker 等文章。

减少访问时间元数据的更新

由于Btrfs的写时复制性质，访问文件就能够触发元数据的写时复制。减少访问时间的更新频率可能会消除这种不希望的硬盘使用并提高性能。参见Fstab#atime 参数

增量备份到外置设备

下列包利用 `btrfs send` 和 `btrfs receive` 将备份增量发送到外置设备上。 参考它们的文档来查看实现，功能和需求的不同。

- **btrbk** — Btrfs子卷的快照和远程备份工具

: https://github.com/digint/btrbk || btrbk包

- **snap-sync** — 使用Snapper快照并备份到外置设备或远程机器

: https://github.com/wesbarnett/snap-sync || snap-sync包

下列工具运行备份snapper快照到非Btrfs文件系统。

- **snapborg** — 一个类似borgmatic的工具，将snapper快照与borg包整合起来

: https://github.com/enzingerm/snapborg || snapborgAUR

自动快照

要管理并自动创建快照，你可以使用像Snapper，Timeshift或Yabsnap这样的快照管理器。

自动通知

桌面通知能帮助您及时察觉严重的 Btrfs 问题，相较于完全不提供通知，能提供更好的问题感知。

btrfs-desktop-notificationAUR 为以下事件提供桌面通知：

- 启动进入任何只读快照或系统时
- dmesg 日志中出现 Btrfs 警告、错误或致命消息

更多信息及配置方法，请参见 https://gitlab.com/Zesko/btrfs-desktop-notification .

空间缓存v1正在被弃用

如果你的 Btrfs 文件系统是用旧的默认设置创建的，你可能会在你的系统日志中看到类似的警告:

```
BTRFS warning (device sdb4): space cache v1 is being deprecated and will be removed in a future release, please use -o space_cache=v2
```

将 Btrfs 空间缓存从旧版 v1 格式转换为新版 v2 格式并解决此警告. 你可以修改分区挂载选项，在 fstab 中包含 `space_cache=v2` 或者手动进行转换：

```
# umount /dev/partition
# mount /dev/partition -o rw,space_cache=v2 /mnt
# umount /mnt
```

注意：在此转换后，Btrfs会在文件系统上设置一个compat-ro功能标志。因此，该文件系统可能无法再被非常旧的Btrfs内核实现挂载。

疑难解答

请查阅 Btrfs 疑难解答和Btrfs 常见问答集（英文）以获得排除一般问题的信息。

启动缓慢

有时，大于4TB的Btrfs卷需要较长时间才能挂载，从而导致系统启动变慢。 将群组树更改为块组树有助于解决该问题：

```
# btrfstune --convert-to-block-group-tree /dev/device
```

GRUB

分区偏移

注意：当您试图将 `core.img` 嵌入到已分区磁盘上时，可能会发生偏移问题。这意味着可以将 GRUB 的 `corg.img` 直接嵌入到无分区磁盘 (例如 `/dev/sd*X*`) 上的 Btrfs 存储池中。

GRUB 可以引导启动 Btrfs 分区，但是因为模块可能会比其它文件系统大，`grub-install` 生成的 `core.img` 文件超过了 MBR 与第一个分区之间的空间大小 (63 扇区/31.5KiB)。最新版的 `fdisk` 和 `gdisk` 等磁盘工具会通过第一个分区前空出 1-2MiB 的空间来避免此问题。

根丢失

**本文或本章节的事实准确性存在争议。**

原因： Suggests editing a non-configuration file manually.（在 Talk:Btrfs#Should not suggest to edit files in /usr/share 中讨论）

当从一个RAID配置的系统中启动用户可能会遇到错误：`error no such device: root`。编辑`/usr/share/grub/grub-mkconfig_lib`，移除行`echo " search`

`--no-floppy --fs-uuid --set=root ${hints} ${fs_uuid}`"中的两个引号。重新生成grub的配置文件，后系统应该能够正常启动。

警告：不建议覆盖软件包提供的文件，因其会在下次更新时被覆盖。

挂载超时

有时（尤其在大型 RAID1 阵列上）系统启动时会出现挂载超时的现象，并带有如下日志信息：

```
Jan 25 18:05:12 host systemd: storage.mount: Mounting timed out. Terminating.
Jan 25 18:05:46 host systemd: storage.mount: Mount process exited, code=killed,
status=15/TERM
Jan 25 18:05:46 host systemd: storage.mount: Failed with result 'timeout'.
Jan 25 18:05:46 host systemd: Failed to mount /data.
Jan 25 18:05:46 host systemd: Startup finished in 32.943s (firmware) + 3.097s (loader)
+ 7.247s (kernel)>
Jan 25 18:05:46 host kernel: BTRFS error (device sda): open_ctree failed
```

这可以通过 fstab 中特定的 systemd 挂载选项 `x-systemd.mount-timeout` 提供系统以更长的超时时间来轻松解决。例如：

```
 UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  /data  btrfs  rw,relatime,x-systemd.mount-
timeout=5min  0 0
```

出现 BTRFS: open_ctree failed 错误

**本文或本章节的事实准确性存在争议。**

原因：内容过时。自从systemd和udev都使用systemd-udevd后，用systemd替换udev钩子没有任何意义。（在 Talk:Btrfs 中讨论）

截至 2014 年 11 月，一个似乎存在于 systemd 或 mkinitcpio 中的 Bug 可能会导致在 `mkinitcpio.conf` 中使用 `btrfs` 钩子（hook）的用户在启动多设备文件系统的 Btrfs 卷时遇到以下错误：

```
BTRFS: open_ctree failed
mount: wrong fs type, bad option, bad superblock on /dev/sdb2, missing codepage or
helper program, or other error

In some cases useful info is found in syslog - try dmesg|tail or so.

You are now being dropped into an emergency shell.
```

一种解决办法是，在`/etc/mkinitcpio.conf`中，将 `HOOKS` 中的 `btrfs` 移入 `MODULES` 中，然后重新生成 initramfs 并重新启动。

另外，如果在挂载 RAID 卷组缺少某个卷时，也有可能会发生这个错误。这种情况下，需要把 `degraded` 加入到 `/etc/fstab` 中；如果根目录在卷组上，需要同时加入内核参数 `rootflags=degraded`。

接上文，截至 2016 年 8 月，针对这一问题，一个可能的解决方案是在 `/etc/fstab` 中仅靠单个硬盘来挂载阵列，然后让 Btrfs 自动发现并追加其它硬盘。UUID 和 LABEL 等基于组的标识符似乎是导致出现错误的原因。比如说，由“disk1”（磁盘 1）

和“disk2”（磁盘 2）组成的双设备 RAID1 阵列会被分配到一个 UUID。但是请在 `/etc/fstab` 中只使用 `/dev/mapper/disk1` 来指定磁盘阵列，而不要使用 UUID。更多解释，参见这个博客文章。

另一个可能的解决方法是在 mkinitcpio.conf 中移除 `udev` 钩子并替换为 `systemd` 钩子。此时 `btrfs` 不应出现在 `HOOKS` 或 `MODULES` 列表中。

请查阅原论坛讨论和 FS#42884 获得更多的讨论内容和信息。

检查 Btrfs 文件系统

警告：Btrfs（特别是 `btrfs check` 命令工具）仍处在大开发阶段，强烈建议在加上 `--repair` 参数运行 `btrfs check` 前先做一个备份，并提前查阅 btrfs-check(8)。

btrfs-check(8) 可以检查并修复一个未挂载的 Btrfs 文件系统。但是由于它尚未开发完成，它并不能修复某些错误（即使这些错误没有导致文件系统无法挂载）。

持续的硬盘活动

自内核版本6.2开始， mount(8) 默认启用 `discard=async` 选项。 这个设置被报告会造成硬盘的持续活动（甚至是待机状态下），因为丢弃队列填充的速度快于其处理速度。这会造成电源使用增加，特别是NVMe设备。

到了内核版本6.3后，问题的解决办法是将`iops_limit`默认值从100改为1000。在旧版内核上，你可以手动设置到一个需要的值，例如：

```
# echo 1000 > /sys/fs/btrfs/uuid/discard/iops_limit
```

其中`*uuid*`是btrfs文件系统的UUID。限制值`1000`需要通过实验进行调整。

要在启动时设置该参数，可能要用到Systemd#systemd-tmpfiles - 临时文件，例如创建下列文件：

```
/etc/tmpfiles.d/btrfs-discard.conf
w /sys/fs/btrfs/uuid/discard/iops_limit - - - - 1000
```

或者在fstab中使用`nodiscard`挂载选项禁用异步丢弃功能，使用定期TRIM来替代。

Device total_bytes should be at most X but found Y

若驱动器从其他计算机迁移或设备顺序变更，且报告的大小差异极小（至多几MB），可能是启用了 HPA（主机保护区域）。

可使用 hdparm包 验证 HPA 是否启用：

```
# hdparm -N 设备
```

输出将显示两个数值：可见扇区数与实际扇区数。若两者不同则表明HPA已启用。

若主板强制设置此功能且固件未提供关闭选项，唯一解决方案是收缩受影响文件系统。此操作最易在原始计算机或未应用HPA的机器上执行。

设备空间不足

博客文章《解决Btrfs文件系统空间不足问题》建议并解释了以下检查步骤：

1. 立即清理空间（删除历史快照）
2. 文件系统是否真满？元数据和/或数据块分布不均（运行 `btrfs balance`）
3. 文件系统是否真满？数据块分布不均
4. 文件系统是否真满？元数据分布不均
5. 因空间不足无法执行平衡操作（运行 `btrfs balance` 前，先用 `btrfs device add` 临时添加设备如U盘或环回设备）

另可参阅 btrfs 元数据满了怎么办 - 依云's Blog。

有关 ENOSPC（“磁盘空间不足”）的最新处理指南，请参阅 ENOSPC - No available disk space | Forza's Ramblings。

另请参阅

- **官方网站**
- Btrfs 文档
- 已归档的 Btrfs 百科
- **性能相关**
- Btrfs on raw disks?
- Varying leafsize and nodesize in Btrfs
- Btrfs support for efficient SSD operation (data blocks alignment)
- Is Btrfs optimized for SSDs?
- Lzo vs. zLib
- **杂项**
- Funtoo:BTRFS Fun
- Avi Miller presenting Btrfs at SCALE 10x, January 2012.
- Summary of Chris Mason's talk from LFCS 2012
- Btrfs: stop providing a bmap operation to avoid swapfile corruptions 2009-01-21
- Doing Fast Incremental Backups With Btrfs Send and Receive

1. ↑ bees 的 issue 中有讨论过，具体链接以后再补。

1.21 ZFS

难度 高级 | **篇幅** 82.0 KB | **所属** 文件系统

本节将讲到

- 概念
- 位于内核树外导致的问题
- 安装
- 配置
- 存储池

翻译状态：

- 本文（或部分内容）译自 ZFS，最近一次同步于 2025-01-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

ZFS 最早是由太阳计算机公司为 solaris 操作系统开发并发布的一个先进文件系统，现在的 ZFS 一词通常指 OpenZFS 分支，该分支将原先的实现移植到了包括 Linux 等其它操作系统上，同时继续开发 solaris ZFS。本文将把 ZFS 和 OpenZFS 视为同义词。

ZFS 具有丰富的特性，其中包括：更好的页缓存算法：ARC、去重、池化存储、快照)、复制、数据完整性检查及自动修复（scrubbing）、RAID-Z 等等。

概念

与将数据存放在单个块设备上的常见文件系统不同，ZFS 将数据存放在存储池中。存储池由 vdev（虚拟设备）组成，而 vdev 又由块设备构成。存储池始终将数据写入到剩余空间占比最大的 vdev 中，使数据条带化分布在各个 vdev 内。另一方面，vdev 可以使用如 RAIDZ 和镜像更复杂的配置。在最简单的配置中，可以在由单个分区组成的单个 vdev 上创建一个池，其行为与常规文件系统类似。

创建后，就可以从池中分配存储资源。这些资源被分组为称为数据集的单元。数据集有 4 种类型：

1. 文件系统：文件系统基本上就是一个目录树，可以像常规文件系统一样被挂载到系统命名空间中
2. 卷（zvol）：表现为块设备的卷
3. 快照：文件系统或卷的快照
4. 书签：不存储任何数据的快照，用于增量复制

数据集被标识为一条唯一路径，格式如下：

```
pool(/segment)+((#|@)bookmark/snapshot name)?
```

其中 `#` 被用于书签，`@` 被用于快照。

注意：上述内容是 zpoolconcepts(7) 和 zfsconcepts(7) 的简短摘要。强烈建议阅读这两篇文档，以熟悉其中的概念以及此处未涉及的技术术语。

位于内核树外导致的问题

由于复杂的法律问题，Linux 内核维护者拒绝将 ZFS 并入到 Linux 内核中，因此 ZFS 被作为树外内核模块进行开发。由此导致的一个问题是：内核升级经常会导致 ZFS 使用的内核 API 被修改。在这种情况下，ZFS 将被迫修改代码以使用新的 API，也就意味着在一段时间内 ZFS 将不兼容最新的主线内核版本。

提示：由于 linux包 会密切跟踪最新稳定内核版本，如果你不想将 linux包 包固定在特定版本，最好是使用 linux-lts包。

安装

作为树外模块，有两种类型的包可供选择安装：一种是为特定内核版本构建的二进制内核模块，另一种是将其源码安装为 DKMS 模块，并在内核更新时自动重新构建。

除了内核模块外，用户还需要安装如 zpool(8) 和 zfs(8) 的用户空间工具。这些工具通常被打包为单个软件包，名称为 `zfs-utils*`。

下面提到的所有内核模块都已在依赖中指定所需的 `zfs-utils*` 包，因此在安装时你只需满足其依赖即可。

二进制内核模块

<table class="wikitable"><caption>各软件包的比较</caption><tbody><tr><th>包名</th><th>软件源</th><th>ZFS 发布类型</th><th>目标 kernel</th><th>二进制包</th><th>附注</th></tr><tr><td>zfs-linux-lts^{<small>AUR</small>}</td><td>AUR</td><td>稳定版</td><td>linux-lts^{<small>包</small>}</td><td>否</td><td rowspan="2">强烈建议在构建新版 ZFS 包时使用 devtools，从而无需在升级内核时卸载当前安装的 ZFS 包。</td></tr><tr><td>zfs-linux^{<small>AUR</small>}</td><td>AUR</td><td>

稳定版</td><td>linux^{<small>包</small>}</td><td>否</td></tr><tr><td>zfs-linux</td><td>archzfs</td><td>稳定版</td><td>linux</td><td>是</td><td></td></tr><tr><td>zfs-linux-lts</td><td>archzfs</td><td>稳定版</td><td>linux-lts</td><td>是</td><td></td></tr></tbody></table>

DKMS

- zfs-dkmsAUR：稳定版本
- zfs-dkms-staging-gitAUR：稳定版本，包含为最新 linux包 内核版本提供支持的补丁

使用 ZFS 作为根分区

参见在 ZFS 上安装 Arch Linux。

配置

启动时导入存储池

ZFS 提供了用于自动导入存储池的 systemd 服务，和为其它单元提供 ZFS 初始化状态的目标，其中包括：

- `zfs.target`：全部 ZFS 服务完成后激活
- `zfs-import.target`：完成 ZFS 存储池导入后激活
- `zfs-volumes.target`：所有 zvol 都出现在 `/dev` 下后激活
- `zfs-import-scan.service`：使用 libblkid 扫描设备并导入存储池
- `zfs-import-cache.service`：检查 `zpool.cache` 并导入存储池
- `zfs-volume-wait.service`：等待所有 zvol 都可用

你需要在 `zfs-import-scan.service` 和 `zfs-import-cache.service` 间二选一，并启用其它全部服务和目标。

zfs-import-scan

`zfs-import-scan.service` 使用了 `zpool import` 的默认逻辑：使用 blkid 扫描设备，意味着其不需要使用 `zpool.cache` 文件。鉴于 `zpool.cache` 已被废弃，建议使用该方法。

如果 `zpool.cache` 存在且不为空，那么 `zfs-import-scan.service` 就不会启动，因此需要确保你的所有存储池导入时都没有启用 `cachefile` 选项，该操作可通过启用 `zfs` 模块的 C4 选项实现。你还需要删除现有的 `zpool.cache`，或在启动时将所有导入的存储池的 `cachefile` 选项设为 `none`。

zfs-import-cache

`zfs-import-cache.service` 在导入存储池时使用 `zpool import -c <zpool.cache>`，该操作会从 `zpool.cache` 读取设备路径。

由于重启或出现硬件更改后设备路径可能会变化，因此在使用该方法时需注意创建 ZFS 存储池时使用的设备路径，否则会导致缓存过时并导入失败。关于如何选择持久化设备路径的信息请参考块设备持久化命名。

自动挂载文件系统

`zfs-import-scan.service` 和 `zfs-import-cache.service` 服务会导入存储池，但不会挂载任何文件系统。根据你的系统是否配置了 `mountpoint=legacy`，有两种方法可以在启动时导入文件系统。如果你的文件系统混用了 legacy 与非 legacy 配置，那就需要同时使用两种方式。

zfs-mount-generator

如果你的文件系统使用非 legacy 挂载，那么建议使用 zfs-mount-generator，这是一个 systemd.generator(7)，可以为导入的 ZFS 存储池中的所有文件系统生成带有 `canmount=on` 选项的 systemd 挂载单元文件，以便在启动时挂载文件系统。由于其需要使用 `zfs list` 缓存，默认情况下运行 zfs-mount-generator 不会产生任何效果，你需要进行以下操作：

1. 启用并启动 `zfs-zed.service`
2. 创建 `/etc/zfs/zfs-list.cache` 目录
3. 在 `/etc/zfs/zfs-list.cache` 中创建以你的存储池为名的空文件，ZFS Event Daemon（ZED）只会在存储池对应文件存在并可写入时更新文件系统清单：

```
# touch /etc/zfs/zfs-list.cache/<pool-name>
```

1. 检查 `/etc/zfs/zfs-list.cache/<pool-name>` 的内容。如果其内容为空，需要执行以下命令修改任意 ZFS 文件系统的 canmount 属性，以生成 ZFS 事件来触发 ZED：

```
zfs set canmount=off zroot/fs1
```

，然后执行

```
zfs inherit canmount zroot/fs1
```

fstab

如果你的文件系统使用 legacy 挂载，那就需要在 fstab 文件中指定挂载点。设备一项需填入文件系统的完整路径名称，dump 及 fsck 项需填为 0。

创建 hostid 文件

通常没有需要，但还是建议创建 `/etc/hostid` 文件：

```
# zgenhostid $(hostid)
```

存储池

尝试使用 ZFS

如果有用户希望在不会造成数据丢失的情况下试用 ZFS，可参考 ZFS/Virtual disks。

创建 ZFS 池

提示：你可能需要先阅读 #ashift 属性，因为建议在创建池时设置 `ashift`。

要创建 ZFS 池，请使用如下命令：

```
# zpool create -R <root> -o <poolopts> -O <dsetprops> <pool> <vdevs>
```

其中 `vdev` 是单个设备或使用以下格式：

```
<vdev type> <device> ... <device>
```

- **\-R**：在该文件夹下挂载所有文件系统，用于不影响当前系统
- **\-o**：指定存储池属性，可多次使用。类似 `ashift` 等属性在创建后就无法修改（理论上 `ashift` 是各 vdev 独立的，但要为 vdev 单独配置该参数就需要执行 `zpool add`）。
- **\-O**：指定存储池根数据集的属性，可多次使用。类似 `normalization` 等属性在创建后就无法更改。
- **pool**：存储池的名称
- **vdev type**：受支持的 vdev 清单请参考 zpoolconcepts(7)。
- **device**：块设备，可以是完整路径或路径的文件名部分

注意：取决于挂载存储池使用的具体方法，你可能要注意创建存储池时使用的设备路径。

以在单个分区上创建存储池为例：

```
# zpool create -R /mnt pool /dev/sda
```

使用单个 `raidz1` vdev 创建池：

```
# zpool create -R /mnt pool \
               raidz1 \
                  ata-ST3000DM001-9YN166_S1F0KDGY \
                  ata-ST3000DM001-9YN166_S1F0JKRR \
                  ata-ST3000DM001-9YN166_S1F0KBP8 \
                  ata-ST3000DM001-9YN166_S1F0JTM1
```

使用两个镜像（`mirror`）vdev 创建池：

```
# zpool create -R /mnt pool \
               mirror \
                  ata-ST3000DM001-9YN166_S1F0KDGY \
                  ata-ST3000DM001-9YN166_S1F0JKRR \
               mirror \
                  ata-ST3000DM001-9YN166_S1F0KBP8 \
                  ata-ST3000DM001-9YN166_S1F0JTM1
```

ashift 属性

`ashift` 是一个不可修改的 vdev 独立属性，决定了（逻辑）扇区大小为 2^ashift 字节。为提升性能，逻辑扇区大小需要大于或等于硬盘的物理扇区大小。

默认情况下，`zpool create` 可以自动识别设备的物理扇区大小，适用于单硬盘配置场景。

但如果你需要（或打算）配置如 `mirror` 或 `raidzX` 的可替换故障硬盘的 vdev 环境，建议始终设定 `ashift=12`。这是因为在 512 字节硬盘上使用 4kb 逻辑扇区大小没有问题，但反过来操作就会导致性能下降（除非你的硬盘是比较少见的 8kb 扇区大小 SSD）。

提示：可以使用

```
$ lsblk --filter 'TYPE=="DISK"' -o NAME,PHY-SEC
```

来检查硬盘的物理扇区大小。

另外，如果你在使用 NVMe 硬盘，有概率可以将其格式化为更高性能的 LBA 格式（具体参考 nvme-format(1)）。

创建兼容 GRUB 的存储池

默认情况下，*zpool create* 会为存储池启用所有特性。如果使用 GRUB 时将 `/boot` 放置到了 ZFS 下，就需要将 GRUB 不支持的特性全部禁用，否则 GRUB 将无法读取池中的数据。ZFS 内置了兼容性文件（参见 `/usr/share/zfs/compatibility.d`），可以帮助创建仅包含部分特性集的存储池，其中就包括了 grub2。

可以通过如下命令创建包含部分特性集的存储池：

```
# zpool create -o compatibility=grub2 $POOL_NAME $VDEVS
```

验证存储池状态

如果命令成功执行，则不会有任何输出。使用 mount 命令会显示存储池已被挂载。使用 `zpool status` 会显示存储池已被创建：

```
# zpool status
  pool: bigdata
 state: ONLINE
 scan: none requested
config:

        NAME                                        STATE     READ WRITE CKSUM
```

```
        bigdata                                     ONLINE       0     0     0
          -0                                        ONLINE       0     0     0
            ata-ST3000DM001-9YN166_S1F0KDGY-part1   ONLINE       0     0     0
            ata-ST3000DM001-9YN166_S1F0JKRR-part1   ONLINE       0     0     0
            ata-ST3000DM001-9YN166_S1F0KBP8-part1   ONLINE       0     0     0
            ata-ST3000DM001-9YN166_S1F0JTM1-part1   ONLINE       0     0     0

errors: No known data errors
```

删除存储池

要删除整个存储池：

```
# zpool destroy <pool>
```

接下来检查下存储池状态：

```
# zpool status
no pools available
```

导出存储池

通过以下命令导出存储池：

```
# zpool export <pool>
```

扩展现有存储池

可以通过如下命令将一个设备（单个分区或硬盘）添加到现有 zpool：

```
# zpool add <pool> <device-id>
```

添加设备为镜像

可以将设备（分区或硬盘）作为镜像附加到现有设备上（与 RAID1 类似）：

```
# zpool attach <pool> <device-id|mirror> <new-device-id>
```

你可以将新设备添加到现有镜像 vdev 中（例如从 2 盘镜像变为 3 盘镜像）或将其附加到单个设备上以构成新的镜像 vdev。

重命名存储池

可以用以下两步重命名已创建的存储池：

```
# zpool export oldname
# zpool import oldname newname
```

更换挂载点

可以通过如下命令修改 zpool 的挂载点：

```
# zfs set mountpoint=/foo/bar poolname
```

升级存储池

将 ZFS 升级到新版本后，有可能会获得一些新功能。出于兼容性考虑，ZFS 不会自动为之前创建的存储池启用新功能，而需要单独为每个存储池手动启用。

要检查是否可以升级：

```
$ zpool upgrade
This system supports ZFS pool feature flags.
```

```
All pools are formatted using feature flags.

Every feature flags pool has all supported and requested features enabled.
```

如果有可以升级的存储池，会出现如下输出：

```
This system supports ZFS pool feature flags.

All pools are formatted using feature flags.

Some supported features are not enabled on the following pools. Once a
feature is enabled the pool may become incompatible with software
that does not support the feature. See zpool-features(7) for details.

Note that the pool 'compatibility' feature can be used to inhibit
feature upgrades.

POOL  FEATURE

rpool
      redaction_list_spill
      raidz_expansion
      fast_dedup
      longname
      large_microzap
```

升级单个存储池：

```
# zpool upgrade <pool>
```

使用如下命令来升级所有存储池：

```
# zpool upgrade -a
```

创建数据集

相对于在存储池中创建文件夹，用户可以选择在存储池中创建数据集。除了快照外，数据集还提供了如配额控制等更强大的控制功能。在创建并挂载数据集前，需确保存储池中不存在与数据集同名的文件夹。以下命令可用于创建数据集：

```
# zfs create <存储池名>/<数据集名>
```

可以对数据集应用 ZFS 特定属性。例如，你可以对数据集中的文件夹设定配额限制：

```
# zfs set quota=20G <存储池名>/<数据集名>/<文件夹>
```

如需了解更多 ZFS 命令，可查阅 zfs(8) 或 zpool(8)。

原生加密

ZFS 支持如下几种加密方式：`aes-128-ccm`, `aes-192-ccm`, `aes-256-ccm`, `aes-128-gcm`, `aes-192-gcm` 及 `aes-256-gcm`。当加密设置为 `on` 时，将使用 `aes-256-gcm` 进行加密。See zfs-change-key(8) for a description of the native encryption, including limitations.

支持下列几种密钥格式：`passphrase`, `raw`, `hex`。

One can also specify/increase the default iterations of PBKDF2 when using `passphrase` with `-o pbkdf2iters <n>`, although it may increase the decryption time.

注意：

- To import a pool with keys, one needs to specify the `-l` flag, without this flag encrypted datasets will be left unavailable until the keys are loaded. See #Importing a pool created by id.
- Native ZFS encryption has been made available in the stable 0.8.0 release or newer. Previously it was only available in development versions provided by packages like zfs-linux-gitAUR, zfs-dkms-gitAUR or other development builds. Users who were only using the development versions for the native encryption, may now switch to the stable releases if they wish.
- The default encryption suite was changed from `aes-256-ccm` to `aes-256-gcm` in the 0.8.4 release.

使用如下命令创建通过密码短语加密的数据集：

```
# zfs create -o encryption=on -o keyformat=passphrase <存储池名>/<数据集名>
```

使用密钥而不是密码短语进行加密：

```
# dd if=/dev/random of=/path/to/key bs=32 count=1 iflag=fullblock
# zfs create -o encryption=on -o keyformat=raw -o keylocation=file:///path/to/key <存储池名>/<数据集名>
```

The easy way to make a key in human-readable form (`keyformat=hex`):

```
# od -Anone -x -N 32 -w64 /dev/random | tr -d [:blank:] > /path/to/hex.key
```

验证密钥位置：

```
# zfs get keylocation <存储池名>/<数据集名>
```

更改密钥位置：

```
# zfs set keylocation=file:///path/to/key <存储池名>/<数据集名>
```

你也可以下列任意一条命令手动加载密钥：

```
# zfs load-key <存储池名>/<数据集名> # load key for a specific dataset
# zfs load-key -a # load all keys
# zfs load-key -r zpool/dataset # load all keys in a dataset
```

挂载加密数据集：

```
# zfs mount <存储池名>/<数据集名>
```

启动时解锁/挂载：systemd

可以使用 systemd 单元在启动时自动解锁数据集。例如，可以创建如下服务来解锁特定的数据集：

```
/etc/systemd/system/zfs-load-key@.service
[Unit]
Description=Load %I encryption keys
Before=systemd-user-sessions.service zfs-mount.service
After=zfs-import.target
Requires=zfs-import.target
DefaultDependencies=no

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/bash -c 'until (systemd-ask-password "Encrypted ZFS password for %I"
--no-tty | zfs load-key %I); do echo "Try again!"; done'

[Install]
WantedBy=zfs-mount.service
```

接下来为每个加密数据集启动/启用该服务 (例如 `zfs-load-key@pool0-dataset0.service`)。注意，`-` 在 systemd 单元中的定义为 `/`，详细资料可参考 `systemd-escape(1)`。

注意：The `Before=systemd-user-sessions.service` ensures that systemd-ask-password is invoked before the local IO devices are handed over to the desktop environment.

另一种方法是加载所有可能用到的密钥：

```
/etc/systemd/system/zfs-load-key.service
[Unit]
Description=Load encryption keys
DefaultDependencies=no
After=zfs-import.target
Before=zfs-mount.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/zfs load-key -a
StandardInput=tty-force

[Install]
WantedBy=zfs-mount.service
```

接下来启动/启用 `zfs-load-key.service`。

登录时解锁：PAM

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Missing prompts in front of commands.（在Talk:ZFS讨论）

If you are not encrypting the root volume, but only the home volume or a user-specific volume, another idea is to wait until login to decrypt it\[失效链接 2024-11-06 ⓘ\]. The advantages of this method are that the system boots uninterrupted, and that when the user logs in, the same password can be used both to authenticate and to decrypt the home volume, so that the password is only entered once.

First set the mountpoint to legacy to avoid having it mounted by `zfs mount -a`:

```
zfs set mountpoint=legacy zroot/data/home
```

Ensure that it is in /etc/fstab so that `mount /home` will work:

```
/etc/fstab
zroot/data/home          /home           zfs             rw,xattr,posixacl,noauto
0 0
```

Alternatively, you can keep using ZFS mounts if you use both:

```
zfs set canmount=noauto zroot/data/home
zfs set org.openzfs.systemd:ignore=on zroot/data/home
```

The first will stop ZFS automatically mounting it, and the second systemd, but you will still be able to manually (or through the following scripts) mount it. If you have child datasets, `org.openzfs.systemd:ignore=on` will be inherited, but you will

need to set `canmount=noauto` on each as it is not inheritable, otherwise they will try to mount without a mountpoint.

On a single-user system, with only one `/home` volume having the same encryption password as the user's password, it can be decrypted at login as follows: first create `/usr/local/bin/mount-zfs-homedir`

```
/usr/local/bin/mount-zfs-homedir
#!/bin/bash
set -eu

# $PAM_USER will be the username of the user, you can use it for per-user home volumes.
HOME_VOLUME="zroot/data/home"

if [ "$(zfs get keystatus "${HOME_VOLUME}" -Ho value)" != "available" ]; then
  PASSWORD=$(cat -)
  zfs load-key "${HOME_VOLUME}" <<< "$PASSWORD" || continue
fi

# This will also mount any child datasets, unless they use a different key.
echo "$(zfs list -rHo name,keystatus,mounted "${HOME_VOLUME}")" | while IFS=$'\t' read
-r NAME KEYSTATUS MOUNTED; do
  if [ "${MOUNTED}" != "yes" ] && [ "${KEYSTATUS}" == "available" ]; then
    zfs mount "${NAME}" || true
  fi
done
```

do not forget to make it executable; then get PAM to run it by adding the following line to /etc/pam.d/system-auth:

```
/etc/pam.d/system-auth
auth       optional                    pam_exec.so
expose_authtok /usr/local/bin/mount-zfs-homedir
```

Now it will transparently decrypt and mount the /home volume when you log in anywhere: on the console, via ssh, etc.

SSH

A caveat is that since your `~/.ssh` directory is not mounted, if you log in via ssh, you must use password authentication the first time rather than relying on `~/.ssh/authorized_keys`.

If you do not wish to enable (insecure) password authentication, you can instead move `~/.ssh/authorized_keys` to a new location.
Make `/etc/ssh/user_config/` and inside it a folder for each user, owned by that user and with `700` permissions. Then move each user's `authorized_keys` into their respective folders, and edit the system sshd configuration:

```
/etc/ssh/sshd_config
AuthorizedKeysFile /etc/ssh/user_config/%u/authorized_keys
```

Then restart `sshd.service`. You can also optionally make a link for each user from `~/.ssh/authorized_keys` to the new location so users can still edit it as they are used to.

This will let you log in, but your home partition will not be mounted, and you will need to do so manually. There are multiple options to work around this:

SSH Key & Password when required

It is possible to set up PAM to only prompt for a password via SSH when it is necessary to decrypt your home partition. You will need to enable both `publickey` and `keyboard-interactive` authentication methods:

```
/etc/ssh/sshd_config
PubkeyAuthentication yes
KbdInteractiveAuthentication yes
AuthenticationMethods publickey,keyboard-interactive

## Example of excluding a certain user who does not have an encrypted home directory.
#Match User nohome
#  KbdInteractiveAuthentication no
#  AuthenticationMethods publickey
```

警告：Note the comma in `AuthenticationMethods publickey,keyboard-interactive`, this means that you need to do *both* authentication methods to log in with SSH. The very similar `AuthenticationMethods publickey keyboard-interactive` means you can do **either** to log in, which would let someone bypass your public key auth.

注意：You may ask why `keyboard-interactive` and not `password`? `password` is done client-side, so even if the auth is skipped, the user is still prompted and the password is just thrown away. With `keyboard-interactive` the user does not get prompted at all when we skip it.

This will mean it asks for the password after validating the key, but using PAM we can stop it asking for the password when not needed. We make a script that will fail when the key is not available to us:

```
/usr/local/bin/require-encrypted-homedir
#!/bin/bash
set -eu

HOME_VOLUME="zroot/data/home" # You can use $PAM_USER to use the username in the volume
for a per-user solution.

if [ "$(zfs get keystatus "${HOME_VOLUME}" -Ho value)" != "available" ]; then
  exit 27 # PAM_TRY_AGAIN
elif [[ "${SSH_AUTH_INFO_0:-""}" =~ ^"publickey " ]]; then
  exit 0
else
  # If this happens, it implies a configuration error: either you are allowing auth
without a public
  # key, or have enabled this in a non-SSH PAM service. Both are dangerous and this
should block it,
  # but if you see it, fix your configuration.
  exit 3 # PAM_SERVICE_ERR
fi
```

And make it executable.

Now we want to configure PAM to call this, and skip asking for the password if the script succeeds because we already have the key available. Add this line above the existing auth line(s) you want to skip (all of them unless you have something else set up) for the SSH service:

```
/etc/pam.d/sshd
auth sufficient pam_exec.so /usr/local/bin/require-encrypted-homedir
```

警告：This is for `/etc/pam.d/sshd` **not** `/etc/pam.d/system-auth` as above. You do not want local users without a public key to be able to skip the password. There a safeguard in the script against this, but still best to be careful.

注意：When using private keys, the auth step is skipped in PAM as the private key authentication is handled entirely by `sshd`. This means that the script we are adding here will never be run for private keys and they cannot be skipped, however, we still do a check for defence-in-depth to try and ensure a key has been checked.

With this, you will be prompted for a password only when the key is not loaded.

SSH Key & Password

A simpler option is to just enable both methods, meaning your key still gets checked, but then you have to type the password too, which will decrypt your home partition.

```
/etc/ssh/sshd_config
PubkeyAuthentication yes
PasswordAuthentication yes
AuthenticationMethods publickey,password
```

警告：Note the comma in `AuthenticationMethods publickey,password`, this means that you need to do *both* authentication methods to log in with SSH. The very similar `AuthenticationMethods publickey password` means you can do **either** to log in, which would let someone bypass your public key auth.

This works (and will not let anyone authenticate with *just* a password), but has the downside of requiring your password every time.

You can also specify something like:

```
AuthenticationMethods publickey password,publickey
```

This allows clients to either use either just a public key, or one and a password. Which the client will do will be based on the `PreferredAuthentications` option. `-o PreferredAuthentications=password,publickey` will ask for the password, while `-o PreferredAuthentications=publickey` will not. This is more manual than automated fallback, but has less moving parts, and avoids asking you every time if you prefer `publickey` by default (you can use host-specific options on clients to simplify setting these options).

交换卷

警告：

- 如果您的系统内存压力较大，则不管剩余多少交换空间可用，将 zvol 用作交换卷都可能会导致文件系统锁起。这个问题现在正在 OpenZFS issue #7734 中调查。
- zvol 上的交换空间不支持从休眠中唤醒，如果尝试从休眠中恢复将会导致存储池损坏。可能的解决方案见：https://github.com/openzfs/zfs/issues/260#issuecomment-758782144

ZFS 不允许使用交换文件，但您可以将一个 ZFS 卷 (ZVOL) 用作交换空间。需要注意的是必须将 ZVOL 的块大小设置为系统的 PAGESIZE，后者可以通过运行 `getconf PAGESIZE` 命令来获得（对于 x86_64 系统来说，其默认值为 4KiB）。关闭 ZVOL 上的写入缓存也可以让系统在低内存状态下更好运行。

创建一个 8 GiB 的 ZFS 卷：

```
# zfs create -V 8G -b $(getconf PAGESIZE) -o compression=zle \
```

```
             -o logbias=throughput -o sync=always\
             -o primarycache=metadata -o secondarycache=none \
             -o com.sun:auto-snapshot=false <pool>/swap
```

将其格式化为交换空间：

```
# mkswap -f /dev/zvol/<pool>/swap
# swapon /dev/zvol/<pool>/swap
```

要将其永久自动挂载，编辑 `/etc/fstab`。ZVOLs 支持垃圾回收，这对 ZFS 的块分配器有潜在帮助，同时当交换空间仍有剩余时有助于减少其他数据集上的磁盘碎片。

在 `/etc/fstab` 中添加如下行：

```
/dev/zvol/<pool>/swap none swap discard 0 0
```

访问控制列表（Access Control Lists，ACL）

要对数据集使用 ACL，请使用如下命令：

```
# zfs set acltype=posixacl <nameofzpool>/<nameofdataset>
# zfs set xattr=sa <nameofzpool>/<nameofdataset>
```

出于性能原因，建议配置 `xattr` 。

鉴于数据集会继承 ACL 参数，最好是对 zpool 启用 ACL。默认模式为 `restricted`，你可能会需要修改其设置：`aclinherit=passthrough` ；但要注意的是，`aclinherit` 不影响 POSIX ACL：

```
# zfs set aclinherit=passthrough <nameofzpool>
# zfs set acltype=posixacl <nameofzpool>
# zfs set xattr=sa <nameofzpool>
```

Databases

ZFS, unlike most other file systems, has a variable record size, or what is commonly referred to as a block size. By default, the recordsize on ZFS is 128KiB, which means it will dynamically allocate blocks of any size from 512B to 128KiB depending on the size of file being written. This can often help fragmentation and file access, at the cost that ZFS would have to allocate new 128KiB blocks each time only a few bytes are written to.

**本文或本章节的事实准确性存在争议。**

原因： At least MariaDB uses a default of 16Kib pages! Check your specific DBMS before setting this value.（在 Talk:ZFS 中讨论）

Most RDBMSes work in 8KiB-sized blocks by default. Although the block size is tunable for MySQL/MariaDB, PostgreSQL, and Oracle database, all three of them use an 8KiB block size *by default*. For both performance concerns and keeping snapshot differences to a minimum (for backup purposes, this is helpful), it is usually desirable to tune ZFS instead to accommodate the databases, using a command such as:

```
# zfs set recordsize=8K <pool>/postgres
```

These RDBMSes also tend to implement their own caching algorithm, often similar to ZFS's own ARC. In the interest of saving memory, it is best to simply disable ZFS's caching of the database's file data and let the database do its own job:

注意：L2ARC requires `primarycache` to function, because it is fed with data evicted from `primarycache`. If you intend to use the L2ARC, do not set the option below, otherwise no actual data will be cached on L2ARC.

```
# zfs set primarycache=metadata <pool>/postgres
```

ZFS uses the ZIL for crash recovery, but databases are often syncing their data files to the file system on their own transaction commits anyway. The end result of this is that ZFS will be committing data **twice** to the data disks, and it can severely impact performance. You can tell ZFS to prefer to not use the ZIL, and in which case, data is only committed to the file system once. However, doing so on non-solid state storage (e.g. HDDs) can result in decreased read performance due to fragmentation (OpenZFS Wiki) -- with mechanical hard drives, please consider using a dedicated SSD as ZIL rather than setting the option below. In addition, setting this for non-database file systems, or for pools with configured log devices, can also *negatively* impact the performance, so beware:

```
# zfs set logbias=throughput <pool>/postgres
```

These can also be done at file system creation time, for example:

```
# zfs create -o recordsize=8K \
             -o primarycache=metadata \
             -o mountpoint=/var/lib/postgres \
             -o logbias=throughput \
              <pool>/postgres
```

Please note: these kinds of tuning parameters are ideal for specialized applications like RDBMSes. You can easily *hurt* ZFS's performance by setting these on a general-purpose file system such as your /home directory.

/tmp

If you would like to use ZFS to store your /tmp directory, which may be useful for storing arbitrarily-large sets of files or simply keeping your RAM free of idle data, you can generally improve performance of certain applications writing to /tmp by disabling file system sync. This causes ZFS to ignore an application's sync requests (eg, with `fsync` or `O_SYNC`) and return immediately. While this has severe application-side data consistency consequences (never disable sync for a database!), files in /tmp are less likely to be important and affected. Please note this does *not* affect the integrity of ZFS itself, only the possibility that data an application expects on-disk may not have actually been written out following a crash.

```
# zfs set sync=disabled <pool>/tmp
```

Additionally, for security purposes, you may want to disable **setuid** and **devices** on the /tmp file system, which prevents some kinds of privilege-escalation attacks or the use of device nodes:

```
# zfs set setuid=off <pool>/tmp
# zfs set devices=off <pool>/tmp
```

Combining all of these for a create command would be as follows:

```
# zfs create -o setuid=off -o devices=off -o sync=disabled -o mountpoint=/tmp
<pool>/tmp
```

Please note, also, that if you want /tmp on ZFS, you will need to mask (disable) systemd's automatic tmpfs-backed /tmp (`tmp.mount`, else ZFS will be unable to mount your dataset at boot-time or import-time.

使用 ZFS Send 和 ZFS Recv 传输快照

通过搭配使用 `zfs send` 和 `zfs recv`，可以将 ZFS 快照传输到任意目标。该操作通过标准输出实现，可以将数据发送到任意文件、设备或网络目标，也可以在管道中加入其它程序对数据流进行中间操作。

以下为常见用例：

ZFS Send 基本用法

首先为一个 ZFS 文件系统创建快照：

```
# zfs snapshot zpool0/archive/books@snap
```

然后将快照发送到另一 zpool 上的新位置：

```
# zfs send -v zpool0/archive/books@snap | zfs recv zpool4/library
```

现在 `zpool0/archive/books@snap` 上的内容就被发送到了 `zpool4/library`。

提示：各命令行选项的具体信息请参考 man zfs-send 和 man zfs-recv。

传入/传出到文件

首先为一个 ZFS 文件系统创建快照：

```
# zfs snapshot zpool0/archive/books@snap
```

将快照写入到 gzip 文件：

```
# zfs send zpool0/archive/books@snap > /tmp/mybooks.gz
```

警告：如果想在发送时保留加密，需对 `zfs send` 使用 `-w` 选项。

从文件恢复快照：

```
# gzcat /tmp/mybooks.gz | zfs recv -F zpool0/archive/books
```

通过 SSH 发送

首先为一个 ZFS 文件系统创建快照：

```
# zfs snapshot zpool1/filestore@snap
```

下一步是通过管道将数据“发送”到运行“recv”的 ssh 会话：

```
# zfs send -v zpool1/filestore@snap | ssh $HOST zfs recv coldstore/backups
```

使用 `-v` 选项将会输出数据流的信息。如果你使用密码或密钥，会出现提示要求进行输入。

增量备份

You may wish update a previously sent ZFS filesystem without retransmitting all of the data over again. Alternatively, it may be necessary to keep a filesystem online during a lengthy transfer and it is now time to send writes that were made since the initial snapshot.

首先为一个 ZFS 文件系统创建快照：

```
# zfs snapshot zpool1/filestore@initial
```

下一步是通过管道将数据“发送”到运行“recv”的 ssh 会话：

```
# zfs send -v -R zpool1/filestore@initial | ssh $HOST zfs recv coldstore/backups
```

在写入更改后，再创建一个快照：

```
# zfs snapshot zpool1/filestore@snap2
```

接下来发送本地 zpool1/filestore@initial 和 zpool1/filestore@snap2 两个快照的区别，然后为远程文件系统 coldstore/backups 再创建一个快照：

```
# zfs send -v -i -R zpool1/filestore@initial | ssh $HOST zfs recv coldstore/backups
```

现在 zpool1/filestore 和 coldstore/backups 都存有 @initial 和 @snap2 两个快照。

你可能会想在远程主机上将最新快照作为当前活动文件系统：

```
# rollback coldstore/backups@snap2
```

调校

通用

可以使用参数进一步调整 ZFS 池和数据集。

注意：除配额和预留外，所有可设置的属性值都会从父数据集继承。

要检索当前 ZFS 池的参数状态，请执行以下操作：

```
# zfs get all <pool>
```

要检索指定数据集的参数状态，请执行以下操作：

```
# zfs get all <pool>/<dataset>
```

要禁用默认启用的访问时间功能（atime），请执行以下操作：

```
# zfs set atime=off <pool>
```

要禁用特定数据集的访问时间功能（atime），请执行以下操作：

```
# zfs set atime=off <pool>/<dataset>
```

除了完全关闭 atime 之外，您还可以使用 `relatime`。这为ZFS带来了默认的 ext4/XFS atime 语义，只有在修改或更改时间发生变化时，或者访问时间在过去24小时内没有变化时，才更新访问时间。这是 `atime=off` 和 `atime=on` 之间的折衷。该属性*只在* `atime` 为 `on` 时生效：

```
# zfs set atime=on <pool>
```

```
# zfs set relatime=on <pool>
```
压缩功能则是对数据的透明压缩。ZFS 支持数种不同的压缩算法，目前默认采用 lz4 。*gzip* 比较适合用于那些不频繁写入并且可压缩率较高的数据。请参考 OpenZFS Wiki 以获得更多信息。

要启用压缩，请执行：

```
# zfs set compression=on <pool>
```
若要将池和/或数据集的属性重置为默认状态，请使用 `zfs inherit`：

```
# zfs inherit -rS atime <pool>
# zfs inherit -rS atime <pool>/<dataset>
```
注意：使用 `-r` 标志将递归重置ZPool中的所有数据集。

Scrubbing

当 ZFS 在读取数据过程中检测到错误时，它会在可能时静默修复数据，写回到磁盘并记录日志，使得你可以获得存储池中错误的概览。ZFS 没有 fsck 一类的工具，但提供了称为 scrubbing 的特性。它会遍历存储池中的所有数据，并验证是否所有块都可被正常读取。

要对存储池执行 scrub：

```
# zpool scrub <pool>
```
要中断运行中的 scrub：

```
# zpool scrub -s <pool>
```
多久需要运行一次呢？

根据 Oracle 的博客文章 Disk Scrub - Why and When?:

: 这一问题对支持人员来说有难度，因为最贴切的回答是“看情况”。所以，在我给出一个较通用的回答前，有些可以用来创建更贴合你需求的答案的提示。

: - 你最旧的备份的有效期是多久？对数据执行 scrub 操作的频率因至少与你最旧备份的有效期相当，以确保回复点可用。

- 你通常多久会碰到一次磁盘故障？While the recruitment of a hot-spare disk invokes a "resilver" -- a targeted scrub of just the VDEV which lost a disk -- you should probably scrub at least as often as you experience disk failures on average in your specific environment.
- 你多久会读取一次磁盘上最旧的数据？你应偶尔进行一次 scrub，以防止旧数据在你不知道的情况下出现位腐坏。

: 如果针对上述任一问题的答案是“我不知道”，那最通用的回答是：你应至少每月对 zpool 执行一次 scrub 操作。这一周期对多数用例来说都较为合适，提供了足以在各

种高负载环境下完成运行的时间，并快于大型 zpools（192+ 磁盘）出现磁盘故障的时间。

根据 Aaron Toponce 的 ZFS Administration Guide，他建议对消费级磁盘每周执行一次 scrub。

根据服务或定时器运行

注意：从 OpenZFS 2.1.3 开始提供了每周/月运行的 systemd 定时器/服务。使用时需根据目标存储池 启用/启动 `zfs-scrub-weekly@*pool-to-scrub*.timer` 或 `zfs-scrub-monthly@*pool-to-scrub*.timer`。

可以使用 systemd 定时器/服务来自动对存储池执行 scrub。

要对特定存储池执行每月 scrubbing：

```
/etc/systemd/system/zfs-scrub@.timer
[Unit]
Description=Monthly zpool scrub on %i

[Timer]
OnCalendar=monthly
AccuracySec=1h
Persistent=true

[Install]
WantedBy=multi-user.target
/etc/systemd/system/zfs-scrub@.service
[Unit]
Description=zpool scrub on %i

[Service]
Nice=19
IOSchedulingClass=idle
KillSignal=SIGINT
ExecStart=/usr/bin/zpool scrub %i

[Install]
WantedBy=multi-user.target
```

启用/启动 `zfs-scrub@*pool-to-scrub*.timer` 单元以为特定 zpool 启用月度 scrubbing。

启用 TRIM

要检查你的 vdev 是否支持 TRIM，你可以通过 `-t` 为 `zpool status` 输出添加 TRIM 信息：

```
$ zpool status -t tank
pool: tank
 state: ONLINE
  scan: none requested
 config:

        NAME                                    STATE     READ WRITE CKSUM
        tank                                    ONLINE       0     0     0
          ata-ST31000524AS_5RP4SSNR-part1       ONLINE       0     0     0  (trim unsupported)
          ata-CT480BX500SSD1_2134A59B933D-part1  ONLINE       0     0     0  (untrimmed)
```

```
errors: No known data errors
```

ZFS 可以手动或通过 `autotrim` 定时对支持的设备进行 TRIM。

对 zpool 手动进行 TRIM：

```
 # zpool trim <zpool>
```

为数据池中所有支持的 vdev 启用自动 TRIM：

```
 # zpool set autotrim=on <zpool>
```

注意：因为自动 TRIM 与 `zpool trim` 的操作有所不同，你可能会想偶尔手动执行 TRIM。

要使用 systemd 定时器/服务对特定存储池每月执行一次完整的 `zpool trim`：

```
/etc/systemd/system/zfs-trim@.timer
[Unit]
Description=Monthly zpool trim on %i

[Timer]
OnCalendar=monthly
AccuracySec=1h
Persistent=true

[Install]
WantedBy=multi-user.target
/etc/systemd/system/zfs-trim@.service
[Unit]
Description=zpool trim on %i
Documentation=man:zpool-trim(8)
Requires=zfs.target
After=zfs.target
ConditionACPower=true
ConditionPathIsDirectory=/sys/module/zfs

[Service]
Nice=19
IOSchedulingClass=idle
KillSignal=SIGINT
ExecStart=/bin/sh -c '\
if /usr/bin/zpool status %i | grep "trimming"; then\
exec /usr/bin/zpool wait -t trim %i;\
else exec /usr/bin/zpool trim -w %i; fi'
ExecStop=-/bin/sh -c '/usr/bin/zpool trim -s %i 2>/dev/null || true'

[Install]
WantedBy=multi-user.target
```

启用/启动 `zfs-trim@*pool-to-trim*.timer` 单元以对特定存储池启用 TRIM。

SSD Caching

If your pool has no configured log devices, ZFS reserves space on the pool's data disks for its intent log (the ZIL, also called SLOG). If your data disks are slow (e.g. HDD) it is highly recommended to configure the ZIL on solid state drives for better write performance and also to consider a layer 2 adaptive replacement cache (L2ARC). The process to add them is very similar to adding a new VDEV.

All of the below references to device-id are the IDs from `/dev/disk/by-id/*`.

ZIL

To add a mirrored ZIL:

```
# zpool add <pool> log mirror <device-id-1> <device-id-2>
```
Or to add a single device ZIL:

```
# zpool add <pool> log <device-id>
```
Because the ZIL device stores data that has not been written to the pool, it is important to use devices that can finish writes when power is lost. It is also important to use redundancy, since a device failure can cause data loss. In addition, the ZIL is only used for sync writes, so may not provide any performance improvement when your data drives are as fast as your ZIL drive(s).

L2ARC

使用如下命令添加 L2ARC：

```
# zpool add <pool> cache <device-id>
```
L2ARC 是只读缓存，所以不需要任何冗余。从 ZFS 2.0.0 版本开始，L2ARC 可以在重启后保留。

L2ARC 通常只在热数据量比系统内存更大，但又小到能放入 L2ARC 的情况下有用。L2ARC 由系统内存中的 ARC 进行索引，每条记录（默认为 128KiB）消耗 70 字节内存。所以，对应的内存用量可用以下公式计算：

```
(L2ARC 大小) / (记录大小) * 70 字节
```
因此，由于 L2ARC 占用了 ARC 的内存空间，在某些情况下它会造成存储性能的下降。

ZVOLs

ZFS volumes (ZVOLs) can suffer from the same block size-related issues as RDBMSes, but it is worth noting that the default recordsize for ZVOLs is 8 KiB already. If possible, it is best to align any partitions contained in a ZVOL to your recordsize (current versions of fdisk and gdisk by default automatically align at 1MiB segments, which works), and file system block sizes to the same size. Other than this, you might tweak the **recordsize** to accommodate the data inside the ZVOL as necessary (though 8 KiB tends to be a good value for most file systems, even when using 4 KiB blocks on that level).

RAIDZ and Advanced Format physical disks

Each block of a ZVOL gets its own parity disks, and if you have physical media with logical block sizes of 4096B, 8192B, or so on, the parity needs to be stored in whole physical blocks, and this can drastically increase the space requirements of a ZVOL, requiring 2× or more physical storage capacity than the ZVOL's logical capacity. Setting the **recordsize** to 16k or 32k can help reduce this footprint drastically.

See OpenZFS issue #1807 for details.

I/O Scheduler

While ZFS is expected to work well with modern schedulers including, `mq-deadline`, and `none`, experimenting with manually setting the I/O scheduler on ZFS

disks may yield performance gains. The ZFS recomendation is "\[...\] users leave the default scheduler “unless you’re encountering a specific problem, or have clearly measured a performance improvement for your workload”"

排障

Creating a zpool fails

If the following error occurs then it can be fixed.

```
# the kernel failed to rescan the partition table: 16
# cannot label 'sdc': try using parted(8) and then provide a specific slice: -1
```

One reason this can occur is because ZFS expects pool creation to take less than 1 second. This is a reasonable assumption under ordinary conditions, but in many situations it may take longer. Each drive will need to be cleared again before another attempt can be made.

```
# parted /dev/sda rm 1
# parted /dev/sda rm 1
# dd if=/dev/zero of=/dev/sdb bs=512 count=1
# zpool labelclear /dev/sda
```

A brute force creation can be attempted over and over again, and with some luck the ZPool creation will take less than 1 second. One cause for creation slowdown can be slow burst read writes on a drive. By reading from the disk in parallell to ZPool creation, it may be possible to increase burst speeds.

```
# dd if=/dev/sda of=/dev/null
```

This can be done with multiple drives by saving the above command for each drive to a file on separate lines and running

```
# cat $FILE | parallel
```

Then run ZPool creation at the same time.

ZFS is using too much RAM

By default, ZFS caches file operations (ARC) using up to half of available system memory on the host. To adjust the ARC size, add the following to the Kernel parameters list:

```
zfs.zfs_arc_max=536870912 # (for 512MiB)
```

In case that the default value of `zfs_arc_min` (1/32 of system memory) is higher than the specified `zfs_arc_max` it is needed to add also the following to the Kernel parameters list:

```
zfs.zfs_arc_min=268435456 # (for 256MiB, needs to be lower than zfs.zfs_arc_max)
```

You may also want to increase `zfs_arc_sys_free` instead (in this example to 8GiB):

```
# echo $((8*1024**3)) > /sys/module/zfs/parameters/zfs_arc_sys_free
```

For a more detailed description, as well as other configuration options, see Gentoo:ZFS#ARC.

ZFS should release ARC as applications reserve more RAM, but some applications still get confused, and reported free RAM is always wrong. But in case all your

applications work as intended and you have no problems, there is no need to change ARC settings.

No hostid found

An error that occurs at boot with the following lines appearing before initscript output:

```
ZFS: No hostid found on kernel command line or /etc/hostid.
```

This warning occurs because the ZFS module does not have access to the spl hosted. There are two solutions, for this. Either place the spl hostid in the 内核参数 in the boot loader. For example, adding `spl.spl_hostid=0x00bab10c`.

The other solution is to make sure that there is a hostid in `/etc/hostid`, and then regenerate the initramfs image. Which will copy the hostid into the initramfs image.

Pool cannot be found while booting from SAS/SCSI devices

In case you are booting a SAS/SCSI based, you might occassionally get boot problems where the pool you are trying to boot from cannot be found. A likely reason for this is that your devices are initialized too late into the process. That means that zfs cannot find any devices at the time when it tries to assemble your pool.

In this case you should force the scsi driver to wait for devices to come online before continuing. You can do this by putting this into `/etc/modprobe.d/zfs.conf`:

```
/etc/modprobe.d/zfs.conf
options scsi_mod scan=sync
```

Afterwards, regenerate the initramfs.

This works because the zfs hook will copy the file at `/etc/modprobe.d/zfs.conf` into the initcpio which will then be used at build time.

On boot the zfs pool does not mount stating: "pool may be in use from other system"

Unexported pool

If the new installation does not boot because the zpool cannot be imported, chroot into the installation and properly export the zpool. See #Emergency chroot repair with archzfs.

Once inside the chroot environment, load the ZFS module and force import the zpool,

```
# zpool import -a -f
```

now export the pool:

```
# zpool export <pool>
```

To see the available pools, use,

```
# zpool status
```

It is necessary to export a pool because of the way ZFS uses the hostid to track the system the zpool was created on. The hostid is generated partly based on the network setup. During the installation in the archiso the network configuration could be different generating a different hostid than the one contained in the new installation. Once the zfs filesystem is exported and then re-imported in the new

installation, the hostid is reset. See Re: Howto zpool import/export automatically? - msg#00227.

If ZFS complains about "pool may be in use" after every reboot, properly export pool as described above, and then regenerate the initramfs in normally booted system.

Incorrect hostid

Double check that the pool is properly exported. Exporting the zpool clears the hostid marking the ownership. So during the first boot the zpool should mount correctly. If it does not there is some other problem.

Reboot again, if the zfs pool refuses to mount it means the hostid is not yet correctly set in the early boot phase and it confuses zfs. Manually tell zfs the correct number, once the hostid is coherent across the reboots the zpool will mount correctly.

Boot using zfs_force and write down the hostid. This one is just an example.

```
$ hostid
0a0af0f8
```

This number have to be added to the 内核参数 as `spl.spl_hostid=0x0a0af0f8`. Another solution is writing the hostid inside the initram image, see the installation guide\[损坏的链接：无效的章节\] explanation about this.

Users can always ignore the check adding `zfs_force=1` in the 内核参数, but it is not advisable as a permanent solution.

Devices have different sector alignment

Once a drive has become faulted it should be replaced A.S.A.P. with an identical drive.

```
# zpool replace bigdata ata-ST3000DM001-9YN166_S1F0KDGY ata-ST3000DM001-1CH166_W1F478BD -f
```

but in this instance, the following error is produced:

```
cannot replace ata-ST3000DM001-9YN166_S1F0KDGY with ata-ST3000DM001-1CH166_W1F478BD: devices have different sector alignment
```

ZFS uses the ashift option to adjust for physical block size. When replacing the faulted disk, ZFS is attempting to use `ashift=12`, but the faulted disk is using a different ashift (probably `ashift=9`) and this causes the resulting error.

For Advanced Format disks with 4 KiB block size, an `ashift` of `12` is recommended for best performance. See OpenZFS FAQ: Performance Considerations and ZFS and Advanced Format disks.

Use zdb to find the ashift of the zpool: `zdb` , then use the `-o` argument to set the ashift of the replacement drive:

```
# zpool replace bigdata ata-ST3000DM001-9YN166_S1F0KDGY ata-ST3000DM001-1CH166_W1F478BD -o ashift=9 -f
```

Check the zpool status for confirmation:

```
# zpool status -v
```

```
pool: bigdata
state: DEGRADED
status: One or more devices is currently being resilvered.  The pool will
        continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
scan: resilver in progress since Mon Jun 16 11:16:28 2014
    10.3G scanned out of 5.90T at 81.7M/s, 20h59m to go
    2.57G resilvered, 0.17% done
config:

        NAME                                    STATE     READ WRITE CKSUM
        bigdata                                 DEGRADED     0     0     0
        raidz1-0                                DEGRADED     0     0     0
            replacing-0                         OFFLINE      0     0     0
            ata-ST3000DM001-9YN166_S1F0KDGY     OFFLINE      0     0     0
            ata-ST3000DM001-1CH166_W1F478BD     ONLINE       0     0     0
(resilvering)
            ata-ST3000DM001-9YN166_S1F0JKRR     ONLINE       0     0     0
            ata-ST3000DM001-9YN166_S1F0KBP8     ONLINE       0     0     0
            ata-ST3000DM001-9YN166_S1F0JTM1     ONLINE       0     0     0

errors: No known data errors
```

Pool resilvering stuck/restarting/slow?

According to ZFS issue #840, this is a known issue since 2012 with ZFS-ZED which causes the resilvering process to constantly restart, sometimes get stuck and be generally slow for some hardware. The simplest mitigation is to stop zfs-zed.service until the resilver completes.

Fix slow boot caused by failed import of unavailable pools in the initramfs zpool.cache

Your boot time can be significantly impacted if you update your intitramfs (eg when doing a kernel update) when you have additional but non-permanently attached pools imported because these pools will get added to your initramfs zpool.cache and ZFS will attempt to import these extra pools on every boot, regardless of whether you have exported it and removed it from your regular zpool.cache.

If you notice ZFS trying to import unavailable pools at boot, first run:

```
$ zdb -C
```

To check your zpool.cache for pools you do not want imported at boot. If this command is showing (a) additional, currently unavailable pool(s), run:

```
# zpool set cachefile=/etc/zfs/zpool.cache zroot
```

To clear the zpool.cache of any pools other than the pool named zroot. Sometimes there is no need to refresh your zpool.cache, but instead all you need to do is regenerate the initramfs.

ZFS Command History

ZFS logs changes to a pool's structure natively as a log of executed commands in a ring buffer (which cannot be turned off). The log may be helpful when restoring a degraded or failed pool.

```
# zpool history zpool
History for 'zpool':
2023-02-19.16:28:44 zpool create zpool
raidz1 /scratch/disk_1.img /scratch/disk_2.img /scratch/disk_3.img
2023-02-19.16:31:29 zfs set compression=lz4 zpool
```

```
2023-02-19.16:41:45 zpool scrub zpool
2023-02-19.17:00:57 zpool replace zpool /scratch/disk_1.img /scratch/bigger_disk_1.img
2023-02-19.17:01:34 zpool scrub zpool
2023-02-19.17:01:42 zpool replace zpool /scratch/disk_2.img /scratch/bigger_disk_2.img
2023-02-19.17:01:46 zpool replace zpool /scratch/disk_3.img /scratch/bigger_disk_3.img
```

小技巧

创建带有 ZFS 支持的 Archiso 映像

**此页面或章节适合移动到 在自定义的 archiso 安装介质中启用 ZFS 模块。**

附注： The target links here for its Arch instructions, but is the natural context.（在 Talk:ZFS 讨论）

创建 Arch Linux live CD/DVD/USB 映像的具体步骤在 Archiso 中已有描述。如需在映像中加入 ZFS 支持，可以选择手动构建 AUR 上的 PKGBUILDs，或是在映像中加入非官方用户仓库中的预构建包。

使用 AUR 自行构建 ZFS 包

参考正常流程自行构建你需要的 ZFS 包。如果你不确定需要哪个包，zfs-dkmsAUR 和 zfs-utilsAUR 可以支持你在 Archiso 映像上做出的多数改动。下一步需要创建一个本地仓库，并将仓库添加到新配置的 Pacman 配置文件中。

将构建出的包添加到要安装的包列表中。下面的例子假设你仅想安装 zfs-dkmsAUR 和 zfs-utilsAUR 包：

```
packages.x86_64
...
zfs-dkms
zfs-utils
```

如果你添加了任何 DKMS 包，请确保你同时添加了 ISO 所用内核对应的头文件包（默认内核对应为 linux-headers包）。

使用 archzfs 非官方用户仓库

将 archzfs 非官方用户仓库添加到新 Archiso 配置中的 `pacman.conf` 文件中。

将 `archzfs-linux` 软件包组添加到要安装的软件包包列表中（`archzfs` 仓库提供的包仅支持 x86_64 架构）：

```
packages.x86_64
...
archzfs-linux
```

注意：如果你稍后在运行 modprobe zfs 的过程中出现报错，需在 packages.x86_64 中加入 linux-headers 包。

收尾

无论你使用了哪种方法，最后都需要构建 ISO 映像。

自动快照

zrepl

zreplAUR 包提供了一个 ZFS 自动复制服务，可被用作类似 snapper 的快照服务。

关于配置 zrepl 守护进程的详细方法请参考 zrepl 文档，配置文件位于 `/etc/zrepl/zrepl.yml`。完成后，先使用 `zrepl configcheck` 检查配置文件是否有语法问题，再启用 `zrepl.service` 服务。

sanoid

sanoidAUR 是一个策略驱动的快照工具，它还提供了 `syncoid`，可用于复制快照。它也附带了 systemd 服务和定时器。

Sanoid 只会清理本地系统的快照，如果要清理远程系统的快照，需要在远程系统上使用 prune 选项运行 sanoid。可以使用 `--prune-snapshots` 命令行选项，也可以将 `--cron` 命令行选项搭配 `autoprune = yes` 和 `autosnap = no` 配置选项使用。

zfs-auto-snapshot

注意： zfs-auto-snapshot-gitAUR 从 2019 年开始就没有被更新过了，而且功能上比较受限。建议使用如 zreplAUR 等更新的工具。

The zfs-auto-snapshot-gitAUR 包提供了一个 shell 脚本，可以自动为所有 ZFS 数据集生成并管理由日期和标签（如每小时，每日等）命名的快照。该包还会安装每十五分钟、每小时、每天、每周、每月进行快照的 cron 任务。你还可以调整 `--keep parameter` 选项来设定要保留快照的时长（每月快照的脚本默认将数据保留一年）。

如果不想对某个数据集进行快照，可以对其设置 `com.sun:auto-snapshot=false`。如果不想进行月度快照，还可以通过标签进行更精细化的配置：`com.sun:auto-snapshot:monthly=false`。

注意：在 scrubbing 期间，zfs-auto-snapshot-git 不会创建快照。你可以修改提供的 systemd 单元文件，移除 `ExecStart` 中的 `--skip-scrub` 选项来绕过该限制，但不清楚是否有任何后果，有知道的请协助添加相关内容。

安装该软件包后，启用并启动你想要的定时器（`zfs-auto-snapshot-{frequent,daily,weekly,monthly}.timer`）。

创建共享

ZFS 支持创建 NFS 或 SMB 共享。

NFS

首先，确保系统已经安装并配置了 NFS。注意：无需编辑 `/etc/exports`。对于 NFS 共享，确保已经启动 `nfs-server.service` 和 `zfs-share.service`。

要将存储池共享到网络：

```
# zfs set sharenfs=on 存储池名
```

要将数据集共享到网络：

```
# zfs set sharenfs=on 存储池名/数据集名
```

要为特定 IP 段启用读写权限：

```
# zfs set sharenfs="rw=@192.168.1.100/24,rw=@10.0.0.0/24" 存储池名/数据集名
```

要确认数据集是否已成功导出：

```
# showmount -e `hostname`
Export list for hostname:
/path/of/dataset 192.168.1.100/24
```

要确认当前导出状态的详细信息：

```
# exportfs -v
/path/of/dataset

192.168.1.100/24(sync,wdelay,hide,no_subtree_check,mountpoint,sec=sys,rw,secure,no_root_squash,no_all_squash)
```

要通过 ZFS 查看当前 NFS 共享列表：

```
# zfs get sharenfs
```

SMB

注意： SMB functionality is very limited. The usershare path must be `/var/lib/samba/usershares` and the only supported sharesmb options are `on` and `off`. Enabling guest access via `sharesmb=guest_ok=y` is not supported.

When sharing through SMB, using `usershares` in `/etc/samba/smb.conf` will allow ZFS to setup and create the shares. See Samba#Enable Usershares for details.

```
/etc/samba/smb.conf
[global]
    usershare path = /var/lib/samba/usershares
    usershare max shares = 100
    usershare allow guests = yes
    usershare owner only = no
```

Create and set permissions on the user directory as root

```
# mkdir /var/lib/samba/usershares
# chmod +t /var/lib/samba/usershares
```

To make a pool available on the network:

```
# zfs set sharesmb=on nameofzpool
```

To make a dataset available on the network:

```
# zfs set sharesmb=on nameofzpool/nameofdataset
```

To check if the dataset is exported successfully:

```
# smbclient -L localhost -U%
```

```
        Sharename       Type      Comment
        ---------       ----      -------
        IPC$            IPC       IPC Service (SMB Server Name)
        nameofzpool_nameofdataset        Disk      Comment: path/of/dataset
SMB1 disabled -- no workgroup available
```

To view the current SMB share list by ZFS:

```
# zfs get sharesmb
```

Encryption in ZFS using dm-crypt

Before OpenZFS version 0.8.0, ZFS did not support encryption directly (See #Native encryption). Instead, zpools can be created on dm-crypt block devices. Since the zpool is created on the plain-text abstraction, it is possible to have the data encrypted while having all the advantages of ZFS like deduplication, compression, and data robustness. Furthermore, utilizing dm-crypt will encrypt the zpools metadata, which the native encryption can inherently not provide.

dm-crypt, possibly via LUKS, creates devices in `/dev/mapper` and their name is fixed. So you just need to change `zpool create` commands to point to that names. The idea is configuring the system to create the `/dev/mapper` block devices and import the zpools from there. Since zpools can be created in multiple devices (raid, mirroring, striping, ...), it is important all the devices are encrypted otherwise the protection might be partially lost.

For example, an encrypted zpool can be created using plain dm-crypt (without LUKS) with:

```
# cryptsetup open --type=plain --hash=sha256 --cipher=aes-xts-plain64 --offset=0 \
            --key-file=/dev/sdZ --key-size=512 /dev/sdX enc
# zpool create zroot /dev/mapper/enc
```

In the case of a root filesystem pool, the `mkinitcpio.conf` HOOKS line will enable the keyboard for the password, create the devices, and load the pools. It will contain something like:

```
HOOKS=(... keyboard encrypt zfs ...)
```

Since the `/dev/mapper/enc` name is fixed no import errors will occur.

Creating encrypted zpools works fine. But if you need encrypted directories, for example to protect your users' homes, ZFS loses some functionality.

ZFS will see the encrypted data, not the plain-text abstraction, so compression and deduplication will not work. The reason is that encrypted data has always high entropy making compression ineffective and even from the same input you get different output (thanks to salting) making deduplication impossible. To reduce the unnecessary overhead it is possible to create a sub-filesystem for each encrypted directory and use eCryptfs on it.

For example to have an encrypted home: (the two passwords, encryption and login, must be the same)

```
# zfs create -o compression=off -o dedup=off -o mountpoint=/home/<username> <zpool>/
<username>
# useradd -m <username>
# passwd <username>
```

```
# ecryptfs-migrate-home -u <username>
<log in user and complete the procedure with ecryptfs-unwrap-passphrase>
```

Emergency chroot repair with archzfs

To get into the ZFS filesystem from live system for maintenance, there are two options:

1. Build custom archiso with ZFS as described in #Create an Archiso image with ZFS support.
2. Boot the latest official archiso and bring up the network. Then enable archzfs repository inside the live system as usual, sync the pacman package database and install the *archzfs-archiso-linux* package.

To start the recovery, load the ZFS kernel modules:

```
# modprobe zfs
```

Import the pool:

```
# zpool import -a -R /mnt
```

Mount the boot partition and EFI system partition (if any):

```
# mount /dev/sda2 /mnt/boot
# mount /dev/sda1 /mnt/efi
```

Chroot into the ZFS filesystem:

```
# arch-chroot /mnt /bin/bash
```

Check the kernel version:

```
# pacman -Qi linux
# uname -r
```

uname will show the kernel version of the archiso. If they are different, run depmod (in the chroot) with the correct kernel version of the chroot installation:

```
# depmod -a 3.6.9-1-ARCH (version gathered from pacman -Qi linux but using the matching kernel modules directory name under the chroot's /lib/modules)
```

This will load the correct kernel modules for the kernel version installed in the chroot installation.

Regenerate the initramfs. There should be no errors.

Bind mount

Here a bind mount from /mnt/zfspool to /srv/nfs4/music is created. The configuration ensures that the zfs pool is ready before the bind mount is created.

fstab

See systemd.mount(5) for more information on how systemd converts fstab into mount unit files with systemd-fstab-generator(8).

```
/etc/fstab
/mnt/zfspool         /srv/nfs4/music             none    bind,defaults,nofail,x-systemd.requires=zfs-mount.service 0 0
```

Monitoring / Mailing on Events

See ZED: The ZFS Event Daemon for more information.

An email forwarder, such as S-nail, is required to accomplish this. Test it to be sure it is working correctly.

Uncomment the following in the configuration file:

```
/etc/zfs/zed.d/zed.rc
 ZED_EMAIL_ADDR="root"
 ZED_EMAIL_PROG="mailx"
 ZED_NOTIFY_VERBOSE=0
 ZED_EMAIL_OPTS="-s '@SUBJECT@' @ADDRESS@"
```

Update 'root' in `ZED_EMAIL_ADDR="root"` to the email address you want to receive notifications at.

If you are keeping your mailrc in your home directory, you can tell mail to get it from there by setting `MAILRC`:

```
/etc/zfs/zed.d/zed.rc
export MAILRC=/home/<user>/.mailrc
```

This works because ZED sources this file, so `mailx` sees this environment variable.

If you want to receive an email no matter the state of your pool, you will want to set `ZED_NOTIFY_VERBOSE=1`. You will need to do this temporary to test.

Start and enable `zfs-zed.service`.

With `ZED_NOTIFY_VERBOSE=1`, you can test by running a scrub as root: `zpool scrub <pool-name>`.

Wrap shell commands in pre & post snapshots

Since it is so cheap to make a snapshot, we can use this as a measure of security for sensitive commands such as system and package upgrades. If we make a snapshot before, and one after, we can later diff these snapshots to find out what changed on the filesystem after the command executed. Furthermore we can also rollback in case the outcome was not desired.

znp

E.g.:

```
# zfs snapshot -r zroot@pre
# pacman -Syu
# zfs snapshot -r zroot@post
# zfs diff zroot@pre zroot@post
# zfs rollback zroot@pre
```

A utility that automates the creation of pre and post snapshots around a shell command is znp.

E.g.:

```
# znp pacman -Syu
# znp find / -name "something*" -delete
```

and you would get snapshots created before and after the supplied command, and also output of the commands logged to file for future reference so we know what command created the diff seen in a pair of pre/post snapshots.

Remote unlocking of ZFS encrypted root

As of PR #261, `archzfs` supports SSH unlocking of natively-encrypted ZFS datasets. This section describes how to use this feature, and is largely based on dm-crypt/特殊应用#基于BusyBox的initramfs（使用mkinitcpio）构建.

1. Install mkinitcpio-netconf包 to provide hooks for setting up early user space networking.
2. Choose an SSH server to use in early user space. The options are mkinitcpio-tinyssh包 or mkinitcpio-dropbear包, and are mutually exclusive.
3. If using mkinitcpio-tinyssh包, it is also recommended to install tinyssh包 or tinyssh-convert-gitAUR. This tool converts an existing OpenSSH hostkey to the TinySSH key format, preserving the key fingerprint and avoiding connection warnings. The TinySSH and Dropbear mkinitcpio install scripts will automatically convert existing hostkeys when generating a new initcpio image.
4. Decide whether to use an existing OpenSSH key or generate a new one (recommended) for the host that will be connecting to and unlocking the encrypted ZFS machine. Copy the public key into `/etc/tinyssh/root_key` or `/etc/dropbear/root_key`. When generating the initcpio image, this file will be added to `authorized_keys` for the root user and is only valid in the initrd environment.
5. Add the `ip=` 内核参数 to your boot loader configuration. The `ip` string is highly configurable. A simple DHCP example is shown below.

```
ip=:::::eth0:dhcp
```

1. Edit `/etc/mkinitcpio.conf` to include the `netconf`, `dropbear` or `tinyssh`, and `zfsencryptssh` hooks before the `zfs` hook:

```
HOOKS=(... netconf <tinyssh>|<dropbear> zfsencryptssh zfs ...)
```

1. Regenerate the initramfs.
2. Reboot and try it out!

Changing the SSH server port

By default, mkinitcpio-tinyssh包 and mkinitcpio-dropbear包 listen on port `22`. You may wish to change this.

For **TinySSH**, copy `/usr/lib/initcpio/hooks/tinyssh` to `/etc/initcpio/hooks/tinyssh`, and find/modify the following line in the `run_hook()` function:

```
/etc/initcpio/hooks/tinyssh
/usr/bin/tcpserver -HRDl0 0.0.0.0 <new_port> /usr/sbin/tinysshd -
v /etc/tinyssh/sshkeydir &
```

For **Dropbear**, copy `/usr/lib/initcpio/hooks/dropbear` to `/etc/initcpio/hooks/dropbear`, and find/modify the following line in the `run_hook()` function:

```
/etc/initcpio/hooks/tinyssh
```

```
/usr/sbin/dropbear -E -s -j -k -p <new_port>
```

Regenerate the initramfs.

Unlocking from a Windows machine using PuTTY/Plink

First, we need to use `puttygen.exe` to import and convert the OpenSSH key generated earlier into PuTTY's *.ppk* private key format. We will call it `zfs_unlock.ppk` for this example.

The mkinitcpio-netconf process above does not setup a shell (nor do we need need one). However, because there is no shell, PuTTY will immediately close after a successful connection. This can be disabled in the PuTTY SSH configuration (*Connection > SSH > \[X\] Do not start a shell or command at all*), but it still does not allow us to see stdout or enter the encryption passphrase. Instead, we use `plink.exe` with the following parameters:

```
plink.exe -ssh -l root -i c:\path\to\zfs_unlock.ppk <hostname>
```

The plink command can be put into a batch script for ease of use.

Enabling bclone support

To use `cp --reflink` and other commands needing bclone support, it is necessary to upgrade the *feature flags* if coming from a version prior to 2.2.2. This will allow the pool to have support for bclone. This is done with `zpool upgrade`, if the status of the pool show this is possible.

It is also required to enable a module parameter, otherwise userspace apps will not be able to use this feature. You can do this by putting this into `/etc/modprobe.d/zfs.conf`:

```
/etc/modprobe.d/zfs.conf
options zfs zfs_bclone_enabled=1
```

Check that is working, and how much space is being saved with the command: `zpool get all POOLNAME | grep clon`

参考

- Aaron Toponce's 17-part blog on ZFS
- OpenZFS releases
- OpenZFS FAQ
- FreeBSD Handbook - The Z File System
- Oracle Solaris ZFS Administration Guide
- ZFS Best Practices Guide
- ZFS Troubleshooting Guide
- How Pingdom uses ZFS to back up 5TB of MySQL data every day
- Tutorial on adding the modules to a custom kernel
- How to create cross platform ZFS disks under Linux
- How-To: Using ZFS Encryption at Rest in OpenZFS (ZFS on Linux, ZFS on FreeBSD, …)

- Archzfs iso download page: Frequently updated and downloadable archzfs linux iso with full OpenZFS support since 2016

内核

内核这一层要解决的是换成自己编译的版本、让模块按需加载、让启动参数与 initramfs 配合。《内核》《内核模块》《内核参数》必读，依次解决内核版本、模块行为与启动方式；编译路线在《内核/Arch 构建系统》与《内核/传统编译》中择一。《DKMS》《内核模块打包指南》用于树外模块；Zram、Zswap、SystemTap、热补丁、Linux-ck、内核级显示模式设置、Sysctl、PAM 均可缓读。

坑是改完参数忘了重建 initramfs、以为写进配置就生效；两条编译路线来回切换，内核与模块版本错位。

2.1　内核热补丁

难度 入门 | **篇幅** 3.2 KB | **所属** 内核

本节将讲到

- kpatch
- kGraft
- 参考

内核热补丁 (Kernel Live Patching, KLP) 允许在不重启整个操作系统的前提下进行内核的快速修复。从 4.0 版本开始，这个功能被加入内核 ，用户可以配置他们的的内核来启用这项功能。通常来讲，内核热补丁通过以下的步骤实现：

1. 获取正在运行的源代码树
2. 准备针对内核的补丁
3. 使用一些工具(随后)来修正并应用补丁

在内核热补丁被官方支持前，一些项目提供了热补丁工具。如甲骨文的 ksplice，SuSE 的 #kGraft，和红帽的 #kpatch。他们用不同的方法实现了内核热补丁。而内核加入的最基础的功能是从 kGraft 和 kpatch 派生出来的。

kpatch

安装

从 kpatchAUR 安装一个适当的内核，和 kpatch-gitAUR 对应的用户界面工具。

你也可以通过启用`CONFIG_LIVEPATCH`, `CONFIG_DEBUG_INFO`, 和 `CONFIG_KALLSYMS`，来手动编译支持kpatch的内核。

注意：安装特定的内核后需要更新 启动加载器。

使用方法

两个包都安装好、并重启后，你可以执行

```
$ export ROOTDIR=some/dir/aur/linux-kpatch/src/linux-x-y
$ cd $ROOTDIR
```

假设你已经完成了一些修改，并在工作目录下生成了一个*some.patch*补丁 (是执行`makepkg -o`后针对您自己的源代码树的补丁，而不是针对*x.y*版本的原始内核补丁)。启动kpatch实用工具：

```
$ kpatch-build -s $(pwd) -v $(pwd)/vmlinux some.patch
```

这个命令涉及原始的和修改后的内核的编译，因此可能会多花一些时间。编译结束后，同目录下会出现一个*kpatch-some.ko*模块。现在，

```
# insmod kpatch-some.ko
```

就可以为内核打上补丁了。

更多信息，请查阅手册页或GitHub存储库。

kGraft

KGraft没有在Arch linux环境下经过测试。

参考

- 热补丁的内核文档
- wikipedia:Kpatch
- wikipedia:KGraft
- wikipedia:Ksplice

2.2 Linux-ck

难度 入门 | **篇幅** 3.5 KB | **所属** 内核

本节将讲到

- 一般包细节
- 关于 MuQSS 的更多信息
- 在 Linux-ck 中使用树外模块
- 另请参见

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： No updates since 2020-08, English page has seen a large update since.（在 Talk:Linux-ck# 中讨论）

一般包细节

Linux-ckAUR 是 AUR 和用户预编译包中都可用的软件包，它允许用户运行用 Con Kolivas 的 ck 补丁集修补的内核和标头设置，包括名为 MuQSS 的 CPU 调度程序（*Multiple Queue Skiplist Scheduler*，发音为 *mux*，代替了他先前的著作 Brain Fuck Scheduler（BFS）。许多 Arch Linux 用户选择此内核是因为其在任何负载情况下均具有出色的桌面交互性和响应能力。

CK 补丁集旨在用于台式机/笔记本电脑，但不适用于服务器。它提供了低延迟环境，并且非常适合 16 个或更少的 CPU。为了能够引导到内核，必须适当配置启动加载器。

发行周期

Linux-ck 大致遵循官方 Arch 内核的发布周期，但不仅如此。以下是新软件包发布的要求：

- CK 补丁集与当前内核版本兼容

长期支持 (Long-Term Support) (LTS) CK 版本

除了 linux-ckAUR 软件包之外，还有 LTS 内核版本，其中包括上述补丁集和先前提到的修改：

- linux-lts-ckAUR\[损坏的链接：package not found\] - 使用 CK 补丁集修补的当前 Arch Linux LTS 内核

注意：该软件包由 vishwin 维护，因此预编译版本将不会出现在非官方的ck仓库中。

关于 MuQSS 的更多信息

请参阅 CK 发布的 LKML 公告。

检查是否启用了 MuQSS

启用 MuQSS 时，此启动消息应出现在内核环形缓冲区中：

```
# dmesg | grep -i muqss
...
MuQSS CPU scheduler v0.120 by Con Kolivas.
```

应用了 MuQSS 补丁的内核和 systemd

认为 MuQSS 不支持 *cgroups* 是一个常见的错误。它支持但不支持所有 cgroup 功能（例如，CPU 限制将不起作用）。

在 Linux-ck 中使用树外模块

使用 DKMS 可以轻松地编译和管理许多树外模块（broadcom-wl，nvidia，virtualbox 等）。

另请参见

- Con Kolivas 的内核补丁仓库
- Con Kolivas 的博客
- Con Kolivas 在 Linux 内核邮件列表上的第一个 BFS 公告
- Wikipedia 的 Con Kolivas 页面
- Wikipedia 的 BFS 文章
- Arch ck Linux 论坛支持主题

2.3 DKMS

难度 入门 | **篇幅** 3.6 KB | **所属** 内核

本节将讲到

- 安装
- 升级
- 使用方法
- 创建 DKMS 包
- 相关链接

摘自维基百科：

: 动态内核模块支持（DKMS）是用来生成 Linux 的内核模块的一个框架，这种模块的源代码一般不在 Linux 内核源代码树中。当新的内核安装时，DKMS 支持的内核模块会自动重建。

这意味着你不再需要等待某个公司、项目组或者包维护者释出新版本的内核模块。自从 pacman 支持钩子之后，内核更新时就会自动重新构建模块。

安装

安装 dkms包 包和目标内核的头文件。例如，标准 linux包 内核的头文件可以用软件包 linux-headers包 安装。其他内核也有其相应的头文件包。

有许多位于内核源码树之外的内核模块都有 DKMS 变体；有一些位于官方软件仓库，大多数可以在 AUR 找到。

升级

虽然在内核升级时，DKMS 的编译自动执行，但是依然有可能编译报错。所以需要特别注意 pacman 的输出。当系统需要这些模块才能启动，或者使用不在官方软件仓库中的内核时，需要额外注意。

若要应用内核中的变化、修复 bug、或是添加特性，请在重启前先升级 DKMS 包。

使用方法

如何手动调用DKMS：

可以通过执行以下命令来使能使用DKMS时的Tab补全：

```
# source /usr/share/bash-completion/completions/dkms
```

列出内核模块

列出当前模块的状态，版本，包括源码树内的模块：

```
# dkms status
```

重新构建模块

为当前内核重新构建所有的模块：

```
# dkms autoinstall
```

或者为指定版本的内核构建：

```
# dkms autoinstall -k 3.16.4-1-ARCH
```

为当前内核构建一个特定的模块（例如: 对于当前内核）：

```
# dkms install -m nvidia -v 334.21
```

或者简单地：

```
# dkms install nvidia/334.21
```

移除模块

移除一个内核模块（旧的内核模块并不会被自动移除）：

```
# dkms remove -m nvidia -v 331.49 --all
```

或者简单的：

```
# dkms remove nvidia/331.49 --all
```

如果你卸载了 dkms包 包，那么以前构建内核模块使用的相关文件信息就会丢失。如果这样，去 `/usr/lib/modules/kernel_release` 和 `/var/lib/dkms/package_name` 下删除不再需要的文件和目录。

创建 DKMS 包

请参见 DKMS 包指南。

相关链接

- Linux Journal: 探寻动态内核模块支持

2.4 内核模块打包指南

难度 入门 | **篇幅** 4.5 KB | **所属** 内核

本节将讲到

- 打包分离
- 文件放置

打包分离

包含内核模块的软件包应予以特殊对待，以支持希望在系统上安装多个内核的用户。

当打包包含内核模块和其他非模块支持文件/实用程序的软件时，将内核模块与支持文件分开很重要。

准则

打包此类软件时（以 NVIDIA 驱动程序为例），约定为：

- 创建一个 `nvidia` 包，其中只包含为vanilla内核构建的内核模块
- 创建包含支持文件的 `nvidia-utils` 包
- 确保 `nvidia` 依赖于 `nvidia-utils`（除非有充分的理由不这样做）
- 对于另一个内核，比如 `linux-mm`，创建 `nvidia-mm` ，其中包含针对该内核构建的内核模块，该内核提供 `nvidia` ，并且还依赖于 `nvidia-utils`
- 确保 `nvidia` 依赖于当前的主要内核版本，例如：

```
    depends=('linux' 'nvidia-utils')
```

基本原理

虽然为不同内核构建的内核模块始终位于不同的目录中并且可以和平共处，但是支持文件预计会在一个位置找到。如果一个软件包包含模块和支持文件，则您将无法为多个内核安装模块，因为软件包中的支持文件将导致 pacman 文件冲突。

模块和随附文件的分离允许在同一系统上为多个内核安装多个内核模块软件包，同时在预期位置之间共享支持文件。

文件放置

如果软件包中包含旨在覆盖现有同名模块的内核模块，则应将该模块放在 `/lib/modules/*X*.*Y*.*Z*-arch1-1/updates` 目录中。当 *depmod* 运行时，此目录中的模块将优先考虑。

2.5 SystemTap

难度 入门 | **篇幅** 5.6 KB | **所属** 内核

本节将讲到

- SystemTap
- 标准内核
- 内核重新编译
- 编译自定义内核
- 问题处理

Systemtap 是一种在运行时收集 Linux 系统信息的自由软件(GPL)框架。

SystemTap

安装 systemtapAUR 或 systemtap-gitAUR。和上游版本的对比：.

要从源代码编译，请访问这里。最新代码会包含新内核版本和发行版的支持。

标准内核

至少需要安装软件包 linux-headers包。

Arch 会从发布的二进制文件,包括内核中删除调试数据。很多 systmetap 功能就不能使用了。stapprobes 手册记录了 NON-DWARF 和 AUTO-DWARF 类型下什么功能可用：

- kernel tracepoints: kernel.trace("*")
- user-space probes: process("...").function("...") (for programs you build yourself with -g)
- user-space markers: process("...").mark("...") (if they were configured with the *<sys/sdt.h>* markers)
- perfctr-based probes: perf.*
- non-dwarf kernel probes: kprobe.function("...") and nd_syscall.\ *tapset (if a /boot/System.map*\ file is available, see below).

内核重新编译

可用自定义 *linux-custom* 软件包之后再运行 SystemTap。重新编译 linux包 软件包也非常方便。参考 Kernels/Traditional compilation。

准备

首先，运行 `cd ~/ && mkdir build && cd build/ && asp checkout linux && cd linux/trunk` 来获取原始内核构建文件。然后使用 `makepkg --`

`verifysource` 获取附加文件。通过执行验证，您可以安全地跳过“更新校验和”步骤。

修改config文件

编辑 **config** (32位内核) 或 **config.x86_64** (64位内核), 确保打开这些选项:

- CONFIG_KPROBES=y
- CONFIG_KPROBES_SANITY_TEST=n
- CONFIG_KPROBE_EVENT=y
- CONFIG_NET_DCCPPROBE=m
- CONFIG_NET_SCTPPROBE=m
- CONFIG_NET_TCPPROBE=y
- CONFIG_DEBUG_INFO=y
- CONFIG_DEBUG_INFO_REDUCED=n
- CONFIG_X86_DECODER_SELFTEST=n
- CONFIG_DEBUG_INFO_VTA=y

默认只有*CONFIG_DEBUG_INFO* 和*CONFIG_DEBUG_INFO_REDUCED*没被打开，修改这两个即可.

更新校验值

执行 `md5sum config[.x86_64]` 获得新的文件校验值.

编辑 **PKGBUILD** 文件, 这一部分 `md5sums=('sum-of-first' ... 'sum-of-last')` 和这一部分 `source=('first-source' ... 'last-source')` 是个数相同，顺序相同的, 把新获得的校验值在合适的位置替换.

`makepkg --skipchecksums` 使用命令可以跳过校验，但这样做对其它文件（比如下载的内核源码包）来说不安全，**因此建议按这里给出的方法操作**。

编译并安装

可选步骤: 可以在 `/etc/makepkg.conf` 文件中设置 `MAKEFLAGS="-j16"` 加速编译.

执行 `makepkg` 开始编译, 然后 `sudo pacman -U *.pkg.tar.gz` 安装编译好的包. **pacman** 会提示你这是重新安装 （**reinstall**）, 这就对了!

linux包 和 linux-headers包 需要安, linux-docs包 则随意.

通过这个方法, 外部内核模块 (例如 nvidia包 和 virtualbox包) 就不需要被重新编译了.

Systemtap

从 AUR 中安装systemtap即可: systemtapAUR, 完成.

编译自定义内核

参考这个官方README

问题处理

版本匹配问题

如果出现如下错误：

```
   /usr/share/systemtap/runtime/stat.c:214:2: error: 'cpu_possible_map' undeclared
(first use in this function)
```

请安装 systemtap-git。

System.map丢失

你可以再启用 DEBUG_INFO 的情况下构建Linux内核时恢复它：

```
   cp src/linux-3.6/System.map /boot/System.map-3.6.7-1-ARCH
```

或者执行以下命令，

```
   sudo cp /proc/kallsyms /boot/System.map-`uname -r`
```

Process return probes not available

如果你确定内核配置正确，但再启动 `stap` 时收到以下两条信息：

```
   WARNING: Kernel function symbol table missing [man warning::symbols]
   semantic error: process return probes not available [man error::inode-uprobes]
```

那么SystemTap可能未能验证此功能的支持，你可以在 System.map丢失这个步骤中解决此问题。

2.6 Zswap

难度 进阶 | **篇幅** 6.4 KB | **所属** 内核

本节将讲到

- 开关 zswap
- 定制 zswap
- 另见

翻译状态：

- 本文（或部分内容）译自 zswap，最近一次同步于 2023-3-24，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

zswap 是一个内核功能，它为交换页提供了一个压缩的内存缓存。原本会交换到磁盘的页被压缩并存储到内存中的存储池中。一旦池已满或内存耗尽，最近最少使用的

（LRU）页就会被解压缩并写入磁盘，就好像它没有被拦截一样。将页解压缩到交换缓存后，可以释放池中的压缩版本。

与 zram 相比的区别在于，zswap 与 swap 设备协同工作，而 *zram* 是内存中的交换设备，不需要后备交换设备。

开关 zswap

在稳定版（stable）的 linux包 官方内核中，zswap 会被默认启用。这可以通过稳定版内核配置中的 `CONFIG_ZSWAP_DEFAULT_ON` 标志进行验证。

要在运行时禁用 zswap，请执行下面的命令：

```
# echo 0 > /sys/module/zswap/parameters/enabled
```

要永久禁用 zswap，添加 `zswap.enabled=0` 到你的内核参数。

定制 zswap

当前参数

zswap 有几个可自定义的参数。可以使用以下方式显示实时设置：

```
$ grep -R . /sys/module/zswap/parameters
/sys/module/zswap/parameters/same_filled_pages_enabled:Y
/sys/module/zswap/parameters/enabled:Y
/sys/module/zswap/parameters/max_pool_percent:20
/sys/module/zswap/parameters/compressor:lz4
/sys/module/zswap/parameters/zpool:z3fold
/sys/module/zswap/parameters/accept_threshold_percent:90
```

见 zswap 文档获取不同参数的描述。

显示初始配置的引导时加载消息可以通过以下方式检索：

```
# dmesg | grep zswap:
[    0.317569] zswap: loaded using pool lz4/z3fold
```

设定参数

使用 sysfs

每个设置都可以在运行时通过 sysfs 接口进行更改。作为示例，要更改 `compressor` 参数：

```
# echo lz4 > /sys/module/zswap/parameters/compressor
```

使用内核引导参数

要持久化参数更改，必须在内核引导参数中添加相应的选项，例如 `zswap.compressor=lz4`。因此，要永久设定上述所有设置，必须添加以下内核参数：

```
zswap.enabled=1 zswap.compressor=lz4 zswap.max_pool_percent=20 zswap.zpool=z3fold
```

当通过引导参数更改压缩算法时，需要确保在引导期间尽早加载相应的压缩模块（参考 #压缩算法）。

最大池大小

存储池不是预先分配的，它可以增长到可用内存总量的一定百分比，默认情况下最多占内存总量的20%。一旦达到此阈值，就会将页从池中逐出到交换设备中。压缩池的最大大小由参数 `max_pool_percent` 控制。

压缩存储池分配器

zpool 参数控制压缩存储池的管理。

在 6.3 版本后的内核中，添加了 zsmalloc 分配器。它在低内存条件下表现良好，并且可以节省更多内存。

对于较旧的内核，可以使用 zbud 数据分配器，它将恰好两个压缩页面存储到一个中，这将压缩比限制为 2 或更低。

同样， z3fold 分配器允许每页最多 3 个压缩对象（与 zbud 的 1.7 相比，典型的压缩比平均约为 2.7）。该分配器以前被建议优于 zsmalloc，因为它具有更好的性能，但由于这一点不再成立且存在众多错误，自 6.13 起已被弃用并默认禁用（在 6.6.55 中回移植到 6.6 LTS），并计划移除。

默认情况下会创建一个类型为 zsmalloc 的 zpool。使用内核参数 `zswap.zpool` 可以在启动时选择其他分配器。数据分配器也可以在后期通过 sysfs 接口进行更改。

压缩算法

对于页的压缩，zswap 使用内核加密 API 提供的压缩器模块。默认情况下会使用 *lz4* 压缩算法，但也可以在启动时使用 `zswap.compressor` 更改压缩算法。其他选项包括 *deflate*, *lz4hc*, *lzo*, *lzo-rle*, *842* 和 *zstd*。

使用 *sysfs* 在运行时更改压缩不会有问题，但在本例中，zswap 从 *lz4* 开始，并在稍后阶段切换到定义的算法。要立即使用另一种算法启动 zswap，必须通过内核引导参数进行设定，并且内核必须尽早加载相应的模块。这可以通过以下步骤来实现：

1. 将与所选压缩器相关的模块添加到 mkinitcpio#模块（MODULES）数组中。
2. 修改 mkinitcpio 配置后重新生成 ramdisk 环境：见 mkinitcpio#创建和启用映像。
3. 在内核参数中将 `zswap.compressor` 设定为你选择的算法。

在下次启动时，见 #当前参数来检查 zswap 现在是否使用了请求的压缩器。

另见

- zswap：如何确定是否压缩交换页？。
- IBM 支持文章“新的 Linux zswap 压缩功能”（基准图像不加载）。
- Ask Ubuntu: zram vs. zswap vs. zcache。
- Arch Linux 论坛讨论串。
- 由 zswap 的主要开发者撰写的 LWN.net 技术文章。

2.7 内核/Arch 构建系统

难度 进阶 | **篇幅** 9.3 KB | **所属** 内核

本节将讲到

- 获取所需内容
- 修改 PKGBUILD
- 编译
- 安装
- 引导加载程序

参阅内核。

利用 Arch 编译系统，可以基于官方的 linux包 包编译自定义内核。这种编译方法可以自动化整个过程，并且是基于一个已经经过详细测试过的内核包。你可以编辑 PKGBUILD 来使用一个自定义内核配置或者添加附加的补丁。

获取所需内容

因为要使用到 makepkg, 请先了解 makepkg 的使用方法和最佳实践建议。例如不要用 root/sudo 运行 makepkg.

首先建立一个编译目录 `build`：

```
$ mkdir ~/build/
$ cd ~/build/
```

安装软件包 devtools包 和 base-devel包组.

首先需要一个原始内核作为自定义的基础，从Git获取PKGBUILD等文件，放到编译目录：

```
$ pkgctl repo clone --protocol=https linux
```

此时，目录树应该看起来像这样：

```
~/build/linux/-+
               +--config
                __PKGBUILD
```

然后从相应的来源获取其他需要的文件 (例如自定义配置文件、补丁等)。

修改 PKGBUILD

编辑 `PKGBUILD`，找到 `pkgbase` 修改为自定义软件包的名称:

```
PKGBUILD
pkgbase=linux-custom
```

警告：切勿将`linux`添加到`provides`数组中，因为你的定制内核和为原`linux`包编译的内核模块不兼容，不满足`provides`所需的依赖关系。同样，也不要把`linux-headers`添加到头文件包的`provides`数组。

避免创建文档

编译内核的冗长过程中，很大一部分时间被用于编译文档。2022年8月25日之后，可以应用以下补丁到PKGBUILD中，来避免生成文档：

```
63c63
<   make htmldocs all
---
>   make all
195c195
< pkgname=("$pkgbase" "$pkgbase-headers" "$pkgbase-docs")
---
> pkgname=("$pkgbase" "$pkgbase-headers")
```

这一补丁删除了#63行，更改了#195行。如果补丁没法直接应用，那就需要手动进行修改。

修改 prepare()

可以在`prepare()`中打上需要的补丁，或修改内核配置文件。

若要修改配置选项，可以直接编辑源文件。

亦可改用 GUI 工具调整编译选项。从 PKGBUILD 的 prepare() 函数中取消注释`make olddefconfig`，然后添加你所需的工具：

```
PKGBUILD
...
  msg2 "Setting config..."
  cp ../config .config
  #make olddefconfig

  make nconfig     # new CLI menu for configuration
  #make menuconfig # CLI menu for configuration
  #make xconfig    # X-based configuration
  #make oldconfig  # using old config from previous kernel version
  # ... or manually edit .config
  make prepare
...
```

警告：systemd包对内核配置有一些要求，无论是一般使用、特定使用条件（比如UEFI）还是某个功能（比如bootchart），不满足这些要求可能导致不稳定或者性能下降。systemd包推荐的配置列表可在`/usr/share/doc/systemd/README`中查看。在编译前应先行检查是否已经满足要求。另外，随着其版本更迭，这些要求可能会改变，arch不会对这些改变进行公告。为了确保你的定制内核满足systemd的要求，在更新systemd之前应检视其发行注记。

生成新校验和

#修改 prepare())步骤意味着你已经修改了`$_srcname/.config`文件。需要生成新的校验和（需安装pacman-contrib包）：

```
$ updpkgsums
```

编译

现在可以用`makepkg`命令编译内核了，如果配置时选择了交互编译(例如 menuconfig)，编译时需要进行配置。

```
 $ makepkg -s
```

选项 `-s` 会在编译时下载需要的依赖关系，比如 xml 和 docs.

注意：

- 内核代码是 PGP 签名, makepkg 会尝试进行校验，详情请参考 makepkg#Signature checking.
- 编译时间可能长达数小时，取决于你的硬件性能。并行编译可以在多核系统上显著减少编译时间。
- 可以用`time makepkg`来知道这次生成在你的系统上占用了多长时间。

安装

编译完成后，`build/linux`文件夹中应有两个包，分别是内核和内核的头文件。命名类似：

```
linux-custom-5.8.12-x86_64.pkg.tar.zst
linux-custom-headers-5.8.12-x86_64.pkg.tar.zst
```

最好同时安装这两个包，因为他们可能相互依赖（比如DKMS）：

```
# pacman -U linux-custom-5.8.12-x86_64.pkg.tar.zst linux-custom-headers-5.8.12-
x86_64.pkg.tar.zst
```

（需要执行的命令是你实际的包名）

引导加载程序

如果你的定制内核和默认内核共存（修改`pkgbase`），为了开机时能够引导你的内核，安装完成后需更新引导加载程序的配置文件，为新的内核和initramfs映像添加新的引导菜单项。

如果使用grub包，可以执行以下命令自动生成新的配置文件（假设配置文件是`/boot/grub/grub.cfg`）：

```
# grub-mkconfig -o /boot/grub/grub.cfg
```

更新

假设已有arch内核的源码，要对其进行更新，一种方法是使用https://github.com/archlinux/linux。本节以下内容假设内核源码树根目录位于`~/build/linux/`。

一般而言，放置内核源码需要使用两个本地git仓库，一个本地裸git仓库位于`archlinux-linux`，指向`https://github.com/archlinux/linux.git`。另一个拉取自裸仓库，位于`**src/**archlinux-linux/`。本地的补丁、构建很可能位于`**src/**archlinux-linux/`。

在本例中，本地位于`archlinux-linux/`的裸仓库的HEAD指向

```
$ cd ~/build/linux/archlinux-linux/
$ git log --oneline --max-count 1 HEAD
4010b622f1d2 Merge branch 'dax-fix-5.3-rc3' of
git://git.kernel.org/pub/scm/linux/kernel/git/nvdimm/nvdimm
```

可以看到，这个HEAD的指向位于 v5.2.5-arch1 和 v5.2.6-arch1 之间的某个版本。

```
$ git fetch --verbose
```

执行后可以看到它获取了v5.2.7-arch（编写wiki时最新的archlinux版本）。若没有获取到新的版本，则说明已是最新。

此时可把源码更新到进行编译的位置。

```
$ cd ~/build/linux/src/archlinux-linux/
$ git checkout master
$ git pull
$ git fetch --tags --verbose
$ git branch --verbose 5.2.7-arch1 v5.2.7-arch1
$ git checkout 5.2.7-arch1
```

可进行检查

```
$ git log --oneline 5.2.7-arch1 --max-count=7
13193bfc03d4 Arch Linux kernel v5.2.7-arch1
9475c6772d05 netfilter: nf_tabf676926c7f60les: fix module autoload for redir
498d650048f6 iwlwifi: Add support for SAR South Korea limitation
bb7293abdbc7 iwlwifi: mvm: disable TX-AMSDU on older NICs
f676926c7f60 ZEN: Add CONFIG for unprivileged_userns_clone
5e4e503f4f28 add sysctl to disallow unprivileged CLONE_NEWUSER by default
5697a9d3d55f Linux 5.2.7
```

上述信息显示了一些在`Arch Linux kernel v5.2.7-arch1` and `Linux 5.2.7`之间的archlinux补丁。

使用下列命令拉取最新的PKGBUILD和内核配置文件：

```
$ cd ~/build/linux/
$ git pull
```

然后应将`~/build/linux/linux/*`合并到`~/build/linux/`中。合并可以手动完成，也可以使用合并工具。参照#修改 prepare())一节进行所需的修改。

此时应可成功执行`makepkg --verifysource`。

然后可进行#编译，确保执行`makepkg`时添加`--noextract`选项。它应该能成功构建，因为源代码是`makepkg --nobuild`解压出的。编译完成后，可进行#安装。

清理

合并之后可以删除`~/build/linux/linux/`。另外，随着更新，`~/build/linux/src/archlinux`会积累很多分支（形如`5.2.7-arch1`）。这些可以使用下列命令清理掉：

```
$ cd ~/build/linux/src/archlinux
$ git branch --delete --force --verbose 5.2.7-arch1
```

参阅

- https://docs.kernel.org/kbuild/kconfig.html and the parent directory

2.8 Zram

难度 进阶 | **篇幅** 10.2 KB | **所属** 内核

本节将讲到

- 作为 swap 的使用
- 提示与技巧
- 另请参阅

zram（原名 compcache）是 Linux 内核模块，在内存中创建压缩块设备，实现实时压缩的 ramdisk。其主要用途为：

- 作 swap（最常见）
- 作 /tmp 等临时文件存储

早期仅用于压缩 swap，故名 compcache。

作为 swap 的使用

在最开始，创建出的 zram 块设备并不会预留或使用任何内存。仅当有文件需要被或者想要被交换出内存时，它们才会被压缩并移入 zram 块设备。因此，zram 块设备将会根据需要动态地增长或收缩。

即使保守估计 zstd 只能达到 1:2 压缩比（实际中压缩比通常为 1:3），zram 也能提供在内存中存储比不进行内存压缩更多内容的优势。

注意：

- 在配置 zram 时设置的 zram 大小指代可存储的最大未压缩数据量，*不是*指压缩后的数据大小。只要内存中的压缩后数据量小于系统物理内存容量，你甚至可以将 zram 大小配置为等于或大于系统物理内存容量。

- 假如相关的 zswap 内核功能为启用状态，它将阻碍 zram 的有效使用。这是因为 zswap 会在 zram 之前被用作 swap 缓存，并在换出的内存分页到达 zram 前对其进行拦截和压缩。在这种情况下，尽管 zramctl(8) 的输出可能并非如此，实际上大部分 zram 并未被使用。因此，建议在开始前通过内核参数或者 sysfs 设置永久禁用 zswap。
- 不支持在休眠时将内存换出至 zram，即便 zram 被配置在位于永久性存储的设备上。*logind* 会阻止休眠到配置在 zram 上的交换空间的尝试。

提示：你可以通过 `mem_limit` 参数指定 ZRAM 可存储的最大压缩后数据量。

一开始可以先尝试配置大小为物理内存的一半。

临时应用

参考如下配置一个采用 zstd 压缩、容量为系统内存大小一半，优先度较高且非持久的 zram 设备 (重启后失效)：

```
# modprobe zram
# zramctl /dev/zram0 --algorithm zstd --size "$(($(grep -Po 'MemTotal:
\s*\K\d+' /proc/meminfo)/2))KiB"
# mkswap -U clear /dev/zram0
# swapon --discard --priority 100 /dev/zram0
```

如需禁用，可以重启或执行以下命令：

```
# swapoff /dev/zram0
# modprobe -r zram
```

关于每一步操作、配置选项及潜在问题的详细信息可以参考 zram 模组的官方文档。

若需一个持久化的解决方案，请在下述章节所述的方法中任选其一。

使用 udev 规则

以下的案例描述了如何通过单个 udev 规则自动在启动时配置 zram 内 swap。该案例无需额外的软件包。

显式地在启动时加载模组：

```
/etc/modules-load.d/zram.conf
zram
```

创建如下的 udev 规则（请按需调整 `disksize` 属性）：

```
/etc/udev/rules.d/99-zram.rules
ACTION=="add", KERNEL=="zram0", ATTR{initstate}=="0", ATTR{comp_algorithm}="zstd",
ATTR{disksize}="4G", RUN="/usr/bin/mkswap -U clear %N", TAG+="systemd"
```

将 `/dev/zram` 以一个高于默认值的优先度添加到您的 fstab：

```
/etc/fstab
/dev/zram0 none swap defaults,discard,pri=100 0 0
```

使用 zram-generator

zram-generator包 提供了 `systemd-zram-setup@zram*N*.service` 单元，可自动初始化 zram 设备而无需用户启动/启用相关模板或实例。详见 zram-generator(8)。

如要使用，只需安装 zram-generator包，并创建包含如下内容的 /etc/systemd/zram-generator.conf 配置文件：

```
/etc/systemd/zram-generator.conf
[zram0]
zram-size = min(ram / 2, 4096)
compression-algorithm = zstd
```

zram-size 是 zram 设备的大小，单位为 MiB，你可以使用 ram 指代总内存容量。

compression-algorithm 指定了 zram 设备使用的压缩算法。执行 cat /sys/block/zram0/comp_algorithm 将显示所有可用的压缩算法（包括当前选用的算法）。

详细信息请参考 zram-generator.conf(5)。

然后执行 daemon-reload，并启动你配置的 systemd-zram-setup@zram*N*.service 示例（*N* 需要与实例编号匹配，该示例中为 systemd-zram-setup@zram0.service） 。

您可使用 zramctl(8) 或 swapon(8)，或是通过查阅 systemd-zram-setup@zram*N*.service 实例的单元状态，来检查您所配置的 /dev/zram*N* 设备的 swap 状态。

使用 zramswap

zramswapAUR 提供一个自动化脚本，可配置一个优先度较高、默认为 20% 系统内存容量的 swap。如需让其自动在启动时执行，请启用 zramswap.service。

提示与技巧

查看 zram 状态

使用 zramctl(8)。示例：

```
$ zramctl
NAME       ALGORITHM DISKSIZE  DATA  COMPR  TOTAL STREAMS MOUNTPOINT
/dev/zram0 zstd           32G  1.9G 318.6M 424.9M      16 [SWAP]
```

- DISKSIZE = 32G: 该 zram 设备最多会存储 32 GiB 的未压缩数据
- DATA = 1.9G: 目前, 该 zram 设备上存储着 1.9 GiB (未被压缩的) 数据
- COMPR = 318.6M: 这 1.9 GiB 未被压缩的数据被压缩到了 318.6 MiB
- TOTAL = 424.9M: 包含元信息在内，这 1.9 GiB 未压缩的数据使用了 424.9 MiB 的物理内存

多个 zram 设备

初始情况下，加载 zram 模块会创建一个 /dev/zram0 设备。

如果您需要更多的 `/dev/zram` 设备，使用 `num_devices` 内核模块参数指定设备的量，或在之后按需添加它们。

优化 zram 上的 swap

介于 zram 与磁盘 swap 的行为不同，可以配置系统的 swap 以充分利用 zram 的优势：

```
/etc/sysctl.d/99-vm-zram-parameters.conf
vm.swappiness = 180
vm.watermark_boost_factor = 0
vm.watermark_scale_factor = 125
vm.page-cluster = 0
```

该配置的解释：

这些值与Pop!_OS 中使用的值相同。这条 GitHub 上 Pop!_OS 的 pull request 同时链接到了r/Fedora 上一些用户所进行的测试，结果显示 `vm.page-cluster = 0` 是较为合适的值。此外，他们也发现建议使用较高的 swappniess，这与内核文档中的建议相符。

默认情况下该值为 60。

: 对于像 zram 或 zswap 这样在内存中的 swap，以及一些混合式的、在相较文件系统来说更为快速的设备上 swap 的配置，应当考虑 100 以上的值。例如，若 swap 设备上的随机读写平均快于文件系统上随机读写的 2 倍，则 swappiness 值应为 133 (x + 2x = 200, 2x = 133.33)。

在配备硬盘驱动器的系统上，在内存设备上的随机读写会比在文件系统上的读写快若干数量级，因此 swappiness 应为 200 左右。甚至是在配备高速固态硬盘的系统上，较高的 swappiness 值也是理想的。

为 zram 块设备启用后备设备

可以配置 zram 将未压缩内存页放入指定的块设备中。

如要手动添加后备设备：

```
# echo /dev/sdX > /sys/block/zram0/backing_dev
```

如要通过 *zram-generator* 为 zram 块设备添加后备设备，可在 `/etc/systemd/zram-generator.conf` 中对应的 `[zram*X*]` 设备下添加如下内容：

```
/etc/systemd/zram-generator.conf
writeback-device=/dev/disk/by-partuuid/XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX
```

然后可以通过执行以下命令将未压缩的页面推送到块设备：

```
# echo huge > /sys/block/zramX/writeback
```

将 zram 用于非 swap 用途

zram 也可以被用作普通的内存块设备，像是占用较少物理内存，但性能略微下降的 `/dev/ram` 设备。然而，有几个问题需要注意：

- 无分区表支持（不会自动创建 `/dev/zram*x*p*y*`）。
- 块大小固定为 4 kiB。

显然，可以通过在 zram 上叠加回环设备来绕过这一问题。使用 *losetup*，可以使用 `-b` 参数指定块大小，使用 `-P` 参数处理分区表并自动创建分区回环设备。

```
# zramctl -f -s NG
/dev/zramx
```

将磁盘镜像复制到新创建的 `/dev/zram*x*`，然后创建回环设备。如果磁盘镜像有分区表，回环设备的块大小必须与分区表使用的块大小匹配，通常为 512 或 4096 字节。

```
# losetup -f -b 512 -P /dev/zramx
# ls /dev/loop*
/dev/loop0 /dev/loop0p1 /dev/loop0p2
# mount /dev/loop0p1 /mnt/boot
# mount /dev/loop0p2 /mnt/root
```

注意：

- zram 设备编号取决于已有的 zram 设备，其大小应足以存放磁盘镜像。
- `ls /dev/loop*` 的输出结果取决于磁盘镜像中的内容。

另请参阅

- Wikipedia:zram
- https://github.com/pop-os/default-settings/pull/163
- https://www.reddit.com/r/pop_os/comments/znh9n6/help_test_a_zram_optimization_for_pop_os/

2.9 PAM

难度 进阶 | **篇幅** 11.0 KB | **所属** 内核

本节将讲到

- 安装
- 配置文件
- 配置方法
- 更多 PAM 包
- 提示与技巧

Linux 可插拔认证模块（PAM，Pluggable Authentication Modules）提供了系统级的用户认证框架。如下描述引用自项目介绍:

: PAM 将程序开发与认证方式进行分离，程序在运行时调用附加的“认证模块”完成自己的工作。本地系统管理员可通过配置选择要使用哪些认证模块。

本文描述在 Arch Linux 下用于认证本地和远程用户的基础默认设置。对默认设置的更改在链接的专门文章内展开。

安装

pam包 包受 base包 元软件包依赖，默认已经安装在系统。PAM 模块放置于 `/usr/lib/security` 目录。

软件仓库中还包括其它一些可选的 PAM 包，详见 #配置方法。

配置文件

`/etc` 目录有多个子目录与 PAM 相关，使用命令 `pacman -Ql pam | grep /etc` 查看创建的默认配置文件。这些配置与模块的 #安全性参数或 #PAM 基础配置有关。

安全性参数

`/etc/security` 包含了对认证方法参数的系统级配置，基础安装使用上游默认配置。

注意 Arch Linux 没有对这些配置文件进行定制。例如 `/etc/security/pwquality.conf` 可用于配置系统级的密码强度要求，但需要手动将 `pam_pwquality.so` 模块加入到 #PAM 基础栈内（Arch Linux 默认没有包含该模块）。

详见 #安全性参数配置。

PAM 基础栈

`/etc/pam.d/` 目录专门用于存放 PAM 配置，用于为具体的应用程序设置独立的认证方式。配置文件由以下安装包提供：

- pambase包。包含供应用程序使用的特定于 Arch Linux 的 PAM 基础配置。
- 其它基础包。例如 util-linux包 添加了为 `login` 及其它一些应用的认证配置，shadow包 提供 Arch Linux 默认的用户数据库认证方式（参见用户和用户组）。

基础安装的不同配置文件相互链接，在运行时“堆叠”，称为“PAM 栈”。例如，在用户登录时，`login` 程序将加载 `system-local-login` 策略，而该策略又会加载其他策略：

```
/etc/pam.d/
```

```
login -> system-local-login -> system-login -> system-auth
```

不同的应用程序加载配置的链路可能不同。例如，openssh包 加载 `sshd` PAM 策略的链路如下：

```
/etc/pam.d/
sshd -> system-remote-login -> system-login -> system-auth
```

因此，需要从链路中选择合适的配置文件。对于上述示例，可以通过 `sshd` 设置其特殊的验证方式，或者通过 `system-remote-login` 更改所有远程登录的验证方式。这两种方式不影响本地登录，修改 `system-login` 或 `system-auth` 则会同时影响本地和远程登录。

如 `sshd` 的例子，任何**接入 PAM（pam-aware）**的应用程序都需要在 `/etc/pam.d` 目录下安装的单独策略，以便正确集成并依赖 PAM 栈。否则应用程序将使用默认配置 `/etc/pam.d/other`，拒绝认证并在日志中记录警告。

提示：

接入 PAM 的程序在运行时动态链接 PAM，例如：

```
$ ldd /usr/bin/login | grep pam
libpam.so.0 => /usr/lib/libpam.so.0 (0x000003d8c32d6000)
libpam_misc.so.0 => /usr/lib/libpam_misc.so.0 (0x000003d8c30d2000)
```

`login` 程序接入了 PAM，因此相应的认证策略**必须**存在。

PAM 的手册页 pam(8) 和 pam.d(5) 描述了配置文件的标准规范。其中解释了四个 PAM 管理组——账户管理（account）、认证管理（auth）、密码管理（password）、会话管理（session）——和各种控制值（用于配置模块堆叠方式和行为）。

此外，详尽的文档安装在 `/usr/share/doc/Linux-PAM/index.html`，包含多个指南，其中提供每个标准模块的手册页。

警告：对 PAM 配置的修改会影响用户认证。不正确的修改可能导致**所有用户**都无法/都可以登录。

提示：更改对于已通过身份验证的用户无效；测试 PAM 的一种比较好的方法是在测试机器上本地登录并进行开发，保持会话持续运行，同时在另一个控制台用另一个用户测试结果。

配置方法

本节提供了一个目录，包括关于如何修改 PAM 配置和如何添加特殊的 PAM 模块的文章/章节。注意，模块的 man 手册通常与模块名一致（不包括 `.so` 后缀）。

安全性参数配置

下面的章节描述如何修改 PAM 默认参数配置：

- 安全#禁用空密码

: 更改允许空密码的默认设置。

- 安全#用 pam_pwquality 强制使用强密码

: 使用 `pam_crackib.so` 强制使用强密码。

- 安全#在三次登录尝试失败后封锁用户

: 使用 `pam_tally.so` 限制尝试登录次数。

- 安全#仅允许特定用户

: 使用 `pam_wheel.so` 限制用户登录。

- 实时进程管理#配置PAM 和安全#限制进程数量

: 使用 `pam_limits.so` 来配置系统进程限制。

- 环境变量#使用 pam_env

: 使用 `pam_env.so` 设置环境变量的示例。

PAM 栈和模块配置

下面的章节说明对于具体的用例，如何修改 #PAM 基础栈：

- pam_mount（英语：pam_mount）

: 使用 `pam_mount.so` 在用户登录时自动挂载加密目录。

- ECryptfs#自动挂载（英语：ECryptfs#Auto-mounting）

: 使用 `pam_ecryptfs.so` 自动挂载加密目录。

- Dm-crypt/启动时挂载（英语：Dm-crypt/Mounting at login）

: 使用 `pam_exec.so` 在用户登录时执行自定义脚本。

- 活动目录集成#配置 PAM（英语：Active Directory integration#Configuring PAM）

: 使用 `pam_winbind.so` 和 `pam_krb5.so` 通过活动目录（LDAP、Kerberos）服务进行用户认证。

- LDAP 认证#PAM 配置

: 使用 `pam_ldap.so` 集成 LDAP 客户端或服务端认证。

- YubiKey#使用 PAM 进行 Linux 用户认证

: 使用 U2F（`pam_u2f.so`）和 YubiKey 提供的专有 Yubico OTP 实现（`pam_yubico.so`）。

- pam_oath

: 使用 `pam_oath.so` 进行软件双因素认证。

- fprint

: 使用 `pam_fprintd.so` 进行指纹认证。

- pam_autologin

: 保存用户名和密码以自动登录。

- pam_usb（英语：pam_usb）

: 使用 `pam_usb.so` 通过 USB 设备进行（可作为双因素）认证。

- SSH 密钥#pam_ssh

: 使用 `pam_ssh.so` 认证远程用户身份。

- pam_abl（英语：pam_abl）

: 使用 `pam_abl.so` 限制通过 ssh 的暴力破解攻击。

- EncFS（英语：EncFS#/etc/pam.d/）

: 使用 `pam_encfs.so` 自动挂载加密目录。

- Google 身份验证器

: 使用 `pam_google_authenticator.so` 进行双因素认证。

- Very Secure FTP Daemon#虚拟用户的 PAM 认证

: 使用 `pam_pwdfile.so` 配置 FTP chroot，以便在没有本地系统账户的情况下对用户进行身份验证。

更多 PAM 包

除了上面提到的包，AUR 中还包括许多另外的 PAM 模块和工具：

- **Pamtester** — PAM 测试工具集

: https://pamtester.sourceforge.net/ || pamtesterAUR

注意，AUR中有一个 PAM 关键词标签，但并不是所有可用的相关软件包都包含。因此，可能还是需要搜索软件包描述。

提示与技巧

被封锁

如果被 PAM 封锁，可能是密码输错太多次了，参见安全#在三次登录尝试失败后封锁用户。

参阅

- linux-pam.org——项目主页
- Understanding and configuring PAM——一篇入门文章
- Login managers: An introduction——结合显示管理器说明 PAM 的必要性

2.10 内核模块

难度 进阶 | **篇幅** 11.3 KB | **所属** 内核

本节将讲到

- 获取信息
- 自动加载模块
- 手动加载模块
- 配置模块参数
- 别名

内核模块是可以按需加载或卸载的内核代码，可以不重启系统就扩充内核的功能。

要创建内核模块，请阅读此指南。模块可以设置成内置或者可加载，要动态加载或删除模块，必须在内核配置中将其配置为可加载模块（因此与模块相关的行将显示字母 M）。

如果您想在新内核安装后自动重建模块，参见 Dynamic Kernel Module Support(DKMS)。

获取信息

模块保存在 `/lib/modules/*kernel_release*` (使用 `uname -r` 命令显示当前内核版本)。

注意：模块名通常使用 (_) 或 - 连接，但是这些符号在 `modprobe` 命令和 `/etc/modprobe.d/` 配置文件中都是可以相互替换的。

显示当前装入的内核模块：

```
$ lsmod
```
显示模块信息：

```
$ modinfo module_name
```
显示所有模块的配置信息：

```
$ modprobe -c | less
```
显示某个模块的配置信息：

```
$ modprobe -c | grep module_name
```
显示一个装入模块使用的选项，可以使用sysfsutils包中的systool(1)：

```
$ systool -v -m module_name
```
显示模块的依赖关系：

```
$ modprobe --show-depends module_name
```

自动加载模块

目前，所有必要模块的加载均由 udev 自动完成。所以，如果不需要使用任何额外的模块，就没有必要在任何配置文件中添加启动时加载的模块。但是，有些情况下可能需要在系统启动时加载某个额外的模块，或者将某个模块列入黑名单以便使系统正常运行。

systemd

内核模块可以在`/etc/modules-load.d/` 下的文件中明确列出，以便 systemd 在引导过程中加载它们。 每个配置文件都以 `/etc/modules-load.d/*program*.conf`的样式命名。 配置文件仅包含要加载的内核模块名称列表，以换行符分隔。 空行和第一个非空白字符为`#`或`;`的行被忽略。

```
/etc/modules-load.d/virtio-net.conf
# Load virtio_net.ko at boot
virtio_net
```
另见modules-load.d(5)。

早期模块加载

initramfs 映像可能不包含 `/etc/modules-load.d/` 中要求的内核模块，它也可能缺少在该文件夹中设置的文件。早期模块加载取决于使用的 initramfs 生成器：

- mkinitcpio包: 参见 Mkinitcpio#MODULES
- dracut包: 参见 Dracut#Early kernel module loading
- booster包: 参见 Booster#Early module loading

手动加载模块

内核模块由 kmod包 包提供的工具处理。您可以手动使用这些工具。

注意：如果你已经升级了你的内核但还没有重新启动，*modprobe* 将失败并且没有错误消息并以代码 1 退出，因为路径 `/usr/lib/modules/$(uname -r)/` 不再存在。当 *modprobe* 无法确定是否存在这种情况时，请手动检查此路径是否存在。

要手动装入模块的话，执行:

```
# modprobe module_name
```

按文件名加载模块:

```
# insmod filename [args]
```

如果要移除一个模块：

```
# modprobe -r module_name
```

或者:

```
# rmmod module_name
```

配置模块参数

要将参数传递给内核模块，您可以使用 modprobe 手动传递它们，或者使用 modprobe 配置文件或使用内核命令行确保始终应用某些参数。如果该模块内置于内核中，则必须使用内核命令行，其他方法将不起作用。

手动加载时设置

将参数传递给模块的基本方法是使用 *modprobe* 命令。使用简单的 `*key=value*` 赋值在命令行上指定参数：

```
# modprobe module_name parameter_name=parameter_value
```

使用 /etc/modprobe.d/中的文件

`/etc/modprobe.d/` 目录中的文件可用于将模块设置传递给 udev，它将使用 `modprobe` 管理系统启动期间的模块加载.此目录中的配置文件可以具有任何名称，只要它们以 *.conf* 扩展名结尾。语法是：

```
/etc/modprobe.d/myfilename.conf
options module_name parameter_name=parameter_value
```

多个模块参数用空格分隔，而一个参数可以接收一个值列表，列表用逗号分隔：

```
/etc/modprobe.d/myfilename.conf
options module_name param1=value1 param2=value2a,value2b …
```

例如：

```
/etc/modprobe.d/thinkfan.conf
# On ThinkPads, this lets the 'thinkfan' daemon control fan speed
options thinkpad_acpi fan_control=1
```

注意：如果任何受影响的模块是从 initramfs 加载的，那么您需要将适当的 *.conf* 文件添加到 mkinitcpio.conf 中的 `FILES` 或使用`modconf` hook，然后 重新生成 initramfs 以包含“.conf”文件。要查看默认 initramfs 的内容，请使用 lsinitcpio。

使用内核命令行

还可以使用内核命令行向模块传递选项。这也是向内核内置模块传递选项的唯一途径。对于所有常见的引导加载程序，以下语法是正确的：

```
modname.parametername=parametercontents
```

例如:

```
thinkpad_acpi.fan_control=1
```

只需将其添加到引导加载程序的内核行，如内核参数中所述。

别名

别名是模块的替代名称。例如：`alias my-mod real_long_modulename` 表示您可以使用 `modprobe my-mod` 代替 `modprobe real_long_modulename`。您也可以使用 shell 风格的通配符，所以 `alias my-mod* really_long_modulename` 意味着 `modprobe my-mod` 和 `modprobe my-mod-something` 有相同的效果。创建别名：

```
/etc/modprobe.d/myalias.conf
alias mymod really_long_module_name
```

一些模块具有别名，用于在应用程序需要它们时自动加载它们。禁用这些别名可以防止自动加载，但仍允许手动加载模块。

```
/etc/modprobe.d/modprobe.conf
# Prevent Bluetooth autoload
alias net-pf-31 off
```

黑名单

在内核模块的上下文中，黑名单是一种防止内核模块加载的机制。如果不需要相关的硬件，或者如果加载该模块会导致问题，这可能很有用：例如，可能有两个内核模块试图控制同一个硬件，将它们一起加载会导致冲突。

一些模块作为 initramfs 的一部分加载。`mkinitcpio -M` 将打印出所有自动检测到的模块：为了防止 initramfs 加载其中一些模块，请将它们列入黑名单中的 `/etc/modprobe.d` 下的 `*.conf*` 文件 并且它应该在图像生成期间由 `modconf` 挂钩添加。运行 `mkinitcpio -v` 将列出由各种挂钩（例如 `filesystems` 挂钩、`block` 挂钩等）拉入的所有模块。如果您的文件中没有 `modconf` 挂钩，请记住将该 *.conf* 文件添加到 `/etc/mkinitcpio.conf` 中的 `FILES` 列表中`HOOKS` 列表（比如您改变了默认配置），将模块列入黑名单 regenerate the initramfs，然后重新启动。

使用 /etc/modprobe.d/ 中的文件

在 `/etc/modprobe.d/` 中创建一个 *.conf* 文件，并使用 `blacklist` 关键字为要列入黑名单的每个模块附加一行。例如，如果您想阻止 `pcspkr` 模块加载以避免通过 PC 扬声器发出声音：

```
/etc/modprobe.d/nobeep.conf
# Do not load the 'pcspkr' module on boot.
blacklist pcspkr
```

注意：`blacklist` 命令将一个模块列入黑名单，这样它就不会自动加载，但如果另一个未列入黑名单的模块依赖于它或者它是手动加载的，则该模块可能会被加载。

但是，对于该情况有一个解决方法； `install` 命令指示 modprobe 运行自定义命令，而不是像往常一样将模块插入内核，因此您可以强制使模块加载失败：

```
/etc/modprobe.d/blacklist.conf
...
install module_name /bin/true
...
```

这将有效地将该模块和任何其他依赖于它的模块列入黑名单。

使用内核命令行

提示：如果模块损坏导致无法引导系统，这将非常有用。

您也可以从引导加载程序中将模块列入黑名单。

如Kernel参数.中所述，只需将 `module_blacklist=modname1,modname2,modname3` 添加到引导加载程序的内核行中即可。

注意：

- 将多个模块列入黑名单时，请注意，它们之间仅用逗号分隔。 空格或其他内容可能会破坏语法。
- `module_blacklist` 会让内核完全拒绝加载模块。如果只想阻止隐式加载，可能会在稍后手动加载模块，那么正确的参数是 `modprobe.blacklist=modname1,modname2,modname3`。这并不能阻止启动过程中的显式加载，例如通过 systemd 或其他模块。

问题处理

模块未加载

如果特定模块未加载并且启动日志（可通过以 root 身份运行 `journalctl -b` 访问）显示该模块已列入黑名单，但目录 `/etc/modprobe.d/` 没有显示相应的条目，请检查 `/usr/lib/modprobe.d/` 的另一个 modprobe 源目录以获取黑名单条目。

如果内核模块中包含的“vermagic”字符串与当前运行的内核的值不匹配，则不会加载模块。如果已知模块与当前运行的内核兼容，则可以使用 `modprobe --force-vermagic` 忽略“vermagic”检查。

警告：忽略内核模块的版本检查可能会导致内核崩溃或系统由于不兼容而表现出未定义的错误。请谨慎使用 `--force-vermagic`。

参见

- 禁用 PC 喇叭
- 编写 WMI 驱动程序 - 一个 LWM 介绍

2.11 内核参数

难度 进阶 | **篇幅** 14.0 KB | **所属** 内核

本节将讲到

- 启动管理器配置
- dracut
- EFI 启动存根
- 劫持 cmdline
- 参数列表

共有三种办法向内核传递参数以控制其行为：

1. 在编译内核时：在内核的 `config` 文件中。详见内核#编译。
2. 在内核启动时：通过命令行参数（通常通过启动管理器设置，或在统一内核镜像中设置）。
3. 在运行时：通过 `/proc/sys/`（参见 sysctl）和 `/sys/` 中的文件。

提示：可加载模块的选项可通过 `/etc/modprobe.d/` 中的 *.conf* 文件设置。参见 内核模块#使用_/etc/modprobe.d/中的文件。

不同的方式有不同的命令名称、是否可见以及设置方式。本文仅说明第二种方式(内核命令行)，并记录适用 Arch linux 内核的常用参数。

大部分参数和某个子系统相关，需要在编译或加载时加入子系统，而且相关硬件需要存在。

内核命令行参数的格式为 `parameter` 或 `parameter=value`，或 `module.parameter=value`。

注意：

- 通过 `cat /proc/cmdline` 可以查看当前使用的命令行参数，验证修改是否生效。
- 所有内核参数均区分大小写。

启动管理器配置

注意：Arch Linux 安装镜像 在 UEFI 系统上使用 systemd-boot，在 BIOS 系统上使用 Syslinux。

内核参数可以在启动时临时修改，也可以永久性写到启动管理器的配置文件中，永远起作用。

下面示例：把参数`quiet` 和 `splash` 加到启动管理器 Clover、GRUB、GRUB Legacy、LILO、Limine、rEFInd、Syslinux 和 systemd-boot 中。

Clover

- 将以下内容添加到 `*esp*/EFI/CLOVER/config.plist`：

: ``` <string>initrd=\initramfs-linux.img root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash</string>

GRUB

- 在菜单出现后按 `e` 然后将它们添加至 `linux` 行：

: ```
linux /boot/vmlinuz-linux root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
: 按 `Ctrl+x` 以便用这些参数启动。

- 要使改变在重启后仍生效，您*可以*手动编辑 `/boot/grub/grub.cfg` 中的如上内容。对于初学者，建议：

: 编辑 `/etc/default/grub` 并将您的内核选项添加至 `GRUB_CMDLINE_LINUX_DEFAULT` 行：

: : ``` GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"

: 然后重新生成 `grub.cfg` 文件：

: : ```
grub-mkconfig -o /boot/grub/grub.cfg
有关配置GRUB的更多信息，请参阅 GRUB 。

GRUB Legacy

- 在菜单出现后按 `e` 然后将它们添加至 `kernel` 行：

: ``` kernel /boot/vmlinuz-linux root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash

: 按 `b` 以便用这些参数启动。

- 要使改变在重启后仍生效，编辑 `/boot/grub/menu.lst` 并将它们添加至 `kernel` 行，如上所示。

有关配置GRUB Legacy的更多信息，请参阅 [GRUB Legacy](../../01_安装与包管理/启动加载程序/GRUB%20Legacy.md) 。

LILO

- 将它们添加到 `/etc/lilo.conf` 中，使用 `append` 或 `addappend`：

: ```
image=/boot/vmlinuz-linux
        ...
        append="quiet splash"
有关配置LILO的更多信息，请参阅 LILO 。

Limine

- 要临时添加内核参数，在启动条目选择屏幕出现时按 e 并修改 cmdline 行：

: ``` cmdline: root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash

- 要使更改永久生效，编辑 Limine 配置文件中的 `kernel_cmdline` 行：

: ```
/+Arch Linux
    ...
    kernel_cmdline: root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
kernel_cmdline 是 cmdline 的别名

rEFInd

- 在所需的菜单条目上按 Insert, F2, Tab, 或 +，然后在子菜单条目上再次按下。在字符串末尾添加内核参数：

: ``` root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw initrd=\boot\initramfs-linux.img quiet splash

: 按 `Enter` 以使用这些参数启动。

- 要使更改在重启后永久生效，编辑 `/boot/refind_linux.conf` 并在所有需要的行中的引号之间追加它们，例如：

: ```
"Boot using default options"   "root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash"

- 如果您在 rEFInd 中禁用了操作系统的自动检测，并在 *esp*/EFI/refind/refind.conf 中定义了操作系统条目来加载您的操作系统，您可以像这样编辑它：

: ``` menuentry "Arch Linux" { ... options "root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash" ... }

有关配置 rEFInd 的更多信息，请参阅 [rEFInd](../../01_安装与包管理/启动加载程序/REFInd.md) 。

Syslinux

- 当出现启动选择菜单的时候，按 `Tab` 进入修改模式：

: ```
linux /boot/vmlinuz-linux root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw initrd=/boot/initramfs-linux.img quiet splash
: 按 Enter 键以当前设置启动.

- 要将设置永久生效，编辑 /boot/syslinux/syslinux.cfg 并添加 APPEND 行:

: ``` APPEND root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash

更多详情请见[Syslinux](../../01_安装与包管理/启动加载程序/Syslinux.md) 。

systemd-boot

- 当启动菜单出现时 按 `e`进入编辑界面:

: ```
initrd=\initramfs-linux.img root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash
: 按 Enter 即可按照输入的参数启动。

注意：

- 如果没有设置显示启动菜单, 你需要按住Space启动电脑来进入启动菜单 。
- 如果不能够从启动菜单上进行编辑，修改 /boot/loader/loader.conf 加入 editor 1 来开启编辑功能。
- 如果想永久加入参数，编辑 /boot/loader/entries/arch.conf (假设你已经设置好了 EFI system partition) 的options 行:

: ``` options root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 rw quiet splash

- 若使用 [统一内核映像](../../01_安装与包管理/启动加载程序/统一内核映像.md) 启动，请编辑 `/etc/kernel/cmdline`。

更多信息请参见 [systemd-boot](../../01_安装与包管理/启动加载程序/Systemd-boot.md) .

dracut

[dracut](../../01_安装与包管理/Initramfs/Dracut.md) 可以将内核参数嵌入 initramfs，并跳过启动加载器的配置，参考：[dracut#内核命令行参数](../../01_安装与包管理/Initramfs/Dracut.md#内核命令行参数)。注意这种方式只对 dracut 所能接受的参数生效，例如 `root=` 和 `rd.*`，这些参数并不会变成真正的内核参数。

EFI 启动存根

参阅 [EFISTUB#直接使用 UEFI](../../01_安装与包管理/启动加载程序/EFISTUB.md#直接使用_UEFI).

劫持 cmdline

即使没有访问启动加载器的权限，也可以通过更改内核参数来启用调试（如果您有 root 权限）。这可以通过覆盖存储内核参数的 `/proc/cmdline` 来实现。然而，`/proc/cmdline` 即使作为 root 也不可写，因此这个技巧通过使用绑定挂载来掩盖路径。

> **注意：**这种技巧仅劫持读取 `/proc/cmdline` 的用户空间进程，无法作用于被内核读取的参数。例如，通过这种方式添加 `debug` 不会使内核进入调试模式。

首先创建一个包含所需内核参数的文件：
/root/cmdline

root=UUID=0a3407de-014b-458b-b5c1-848e92a327a3 ro console=tty1 logo.nologo debug

然后使用绑定挂载覆盖参数：

`-n` 选项跳过将挂载添加到 `/etc/mtab`，因此即使根文件系统以只读方式挂载也能工作。您可以通过 `cat /proc/cmdline` 确认更改是否成功。

参数列表

本表并不完整。除了内核，其他程序也能从 `/proc/cmdline` 读取参数并受到影响。

- 所有选项的完整列表请参见[内核文档](https://docs.kernel.org/admin-guide/kernel-parameters.html)（包含内核与引导加载程序的可用参数）。[bootparam(7)](https://man.archlinux.org/man/bootparam.7) 中有一份基本的参数列表可供参考。
- [systemd](../../01_安装与包管理/初始化/Systemd.md) 的可用参数请参见 [kernel-command-line(7)](https://man.archlinux.org/man/kernel-command-line.7)。
- 带有基于 busybox 的 initial ramdisk 的 [mkinitcpio](../../01_安装与包管理/Initramfs/Mkinitcpio.md) 的可用参数请参见 [mkinitcpio(8) § EARLY INIT ENVIRONMENT](https://man.archlinux.org/man/mkinitcpio.8#EARLY_INIT_ENVIRONMENT) 和 [mkinitcpio#运行时配置](../../01_安装与包管理/Initramfs/Mkinitcpio.md#运行时配置)。
- [dracut](../../01_安装与包管理/Initramfs/Dracut.md) 的可用参数请参见 [dracut.cmdline(7)](https://man.archlinux.org/man/dracut.cmdline.7)。

参数	描述
init	运行指定的二进制文件而不是 `/sbin/init` 作为 init 进程。[systemd-sysvcompat](https://archlinux.org/packages/?name=systemd-sysvcompat)包 包将 `/sbin/init` 符号链接到 `/usr/lib/systemd/systemd` 以使用 [systemd](../../01_安装与包管理/初始化/Systemd.md)。将其设置为 `/bin/sh` 以启动到 shell。
initrd	指定初始 ramdisk 的位置。对于 UEFI 启动管理器 和 EFI 启动存根，路径必须使用反斜杠 (`\`) 作为路径分隔符。
[cryptdevice](../../08_安全与加密/静态数据加密/Dm-crypt_系统配置.md#cryptdevice)	指定 [dm-crypt](../../08_安全与加密/静态数据加密/Dm-crypt.md) 加密分区的位置以及 [设备映射器](https://en.wikipedia.org/wiki/device_mapper) 名称。
debug	启用内核调试（事件日志级别）。
lsm	设置 Linux 安全模块的初始化顺序，用于启用 [AppArmor](../../08_安全与加密/访问控制/AppArmor.md)，[SELinux](../../06_软件与应用/红帽/SELinux.md) 或 TOMOYO。
maxcpus	SMP 内核在启动期间将启动的最大处理器数量。
mem	强制使用特定数量的内存。
netdev	网络设备参数。
nomodeset	禁用 [Kernel mode setting](内核级显示模式设置.md)。
panic	内核崩溃后自动重启的时间。
resume	指定从 休眠 唤醒时使用的交换设备。
ro	在启动时以只读方式挂载根设备。这是 [mkinitcpio](../../01_安装与包管理/Initramfs/Mkinitcpio.md) 的默认值1。
root	根文件系统。参见 [init/do_mounts.c](https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/init/do_mounts.c#n254) 了解内核支持的设备名称格式。请注意，带有 [udev](../../04_硬件与外设/硬件检测和故障排除/Udev.md) 的 [initramfs](../../01_安装与包管理/启动过程/Arch%20的启动流程.md) 支持 更多名称格式。与 [systemd#GPT 分区自动挂载](../../01_安装与包管理/初始化/Systemd.md#GPT_分区自动挂载) 兼容的设置允许完全省略该参数，或者使用 `root=dissect`。
rootflags	根文件系统挂载选项。对于无法通过重新挂载应用的选项（例如，由 [systemd-remount-fs.service(8)](https://man.archlinux.org/man/systemd-remount-fs.service.8) 应用）非常有用。例如，[XFS](../文件系统/XFS.md) 根卷的 `discard` 选项或 [Btrfs 使用子卷作为根](../文件系统/Btrfs.md#Mounting_subvolume_as_root) 时的 `subvol=` 选项。
rw	在启动时以读写方式挂载根设备。这是内核的默认值1。
systemd.unit	启动到指定的 [目标](../../01_安装与包管理/初始化/Systemd.md#Targets)。
video	覆盖帧缓冲视频默认值。

1. 如果内核命令行上未明确设置 `ro` 或 `rw`，内核将使用 `rw`（参见 [bootparam(7) § General non-device-specific boot arguments](https://man.archlinux.org/man/bootparam.7#General_non-device-specific_boot_arguments)）。然而，[mkinitcpio](../../01_安装与包管理/Initramfs/Mkinitcpio.md) 使用 `ro` 作为默认值，覆盖内核的默认值（参见 [mkinitcpio(8) § EARLY INIT ENVIRONMENT](https://man.archlinux.org/man/mkinitcpio.8#EARLY_INIT_ENVIRONMENT)）。启动加载器也可能有自己的配置默认值，例如，*grub-mkconfig* 使用 `rw`（参见 [FS#36275](https://bugs.archlinux.org/task/36275) 作为参考）。

    > **注意：**当使用 mkinitcpio 的 [fsck 钩子](../文件系统/Fsck.md#启动时检查)（参见 ）或 [使用 F2FS 作为根文件系统](../文件系统/F2FS.md#Remounting_impossible_with_some_options) 时，需要 `rw`。

参见

- [bootparam(7)](https://man.archlinux.org/man/bootparam.7)
- [kernel-command-line(7)](https://man.archlinux.org/man/kernel-command-line.7)
- 省电相关内核参数
- [内核启动命令行参数参考](http://files.kroah.com/lkn/lkn_pdf/ch09.pdf)——[Linux Kernel in a Nutshell](http://www.kroah.com/lkn/) 的第 9 章，作者 [Greg Kroah-Hartman](https://en.wikipedia.org/wiki/Greg_Kroah-Hartman)

2.12 内核级显示模式设置

难度 进阶 | **篇幅** 16.5 KB | **所属** 内核

本节将讲到

- 背景
- 配置
- 问题解决
- 强制设置显示模式与 EDID
- 禁用 KMS

内核级显示模式设置 (KMS)，作用是可以在内核级别而不是最终用户级别切换显示分辨率和颜色深度。

Linux 内核的 KMS 实现支持在 framebuffer 中使用原生分辨率和即时终端(tty)切换。KMS 使用了更新的技术(例如 DRI2)，可以减少失真、增强3D性能，甚至使用内核空间节能功能。

注意：NVIDIA 闭源驱动从 364.12 开始也实现了 KMS，但是没有利用 Linux 内核内置的实现；从 545.29 开始，高分辨率终端的 fbdev 驱动作为需要手动启用的实验性功能提供。

背景

以前，设定显卡是 X 服务器的工作。所以虚拟终端不可能提供漂亮的图像效果。同时，每次使用Ctrl+Alt+F1~7从X切换到虚拟终端时，x服务器必须将显卡的控制权交给内核，这个流程显得低效并且会导致闪烁。将控制权切回到X服务器同样是一个“痛苦”的过程。

使用内核模式设置后，内核可以设定显卡的模式。这样开机启动即可看到漂亮的显示画面，在 X 图形界面 和 终端 之间也可以快速切换，还有其他的一些优点。

配置

不管使用什么方法，都需要先“禁用”下列项：

- 所有引导加载程序中的 `vga=` 选项，这个选项与 KMS 的原生分辨率设置冲突。
- 所有 `video=` 选项，这个选项会启动帧缓冲，导致驱动冲突。

KMS 晚启动

Intel, Nouveau, ATI 和 AMDGPU 驱动已经为所有芯片组提供了 KMS 支持。这些驱动也会自动启用 KMS，无须额外操作。

闭源的 NVIDIA 驱动从 364.12 开始支持 KMS，并且从 560.35.03-5 版本开始默认启用。旧版本需要手动启用。

KMS 早启动

提示：如果你有分辨率方面的问题，可以参考一下 强设模式 ，也许会有帮助。

KMS通常是在initramfs 阶段之后开始初始化，但是你也可以在initramfs的阶段启用KMS。将所需的视频驱动模块加入 initramfs 配置文件：

- AMDGPU 驱动加入 `amdgpu`; 老的 ATI 驱动加入 `radeon`
- Intel 卡加入`i915`
- 开源的 Nouveau 驱动加入`nouveau`
- 对于树外 NVIDIA 驱动，加入 `nvidia nvidia_modeset nvidia_uvm nvidia_drm`。见 NVIDIA#DRM 内核级显示模式设置 以获取详细信息。
- Matrox 显卡加入 `mgag200`。
- QEMU 显卡使用（*qemu* 选项 `-vga *type*` 或 libvirt `<video><model type='*type*'>`）：
- `bochs` 对应 `std` (*qemu*) 和 `vga`/`bochs` (*libvirt*),
- `virtio-gpu` 对应 `virtio`,
- `qxl` 对应 `qxl`,
- `vmwgfx` 对应 `vmware` (*qemu*) 和 `vmvga` (*libvirt*),
- `cirrus` 对应 `cirrus`。
- VirtualBox 显示控制器：
- `vmwgfx` 对应 VMSVGA，

- `vboxvideo` 对应 VBoxVGA 或 VBoxSVGA。

根据使用的 initramfs 生成器，具体的配置方法略有不同。

mkinitcpio

使用树内模块时，可以将 `kms` 加入到 `/etc/mkinitcpio.conf` 的 HOOKS 数组中（自 mkinitcpio v33 起为默认设置），或将模块名称加入 MODULES 数组。

对于树外 NVIDIA 模块，必须将模块的名字加入到 MODULES 数组中，如下所示：

```
/etc/mkinitcpio.conf
MODULES=(... nvidia nvidia_modeset nvidia_uvm nvidia_drm ...)
```

注意：如果你使用 PRIME GPU，并且主要的 GPU 是英特尔核显，而 AMD 显卡是独立显卡。`intel_agp` 可能会在从显示器无信号状态恢复时造成麻烦。详见 。

如果你在使用 #强制设置显示模式与 EDID 的方法，则也需要将选定的文件集成到 initramfs 去：

```
/etc/mkinitcpio.conf
FILES=(/usr/lib/firmware/edid/your_edid.bin)
```

完成这一切后，重新生成 initramfs。

Booster

如果您在使用 Booster，可通过以下的方式改变设置来加载需要的模块。

```
/etc/booster.yaml
modules_force_load: i915
```

如果您在使用 #强制设置显示模式与 EDID 的方法，则也需要将选定的文件集成到 Booster 镜像中：

```
/etc/booster.yaml
extra_files: /usr/lib/firmware/edid/your_edid.bin
```

之后，重新生成 Booster 镜像。

Dracut

Dracut 可通过 `--force_drivers` 命令行选项，或配置文件中的 `force_drivers+=""` 设置，在 initramfs 阶段使用 `modprobe` 提前加载驱动。例如：

```
/etc/dracut.conf.d/myflags.conf
# ...
force_drivers+=" amdgpu "
# ...
```

如果您在使用 #强制设置显示模式与 EDID 的方法，则也应包含自定义文件：

```
/etc/dracut.conf.d/myflags.conf
install_items+=" /usr/lib/firmware/edid/your_edid.bin "
```

之后，重新生成 Dracut 镜像。

问题解决

字体太小

Linux 控制台#字体介绍了如何在终端中使用大字体。软件仓库中的 (terminus-font包) 字体提供了很多字号，比如更大一些的 `ter-132b`。 或者，可以#禁用 KMS 以切换为更低的分辨率，使得字体外观显得更大一些。另一种选择是 KMSCON。

强制设置显示模式与 EDID

如果无法自动配置您本机的分辨率，或者根本无法侦测到显示器；您的显示器很有可能没有或发送了破损的 EDID 文件。内核将尝试捕获这种情况并设置为某一通用分辨率。

如果您拥有贵显示器的 EDID 文件，您只需明确地强制设置即可(详见下文)。但是，您在大多数情况下可能无法直接取得一个健全的 EDID 文件。您可能需要提取现有 EDID 文件中的信息并试图修复它，从而获得了一个新的 EDID 文件。

在内核编译期间，可以通过遵循 上游文档 来生成用於各种分辨率和配置的新EDID二进製文件（另请参见 此处 的简短指南）。 这篇文章 也详细概述了其他解决方案。在大多数情况下，提取现有文件比较容易。例如从Windows下的驱动程序中提取EDID，如果您的显示器在那里运行良好。或者您可以使用来自 read-edid包 包的 get-edid(1) 命令，从具有相同设置的类似显示器中提取。可以从 `/sys/class/drm/*/edid` 中查找一下。

在您的EDID准备就绪后，请将它放置于特定的文件夹。例如一个在 `/usr/lib/firmware/` 下名为 `edid` 的文件夹，您可以将您的二进制文件放入其中。

请在内核参数中设置下述参数来在启动时加载：

```
drm.edid_firmware=edid/你的edid.bin
```

用下述参数来设定特定的连接器：

```
drm.edid_firmware=VGA-1:edid/你的edid.bin
```

如果你希望设置多个 edid 文件，用下面的参数：

```
drm.edid_firmware=VGA-1:edid/你的edid.bin,VGA-2:edid/你的另一个edid.bin
```

如果您要进行#KMS 早启动，则必须在 initramfs 中包含自定义 EDID 文件，否则可能会遇到问题。

启动后，也可以通过写入 `/sys/module/drm/parameters/edid_firmware` 修改 `drm.edid_firmware` 参数：

```
# echo edid/你的edid.bin > /sys/module/drm/parameters/edid_firmware
```

此更改只会对新连接的显示器生效；已经连接的显示器会继续使用原来的 EDID 设置。对于外接显示器，重新插拔即可使更改生效。

如果内核不处于锁定模式，可以使用 debugfs 在启动后加载 EDID，而不必通过内核命令行参数。这对于更换连接器上的显示器或测试 EDID 特别有用。准备好上述 EDID 文件后执行：

```
# cat correct-edid.bin > /sys/kernel/debug/dri/0/HDMI-A-2/edid_override
```

若要取消覆盖：

```
# echo -n reset > /sys/kernel/debug/dri/0/HDMI-A-2/edid_override
```

如果显示器支持热插拔，还可以触发热插拔事件，让显示器使用刚刚加载的新 EDID（例如写入 `edid_override` 的 EDID），无需实际重新插拔显示器或重新启动：

```
# echo 1 > /sys/kernel/debug/dri/0/HDMI-A-2/trigger_hotplug
```

Forcing modes

注意：下面描述的方法并不完美，例如 Xorg 不会考虑指定的分辨率，因此建议用户使用上述方法。但通过 `video=` 内核命令行参数指定分辨率，对所有 Wayland 混成器 都很有用。

来自 nouveau wiki：

用内核命令行可以强制设置使用的模式。但是 DRM 内核选项的文档很少，使用的方法位于：

- https://docs.kernel.org/fb/modedb.html
- https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/gpu/drm/drm_fb_helper.c

格式：

```
video=<driver>:<conn>:<xres>x<yres>[M][R][-<bpp>][@<refresh>][i][m][eDd]
```

- `<driver>`：指定启动时的视频模式。
- `<conn>`：指定视频连接类型，例如 VGA、DVI、HDMI 等；可通过 `/sys/class/drm/` 查看可用连接器。
- `<xres>`：水平分辨率（像素）。
- `<yres>`：垂直分辨率（像素）。
- `[M]`：使用 VESA 协调视频时序（CVT）计算模式时序，而不是从数据库中查找模式。
- `[R]`：在使用 CVT 时为数字显示器启用减少消隐时间计算，缩短水平及垂直消隐间隔以节省带宽。
- `[-<bpp>]`：颜色深度，即每像素位数（例如 `-24` 表示 24 位色）。

- `[@<refresh>]`：刷新率（Hz）。
- `[i]`：启用隔行扫描模式。
- `[m]`：在 CVT 计算中添加边距（水平分辨率的 1.8%，向下取整为 8 的倍数；垂直分辨率的 1.8%）。
- `[e]`：强制启用输出。
- `[D]`：强制启用数字输出（例如 DVI-I 连接器）。
- `[d]`：强制关闭输出。

可以使用多个 `video=` 设置不同输出的分辨率，例如要强制 DVI 使用 1024x768 85 Hz，同时禁用 TV-out：

```
video=DVI-I-1:1024x768@85 video=TV-1:d
```

也可以在模式之后用逗号分隔并传入其他选项：

```
video=DVI-I-1:720x480,rotate=180
```

有效选项包括：

- `margin_top`、`margin_bottom`、`margin_left`、`margin_right`（整数）：边距的像素数，通常用于处理电视的过扫描问题。
- `reflect_x`（布尔值）：沿 X 轴进行镜像反射。
- `reflect_y`（布尔值）：沿 Y 轴进行镜像反射。
- `rotate`（整数）：将初始 framebuffer 旋转指定角度，可用值为 0、90、180、270。
- `tv_mode`：模拟电视制式，可用值包括 "NTSC"、"NTSC-443"、"NTSC-J"、"PAL"、"PAL-M"、"PAL-N" 或 "SECAM"。
- `panel_orientation`：可用值为 "normal"、"upside_down"、"left_side_up" 或 "right_side_up"。仅适用于 KMS 驱动，用于设置 KMS 连接器的 "panel orientation" 属性，向 KMS 用户提供显示面板方向的提示。

要获取当前连接器的状态，使用下面命令：

```
$ for p in /sys/class/drm/*/status; do con=${p%/status}; echo -n "${con#*/card?-}: ";
cat $p; done
DVI-I-1: connected
HDMI-A-1: disconnected
VGA-1: disconnected
```

禁用 KMS

有时需要禁用 KMS。只需在内核参数中加入 `nomodeset` 即可，设置方法请阅读内核参数页面。

使用 `nomodeset` 内核参数的同时，Intel 卡需要添加 `i915.modeset=0`，Nvidia 卡需要添加 `nouveau.modeset=0`。Nvidia Optimus 双显卡系统，需要添加三个内核参数：

```
nomodeset i915.modeset=0 nouveau.modeset=0
```

注意：有些 Xorg 驱动必须启用 KMS 才能工作，参阅所用驱动的页面。

2.13 内核/传统编译

难度 进阶 | **篇幅** 16.9 KB | **所属** 内核

本节将讲到

- 准备
- 配置
- 编译
- 安装
- 引导加载程序设置

本文介绍从 **kernel.org 的源代码**编译自己的内核。这是一种对所有发行版均适用的传统方法。取决于你的背景，这也可能比用 ABS 工具编译内核更复杂。请考虑使用 Arch 构建系统以使重复的编译任务更加有效和安全。

准备

准备编译时，不需要也不建议使用 root 账号或用 sudo 等工具获取 root 权限。

安装核心软件包

安装 base-devel包 元软件包，这个组包含了 make包 和 gcc包 等我们需要的软件包。参考 Arch 默认内核的 PKGBUILD，同时建议安装以下软件包：xmlto包、kmod包、inetutils包, bc包、libelf包、git包、cpio包、perl包、tar包、xz包。

创建内核编译目录

建议创建单独的内核编译目录，本文使用 `home` 下的 `kernelbuild` 目录。

```
$ mkdir ~/kernelbuild
```

获取源码

警告： systemd 需要至少 3.12 版本的内核（cgroups 支持需要 4.2 及以上的版本）。参考 `/usr/share/doc/systemd/README`。

从 https://www.kernel.org 下载源码，得到你所选的内核版本对应的 tarball) (`tar.xz`) 文件。

可以在浏览器中简单地右键 `tar.xz` 链接并选择 保存链接为 下载；也可以通过 HTTP、TFTP、Rsync 或者 Git 的各种图形化或命令行工具下载。

注意：

- 验证下载到的任何内核 tarball 文件的 PGP 签名总是好的做法。这样可以确保它是合法的，并有助于建立信任网络。参见 kernel.org/signature。
- 对于某些版本（例如 `-rc` 版本），下载的存档和生成的目录不会严格遵循本页示例中使用的 `*A*.*B*.*C*` 命名，请根据你的需要进行调整。

下面例子中使用 wget包 下载 *A.B.C* 内核到 `~/kernelbuild` 目录:

```
$ cd ~/kernelbuild
$ wget https://cdn.kernel.org/pub/linux/kernel/vA.x/linux-A.B.C.tar.xz
```

在信任下载的压缩包之前，你还得验证下载的正确性。 首先抓取签名，然后用它来抓取签名密钥的指纹，然后使用指纹来获取实际的签名密钥：

```
$ wget https://cdn.kernel.org/pub/linux/kernel/vA.x/linux-A.B.C.tar.sign
$ gpg --list-packets linux-A.B.C.tar.sign | grep -i keyid | awk '{print $NF}' | xargs gpg --recv-keys
```

注意签名是为 `tar` 存档（即扩展名 `.tar`）生成的，而不是为下载的压缩 `.tar.xz` 文件生成的。 你需要解压后者而不解压它。 检查你是否已安装 `xz`，然后这样继续：

```
$ unxz linux-A.B.C.tar.xz
$ gpg --verify linux-A.B.C.tar.sign linux-A.B.C.tar
```

如果没看到"Good signature"就别继续。

如果在构建目录中没使用`wget`，就需要手动将压缩包移入其中，例如：

```
$ mv /path/to/linux-A.B.C.tar.xz ~/kernelbuild/
```

半官方内核镜像

一些 kernel.org Git 存储库的半官方镜像由其各自的维护者提供。这些通常比 kernel.org 的版本更新。

- Linus Torvalds 放在 GitHub 上的主线内核

https://github.com/torvalds/linux.git.

- Greg Kroah-Hartman 放在 GitHub 上的稳定分支

https://github.com/gregkh/linux.

解压内核代码

- 将其解压后进入源码目录:

```
$ tar -xvf linux-A.B.C.tar
```

为了绝对确保不发生权限错误，需要运行 chown 将文件夹的所有权转移给当前用户。

要将包含其中每个文件的文件夹的所有权转移，运行 `chown` 命令。

```
$ chown -R $USER:$USER linux-A.B.C
```

文件夹中每个文件的所有权转移给你了，所以不会遇到任何与权限相关的错误。

为了完成准备工作，内核树需要绝对干净；不要觉得解压后源代码树是干净的。所以，切换到创建的新内核源目录，然后运行 `make mrproper` 命令：

```
$ cd linux-A.B.C
$ make mrproper
```

注意：`make mrproper` 的目标依赖于 `make clean` 的目标，所以没必要同时执行两者。参考。

配置

这是自定义内核以适应计算机的精确规格的最关键步骤。内核配置在 `.config` 文件中设置，其中包括 Kernel modules 的使用情况。正确设置 `.config` 中的选项，你的计算机会获得最高的运行效率。

注意：现阶段无需使用 root 帐户或 root 权限。

参考使用下面内容可以节省时间：

- 使用 Arch 官方内核的设置(推荐)
- 生成正在运行内核的配置文件和设置（手动，适用于高级用户但不推荐）

导出当前内核设置

这个命令将使用默认的 Arch 内核设置为自定义内核创建一个 `.config` 文件。如果正在运行官方 Arch 内核，你可以在内核源码目录中使用以下命令：

```
$ zcat /proc/config.gz > .config
```

也可以在 official Arch Linux kernel 中找到默认配置。

提示：

- 如果你在升级内核，一些选项可能改了或被删掉了。在这种情况下，在 #Compilation 下运行 `make` 时，它会要求你提供版本之间更改的每个配置选项的答案。要使用默认值而不用一直按 Enter，请运行 `make olddefconfig`。
- modprobed-db 可用于从默认 Arch `.config` 中删除不需要的模块。获得 properly populated database 后，运行 `make LSMOD=$HOME/.config/modprobed.db localmodconfig` 从 `modprobed.db` 中删除所有不存在的模块。

警告：如果你用当前的 `.config` 文件编译内核，别忘了在新的 `.config` 中的重命名内核版本，就是"CONFIG_LOCALVERSION"这项。或者用#高级配置中列出的用户界面之一中的"General Setup > Local version - append to kernel release"更改。如果跳过这一步，就有把当前内核覆盖的风险。

高级配置

提示：如果不想在启动和关闭时看到很多内核输出，可以禁用相关的调试选项。

有多种方式可以协助配置内核：

注意：下面列出的工具提供每个内核功能的三个配置选项：`y` 表示启用，`n` 表示禁用，`m` 表示启用作为内核模块（必要时加载）。

- `make nconfig`: 新的命令行 ncurses 界面，用来代替 `menuconfig`。
- `make xconfig`: 用户友好的图形界面，需要安装 Qt。这是推荐的方法，尤其对于没有经验的用户，因为它更容易导航，并且还会显示每个选项的信息。
- `make gconfig`: 和 xconfig 类似的图形界面，使用 GTK.

在内核源代码目录执行选择的方法，将生成一个全新的`.config`，可选参数默认是选中的，但是如果有老内核的 `.config` 文件，这些参数不一定会被选中。

改变配置文件并保存。建议在源代码目录外备份`.config`,因为你可能需要编译多次，直到所有的选项都配置正确。如果不确定每个选项的影响，每次编译只改变少量选项。如果无法启动新构建的内核，参见这里查看必须的配置项列表。从liveCD运行`$ lspci -k #`以列出使用中的内核模块的名字。最重要的是，你必须保留 systemd 需要的 CGROUPS 支持。

从live media运行 `lspci -k #` 会列出正在使用内核模块的名称。重中之重，你必须保留cgroup支持，这是systemd的依赖。参阅 Gentoo:Kernel/Gentoo Kernel Configuration Guide 和 Gentoo:Intel#Kernel 或 Gentoo:Ryzen#Kernel（适用于 Intel 或 AMD Ryzen CPU）。

编译

如果希望 gcc包 根据处理器指令集进行优化， 编辑内核目录下的 `arch/x86/Makefile` (i686) 或 `arch/x86_64/Makefile` (86_64)文件：

- 查找 `Processor type and features > Processor Family` 中设置的处理器系列 `CONFIG_MK8,CONFIG_MPSC,CONFIG_MCORE2,CONFIG_MATOM,CONFIG_GENERIC_CPU`
- 将 cc 调用选项设置为 `-march=native`，例如 `cflags-$(CONFIG_MK8) += $(call cc-option,-march=native)`.

对 32bit 内核，需要编辑 `arch/x86/Makefile_32.cpu` 并设置处理器的 `-march=native`。

编译时间将从15分钟到超过一小时不等。这很大程度依赖于选择了多少选项/模块和处理器性能。详情参考 Makeflags。 `.config` 配置好之后，在内核目录运行：

```
$ make
```

提示：为了更快地编译，可以使用 `-j*X*` 参数运行 *make* 以并行编译，其中 `*X*` 是整数。最快的编译通常是通过使用机器中 CPU 核心的数量来实现的； 例如，对于双核处理器，运行 `make -j2`。 参阅 Makepkg#减少编译时间。

安装

安装模块

内核编译了，它的模块也随之而来。首先构建模块：

```
$ make modules
```

警告：从这里开始，需要 root 权限执行命令，否则会失败.

编译完内核后编译模块:

```
# make modules_install
```

这会将编译后的模块复制到 `/lib/modules/*A*.*B*.*C*/` 中。 这使得单独使用的各个内核的模块保持分离。

提示：如果系统需要正常 Linux 内核外的模块，需要在定制内核安装完成之后重新编译它们。Nvidia 显卡模块可以参考 NVIDIA#Custom kernel。

拷贝内核到 /boot 目录

注意：确保已从你的系统架构的目录复制 `bzImage` 内核文件。见下文。

内核编译过程将生成该内核的压缩 `bzImage`，如果没有，运行：

```
make bzImage
```

必须将这个 bzImage 复制到 `/boot` 目录并重命名。只要名称带有 `vmlinuz-` 前缀，你就可以根据需要命名内核。 在下面的示例中，已安装并编译的 *A.B.C* 内核已被复制并重命名为 `vmlinuz-linux*AB*`：

```
# cp -v arch/x86/boot/bzImage /boot/vmlinuz-linuxAB
```

制作初始化内存盘

初始化内存盘，是一个在真正的根文件系统可访问之前被挂载的初始化根目录文件系统。初始化内存盘（initrd）被绑定到对应的内核，且作为内核启动过程的一部分被载入。内核将该初始化内存盘（initrd）作为两阶段启动过程的一部分挂载起来，以载入模块使得真正的文件系统可用，然后得到真正的根文件系统。初始化内存盘包含实现以上过程的目录和可执行文件的最小集合，例如用来将内核模块安装到内核的insmod工具。对桌面或服务器Linux系统来说，初始化内存盘(initrd)就是一个临时的文件系统。其生命周期短暂，仅作为到真正的根文件系统的桥梁。在没有可变存储的嵌入式系统中，初始化内存盘就是永久的根文件系统。参考 Initramfs on Wikipedia 和 mkinitcpio。

/boot 中的文件可以取任何名字。然而，为了方便区分，建议采取统一的命名规范，例如使用 `linux<major revision><minor revision>`。

如果你使用了 LILO 引导加载程序且其无法与内核设备映射器驱动连通，你可能需要先运行`modprobe dm-mod`命令。

自动生成

可以复制和修改现有的 mkinitcpio preset，以便可以按照与官方内核相同的方式生成自定义内核 initramfs image。 这在重新编译内核（例如更新时）时会节省很多时间。在下面的示例中，将为上面安装的内核*A.B.C*复制并修改库存 Arch 内核的预设文件。

首先，复制现有的预设文件，将其重命名为与复制 `bzImage` 时指定为 `/boot/vmlinuz-` 后缀的自定义内核的名称相匹配：

```
# cp /etc/mkinitcpio.d/linux.preset /etc/mkinitcpio.d/linuxAB.preset
```

其次，编辑文件并针对自定义内核进行修改。注意 `ALL_kver=` 参数也与复制 `bzImage` 时指定的自定义内核的名称匹配：

```
/etc/mkinitcpio.d/linuxAB.preset
...
ALL_kver="/boot/vmlinuz-linuxAB"
...
default_image="/boot/initramfs-linuxAB.img"
...
fallback_image="/boot/initramfs-linuxAB-fallback.img"
```

最后，生成 initramfs 映像：

```
# mkinitcpio -p linuxAB
```

手动方法

不使用 preset 文件，可以用 mkinitcpio 手动生成：

```
# mkinitcpio -k <kernelversion> -g /boot/initramfs-<file name>.img
```

- `-k` (--kernel <kernelversion>): 生成 initramfs image 的内核版本. `<kernelversion>` 需要和内核源代码目录和`/usr/lib/modules/`下的模块目录一致.
- `-g` (--generate <filename>): 要生成的 initramfs 文件。

例如，上面安装的*A.B.C*自定义内核的命令为：

```
# mkinitcpio -k A.B.C -g /boot/initramfs-linuxAB.img
```

拷贝 System.map

`System.map` 文件并非 Linux 启动所必须的。它包含一张该次构建的内核的标识符之列表（例如函数名，变量名等等）和它们相应的地址。该“标识符到地址的映射”是用于：

- 某些进程，像 klogd, ksymoops 等
- OOPS 处理程序，当内核崩溃出错信息释放到屏幕时（例如类似于内核在某个函数中崩溃了的信息）。

提示：UEFI 分区使用 FAT32 文件系统，不支持软链接。

如果 `/boot` 支持软链接（即不是 FAT32），将 `System.map` 复制到 `/boot`, 然后创建 `/boot/System.map` 软链接到 `/boot/System.map-YourKernelName`:

```
# cp System.map /boot/System.map-YourKernelName
# ln -sf /boot/System.map-YourKernelName /boot/System.map
```

如果 `/boot` 是不支持软链接的 FAT32，则需要将第二条命令换成 cp 来复制文件。

完成以上所有步骤之后，你的/boot目录中应该多出以下三个文件和一个软链接：

- Kernel: `vmlinuz-YourKernelName`
- Initramfs: `Initramfs-YourKernelName.img`
- System Map: `System.map-YourKernelName`
- System.map 软链接（或者复件）

引导加载程序设置

在你的引导加载程序（bootloader）配置文件中为你神奇的新内核添加一个入口。范例见GRUB, LILO, GRUB2 或 Syslinux。

提示：内核源码包含一个用于自动化 LILO 安装过程的脚本：如果你正在使用其他引导加载程序，可以安全地忽略它。

2.14 内核

难度 高级 | **篇幅** 20.4 KB | **所属** 内核

本节将讲到

- 官方支持的内核
- 编译
- 疑难解答
- 参见

根据维基百科：

: Linux 内核是一个开源的、类 Unix 的、操作系统宏内核。

Arch Linux 基于 Linux 内核。除了最新的稳定内核之外，还有许多各种各样的 Linux 内核可供选择。本文列出了仓库中提供的一些选择，并对它们简要介绍。本文还介绍了一些可用的内核补丁。文章的最后总览了自行编译内核的方法，并包含各种方法的链接。

内核包安装在 `/usr/lib/modules/` 路径下，随后其包含的 vmlinuz 可执行镜像会被复制到 `/boot/` 目录。安装新的内核包后，需重新配置引导加载程序以适配变更。若需降级内核，请参阅降级软件包#降级内核。

官方支持的内核

官方支持的内核可通过论坛和漏洞报告获取社区支持。

- **Stable** — 采用了一些补丁的原版 Linux 内核和模块。

: https://www.kernel.org/ || linux包

- **Hardened** — 注重安全的 Linux 内核，采用一系列加固补丁以缓解内核和用户空间漏洞。和 linux包 相比，它启用了上游更多的内核加固功能。

: https://github.com/anthraxx/linux-hardened || linux-hardened包

- **Longterm** — 长期支持（LTS）的 Linux 内核及模块，提供针对服务器使用场景优化的配置选项。

: https://www.kernel.org/ || linux-lts包

- **Realtime kernel（英语：Realtime kernel）** — 由 Ingo Molnar 领导的核心开发者小组维护。该补丁使内核的几乎所有部分可被抢占（除少量“raw_spinlock critical regions”（“原始自旋锁关键区域”）代码段外），通过将大多数内核自旋锁替换为支持优先级继承的互斥锁，并将所有硬件中断和软件中断迁移至内核线程处理实现。

: https://wiki.linuxfoundation.org/realtime/start || linux-rt包、linux-rt-lts包 : > **注意**：实时内核支持已并入 Linux 6.12

- **Zen Kernel** — 一些内核黑客合作的结果，致力于提供最适合日常使用的内核。详细信息请参阅常见问题解答和详细特性列表。

: https://github.com/zen-kernel/zen-kernel || linux-zen包

编译

可通过以下方式编译内核：

/Arch 构建系统 : 利用已有的高质量的 `linux` PKGBUILD 且可受益于包管理系统。 **/传统编译** : 需要手动下载内核源代码包，并以普通用户身份在自己的主目录中进行编译。

警告：

- 使用自定义内核可能会引起各种稳定性和可靠性问题，包括数据丢失。强烈建议操作前进行备份。
- Arch Linux 仅对 #官方支持的内核提供官方支持。若使用非官方支持的内核，务必在请求支持时提及。

提示：

- 提高系统速度的最佳方法是先根据架构和处理器类型定制内核配置。
- 可以通过移除不使用的功能模块来减小内核体积（从而缩短编译时间）。例如禁用蓝牙、video4linux、千兆以太网等支持。

Arch 内核包的 `config` 文件位于 Arch 软件包的源文件中（例如 linux包 链接到的 ）。如果启用了 `CONFIG_IKCONFIG_PROC` 内核选项，当前运行的内核的 `config` 文件也可通过 `/proc/config.gz` 路径获取。

部分列出的软件包也可通过非官方用户仓库获取预编译版本。

kernel.org 内核

- **Git** — 使用 Linus Torvalds Git 仓库源码构建的 Linux 内核及模块。

: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git || linux-gitAUR

- **Mainline** — 每 2～3 个月发布一次的引入所有新功能的内核。

: https://www.kernel.org/ || linux-mainlineAUR

- **Next** — 包含待合并至下个 Mainline 版本的功能的前沿内核。

: https://www.kernel.org/doc/man-pages/linux-next.html || linux-next-gitAUR

- **DRM** — 包含前沿 GPU 驱动程序的 Linux 内核。

: https://gitlab.freedesktop.org/drm || linux-drm-tip-gitAUR、linux-drm-next-gitAUR

- **Longterm** — 长期支持（LTS）的 Linux 内核和模块。

: https://www.kernel.org/ || linux-lts612AUR、linux-lts66AUR、linux-lts61AUR、linux-lts515AUR、linux-lts510AUR

非官方内核

- **linux-lily** — 本站站长 Lilydjwg 使用的内核，包含 cjktty、Android binder 和默认 BBR。这是一个较少更新的内核包，可以通过添加 archlinuxcn 仓库来安装。

: https://github.com/archlinuxcn/repo/tree/master/archlinuxcn/linux-lily || linux-lilyCNRepo

- **Libre** — 移除了所有专有或被混淆的设备驱动的纯净内核。

: https://www.fsfla.org/ikiwiki/selibre/linux-libre/ || linux-libreAUR

- **Liquorix** — 基于 Zen 内核源码并使用 Debian 配置方案构建的替代内核，专为桌面、多媒体和游戏场景优化，常用作 Debian 性能增强内核。维护者 Damentz 同时也是 Zen 补丁集的开发者。

: https://liquorix.net || linux-lqxAUR

- **pf-kernel** — 包含多项未合并入主线的前沿特性，由内核工程师维护。若补丁未正式移植到新版内核，项目将提供支持。当前主要特性包含 UKSM、DDCCI、v4l2loopback 和 BBRv3。

: https://pfkernel.natalenko.name || 软件包：

- 通过 post-factum kernels 仓库获取或使用 linux-pfAUR（由开发者 post-factum 维护）
- **Project C** — 包含 Alfred Chen 的 Project C 补丁集（含 BMQ 和 PDS 调度器）。

: https://gitlab.com/alfredchen/projectc || linux-prjcAUR

- **Nitrous** — 针对 Skylake 及更新架构优化的修改版内核。

: https://gitlab.com/xdevs23/linux-nitrous || linux-nitrousAUR

- **Tachyon** — Intel Clear Linux 项目的补丁，由于 Clear Linux 项目已被放弃，现已交接给新项目。提供性能和安全优化。

: https://git.staropensource.de/StarOpenSource/Linux-Tachyon || linux-tachyonAUR

- **tkg** — 高度可定制的内核构建系统，提供多项性能优化补丁和调校选项（含 CFS、Project C PDS/BMQ、MuQSS 和 CacULE 调度器），由 Etienne Juvigny 维护。

: https://github.com/Frogging-Family/linux-tkg || 通过 chaotic-aur 获取

- **VFIO** — 包含 Alex Williamson 开发的补丁（acs override 和 i915），用于在特定设备上实现 KVM PCI 直通功能。

: https://lwn.net/Articles/499240/ || linux-vfioAUR、linux-vfio-ltsAUR

- **XanMod** — 专为高性能工作站、游戏主机和媒体中心优化，采用 BFQ I/O 调度器、TCP BBRv3 拥塞控制、x86_64 高级指令集支持和部分 Clear Linux 补丁集。

: https://xanmod.org/ || linux-xanmodAUR、linux-xanmod-ltsAUR、linux-xanmod-rtAUR、linux-xanmod-boreAUR

- **linux-cachyos** — 由 CachyOS 开发的 SCHED-EXT + BORE + Cachy Sauce 增强内核，包含多项优化补丁。

: https://github.com/CachyOS/linux-cachyos || linux-cachyosAUR

这里的许多非官方内核都包含需要手动启用的功能，请阅读补丁内附带的文档（其中许多已经包含了对内核源码的 `Documentation/` 目录的更改）或在网络上搜索补丁名称。

疑难解答

内核 panic

当 Linux 内核进入不可恢复的故障状态时，会触发内核 panic（*kernel panic*）。该状态通常由存在缺陷的硬件驱动引发，导致系统死锁、无响应并需强制重启。在死锁发生前，内核会生成诊断信息，包含以下内容：故障发生时的机器状态（*machine state*）、指向识别故障的内核函数的调用轨迹（*call trace*），以及当前已加载的模块列表。值得庆幸的是，使用主线版本内核（例如官方仓库所提供的）时，内核 panic 概率较低。但仍需掌握遭遇此情况时的应对方法。

注意：内核 panic 有时也被称作 oops 或内核 oops（*kernel oops*）。尽管 panic 和 oops 均由故障状态引发，但 oops 的范畴更广——其不一定导致系统死锁：某些情况下内核可通过终止问题进程继续运行。

提示：通过以下方式之一可将可恢复的 oops 强制转为 panic：

- 启动时添加内核参数 `oops=panic`
- 向 `/proc/sys/kernel/panic_on_oops` 写入 `1`

此设置适用于需严格排查因 oops 恢复可能导致的隐蔽性系统不稳定的场景。

检查 panic 信息

若内核 panic 发生在启动流程早期，控制台可能显示含“Kernel panic - not syncing:”的提示信息，而一旦 Systemd 开始运行，内核消息**应**被捕获并写入系统日志。然而，由于 panic 发生时系统会在 `systemd-journald` 记录前即死锁，故内核输出的诊断信息几乎从不写入磁盘日志。

蓝屏二维码

自 linux包 6.10（引入了 `drm_panic`）起，内核默认会以蓝屏二维码的形式显示 panic，可通过二维码提供的 URL 查看*堆栈回溯*。对于 Arch Linux，其是指向 https://panic.archlinux.org/panic_report 的链接。该 URL 包含多种信息以及经 gzip 压缩并编码在 URL 片段中的堆栈回溯，这些数据不会被发送到服务器（仅在客户端处理）。

可在这篇论坛帖子查看附有链接和截图的 panic 示例。

欲回到旧方式，请向 `drm` 内核模块传递 `panic_screen=kmsg` 参数（或直接传递 `drm.panic_screen=kmsg` 内核参数）以在控制台显示堆栈回溯。

C控制台

在触发时通过控制台实时观察以查看 panic 信息（不配置 *kdump crashkernel* 的情况下）的旧方式依旧可行。可通过以下内核参数启动系统，并尝试在 tty1 复现 panic：

```
systemd.journald.forward_to_console=1 console=tty1
```

提示：若 panic 信息滚动过快难以查看，可尝试添加内核参数 `pause_on_oops=*seconds*`（*seconds* 为暂停秒数）来冻结屏幕输出。

示例：问题模块分析

通过诊断信息可推测引发 panic 的子系统或模块。本例展示某虚拟设备启动时发生的 panic，请关注**粗体**标记的关键行：

```
kernel: BUG: unable to handle kernel NULL pointer dereference at (null) 1
kernel: IP: fw_core_init+0x18/0x1000 [firewire_core] 2
kernel: PGD 718d00067
kernel: P4D 718d00067
kernel: PUD 7b3611067
kernel: PMD 0
kernel:
kernel: Oops: 0002 [#1] PREEMPT SMP
kernel: Modules linked in: firewire_core(+) crc_itu_t cfg80211 rfkill ipt_REJECT
nf_reject_ipv4 nf_log_ipv4 nf_log_common xt_LOG nf_conntrack_ipv4 ... 3
kernel: CPU: 6 PID: 1438 Comm: modprobe Tainted: P           O    4.13.3-1-ARCH #1
kernel: Hardware name: Gigabyte Technology Co., Ltd. H97-D3H/H97-D3H-CF, BIOS F5
06/26/2014
kernel: task: ffff9c667abd9e00 task.stack: ffffb53b8db34000
kernel: RIP: 0010:fw_core_init+0x18/0x1000 [firewire_core]
kernel: RSP: 0018:ffffb53b8db37c68 EFLAGS: 00010246
kernel: RAX: 0000000000000000 RBX: 0000000000000000 RCX: 0000000000000000
kernel: RDX: 0000000000000000 RSI: 0000000000000008 RDI: ffffffffc16d3af4
kernel: RBP: ffffb53b8db37c70 R08: 0000000000000000 R09: ffffffffae113e95
kernel: R10: ffffe93edfdb9680 R11: 0000000000000000 R12: ffffffffc16d9000
kernel: R13: ffff9c6729bf8f60 R14: ffffffffc16d5710 R15: ffff9c6736e55840
kernel: FS:  00007f301fc80b80(0000) GS:ffff9c675dd80000(0000) knlGS:0000000000000000
kernel: CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
kernel: CR2: 0000000000000000 CR3: 00000007c6456000 CR4: 00000000001406e0
kernel: Call Trace:
kernel:  do_one_initcall+0x50/0x190 4
kernel:  ? do_init_module+0x27/0x1f2
kernel:  do_init_module+0x5f/0x1f2
kernel:  load_module+0x23f3/0x2be0
kernel:  SYSC_init_module+0x16b/0x1a0
kernel:  ? SYSC_init_module+0x16b/0x1a0
```

```
kernel:  SyS_init_module+0xe/0x10
kernel:  entry_SYSCALL_64_fastpath+0x1a/0xa5
kernel: RIP: 0033:0x7f301f3a2a0a
kernel: RSP: 002b:00007ffcabbd1998 EFLAGS: 00000246 ORIG_RAX: 00000000000000af
kernel: RAX: ffffffffffffffda RBX: 0000000000c85a48 RCX: 00007f301f3a2a0a
kernel: RDX: 000000000041aada RSI: 000000000001a738 RDI: 00007f301e7eb010
kernel: RBP: 0000000000c8a520 R08: 0000000000000001 R09: 0000000000000085
kernel: R10: 0000000000000000 R11: 0000000000000246 R12: 0000000000c79208
kernel: R13: 0000000000c8b4d8 R14: 00007f301e7fffff R15: 0000000000000030
kernel: Code: <c7> 04 25 00 00 00 00 01 00 00 00 bb f4 ff ff ff e8 73 43 9c ec 48
kernel: RIP: fw_core_init+0x18/0x1000 [firewire_core] RSP: ffffb53b8db37c68
kernel: CR2: 0000000000000000
kernel: ---[ end trace 71f4306ea1238f17 ]---
kernel: Kernel panic - not syncing: Fatal exception 5
kernel: Kernel Offset: 0x80000000 from 0xffffffff810000000 (relocation range:
0xffffffff800000000-0xffffffffffbfffffff
kernel: ---[ end Kernel panic - not syncing: Fatal exception
```

1. 指出引发 panic 的错误类型，本例为程序缺陷
2. 显示 panic 发生在 `firewire_core` 模块的 `fw_core_init` 函数
3. 显示最近加载的模块是 `firewire_core`
4. 显示调用 `fw_core_init` 的上层函数是 `do_one_initcall`
5. 确认该 oops 实际已成为 panic，系统进入死锁

由此可推断 panic 发生于 `firewire_core` 模块加载时的初始化例程（可能是该版本 FireWire 驱动存在程序错误，导致与硬件不兼容）。临时解决方案是阻止该模块加载：

- 若模块在 initramfs 阶段加载：使用内核参数 `rd.blacklist=firewire_core` 重启
- 其他情况：使用内核参数 `module_blacklist=firewire_core` 重启

调试回归问题

参见常规故障排除#处理软件倒退问题。

可尝试安装 linux-mainlineAUR 以确认问题是否已在上游修复。软件包评论区置顶说明中提及的预编译内核仓库可避免耗时的手动编译。

若问题非近期出现，建议测试 Arch Linux Archive 中的旧版 LTS 内核（linux-lts包）。

若问题仍存，请对 linux-gitAUR 内核进行二分排查（英语：bisect）（bisect），并按内核项目回归问题报告流程提交错误报告。具体报告途径需根据 C0 文件中标注的 Bugtracker（B：条目）选择：子系统邮件列表、Kernel Bugzilla 或 DRM Gitlab 等平台。测试时需确保使用未打补丁的「原版」内核以排除第三方修改影响。若确认补丁引发问题，请直接联系补丁作者。

注意：由于需要多次编译不同版本内核，内核二分排查可能耗时较长。

精简内核构建

通过以下方式可显著缩短内核编译时间：

- 使用 modprobed-db（英语：modprobed-db） 工具仅编译本地系统所需的模块
- 运行 `make localmodconfig` 自动生成最小化配置

调试特定子系统时（如网络问题），可完全移除无关驱动（例如声卡驱动）以进一步精简配置。

参见

- O'Reilly - Linux Kernel in a Nutshell（免费电子书）
- What Stable Kernel Should I Use?（Greg Kroah-Hartman 撰文）
- Linux 内核官方文档

2.15 Sysctl

难度 高级 | **篇幅** 22.9 KB | **所属** 内核

本节将讲到

- 安装
- 配置
- 安全
- 网络
- 虚拟内存

sysctl 是一种用于在运行时检查和更改内核参数的工具。sysctl 在 procfs 中实现，procfs 是位于`/proc/`的虚拟进程文件系统。

安装

procps-ng包 包应该已经安装了，因为它是 base包 元包的依赖项。

配置

sysctl 预加载/配置文件可以在 `/etc/sysctl.d/99-sysctl.conf` 中创建。对于 systemd，`/etc/sysctl.d/` 和 `/usr/lib/sysctl.d/` 是内核 sysctl 参数的配置目录。命名和源目录决定了处理的顺序，这很重要，因为处理的最后一个参数可能会覆盖前面的参数。例如，`/usr/lib/sysctl.d/50-default.conf` 中的参数将被 `/etc/sysctl.d/50-default.conf` 中的相同参数以及之后从这两个目录处理的任何配置文件覆盖。

要手动加载所有配置文件，可以执行：

```
# sysctl --system
```
这也将输出应用的层次结构。单个参数文件也可以显式加载：

```
# sysctl --load=filename.conf
```
有关详细信息，请参阅新的配置文件，更具体地说是 sysctl.d(5)。

可用参数列在 `/proc/sys/` 下。 例如，`kernel.sysrq` 参数指的是文件系统上的文件 `/proc/sys/kernel/sysrq`。`sysctl --all` 命令可用于显示所有当前可用的值。

注意：如果您安装了内核文档 (linux-docs包)，您可以在 `/usr/lib/modules/$(uname -r)/build/Documentation/admin-guide/sysctl/` 中找到有关 sysctl 设置的详细信息。在线版本由本文的#参阅部分引用。强烈建议在更改 sysctl 设置之前阅读这些内容。

也可以通过文件操作或使用 sysctl(8) 实用程序更改设置。例如，临时启用Magic SysRq 组合键：

```
# sysctl kernel.sysrq=1
```
或者:

```
# echo "1" > /proc/sys/kernel/sysrq
```
有关 `kernel.sysrq` 的详细信息，请参阅 Linux 内核文档。

要在重新启动之间保留更改，在 `/etc/sysctl.d/99-sysctl.conf` 以及 `/etc/sysctl.d/` 中的配置文件中添加或修改对应的行。

提示：

- 一些参数是否生效可能取决于内核模块，而这些内核模块又可能不会被加载。 例如 `/proc/sys/net/bridge/*` 中的参数依赖于 `br_netfilter` 模块。如果 `br_netfilter` 没有在运行时（或重启后）加载，那么这些参数将*安静的*不生效。参见内核模块。
- 还可以使用内核命令行参数应用 sysctl 参数，方法为对参数名称添加 `sysctl.` 前缀（例如 `sysctl.vm.swappiness=40`）。这样就可以在不修改系统全局配置的情况下应用永久性更改。

安全

参阅安全#内核加固，以及本文的其余部分。

限制用户模式辅助工具的能力

内核通过 modprobe(8) 使用用户模式辅助工具来加载模块，还可以用来进行核心转储。辅助工具只需要一部分能力（capabilities），因此可以对其进行限制。需要用到的能力包括 CAP_SYS_MODULE，CAP_DAC_READ_SEARCH，CAP_SETGID，CAP_SETUID，CAP_SETPCAP 和 CAP_SYS_PTRACE，合在一起就是 `590276`。限制方法如下：

```
/etc/sysctl.d/51-usermodehelpers.conf
kernel.usermodehelper.bset = 590276 0
kernel.usermodehelper.inheritable = 590276 0
```

如果用不到 systemd-coredump 或需要完全禁用核心转储，可以将上文的 `590276` 替换为 `65536`（只允许 CAP_SYS_MODULE），只保留加载内核请求的模块的功能。如果需要禁用模块，但保留核心转储并启用 systemd-coredump，可以使用 `524740` 以移除 CAP_SYS_MODULE（systemd-coredump 不需要该项）。`0` 代表不向辅助工具授予任何能力。

网络

性能优化

增加接收队列的大小

进入系统的网络数据帧在从网卡上的环形缓冲区中取出后，会存放在这个队列中。

在使用高速网卡时增加系统的这个设定值可能有助于防止数据包丢失：

```
net.core.netdev_max_backlog = 16384
```

注意：在 SIP 路由器等实时应用中，此选项需要高速 CPU，否则队列中的数据将过时。

增加最大连接数

可以更改设置内核接受的连接数上限（默认 4096）：

```
net.core.somaxconn = 8192
```

警告：增加此值可能只会提高高负载服务器的性能，并可能导致处理速度变慢（例如单线程阻塞服务器）或工作线程/进程数量不足 。

增加专用于网络接口的内存

**本文或本章节的事实准确性存在争议。**

原因： 本节似乎受到 Cloudflare 博客文章的启发，但 Red Hat 的调优配置建议使用更小的值，并声称“这似乎只在 40Gb 速度下才有必要”。 因此，这些设置似乎对消费级硬件不实用。（在 Talk:Sysctl 中讨论）

默认情况下，Linux 网络堆栈未配置为通过 WAN 链接进行高速大文件传输（即处理更多网络数据包），设置正确的值可以节省内存资源：

```
net.core.rmem_default = 1048576
net.core.rmem_max = 16777216
net.core.wmem_default = 1048576
net.core.wmem_max = 16777216
net.core.optmem_max = 65536
net.ipv4.tcp_rmem = 4096 1048576 2097152
net.ipv4.tcp_wmem = 4096 65536 16777216
```

也可以增加默认的 `4096` UDP 限制：

```
net.ipv4.udp_rmem_min = 8192
net.ipv4.udp_wmem_min = 8192
```

有关详细信息和推荐值，请参阅以下来源：

- http://www.nateware.com/linux-network-tuning-for-2013.html

- https://blog.cloudflare.com/the-story-of-one-latency-spike/

启用 TCP Fast Open

TCP Fast Open 是传输控制协议 (TCP) 的扩展，它通过在发送方的初始 TCP SYN 时使用值 `3` 而不是默认值 `1` 允许传入和传出连接的 TCP 快速打开：

```
net.ipv4.tcp_fastopen = 3
```

调整挂起的连接处理

`tcp_max_syn_backlog` 是挂起连接等*待确认*的最大队列长度。

在碰到 synflood DOS 攻击的情况下，这个队列会很快填满，此时 TCP SYN cookies 将开始允许您的系统继续响应合法流量，并允许您获得阻止恶意 IP 的权限。

如果服务器在高峰时间系统网络过载，您可能需要稍微增加此值：

```
net.ipv4.tcp_max_syn_backlog = 8192
```

`tcp_max_tw_buckets` 表示系统内允许处于 TIME_WAIT 状态的最大 socket 连接数。

达到此数字后，系统将开始销毁处于此状态的 socket 连接。

增加该值可以防止简单的 DOS 攻击：

```
net.ipv4.tcp_max_tw_buckets = 2000000
```

`tcp_tw_reuse` 设置当新时间戳严格大于上一个连接记录的最新时间戳时，TCP 是否应该为新的传出连接重用 TIME-WAIT 状态下的现有连接。

默认值为 `2`，表示仅对回环连接启用。您可以将其设置为 `1` 以对所有连接启用，这有助于避免用完可用的网络 sockets：

```
net.ipv4.tcp_tw_reuse = 1
```

指定在强制关闭 socket 之前需要等待的最终 FIN 数据包时间（秒数）。这完全违反了 TCP 规范，但防止拒绝服务攻击需要这个：

```
net.ipv4.tcp_fin_timeout = 10
```

`tcp_slow_start_after_idle` 设置 TCP 是否应仅对新连接或空闲时间过长的现有连接开启'以默认窗口大小启动连接'。

此设置会破坏持久的单连接性能，可以将其关闭：

```
net.ipv4.tcp_slow_start_after_idle = 0
```

更改 TCP 保活参数

TCP 保活是一种 TCP 连接机制，有助于确定另一端是否已停止响应。TCP 会在一段时间的空闲时间后，多次向网络对端发送包含空数据的保活探测。如果对端没有响应，socket 将自动关闭。默认情况下，TCP 保活进程在发送第一个保活探测之前等待

socket 活动两个小时（7200 秒），然后每 75 秒重新发送一次。只要有 TCP/IP socket 通信在进行并处于活动状态，就不需要保活数据包。

注意：使用以下设置，您的应用程序将在 120 秒后检测失效 TCP 连接 (60s + 10s + 10s + 10s + 10s + 10s + 10s)。

```
net.ipv4.tcp_keepalive_time = 60
net.ipv4.tcp_keepalive_intvl = 10
net.ipv4.tcp_keepalive_probes = 6
```

启用 MTU 探测

最大传输单元 (MTU) 越长，性能越好，但可靠性越差。

这是因为 MTU 设置大了一旦发生数据包丢失就意味着需要重新传输的数据更多，而且互联网上的许多路由器并不能传送非常长的数据包：

```
net.ipv4.tcp_mtu_probing = 1
```

更多信息，可以参阅这篇文章。

TCP 时间戳

警告：TCP 时间戳可以防止 TCP 实现中的（以Gbps速度传输的）序号回绕（sequence numbers wrap-around）问题和时间的重复计算问题，参考 RFC 1323。不建议关闭 TCP 时间戳，因为它可能会导致安全风险。

禁用时间戳生成可以减少低性能尖峰的发生并可能提高千兆网络的性能：

```
net.ipv4.tcp_timestamps = 0
```

TCP 选择性确认

TCP 选择性确认 (TCP SACK)，由布尔值 `tcp_sack` 控制，允许接收方向发送方提供有关丢失段的更多详细信息，从而减少重传量。这在延迟较高的网络上非常有用，但在高速局域网上禁用此功能可以提高吞吐量。如果您不发送 SACK，也请禁用 `tcp_dsack`，因为您肯定不希望发送重复数据！前向确认在 SACK 之上工作，如果 SACK 被禁用，它也将被禁用。

```
net.ipv4.tcp_sack = 1
```

启用 BBR

BBR 拥塞控制算法可以帮助实现更高的带宽和更低的互联网流量延迟。

启用 BBR 需要先加载 `tcp_bbr` 模块。

注意：BBR GitHub says, "这不是 Google 的官方产品。"

```
net.core.default_qdisc = cake
net.ipv4.tcp_congestion_control = bbr
```

增加临时端口范围

**本文或本章节的事实准确性存在争议。**

原因： 此更改可以提高性能？原理是什么？（在 Talk:Sysctl#net.ipv4.ip_local_port_range 中讨论）

通常传输控制协议 (TCP)、用户数据报协议 (UDP) 或流控制传输协议 (SCTP) 会使用 Wikipedia:Ephemeral port 作为客户端与服务器端之间的端口分配协商。

```
net.ipv4.ip_local_port_range = 30000 65535
```

TCP/IP 协议栈加固

下面指定了一个参数集，用于加强 IPv4 协议内核的网络安全选项，并且也存在等效项的相关 IPv6 参数。

对于某些用例，例如将系统用作路由器，其他参数也可能有用或需要。

TCP SYN cookie 保护

此项设置有助于抵御 SYN 洪水攻击。只有访问请求达到 `net.ipv4.tcp_max_syn_backlog` 时才会启动。更多详细信息，例如 。从 linux 包 5.10 开始默认启用。

```
net.ipv4.tcp_syncookies = 1
```

TCP rfc1337

**本文或本章节的事实准确性存在争议。**

原因： 这好像不是 TCP 标准的一部分？ 描述可能不准确。（在 Talk:Sysctl#net.ipv4.tcp_rfc1337 中讨论）

可以防止 tcp time-wait assassination 缺陷，丢弃处于 time-wait 状态的 socket 的 RST 数据包。 除 Linux 之外并不广泛支持，但符合 RFC：

```
net.ipv4.tcp_rfc1337 = 1
```

反向路径过滤

通过启用反向路径过滤，内核将对从机器上所有网络接口接收到的数据包进行来源验证。这可以防止使用 IP 欺骗方法的攻击者破坏系统。

内核的 `net.ipv4 .conf.all.rp_filter` 默认值为 `0`（关闭来源验证），但 systemd 提供默认 `/usr/lib/sysctl.d/50-default.conf` 将其设置到 `2`（宽松模式）。

以下配置将反向路径过滤机制设置为值 `1` （严格模式）：

```
net.ipv4.conf.*.rp_filter = 1
-net.ipv4.conf.all.rp_filter
```

ip-sysctl.html 这里解释了 `net.ipv4.conf.default.*`、`net.ipv4.conf.*interface*.*` 和 `net.ipv4.conf.all.*` 之间的关系和行为。

记录 martian 数据包

martian 数据包是指那些包内的源或目标地址被设置为由互联网号码分配局（Internet Assigned Numbers Authority，IANA）为了特殊用途而保留的地址。更多信息请参考 IP 保留地址。

martian 和无法被路由的数据包可能会用于恶意攻击。记录这些数据包有助于日后进一步进行分析：

```
net.ipv4.conf.*.log_martians = 1
-net.ipv4.conf.all.log_martians
```

注意：这可能会导致大量信息填满您的日志，建议仅在测试时启用它。

禁用 ICMP 重定向

相关背景知识可以参考什么是 ICMP 重定向？ 他们应该被阻止吗?

下面的配置用来禁用 ICMP 重定向的接收功能：

```
net.ipv4.conf.*.accept_redirects = 0
-net.ipv4.conf.all.accept_redirects
net.ipv6.conf.*.accept_redirects = 0
-net.ipv6.conf.all.accept_redirects
```

下面的配置用来在非路由器上禁用 ICMP 重定向的发送功能：

```
net.ipv4.conf.*.send_redirects = 0
-net.ipv4.conf.all.send_redirects
```

忽略 ICMP 回应请求

要禁用 ICMP 回应（又叫 ping）请求：

```
net.ipv4.icmp_echo_ignore_all = 1
net.ipv6.icmp.echo_ignore_all = 1
```

注意：请注意，这可能会导致依赖 ICMP 回显（ping）响应的网络记录工具和或应用程序出现问题。

其它

允许非特权用户创建 IPPROTO_ICMP socket

IPPROTO_ICMP（icmp(7)）socket 类型增加了在不打开 raw(7) socket 的情况下发送 ICMP_ECHO 消息和接收相应 ICMP_ECHOREPLY 消息的功能，该操作需要设置 CAP_NET_RAW 功能权限或具有指定特权用户的 SUID 位。这些 ICMP_ECHO 消息由 ping 应用程序发送，因此除了 ICMP Echo socket 之外，IPPROTO_ICMP socket 也称为 ping socket。

`ping_group_range` 确定允许其用户创建 IPPROTO_ICMP socket 的组的 GID 范围。此外，还允许有 CAP_NET_RAW 功能权限的用户创建 IPPROTO_ICMP socket。默认情况下这个范围是 `1 0`，意味着除了 root 之外没有人被允许创建 IPPROTO_ICMP socket。要利用这项设置，当前使用原始 socket 的程序需要移植到

使用 IPPROTO_ICMP socket。例如，QEMU 将 IPPROTO_ICMP 用于 SLIRP，即用户模式网络，因此允许运行 QEMU 的用户创建 IPPROTO_ICMP socket，这样就可以在客户机内进行 ping 操作了。

下面的配置只允许 GID 为 100 的组成员的用户创建 IPPROTO_ICMP 套接字：

```
net.ipv4.ping_group_range = 100 100
```

下面的配置允许系统中的所有用户创建 IPPROTO_ICMP 套接字：

```
net.ipv4.ping_group_range = 0 65535
```

虚拟内存

有几个关键参数可以调整 Linux 内核的虚拟内存子系统的操作以及将脏数据（dirty data）写出到磁盘。有关详细信息，请参阅官方 Linux 内核文档。例如：

- `vm.dirty_ratio = 10`

: 包括占可用内存总量（可用页面和可回收页面）的百分比，和正在发生磁盘写入的进程本身将开始写出脏数据的页数。

- `vm.dirty_background_ratio = 5`

: 包括可用页面和可回收页面的总可用内存的百分比以及后台内核刷新线程将要开始写出脏数据的页数。

如参数注释中所述，在设置这些值时需要考虑 RAM 总量。例如，可以通过用已安装的系统内存而不是可用内存来简化计算：

警告：

- 较高的比率值可能会提高性能，但也会增加数据丢失的风险。
- 将此值设置为 `0` 可能会导致更高的磁盘延迟和低性能尖峰。

更多信息请参考 。

- 共识是，如果 RAM 为 1 GB，则将 `vm.dirty_ratio` 设置为 RAM 的 10% 是一个合理的值（10% 也就是 100 MB）。 但是如果机器有更多的 RAM，比如 16 GB（10% 就是 1.6 GB），这个百分比可能因为机械磁盘上旋转回写的几秒钟时间差异而不是线性的。 在这种情况下更合理的值可能是 `3`（16 GB 的 3% 大约是 491 MB）。
- 同样，将 `vm.dirty_background_ratio` 设置为 `5` 可能只适用于小内存值，所以要考虑并根据特定系统上的内存容量进行相应调整。

VFS 缓存

降低虚拟文件系统 (VFS) 缓存参数值可能会提高系统响应能力：

- `vm.vfs_cache_pressure = 50`

: 这个值控制内核回收用于缓存目录和 inode 对象（即 VFS 缓存）内存的积极性。 将它从默认值 100 降低会使内核不太愿意回收 VFS 缓存（不要将其设置为 0，这可能会产生内存不足的情况）。

MDADM

参考 RAID#修改同步速度限制。

故障排除

周期性的短时系统冻结

如果遇到这个问题，可以将脏字节（dirty bytes）设置为足够小的值（例如 4 MiB）：

```
vm.dirty_background_bytes = 4194304
vm.dirty_bytes = 4194304
```

注意：`dirty_background_bytes` 和 `dirty_bytes` 参数是 `dirty_background_ratio` 和 `dirty_ratio` 的对应项（如 #虚拟内存 中所示）。一次只能指定一个参数。

参考资料

- sysctl(8) 和 sysctl.conf(5)
- `/proc/sys/` 的内核文档
- 内核文档: IP Sysctl
- sysctl的内核网络参数
- sysctl-explorer.net – 促进使用 Linux 的 sysctl 参考文档的倡议
- Disable Source Routing - Red Hat 客户门户
- SUSE 内核安全特性手册
- Linux sysctl 调优

系统管理

系统管理要解决的是让运维可重复：环境变量与默认应用程序决定程序读到什么、由谁打开，自动启动、Systemd/定时器与 Cron 决定事情何时发生，系统维护管检查错误、备份与清理，调试管故障后怎么往下挖。

系统维护与环境变量必读，是其余内容的地基；调度在 Systemd/定时器与 Cron 中择一精读，自动启动补足各种时机，调试等真出故障再对照；IPMI、Scx-scheds、Windows PE 等专题可跳过。最大的误区是把系统维护等同于升级，省去检查错误与备份；再是把开机、登录与插拔当成同一个启动入口。

3.1　IPMI

难度 入门 | **篇幅** 2.3 KB | **所属** 系统管理

本节将讲到

- 固件
- 参见

智能平台管理接口 (IPMI) 是用于自治计算机子系统的一组计算机接口规范，该子系统提供独立于主机系统的 CPU、固件（BIOS 或 UEFI）和操作系统的管理和监视功能。

Arch Linux 具有以下与 IPMI 相关的软件包：

- **freeipmi** — Sensor monitoring, system event monitoring, power control, and serial-over-LAN (SOL).

: https://www.gnu.org/software/freeipmi/ || freeipmi包

- **openipmi** — Full-function IPMI (Intelligent Platform Management Interface) system.

: http://openipmi.sourceforge.net/ || openipmi包

- **ipmiutil** — A simple program that lists results from the hardware detection library.

: https://sourceforge.net/projects/ipmiutil/ || ipmiutilAUR

- **ipmitool** — Command-line interface to IPMI-enabled devices.

: https://github.com/ipmitool/ipmitool || ipmitool包

- **ipmiview** — Supermicro IPMI tool.

: https://www.supermicro.com/products/nfo/ipmi.cfm || ipmiviewAUR

固件

可以使用 supermicro-update-managerAUR 更新 Supermicro 服务器上的 bmc 固件。

参见

- Wikipedia:out-of-band management

3.2 D-Bus

难度 入门 | **篇幅** 2.8 KB | **所属** 系统管理

本节将讲到

- 替代实现
- 提示与技巧
- 调试
- 另见

D-Bus 是一个为进程间通信提供简单方法的消息总线系统。它由一个可为全系统加载或是为单个用户会话加载的守护进程和一系列供应用程序使用 D-Bus 的库组成。

dbus包 作为 systemd包 的依赖被拉取和安装，并且用户会话总线会为每个用户自动启动。

替代实现

dbus-broker

dbus-broker — A drop-in replacement for the *libdbus* reference implementation, which aims "to provide high performance and reliability, while keeping compatibility to the D-Bus reference implementation".

: https://github.com/bus1/dbus-broker || dbus-broker包

要将 *dbus-broker* 作为系统总线启用，要先禁用 `dbus.service` 然后启用 `dbus-broker.service`

要将其作为用户总线启用，可以为单个或全部用户启用`dbus-broker.service` 用户单元。

最后要重启来让这些设置生效。

提示与技巧

覆盖 dbus 服务

你可以在 `$XDG_DATA_HOME/dbus-1/services` 覆盖 dbus 服务。

如果服务已经被启动了，覆盖工作不会立即生效。必须先杀死已有的服务进程，或者更早地启动服务。

You can use this to always use a particular service when you have several installed providing the same well-known bus name.

调试

- **D-Feet** — Easy to use D-Bus debugger GUI tool. D-Feet can be used to inspect D-Bus interfaces of running programs and invoke methods on those interfaces.

: https://wiki.gnome.org/Apps/DFeet || d-feet包

- **QDbusViewer** — GUI D-Bus debugger. Can be used to inspect D-Bus services and invoke methods on them.

: https://doc.qt.io/qt-5/qdbusviewer.html || qt5-tools包

你也可以用 systemd 的 busctl(1)。

另见

- https://freedesktop.org/wiki/Software/dbus/
- https://freedesktop.org/wiki/IntroductionToDBus/

3.3 Preload

难度 入门 | **篇幅** 2.9 KB | **所属** 系统管理

本节将讲到

- Go-preload
- preload
- 参阅

预加载的主要作用是让把文件放入RAM中，或者让文件保持在RAM中。这样做的好处是让应用预加载从而更快的打开，因为从内存读取要比从硬盘读取速度更快。虽然这样会使用一部分内存，但不会比应用打开后所需的内存多。所以，预加载最好用于大型的，经常打开的应用，比如 Firefox 和 LibreOffice。

Go-preload

gopreload-gitAUR 是个轻量级的守护进程，来自 Gentoo forum。为了使它发光发热，得启动后先在一个终端里为每个你想要preload的程序运行以下命令：

```
# gopreload-prepare program
```

对于普通用户，给予 `/usr/share/gopreload/enabled` 和 `/usr/share/gopreload/disabled` 的所有权：

```
# chown username:users /usr/share/gopreload/enabled /usr/share/gopreload/disabled
```
接着为每个你想要preload的程序“gopreload”：

```
$ gopreload-prepare program
```
接着，按照提示，当程序完全加载好时回车。这样就会
在 `/usr/share/gopreload/enabled` 添加preload所需的列表。如果要在启动时加载列表中所有的程序，启用该systemd服务 `gopreload.service`.

想要禁用一个程序的加载，在 `/usr/share/gopreload/enabled` 中移除你想要的程序或把那个程序移到文件 `/usr/share/gopreload/disabled` 里。

建议在系统升级后运行gopreload prepare以刷新文件列表。 运行以下脚本文件可以非常方便地做到这一点：

```
# gopreload-batch-refresh.sh
```
只需要在不使用系统时运行它。

配置

配置文件位于 `/etc/gopreload.conf`

preload

preload是由Behdad Esfahbod编写的程序，它作为一个守护进程运行，并使用马尔可夫链统计程序的使用情况；在计算机空闲时，使用频率较高的程序的文件会加载到内存中。这会加快程序的启动时间，因为需要从磁盘读取的数据更少。

安装

安装包preloadAUR。您现在可以开启systemd服务`preload`，然后也可以启用这个服务以便在开机时启动。

配置

配置文件位于`/etc/preload.conf`，它包含适合普通用户的默认设置。`cycle`选项允许您配置调用preload的频率，以更新其要缓存的应用程序和库的信息。

参阅

- wikipedia:Preload_(software))
- Improving performance

3.4 自动启动

难度 入门 | **篇幅** 4.6 KB | **所属** 系统管理

本节将讲到

- 开关机时

- 用户登录登出时
- 插入拔出设备时
- 基于时间事件
- 基于文件系统事件

有许多方法可在特定事件发生时自动执行脚本或启动应用程序，本文提供了这些方法的链接。

开关机时

启用相关的 systemd 服务。如果应用程序没有提供 systemd 服务，请自行编写。

用户登录登出时

启用相关的用户单元。

插入拔出设备时

使用 udev 规则。

基于时间事件

在特定时间、日期或间隔定期执行：

- systemd/Timers
- Cron

仅在某日或某时执行一次：

- systemd/Timers
- at包

基于文件系统事件

使用 inotify 事件监视器：

- inotify-tools包，参见 inotifywait(1)
- incron
- fswatchAUR
- entr包
- systemd 可以使用 systemd.path(5) 文件在文件系统事件发生时激活单元

shell 登录登出时

参见命令行解释器#配置文件。

Xorg 启动时

- 如果使用 xinit 手动启动 Xorg，请参见 xinitrc。
- 如果使用显示管理器启动，请参见 xprofile。

桌面环境启动时

大多数桌面环境实现了 XDG Autostart。

如果某个桌面环境有相关文章，请参见其“自动启动”（*Autostart*）章节。

- GNOME#自启动
- KDE#自启动
- Xfce#自动运行
- LXDE#程序自动开启
- LXQt#自动启动

窗口管理启动时

许多窗口管理器（和Wayland 混成器）实现了 XDG Autostart。

如果有相关文章，请参见其“自动启动”（*Autostart*）章节。

- Awesome#Autostart
- dwm#Autostart
- Fluxbox#自启动程序
- FVWM#Autostart
- Hyprland#自动启动
- i3#自启动程序
- IceWM#Autostart
- JWM#Autostart
- Labwc#Autostart
- Niri#自动启动
- Openbox#自动启动
- PekWM#Autostart
- Qtile#Autostart
- Ratpoison#Autostart
- River#Autostart
- Sway#程序自启动

3.5 Scx-scheds

难度 进阶 | **篇幅** 6.9 KB | **所属** 系统管理

本节将讲到

- 安装
- 部署
- 参见

scx-scheds包 是 scheds-ext 的程序实现，**scheds-ext** （Scheduler Extensions）是 Linux 的一个可拓展调度器框架，允许在不修改内核代码的情况下通过 **BPF**（Berkeley Packet Filter）或 eBPF 来实现自定义调度策略，具有少量的性能提升效果。

安装

需要先检查内核是否支持 bpf，使用 root 权限执行指令：

```
# zcat /proc/config.gz | grep -i BPF
```

通常情况下会输出这些内容：

```
# zcat /proc/config.gz | grep -i BPF
CONFIG_BPF=y
CONFIG_HAVE_EBPF_JIT=y
CONFIG_ARCH_WANT_DEFAULT_BPF_JIT=y
.....
```

如果得到如上结果，代表内核支持 BPF。 安装 scx-scheds包 与 bpf包。

部署

scx-scheds 安装后会提供两个 systemd 服务 `scx_loader.service` 和 `scx.service`：`scx.service` 是 scx 调度程序，`scx_loader.service` 则是通过 D-Bus 来实现 scheds-ext 框架加载器和管理器的实用程序。

警告：现在不再推荐使用 `scx.service`，推荐使用 `scx_loader.service`，且不要同时运行，否则它们不会执行任何东西。

警告：在使用 scheds-ext 框架的任何一个调度器时强烈建议禁用并停止使用 ananicy-cpp-gitAUR 等相关的包，因为 anaicy-cpp-gitAUR 等相关的包会干扰系统优先级导致 scheds-ext watchdog 超时，使调度程序被“杀死”。

提示：内核的 Lockdown 模式设置为 `integrity` 并不会影响 `scx-scheds` 调度程序及 dae 代理程序的正常运行。

启用 scx_loader 服务。

scx-scheds包 在安装后会生成一个默认配置文件位于 `/etc/default/scx`，用于 `scx.service`。

注意：`scx_loader.service` 不会使用 `/etc/default/scx` 配置文件。

配置 scx 服务

在 scx.service 中分为两个部分： SCX_SCHEDULER 和 SCX_FLAGS 。

```
/etc/default/scx
# List of scx_schedulers: scx_bpfland scx_central scx_flash scx_lavd scx_layered
scx_nest scx_qmap scx_rlfifo scx_rustland scx_rusty scx_simple scx_userland scx_p2dq
scx_tickless
SCX_SCHEDULER=scx_bpfland

# Set custom flags for each scheduler, below is an example of how to use
#SCX_FLAGS='-p -m performance'
```

SCX_SCHEDULER 为 scx 调度器，总共 14 个调度器，主要调度器有 4 个：scx_bpfland、scx_rusty、scx_flash 和 scx_lavd 。

SCX_FLAGS 是调度器的启动参数，例如：

```
SCX_FLAGS='-p -m performance'
```

-p 参数表示**启用每颗CPU的任务优先级划分**。-m 表示使用模式 performance 代表 gaming 或 lowlatency（低延迟）。

适用环境：scx_rustland 旨在优先处理交互式工作负载，而不是后台 CPU 密集型工作负载，如游戏，实时直播和视频会议等，但不适用于生产环境；scx_lavd 使用了 **LAVD** （延迟关键性感知虚拟截止时间）调度算法，旨在提高 Linux 在游戏上的性能提升，虽然仍在开发，但已经能用于生产环境；scx_bpfland 是 scx_rustland 的改进版本，完全能用于平常使用和生产环境；scx_rusty 为一个多域、BPF/用户空间混合调度器，可以适应不同架构和工作负载，可以投入生产环境，但需要调整。

配置 scx_loader 服务

注意： scx_loader 服务的配置文件是不会自动生成的，需要用户手动配置。

scx_loader 的配置文件有两个：/etc/scx_loader.toml 和 /etc/scx_loader/config.toml 。

文件结构为：

```
/etc/scx_loader/config.toml
default_sched = "scx_bpfland"
default_mode = "Auto"

[scheds.scx_bpfland]
auto_mode = ["-m","performance"]
```

default_sched 意思是默认使用的调度器。scx 和 scx_loader 的默认调度器都是 scx_bpfland，如果字段为空则什么都不启用。 default_mode 指默认启动模式，主要参数有： "Auto"、"LowLatency"、"PowerSave"、"Gaming" 和 "Server"。如果没有参数则为默认模式，即 "Auto" 。 [scheds.scx_name] 指对调度器的自定义标志，把 scx_name 修改为实际的调度程序。参数标志有：auto_mode、gaming_mode、lowlatency_mode、powersave_mode 和

`server_mode`；每个字段是一个字符串且每个字符串表示一个参数标志。如果没有参数标志则使用默认。

注意：`server_mode` 仅 `scx_bpfland` 可用

参考样例：

```
/etc/scx_loader.toml
default_sched = "scx_bpfland"
default_mode = "LowLatency"

[scheds.scx_bpfland]
auto_mode = []
gaming_mode = ["-m", "performance"]
lowlatency_mode = ["-k", "-s", "5000", "-l", "5000"]
powersave_mode = ["-m", "powersave"]
```

提示：如无微调设置需求，仅需设置 `default_sched` 与 `default_mode` 项，因其它项具有默认值。

参数标签

scx_bpfland

- Gaming Mode : `-m performance`
- Low Latency Mode : `-s 5000 -S 500 -l 5000`
- PowerSave Mode : `-m powersave`

scx_rusty

没有自定义参数标志，使用默认参数标志。

scx_lavd

- Gaming Mode：`--performance`
- Low Latency Mode：`--performance`
- AutoPower Mode：`--autopower`
- PowerSave Mode：`--powersave`

scx_flash

没有自定义参数标签，使用默认参数标签。

启动和检查

通过 systemd 启动后，使用 root 权限进行检查。通过命令检查是否启用以及查看调度器：

```
# cat /sys/kernel/sched_ext/state /sys/kernel/sched_ext/*/ops 2>/dev/null
```

回退模式

如果配置文件中未定义特定参数标志， `scx_loader` 会自动使用默认的参数标志。

参见

- CachyOS sched-ext教程

- sched-ext配置
- crates的scx_bpfland
- crates的scx_rusty
- crates的scx_rustland
- crates的scx_lavd

3.6 调试

难度 进阶 | **篇幅** 6.9 KB | **所属** 系统管理

本节将讲到

- 检查核心转储的可用性
- 段错误
- 缺少库或文件
- 非二进制文件
- 报告漏洞

翻译状态：

- 本文（或部分内容）译自 Debugging，最近一次同步于 2024-08-28，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

本页主要是关于如何收集与漏洞报告相关的更多信息。尽管使用了 “调试” 一词，但它并不是在开发时如何调试程序的指南。

检查核心转储的可用性

核心转储是进程意外终止时包含进程地址空间（内存）的文件。如果应用程序以易于调试的方式编译，则可以使用核心转储文件找出问题所在。

核心转储的位置可能因操作系统配置而异。请参阅核心转储以查找系统上是否启用了核心转储文件的生成以及它们的位置。

段错误

有几种方法可用于找出问题所在。准备好成为一个真正的测试员吧！

Gdb

gdb包 是一个古老且经过充分测试的调试器。有关如何使用它来获取跟踪的更多说明，请参见调试/获取跟踪数据#获取跟踪数据。从 `gdb` 运行时，您可能必须等待段错误。然后，将跟踪发布到网络剪切板上并将 URL 包含在你的错误报告中。

如果你有一个核心转储文件，它可以与 gdb 一起使用来获得回溯：

```
$ gdb appname core
bt full
```

Valgrind

假设你有一个没有内联函数的未剔除符号表的二进制文件，通过 valgrind包 运行该程序也是一个好方法。*valgrind* 是一个模拟 CPU 的工具，通常显示哪里出了问题，或者为提供一些 gdb 不能提供的附加信息。

```
$ valgrind appname
```

如果发生崩溃，它将提供许多有用的调试输出。考虑使用 `-v` 和 `--leak-check=full` 参数来获取更多信息。

或者使用：

```
$ valgrind --tool=callgrind appname
```

并通过 kcachegrind包 运行输出，这样就可以以图形方式浏览程序使用的函数。如果程序挂起，这样可以更轻松地查明错误的精确位置。

缺少库或文件

Strace

strace包 会查找应用程序执行的详细操作。如果应用程序尝试访问不存在的文件，则 strace 将会发现该文件。

要查找名为 *appname* 的程序尝试访问哪些文件：

```
$ strace -eopen appname
```

提示：如果你想从 strace 的输出中查找特定信息，你可以运行：`strace -o /dev/stdout *appname* | grep *string*`。这将返回所有 *string* 出现位置周围的输出。

LD_DEBUG

设置 `LD_DEBUG=files` 同样可以大概了解应用程序正在查找的文件。对于名为 *appname* 的应用程序，运行：

```
$ LD_DEBUG=files appname > appname.log 2>&1
```

程序结束后，输出存储在 `*appname*.log` 中。

对于更相信的信息，参见 ld-linux(8)。

Readelf

如果您在运行应用程序时收到 `no such file or directory` 的错误提示，请尝试以下命令：

```
$ readelf -a /usr/bin/appname | grep interp
```

（将 `/usr/bin/*appname*` 替换为可执行文件的位置）

确保有问题的解释器（如 `/lib/ld-linux-x86-64.so.2`）确实存在。如果需要，请安装 ld-lsb包。

非二进制文件

在可执行文件上使用 file包 以获取更多信息：

```
$ file /usr/bin/appname
```

如果它显示 `ELF`，则它是一个二进制可执行文件。如果它显示 `Python script`，说明它是一个用 Python 编写的应用程序。

如果它是一个 shell 脚本，请在文本编辑器中打开 shell 脚本，然后查看（通常在文件底部）是否能找到真实应用程序（ELF 文件）的名称。然后，您可以暂时将 “gdb” 放在 shellscript 中可执行文件名称之前，以便进行调试。请参阅前面关于 gdb 的部分。如果需要调试的可执行文件也需要参数，请在命令前加上 `gdb --args`。

对于纯 shell 脚本，您还可以使用 `bash -x *script_name*` 或 `bash -xv *script_name*` 来调试。

对于 Python 应用程序，输出通常会说明崩溃发生在哪个文件和行号。如果您精通 Python，则可以尝试修复此问题并将修复包含在 bug 报告中。

报告漏洞

首先检查有问题的 bug 是否是打包 bug。如果由于 Arch Linux 打包此应用程序的方式不当而引入该漏洞，请向 https://gitlab.archlinux.org/groups/archlinux/packaging/-/issues 报告。这还包括库或依赖项的问题（例如，如果其中一个并不是被构建特定功能的所需）。

检查包的 PKGBUILD，这可以通过 Arch 构建系统进行，看看它是如何打包的。

有关更多信息，请参见漏洞报告准则#上游还是_Arch? 。

如果该漏洞与 Arch Linux 无关，并且可以在其他任何地方重现，则仅将其报告给上游。Arch Linux 无法神奇地修复上游错误。将其报告给 Arch 错误跟踪器无济于事，甚至可能适得其反，因为它往往会错误地浪费管理者的时间。

参见

- Gentoo:Project:Quality Assurance/Backtraces

3.7 Anything-sync-daemon

难度 进阶 | **篇幅** 7.1 KB | **所属** 系统管理

本节将讲到

- 安装
- 配置
- 使用
- 提示以及技巧
- 支持

翻译状态：

- 本文（或部分内容）译自 Anything-sync-daemon，最近一次同步于 2023-06-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

anything-sync-daemon包 (asd) 是一个小型的、用以将用户指定的目录（同步的目标）移至 tmpfs 并定期同步回硬盘（HDD/SSD）的伪守护进程。其原理为通过 bind 挂载以及创造性地使用 rsync 来同步两处的文件。此外，ASD 有着多种用于崩溃恢复的特性。

ASD 的设计目标以及优点有：

1. 透明的用户体验
2. 减少对硬盘的损耗
3. 速度

当目标目录被移至 tmpfs 之后，相应的读写操作也将从硬盘转移到内存，因而可以减少硬盘损耗，同时提升运行速度与响应速度。内存的访问时间以纳秒计，而硬盘则是以毫秒计，这中间差了六个数量级，或者说，内存比硬盘快出一百万倍。

警告：如果你只想同步浏览器的 profile，我们建议不要使用 ASD，而是使用专门为此设计的、可以检查浏览器运行状况的 PSD。ASD 并不会做这些检查，在某些情况下，浏览器 profile 的数据可能丢失。

安装

安装 anything-sync-daemon包 包。

配置

配置文件在随软件包安装的 `/etc/asd.conf`。

- 至少需要在 `WHATTOSYNC` 数组里定义由 ASD 管理的同步目标。语法见下方。
- 选做：取消 `VOLATILE` 变量的注释并定义 tmpfs 的位置。

- 选做：启用 overlayfs 来进一步提升同步的速度并降低内存的使用。注意该选项需要你的内核进行任一 overlay 内核模块的使用的配置。阅读 #Overlayfs 模式 以了解更多信息。

注意：对于 VOLATILE 设定来说，默认值 `/tmp` 就应该很好了。如果你正在使用 bleachbit包，请不要用 `--clean system.tmp` 选项调用它，否则你会把一个 ASD 用来记录同步状态的关键的文件（.foo）给移除掉。另外，请注意使用设置成 `/dev/shm` 可能会带来问题，此时 systemd 的 NAMESPACE 可能只会在用户开启 overlayfs 选项时出现。

例如：

```
WHATTOSYNC=('/var/lib/monitorix' '/srv/http' '/foo/bar')
```

或者

```
WHATTOSYNC=(
'/var/lib/monitorix'
'/srv/http'
'/foo/bar'
)
```

使用

启动/启用 `asd.service`。此外，程序提供的同步定时器会每小时运行一次，将文件从 tmpfs 同步回硬盘。同步定时器在 `asd.service` 启动后会自动启动，所以不必手动启动。

预览（parse）模式

运行 `asd parse` 来查看在 `/etc/asd.conf` 的设定下 ASD 将会进行或是正在进行什么操作。它同时也会提供目录大小、路径、以及是否存在恢复快照等有用信息。

提示以及技巧

更频繁地进行同步

软件提供了同步的计时器（timer），每小时运行一次。用户可以通过扩展对应的 systemd unit对其重新设定。下面的例子将定时器设定为每十分钟同步一次（注意要先清除 `OnUnitActiveSec` 再设定新的值 ）：

```
/etc/systemd/system/asd-resync.timer.d/frequency.conf
[Unit]
Description=Timer for Anything-sync-daemon - 10min

[Timer]
OnUnitActiveSec=
OnUnitActiveSec=10min
```

查看 systemd.timer(5) 以获取更多选项。

Overlayfs 模式

注意：生产环境下的不同发行版的 Linux 内核有好几个版本的 overlayfs。版本 22 以及更低的版本的模块是“overlayfs”，而新版本（23 以及更高）是“overlay”——注意新版本里没有了“fs”的后缀。如果被设定使用其中一种 overlayfs 的话，ASD 会自动检测你的内核里可用的版本。

Overlayfs 是 Linux 内核版本 3.18.0 里正式引入的简单的联合文件系统。从 ASD 版本 5.54 开始，overlayfs 就可以被用于减低 ASD 的 tmpfs 内存使用并加速同步操作（以及取消同步的操作）。关键在于 overlay 的挂载只会写入更改了的数据写入而不会写入写入整个同步目录。在使用 overlayfs 模式时，ASD 默认启用的崩溃恢复特性也生效。Overlayfs 模式可以通过取消 `/etc/asd.conf` 里的 `USE_OVERLAYFS="yes"` 一行的注释并重启守护进程来开启。

快照

有时候你的同步目录的最新的完好的备份还会在你的文件系统里保存着。在重启时，ASD 会检查上次退出时是否处在某种损坏的状态。如果检测到是的话，ASD 会先将最新的完好备份快照下来，之后再用这份备份恢复。注意，ASD 为了试图减少硬盘占用，它不会把整个目录都复制下来。它实际上使用了硬链接，在同步的时候才创建新的文件并保留原有的硬链接。所以在 ASD 正在备份的时候尝试更改目录可能会导致目录变为某种损坏的状态。

注意：如果真的想的话，用户可以通过在 `/etc/asd.conf` 里取消 `USE_BACKUPS` 变量的注释并设成 `"no"` 来将快照/备份特性直接关闭

快照会和同步目标在一个目录里，并且它的文件名会带有对应了恢复时间的时间戳。例如，`/foo/bar` 的快照会是 `/foo/.bar-backup_asd-crashrecovery-20141221_070112.tar.zstd`——当然你对应的时间戳后缀会不同。

要恢复快照的话：

- 停止 `asd.service`.
- 确认不存在 ASD 创建的目录。如果存在的话，那么 ASD 上次因为某些其它的原因没有正确退出。
- 将损坏的同步目录转移至备份位置（不要盲目地删除任何东西）。
- 将快照存档的 tar 压缩包解压到同步目录。

以 `/foo/bar` 为例：

```
$ cd /foo
$ mv bar bar-bad
$ tar -xvf .bar-backup_asd-crashrecovery-20141221_070112.tar.zstd
```

使用清除（clean）模式清除所有快照

运行 `asd clean` 会删除所有的恢复快照。请只在确定你想要删除的时候再运行这个命令。

注意：如果同步目录的权限在 root 或者另一用户手中而 ASD 被调用来清除快照，那么它会因为同步目标的权限而出错。用户可以通过 sudo 或者以 root 身份调用这个功能来避免此类错误。

支持

请至讨论贴发表评论或提问。

3.8 设备文件

难度 进阶 | **篇幅** 9.4 KB | **所属** 系统管理

本节将讲到

- 块设备
- 伪设备
- 另请参阅

根据维基百科：设备文件在类 Unix 系统中是一个设备驱动程序的接口，并如常规文件一样出现在文件系统中。

根据文件系统层次结构标准 (FHS)，设备文件在 `/dev` 目录中。

在 Arch Linux 上设备节点由 udev 管理。

块设备

块设备是一种特殊文件，能通过缓冲区访问硬件设备。有关详细说明和虚拟文件系统设备之间的比较，请参阅维基百科。

块设备名

设备名的开头指定了内核操作块设备时使用的驱动子系统。

警告：块设备的内核名称描述符不固定，在每次启动时可能会变化。不应在配置文件里使用块设备名。

SCSI

支持 SCSI 命令 (SCSI、SAS、UASP)、ATA (PATA、SATA) 的存储设备或 USB 大容量存储设备，比如机械硬盘、固态硬盘和闪存驱动器，均由内核 SCSI 驱动子系统处理。这些设备的命名规范一致。

这些设备的名称以 `sd` 开头。之后是一个从 `a` 开始的小写字母，`a` 表示第一个发现的设备，`b` 表示第二个发现的设备，以此类推。

例如：

- `/dev/sda` - 设备 `a`，第一个发现的设备。
- `/dev/sde` - 设备 `e`，第五个发现的设备。

NVMe

通过 NVM Express(NVMe) 连接的设备，比如固态硬盘，名称以 `nvme` 开头。之后是一个从 `0` 开始的数字，表示设备控制器号。`nvme0` 表示第一个发现的 NVMe 控制器，`nvme1` 表示第二个发现的，以此类推。然后是字母 "n" 和一个从 `1` 开始的数字，表示控制器上的设备号。比如 `nvme0n1` 表示第一个控制器上第一个发现的设备，`nvme0n2` 表示第一个控制器上第二个发现的设备，以此类推。

例如：

- `/dev/nvme0n1` - 控制器 `0` 上的设备 `1`，第一个控制器上第一个发现的设备。
- `/dev/nvme2n5` - 控制器 `2` 上的设备 `5`，第三个控制器上第五个发现的设备。

MMC

SD 卡、多媒体存储卡 (MMC 卡) 和 eMMC 存储设备由内核的 `mmc` 驱动处理。这些设备的名称以 `mmcblk` 开头，之后是一个从 `0` 开始的数字表示设备号。比如 `mmcblk0` 表示第一个发现的设备，`mmcblk1` 表示第二个发现的设备，以此类推。

例如：

- `/dev/mmcblk0` - 设备 `0`，第一个发现的设备。
- `/dev/mmcblk4` - 设备 `4`，第五个发现的设备。

SCSI 光盘驱动器

通过受 SCSI 驱动子系统支持的接口连接的光盘驱动器，其名称以 `sr` 开头。之后是一个从 `0` 开始的数字，表示设备号。比如 `sr0` 表示第一个发现的设备，`sr1` 表示第二个发现的设备，以此类推。

Udev 也提供到 `/dev/sr0` 的符号链接，名为 `/dev/cdrom`。符号链接的名称 `/dev/cdrom` 与驱动器支持的光盘类型和插入的介质无关。

例如：

- `/dev/sr0` - 光盘驱动器 `0`，第一个发现的光盘驱动器。
- `/dev/sr4` - 光盘驱动器 `4`，第五个发现的光盘驱动器。
- `/dev/cdrom` - 到 `/dev/sr0` 的符号链接。

virtio-blk

连接到 virtio 块设备 (virtio-blk) 接口的驱动器，其名称以 `vd` 开头。之后是一个从 `a` 开始的小写字母，`a` 表示第一个发现的设备 (`vda`)，`b` 表示第二个发现的设备 (`vdb`)，以此类推。

注意：不要将 virtio-blk 与 virtio-scsi 混淆。后者模拟 SCSI 控制器，因此遵循 SCSI 命名规则。

例如：

- `/dev/vda` - 设备 `a`，第一个发现的设备。
- `/dev/vde` - 设备 `e`，第五个发现的设备。

分区

分区设备名是驱动器设备名与在分区表中分配的分区编号的组合 (可表示为 `/dev/*drivepartition*`)。对于设备名以数字结尾的驱动器，需要用字母 "p" 分隔驱动器名和分区编号 (可表示为 `/dev/*drive***p***partition*`)。

例如：

- `/dev/sda1` - `/dev/sda` 上的分区 `1`。
- `/dev/nvme2n5p3` - `/dev/nvme2n5` 上的分区 `3`。
- `/dev/mmcblk3p4` - `/dev/mmcblk3` 上的分区 `4`。
- `/dev/vda1` - `/dev/vda` 上的分区 `1`。
- `/dev/loop0p2` - `/dev/loop0` 上的分区 `2`。

实用工具

lsblk

util-linux包 软件包提供了列出块设备的实用工具 lsblk(8)，例如：

```
$ lsblk --fs
NAME   FSTYPE   LABEL       UUID                                 MOUNTPOINT
sda
├─sda1 vfat                C4DA-2C4D                            /boot
├─sda2 swap                5b1564b2-2e2c-452c-bcfa-d1f572ae99f2 [SWAP]
└─sda3 ext4                56adc99b-a61e-46af-aab7-a6d07e504652 /
```

在上述例子中，只有一个设备可用 (`sda`)。此设备有 3 个分区 (从 `sda1` 到 `sda3`)，每个分区都有不同的文件系统。

wipefs

wipefs 能列出或擦除指定设备上的文件系统、RAID 或分区表签名 (魔术字符串)，从而使签名对 libblkid(3) 不可见。它不会擦除文件系统自身，也不会擦除设备上的任何其他数据。

更多信息请参阅 wipefs(8) 。

例如，以下命令将擦除设备 `/dev/sdb` 的所有签名，并给每个签名创建签名备份文件 `~/wipefs-sdb-*offset*.bak`：

```
# wipefs --all --backup /dev/sdb
```

伪设备

无物理设备的设备节点。

- /dev/random，参见 random(4)
- /dev/shm
- /dev/null，参见 /dev/zero 和 null(4)
- /dev/full，参见 full(4)
- /dev/ttyX (其中 X 是一个数字)

另请参阅

- Linux 内核文档 - Linux 已分配的设备 (English)
- Gentoo：设备文件 (English)

3.9 Windows PE

难度 进阶 | **篇幅** 12.3 KB | **所属** 系统管理

本节将讲到

- 使用场合
- 创建可启动的 Windows PE 镜像
- 引导启动 Windows PE
- 从 Windows PE 安装 Windows
- 问题解决

Windows PE.aspx) 是 Windows 的轻量级版本，旨在用于安装 Windows Vista 或更高版本的 Windows，或是用于系统维护。它完全运行在内存中，并可以从网络启动。本页面描述了如何创建定制化的 Windows PE 镜像文件（也可选择发布在网络上的），我们只需用到 Arch Linux 设备上的自由软件包和微软 Windows 自动安装工具包（Microsoft's Windows Automated Installation Kit，简称 WAIK）。WAIK 可以免费下载，并且我们只需解压其中的 `boot.wim` 文件，该文件包含了 Windows PE 的最初备份, 还有一些启动文件。

警告：下载 Windows Automated Installation Kit 时, 您可能会受到其许可的约束，这会阻止您将 Windows PE 用作通用操作系统。

使用场合

通常，只能使用 Windows 计算机上的 Windows 自动安装工具包 (WAIK) 创建 Windows PE 的映像。但是，也可以使用 (Arch) Linux 机器创建和修改 Windows PE 的映像，并可选择将它们发布到网络上以进行 PXE 引导。不使用 Windows 机器是可能的。在以下情况出现时，您可能想要这样做：

- 您需要使用 Arch Linux 的服务器从网络安装 Windows ，或者从网络启动 Windows PE 来进行系统管理。这样做可能是因为您没有基于 Windows 的服务器，或者因为绝佳的安全性和可配置特性就是喜欢用 Linux 服务器，又或是您因为别的原因已经拥有了一台 Linux 服务器。
- 您需要在 Windows 环境下运行 Win32 应用程序，但您并没有可用的 Windows 设备，也不想用 Wine 或者您的程序在 Wine 下没有正确执行。

创建可启动的 Windows PE 镜像

安装 wimlib包.

配置 Windows PE 镜像

要启动到命令提示符，请创建一个启动脚本，该脚本将包含在下一步的可启动映像中：

```
start.cmd
cmd.exe
pause
```

`mkwinpeimg` 脚本支持使用 `--overlay` 选项对 Windows PE 进行进一步修改。 有关 `mkwinpeimg` 的更多信息，请参阅它的手册页。您可能希望这样做以添加要在 Windows PE 中运行的 Windows 应用程序或者额外的驱动程序。(可以用 Windows PE 中的命令 `drvload` 加载所需的驱动程序).

获取 Windows ISO 或 WAIK 镜像

`mkwinpeimg` 可以从各种来源创建可启动的 Windows PE ISO。您首先需要获得：

- **选项 A** （适用于 Windows 7 及更高版本）: Windows 安装镜像ISO文件。它可以在微软网站上免费下载： Windows 10 ISO. 这是最简单的方法。
- **选项 B**: Windows自动安装工具包镜像 (WAIK) 文件。 对于 Windows Vista/7, 它可以在此处下载：一个单独的 WAIK 发行版. 从 Windows 8 开始, WAIK 被重命名为 WADK 并通过 `adksetup.exe` 分发。

警告：确保根据您的实际需要选择正确的目标架构: x86 (32 位) 或 x64 (64 位)。在 x64 版 Windows PE 上运行 x86 程序并没有那么简单。

不同版本的 Windows 安装介质包含不同版本的 Windows PE。Windows 版本和 Windows PE 版本之间的关系，请参考维基百科.

提示：也许可以使用 httpfs 来避免下载整个镜像文件。Windows PE 只占其中的 118MB。

准备可启动的 Windows PE ISO

成功获取 Windows 安装镜像或者 WAIK/WADK 镜像后，你需要挂载它。假设它叫做 `winimg.iso`:

```
# mkdir /media/winimg
# mount winimg.iso /media/winimg
```

使用 wimlib包 提供的 `mkwinpeimg` 脚本创建可启动的 Windows PE ISO 文件 `winpe.iso`, 启动脚本在上一节中已经创建:

选项 A: 源镜像是 Windows 安装介质

```
$ mkwinpeimg --iso --windows-dir=/media/winimg --start-script=start.cmd winpe.iso
```

选项 B: 源镜像是 WAIK/WADK

```
$ mkwinpeimg --iso --waik-dir=/media/winimg --start-script=start.cmd winpe.iso
```

有关更多信息，请参阅 mkwinpeimg(1), 包含 `--overlay` 选项以把文件复制到镜像中。

卸载源 ISO:

```
# umount /media/winimg
```

为 UEFI 系统准备 Windows PE 启动U盘

不幸的是，从 1.13.1-1 版的 wimlib包 开始，`mkwinpeimg` 无法构建可引导的 UEFI 系统，但是我们依然可以构建包含必要的 UEFI 启动文件的 Windows 10 安装 ISO。

警告：确保为 Windows ISO 映像（x86 或 x64）选择了合适的体系结构。本指南仅在 x64 上进行过测试，即它使用 `/efi/boot/bootx64.efi`。能否以这种方式启动 Windows 的 x86 映像尚不得而知。如果您可以测试，请更新此页面。

在启动盘上, 创建一个 GPT 分区表，其中包含一个类型为“EFI System”的 FAT32 分区。这可以通过 `cfdisk` 或 fdisk 交互式地完成，比如使用 fdisk :

```
# fdisk /dev/sdX
(fdisk) g  # 建立新的 GPT 分区表
(fdisk) n  # 使用默认设置创建新分区
(fdisk) t  # 修改分区类型
(fdisk) 1  # 类型 1 表示 "EFI System Partition"
(fdisk) w  # 保存并退出
```

然后把此分区格式化为 FAT32:

```
# mkfs.vfat /dev/sdX1
```

挂载由 `mkwinpeimg` 创建的 `winpe.iso` 文件，挂载你的启动盘, 复制以下内容:

```
# mkdir /media/{winpe,usb}
```

```
# mount winpe.iso /media/winpe
# mount /dev/sdX1 /media/usb
# cp -r /media/winpe/* /media/usb/
```

最后，挂载原始 Windows ISO 文件并复制 `efi` 中的所有文件:

```
# mount winimg.iso /media/winimg
# cp -r /media/winimg/efi /media/usb/
```

现在，你可以卸载所有 ISO 和启动盘，你的启动盘已经准备就绪。

注意：不需要 syslinux 或 grub magic。UEFI 固件会找到 FAT 分区并加载 Windows 提供的 `/efi/boot/bootx64.efi` 引导文件。

引导启动 Windows PE

按照上一节所述创建可启动的 Windows PE ISO (`winpe.iso`) 之后，你也许想通过如下几种方式引导启动 Windows PE 。

在虚拟机中

在虚拟机中运行 `winpe.iso` 作为 CD-ROM。确保分配了足够的内存，绝对要大于 ISO 文件的大小，因为 Windows PE 是从内存启动的。

使用启动盘

如果你已经如前文所述为 UEFI 系统准备好了启动盘，你只需从启动盘启动即可。启动过程可能需要耗费一些时间（10至20秒是正常的，具体取决于启动盘的读取速度），因为加载程序需要把数据复制到内存中。

使用光盘

只需把 `winpe.iso` 烧录到光盘上，就可以从该光盘启动 Windows PE。

从网络启动

在 BIOS 系统中，Windows PE 可以从网络启动，通过 PXELINUX 和它的 MEMDISK 模块。对于 UEFI 系统,可以使用 wimboot 和 iPXE。

安装 syslinux包 和 tftp-hpa包。

复制需要的 PXELINUX 文件到 TFTP server 的根目录。

```
# rsync -aq /usr/lib/syslinux/bios/ /var/tftpboot/
```

把 `winpe.iso` 文件放进 TFTP server 的根目录。

```
# mv winpe.iso /var/tftpboot
```

创建 PXELINUX 的配置文件，类似这个:

```
/var/tftpboot/pxelinux.cfg/default
UI         menu.c32
MENU TITLE Network Boot
TIMEOUT    50

LABEL      winpe
```

```
MENU LABEL Boot Windows PE from network
KERNEL     /memdisk
INITRD     winpe.iso
APPEND     iso raw

LABEL      localboot
MENU LABEL Boot from local disk
LOCALBOOT  0
```

启动 TFTP server.

配置你的 DHCP server (例如 Dhcpd 和 Dnsmasq)，用 Linux 服务器的 IP 地址指向作为引导文件的 `pxelinux.0`。注意：如果您的 DHCP 服务器在路由器上，则可能无法在不安装自定义固件的情况下执行此操作。

完成以上全部步骤后，你将能够从网络启动 Windows PE 。

注意：根据上面给出的 PXELINUX 配置文件, Windows PE 将在 5 秒后默认启动

提示：TFTP 并非为传输大文件而设计, `winpe.iso` 足足有 118MB 或更大，可能需要花费超过 30 秒的时间加载。使用 `gpxelinux.0` bootloader 而不是 `pxelinux.0`，使用 HTTP 而不是 TFTP 加载 `winpe.iso` 或许可以提高性能

从 Windows PE 安装 Windows

启动到 Windows PE 后，您可以从安装介质安装 Windows。

安装介质可以是网络共享 (Samba)。有关在局域网中的另一台机器上设置 Samba 服务器的信息，请参阅 Samba。要分享挂载在 `/media/winimg` 的安装镜像，请加以下内容添加到 `/etc/samba/smb.conf`:

```
/etc/samba/smb.conf
[REMINST]
browsable = true
read only = no
guest ok = yes
path = /media/winimg
```

启动到 Windows PE 命令提示符后，运行以下命令来初始化网络接口，获取 Samba 服务器的 IP (假设 Windows PE 是从运行 DHCP、TFTP 和 Samba 服务器的机器上通过 PXE 启动的，服务器 IP 通常也是网关 IP），挂载网络共享，并运行图形化安装程序:

```
> wpeinit
> ipconfig
> net use I: \\IP.ADDRESS.OF.SAMBA.SERVER\REMINST
> I:\setup.exe
```

问题解决

出现系统错误 58，指定的服务器无法执行请求的操作

如果你在执行 `net use` 命令时看到以下错误提示:

```
System error 58 has occurred.
```

```
The specified server cannot perform the requested operation.
```

1\. 确保你没有误卸载 `/media/winimg` 文件夹.

2\. 添加一个 `map to guest` 到 `/etc/samba/smb.conf` 文件中。即在文件开头添加如下内容:

```
/etc/samba/smb.conf
[global]
map to guest = Bad User
...
```

3\. 重启 `smbd.service`。

4\. 使用命令在 net 中明指定任意一个用户名和对应密码:

```
net use I: \\IP.ADDRESS.OF.SAMBA.SERVER\REMINST /user:user pass
```

发生这种情况，是因为 Windows 10 连接到匿名共享时，需要用一对用户名和密码检查是否能够登录，如果可以，则允许匿名连接。显然，只要有地方阻止用户验证是否可以登录，就无法在 PE 系统中访问 Samba 共享。

参阅

- 关于 Windows PE 的微软文档.aspx)
- 另一篇有关在 Linux 上制作 Windows PE 镜像的文章
- A guide with scripts for unattended installation of Windows 7 from Linux using Windows PE\[失效链接 2022-09-23 ⓘ\]
- Windows 10 PE Unable to map network drive anonymously

3.10 Systemd/定时器

难度 进阶 | **篇幅** 13.2 KB | **所属** 系统管理

本节将讲到

- 定时器单元
- 服务单元
- 管理
- 示例
- 瞬态 .timer 单元

Timers 是以 `.timer` 为后缀名的 systemd 单元文件，用于控制 `.service` 文件或事件。Timers 可用来替换 cron（阅读 #替代 cron）。Timers 内置了实时定时事件和单调定时事件的支持，并可以异步执行这些事件。

定时器单元

Timers 是以 `.timer` 为后缀的 systemd 单元文件。Timers 和其他单元配置文件是类似的，它通过相同的路径加载，不同的是包含了 `[Timer]` 部分。`[Timer]` 部分定义了何时以及如何激活定时事件。Timers 可以被定义成以下两种类型：

- **实时定时器**（亦称“挂钟定时器”）通过日历事件激活（类似于 cronjobs）定时任务。使用 `OnCalendar=` 来定义实时定时器。
- **单调定时器**即在一个时间点经过一段时间后激活定时任务。所有的单调计时器都遵循如下形式： `On*Type*Sec=`。 `OnBootSec` 和 `OnActiveSec` 是常用的单调定时器。

要查阅完整的定时器选项，参见 systemd.timer(5)。关于日历事件和时间段的定义参见 systemd.time(7)。

服务单元

每个 `.timer` 文件所在目录都得有一个对应的 `.service` 文件（如 `foo.timer` 和 `foo.service`）。`.timer` 文件激活并控制 `.service` 文件。对应的 `.service` 文件中不需要包含 `[Install]` 部分，因为这由 *timer* 单元接管。必要时可以通过在定时器的 `[Timer]` 部分指定 `Unit=` 选项来控制一个与定时器不同名的服务单元。

管理

使用 timer 单元只需像其他单元一样 enable 或 start 即可（别忘了添加 `.timer` 后缀）。要查看所有已启用的定时器，运行：

```
$ systemctl list-timers
NEXT                         LEFT        LAST                         PASSED     UNIT
ACTIVATES
Thu 2014-07-10 19:37:03 CEST  11h left    Wed 2014-07-09 19:37:03 CEST  12h ago
systemd-tmpfiles-clean.timer systemd-tmpfiles-clean.service
Fri 2014-07-11 00:00:00 CEST  15h left    Thu 2014-07-10 00:00:13 CEST  8h ago
logrotate.timer              logrotate.service
```

注意：

- 列出所有定时器（包括非活动的），使用下列命令： `systemctl list-timers --all`。
- 一个由定时器启动的服务的状态，如果不是正好被触发的话，通常是未激活的。
- 若一个定时器不再同步，它可能会删除它在 `/var/lib/systemd/timers` 下的 `stamp-*` 文件。这些空文件只用于表示每个定时器上次运行的时间。删除后，他们将在下次定时器运行时自动重建。

示例

通过定时器预定执行的服务文件一般无需任何修改。以下示例将预定执行 `foo.service`，因此它的定时器应该被命名为 `foo.timer`。

单调定时器

定义一个在系统启动 15 分钟后执行，且之后每周都执行一次的定时器：

```
/etc/systemd/system/foo.timer
[Unit]
Description=Run foo weekly and on boot

[Timer]
OnBootSec=15min
OnUnitActiveSec=1w

[Install]
WantedBy=timers.target
```

实时定时器

定义一个每周执行一次（具体来讲，指周一凌晨零点）的定时器。如果上次未执行（比如说系统当时没有开机，这个行为由 `Persistent=true` 定义）就立即执行服务。

```
/etc/systemd/system/foo.timer
[Unit]
Description=Run foo weekly

[Timer]
OnCalendar=weekly
Persistent=true

[Install]
WantedBy=timers.target
```

更精确的时间可以通过 `OnCalendar` 参数以下列方式指定：

```
星期 年-月-日 时:分:秒
```

可以用星号来表示任意值，用逗号分列可能值。以 `..` 分隔的两个值指两个值间的连续序列。

这是在每个月的前四天晚上中午 12 点，但仅当这一天是周一或周二时运行的例子：

```
OnCalendar=Mon,Tue *-*-01..04 12:00:00
```

在每个月的第一个星期六运行：

```
OnCalendar=Sat *-*-1..7 18:00:00
```

如果不需要以星期几来界定，比如说在每天四点运行，请去掉星期项：

```
OnCalendar=*-*-* 4:00:00
```

在不同的时间运行一个服务可以以多次指定 `OnCalendar` 的方式表示。在下面的例子中，这个服务在周内的 22:30 和周末的 20:00 运行：

```
OnCalendar=Mon..Fri 22:30
OnCalendar=Sat,Sun 20:00
```

更多信息请参阅 systemd.time(7)。

提示：

- 可以用 *systemd-analyze* 工具检测 `OnCalendar` 项是否填写正确，并计算下一次这个条件成立的时间。比如说可以用 `systemd-analyze calendar weekly` 或 `systemd-analyze calendar "Mon,Tue *-*-01..04 12:00:00"`。
- libfaketime包 包提供的 `faketime` 命令在测试各种情况时非常有用。
- 特殊的事件表达式如 `daily` 和 `weekly` 表示 *特定的启动时间*，因此任何共享该日历事件的定时器将同时启动。如果该定时器的服务会竞争系统资源，那么共享该日历事件的定时器可能会引起系统性能下降。使用 `[Timer]` 部分中的 `RandomizedDelaySec` 选项可以通过随机推迟定时器启动来避免这个问题。参见 systemd.timer(5)。
- 向 `[Timer]` 部分添加 `AccuracySec=1us` 选项可以避免原有默认的精确值 *1 分钟*造成的误差。参见 systemd.timer(5)。

瞬态 .timer 单元

可以使用 `systemd-run` 创建一个瞬态 `.timer` 单元。即可以在不创建服务文件的情况下设置定时运行某个命令。比如说下面的命令可以在 30 秒钟后创建一个文件：

```
# systemd-run --on-active=30 /bin/touch /tmp/foo
```

也可以建立指定一个已经存在但没有定义 timer 的服务文件。比如说，下面的命令会在 12 小时 30 分钟后启动名为 `*someunit*.service` 的服务：

```
# systemd-run --on-active="12h 30m" --unit someunit.service
```

更多信息和例子参见 systemd-run(1)。

替代 cron

尽管 cron 毋庸置疑是最有名的计划任务管理器，*systemd* 定时器仍可以作为一个替代品。

优势

使用定时器的最主要的优势在于每个任务都有它自己的 *systemd* 服务。这样做的好处包括：

- 任务可以简单地独立于他们的定时器启动，简化调试。
- 每个任务可配置运行于特定的环境中（参见 systemd.exec(5)）。
- 任务可以使用 cgroups 特性。
- 任务可以配置依赖于其他 *systemd* 单元。
- 任务会被记录于 *systemd* 日志，便于调试。

注意事项

有些可以用 cron 轻易做到的事情仅仅使用 timer 组件会比较难办：

- 创建过程：相比于在 crontab 中只需添加一行任务，使用 *systemd* 配置计划任务需要创建两个文件并运行好几次 `systemctl` 命令。

- 邮件：目前还没有内置与 cron `MAILTO` 类似的任务失败时发送邮件的功能。可以在每个服务文件中配置 `OnFailure=` 来实现同样的功能。

另外， systemd用户服务定时器默认只会在用户登录时进行。但是，lingering 允许在启动，且用户没有活动登录会话时激活。

- 随机延时：目前没还有内置与 cron 类似的 `RANDOM_DELAY` 功能来指定一个数字用于定时器延时执行。（参见 bug report）。 你不想同时执行的服务必须手动设置它们的定时器。

: > **注意**：`AccuracySec` 选项对于随机错开定时器执行时间是 **没有** 作用的，因为它"会在所有本地定时器单元间同步" (`systemd.timer(5)`)。换句话说，`AccuracySec` 会以相同的量改变所有定时器激活时间。例如，所有 `OnCalendar=daily` 的定时器单元，指定 `AccuracySec=15m` 将同时在 00:00 到 00:15 触发相关的服务。

发送邮件

可以配置 systemd 在单元失败时发送电子邮件。Cron 的 `MAILTO` 会在任务向标准输出或标准错误输出时发送邮件，许多任务只在发生错误时输出。首先需要两个文件：一个用来发送邮件的可执行文件和一个用于启动这个可执行文件的 *.service* 文件。在本例中，可执行文件只是一个由提供 `smtp-forwarder` 的包中给出的调用 `sendmail` 的shell 脚本：

```
/usr/local/bin/systemd-email
#!/bin/sh

/usr/bin/sendmail -t <<ERRMAIL
To: $1
From: systemd <root@$HOSTNAME>
Subject: $2
Content-Transfer-Encoding: 8bit
Content-Type: text/plain; charset=UTF-8

$(systemctl status --full "$2")
ERRMAIL
```

不论你用什么可执行文件，应该都会像这个脚本一样接收至少两个参数：目标邮箱地址和用来获取状态的单元文件名。我们创建的 *.service* 文件会传递这些参数：

```
/etc/systemd/system/status_email_user@.service
[Unit]
Description=status email for %i to user

[Service]
Type=oneshot
ExecStart=/usr/local/bin/systemd-email address %i
User=nobody
Group=systemd-journal
```

其中，`*user*` 是被通知的用户名，`*address*` 是该用户的邮件地址。尽管这个地址被硬编码在代码里，但是报告的单元文件会在实例参数中指定，因此可以作为许多其他单元的邮件服务端。此时你可以 start `status_email_*user*@dbus.service` 服务来检查你能否收信。

成功后，仅需编辑你需要发送失败邮件的服务，向 `[Unit]` 部分添加 `OnFailure=status_email_*user*@%n.service`。`%n` 会向模板传递单元的名字。

注意：

- 如果你根据 sSMTP#Security 配置了 sSMTP 安全，`nobody` 用户会没有访问 `/etc/ssmtp/ssmtp.conf` 文件的权限，`systemctl start status_email_*user*@dbus.service` 命令就会失败。一个解决办法是使用 `root` 作为 `status_email_*user*@.service` 单元的执行者。
- 如果你希望在你的邮件脚本中使用 `mail -s somelogs *address*`，`mail` 会以 fork 的形式启动，systemd 会把在你的脚本退出时一并将这个进程结束掉。可以用 `mail -Ssendwait -s somelogs *address*` 来让 mail 不以 fork 的形式启动。

使用 crontab

有些注意事项中的部分可以通过安装一些将传统 crontab 项目翻译成 timers 配置的包来完成，例如AUR 中的 systemd-crontab-generatorAUR\[损坏的链接：package not found\] 和 systemd-cronAUR 包。他们也可以提供缺失的 `MAILTO` 特性。

同样，与 crontab 一样，可以通过 `systemctl` 来获取一个统一的计划任务视图。参见 #管理章节。

参见

- systemd.timer(5)
- Fedora Project wiki page *systemd* 的实时定时器
- Gentoo wiki section on *systemd* timer services
- **systemd-cron-next** — 一个从 crontab 和 anacrontab 文件生成 timers/services 文件的工具

: https://github.com/systemd-cron/systemd-cron-next || systemd-cron-nextAUR

- **systemd-cron** — 提供 systemd 单元运行 cron 脚本；使用 *systemd-crontab-generator* 转换 crontabs

: https://github.com/systemd-cron/systemd-cron || systemd-cronAUR

- Systemd Timers for Scheduling Tasks

3.11　默认应用程序

难度 进阶 | **篇幅** 13.7 KB | **所属** 系统管理

本节将讲到

- 背景信息
- 资源打开器

不同程序会使用不同的方式实现默认应用程序关联。命令行程序通常使用环境变量，而图形应用程序则倾向于通过调用 GIO) API、Qt API 或执行 `/usr/bin/xdg-open`（xdg-utils 的一部分），使用 XDG MIME 应用程序作为默认。由于 xdg-open 和 XDG MIME 应用程序相当复杂，因此开发了各种替代的资源打开器。下表列出了使用各种方法的应用程序示例。

方法	使用 XDG	应用程序示例	检查关联
GIO 的 GAppInfo	是	Firefox、GNOME 文件、PCManFM、Thunar、Thunderbird、Telegram	`gio mime *mimetype*`
`/usr/bin/xdg-open`	默认	Chromium（打开下载的文件）	`xdg-mime query default *mimetype*`
自定义	通常没有	mc、ranger	–
环境变量	否	man、sudoedit、systemctl	–
D-Bus 的 FileManager1	org.freedesktop.FileManager1	Firefox（打开包含文件的文件夹）、Zotero（显示文件）、Telegram（在文件夹中显示）	–

许多桌面环境和图形文件管理器提供用于配置默认应用程序的图形界面。

注意：XDG MIME 应用程序规范允许 MIME 类型定义为其它类型的子类型或别名。对于没有默认程序的子类型，这种处理方式可能不够直观——会使用其父类型的默认程序。例如，Firefox 可能通过 GIO 将 `application/rss+xml` 解析为由自身打开，就是因为该类型是 `application/xml` 的子类，而 Firefox 正是 `application/xml` 的默认处理程序。

背景信息

程序有时需要使用用户首选的应用程序打开文件或 URI，这需要检测文件类型（通常通过扩展名或魔术数字映射到 MIME 类型），并将其关联到应用程序。

传承自 UNIX 的程序（Heirloom UNIX programs）使用 mime.types 检测 MIME 类型，使用 mailcap 将其关联到应用程序。

资源打开器

- **XDG MIME 应用程序**：遵循 XDG MIME 应用程序规范

- **正则表达式**：允许使用正则表达式将 MIME 类型与应用程序关联
- **URI 支持**：允许将任意 URI 方案（scheme）与应用程序关联

名称	包	XDG MIME 应用程序	正则表达式	URI 支持
xdg-open	xdg-utils	是	否	是
gio(1)	glib2包	是	–	是
mimeopen(1p)	perl-file-mimeinfo包	是	否	否
mimeo	mimeoAUR	是	是	是
mimi	mimiAUR、mimi-gitAUR	否	否	部分
busking	busking-gitAUR	否	是	是
rifle(1)	ranger	否	是	否
handlr	handlr-regex包	是	是	是
clifm	clifmAUR	否	是	部分
mimejs-git	mimejs-gitAUR	否	否	是

xdg-open

xdg-open（xdg-utils 的一部分）遵循 XDG MIME 应用程序规范，并由许多程序使用。

因为 xdg-utils 版本的 xdg-open 比较复杂，默认应用程序错误时可能很难调试，所以出现了许多尝试改进它的替代方案。其中一些替代方案通过直接替换 `/usr/bin/xdg-open` 可执行文件，能够改变大多数应用程序的行为。其它则仅提供选择默认应用程序的一种替代方法。

perl-file-mimeinfo

perl-file-mimeinfo包 提供 mimeopen(1p) 和 mimetype(1p) 两个工具。提供的界面比 xdg-utils包 的对应工具更好一些：

```
# 确定文件的 MIME 类型
$ mimetype photo.jpeg
photo.jpeg: image/jpeg

# 关联文件类型到默认应用程序
$ mimeopen -d photo.jpeg
Please choose a default application for files of type image/jpeg

    1) Feh (feh)
    2) GNU 图像处理程序 (gimp)
    3) Pinta (pinta)

use application #

# 使用默认应用程序打开文件
$ mimeopen -n photo.jpeg
```

最重要的是，如果没有检测到桌面环境，xdg-utils 会调用 `file` 而不是 `mimetype` 来进行 MIME 类型检测。这很重要，因为 `file` 并不遵循 XDG 规范。

注意：perl-file-mimeinfo包 不完全遵循 XDG 规范。

- 在 0.28-1 版本之前，它不读取发行版范围的默认设置，并且将其配置保存在已弃用的位置。
- 截至 0.35-3 版本，它仍未遵循规范，没有实现 glob 解析优先级、glob 权重和魔术数字优先级。安装该软件包可能会悄然改变 `xdg-mime` 和 `xdg-open` 对于一组与**相同**文件扩展名关联的 MIME 类型的处理行为。

mimeo

mimeoAUR 提供 `mimeo` 工具，兼具 `xdg-open` 和 `xdg-mime` 的功能。

以下示例展示了如何将 SVG 文件与 Inkscape 关联：

```
# 确定文件的 MIME 类型
$ mimeo --mimetype Svg_example2.svg
Svg_example2.svg
 image/svg+xml

# 确定与可执行文件 "inkscape" 相关联的桌面项
$ mimeo --app2desk inkscape
inkscape
 org.inkscape.Inkscape.desktop

# 查找桌面项路径
$ mimeo --finddesk org.inkscape.Inkscape.desktop
/usr/share/applications/org.inkscape.Inkscape.desktop

# 选择此 MIME 类型的默认应用程序
$ mimeo --prefer image/svg+xml org.inkscape.Inkscape.desktop

# 检查关联
$ mimeo --mime2desk image/svg+xml
image/svg+xml
 org.inkscape.Inkscape.desktop

# 使用默认应用程序打开文件
$ mimeo Svg_example2.svg
```

还可以查找 `mimeapps.list` 文件的路径：

```
$ mimeo --mimeapps-list
/home/user/.config/mimeapps.list
```

与 xdg-utils 的一个主要区别是，mimeo 还支持自定义的“关联文件”，允许进行更复杂的关联。例如，可以根据匹配的正则表达式，传递特定的参数：

```
# 在 VLC 中打开 YouTube 链接而不打开新实例
vlc --one-instance --playlist-enqueue %U
  ^https?://(www.)?youtube.com/watch\?.*v=
```

xdg-utils-mimeoAUR 修补了 xdg-utils，使得 `xdg-open` 在未检测到桌面环境时回退到 mimeo。

handlr

handlr-regex包 由 Rust 编写，提供了与 `xdg-open` 和 `xdg-mime` 相同的功能，但界面更加简洁。 handlr-regex包 是 handlrAUR 的一个带有正则表达式支持的分支。

与 `xdg-utils` 相比，handlr 增加了以下功能：

- 通过扩展名设置关联，省去了查找或记住 MIME 类型的必要

- 校验 MIME 类型和扩展名是否正确
- 从 `mimeapps.list` 中删除无效条目
- 扩展名不明确时，通过文件内容智能检测 MIME 类型
- 自动补全 MIME 类型、扩展名和桌面条目
- 设置任意命令作为正则表达式匹配时的处理程序

```
# 以下两个命令是相同的
handlr set .png feh.desktop
handlr set image/png feh.desktop

# 列出默认应用程序
handlr list

# 获取 MIME 类型/扩展名的处理程序
$ handlr get .png --json
{"handler":"pqiv.desktop","name":"pqiv","cmd":"/usr/bin/pqiv"}

# 启动处理程序，可选传递路径/URL
handlr launch x-scheme-handler/https
handlr launch x-scheme-handler/https -- https://google.ca
```

要用 `handlr` 替代 `xdg-open`，可以使用以下脚本遮蔽 `xdg-open` 文件：

```
#!/usr/bin/bash
handlr open "$@"
```

clifm

`clifm` 内置 Lira 作为资源打开器，可以通过 `--open` 命令行选项作为独立资源打开器使用。其配置文件（`~/.config/clifm/profiles/PROFILE_NAME/mimelist.clifm`）支持对 MIME 类型和文件名（及扩展名）使用正则表达式。以下是几个示例：

```
# 打开常规文件
clifm --open /etc/hosts

# 打开目录
clifm --open /media/data

# 打开 URL（通过配置文件中与 text/http MIME 类型关联的应用程序）
clifm --open www.archlinux.org
```

轻量替代品

以下包冲突并提供 xdg-utils包，通过提供自己的 `/usr/bin/xdg-open` 脚本直接替代 xdg-utils。

如果使用这些资源打开器但仍需要使用 xdg-utils包，请将这些资源打开器手动安装到在 PATH 中位于 `/usr/bin` 之前的目录下。

- mimiAUR、mimi-gitAUR —— 130 行 Bash 脚本，可为每种 MIME 类型自定义命令参数

- busking-gitAUR —— 80 行 Perl 脚本，与 `mimi` 类似，额外支持正则表达式规则

run-mailcap

警告：如果使用 run-mailcapAUR，`xdg-open` 在未检测到桌面环境时可能将请求委托给它处理。若同时使用了 mailcap包 默认提供的 `/etc/mailcap`，由于该文件中的条目会调用 `xdg-open`，这将导致无限循环。

3.12 环境变量

难度 进阶 | **篇幅** 19.2 KB | **所属** 系统管理

本节将讲到

- 工具
- 定义变量
- 示例
- 参阅

环境变量是一个具名对象，包含可被一个或多个应用程序使用的数据。简单来说，它是具有一对名称和值的变量。环境变量的值可以是文件系统上所有可执行程序的位置，默认的编辑器，系统本地化设置等。Linux 新用户可能觉得这种管理变量的方式有点混乱。但是环境变量提供了一种在多个程序和进程间共享配置的简单方式。

工具

coreutils包 软件包含程序 *printenv* 和 *env* . 要显示当前环境变量的值：

```
$ printenv
```

注意：一些环境变量是属于特定用户的. 比较一般用户和 *root* 用户的 *printenv* 就可以看到差异。

工具 `env` 可以用指定的环境变量执行一个命令。下面例子把环境变量 `EDITOR` 设置为 `vim` 然后执行命令 *xterm*. 这个操作不会影响全局环境变量 `EDITOR`.

```
$ env EDITOR=vim xterm
```

shell 内置的set(1p) 命令可以设置 shell 选项值以及位置参数，或显示 shell 变量的名称和值。

每个进程都把它们的环境变量保存在 `/proc/$PID/environ` 文件中。此文件包含用 nul (`\x0`) 字符分隔的键值对。用 sed 可以获得用户可读的内容，如：`sed 's:\x0:\n:g' /proc/$PID/environ`.

定义变量

为了避免过多的环境设置，应该限制变量的作用域。图形会话和 systemd 服务要求在明确的位置设置环境变量。变量的作用域可以分为:

- 全局: 任何用户运用的所有程序，不包含 systemd 服务.
- 按用户: 某个用户运行的程序，不包含用户 systemd 服务 (需参考 Systemd/User#Environment variables) 或图形程序(需参考 #图形环境).

全局

使用 shell 初始化文件

大多数 Linux 发行版告诉您在 `/etc/profile` 或其他位置更改或添加环境变量定义。请记住，还有一些特定于包的配置文件，其中包含变量设置，比如 `/etc/locale.conf`. 确保维护和管理环境变量，并注意可能包含环境变量的大量文件。原则上，任何 shell 脚本都可以用于初始化环境变量，但按照传统的 UNIX 约定，这些语句只应出现在某些特定的文件中。

下列文件可用于定义系统上的全局环境变量，每个文件都有不同的限制：

- `/etc/environment` 被 pam_env 模块使用，与 shell 无关，所以无法使用脚本或通配符展开。此文件仅接受 `*variable=value*` 键值对格式。
- `/etc/profile` 仅初始化登录 shell 的变量。但它可以执行脚本(比如 `/etc/profile.d/` 中的脚本) 并支持所有兼容 Bash 的 shell。
- 特定于 shell 的配置文件——您的 Shell 的全局配置文件，初始化变量并运行脚本。例如 Bash#Configuration files （例如 `~/.bashrc`） 或 Zsh#Startup/Shutdown files （例如 `~/.zshrc`）。

以将 `~/bin` 和 `~/scripts` 加入对应用户的 `PATH` 为例，可以将其放入首选的全局环境变量配置文件 `/etc/profile` 或 `/etc/bash.bashrc`：

```
set_path(){

    # Check if user id is 1000 or higher
    [ "$(id -u)" -ge 1000 ] || return

    for i in "$@";
    do
        # Check if the directory exists
        [ -d "$i" ] || continue

        # Check if it is not already in your $PATH.
        echo "$PATH" | grep -Eq "(^|:)$i(:|$)" && continue

        # Then append it to $PATH and export it
        export PATH="${PATH}:$i"
    done
}

set_path ~/bin ~/scripts
```

在不同 shell 间共享环境变量

在不同 shell 间共享环境变量的一种方法是创建一个 `envsubst` 可以直接读取的**不带任何注释、空行或空格（bash）**的文件（源于 ）：

```
.env
EDITOR=vim
XDG_CACHE_HOME=$HOME/.cache
XDG_CONFIG_HOME=$HOME/.config
XDG_DATA_HOME=$HOME/.local/share
XDG_STATE_HOME=$HOME/.local/state
~/.bashrc
export $(envsubst < .env)
~/.config/fish/config.fish
export (envsubst < .env)
```

注意：这种做法的缺点是：变量值不可含空格。比如，不能使用这种做法定义`EDITOR="emacsclient -t"`。

使用 pam_env

PAM 模块 pam_env(8) 从以下文件中按顺序加载要在环境中设置的变量：`/etc/security/pam_env.conf` 和 `/etc/environment`.

注意：这些文件在其他文件之前读取，特别是 `~/.profile`, `~/.bash_profile` and `~/.zshenv` 之前。

警告：读取 `~/.pam_environment` 已被弃用，不再起作用，参考 FS#68945。

`/etc/environment` 必须由每行单独的一个 `*VARIABLE*=*value*` 键值对组成，例如：

```
/etc/environment
EDITOR=nano
```

`/etc/security/pam_env.conf` 使用以下格式：

```
/etc/security/pam_env.conf
VARIABLE [DEFAULT=value] [OVERRIDE=value]
```

`@{HOME}` 和 `@{SHELL}` 是特殊的变量，展开到 `/etc/passwd` 中的定义。以下示例说明了如何将 `HOME` 环境变量展开到另一个变量：

```
/etc/security/pam_env.conf
XDG_CONFIG_HOME   DEFAULT=@{HOME}/.config
```

注意：变量 `${HOME}` 和 `${SHELL}` 默认是未设置的，并不链接到 `HOME` 和 `SHELL` 环境变量。

该格式还允许在其它变量值中使用 `${*VARIABLE*}` 展开已定义的变量，如下所示：

```
/etc/security/pam_env.conf
GOPATH DEFAULT=${XDG_DATA_HOME}/go
```

`*VARIABLE*=*value*` 键值对也是允许的，但不支持变量展开。请参考 pam_env.conf(5).

按用户

有时并不希望定义全局环境变量，比如要把 `/home/my_user/bin` 加入 `PATH` 变量但是不影响其它用户。本地环境变量可以在下面文件定义：

- shell 的用户配置文件，例如 Bash#Configuration files 或 Zsh#Startup/Shutdown files。
- 除非您将变量的范围限制为您打开的终端（例如命令行程序），您需要修改登录 shell。
- systemd 用户环境变量读取自 `~/.config/environment.d/*.conf`。

要修改本地用户的 `PATH` 变量，将以下内容加入 `~/.bash_profile`

```
export PATH="${PATH}:/home/my_user/bin"
```

要更新变量，重新登录或 source 文件： `$ source ~/.bash_profile`

注意：dbus 守护进程和 systemd 用户实例不会继承任何 .bashrc 等文件定义的环境变量。这意味着 dbus 启动的程序如 GNOME Files 默认不会使用这些变量，请参考 Systemd/用户#环境变量。

提示：您可以用 `export -p` 来查看为用户会话设置的全局和局部环境变量。

图形环境

如果环境变量仅影响图形应用程序，您可能希望仅通过在图形会话中设置它来限制其范围。

Xorg 会话

修改 Xorg 会话环境的过程取决于其启动方式：

- 大多数显示管理器读取 xprofile.
- startx 和 SLiM 执行 xinitrc.
- XDM 执行 `~/.xsession`；请参考 XDM#Defining the session.
- LightDM 和 SDDM 为登录 shell 读取额外启动脚本，比如 `~/.bash_profile` 为 bash 或 `~/.zprofile` 和 `~/.zlogin` 为 zsh

虽然脚本的结尾取决于它是哪个文件，且任何高级语法都取决于所使用的 shell，但基本用法是通用的：

```
~/.xprofile, ~/.xinitrc, or ~/.xsession
...
export GUI_VAR=value
...
```

Wayland 会话

由于 Wayland 不启动任何 Xorg 相关文件，因此 GDM 和 KDE Plasma 读取 systemd 用户环境变量。

```
~/.config/environment.d/envvars.conf
GUI_VAR=value
```

其他支持 Wayland 会话的显示管理器（例如 SDDM）还没有提供这种支持。然而，LightDM 和 SDDM 也为 Wayland 会话的登录 shell 读取启动脚本。

greetd 同时读取 `/etc/profile` 和 `~/.profile`——该行为由其 `source_profile` 控制，默认开启。

本文或本章节的语言、语法或风格需要改进。参考：帮助:风格

原因：The following depends on run-parts包.（在Talk:环境变量讨论）

If your display manager sources startup scripts like `~/.bash_profile` and you want to use `environment.d`, you can source it like so:

```
~/.bash_profile
# use systemd-environment-d-generator(8) to generate environment, and export those variables
set -o allexport
source <(/usr/lib/systemd/user-environment-generators/30-systemd-environment-d-generator)
set +o allexport
```

注意：Other generators in `/usr/lib/systemd/user-environment-generators` like `60-flatpak` may not quote the values of environment variables. In this case the output should be sourced with `export -- "$(/usr/lib/systemd/user-environment-generators/60-flatpak)"`

按桌面会话

一些图形环境（如 KDE Plasma）支持在登录时执行 shell 脚本，它们可以用来设置环境变量。示例请参考 KDE#Autostart

按单个程序

要仅为特定应用程序而不是整个会话设置环境变量，请编辑应用程序的 *.desktop* 文件。请参考 Desktop entries#修改环境变量。

可以通过 Steam#Launch options 选项修改 Steam 游戏的环境变量。

按会话或 shell

本文或本章节的事实准确性存在争议。

原因： Since `PATH` is exported by default (at least in Bash; see `declare -p | grep PATH`), both `PATH=...` and `export PATH=...` do the same; it is probably better to use another variable for at least one of these examples.（在 Talk:环境变量 中讨论）

有时只需要临时变量。您可能希望从创建的特定目录临时运行可执行文件，而无需键入每个文件的绝对路径，或者在简短的临时 shell 脚本中使用该路径。

例如，可以在当前 shell 中定义 `PATH` 变量，或者使用 *export* 命令为所有 shell 定义，直到注销会话。添加一个特定会话目录到 `PATH`：

```
$ export PATH="${PATH}:/home/my_user/tmp/usr/bin"
```

添加一个特定 shell 目录到 PATH：

```
$ PATH="${PATH}:/home/my_user/tmp/usr/bin"
```

示例

以下部分列出了 Linux 系统使用的一些常见环境变量，并描述了它们的值。

- XDG_CURRENT_DESKTOP 是一个 freedesktop.org 变量，包含冒号分隔的字符串列表，标识当前 desktop environment . 活跃开发中环境的标准化值有 GNOME, GNOME-Flashback, KDE, LXDE, LXQt, MATE, TDE, Unity, XFCE, EDE, Cinnamon, Pantheon 和 DDE .
- Cinnamon 的注册时间比其他桌面环境晚. 因此，一些软件依旧使用注册前的值 X-CINNAMON, 比如 Qt.
- Hyprland is informally recognized for Hyprland.
- XDG_SESSION_DESKTOP 类似于 XDG_CURRENT_DESKTOP, 但只允许单个字符串。尽管名字包含 XDG，但它并没有由 freedesktop.org 标准化。
- DE 是一个过时的变量，表明使用的桌面环境。没有权威的文档说明可能的值，但 xdg-utils 为许多桌面环境提供了参考。
- DESKTOP_SESSION 类似于 DE，但是用于 LXDE 桌面环境：当 DESKTOP_SESSION 设置为 LXDE, *xdg-open* 会使用 *pcmanfm* 文件关联。
- WINDOW_MANAGER 变量有时用于**选择**桌面环境要使用的 window manager，与其他变量不同，这里的变量是由已经选择的显示管理器或桌面环境设置的，供其他程序读取。
- PATH 包含以冒号分隔的目录列表，系统在其中查找可执行文件。当一个一般命令 (例如 *ls*, *systemctl* 或 *pacman*) 由 shell 解释时 (如 *bash* 或 *zsh*)， shell 从列表包含的目录中寻找与命令同名的可执行程序并执行。要运行不在 PATH 中的应用程序，必须给出可执行程序的相对或绝对路径，例如 ./a.out 或 /bin/ls.

注意：安全起见，建议不要将当前工作目录 (.) 加入 PATH，因为它可能欺骗用户执行恶意命令。

- HOME 包含当前用户主目录的路径。应用程序可以使用此变量将配置文件等与运行它的用户相关联。
- PWD 包含工作目录的路径。
- OLDPWD 包含前一个工作目录的路径，即执行最后一个 *cd* 之前的 PWD 的值。
- TERM 包含正在运行的终端类型，例如 xterm-256color. 它由运行在终端中，希望使用终端特定功能的程序使用。

- `MAIL` 包含传入电子邮件的位置。传统的设置是 `/var/spool/mail/$LOGNAME`.
- `ftp_proxy` 和 `http_proxy` 分别包含 FTP 和 HTTP 代理服务器：

```
ftp_proxy="ftp://192.168.0.1:21"
http_proxy="http://192.168.0.1:80"
```

- `MANPATH` 包含以冒号分隔的目录列表，*man* 命令在其中搜索手册页。

注意：在 `/etc/profile`中，有一条注释说 "Man 比我们更擅长解决这个问题", 因此该变量通常不设置。请参考 manpath(5).

- `INFODIR` 包含以冒号分隔的目录列表，*info* 命令在其中搜索信息页，比如 `/usr/share/info:/usr/local/share/info`
- `TZ` 可用于为用户设置与系统区域不同的时区。`/usr/share/zoneinfo/` 中列出的时区可作参考，例如 `TZ=":/usr/share/zoneinfo/Pacific/Fiji"`. 当 `TZ` 变量指向 zoneinfo 文件时，应以冒号开头 GNU 手册.

默认程序

- `SHELL` 包含用户首选 shell 的路径。请注意，这不一定是当前运行的 shell。在没有值的情况下，Bash 将自动将此变量设置为 `/etc/passwd`中定义的用户登录 shell，如果无法确定，则将此变量设为 `/bin/sh`.
- `PAGER` 包含运行用于列出文件内容的程序的命令，例如 `/bin/less`.
- `EDITOR` 包含运行用于编辑文件的轻量级程序的命令，例如 `/usr/bin/nano`. 本例中，你可以编写 X 下的*gedit* 或 *nano* 的交互式切换：

```
[ -n "$DISPLAY" ] && export EDITOR=gedit || export EDITOR=nano
```

- `VISUAL` 包含运行完整编辑器的命令，该编辑器用于更高要求的任务，如编辑邮件 (例如 `vi`, vim, emacs 等)。
- `BROWSER` 包含 web 浏览器的路径。有助于在交互式 shell 配置文件中进行设置，以便根据图形环境的可用性对其进行动态更改，例如 X:

```
[ -n "$DISPLAY" ] && export BROWSER=firefox || export BROWSER=links
```

提示：这些默认程序也可以根据是否运行 Wayland compositor 而配置，需要检测的是 `WAYLAND_DISPLAY` 环境变量。

参阅

- Gentoo:Handbook:X86/Working/EnvVar
- Ubuntu Community Wiki - Environment Variables

3.13 系统维护

难度 高级 | **篇幅** 20.7 KB | **所属** 系统管理

本节将讲到

- 检查错误
- 备份
- 升级系统
- 使用包管理器安装软件
- 清理文件系统

为了持续使用 Arch Linux，需要进行系统日常维护，每个用户都应该及时维护系统。

检查错误

失败的 systemd 服务

检查是否有 systemd 服务失败:

```
$ systemctl --failed
```

更多信息请参考使用单元。

日志文件

检查 `/var/log/` 目录下的日志文件中的错误信息，同时检查 systemd journal 中记录的信息：

```
# journalctl -b
```

有关 Xorg 的错误日志记录请参阅 Xorg#故障和修复。

备份

应把重要数据的备份作为日常维护任务，因为随着时间推移，数据很可能会由于人为和机器处理错误损坏，且存储数据的物理介质也注定会发生故障。

请参见同步和备份程序寻找最符合实际需要的用于备份的应用程序。其它相关文章请参阅分类:系统恢复。

强烈建议自动化备份并测试恢复流程，以确保一切按预期运行。关于自动化操作，请参阅系统备份#自动化。

配置文件

在编辑任何配置文件前，都先进行备份，这样若出现问题便可以直接还原。可以配置 Vim、Emacs 等编辑器自动进行此操作。在更复杂的应用场景中，可考虑使用配置管理器。

对于 dotfiles（即主文件夹内的配置文件），可参阅 dotfiles#使用 Git 跟踪点文件。

安装的软件包

维护一个所有安装软件包的列表，这样在不得已进行系统重装时，可以很快恢复到初始环境。

详见 pacman/提示和技巧#已安装软件包的列表。

pacman 数据库

参见 pacman/提示和技巧#备份 pacman 数据库。

加密元数据

参见静态数据加密#用于磁盘加密场景的数据备份。

系统和用户数据

参见系统备份。

升级系统

建议定期通过 pacman#升级软件包进行完整系统升级，这样既能享受到最新的问题修复和安全更新，还可以避免一次性处理过多需要手动干预的软件包升级。从社区寻求帮助时，其他人通常会假定系统处于最新状态。

手头上保留 Arch 安装盘或其它 Linux“Live”环境介质，以便在更新后出现问题时能轻松恢复系统。如果在生产环境使用 Arch，或因其它原因不能接受停机，那么在更新前请在非关键系统上测试新的配置文件和软件包，确认没有问题后再部署到生产环境。

如果系统中装有来自 AUR 的软件包，请注意小心地升级每个软件包。

pacman 是一款强大的包管理工具，但并不试图处理所有极端情况。用户必须保持警惕，并承担维护自身系统的责任。

在升级前阅读新闻

升级前建议访问 Arch Linux 主页查看最新消息（中文翻译见 Arch Linux 中文社区主页），或订阅 RSS（中文翻译）、arch-announce 邮件列表 或 Telegram Arch Linux 中文新闻频道。如果升级需要不寻常的（即无法简单地按照 pacman 的输出信息处理的）用户操作介入时，以上信息源总会给出合适的方法。

在更新内核、Xorg、systemd、glibc包 等核心软件之前，查看论坛（中文论坛）了解是否已有任何问题报告。

必须同样意识到的是，升级软件包可能引发**意想不到**的问题，需要用户立即进行一些手动调整。因此，请不要在重要任务前进行系统更新，而要留出足够的时间用于应付可能的问题。

提示：可以使用 informantAUR、newscheckAUR 或 arch-manwarnAUR 一类的 pacman 钩子，避免在阅读最新 Arch 新闻前进行完整系统升级。

避免某些 pacman 命令

避免部分升级——**总是**使用 `pacman -Syu` 而**不要**使用 `pacman -Sy`。

一般情况下，避免使用 pacman 的 `--overwrite` 选项。`--overwrite` 选项接受一个可包含通配符的模式作为参数。使用时，pacman 将跳过匹配模式的文件的冲突检查，直接覆盖文件。在一个维护得当的系统中，该选项应仅在 Arch 开发者明确推荐（参见 #在升级前阅读新闻）时使用。

避免使用 pacman 的 `-d` 选项。`pacman -Rdd *软件包*` 在移除软件包时不会进行依赖关系检查，因此可能仅移除某个提供关键依赖的软件包，从而导致系统损坏。

不支持部分升级

Arch Linux 是滚动更新的发行版：当新版本的库被推送到软件仓库时，开发者和软件包维护者会重新构建仓库中所有需要针对新库重新构建的软件包。如果两个软件包依赖于同一个库，仅升级其中一个软件包时，若需要同时升级该库（作为依赖项），就会破坏另一个依赖于该库旧版本的软件包。

这就是为什么 Arch **不支持**部分升级。**切勿**：

- 单独执行 `pacman -Sy *软件包*`
- 先执行 `pacman -Sy` 再执行 `pacman -S *软件包*`（可以先 `pacman -Sy` 再 `pacman -S**u** *软件包*`）

注意：

- `pacman -Syuw` 命令存在与 `pacman -Sy` 相同的风险，两者都不进行完整系统升级，却会同步 pacman 的数据库。
- pacman-contrib包 提供的 bash 脚本 `checkupdates` 可用于安全地检查已安装软件包的可用更新，而无需进行完整系统升级。脚本选项 `-d` 可以将这些可用更新下载到 pacman 缓存中，但不会同步 pacman 的数据库，所以即使之后尝试使用 `pacman -S *软件包*` 安装软件包也不会导致部分升级问题。

当更新或安装软件包时，**总是**使用 `pacman -Syu` 进行完整升级。

注意：如果 `pacman -Syu` 因错误而未能完成完整升级（即 `-Su` 部分），必须在进行其它包管理操作之前处理错误并完成升级操作，因为数据库已经成功同步（`-Sy` 部分）。

出于同样的原因，使用 `IgnorePkg` 和 `IgnoreGroup` 时要非常小心。如果系统有本地构建的软件包（例如 AUR 包），当其依赖项发生 soname 升级时，需要重新构建。

如果不慎造成了部分升级，导致二进制文件找不到链接库，**不要简单地通过创建符号链接来"修复"问题**。当库不再向后兼容时，会进行 soname 升级。只要 pacman 本身没有损坏，对一个同步正常的镜像执行简单的 `pacman -Syu` 即可解决问题。

注意升级时的提醒

升级系统时，请一定要注意 pacman 输出的注意信息。如果有需要手动操作的，请一定要立即搞定。如果不明白 pacman 输出的信息，请去论坛搜索或检查 Arch Linux 首页的新闻（参见 #升级前阅读新闻）。

处理配置文件更新

pacman 运行时可能会创建 `.pacnew` 和 `.pacsave` 文件并发出通知。这些文件必须及时处理，详细的操作说明请参阅 pacman/pacnew 与 pacsave。

此外，请留意之前复制来的或创建的其他配置文件。例如，若曾将软件包的示例配置文件复制到主目录，请检查示例文件是否更新。

在升级后重启程序或系统

升级通常不立即应用于运行中的程序，只有重新启动程序才能完全应用升级。

可使用 archlinux-contrib包 提供的 checkservices 脚本，其运行 pacdiff 合并 `.pacnew` 文件，然后检查正在运行且使用过期库的进程，并提示用户是否想要重启这些进程。

内核尤其难以应用升级，一般需要重启系统。升级内核后重启系统永远是最安全的选择，但如果非常不便，也可使用内核热补丁在不重启系统的情况下应用更新。

回退有问题的更新

如果已知升级某个软件包可能会引发问题，软件包维护者会确保在 pacman 升级该包时给出提示信息。如果在升级后遇到问题，可以查看 `/var/log/pacman.log` 再次确认 pacman 的输出内容。

提示：可以使用日志查看器（例如 wat-gitAUR）搜索 pacman 日志。

只有在确保无法从 pacman 日志中找到任何相关信息、已 #在升级前阅读新闻且没有关于该更新的论坛帖子之后，才应考虑在论坛（中文论坛）或 IRC 上寻求帮助。降级软件包回退有问题的更新是最后的解决方案。

检查孤立与弃用的软件包

系统升级后，有些软件包可能不再需要，或者已从远程仓库中移除。

用 `pacman -Qtd` 命令查找之前作为依赖安装，现在不再需要的软件包（即孤立软件包）。如果还需要孤立包，可以将安装原因改为“显式安装”；如果不再需要，可以直接将其删除。详见 pacman/提示和技巧#删除未使用的软件包（孤立软件包）。

除此之外，有些软件包可能已从远程仓库中移除，但是还安装在本地（即弃用的软件包）。`pacman -Qm` 命令可以列出所有弃用的软件包。注意输出的列表会包含手动安装的软件（包括来自 AUR 的包）。若要排除那些在 AUR 上可用的软件包（曾通过 AUR 安装，或从远程仓库移至 AUR），可使用 BBS#288205 中提供的脚本，或尝试使用 ancient-packagesAUR 工具。

检查固件更新

除了一般软件，固件也需要日常更新。

fwupd 是可用于 Linux 系统的一个全面的固件更新工具。

使用包管理器安装软件

pacman 在跟踪安装的文件方面远比人工操作更可靠。如果手动安装软件，**迟早会**忘记做过的操作、忘记安装位置、安装冲突的软件、安装到错误的位置等等。

- 使用 pacman#安装软件包中介绍的方法安装官方软件包。
- 如果官方源中没有所需程序，请检查 AUR 中是否有人已为其创建了软件包。如果有，按照 AUR 文章中的方法进行安装。
- 如果在 AUR 中也没有找到，可以考虑为其创建软件包。

要清理自己安装的文件，可以参考查找不属于任何软件包的文件。

选择开源驱动

尽可能选择开源驱动，避免私有驱动。大多数时候，开源驱动比私有驱动更加稳定可靠且开源驱动的问题修改起来更加容易快速。私有驱动尽管提供更多的特性和性能，但往往以牺牲稳定性为代价。为了避免这个两难选择，可以选择已知有功能完整、成熟的开源驱动支持的硬件。关于硬件和开源 Linux 驱动的信息可以在 linux-drivers.org 找到。

谨慎使用非官方软件包

在使用 AUR 或非官方用户仓库中的软件包时，请务必保持谨慎。这些包大多由普通用户提供，可能无法达到与官方仓库相同的质量。在编译或安装软件包之前，**务必**检查软件包的 PKGBUILD 确保其中不含有错误或恶意的代码。

为简化维护，应限制第三方软件包的使用。定期检查实际使用的包，移除不再需要的包（或使用官方仓库中存在的包替代）。pacman/提示和技巧#维护提供了相关实用命令。系统升级后，可使用 rebuild-detector包 识别可能需要重新构建的非官方软件包。

更新镜像列表

镜像的质量可能会随时间变化，有些镜像会下线或者出现同步或下载问题，所以请注意更新 pacman 的镜像列表。

详见镜像源。

清理文件系统

有助于完成此任务的程序可在应用程序列表/工具#磁盘清理中找到。

无用的大文件

在查找要删除的文件时，关键在于找到那些占用最多磁盘空间的文件。有助于完成此任务的程序可在应用程序列表/工具#磁盘使用情况显示中找到。

软件包缓存

删除 `/var/cache/pacman/pkg/` 中不需要的 `.pkg` 文件可以减少空间占用。

详见 pacman#清理软件包缓存。

不需要的软件包

删除系统中不需要的软件包可以减少空间占用并降低维护难度。

参见 #检查孤立与弃用的软件包。

旧配置文件

新版本的软件可能不兼容旧配置文件。请定期清理不需要的配置文件，尤其是在主目录和 `~/.config` 目录。出于类似原因，在不同系统安装之间共享主目录时需格外谨慎。

检查以下目录：

- `~/.config/`——应用程序存储其配置的位置
- `~/.cache/`——某些程序的缓存可能会占用大量空间
- `~/.local/share/`——旧文件可能残留于此

有关这些目录的更多信息请参阅 XDG 基本目录。

为使主目录免受错误创建于其中的临时文件影响，建议管理一个不需要的文件列表并定期清理它们，例如使用 rmshit.py 脚本。

损坏的符号链接

系统中可能残留有需要删除的已损坏的旧符号链接。方法可以参考这里和这里。但是注意一些损坏的符号链接仍有用处，故不应盲目删除所有的符号链接 。

要快速列出系统中全部损坏符号链接（排除运行时与虚拟目录），请使用：

```
# find / -type d \( -path "/dev" -o -path "/proc" -o -path "/run" -o -path "/sys" \) -prune -o -xtype l -print
```

然后检查并删除列表中不必要的文件。

提示：可使用 rmlint-gitAUR 来查找并选择删除重复文件、空文件、递归空目录和损坏的符号链接。

提示与技巧

以下技巧通常并非必需，但对某些用户可能有用。

使用经测试的软件包

Arch 的滚动更新让用户可以使用最新的功能和上游更新，但是这也增加了系统维护的难度。为简化维护并提升稳定性，可以尽量避免使用尖端软件，使用稳定的、经测试的软件包。此类软件包不太可能出现复杂的升级，例如重大的配置变更或功能移除。选择拥有强大活跃开发社区以及大量资深用户的软件，在出现问题时能够更轻松地获得支持。

避免使用 testing 仓库（即使仅仅单个软件包），这些软件包都属实验性质，不适合安装到稳定系统。同样地，也应避免那些直接从上游开发源代码构建的软件包。这些包通常由 AUR 提供，包名上通常带有 `dev`、`devel`、`svn`、`cvs`、`git` 等字样。

安装内核的长期支持版本

可使用 core 仓库中的 linux-lts包 替代默认内核。内核上游开发者为此版本提供长期支持（LTS），包括安全更新和错误修复。适用于由于使用了树外内核模块（如某些专有驱动）并希望确保其兼容性或其它原因需要长期支持的用户，也可以将此内核作为备用内核，以免升级到新内核后出现问题。

要将其添加到启动选项，需要编辑引导加载程序的配置文件，使用 LTS 内核及其内存磁盘映像：`vmlinuz-linux-lts` 和 `initramfs-linux-lts.img`。

参阅

- 用于获取 Arch 新闻的简单 Bash 脚本
- fwupd

3.14 Cron

难度 高级 | **篇幅** 21.9 KB | **所属** 系统管理

本节将讲到

- 安装
- 配置
- Crontab 格式
- 基本命令
- 范例

摘自 Wikipedia:

: *cron* 是一个在 Unix 及类似操作系统上执行计划任务的程序。cron 使用户能够安排工作(命令或shell脚本)在特定时间、日期或间隔定期运行，通常用于系统的自动化维护或者管理。

安装

cron 有多个实现程序，但是基础系统默认使用 systemd 的 Timers，下面的实现都不被默认安装。参见 Gentoo wiki 的 cron 指南，该指南提供了这些实现之间的比较。

可用的包:

- cronie包
- fcron包
- dcronAUR
- vixie-cronAUR
- fcron-devAUR
- scron-gitAUR

配置

激活及开机启动

安装后，默认的守护进程不会启动。安装的软件包通常提供了可以用 systemctl 控制的服务文件。例如 *cronie* 使用 `cronie.service`。

`/etc/cron.daily/` 和类似的目录包含当前的任务，启动 cron 服务时会触发所有这类任务。

注意：*cronie* 提供了 `0anacron` 任务，每小时执行一次，它允许延迟运行其他某些任务，比如因为未开机而延迟的任务。

处理任务中的错误

cron 会记录 *stdout* 和 *stderr* 的输出并尝试通过 `sendmail` 命令发送邮件给用户。如果 Cronie 未找到 `/usr/bin/sendmail`，则会禁用邮件通知。要发送邮件到用户的 spool，需要在系统上运行一个 smtp 守护进程，例如 opensmtpd包。也可以安装提供 sendmail 命令的软件包，然后配置成通过外部邮件服务器发送邮件。或者使用 `-m` 选项将错误记录到日志并通过定制的脚本进行处理。

提示：通过 Postfix/localmail) 可以发送邮件到本地系统。

1. 编辑 `cronie.service` 服务。
2. 安装 esmtpAUR, mSMTP, opensmtpd包, sSMTP 或编写自定义脚本。

使用 sSMTP 的例子

sSMTP 是一个仅包含发送功能的 sendmail 模拟器，可以从本地计算机向 smtp 服务器发送邮件。尽管目前已经没有活跃维护者，这个程序依然是发送邮件的最简单方式。不需要运行守护进程，配置可以简单到只需在一个配置文件中编辑三行即可（如果你的主机是受信任的，可以通过你的邮件服务提供商转发未经认证的邮件）。sSMTP 无法收取邮件、展开别名或管理队列。

安装 ssmtpAUR，安装时会创建链接 `/usr/bin/sendmail` 指向 `/usr/bin/ssmtp`。安装后编辑 `/etc/ssmtp/ssmtp.conf` 配置文件。详情请参考 sSMTP，到 `/usr/bin/sendmail` 的软链接可以确保 S-nail 等提供 `/usr/bin/mail` 的程序可以无需修改直接使用。

重启 `cronie.service` 以确保 cronie 能够检测到新配置的 `/usr/bin/sendmail` 命令。

使用 mSMTP 的例子

安装 msmtp-mta包, 安装时会创建链接 `/usr/bin/sendmail` 指向 `/usr/bin/msmtp`。重启 `cronie.service` 以确保 cronie 能够检测到新配置的 `/usr/bin/sendmail` 命令。你必须提供一种方法让 `msmtp` 能够将你的用户名转换成电子邮件地址。

然后要么在你的 crontab 中添加 `MAILTO` 行：

```
MAILTO=your@email.com
```

要么创建 `/etc/msmtprc` 文件，并添加这一行：

```
aliases /etc/aliases
```

并创建 /etc/aliases 文件：

```
your_username: your@email.com
# 可选:
default: your@email.com
```

然后修改 *cronie* 的配置，将*cronie* 守护进程的 ExecStart 命令替换为

```
ExecStart=/usr/bin/crond -n -m '/usr/bin/msmtp -t'
```

使用 esmtp 的例子

安装 esmtpAUR 和 procmailAUR。

安装完成后，进行如下配置:

```
/etc/esmtprc
identity myself@myisp.com
       hostname mail.myisp.com:25
       username "myself"
       password "secret"
       starttls enabled
       default
mda "/usr/bin/procmail -d %T"
```

Procmail 需要 root 权限才能在投递模式下工作，但如果你是以 root 身份运行 cronjobs，就不会有这个问题。

要测试一切是否正常工作，请创建一个内容为 "test message" 的 message.txt 文件。

在同一目录下运行：

```
$ sendmail user_name < message.txt
```

紧接着：

```
$ cat /var/spool/mail/user_name
```

现在您应该看到测试信息以及发送的时间和日期。

所有作业的错误输出现在会被重定向到 /var/spool/mail/*user_name*。

由于权限问题，很难创建和发送邮件给root用户（比如 su -c ""）。你可以要求 esmtp 将 root 的所有邮件转发给普通用户，方法是：

```
/etc/esmtprc
force_mda="user-name"
```

注意：如果上述测试没有成功，您可以尝试在 ~/.esmtprc 中创建一个具有相同内容的本地配置。

运行下面的命令以确保它有正确的权限：

```
$ chmod 710 ~/.esmtprc
```

然后用 message.txt 文件重新运行一遍上面的测试。

使用 opensmtpd 的例子

安装 opensmtpd包。

编辑 `/etc/smtpd/smtpd.conf`。下面的配置允许本地投递：

```
listen on localhost
action "local" mbox alias <aliases>
match for local action "local"
```

您现在可以进行测试。运行 `smtpd.service`，然后执行：

```
$ echo test | sendmail user
```

user 可以用任何 能够处理 mbox 格式的邮件阅读器来检查邮件，或者直接查看 `/var/spool/mail/*user*` 文件。如果一切都符合预期，您可以启用 openmtpd 使其开机运行。

这种方法的好处是不需要向远程服务器发送本地 cron 通知。但缺点是需要运行一个新的守护进程。

注意：

- 在写这篇文章的时候，Arch 的 opensmtpd 包还没有在 `/var/spool/smtpd` 下创建好所有需要的目录，但是守护进程会在需要指定所有者和权限时发出警告。只需按照警告创建即可。
- 尽管上文的配置中程序并不会接受远程连接，但使用iptables或类似的方法来阻止端口 25 也是预防措施。

运行时间很长的 cron 任务

假设 cron 调用了这个脚本：

```
#!/bin/sh
echo "我有一个可恢复错误！"
sleep 1h
```

这时会发生以下事件：

1. cron 运行这个脚本
2. 一旦 cron 检查到脚本有输出，就会启动你的邮件传输程序，并向通过管道其传递一些必要的头部内容。因为脚本还没有结束，还会有更多的输出，管道就没有关闭。
3. 邮件传输程序打开到邮件服务器的链接，并等待后续的输出。
4. 邮件服务器会在特定时间后关掉这个空闲的链接，你会受到类似这样的错误：

```
    smtpmsg='421 … Error: timeout exceeded' errormsg='the server did not accept the
mail'（服务器没有接受邮件
```

要解决这个问题，你可以使用 moreutils包 中的 chronic 或 sponge 命令。 它们各自的手册页面中写道：

chronic(1)（时序）：chronic 运行一个命令时，会缓存它的标准输出和标准错误输出，并只在命令失败（返回值非零或崩溃）的情况下才会一并输出。如果命令成功执行，任何无关的输出将被隐藏。 **sponge(1)（海绵）**：sponge 读取标准输入并将其写

入指定的文件。不同于 shell 重定向，sponge 在打开输出文件之前会吸收所有的输入……如果没有指定输出文件，sponge 会输出到标准输出。

Chronic也会在打开标准输出之前缓冲命令输出。

Crontab 格式

crontab 的基本格式是：

分 时 日 月 星期 命令

- *分*值从 0 到 59。
- *时*值从 0 到 23。
- *日*值从 1 到 31。
- *月*值从 1 到 12。
- *星期*值从 0 到 6, 0 代表星期日。

空格用来分开字段，要微调你的时间表，也可以用下面特殊字符来设定范围:

符号	描述
*	通配符，表示所有支持的时间值
,	用逗号分隔多个时间
-	连接两个数值，给出一个范围
/	指定一个周期或频率

例如，下面一行：

```
*/5 9-16 * 1-5,9-12 1-5 ~/bin/i_love_cron.sh
```

将会在6月、7月和8月外的周一至周五，从早上 9 点到下午 4 点 55 分，每隔 5 分钟执行一次脚本 `i_love_cron.sh`。

此外，crontab 还有一些特殊的关键字。

Keyword	Description
@reboot	启动时
@yearly	每年一次
@annually	同 @yearly
@monthly	每月一次
@weekly	每周一次
@daily	每天一次
@midnight	同 @daily
@hourly	每小时一次

例如：

```
@reboot ~/bin/i_love_cron.sh
```

将在启动时执行脚本 `i_love_cron.sh`。

更多信息参见: https://www.adminschoice.com/crontab-quick-reference

更多的例子和高级配置技巧可以在下面找到。

基本命令

Crontabs 绝不应该被直接编辑；你应该使用 `crontab` 程序来处理你的 crontabs。

要查看你的 crontabs：

```
$ crontab -l
```

要编辑你的 crontabs：

```
$ crontab -e
```

要移除你的*所有* crontabs：

```
$ crontab -r
```

如果你有一个保存好的 crontab，而且你想要用它完全覆盖旧的 crontab：

```
$ crontab saved_crontab_filename
```

要从命令行(Wikipedia:stdin)覆盖一个 crontab：

```
$ crontab -
```

要编辑其他用户的 crontab：

```
# crontab -u username -e
```

同一个格式（在命令后追加 `-u *username*`）也可以用来列出或删除 crontabs。

范例

下面的条目:

```
01 * * * * /bin/echo Hello, world!
```

将会在每个月的每一天的每一个小时的第一分钟（例如，在12:01，1:01，2:01等）执行命令 `/bin/echo Hello, world!`

类似地,

```
*/5 * * jan mon-fri /bin/echo Hello, world!
```

将会在一月的每个工作日每五分钟（例如，在12：00，12：05，12：10等）执行一次相同的命令。

和前文 *Crontab 格式*一章相同，这一行（如 crontab(5) 所述）：

```
*0,*5 9-16 * 1-5,9-12 1-5 /home/user/bin/i_love_cron.sh
```

将会在周内从早上 9 点到下午 4 点 55 分，每隔 5 分钟执行一次脚本 `i_love_cron.sh`，夏季除外（6月、7月和8月）。

也可以像这样输入周期性设置：

```
# Chronological table of program loadings
# Edit with "crontab" for proper functionality, "man 5 crontab" for formatting
# User: johndoe
```

```
# mm  hh  DD  MM  W /path/progam [--option]...  ( W = weekday: 0-6 [Sun=0] )
  21  01  *   *   * /usr/bin/systemctl hibernate
  @weekly           $HOME/.local/bin/trash-empty
```

下面是一些不言自明的crontab语法例子：

```
30 4 echo "四点半了。"
0 22 echo "晚上十点了。"
30 15 25 12 echo "现在是圣诞节下午三点半。"
30 3 * * * echo "每天早上三点半提醒我。"
0 * * * * echo "新的一个小时到来了。"
0 6 1,15 * * echo "每月1号和15号的早上六点。"
0 6 * * 2,3,5 echo "周二三四的早上六点。"
59 23 * * 1-5 echo "周内每天的最后一分钟。"
0 */2 * * * echo "每两个小时。"
0 20 * * 4 echo "周四的晚上八点。"
0 20 * * Thu echo "周四的晚上八点。"
*/15 9-17 * * 2-5 echo "周内朝九晚五的每一刻钟。"
@yearly echo "新年好！"
```

默认编辑器

要修改默认编辑器，请在 shell 初始化脚本中定义 `EDITOR` 环境变量，如环境变量所述。

作为普通用户，需要使用 `su` 代替 `sudo` 来正确拉取环境变量：

```
$ su -c "crontab -e"
```

如果希望给这个命令取别名，因为 su 会在一个新启动的子 shell 中启动，为了防止一些以外的发生，需要用 `printf` 加一个任意字符串，来提醒你仍然在 root 下运行：

```
alias scron="su -c $(printf "%q " "crontab -e")"
```

运行基于 X.org 的应用程序

Cron 不在 X.org 下运行，因此它无法知道启动 X.org 应用程序所需的环境变量，因此必须显式定义。我们可以使用类似 xuserrun-gitAUR 这样的程序来完成：

```
17 02 * ... /usr/bin/xuserrun /usr/bin/xclock
```

或者可以手动定义它们（`echo $DISPLAY` 将给出当前的 DISPLAY 环境变量值）：

```
17 02 * ... env DISPLAY=:0 /usr/bin/xclock
```

如果需要在 cron 中运行 notify-send 进行桌面通知，因为 notify-send 通过 dbus 发送值。需要告诉 dbus 连接到正确的总线。 通过检查 DBUS_SESSION_BUS_ADDRESS 环境变量，并设置为相同的值，就可以找到地址。因此：

```
17 02 * ... env DBUS_SESSION_BUS_ADDRESS=your-address notify-send 'Foo bar'
```

如果是通过SSH完成的，需要给与权限：

```
# xhost +si:localuser:$(whoami)
```

异步任务处理

如果你经常关机，但又不想错过任务的执行，这里有一些解决方案（从最简单到最难）：

Cronie

cronie包 内含 anacron。其项目主页介绍道：

Cronie 包含了标准的 UNIX 守护进程 crond，它可以在预定的时间运行指定的程序和相关工具。 它基于最初的 cron，并增强了安全性和配置功能，比如可以使用 pam 和 SELinux。

Dcron

Vanilla dcronAUR 支持异步任务处理。只要用@hourly、@daily、@weekly或者@monthly加上任务名就可以了：

```
@hourly         ID=greatest_ever_job      echo This job is very useful.
```

Cronwhip

cronwhipAUR 是一个自动运行遗漏的 cron 任务的脚本；它与以前的默认 cron 实现 *dcron* 一起工作。 另见这个论坛帖子。

Anacron

Anacron 是 *dcron* 的完全替代者，它可以异步处理任务。

它由 cronie包 提供，配置文件为 `/etc/anacrontab`。关于格式的信息可以在 anacrontab(5) 中找到。运行 `anacron -T` 可以测试 `/etc/anacrontab` 的有效性。

Fcron

和*anacron*一样，fcron包 假设计算机并不总是在运行。但与*anacron*不同的是，它可以在比一天更短的时间内安排这些事件，这对于经常暂停/休眠的系统（例如笔记本电脑）可能很有用。和 cronwhip 一样， fcron 也可以运行那些本应在计算机停机期间运行的作业。

用 fcron 取代 cronie包 时，spool 目录会变为 `/var/spool/fcron`，并使用 `fcrontab` 命令代替 *crontab* 来编辑用户的 crontabs。这些 crontab 以二进制格式存储，其旁边的文本版本在 spool 目录中为 *foo*.orig。由于这种行为上的差异，任何手动编辑的用户 crontabs 可能需要进行调整。

一个快速的脚本小程序，可以帮助您将传统的用户 crontabs 转换为 fcron 格式：

```
cd /var/spool/cron && (
 for ctab in *; do
```

```
  fcrontab ${ctab} -u ${ctab}
 done
)
```

另见这个论坛主题。

确保排他性

如果您有可能运行很久的任务（比如说变化很多或者网速突然变慢，备份可能会偶尔运行很长时间），那么 `flock`（util-linux包）可以确保 cron 任务在同一时间点只有一个运行。

```
  5,35 * * * * /usr/bin/flock -n /tmp/lock.backup /root/make-backup.sh
```

Cronie

cronie 的相关文件层次结构如下：

```
   /etc/
     |----- cron.d/
              | ----- 0hourly
     |----- cron.minutely/
     |----- cron.hourly/
              | ----- 0anacron
     |----- anacrontab
     |----- cron.daily/
     |----- cron.monthly/
     |----- cron.weekly/
     |----- crontab
     |----- cron.deny
```

Cronie 提供了 *cron* 和 *anacron* 两种功能：只要系统在指定的时间可用，*cron* 以固定的时间间隔（粒度为一分钟）运行工作，而*anacron*则在以天为单位指定的时间间隔执行命令。与 cron 不同的是，它并不假设系统连续运行。当系统启动时，*anacron* 就会检查是否有任何漏掉的任务，并进行相应的处理。

*cron*任务可以在 `/etc/cron.d` 目录中类似 crontab 的文件中定义，或者在 `/etc/crontab` 文件中添加。注意后者在默认情况下并不存在，但如果存在就会被使用。按照 `/etc/cron.d/0hourly` 的内容，`/etc/cron.hourly` 中的任何可执行文件将每小时运行一次（默认为每小时的第1分钟），而在 `/etc/cron.minutely` 中的可执行文件将每分钟执行一次（如果 `/etc/cron.d/0minutely` 中有相应的指令）。这些可执行文件通常是 shell 脚本，也可以使用可执行文件的符号链接。 `/etc/cron.deny` 文件包括不允许使用 crontab 的用户列表，如果没有这个文件，只有 `/etc/cron.allow` 中列出的用户才能使用它。

*Anacron*的工作原理类似，通过执行根据所需的作业频率放置在 `/etc/cron.daily`、`/etc/cron.weekly` 和 `/etc/cron.monthly`目录下的文件，cron 作业 `/etc/cron.hourly/0anacron` 确保 *anacron* 每天运行一次，以执行其待办任务。

注意：

- Cronie 使用 `run-parts` 来执行不同目录下的脚本。文件名中不应该包含任何点号（.），因为 `run-parts` 在默认模式下会忽略它们（参见run-parts(8)）。名字必须只由大小写字母、数字、下划线和减号组成。
- `systemctl status cronie` 的输出可能会显示诸如 `CAN'T OPEN (/etc/crontab): No such file or directory` 的内容。您可以忽略这些内容，因为这个文件 cronie 不是必须的。
- Cronie 对 `/etc/cron.d/0hourly` 的权限要求很严格。如果 `/etc/cron.d/{hourly,weekly,daily}...` 中的任务损坏或权限不正确，`/etc/cron.d/0hourly` 中的所有任务都不会被运行（包括 anacron 启动器）。`pacman -Qkk cronie` 可以显示是否有这样的问题。

提示：如果不需要某些命令的输出和电子邮件提醒，请在该 cron 任务的行末添加 `>/dev/null 2>&1`，将输出重定向到/dev/null：

```
0 1 5 10 * /path/to/script.sh >/dev/null 2>&1
```

您也可以在您的 crontab 文件中设置 `MAILTO=""` 变量来禁用全部电子邮件提醒。

Dcron

cron守护进程会解析一个名为`crontab`的配置文件。系统中的每个用户都可以维护一个单独的crontab文件来单独调度命令。root 用户的 crontab 用于调度全系统的任务（用户可以选择使用 `/etc/crontab` 或 `/etc/cron.d` 目录，这取决于他们选择的 cron 实现）。

```
/var/spool/cron/root
# Run command at a scheduled time
# Edit this 'crontab -e' for error checking, man 1 crontab for acceptable format

# <@freq>                       <tags and command>
@hourly         ID=sys-hourly   /usr/sbin/run-cron /etc/cron.hourly
@daily          ID=sys-daily    /usr/sbin/run-cron /etc/cron.daily
@weekly         ID=sys-weekly   /usr/sbin/run-cron /etc/cron.weekly
@monthly        ID=sys-monthly  /usr/sbin/run-cron /etc/cron.monthly

# mm  hh  DD  MM  W /path/command (or tags) # W = week: 0-6, Sun=0
  21  01  *   *   * /usr/bin/systemctl suspend
```

下面几行是 crontab 条目的一种可接受格式，即以空格分隔的字段:

1. @period
2. ID=jobname （这个是 dcron 独有的）
3. command

crontab 条目的另一种标准格式是：

1. minute
2. hour
3. day
4. month
5. day of week

6. command

crontab文件本身通常存储为 `/var/spool/cron/username`。例如，root 的 crontab 文件位于 `/var/spool/cron/root`。

参见 crontab man page 获取更多信息和配置示例。

另请参阅

- Gentoo Linux Cron 指南
- crontab.guru - cronjob 表达式在线编辑器
- cron-notifyAUR 是一个兼容 FreeDesktop.org 的通知服务，用于在执行命令前定期请求确认。命令在自定义配置文件中进行配置。

3.15 Systemd/用户

难度 高级 | **篇幅** 22.6 KB | **所属** 系统管理

本节将讲到

- 工作原理
- 基础设置
- 开发用户单元
- 临时文件
- Xorg 和 systemd

systemd 会给每个用户生成一个 systemd 实例，用户可以在这个实例下管理服务，启动、停止、启用以及禁用他们自己的单元。这个能力大大方便了那些通常在特定用户下运行的守护进程和服务，比如 Music Player Daemon, 还有像拉取邮件等需要自动执行的任务。

工作原理

从 systemd 226 版本开始，`/etc/pam.d/system-login` 默认配置中的 `pam_systemd` 模块会在用户首次登录的时候, 自动运行一个 `systemd --user` 实例。 只要用户还有会话存在，这个进程就不会退出；用户所有会话退出时，进程将会被销毁。当#随系统自动启动 systemd 用户实例启用时, 这个用户实例将在系统启动时加载，并且不会被销毁。systemd 用户实例负责管理用户服务，用户服务可以使用systemd提供的各种便捷机制来运行守护进程或自动化任务，如 socket 激活、定时器、依赖体系以及通过 cgroup 限制进程等。

和系统单元类似，用户单元可以在以下目录找到(按优先级从低到高排序）：

- `/usr/lib/systemd/user/` 这里存放的是各个软件包安装的服务。

- `~/.local/share/systemd/user/` 这里存放的是HOME目录中已安装的软件包的单元。
- `/etc/systemd/user/` 这里存放的是由系统管理员维护的系统范围的用户服务。
- `~/.config/systemd/user/` 这里存放的是用户自身的服务。

当 systemd 用户实例启动时，它会将用户个人的 `default.target` 带起来。其他用户单元可以通过 `systemctl --user` 手动管理。参考 systemd.special(7) § 用户服务管理器管理的单元.

注意：

- `systemd --user` 实例是针对每个用户处理的，而不是针对会话。这样做的原理是用户服务处理的大部分资源，像 socket 或状态文件是针对每个用户的（存活于用户的主目录下）而不是会话。这意味着所有的用户服务是独立于会话之外运行的。最终，我们得出结论：基于会话运行的程序可能会导致用户服务中断。systemd 处理用户会话的方式是非常生硬的（pretty much in flux）。 单会话支持的进展参考 和 。
- `systemd --user` 和 `systemd --system` 运行于不同的进程里面，所以用户单元不能引用或依赖于系统单元或其他用户的单元。

基础设置

所有的用户单元都位于 `~/.config/systemd/user` 路径下。 如果你想在首次用户登录时运行单元，对想要自动启动的单元执行 `systemctl --user enable *unit*` 即可。

提示：如果不是只让发出"systemctl"命令的用户启用某个service单元，而是想要让所有用户都生效，请以root权限执行`systemctl --global enable *service*` 命令。

环境变量

systemd 用户实例不会继承类似 `.bashrc` 中定义的环境变量。systemd 用户实例有三种设置环境变量的方式：

1. 对于有 `$HOME` 目录的用户，可以在 `~/.config/environment.d/` 目录中新建一个".conf"文件，然后在其中写入格式为`NAME=VAL`这样的行。这些设置只对指定用户的用户单元有效 。更多信息可以参考 environment.d(5) 。
2. 使用 `/etc/systemd/user.conf` 文件中的 `DefaultEnvironment` 选项。这个配置在所有的用户单元中可见。
3. 在 `/etc/systemd/system/user@.service.d/` 下增加配置文件设置。 这个配置在所有的用户单元中可见。

4. 在任何时候， 使用 `systemctl --user set-environment` 或 `systemctl --user import-environment`. 对设置环境变量之后启动的所有用户单元有效，但已经启动的用户单元不会生效。
5. 使用由 dbus提供的 `dbus-update-activation-environment --systemd --all` 命令。和`systemctl --user import-environment`有同样的效果，但是会影响D-Bus会话。你可以把这个添加到shell初始化文件的末尾。
6. 对于用户环境的“全局”环境变量，可以使用会被某些生成器解析的 `environment.d` 目录。 更多信息可以参考environment.d(5)和 systemd.generator(7)。
7. 您还可以编写一个systemd.environment-generator(7) 脚本，该脚本可以生成因用户而异的环境变量，如果您需要分别给每个用户环境配置变量，这可能是最好的方法（ `XDG_RUNTIME_DIR`, `DBUS_SESSION_BUS_ADDRESS`等就是这种情况 ）。

: : > 提示：如果想一次设置多个环境变量，可以写一个配置文件，文件里面每一行定义一个环境变量，用 "key=value" 的键值对表示，然后在你的启动脚本里添加`xargs systemctl --user set-environment < /path/to/file.conf`。

一般情况下，你需要设置 `PATH` 这个环境变量。 配置完成后，可以使用命令 `systemctl --user show-environment` 来验证值是否正确。

Service 文件例子

新建 drop-in 目录 `/etc/systemd/system/user@.service.d/` 然后在里面新建一个 `.conf`文件 (例如 `local.conf`):

```
/etc/systemd/system/user@.service.d/local.conf
[Service]
Environment="PATH=/usr/lib/ccache/bin:/usr/local/bin:/usr/bin:/bin"
Environment="EDITOR=nano -c"
Environment="BROWSER=firefox"
Environment="NO_AT_BRIDGE=1"
```

DISPLAY 和 XAUTHORITY

任何一个 X 应用程序都需要使用 `DISPLAY` 来指示使用哪个显示器，而 `XAUTHORITY` 则是保存了用户授权文件 `.Xauthority` 的路径，X 应用需要用户授权文件中的 cookie 信息才能访问 X Server。如果你想通过 systemd 单元启动一个 X 应用，必须先设置这两个环境变量。systemd 提供了一个脚本 `/etc/X11/xinit/xinitrc.d/50-systemd-user.sh`，在 X 启动的时候，将

这些环境变量导入到 systemd 用户会话中。所以除非你不是通过正常的途径启动X，systemd用户服务应该已经包含了这两个变量。

PATH

通过 `.bashrc` 或者 `.bash_profile` 设置的环境变量，对 systemd 都是不可见的。如果你改变了你的 `PATH` 变量，并且准备在 systemd 单元运行的应用中使用这个环境变量，你必须在 systemd 的环境中设置 `PATH`。假设你在 `.bash_profile` 中设置了 `PATH`，让 systemd 感知到这个变化的最好方法是在修改 `PATH` 之后，加入以下行通知 systemd：

```
~/.bash_profile
systemctl --user import-environment PATH
```

注意：

- 不会影响导入 `PATH` 之前启用的程序.
- *systemd* 只有在处理绝对路径二进制时才会使用设置的 `PATH`。

pam_env

注意：这种为用户设置环境变量的方法已弃用且将被移除。

环境变量可以通过 `pam_env.so` 模块来设置。参阅环境变量#使用 pam_env 获取配置详情。

随系统自动启动 systemd 用户实例

systemd 用户实例在用户首次登录时启动，并在最后一个会话退出时终止。 但有时候，对于一些不依赖于会话的用户进程，在系统启动时加载用户实例，在会话全部结束时，也不停止用户实例是比较有用的。Lingering 就是用来实现这个的。 使用以下命令来启用驻留：

```
$ loginctl enable-linger
```

为另一个用户启用驻留：

```
# loginctl enable-linger username
```

警告：systemd 服务是 **没有** 会话的， 它们在 *logind* 状态之外运行， 所以不要在 lingering 中启用自动登录的功能，这会导致 会话中断。

开发用户单元

通用的 systemd unit 文件编写请参考 systemd#编写单元文件。

例子

下面是 mpd 服务用户版本的例子:

```
~/.config/systemd/user/mpd.service
[Unit]
Description=Music Player Daemon
```

```
[Service]
ExecStart=/usr/bin/mpd --no-daemon

[Install]
WantedBy=default.target
```

使用变量的例子

下面是用于 foldingathomeAUR 的用户服务，将主目录作为变量传入使得 Folding@home 能找到指定文件：

```
~/.config/systemd/user/foldingathome-user.service
[Unit]
Description=Folding@home distributed computing client
After=network.target

[Service]
Type=simple
WorkingDirectory=%h/.config/fah
ExecStart=/usr/bin/FAHClient
CPUSchedulingPolicy=idle
IOSchedulingClass=3

[Install]
WantedBy=default.target
```

在 systemd.unit(5) 的 SPECIFIERS 章节中，详细介绍了各种变量，`%h` 指示符将使用运行该服务的用户的主目录替代。更多的变量参考 systemd 的手册页。

查看日志

查看用户单元的日志可以用类似命令：

```
$ journalctl --user
```

查看指定单元的日志，可以使用以下命令：

```
$ journalctl --user-unit myunit.service
```

也可以这样：

```
$ journalctl --user -u myunit.service
```

注意：UID 小于 1000 的用户的日志不会被 journald 记录，需要的话可以 重定向 所有内容至系统日志。

临时文件

systemd-tmpfiles 可以像管理系统临时文件那样管理用户自定义的临时文件和目录（参阅 systemd#systemd-tmpfiles - 临时文件）。用户配置文件以 `~/.config/user-tmpfiles.d/` 和 `~/.local/share/user-tmpfiles.d/` 的顺序读取。为了正常使用本功能，需要为用户启用必要的 systemd 用户单元：

```
$ systemctl --user enable systemd-tmpfiles-setup.service systemd-tmpfiles-clean.timer
```

配置文件的语法格式和系统级的配置是一样的，详情请参阅 systemd-tmpfiles(8) 和 tmpfiles.d(5) 手册页。

Xorg 和 systemd

使用 systemd 单元来运行 Xorg 有好几种方法，下面介绍其中两种，一种是启动一个新的用户会话，在里面运行 Xorg 服务，另外一种是用 systemd 用户服务启动 Xorg。

没有显示管理器的情况下自动登录到 Xorg

**本文或本章节的事实准确性存在争议。**

原因： 这样设置会导致用户有2条 D-Bus 总线，一条用于桌面，另一条用于 systemd。为什么不共用 systemd 的那一条？（在 Talk:Systemd/用户 中讨论）

这种方法通过一个系统单元将用户会话带起来，并在用户会话里面启动一个 xorg 服务，并运行 `~/.xinitrc` 将窗口管理器等启动起来。你需要安装好 xlogin-gitAUR, 并按照 Xinit#xinitrc 来设置好你的 xinitrc。

会话会使用它自己的 dbus 守护，而需要用到 `dbus.service` 的 systemd 工具会自动连接到会话的 dbus 实例上。最后启用 `xlogin@*username*` 服务来达成开机自动登录。 整个用户会话都在 systemd 的作用域下运行，会话内的一切都能正常工作。

将 Xorg 作为 systemd 用户服务

另外一种选择是将 xorg 作为一个 systemd 用户服务。这是一种不错的方案，因为其他的 X-related units 可以依赖于 xorg 服务。 但另一方面，这个方案存在某些倒退，这在下面会提到。

xorg-server包 提供了两种整合到 systemd 的方法:

- 可以在无特权模式下运行，设备管理由 logind 代为管理(参考 Hans de Goede 的这个提交)。
- 可以实现通过 socket 激活服务 (参考这个提交)。

但非常不幸，xorg 的无特权模式需要在用户会话里面运行。所以，xorg 的用户服务只能在 root 权限下运行(和 1.16 版本之前一样)，而不能使用 1.16 版本提供的无特权模式。

注意：这并不是 logind 强加的限制，而是 xorg 需要知道它将要接管的是哪个会话，而现在它通过调用 logind's `GetSessionByPID` 来获取这个信息(使用 xorg 自身的 pid 作为参数)。参见这个话题 和 xorg 源码. 看上去如果 xorg 通过其依附的 tty 来获取会话信息的话，这个问题将得到解决。

警告：在 xorg 1.18 上 socket 激活存在问题，客户端会触发激活死锁。具体查阅上游报告 。临时策略是启动一个不带 socket 激活的 xorg 服务器，并在服务器启动一段时间后客户端再连接，确保连接时服务器已经启动。目前来看没有一个可靠机制来判断 X 服务器是否启动完毕。

下面是从用户服务运行 xorg 的步骤：

1\. 通过编辑 `/etc/X11/Xwrapper.config` 文件，允许所有用户使用 root 权限运行 xorg。这是在 Xorg#Xorg as Root 的基础上，无需用户在实际控制台上登录，而是将 `allowed_user` 的默认 `console` 控制台定义成了 `anybody`。参见 Xorg.wrap(1)。

```
/etc/X11/Xwrapper.config
allowed_users=anybody
needs_root_rights=yes
```

2\. 把下面 systemd 单元加到 `~/.config/systemd/user` 目录下：

```
~/.config/systemd/user/xorg@.socket
[Unit]
Description=Socket for xorg at display %i

[Socket]
ListenStream=/tmp/.X11-unix/X%i
~/.config/systemd/user/xorg@.service
[Unit]
Description=Xorg server at display %i

Requires=xorg@%i.socket
After=xorg@%i.socket

[Service]
Type=simple
SuccessExitStatus=0 1

ExecStart=/usr/bin/Xorg :%i -nolisten tcp -noreset -verbose 2 "vt${XDG_VTNR}"
```

这里 `${XDG_VTNR}` 表示 xorg 将要运行的虚拟终端，可以在服务单元文件里面硬编码，也可像下面那样在环境变量里指定：

```
$ systemctl --user set-environment XDG_VTNR=1
```

注意：xorg应该在用户登录的虚拟终端上运行，否则 logind 会认为会话没有激活。

3\. 确保 `DISPLAY` 环境变量已经配置，参考这里.

4\. 接下来，执行以下命令，使得 xorg 在 display 0 和 tty 2 上可以通过 socket 激活：

```
$ systemctl --user set-environment XDG_VTNR=2      # 这样 xorg@.service 知道该用哪个虚拟终端
$ systemctl --user start xorg@0.socket             # 开始为 display 0 监听 socket
```

现在，在 tty 2上运行任意的X应用，xorg 都会自动启动。

可以在 `.bash_profile` 里面把环境变量 `XDG_VTNR` 设置到 systemd 环境里面。在这之后，你可以使用 systemd 单元启动任意的X应用，包括窗口管理器。当然，这些 systemd 单元必须依赖于 `xorg@0.socket`。

警告：当前，通过用户服务启动窗口管理器意味着它是在会话之外运行的，这将带来以下问题: break the session. 但是，systemd 的开发者看上去更倾向于这样(?)。参见 和

一些用例

窗口管理器

以一个 systemd 服务来运行窗口管理器需要先 #将 Xorg 作为 systemd 用户服务来运行。下面我们以 awesome 举例：

```
~/.config/systemd/user/awesome.service
[Unit]
Description=Awesome window manager
After=xorg.target
Requires=xorg.target

[Service]
ExecStart=/usr/bin/awesome
Restart=always
RestartSec=10

[Install]
WantedBy=wm.target
```

注意：`[Install]` 一节包括一个 `WantedBy` 参数，当使用 `systemctl --user enable` 时会创建一个本服务的软链接到 `~/.config/systemd/user/wm.target.wants/*window_manager*.service` 来使它在登录时启动。推荐采用启用服务的方法来代替手动创建软链接。

终端复用器持久化

比起默认登入一个窗口管理器，你可能更想要自动在后台运行一个终端复用器（比如 GNU Screen 或 tmux）。

创建以下文件：

```
~/.config/systemd/user/multiplexer.target
[Unit]
Description=Terminal multiplexer
Documentation=info:screen man:screen(1) man:tmux(1)
After=cruft.target
Wants=cruft.target

[Install]
Alias=default.target
```

将用户登录与 X 登录分开，对引导到 TTY 而不是显示管理器的用户是有用的，此时你可以将所有要启动的东西绑定到 `mystuff.target`。

依赖项 `cruft.target` 与上述 `mystuff.target` 类似，用来在终端复用器启动前启动你需要的服务（或用来启动那些需要开机启动但时序要求不高的服务），比如 GnuPG 守护程序。

现在可以为你的终端复用器创建一个服务了。以下是一个例子，以 tmux 为例且加载了一个 gpg-agent 会话，这个 gpg-agent 会话会将信息写入 `/tmp/gpg-agent-info`。这个示例会话在将来启动 X 以后也可以运行 X 程序，因为设置了 `$DISPLAY`。

```
~/.config/systemd/user/tmux.service
[Unit]
Description=tmux: A terminal multiplexer
Documentation=man:tmux(1)
After=gpg-agent.service
Wants=gpg-agent.service

[Service]
Type=forking
ExecStart=/usr/bin/tmux start
ExecStop=/usr/bin/tmux kill-server
Environment=DISPLAY=:0
EnvironmentFile=/tmp/gpg-agent-info

[Install]
WantedBy=multiplexer.target
```

启用 `tmux.service`、`multiplexer.target` 和由 `cruft.target` 运行的服务，像往常一样启动 `user@.service` 即可。

在注销后结束用户进程

Arch Linux 用 `--without-kill-user-processes` 参数编译了 systemd包 包，默认下 `KillUserProcesses` 是 `no`。这一设置导致了在用户注销后，用户进程不会结束。要改变这一行为，在用户注销后结束所有用户进程，请在 `/etc/systemd/logind.conf` 中设置 `KillUserProcesses=yes`。

注意，修改这一设置会破坏终端复用器，如 tmux 和 GNU Screen。修改设置后仍想使用终端复用器，需按下列方法使用 `systemd-run`：

```
$ systemd-run --scope --user command args
```

比如要运行 `screen` 可以这样：

```
$ systemd-run --scope --user screen -S foo
```

只有在用户在某处登录过系统至少一次，且 `user@.service` 仍在运行的情况下，使用 `systemd-run` 可以保持进程在用户注销后继续运行。

用户注销所有会话后，默认 `user@.service` 也会终止，除非用户启用了 "lingering" 。为了让用户在注销后也能运行长期任务，需要为他们启用 lingering。详情参阅 #随系统自动启动 systemd 用户实例和 loginctl(1)。

疑难解答

Runtime directory '/run/user/1000' is not owned by UID 1000, as it should

```
systemd: pam_systemd(systemd-user:session): Runtime directory '/run/user/1000' is not owned by UID 1000, as it should.
systemd: Trying to run as user instance, but $XDG_RUNTIME_DIR is not set
```

如果你看到类似错误且无法登录会话，有可能是另一个系统级（非用户级）的服务正在创建该目录。例如正在使用的 docker 容器挂载了 `/run/user/1000`。可以修改容器取消挂载，或禁用、延迟启动 docker 服务来修复此问题。

参阅

- KaiSforza's Bitbucket wiki
- Zoqaeski's units on GitHub
- Arch forum thread about changes in systemd 206 user instances

电源管理

这一章处理从待机到关机的电源行为：选哪个 ACPI 模块、屏幕何时熄灭、挂起交给 systemd 还是 pm-utils、谁能唤醒机器。环节互相咬合，任一处配错都会睡下去醒不来。

建议先读《电源管理》《电源管理/挂起与休眠》，理顺内核 swsusp 与 systemd 的关系，再按需取用 TLP、Powertop、Acpid 等；《DSDT》《Pm-utils》可留到最后，前者要重编译固件表。最常见的误解是把故障归给硬件，实际多源于配置冲突：模块选错、挂起后端并存、唤醒触发器未持久保存、终端与 X 各自对 DPMS 下指令。

4.1　ACPI 模块

难度 入门 | **篇幅** 4.2 KB | **所属** 电源管理

本节将讲到

- 简介
- 目前有哪些模块?
- 如何选择正确的模块
- 问题解决

简介

从 kernel 2.6.20.7 开始，acpi 模块化了, 以避免一些机器上面产生的 acpi 的问题。

本文是对内核的 acpi 模块的一个简单介绍，这些模块可以激活一些特别的 acpi 函数或者添加一些信息到 /proc 下面，以使得 acpid 或者其他监视程序可以处理 acpi 事件。

目前有哪些模块?

- ac (电源连接情况) => 在启动的时候由 initscripts-0.8-7 自动加载
- asus_acpi (使用asus/medion 笔记本电脑的时候有用)
- battery (电池状态) => 在启动的时候由 initscripts-0.8-7 自动加载
- bay (bay status)
- button (捕获按键事件，比如合上显示器或者按下电源按钮) => 在启动的时候由 initscripts-0.8-7 自动加载
- container (container status)
- dock (docking station status) 有些笔记本可以在下面附加一个dock来提供一些额外的功能，比如sony、dell的很多笔记本都有这个接口。
- fan (风扇状态) => 在启动的时候由 initscripts-0.8-7 自动加载

- hotkey (笔记本电脑的热键)
- i2c_ec (EC SMBus 驱动)
- ibm_acpi (使用IBM笔记本电脑的时候有用)(2.6.22后是thinkpad_acpi)
- processor (CPU处理器状态) => 集成到了 kernel 2.6.20.7-2 中
- sbs (smart battery status)
- thermal (status of thermal sensors) => 集成到了 kernel 2.6.20.7-2 中
- toshiba_acpi (使用toshiba笔记本电脑的时候有用)
- video (视频设备的状态)

当前正在使用的内核支持的acpi模块列表可以用下面命令查看：

```
# ls -l /usr/lib/modules/$(uname -r)/kernel/drivers/acpi
total 112
-rw-r--r-- 1 root root  2808 Aug 29 23:58 ac.ko.gz
-rw-r--r-- 1 root root  3021 Aug 29 23:58 acpi_ipmi.ko.gz
-rw-r--r-- 1 root root  3354 Aug 29 23:58 acpi_memhotplug.ko.gz
-rw-r--r-- 1 root root  4628 Aug 29 23:58 acpi_pad.ko.gz
drwxr-xr-x 2 root root  4096 Aug 29 23:59 apei
-rw-r--r-- 1 root root  7120 Aug 29 23:58 battery.ko.gz
-rw-r--r-- 1 root root  3700 Aug 29 23:58 button.ko.gz
-rw-r--r-- 1 root root  2181 Aug 29 23:58 container.ko.gz
-rw-r--r-- 1 root root  1525 Aug 29 23:58 custom_method.ko.gz
-rw-r--r-- 1 root root  1909 Aug 29 23:58 ec_sys.ko.gz
-rw-r--r-- 1 root root  2001 Aug 29 23:58 fan.ko.gz
-rw-r--r-- 1 root root  1532 Aug 29 23:58 hed.ko.gz
-rw-r--r-- 1 root root  3241 Aug 29 23:58 pci_slot.ko.gz
-rw-r--r-- 1 root root 17742 Aug 29 23:58 processor.ko.gz
-rw-r--r-- 1 root root  3073 Aug 29 23:58 sbshc.ko.gz
-rw-r--r-- 1 root root  7098 Aug 29 23:58 sbs.ko.gz
-rw-r--r-- 1 root root  6311 Aug 29 23:58 thermal.ko.gz
-rw-r--r-- 1 root root  8891 Aug 29 23:58 video.ko.gz
```

如何选择正确的模块

你只能自己来测试哪个模块在你机器上面能正常工作：

```
# modprobe <yourmodule>
```

然后使用下面命令检查模块是否可用

```
# dmesg
```

提示： 可以用 grep 进行过滤

```
 # dmesg | grep acpi
[    0.000000] ACPI: LAPIC (acpi_id[0x00] lapic_id[0x00] enabled)
[    0.000000] ACPI: LAPIC (acpi_id[0x01] lapic_id[0x04] enabled)
[    0.000000] ACPI: LAPIC (acpi_id[0x02] lapic_id[0x01] enabled)
[    0.000000] ACPI: LAPIC (acpi_id[0x03] lapic_id[0x05] enabled)
[    0.000000] ACPI: LAPIC_NMI (acpi_id[0x00] high edge lint[0x1])
[    0.000000] ACPI: LAPIC_NMI (acpi_id[0x01] high edge lint[0x1])
[    0.000000] ACPI: LAPIC_NMI (acpi_id[0x02] high edge lint[0x1])
[    0.000000] ACPI: LAPIC_NMI (acpi_id[0x03] high edge lint[0x1])
[    5.066752] ACPI: acpi_idle yielding to intel_idle
[    5.438998] acpi device:04: registered as cooling_device4
```

把需要的模块按照 man modules-load.d 加入配置文件`/etc/modules-load.d`.

问题解决

DSDT 修正

如果装入模块后电源管理问题仍然存在，可能是 Linux 兼容性较差的DSDT导致。请参阅DSDTWiki 文档。

笔记本 ACPI 修正

如果遇到 ACPI: EC: input buffer is not empty, aborting transaction。 可能是 acpi 模块导致。有两个解决方法：

1\. "简单" 在 GRUB 的 menu.lst 或者 grub.cfg 中的内核行加入 acpi=off，这将禁用所有 acpi 功能，包括电池充电。

2\. "难" 用 bugs.launchpad.net 中的补丁重新编译内核。

4.2 Powertop

难度 入门 | **篇幅** 4.4 KB | **所属** 电源管理

本节将讲到

- 安装
- 使用
- 疑难解决
- 参见

Powertop 是一个 Intel 提供的在用户空间、内核和硬件层面的节电工具。它可以监视进程，并显示哪些进程利用 CPU 并从空闲状态唤醒它，从而识别具有特殊高功率需求的应用程序。

安装

安装 powertop包 。

使用

Powertop 的交互模式可以通过以下方式调用：

```
# powertop
```

在交互模式下，您可以在 **Tunables** 和 **WakeUp** 选项卡中修改建议的设置，并可以在 **Overview** 选项卡中监视它们对功耗的影响。但是，任何设置都不会保留，重启后会丢失。

生成报告

Powertop 可以生成 CSV 或 HTML 格式的报告。HTML 导出的是一个交互式文档，显示推荐的设置。在生成报告之前，请确保重启以恢复为系统默认设置！

您也可以通过遵循以下步骤来提取推荐的参数：

1. 使用 Powertop 生成参数报告：

```
# powertop --html=powerreport.html
```

1. 在您喜欢的网页浏览器中打开报告。报告的 **Tuning** 标签现在显示了工具建议节省电量应用的实际参数。提取命令：

```
$ awk -F '</?td ?>' '/tune/ { print $4 }' powerreport.html
```

较新版本的 Powertop 包括了 `--auto-tune-dump` 选项，它将输出 Powertop 的 `--auto-tune` 将运行的命令。如果您不想运行 Powertop 的所有建议，这在脚本中非常有用。

保存设置

有两种方法保存其设置，使其在重启后依然应用先前的设置。

- **推荐**：使用 Kernel modules、 Udev 和sysctl来使其在系统启动时应用设置。相关细节请看Power management。您也可以使用 powertop-to-tmpfileAUR 辅助工具，通过 systemd-tmpfiles 应用设置。
- 使用 Powertop 的 `--auto-tune` 参数，该参数会使得所有的可调整项变成 GOOD,为使其在系统启动时就生效，可使用 systemd 服务使其开启自启动。添加该文件并启用/启动该服务：

```
/etc/systemd/system/powertop.service
[Unit]
Description=Powertop tunings

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/powertop --auto-tune

[Install]
WantedBy=multi-user.target sleep.target
```

您也可以将这一行添加到 `[Service]` 部分，以防止在启动时已经连接到您的系统的USB输入设备断开连接。

```
 ExecStartPost=/bin/sh -c 'for f in $(find /sys/bus/usb/drivers/usbhid -regex '.*\/
[0-9:.-]+' -printf '%f\n' | cut -d ":" -f 1 | sort -u); do echo on >|
"/sys/bus/usb/devices/$f/power/control"; done'
```

疑难解决

Error: Cannot load from file

如果在启动 powertop 时遇到如下错误，可能是因为 powertop 没有收集到足够的数据，请在电池供电的情况下多运行一段时间，收集更多的数据。

```
Loaded 39 prior measurements
Cannot load from file /var/cache/powertop/saved_parameters.powertop
Cannot load from file /var/cache/powertop/saved_parameters.powertop
```

校准测量数据

如果测量结果不准确，可能需要先校准 powertop: 运行 powertop 时增加 `--calibrate` 参数.

注意：校准时会开关背光、wifi 等功能，在校准时不要触碰机器。

```
# powertop --calibrate
```

参见

- Project repository
- Wikipedia article

4.3 Display Power Management Signaling

难度 入门 | **篇幅** 5.0 KB | **所属** 电源管理

本节将讲到

- 在 X 中设定 DPMS
- 用xset修改DPMS和屏保设定
- Linux 终端和 DPMS 的交互
- 参阅

DPMS (显示电源管理信号，简称DPMS) 可以在计算机一定时间无操作时，将显示器置于节电模式。具体的时间设置可以参考 . 注意有些显示器在不同 DPMS 状态下表现不变。

在 X 中设定 DPMS

注意：从 Xorg 1.8 开始，只要内核启用了 ACPI， DPMS 就会自动启用。

在 `/etc/X11/xorg.conf` 的 `Monitor` 段落中加上上:

```
Option "DPMS" "true"
```

把下面的配置加入 `ServerLayout` 小节, 必要时改变时间(以分钟计):

```
Option "StandbyTime" "10"
Option "SuspendTime" "20"
Option "OffTime" "30"
```

注意：If the *"OffTime"* option does not work replace it with the following, (change the *"blanktime"* to *"0"* to disable screen blanking)

```
Option         "BlankTime" "30"
```

比较新的X推荐使用 `.conf` 文件代替 `xorg.conf`, `/etc/X11/xorg.conf.d/10-monitor.conf` 的一个例子如下：

```
Section "Monitor"
    Identifier "LVDS0"
    Option "DPMS" "false"
EndSection

Section "ServerLayout"
    Identifier "ServerLayout0"
    Option "BlankTime"   "0"
    Option "StandbyTime" "0"
    Option "SuspendTime" "0"
    Option "OffTime" "0"
EndSection
```

用xset修改DPMS和屏保设定

可以用 xorg-xset包 提供的`xset`工具关闭屏幕。如果要在shell中关闭显示器，需要在命令前面加上 `sleep 1;` . 例如：

```
sleep 1; xset dpms force off
```

命令示例：

命令	描述
xset s off	禁用屏保清空
xset s 3600 3600	将清空时间设置到 1 小时
xset -dpms	关闭 DPMS
xset s off -dpms	禁用 DPMS 并阻止屏幕清空
xset dpms force off	立即关闭屏幕
xset dpms force standby	待机界面
xset dpms force suspend	休眠界面

注意：通过 `dpms 0 0 0` 可以将 DPMS 超时都设置成零，这样也可以禁用 DPMS. 这种方式更方便，可以使用 `xset dpms force off` 关闭屏幕。

查看当前设置:

```
$ xset q

...

Screen Saver:
  prefer blanking:  yes    allow exposures:  yes
  timeout:  600    cycle:  600
DPMS (Energy Star):
  Standby: 600    Suspend: 600    Off: 600
  DPMS is Enabled
  Monitor is On
```

运行 `xset` 可以查看全部可用命令.

注意：如果在 xinitrc 中使用 `xset` 无法工作，在配置文件中进行设置。

警告：XScreenSaver 和 xfce4-power-manager包 会使用自己的 DPMS 设置，详情参考 XScreenSaver#DPMS and blanking settings 和 Xfce#Display blanking.

Linux 终端和 DPMS 的交互

setterm 工具可以通过终端能够识别的转义字符修改终端。可以写入/echo 终端序列到当前终端设备，包括再显示设备，远程 SSH 终端， 控制台，串口控制台。

setterm Syntax: (0 disables)

```
setterm -blank [0-60|force|poke]
setterm -powersave [on|vsync|hsync|powerdown|off]
setterm -powerdown [0-60]
```

防止屏幕关闭

可以运行以下命令：

```
$ setterm -blank 0 -powerdown 0
```

也可以通过下列命令禁止终端清空:

```
# echo -ne "\033[9;0]" >> /etc/issue
```

将分号后的 0 修改为 3 会在 3 分钟后进入休眠模式。

通过 cat 显示输出中的转义

```
$ setterm -powerdown 2>&1 | exec cat -v 2>&1 | sed "s/\\^\\[/\\\\033/g"
```

将转义输出到任意 tty (with write/append perms) 进行终端修改

```
$ setterm -powerdown 0 >> /dev/tty3
```

注意：使用 >> 而不是 >. 如果脚本有 *sudo* 权限问题，tty 可能允许附加但是不允许写入， 可以使用 **tee** 阻止 setterm 输出到 tty 。

用循环设置 ttys 0-256

```
$ for i in {0..256}; do setterm -powerdown 0 >> /dev/tty$i; done; unset i;
```

参阅

- PC Monitor DPMS specification explanation
- DPMS control in X

4.4 TLP

难度 入门 | **篇幅** 5.6 KB | **所属** 电源管理

本节将讲到

- 安装
- 配置
- 问题解决
- 参见

来自项目主页:

: TLP 是 Linux 系统上一个功能丰富的命令行工具，能够在不需要深入了解技术细节的情况下**节省笔记本电脑电池电量**。

: TLP 的默认设置已经针对电池寿命进行了**优化**，并且开箱即用就实现了 Powertop 的建议。所以你只需要**安装并忘记**它。

: 尽管如此，TLP 仍然是**高度可定制的**，以满足你的特定需求。

TLP 故意排除了一些设置，特别是风扇转速控制和背光。

安装

安装 tlp包。安装可选依赖项可能有助于提供额外的节能效果。

启用/启动 `tlp.service`。

还应该屏蔽服务 `systemd-rfkill.service` 和套接字 `systemd-rfkill.socket`，以避免冲突并确保 TLP 的无线电设备切换选项正常工作。

无线设备向导（tlp-rdw）

当使用无线电设备向导（tlp-rdw包）时，需要使用 NetworkManager 并启用 `NetworkManager-dispatcher.service`。

详情请参阅 TLP 设置。

图形界面

- tlpuiAUR 是用 Python 编写的 TLP 的 GTK 用户界面。
- slimbookbatteryAUR 是另一种与 AMD 和 NVIDIA 等附加驱动程序兼容的 GTK 界面。

对于 ThinkPads

使用 threshyAUR 及其示例 Qt 用户界面 threshy-guiAUR，可以在没有 root 权限的情况下通过 D-Bus 控制充电阈值。

对于 Sandy Bridge 之前的型号（2010 年及以前）

对于 2010 年及以前 ThinkPad 型号，需要 tp_smapi（英语：tp_smapi） 内核模块。请参阅 tp_smapi#安装（英语：tp_smapi#Installation）获取特定内核的安装说明。

配置

配置文件位于 `/etc/tlp.conf`，并且默认情况下提供了大部分优化的节能功能。要获取选项的完整解释，请参阅：TLP 设置。

禁用 USB 自动挂起

使用默认配置启动 TLP 时，一些 USB 设备（如音频 DAC）在运行电池时将由于 TLP 的自动挂起功能而被**关闭电源**。一些设备如键盘和扫描仪默认被排除在自动挂起之外。

您可能只想通过以下设置完全禁用 USB 自动挂起：

```
/etc/tlp.conf
# 不挂起 USB 设备
USB_AUTOSUSPEND=0
```

或者将特定设备列入黑名单以避免自动挂起。详情请查看 TLP 关于 USB 设备的文档。

强制使用电池（BAT）配置

当无法检测到电源时，像台式机和嵌入式硬件这样的设备将使用 AC 设置。

您可能希望在使用 TLP 时强制使用电池（BAT）设置，以启用更多的节能措施：

```
/etc/tlp.conf
# 当无法检测到电源时的操作模式：AC 或 BAT。
TLP_DEFAULT_MODE=BAT

# 操作模式选择：0=依赖电源，1=始终使用 TLP_DEFAULT_MODE
TLP_PERSISTENT_DEFAULT=1
```

Bumblebee 与 NVIDIA 驱动

如果您正在运行 Bumblebee 并使用 NVIDIA 驱动，您需要在 TLP 中禁用 GPU 的电源管理，以便让 Bumblebee 控制 GPU 的电源。

根据您的驱动使用情况，将一个或多个驱动列入黑名单，防止 TLP 管理它们的电源状态：

```
/etc/tlp.conf
RUNTIME_PM_DRIVER_DENYLIST="nouveau nvidia"
```

命令行

TLP 提供多个命令行工具。详情访问 TLP commands。

问题解决

对于调试，您可以显示有关当前使用的模式（AC/BAT）和应用的配置的信息：

```
# tlp-stat
```

另请参阅上游故障排除指南。

hci0: link tx timeout

如果您的蓝牙耳机突然停止工作，并且您从 dmesg 看到这个错误，这可能是由于 TLP 挂起了您的设备。在 `/etc/tlp.conf` 中的 `USB_BLACKLIST` 添加设备 ID：

```
# 禁用蓝牙自动挂起
USB_DENYLIST="8087:0aaa"
```

从 `lsusb -v` 获取您蓝牙设备的设备 ID。重启 TLP 和 `bluetooth` 服务。

参见

- TLP - Linux Advanced Power Management - 项目主页及文档。
- Project FAQ

4.5 电源管理/唤醒触发器

难度 进阶 | **篇幅** 9.0 KB | **所属** 电源管理

本节将讲到

- 唤醒触发器接口
- 持久设置
- 疑难解答

唤醒触发器是可以将系统从任何硬件节能状态唤醒的事件源。显而易见的例子是电源或挂起按钮、网络唤醒（英语：Wake-on-LAN）功能或笔记本电脑系统中的盖开关。唤醒触发器可以通过下面列出的各种内核接口进行控制。没有涵盖所有可能的触发器的统一接口。

唤醒触发器接口

`/proc/acpi/wakeup`

读取 `/proc/acpi/wakeup` 文件将生成 ACPI 注册的唤醒源列表，其中包含相应的 `sysfs` IDs（如果可用）。将条目从*设备*列写入文件可切换其状态。例如，要禁用打开笔记本电脑盖时的唤醒功能，请运行：

```
# echo "LID" > /proc/acpi/wakeup
```

/sys/module/acpi/parameters/ec_no_wakeup

该文件表示 ACPI 内核模块选项 `ec_no_wakeup` 的值，当系统处于挂起到空闲（`s2idle`）电源状态时，该选项控制从嵌入式控制器传递唤醒触发器。在现代笔记本电脑上，嵌入式控制器唤醒在某些（英语：Lenovo ThinkPad Helix 2nd Gen#Disable embedded-controller wake-ups）情况下会导致电池过度消耗。

/sys/devices/

每个支持唤醒的 `sysfs` 设备都在设备的 `power` 子目录中包含文件 `wakeup`。该文件包含唤醒触发器的状态，也可以写入。总线控制器和端点设备能够唤醒系统。例如，若要禁用第一个 USB 控制器（总线）的唤醒，请运行：

```
# echo "disabled" > /sys/bus/usb/devices/usb1/power/wakeup
```

如果启用了触发器，端点设备应该能够唤醒设备，而不管控制器的设置如何，但这可能与依赖于硬件。

编写 PowerTOP 与 `sysfs` 的接口，但它只通过读取 `/sys/class/net/` 和 `/sys/bus/usb/devices/`（包括到 `/sys/devices/` 的符号链接）来列出网络和 USB 设备的唤醒触发器。

*/sys/class/wakeup/**

几乎所有的唤醒触发器都可以在 `/sys/class/wakeup` 目录中找到，该目录包含指向所有相关设备的符号链接。这对于通过子目录查找可能的唤醒触发器非常有用。一些触发器可以对应于虚拟设备，而与硬件相关的唤醒触发器至少包含以下文件之一：

```
/sys/class/wakeup/*/device/physical_node/power/wakeup
/sys/class/wakeup/*/device/power/wakeup
```

`/sys/class/wakeup` 中的一些唤醒触发器还提供了一个以下文件指向所在位置隐藏的 `/proc/acpi/wakeup` 名称的链接：

```
/sys/class/wakeup/*/device/path
```

持久设置

*原先的*方法应该足以设置 `/proc/acpi/wakeup` 状态和 `acpi.ec_no_wakeups` 内核参数，而 udev 的*事件驱动*途径是配置 `sysfs` 设备的可靠方式。

使用 systemd 的原先方法

`ec_no_wakeups` ACPI 内核模块选项可以在启动时设置，如本文所述。在引导时设置 `sysfs` 值的标准解决方案是 systemd services，例如在此故障排除中。`/proc/acpi/wakeup` 的另一个基于 systemd 的管理器是 wakeup-triggersAUR。

一些系统可以在电源状态转换时覆盖一些 ACPI 唤醒触发器，这更像是一个 bug，而不是一个功能。如果硬件能在可预测的时间覆盖触发器，则仍然可以使用适当构建的 systemd units 来解决。`sleep.target` 是一个通用目标，涵盖所有不同的挂起状态，在这种情况下可能会有所帮助，但没有通用的 `wakeup.target` 。

此方法仅适用于始终连接的 `sysfs` 设备。

使用 udev 的事件驱动

使用 udev 规则设置唤醒触发器状态是一种事件驱动的方法，在连接具有唤醒触发器的设备时都能可靠地工作。关键是检测规则中是否添加了新设备（`ACTION=="add"`），并将唤醒触发器状态设置为 `ATTR{power/wakeup}="disabled"`。如果硬件正在重置此设置，udev 可以尝试在每次设备更改时重新应

用规则来规避此设置（`ACTION=="add|change"`）。通过 `udevadm info -q all -a /sys/devices/...` 可以获得一个设备树，其中包含用于匹配 `sysfs` 中找到的特定设备的可能参数。

这里一个典型的常见例子是罗技统一 USB 接收器。它的唤醒触发器应该在默认情况下启用，如果不需要，解决方案可以是 udev 规则，如下所示:

```
/etc/udev/rules.d/logitech-unifying.rules
ACTION=="add", SUBSYSTEM=="usb", DRIVERS=="usb", ATTRS{idVendor}=="046d",
ATTRS{idProduct}=="c52b", ATTR{power/wakeup}="disabled"
```

在 udev 文章中描述了启用必要触发器的相反情况。

udev 触发器在设备枚举中出现得太早，以至于使用上述方法禁用唤醒触发器会导致（某些？）禁用的触发器不在 `/sys/class/wakeup` 中列出。这可能取决于设备在启动时是否已经存在，需要进一步阐明。

疑难解答

列出设备和/或总线树

当试图理解某个系统的所有唤醒触发器时，这些辅助命令会很有用，有助于编写 udev 规则或进行常见的唤醒源故障排除：

```
# lshw -businfo -numeric
# lspci -DPPnn
# lsusb -tvv
```

挂起后被立即唤醒

对于某些带有 LynxPoint 或 LynxPoint-LP 芯片组的 Intel Haswell 系统，有报告称它们会在挂起后就立即被唤醒。这与错误的 BIOS ACPI 实现，以及 `xhci_hcd` 如何在引导期间如何解释它们都有关。作为一项解决方案，内核会将报告的受影响系统逐个添加到拒绝列表（名为 `XHCI_SPURIOUS_WAKEUP`）。

电脑也有可能在其它情况下发生立即恢复，比如，一个 USB 设备在挂起后插入，且启动了 ACPI 唤醒触发器。对于这样的情况，如果受影响的系统尚未被加入上述拒绝列表，一个可行的方案是禁用相关的唤醒触发器。通过 USB 禁用唤醒的例子见后。

要查看当前配置：

```
$ cat /proc/acpi/wakeup
Device  S-state   Status   Sysfs node
...
EHC1      S3    *enabled  pci:0000:00:1d.0
EHC2      S3    *enabled  pci:0000:00:1a.0
XHC       S3    *enabled  pci:0000:00:14.0
...
```

可以看到相关的设备有 `EHC1`、`EHC2` 和 `XHC`（USB 3.0设备）。要切换它们的状态，你需要以 root 权限把设备名写入到文件里。

```
# echo EHC1 > /proc/acpi/wakeup
# echo EHC2 > /proc/acpi/wakeup
# echo XHC > /proc/acpi/wakeup
```

这能使挂起重新工作。但是，此设置只是临时的，需要在每次启动时设置。要实现自动化，请参阅 Systemd#临时文件或论坛帖子以了解可能的解决方案。

nouveau 驱动程序

如果使用了 nouveau 驱动程序，则立即唤醒的原因可能是驱动程序中的错误，这有时会阻止 GPU 挂起。一种可能的解决方法是在休眠前卸载 `nouveau` 内核模块，并在唤醒后重新加载。为此，请创建以下脚本：

```
/usr/lib/systemd/system-sleep/10-nouveau.sh
#!/bin/bash

case $1/$2 in
  pre/*)
    # echo "进入 $2 状态..."
    /usr/bin/echo "0" > /sys/class/vtconsole/vtcon1/bind
    /usr/bin/rmmod nouveau
    ;;
  post/*)
    # echo "从 $2 状态唤醒..."
    /usr/bin/modprobe nouveau
    /usr/bin/echo "1" > /sys/class/vtconsole/vtcon1/bind
    ;;
esac
```

第一个 echo 行将 `nouveaufb` 从帧缓冲控制台驱动程序（`fbcon`）中解除绑定。通常它是 `vtcon1`，如本例所示，但也可能是另一个 `vtcon*`。查看 `/sys/class/vtconsole/vtcon*/name` 其中哪一个是 *frame buffer device* 。

4.6 Acpid

难度 进阶 | **篇幅** 10.3 KB | **所属** 电源管理

本节将讲到

- 安装
- 配置
- 小技巧
- 参考

acpid2是用于处理电源相关事件的守护进程，它非常灵活且易于扩展。当某个事件发生时，执行相关程序来处理该事件。这些事件是由某些动作触发的，比如：

- 按下电源按钮
- 按下睡眠/挂起按钮
- 合上笔记本盖子
- 拔下/插上笔记本外接电源

警告：请注意桌面环境比如GNOME，systemd 和 额外按键处理进程会有它自己的一套管理方法。同时运行多套系统可能产生意想不到的结果，比如，当按下电源键时电脑同时执行挂起和关机；或者当按下睡眠按钮时电脑执行了两次挂起操作。所以，使用多套系统时你应只激活一套系统的电源事件管理方法，以免引起冲突。

安装

使用Pacman 安装 acpid包。 然后 start/enable `acpid.service`.

配置

确定事件名称

不幸的是计算机上的这些电源事件标识并不统一，比如，在一些计算机上睡眠按钮可能被标识为*SLPB*，而在另一些计算机上为*SBTN*。

要想确定各种按钮或`Fn`快捷键在你的计算机上是如何定义的：

```
# journalctl -f
```

现在在你计算机上按下电源按钮或睡眠按钮（比如`Fn+Esc`），你会得到类似以下的结果：

```
logger: ACPI action undefined: PBTN
logger: ACPI action undefined: SBTN
```

如果这不起作用的话，运行：

```
# acpi_listen
```

或者openbsd-netcat包：

```
$ netcat -U /var/run/acpid.socket
```

然后按下电源按钮，你会看到类似以下输出：

```
button/power PBTN 00000000 00000b31
```

`acpi_listen`的输出会被作为$1, $2 , $3 & $4参数发送给`/etc/acpi/handler.sh`。 举例：

```
$1 button/power
$2 PBTN
$3 00000000
$4 00000b31
```

定义事件动作

acpid包预置了许多事件触发行为，比如它定义了当你按下电源按钮时应当发生什么。这些触发行为默认在`/etc/acpi/handler.sh`中定义。
在`/etc/acpi/events/anything`)规定：任何侦测到的电源事件都会按照`/etc/acpi/handler.sh`中定义的触发行为执行相关动作。

下面是一个定义触发行为的简单例子。在这个例子中，当按下睡眠按钮时acpid运行命令`echo -n mem >/sys/power/state`，这将会使你的电脑挂起：

```
button/sleep)
    case "$2" in
        SLPB) echo -n mem >/sys/power/state ;;
        *)    logger "ACPI action undefined: $2" ;;
    esac
    ;;
```

如果睡眠按钮被识别为*SBTN*，而不是 `/etc/acpi/handler.sh`中默认定义的*SLPB*（这是标识符）。需要编辑`/etc/acpi/handler.sh`把*SLPB)*替换为*SBTN)*。

参照上面的例子，你应该可以很容易的通过定制`/etc/acpi/handler.sh`来根据侦测到的电源时间来执行不同的命令。更多例子可参考下面的小技巧部分。

注意：Events such as `button/power`, `button/lid`, `button/suspend` and `button/hibernate` are handled by systemd-logind.service(8) by default, see Power management#Power management with systemd. If handling these events with *acpid*, the handling of these events by logind should either be disabled first, or inhibited.

其它配置方案

默认所有的电源事件都是交由`/etc/acpi/handler.sh`处理的。这是由`/etc/acpi/events/anything`规定的：

```
# Pass all events to our one handler script
event=.*
action=/etc/acpi/handler.sh %e
```

尽管这样配置工作起来没有任何问题，但一些用户可能更喜欢使用各自独立的脚本来定义不同的电源事件。下面举例说明了如何使用不同的事件定义文件和行为定义文件：

作为root，创建以下文件：

```
/etc/acpi/events/sleep-button
event=button sleep.*
action=/etc/acpi/actions/sleep-button.sh %e
```

然后建立以下文件：

```
/etc/acpi/actions/sleep-button.sh
#!/bin/sh
case "$3" in
    SLPB) echo -n mem >/sys/power/state ;;
    *)    logger "ACPI action undefined: $3" ;;
esac
```

最后，给脚本加上 executable 可执行权限： 并 reload `acpid.service`。

这种方法可以简化的独立事件和行为脚本的处理。

小技巧

注意：一些在这里描述的动作，例如无线网络切换和背光控制，可能已经由驱动直接管理。在这种情况下，您应该查询相应的内核模块。

脚本示例

下面给出的例子可以用在你的`/etc/acpi/handler.sh`脚本中，只是在使用的时候记得将事件变量更改为`acpi_listen`侦测到的那个。

实现插拔外接电源时自动设置屏幕亮度（数值可能需要调整，参考`/sys/class/backlight/acpi_video0/max_brightness`）：

```
ac_adapter)
    case "$2" in
        AC*|AD*)
            case "$4" in
                00000000)
                    echo -n 50 > /sys/class/backlight/acpi_video0/brightness
                    ;;
                00000001)
                    echo -n 100 > /sys/class/backlight/acpi_video0/brightness
                    ;;
            esac
```

Enabling backlight control

Similar to volume control, acpid also enables you to control screen backlight. To achieve this you write some handler, like this:

```
/etc/acpi/handlers/bl
#!/bin/sh
bl_dev=/sys/class/backlight/acpi_video0
step=1

case $1 in
  -) echo $(($(< $bl_dev/brightness) - $step)) >$bl_dev/brightness;;
  +) echo $(($(< $bl_dev/brightness) + $step)) >$bl_dev/brightness;;
esac
```

and again, connect keys to ACPI events:

```
/etc/acpi/events/bl_d
event=video/brightnessdown.*
action=/etc/acpi/handlers/bl -
/etc/acpi/events/bl_u
event=video/brightnessup.*
action=/etc/acpi/handlers/bl +
```

音量调整

下面的一系列脚本是用来控制音量的。用上面讲到的方法找到音量键的标识符，然后替换掉下面文件中的事件标识就可以用在自己电脑上了：

```
/etc/acpi/events/vol-d
event=button/volumedown
action=amixer set Master 5-
/etc/acpi/events/vol-m
event=button/mute
action=amixer set Master toggle
/etc/acpi/events/vol-u
event=button/volumeup
action=amixer set Master 5+
```

注意：这些命令在 PulseAudio 下可能有问题。要实现全部功能，以当前用户执行并将设置 `XDG_RUNTIME_DIR` 环境变量 environment variable,例如 `# sudo -u *user* XDG_RUNTIME_DIR=/run/user/*1000* pactl`.

提示：在 Xorg 中禁用或绑定这些音量按键，以避免和其他程序冲突。详情请参考或 Xmodmap.

参阅 .

Disabling ordinary key events

Since b336c96 acpid generates events for some ordinary key presses, such as arrow keys. This results in event/handler spam, visible in system logs or *top*. Events for these buttons can be dropped in the configuration file:

```
/etc/acpi/events/buttons
event=button/(up|down|left|right|kpenter)
action=<drop>
```

获取当前登录的用户名

你可以使用`getuser`函数来获取当前登录的用户名：

```
getuser ()
    {
     export DISPLAY=`echo $DISPLAY | cut -c -2`
     user=`who | grep " $DISPLAY" | awk '{print $1}' | tail -n1`
     export XAUTHORITY=/home/$user/.Xauthority
     eval $1=$user
    }
```

然后这个函数就可以被用在下面的例子中，它可以在你按下电源按钮时正常的关闭 KDE：

```
button/power)
    case "$2" in
        PBTN)
            getuser "$user"
            echo $user > /dev/tty5
            su $user -c "dcop ksmserver ksmserver logout 0 2 0"
            ;;
          *) logger "ACPI action undefined $2" ;;
    esac
    ;;
```

连接到 acpid 套接字

除了规则文件之外，acpid 也接受 UNIX 域套接字连接，默认是 `/var/run/acpid.socket`. 用户程序可以连接到此套接字。

```
#!/bin/bash
coproc acpi_listen
trap 'kill $COPROC_PID' EXIT

while read -u "${COPROC}" -a event; do
    handler.sh "${event[@]}"
done
```

handler.sh 可以参考 `/etc/acpi/handler.sh`.

Disable keyboard and touchpad while laptop lid is closed under Wayland

This example uses inhibited property of input device drivers as a replacement for xinput which does not work under Wayland.

```
       button/lid)
              if echo "$3" | grep -iq "open"; then
                     echo 'LID opened'
                     disabled=0
              fi
              if echo "$3" | grep -iq "close"; then
                     echo 'LID closed'
```

```
                        disabled=1
                fi
                find /sys/devices/platform/i8042/serio* -prune -type d | while IFS= read
-r serionumber; do
                        find $serionumber/input/* -prune -type d | while IFS= read -r
inputdevicepath; do
                                if grep -q -i -e "keyboard" -e "touchpad"
$inputdevicepath/name; then
                                        logger "found $(cat $inputdevicepath/name) and set
to $disabled"
                                        echo $disabled > $inputdevicepath/inhibited
                                fi
                        done
                done
        ;;
```

参考

- http://acpid.sourceforge.net/ - acpid 主页
- Gentoo:ACPI#Configuration

4.7 DSDT

难度 进阶 | **篇幅** 11.1 KB | **所属** 电源管理

本节将讲到

- 在你开始之前
- 自己重新编译
- 使用修改过的代码
- 确认覆盖成功
- 参见

DSDT(Differentiated System Description Table)是ACPI规格的一部分。它提供了关于一个给定系统中受支持的电源事件的信息。ACPI表是由制造商在固件里提供的。通常，Linux遇到的问题是某些ACPI功能的缺失，比如风扇不转，盖子合上时屏幕不熄灭等等。这些问题可以归咎于DSDT是为Windows所定制的。安装后可以打补丁来修复这些问题。这篇文章的目标是分析并且重建一个有错误的DSDT，这样内核就会略过默认的DSDT。

基本上来说，一个DSDT表是运行在ACPI(电源管理)上的代码。

注意：Linux ACPI\[失效链接 2023-09-16 ⓘ\]项目的目标让Linux工作在未修改的固件上。如果你还是觉得这种方法在现代内核上是必要的，那么你应该考虑一下报告bug。

在你开始之前

硬件制造商可能已经发布了更新的固件，修复了与ACPI相关的问题。安装更新的固件通常比这种方法更可取，因为它可以避免重复劳动。

此过程会修改你安装中的一些相当基础的代码。你需要绝对确定所做的更改。你可能还希望在之前克隆你的磁盘。

即使在尝试自己修复DSDT之前，你也可以尝试以下几种捷径：

让内核报告某个Windows版本

使用变量`acpi_os_name`作为内核参数。例如：

```
acpi_os_name="Microsoft Windows NT"
```

要添加一个被识别的操作系统接口，使用变量`acpi_osi`。

```
acpi_osi="Linux"
```

要仅使用一个操作系统接口，添加`acpi_osi=!`。这会告诉固件只有一个支持的操作系统，因此这通常是推荐的解决方案。

```
acpi_osi=! acpi_osi="Windows 2022"
```

要移除一个接口，在字符串的开头使用感叹号。

```
acpi_osi="!Windows 2012"
```

其他可以测试的字符串：

- "Microsoft Windows XP"
- "Microsoft Windows 2000"
- "Microsoft Windows 2000.1"
- "Microsoft Windows ME: Millennium Edition"
- "Microsoft WindowsME: Millennium Edition"
- "Windows 2001"
- "Windows 2006"
- "Windows 2009"
- "Windows 2012"
- "Windows 2015"
- "Windows 2020"

提示：虽然不推荐，但你可以尝试"Linux"。

出于好奇，你可以按照以下步骤提取你的DSDT并搜索*.dsl*文件。只需grep "Windows"并查看结果。

找到一个修复过的DSDT

DSDT文件最初是用ACPI源语言（一个*.asl*/*.dsl*文件）编写的。使用编译器可以生成一个'ACPI机器语言'文件（*.aml*）或一个十六进制表（*.hex*）。要将该文件整合到你的Arch安装中，你需要获取一个编译好的*.aml*文件——无论是自己编译还是信任互联网上的某个陌生人，这取决于你。如果你从互联网上下载文件，它很可能是一个压缩的*.asl*文件。因此，你需要解压并编译它。这样做的好处是你不必自己研究特定的代码修复。

像你一样使用同一种笔记本的Arch用户是少数中的少数中的少数。尝试浏览其他发行版/Linux论坛，寻找关于相同型号的讨论。很可能他们有相同的问题，并且因为有很多人，或者因为他们技术娴熟——有人已经制作了一个可用的DSDT，甚至可能提供预编译版本（再次提醒，使用时需自行承担风险）。 搜索引擎是你的最佳工具。尝试保持简短：'型号名称' + 'dsdt' 可能会产生结果。

自己重新编译

在这个过程中你的最好的资源将会是ACPI Spec homepage，和Linux ACPI Project\[失效链接 2023-09-16 ⓘ\]，这些将会替代*acpi.sourceforge.net*。 简而言之，你可以使用英特尔的ASL编译器将你系统的DSDT表转换成源代码，定位并修复错误，最后重新编译。

你将会需要安装acpica包来修改代码。 **什么编译器编译了原始代码?** 查看你系统的DSDT使用英特尔的还是微软的编译器编译的：

```
 # dmesg | grep DSDT
ACPI: DSDT 00000000bf7e5000 0A35F (v02 Intel  CALPELLA 06040000 INTL 20060912)
ACPI: EC: Look up EC in DSDT
```

如果是微软的编译器，INTL将会变成MSFT。 在这个例子中，在反编译/编译的DSDT时共有五个错误，有两个在谷歌一下和研究ACPI规范之后很容易解决。另外三个是由于使用了不同版本的编译器。后来你会发现，它们三个会在启动时被ACPICA处理。内核的ACPICA部分可以处理编译DSDT时产生的的大部分不重要的错误.所以如果你的系统正在*按照正常方式运行*，请不要为了编译错误烦恼。

1.) 提取ACPI表(作为超级用户): `# cat /sys/firmware/acpi/tables/DSDT > dsdt.dat`

2.) 反编译: `iasl -d dsdt.dat`

3.) 重新编译: `iasl -tc dsdt.dsl`

4.) 检查错误并修复。例如:

```
dsdt.dsl    6727:                            Name (_PLD, Buffer (0x10)
Error    4105 -          Invalid object type for reserved name ^  (found BUFFER,
requires Package)
 nano +6727 dsdt.dsl
(_PLD, Package(1) {Buffer (0x10)...
```

5.) 增添OEM版本，否则内核不会应用修改过的ACPI表。例如在修改前:

```
DefinitionBlock ("DSDT.aml", "DSDT", 2, "INTEL ", "TEMPLATE", 0x00000000)
```

修改后:

```
DefinitionBlock ("DSDT.aml", "DSDT", 2, "INTEL ", "TEMPLATE", 0x00000001)
```

6.) 编译被修复过的代码: `iasl -tc dsdt.dsl` (你可能需要使用命令行参数`-ic`来将C语言头文件插入内核)

如果没有错误，你应该可以接着往下进行了。

使用修改过的代码

警告：每次BIOS更新后你都要重新修复DSDT表并且重复这些步骤！

至少有两种方式来使用定制的DSDT:

- 创建一个由内核在启动早期加载的未压缩CPIO文档
- 把它编译进内核

使用mkinitcpio的acpi_override钩子

mkinitcpio提供了一个`acpi_override`钩子，它会将`/usr/initcpio/acpi_override/`和`/etc/initcpio/acpi_override/`中找到的所有*.aml*文件放入`/kernel/firmware/acpi/`中的一个早期未压缩CPIO文档中。这避免了手动创建单独的CPIO文档或更改引导加载程序配置的需要，因为*mkinitcpio*将未压缩的CPIO文档与主initramfs映像打包到一个文件中。

首先，创建`/etc/initcpio/acpi_override`目录并将所有需要的*.aml*文件复制到其中。例如：

```
# mkdir /etc/initcpio/acpi_override
# cp dsdt.aml /etc/initcpio/acpi_override/
```

将`acpi_override`添加到`/etc/mkinitcpio.conf`中的`HOOKS`数组：

```
/etc/mkinitcpio.conf
HOOKS=(... acpi_override)
```

最后，重新生成initramfs并重启。

使用一个CPIO文档

这个方法有一个优点就是你不用重新编译你的内核，升级内核后你也无需重复这些步骤

这个方法要求内核参数`ACPI_TABLE_UPGRADE=y`被启用(对于linux包被设置为true)。浏览 来查看更多详细内容。

首先，创建下面的目录结构:

```
$ mkdir -p kernel/firmware/acpi
```

将修复过的ACPI表拷贝进刚刚创建的`kernel/firmware/acpi`文件夹，例如:

```
$ cp dsdt.aml ssdt1.aml kernel/firmware/acpi
```

在新创建的`kernel/`目录所在的目录下，运行:

```
$ find kernel | cpio -H newc --create > acpi_override
```
这条命令会创建含有修复了的ACPI表的CPIO文档。将文档拷贝到`boot`目录。

```
# cp acpi_override /boot
```
最后，配置bootloader来加载你的CPIO文档。例如，使用Systemd-boot的话，`/boot/loader/entries/arch.conf`可能看起来像这样:

```
title   Arch Linux
linux   /vmlinuz-linux
initrd   /acpi_override
initrd  /initramfs-linux.img
options  root=PARTUUID=ec9d5998-a9db-4bd8-8ea0-35a45df04701 resume=PARTUUID=58d0aa86-d39b-4fe1-81cf-45e7add275a0 ...
```
现在，剩下需要做的事情就是重启电脑并且确认结果。

编译进内核

你会想要熟悉编译自己的内核。最直接的方式就是用“传统”方法。 编译DSDT之后，iasl产生两个文件:`dsdt.hex`和`dsdt.aml`。

使用 `menuconfig`:

- 禁用"Select only drivers that don't need compile-time external firmware"。位于"Device Drivers -> Generic Driver Options"。
- 启用"Include Custom DSDT"并且指明你修复过的DSDT文件的绝对路径(`dsdt.hex`，而不是`dsdt.aml`)。位于"Power management and ACPI options -> ACPI (Advanced Configuration and Power Interface) Support"。

在GRUB中使用AML

如果你使用GRUB，你可以使用一个更简单的方法。将上面创建的*.aml*文件复制到你的引导分区：

```
# cp dsdt_patch.aml /boot
```
然后在你的GRUB配置中添加以下行：

```
acpi /dsdt_patch.aml
```
你可以将其添加到`/etc/grub.d/40_custom`中，别忘了之后生成你的GRUB配置。

在dracut中使用AML

如果你使用Dracut，你可以简单地将上面创建的*.aml*文件复制到一个定义的位置。必须创建一个相应的配置文件`/etc/dracut.conf.d/acpi-fix.conf`：

```
acpi_override="yes"
acpi_table_dir="/usr/local/lib/firmware/acpi"
```

确认覆盖成功

查找确认覆盖的消息，例如：

```
# dmesg | grep ACPI
```

```
[    0.000000] ACPI: Override [DSDT-   A M I], this is unsafe: tainting kernel
[    0.000000] ACPI: DSDT 00000000be9b1190 Logical table override, new table:
ffffffff81865af0
[    0.000000] ACPI: DSDT ffffffff81865af0 0BBA3 (v02 ALASKA    A M I 000000F3 INTL
20130517)
```

参见

- 通过 initrd 升级 ACPI 表
- 如何使用 _OSI 在 ACPI 中识别 Windows 版本
- 如何使用 dracut 覆盖 ACPI 表

4.8 Pm-utils

难度 进阶 | **篇幅** 13.5 KB | **所属** 电源管理

本节将讲到

- 安装
- 基本配置
- 高级配置
- 工作原理
- 已知问题

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

pm-utils 是一个全新的电源挂起和电源状态设置框架。它被设计来替代 `powersave` 包中的相关脚本。

通常HAL使用pm-utilsl来执行一系列的hacks来解决驱动所不能提供的电源挂起方面的操作。可以很方便的在特定目录当中配置自定义的挂钩，挂钩可以由管理员来建立或者是被包含在安装包当中，如果安装包需要监视电源的挂起和电源状态的改变。

可以和 Cpufrequtils 包联合使用，为笔记本和台式机提供完整的电源管理方案。

安装

pm-utilsAUR 包可以从官方软件仓库安装：

```
# pacman -S pm-utils
```

注意：如果你在 resuming video 时遇到问题，你也许需要从官方源同时安装 vbetool包。

注意：如果你从清洁安装开始，确保你已经安装了 acpi包。

基本配置

挂起 / 休眠到内存

理想情况下，运行 `sudo pm-suspend` 将会保存运行程序的状态到内存并关闭内存之外的设备以节省电能。按下电源键会唤醒电脑。

有时 pm-suspend 会挂住，无法正常完成。原因可能是 "不正常" 的模块。如果知道是哪个模块导致问题，可以在 `/etc/pm/config.d/modules` SUSPEND_MODULES 配置项中加入这些模块：

```
SUSPEND_MODULES="uhci_hd button ehci_hd"
```

在挂起和唤醒电脑时，程序会对它们进行特别处理。

要在合上笔记本盖等电源事件发生时自动执行 pm-suspend，请参考 Acpid。

待机 / 休眠到磁盘

要使用休眠功能，需要以root的身份编辑 `/boot/grub/menu.lst` ，并在内核选项中添加 **resume=/path/to/swap/drive** (e.g. /dev/sda2) 例如：

```
# (0) Arch Linux
title  Arch Linux
root   (hd0,0)
kernel /vmlinuz-linux root=/dev/sda3 resume=/dev/sda2 ro
initrd /initramfs-linux.img
```

当计算机进入休眠后，计算机会把内存中的所有信息保存到交换分区（swap partition）... 那么要求你的交换分区有足够的空间来保存RAM信息。

Raid swap :

```
# (0) Arch Linux
title  Arch Linux
root   (hd0,0)
kernel /vmlinuz-linux root=/dev/md2 resume=/dev/md0 ro md=0,/dev/sda2,/dev/sdb2
md=2,/dev/sda5,/dev/sdb5 vga=773
initrd /initramfs-linux.img
```

如果你想要使用UUID来代替磁盘编号(UUID使用blkid命令找出，需要超级用户权限):

```
# (0) Arch Linux
title  Arch Linux
root   (hd0,0)
kernel /vmlinuz-linux cryptdevice=/dev/sda2:main root=/dev/mapper/main-root
resume=/dev/disk/by-uuid/1d893194-b151-43cd-a89e-6f89bd8b9f99 ro
initrd /initramfs-linux.img
```

当你的机器休眠的时候，数据会从内存转向swap分区，所以最好swap空间至少要等于内存的。

对于**GRUB2**，需要修改 `/boot/grub/grub.cfg`的"linux"行。这是个例子（注意修改你的UUID）：

```
linux   /boot/vmlinuz-linux root=/dev/disk/by-uuid/818dc030-8108-4428-8859-b73a58d0b0f3
ro  quiet resume=/dev/sda2
```

关于永久性的讨论,请移步(英文) 论坛.

即使你的swap分区小于内存,也有很大的可能成功的休眠.根据 kernel documentation, */sys/power/image_size* *控制休眠到磁盘镜像的大小*,其中有一个500M的默认值。*休眠将竭尽所能的确保镜像的大小不会超过这个阀值*。你可以增加或者减少swap分区的大小来提高休眠的速度.

注意：你必须添加`resume` 钩子到 `/etc/mkinitcpio.conf`文件中, 移步 常见问题解答!

在非root下执行挂起/休眠

因为必须在root权限下允许 `pm-utils` 脚本，一般情况下，你希望没有root密码的普通用户来执行这些脚本。这时，你需要使用visudo来编辑 `/etc/sudoers` 文件， 例如：

```
# visudo
```

添加如下几行，记住把 *username* 替换成你自己的登录名：

```
username  ALL = (ALL) NOPASSWD: /usr/sbin/pm-hibernate
username  ALL = (ALL) NOPASSWD: /usr/sbin/pm-suspend
```

保存并推出 visudo

之后，你可以在普通用户下允许下面的脚本：

```
$ sudo pm-hibernate
```

或者是

```
$ sudo pm-suspend
```

同时，把你的用户加入到 *power* 组，这样桌面小应用程序可以使用挂起操作。如果你不这样做，当你使用如 gnome shutdown 的程序来进行挂起/休眠操作的时候，计算机会发出烦人的声音和锁住屏幕。

```
# gpasswd -a username power
```

完成以上步骤后，你可以使用 gnome 的电源管理工具 (也许也能使用 kpowersave) 来进行自动的挂起/休眠，比如在合上笔记本上盖和电池电量不足的时候...

高级配置

主要的配置文件是**`/usr/lib/pm-utils/defaults`**. 但是建议你不要去修改这个文件，因为此文件会随着软件包的升级而被重置。如果需要配置可以修改 Y**`/etc/pm/config.d/`** 。

你可以将下列简单的配置以"modules" 或 "config"的命名加入`/etc/pm/config.d` 中，这个设置将会替代系统的一般配置。

```
SUSPEND_MODULES="button uhci_hcd"
```

配置文件中一些可以使用的变量

```
SUSPEND_MODULES="button" # 列表当中的模块将在系统挂起时被卸载。
```

如何关闭监测（钩子，hook）程序

如果你不喜欢某个监测程序或觉得某个监测程序无用甚至影响正常使用的话，你可以将他视为一个Bug来报告。当然，我们也可以很简单的去禁用它。你可以通过管理`/etc/pm/sleep.d/`目录中所对应设备的文件控制监测程序的运行与否。例如：

```
touch /etc/pm/sleep.d/45pcmcia
```

你就可以建立一个`/usr/lib/pm-utils/sleep.d/45pcmcia`文件，来禁用pcmcia。

对于这个假的钩子程序不要设置可执行属性位。

建立你自己的钩子

如果你想建立一些只属于你自己的挂起/休眠设置，你只需要简单的把你的钩子放到`/etc/pm/hooks`。这个文件夹中的钩子会在挂起的时候按照字母顺序执行，并在唤醒过程中按照字母逆序执行（因此通常它们的名字以两位数字开头使得顺序更加明了）。

以下展示的是一个非实用的钩子，它会把一些信息写入你的log文件：

```
#!/bin/bash
case $1 in
    hibernate)
        echo "Hey guy, we are going to suspend to disk!"
        ;;
    suspend)
        echo "Oh, this time we are doing a suspend to RAM. Cool!"
        ;;
    thaw)
        echo "oh, suspend to disk is over, we are resuming..."
        ;;
    resume)
        echo "hey, the suspend to RAM seems to be over..."
        ;;
    *)  echo "somebody is calling me totally wrong."
        ;;
esac
```

把以上内容写入`/etc/pm/sleep.d/66dummy`文件中，并执行`chmod +x /etc/pm/sleep.d/66dummy`，之后在挂起/唤醒过程中它将产生一些无实际作用的行。

***注意：** 所有的钩子都以root用户执行。这意味着你需要在创建临时文件、检查PATH环境变量等时候多加小心，以避免安全问题。*

工作原理

这个概念看起来很简单：这些系统脚本 (`pm-suspend`、`pm-hibernate`、`pm-suspend-hybrid`)将会运行名为"hooks"的可执行文件，这些脚本将会按照字母顺序进行挂起或休眠。一旦hooks运行成功，电脑就会进入休眠状态。直到下次唤醒PC时，所有的脚本按照相反的顺序从内存或者磁盘恢复工作。hooks做了很多事情，例如准备启动，停止蓝牙工作，或者卸载相关的模块。 通常，挂起和休眠都会从HAL或者桌面部件中启用（例如gnome-power-manager、 kpowersave。）

注意： *`suspend-hybrid` 还是一个半成品，并不保证一定可用。*

也可以设置电脑在高功耗或低功耗模式下工作，利用这条命令`pm-powersave` 使用参数`true` 或 `false`。它的工作状况基本与挂起的情况相同。

管理挂起的hooks文件在以下这些地方：

- `/usr/lib/pm-utils/sleep.d` (发行版提供)
- `/etc/pm/sleep.d` (系统管理员添加)

管理电源状态的hooks文件在以下这些地方：

- `/usr/lib/pm-utils/power.d` (发行版提供)
- `/etc/pm/power.d` (系统管理员添加)

在`/etc/pm/` 的HOOKS会优先于 `/usr/lib/pm-utils/`内的HOOKS, 那么作为系统管理员就可以仅仅设置`/etc/pm/`而不用理会发行版提供的HOOKS文件。

已知问题

如果你的休眠与挂起功能不能正常工作，那么一些有用的信息可能会输出到这个文件**`/var/log/pm-suspend.log`**，比如运行了哪些HOOK以及他们输出了什么。

Resume Hook

系统有时会提示你需要将`resume` hook 添加到 initrd image中,，否则内核将提醒你没有resume. 那么，使用**ROOT**权限编辑`/etc/mkinitcpio.conf` 并且将 `resume` 添加到 HOOKS 中：

```
HOOKS="base udev autodetect ide scsi sata resume filesystems "
```

这只是一个示范，你的HOOKS可以安装自己的需要编辑。

`resume` 必须放在 'ide', 'scsi' and/or 'sata' *之后*，但必须在 'filesystems'之前。这里应该有一个适当的 'resume' 文件 /lib/initcpio/hooks （修改合适你的配置）, hook文件是'mkinitcpio'包的一部分。

最后，你必须重新制作内核镜像，否则那些变更不会生效。

```
# mkinitcpio -p linux
```

注意: 如果你使用的是自己编译的内核，那么必须用 -p 参数重新制作镜像。

例如这样做:

```
$ cat /etc/pm/sleep.d/50-hdparm_pm
#!/bin/dash

if [ -n "$1" ] && ([ "$1" = "resume" ] || [ "$1" = "thaw" ]); then
       hdparm -B 254 /dev/sda > /dev/null
fi
```

重新启动鼠标

某些笔记本电脑在挂起之后会不能使用。要纠正此问题的一种方法是通过 HOOKS`/etc/pm/hooks`(see hooks\[损坏的链接：无效的章节\])强制重新初始化 PS/2 的驱动程序(这里指 `i8042`)

```
#!/bin/sh
echo -n "i8042" > /sys/bus/platform/drivers/i8042/unbind
echo -n "i8042" > /sys/bus/platform/drivers/i8042/bind
```

我点击的挂起但没有任何反应 / 日志文件在哪里

当你从桌面部件使用挂起功能时没有正确执行，那么请尝试ROOT权限手动执行 `pm-suspend` 或 `pm-hibernate` manually from a root shell in a terminal\[损坏的链接：无效的章节\]. 有可能你会在终端看见一些有用的输出信息。 系统挂起的脚本也会在log留下错误信息logfile at **C2**\[损坏的链接：无效的章节\].

在Openbox 菜单添加睡眠模式

Openbox users can add the new scripts as additional shutdown options within the Openbox menu by adding the items to a new or existing sub-menu in `~/.config/openbox/menu.xml`, for example:

```
<menu id="64" label="Shutdown">
       <item label="Lock"> <action name="Execute"> <execute>xscreensaver-command -
lock</execute> </action> </item>
       <item label="Logout"> <action name="Exit"/> </item>
       <item label="Reboot"> <action name="Execute"> <execute>sudo shutdown -r
now</execute> </action> </item>
       <item label="Poweroff"> <action name="Execute"> <execute>sudo shutdown -h now
</execute> </action> </item>
       <item label="Hibernate"> <action name="Execute"> <execute>sudo pm-
hibernate</execute> </action> </item>
       <item label="Suspend"> <action name="Execute"> <execute>sudo pm-
suspend</execute> </action> </item>
</menu>
```

相关资源

HAL Quirk Site - Common solutions and frequently asked questions Cpufrequtils - CPU Frequency Scaling and CPU Power schemes SpeedStep - More information on CPU frequency scaling (some of which is obsolete)

Credits

This wiki entry was originally sourced from the OpenSUSE Wiki (Licensed under GPL). A big thank you goes to the `pm-utils` developers and documenters for their time.

4.9 Wake-on-LAN

难度 进阶 | **篇幅** 15.2 KB | **所属** 电源管理

本节将讲到

- 硬件设置
- 软件设置
- 触发唤醒
- 杂项
- 故障排除

网络唤醒（Wake-on-LAN,WoL）是一个通过网络来打开计算机的功能。

硬件设置

网络唤醒只在满足以下前提条件时有效：

1. 目标计算机的主板和网卡支持网络唤醒。
2. 目标计算机必须*物理地*连接（使用线缆）到一台路由器或者源计算机才能让网络唤醒正常工作，除非你的无线网卡支持无线唤醒（WoWLAN或WoW）。

你需要在你的BIOS或UEFI中准备：

1. 启用网络唤醒功能。不同的主板制造商使用稍微不同的语言来描述这项功能。查找像是“PCI Power up”“Allow PCI wake up event”或“Boot from PCI/PCI-E”的术语。
2. 如果在BIOS/UEFI中可用，确保ErP是**禁用**的，否则你的网卡将不会有电并且将不会从其他设备受到任何网络唤醒包。

注意：

- 有些主板在关机状态下支持网络唤醒，但有的主板只在睡眠或挂起状态下支持网络唤醒。
- 一些主板受到一个Bug的影响，导致当BIOS的网络唤醒功能启用时立刻或随机在关机后唤醒。

软件设置

在网卡上启用网络唤醒

根据硬件的不同，网卡驱动可能默认关闭网络唤醒。

要查询或更改这项设置，安装ethtool包，确定网络配置#网络接口的名字，并用以下命令来查询：

```
# ethtool interface | grep Wake-on
Supports Wake-on: pumbag
Wake-on: d
```

值*Wake-on*定义了触发唤醒的活动：d（禁用）、p（PHY活动）、u（单播活动）、m（多播活动）、b（广播活动）、a（ARP活动）和g（魔术包活动）。 网络唤醒工作需要值为g，如果不是，使用以下命令启用网卡驱动的网络唤醒功能：

```
# ethtool -s interface wol g
```

注意：为了启用该功能，可能还需要设置u、m和b中的一个以及g。

此命令的操作在下一次重启之后可能不会保留，在这种情况下，必须通过某种机制来重复执行。下面列出了常见的解决方案。

让其持久

systemd.link

可以通过systemd-networkd#link文件进行link-level配置。实际配置由udevnet_setup_link内置程序执行。向网络link文件中添加WakeOnLan：

```
/etc/systemd/network/50-wired.link
[Match]
MACAddress=aa:bb:cc:dd:ee:ff

[Link]
NamePolicy=kernel database onboard slot path
MACAddressPolicy=persistent
WakeOnLan=magic
```

另请参阅systemd.link(5)以获取更多信息。

注意：

- 仅应用第一个匹配的文件。必须包括systemd附带的链接文件/usr/lib/systemd/network/99-default.link的内容，否则接口可能配置错误。
- 需要考虑的是，文件名应该按顺序放在默认的99-default.link之前，例如50-wired.link就可以。
- 此配置仅适用于link级别，并且独立于像是NetworkManager和systemd-networkd这样的network级别守护程序。
- 在Match部分，OriginalName=也可以用于识别接口。

systemd service

这相当于前面的systemd.link选项，但使用独立的systemd服务。

```
/etc/systemd/system/wol@.service
[Unit]
Description=Wake-on-LAN for %i
Requires=network.target
After=network.target

[Service]
ExecStart=/usr/bin/ethtool -s %i wol g
```

```
Type=oneshot

[Install]
WantedBy=multi-user.target
```

或者安装wol-systemdAUR，然后启动激活这个新服务`wol@*接口*.service`。

udev

udev能够在设备可见时立即运行任何命令。以下规则将在名称与`en*`匹配的所有网络接口上打开网络唤醒。文件名很重要，它必须以81到99之间的数字开头，这样它才能在`80-net-setup-link.rules`之后运行，后者用可预测的名称重命名接口，否则`NAME`将未定义，规则将不会运行。

```
/etc/udev/rules.d/81-wol.rules
ACTION=="add", SUBSYSTEM=="net", NAME=="en*", RUN+="/usr/bin/ethtool -s $name wol g"
```

`$name`占位符将替换为匹配设备的`NAME`变量的值。

cron

每次使用crontab中的"@reboot"重启计算机时，都可以运行一个命令。首先,确保启用cron，然后为root用户编辑含以下行的crontab：

```
@reboot /usr/bin/ethtool -s interface wol g
```

netctl

如果使用netctl，可以通过添加以下netctl配置文件使此设置永久化：

```
/etc/netctl/profile
ExecUpPost='/usr/bin/ethtool -s interface wol g'
```

NetworkManager

NetworkManager提供网络唤醒支持。通过魔术包启用网络唤醒的一种方式是通过 *nmcli*。

首先，搜索有线连接的名称：

```
# nmcli con show
NAME    UUID                                  TYPE             DEVICE
wired1  612e300a-c047-4adb-91e2-12ea7bfe214e  802-3-ethernet   enp0s25
```

通过以下操作可以查看网络唤醒设置的当前状态：

```
# nmcli c show "wired1" | grep 802-3-ethernet.wake-on-lan
802-3-ethernet.wake-on-lan:             default
802-3-ethernet.wake-on-lan-password:    --
```

在连接上启用魔术包网络唤醒：

```
# nmcli c modify "wired1" 802-3-ethernet.wake-on-lan magic
```

然后重启，可能是两次。要禁用网络唤醒，使用`ignore`替换`magic`。

使用nm-connection-editor包GUI也可以更改网络唤醒设置。

你可以通过添加一个专用的配置文件来为所有连接永久禁用网络唤醒：

```
/etc/NetworkManager/conf.d/wake-on-lan.conf
[connection]
```

```
ethernet.wake-on-lan = ignore
wifi.wake-on-wlan = ignore
```

在TLP启用网络唤醒

当使用TLP进行挂起或休眠时，`/etc/tlp.conf`中的`WOL_DISABLE`设置必须设置为`N`来允许使用网络唤醒来恢复电脑。

触发唤醒

要触发目标设备的网络唤醒，必须知道它的**MAC地址**。要获取它，在设备商执行下面的命令：

```
$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp1s0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br0
state UP group default qlen 1000
    link/ether 48:05:ca:09:0e:6a brd ff:ff:ff:ff:ff:ff
```

这里的MAC地址是`48:05:ca:09:0e:6a`。

在最简单的形式中，网络唤醒将魔术包作为以太网帧进行广播，其中包含当前网络子网中IP协议层下方的MAC地址。不需要了解IP地址，因为它在第二层（数据链路层）上运行。

如果用于通过互联网或其它子网唤醒计算机，通常依赖路由器中继数据包并进行广播。在这种情况下，路由器的外部IP地址必须是已知的。请记住，作为安全措施，大部分路由器默认情况下不会中继子网定向广播，需要明确告知这样做。

能够为网络唤醒发送魔术包的应用程序：

- **gWakeOnLAN** — 通过网络唤醒功能来唤醒关闭的电脑的GTK程序。

: https://www.muflone.com/gwakeonlan/english/ || gwakeonlan包

- **wol** — 在一个小的程序中实现局域网唤醒功能。它唤醒服从魔术包的硬件。注意：此应用程序需要需要使用-p参数将端口默认值（40000）改为9。

: https://sourceforge.net/projects/wake-on-lan/ || wol包

- **wol_qt** — 具有集成ARP扫描的Qt程序，用于查找MAC地址和批量发送局域网唤醒数据包。

: https://github.com/stefmitropoulos/wol_qt || wol_qtAUR

在同一局域网中

如果你通过网络线缆直接连接到另一台电脑，或者局域网内的通信没有防火墙，那么使用网络唤醒会很简单，因为无需担心端口重定向。

在最简单的情况下，使用默认广播地址225.225.225.225：

```
$ wol 目标的MAC地址
```

要仅将魔术包广播到特定的子网或主机，请使用-i：

```
$ wol -i target_IP target_MAC_address
```

通过因特网

当源计算机和目标计算机被NAT路由器分隔时，可以设想不同的解决方案：

- 如果路由器支持网络唤醒，则可以通过它将数据包正确广播到本地网络。

否则可以通过端口转发实现局域网唤醒。路由器需要使用以下两个选项之一进行配置：

- 将不同的端口转发到目标计算机。这需要每台目标计算机在它的局域网有一个静态的IP。
- 将单个端口转发到广播地址。大多数路由器不允许转发到广播，但是你可以通过telnet、SSH、串口（console）或其它方式获得访问路由器的shell的权限，并运行下面的命令：

```
$ ip neighbor add 192.168.1.254 lladdr FF:FF:FF:FF:FF:FF dev net0
```

此示例假设网络为192.168.1.0/24，并使用net0作为网络接口。现在，将UDP端口9转发到192.168.1.254。该解决方案已在运行Tomato)的Linksys WRT54G和Verizon FIOS ActionTec路由器上成功测试。有关如何在具有DD-WRT固件上的路由器执行此操作的说明，请参见这个教程；对于具有OpenWrt固件的路由器，请参阅这个教程。

在任何情况下，从源计算机运行以下命令以触发唤醒：

```
$ wol -p forwarded_port -i router_IP target_MAC_address
```

杂项

检查魔术包的接受情况

为了确保网络唤醒数据包到达目标计算机，可以监听UDP端口，通常是端口9，以获取魔术包。魔术包的帧预计会包含6字节的FF，之后是目标计算机的16个重复（每个6字节），总共102字节。

使用netcat

这可以通过在目标计算机上安装gnu-netcat包并使用以下命令来执行：

```
# nc --udp --listen --local-port=9 --hexdump
```

然后等待传入流量出现在nc终端里。

注意：电脑本身的防火墙不需要打开即可网络唤醒（接口的处理发生在NIC中，在防火墙之前）。但是，为了使用netcat进行调试，你仍然需要临时打开该端口。

使用ngrep

在目标设备上安装ngrep包并执行下面的命令：

```
# ngrep '\xff{6}(.{6})\1{15}' -x port 9
```

网络唤醒脚本示例

下面是一个说明了在不同的机器上使用网络唤醒的示例：

```
#!/bin/bash

# definition of MAC addresses
monster=01:12:46:82:ab:4f
ghost=01:1a:d2:56:6b:e6

echo "Which PC to wake?"
echo "m) monster"
echo "g) ghost"
echo "q) quit"
read input1
case $input1 in
  m)
    /usr/bin/wol $monster
    ;;
  g)
    # uses wol over the internet provided that port 9 is forwarded to ghost on ghost's
router
    /usr/bin/wol --port=9 --host=ghost.mydomain.org $ghost
    ;;
  Q|q)
    break
    ;;
esac
```

故障排除

NetworkManager

网络适配器在关机时处于断电状态

如果通过*nmcli*配置了网络唤醒，并且网络适配器在关机时处于断电状态，则将自动协商设置为`yes`可能会有所帮助。设置使用它：

```
# nmcli c modify "wired1" 802-3-ethernet.auto-negotiate yes
```

在关机后唤醒

众所周知,一些主板会受到一个Bug的影响，无论何时启用BIOS的网络唤醒功能（例如这里所讨论的），该错误都会导致关机后立即或随即唤醒。

使用BIOS修复

BIOS首选项中的以下操作可以解决某些主板的问题：

1. 禁用USB设置中所有对*xHCI*的引用（注意，这也会在启动时禁用USB3.0）。
2. 如果*EuP 2013"是一个明确的选项，禁用它。*
3. 可选地启用键盘唤醒操作。

注意：对于上面第三条的值，有各种各样的意见，它可能取决于主板。

通过内核修复

这个问题也可以通过添加`xhci_hcd.quirks=270336`内核参数来解决。这会激活：

- `XHCI_SPURIOUS_REBOOT`
- `XHCI_SPURIOUS_WAKEUP`

电池排空的问题

一些笔记本电脑在关机后会出现电池耗尽的问题。这可能由启用网络唤醒导致。要解决这个问题，像上面提到的一样禁用网络唤醒：

```
# ethtool -s net0 wol d
```

瑞昱

使用基于瑞昱8168 8169 8101 8111（C）的NIC（网卡和板载网卡）时,可能会注意到NIC在启动时似乎被禁用，并且没有指示灯。请参阅网络配置/以太网#Realtek_没有连接/网络唤醒故障。

如果网络交换机上的链路指示灯在计算机关闭但网络唤醒仍不工作时启用，则至少使用r8168内核模块引导系统一次，然后切换回内核中包含r8169的内核模块似乎至少可以在以下配置中解决该问题：

- MSI B85M-E45主板, BIOS版本V10.9, 板载Realtek 8111G 芯片组

对于r8168模块，你可能需要设置 `s5wol=1`内核模块#配置模块参数来启用网络唤醒功能。

alx驱动支持

对于一些较新的基于Atheros的NIC（如Atheros AR8161和Killer E2500），由于一个导致意外唤醒的Bug,主线alx模块中的网络唤醒支持已被禁用（请参阅该讨论）。可以使用补丁（使用alx-wol-dkmsAUR包安装一个dkms模块），该补丁不仅恢复了网络唤醒支持，还修复了潜在的错误，如这里所述。

另请参阅预修复补丁的源代码。

4.10 电源管理

难度 高级 | **篇幅** 36.3 KB | **所属** 电源管理

本节将讲到

- 用户空间工具
- ACPI 事件
- 节能
- 工具和脚本

- 允许用户关机

电源管理这个功能可以在系统组件不工作时切断其电源或将其切换到低耗电模式。

Arch 中的电源管理包含两个主要部分：

1. 配置与硬件交互的 Linux 内核：
2. 内核参数
3. 内核模块
4. udev 规则
5. 配置用户空间工具，这些工具与内核交互并对其事件作出反应。许多用户空间工具还允许以“用户友好”的方式修改内核配置。有关选项，参见#用户空间工具。

用户空间工具

这些工具允许你更改许多设置，而无需手动编辑配置文件。请勿同时运行多个程序，以避免冲突，因为它们的工作方式基本类似。更多电源管理选项请参考电源管理分类。

下面是一些比较流行的节能工具：

命令行工具

- **acpid** — 一个支持 netlink 的 ACPI 电源管理事件分发进程。

: https://sourceforge.net/projects/acpid2/ || acpid包

- **Laptop Mode Tools** — 配置笔记本电源设置的工具，很多人将其视为省电标准工具，需要的配置比较多。

: https://github.com/rickysarraf/laptop-mode-tools || laptop-mode-toolsAUR

- **libsmbios** — 和 Dell SMBIOS tables 交互的库和工具.

: https://github.com/dell/libsmbios || libsmbios包

- **powertop** — 检查电源消耗和电源管理的工具，可以协助省电模式的配置。

: https://github.com/fenrus75/powertop || powertop包

- **powerstat** — 使用 ACPI 或 Intel RAPL 接口测量功耗的工具。

: https://github.com/ColinIanKing/powerstat || powerstatAUR

- **systemd** — 一个系统和服务管理器。

: https://systemd.io/ || systemd包

- **TLP** — Linux 高级电源管理

: https://linrunner.de/tlp || tlp包

- **TuneD** — 执行设备监控和自适应配置的守护进程。

: https://tuned-project.org || tuned包

- **UPower** — 用于枚举电源设备、监听设备事件以及查询历史和统计信息的抽象层。

: https://upower.freedesktop.org || upower包

- **power-profiles-daemon** — 通过 D-Bus 提供电源配置文件处理功能。

: https://gitlab.freedesktop.org/upower/power-profiles-daemon || power-profiles-daemon包

图形化界面

- **batsignal** — 轻量级电池监视器，使用libnotify来警告电池电量不足。

: https://github.com/electrickite/batsignal || batsignal包

- **cbatticon** — 一款在系统托盘中的轻量且快速的电池图标。

: https://github.com/valr/cbatticon || cbatticon包

- **GNOME Power Statistics** — GNOME的电源信息统计工具。

: https://gitlab.gnome.org/GNOME/gnome-power-manager || gnome-power-manager包

- **KDE Power Devil** — Plasma的电源管理模块。

: https://invent.kde.org/plasma/powerdevil || powerdevil包

- **LXQt Power Management** — LXQt的电源管理模块。

: https://github.com/lxqt/lxqt-powermanagement || lxqt-powermanagement包

- **MATE Power Management** — MATE的电源管理工具。

: https://github.com/mate-desktop/mate-power-manager || mate-power-manager包

- **MATE Power Statistics** — MATE的电源信息统计工具。

: https://github.com/mate-desktop/mate-power-manager || mate-power-manager包

- **poweralertd** — 用于传递 UPower 通知的守护进程。

: https://git.sr.ht/~kennylevinsen/poweralertd || poweralertdAUR

- **powerkit** — 一个独立于桌面的轻量级全功能电源管理器。

: https://github.com/rodlie/powerkit || powerkitAUR

- **Xfce Power Manager** — Xfce的电源管理工具。

: https://docs.xfce.org/xfce/xfce4-power-manager/start || xfce4-power-manager包

- **vattery** — 用Vala编写的电池监控应用程序，它将在系统托盘中显示笔记本电脑电池的状态。

: https://www.jezra.net/projects/vattery.html || vatteryAUR

ACPI 事件

systemd 处理一些与电源相关的 ACPI 事件，这些事件的操作可以在 `/etc/systemd/logind.conf` 或 `/etc/systemd/logind.conf.d/*.conf` 中配置——参见 logind.conf(5)。在没有专用电源管理器的系统上，这可能会取代通常用于响应这些 ACPI 事件的 acpid 守护进程。

每个事件的指定操作可以是 `ignore`、`poweroff`、`reboot`、`halt`、`suspend`、`hibernate`、`hybrid-sleep`、`suspend-then-hibernate`、`lock` 或 `kexec`。在休眠和挂起的情况下，必须正确设置。如果事件未配置，*systemd* 将使用默认操作。

事件处理器	描述	默认操作
`HandlePowerKey`	当电源键/按钮被按下时触发。	`poweroff`
`HandleSuspendKey`	当挂起键/按钮被按下时触发。	`suspend`
`HandleHibernateKey`	当休眠键/按钮被按下时触发。	`hibernate`
`HandleLidSwitch`	当盖子关闭时触发，以下情况除外。	`suspend`
`HandleLidSwitchDocked`	当系统插入扩展坞或连接了多个显示器时，盖子关闭时触发。	`ignore`
`HandleLidSwitchExternalPower`	当系统连接到外部电源时，盖子关闭时触发。	`HandleLidSwitch` 设置的操作

要应用更改，请 reload `systemd-logind.service`。

注意：

- *systemd* 无法处理 AC 和电池 ACPI 事件，因此如果你使用 Laptop Mode Tools 或其他类似工具，仍然需要 acpid。

- 当连续快速进行盖子开关操作时，*logind* 会延迟挂起操作最多 90 秒以检测可能的扩展坞。 从 systemd v220 开始，此延迟可配置。 你可以在 `logind.conf` 或其 drop-in 文件中使用例如 `HoldoffTimeoutSec=30s`。

电源管理器

一些桌面环境包含电源管理器，这些管理器会抑制（临时关闭）部分或所有 *systemd* ACPI 设置。如果此类电源管理器正在运行，则 ACPI 事件的操作可以仅在电源管理器中配置。只有在你想配置未被电源管理器抑制的特定事件的行为时，才需要更改 `/etc/systemd/logind.conf` 或 `/etc/systemd/logind.conf.d/*.conf`。

请注意，如果电源管理器没有为适当的事件抑制 *systemd*，你可能会遇到 *systemd* 挂起系统，然后在系统唤醒时，另一个电源管理器再次挂起它的情况。GNOME、MATE、Plasma 和 Xfce 的电源管理器会发出必要的 *inhibited* 命令。如果没有发出 *inhibited* 命令，例如在使用 acpid 或其他工具处理 ACPI 事件时，请将 `Handle` 选项设置为 `ignore`。另见 systemd-inhibit(1)。

xss-lock

xss-lock包 订阅 systemd 事件 `suspend`、`hibernate`、`lock-session` 和 `unlock-session`，并执行适当的操作（运行锁屏程序并等待用户解锁或终止锁屏程序）。*xss-lock* 还会响应 DPMS 事件，并根据需要运行或终止锁屏程序。

例如，Autostart 以下内容：

```
$ xss-lock -- i3lock -n -i background_image.png &
```

节能

注意：参见 笔记本电脑#电源管理 了解特定于笔记本电脑的电源管理，如电池监控。另见特定于 CPU 和 GPU 的页面（例如，Ryzen, AMDGPU）。

本节是创建自定义脚本和节能设置（如 udev 规则）的参考资料。确保设置不由其他实用程序管理，以避免冲突。

这里列出的几乎所有特性都值得使用，无论计算机使用交流电还是电池电源。其中大多数对性能的影响可以忽略不计，只是由于通常的硬件/驱动程序故障而在默认情况下没有启用。减少能源使用意味着减少热量，由于动态超频，在现代英特尔或 AMD CPU 上甚至可以获得更高的性能。

打印电源设置

此脚本打印 USB 和 PCI 设备的电源设置以及各种其他属性。请注意，需要 root 权限才能查看所有设置。

```
#!/bin/bash

for i in $(find /sys/devices/ -name "bMaxPower")
do
        busdir=${i%/*}
        busnum=$(<$busdir/busnum)
        devnum=$(<$busdir/devnum)
        title=$(lsusb -s $busnum:$devnum)

        printf "\n\n+++ %s\n  -%s\n" "$title" "$busdir"

        for ff in $(find $busdir/power/ -type f ! -empty 2>/dev/null)
        do
                v=$(cat $ff 2>/dev/null|tr -d "\n")
                [[ ${#v} -gt 0 ]] && echo -e " ${ff##*/}=$v";
                v=;
        done | sort -g;
done;

printf "\n\n\n+++ %s\n" "内核模块"
for mod in $(lspci -k | sed -n '/in use:/s,^.*: ,,p' | sort -u)
do
        echo "+ $mod";
        systool -v -m $mod 2> /dev/null | sed -n "/Parameters:/,/^$/p";
done
```

支持 Intel HWP (Intel Hardware P-state) 的处理器

**本文或本章节可能需要合并到CPU frequency scaling。**

附注： More context in the main article.（在 Talk:电源管理 中讨论）

支持 HWP 的处理器的可用能源首选项有 `default`, `performance`, `balance_performance`, `balance_power`, `power`。

这可以通过运行

```
$ cat /sys/devices/system/cpu/cpufreq/policy*/energy_performance_available_preferences
```

为了节省更多的能源，您可以通过创建以下文件进行配置：

```
/etc/tmpfiles.d/energy_performance_preference.conf
w /sys/devices/system/cpu/cpufreq/policy*/energy_performance_preference - - - - balance_power
```

参见 x86_energy_perf_policy(8) 手册页获取关于 Intel 处理器能源性能策略的更多信息。另见 systemd-tmpfiles(8) 和 tmpfiles.d(5) 手册页获取临时文件/目录的详细信息。

音频

默认情况下，音频节能功能是否开启取决于驱动程序，例如 HD Audio 默认开启。识别正在使用的模块，然后运行

```
$ modinfo --field=parm 模块名称 | column --separator=':' --table --table-columns-limit=2
```

并查找调整或禁用节能功能的内核模块参数（如 `power_save`）。

注意：节能功能可能会导致可听见的点击声（爆裂声）和其他问题。请参见相关页面的解决方案：

- Advanced Linux Sound Architecture/Troubleshooting#Power saving
- PipeWire#Noticeable audio delay or audible pop/crack when starting playback
- PulseAudio/Troubleshooting#Pops when starting and stopping playback

背光

见背光。

蓝牙

要完全禁用蓝牙，请将 `btusb` 和 `bluetooth` 模块列入黑名单。

或者，创建以下 udev 规则：

```
/etc/udev/rules.d/50-bluetooth.rules
# 禁用蓝牙
SUBSYSTEM=="rfkill", ATTR{type}=="bluetooth", ATTR{state}="0"
```

要仅暂时关闭蓝牙，使用 rfkill(8)：

```
# rfkill block bluetooth
```

网络摄像头

如果你不使用集成的网络摄像头，将 `uvcvideo` 模块加入黑名单。

内核参数

本节使用 `/etc/sysctl.d/` 中的配置，它是*"内核 sysctl 参数的插入式目录"*。有关更多信息，参见 The New Configuration Files 和更具体的 sysctl.d(5)。

禁用 NMI 看门狗

NMI（不可屏蔽中断）看门狗是一个调试特性，用于捕捉导致内核 panic 的硬件中断。在某些系统上，它会产生大量的中断，导致用电量明显增加。要列出自上次启动以来每个 CPU 核心的中断，可以使用：

```
$ grep NMI /proc/interrupts
NMI:     22     58     24     23     24     39     22     30   Non-maskable interrupts
```

要关闭硬锁检测器，请使用：

```
/etc/sysctl.d/disable_watchdog.conf
kernel.nmi_watchdog = 0
```

或者在内核参数中添加 `nmi_watchdog=0`。

或者，在内核参数中添加 `nowatchdog` 以禁用硬锁和软锁检测器。参见

写回时间

增加虚拟内存脏写回时间有助于将硬盘 I/O 聚合在一起，从而减少跨硬盘写操作，并提高节能。要将值设置为60秒（默认值为5秒）：

```
/etc/sysctl.d/dirty.conf
vm.dirty_writeback_centisecs = 6000
```

要对支持的文件系统（例如ext4, btrfs...）上的日志提交执行同样的操作，请在 fstab 中使用 `commit=60` 作为选项。

注意，这个值是作为下面笔记本模式设置的副作用而修改的。另请参阅 Sysctl#虚拟内存了解其他影响 I/O 性能和节能的参数。

笔记本电脑模式

参见笔记本电脑模式“旋钮”的内核文档\-“旋钮的合理值为5秒”。

```
/etc/sysctl.d/laptop.conf
vm.laptop_mode = 5
```

注意：此设置主要与自旋磁盘驱动器有关。

网络接口

网络唤醒（英语：Wake-on-LAN）可能是一个有用的功能，但如果你不使用它，那么在挂起时等待魔术包只会消耗额外的电量。您可以调整网络唤醒#udev（英语：Wake-on-LAN#udev）规则，以禁用所有以太网接口的功能。要在所有无线接口上用 iw包启用节能，请执行以下操作：

```
/etc/udev/rules.d/81-wifi-powersave.rules
ACTION=="add", SUBSYSTEM=="net", KERNEL=="wl*", RUN+="/usr/bin/iw dev $name set
power_save on"
```

配置文件的名称很重要。通过在 systemd 中使用持久设备名称，在设备重命名为持久名称（例如，`wlan0` 重命名为 `wlp3s0`）后，将应用上述网络规则（以 `80-net-setup-link.rules` 的字典命名）。请注意，在处理完所有规则之后，将执行 `RUN` 命令，并且无论如何都必须使用持久化名称，可在 `$name` 中为匹配的设备使用。

Intel 无线网卡（iwlwifi）

通过向内核模块传递正确的参数，可以启用带有 `iwlwifi` 驱动程序的 Intel 无线网卡的其他节能功能。通过将以下行添加到 `/etc/modprobe.d/iwlwifi.conf` 文件中来实现持久化：

```
options iwlwifi power_save=1
```

此选项可能会增加延迟中值：

```
options iwlwifi uapsd_disable=0
```

在低于5.4的内核上，您可以使用此选项，但它可能会降低您的最大吞吐量：

```
options iwlwifi d0i3_disable=0
```

根据您的无线网卡，这两个选项之一将适用。

```
options iwlmvm power_scheme=3
options iwldvm force_cam=0
```

您可以通过使用下列命令检查这些模块中的哪个模块正在运行来检查哪个模块是相关模块

```
# lsmod | grep '^iwl.vm'
```

请记住，这些节能选项是试验性的，可能会导致系统不稳定。

总线电源管理

活动状态电源管理

来自维基百科：

: **活动状态电源管理**（**ASPM**）是 PCI Express 设备在完全活动状态下节省电源的一种机制。主要是通过活动状态链路电源管理实现的；即，当没有流量通过时，PCI Express 串行链路会关闭电源。它通常用于笔记本电脑和其他移动互联网设备以延长电池寿命。

在启动时，BIOS 根据硬件支持启用或禁用 ASPM。要检查支持情况：

```
# lspci -vv | grep 'ASPM.*abled;'
```

获取可用的 ASPM 策略和当前系统默认值：

```
$ cat /sys/module/pcie_aspm/parameters/policy
[default] performance powersave powersupersave
```

ASPM 可能由于以下原因被禁用 ：

1. BIOS 确定需要这样做。
2. PCIE 需要 ASPM，但 L0s 是可选的，因此您可能禁用了 L0s 而仅启用了 L1。
3. 您有一个有问题的 BIOS。
4. 您没有 BIOS，并且您的系统程序员尚未处理 ASPM。

如果您认为您的硬件支持 ASPM，尽管有上述情况，可以通过 `pcie_aspm=force` 内核参数强制启用它。

警告：

- 在不支持的系统上强制启用 ASPM 可能会导致功耗增加。此外，它可能会导致系统冻结或内核 panic，因此请确保如果该选项不合适，您有办法撤消它。
- 强制启用 ASPM 是在内核中进行的，因此它可能仍然在硬件中保持禁用状态并且不起作用。要检查是否是这种情况，请以 root 身份运行 `dmesg | grep ASPM`。如果可能，请查阅特定于硬件的 Wiki 文章以获取更多信息。

只要 ASPM 支持并启用，就可以为当前会话选择所需的策略。例如，切换到 `powersupersave` 作为当前会话的策略：

```
# echo powersupersave > /sys/module/pcie_aspm/parameters/policy
```

要在系统启动时配置特定的 ASPM 状态（以 `powersupersave` 为例），请添加 `pcie_aspm.policy=powersupersave` 作为内核参数。

PCI 运行时电源管理

```
/etc/udev/rules.d/pci_pm.rules
SUBSYSTEM=="pci", ATTR{power/control}="auto"
SUBSYSTEM=="ata_port", KERNEL=="ata*", ATTR{device/power/control}="auto"
```

上述规则会关闭所有未使用的设备。

某些设备不会再次唤醒。要仅对已知可以工作的设备进行运行时电源管理，请使用对应供应商和设备ID的简单匹配（使用 `lspci -nn` 获取这些值）：

```
/etc/udev/rules.d/pci_pm.rules
# PCI 自动挂起的白名单
SUBSYSTEM=="pci", ATTR{vendor}=="0x1234", ATTR{device}=="0x1234", ATTR{power/control}
="auto"
```

或者，将不能使用 PCI 运行时电源管理的设备列入黑名单，并为所有其他设备启用：

```
/etc/udev/rules.d/pci_pm.rules
# PCI 运行时电源管理的黑名单
SUBSYSTEM=="pci", ATTR{vendor}=="0x1234", ATTR{device}=="0x1234", ATTR{power/control}
="on", GOTO="pci_pm_end"

SUBSYSTEM=="pci", ATTR{power/control}="auto"
LABEL="pci_pm_end"
```

USB 自动挂起

当 USB 设备不使用时，Linux 内核可以自动挂起它们。这有时可以节省相当多的电量，但某些 USB 设备与 USB 节能不兼容，并开始出现错误行为（USB 鼠标/键盘常见）。基于白名单或黑名单过滤的 udev 规则可以帮助缓解问题。

以下示例为所有 USB 设备启用自动挂起，但键盘和鼠标除外：

```
/etc/udev/rules.d/50-usb_power_save.rules
ACTION=="add", SUBSYSTEM=="usb", ATTR{product}!="*Mouse", ATTR{product}!="*Keyboard",
TEST=="power/control", ATTR{power/control}="auto"
```

要仅允许已知可以工作的设备进行自动挂起，请使用对应供应商和产品 ID 的简单匹配（使用 *lsusb* 获取这些值）：

```
/etc/udev/rules.d/50-usb_power_save.rules
# USB 自动挂起的白名单
ACTION=="add", SUBSYSTEM=="usb", TEST=="power/control", ATTR{idVendor}=="05c6",
ATTR{idProduct}=="9205", ATTR{power/control}="auto"
```

或者，将不能使用 USB 自动挂起的设备列入黑名单，并为所有其他设备启用它：

```
/etc/udev/rules.d/50-usb_power_save.rules
# USB 自动挂起的黑名单
ACTION=="add", SUBSYSTEM=="usb", ATTR{idVendor}=="05c6", ATTR{idProduct}=="9205",
GOTO="power_usb_rules_end"

ACTION=="add", SUBSYSTEM=="usb", TEST=="power/control", ATTR{power/control}="auto"
LABEL="power_usb_rules_end"
```

默认的自动挂起空闲延迟时间由 `usbcore` 内建内核模块的 `autosuspend` 参数控制。要将延迟设置为5秒而不是默认的2秒，请为引导加载程序添加以下内核参数。

```
usbcore.autosuspend=5
```

与 power/control 类似，可以通过设置 power/autosuspend 属性来微调每个设备的延迟时间。这意味着，也可以通过将 power/autosuspend 设置为-1（即从不自动挂起）来禁用自动挂起：

```
/etc/udev/rules.d/50-usb_power_save.rules
ACTION=="add", SUBSYSTEM=="usb", ATTR{idVendor}=="05c6", ATTR{idProduct}=="9205",
ATTR{power/autosuspend}="-1"
```

参见 Linux 内核文档获取关于 USB 电源管理的更多信息。

SATA 活动链路电源管理

当前设置可以
从 /sys/class/scsi_host/host*/link_power_management_policy 读取或写入：

```
$ grep . /sys/class/scsi_host/host*/link_power_management_policy
$ echo "med_power_with_dipm" >/sys/class/scsi_host/hostN/link_power_management_policy
```

<table class="wikitable"><caption>可用的 ALPM 设置</caption><tbody><tr><th>设置</th><th>说明</th><th>节能</th></tr><tr><td>max_performance</td><td>当前默认值</td><td>无</td></tr><tr><td>medium_power</td><td>-</td><td>~1.0瓦</td></tr><tr><td>med_power_with_dipm</td><td>推荐设置¹</td><td>~1.5瓦</td></tr><tr><td>min_power</td><td>警告：可能丢失数据²</td><td>~1.5瓦</td></tr></tbody></table>

1. 自 Linux 4.15 起，有一个名为 med_power_with_dipm 的设置，它与 Windows IRST 驱动程序设置的行为相匹配，并且不应导致最近的 SSD 或 HDD 数据丢失。节能效果显著，范围从1.0瓦到1.5瓦（空闲时）。它已成为 Linux 4.16 中基于 Intel 的笔记本电脑的默认设置 。在 Linux 6.11 中，它成为默认设置 。
2. > **警告：**min_power SATA 活动链路电源管理设置可能会导致某些设备上的数据丢失。除非经常备份，否则不要启用此设置。

您可以通过添加 udev 规则文件来持久化配置 link_power_management_policy 设置，例如：

```
/etc/udev/rules.d/hd_power_save.rules
ACTION=="add", SUBSYSTEM=="scsi_host", KERNEL=="host*",
ATTR{link_power_management_policy}="med_power_with_dipm"
```

注意：

- 这会增加访问闲置驱动器时的延迟，因此这是少数几个值得根据您是否使用交流电源进行切换的设置之一。
- 并非所有 SATA 主机控制器和存储设备的组合都能与默认的 med_power_with_dipm 设置良好配合。例如：

- 截至 linux-6.8.1，Intel 7 系列芯片组控制器在访问 Crucial M550 SSD（固件 MU02）时会出现超时和链路退化。
- 截至 linux-6.9.6，AMD 600 系列芯片组 SATA 控制器与 HL-DT-ST BDDVDRW GGC-H20L 光盘驱动器一起使用时，会产生令人讨厌的效果，即每隔几分钟就会重新初始化光盘驱动器。

: : 在这种情况下，使用 `medium_power` 设置可以解决问题。

硬盘驱动器

有关可以设置的驱动器参数，请参见hdparm#电源管理配置。

当有太多程序频繁写入硬盘时，节能效果不佳。跟踪所有程序，获取它们如何以及何时写入硬盘以限制磁盘使用。使用 iotop包 查看哪些程序经常使用磁盘。有关其他提示，请参见性能优化#存储设备。

诸如设置 noatime 选项之类的小事情也会有所帮助。如果有足够的 RAM 可用，请考虑禁用或限制交换值)，因为它可能会限制大量的磁盘写操作。

对于具有 PowerChoice 技术的希捷驱动器，通过 hdparm 设置 APM 的技巧将不起作用，因为 *EPC*（扩展电源条件）功能。与其设置 APM，您可以安装 openseachestAUR 并完全禁用 EPC，如下所示（将 *X* 替换为实际驱动器字母）：

```
# openSeaChest_PowerControl --scan
# openSeaChest_PowerControl -d /dev/sdX -i
# openSeaChest_PowerControl -d /dev/sdX --showEPCSettings
# openSeaChest_PowerControl -d /dev/sdX --EPCfeature disable
# openSeaChest_PowerControl -d /dev/sdX --showEPCSettings
```

最后一次调用将给出以下摘要：

```
================================================================================
===
 openSeaChest_PowerControl - openSeaChest drive utilities - NVMe Enabled
 Copyright (c) 2014-2023 Seagate Technology LLC and/or its Affiliates, All Rights
Reserved
 openSeaChest_PowerControl Version: 3.3.1-4_1_1 X86_64
 Build Date: Jul  4 2023
 Today: Tue Jul  4 17:49:36 2023        User: root
================================================================================
===

/dev/sdX - ST1000NM0008-2F2100 - ZFA19JG2 - SN02 - ATA

===EPC Settings===
        * = timer is enabled
        C column = Changeable
        S column = Savable
        All times are in 100 milliseconds

Name       Current Timer Default Timer Saved Timer   Recovery Time C S
Idle A      0             *10           *10           1             Y Y
Idle B      0             *1200         *1200         3             Y Y
Idle C      0              6000          6000         16            Y Y
Standby Z   0              9000          9000         46            Y Y
```

Zeroes in the first column confirm that parking and spindown were disabled successfully

工具和脚本

本文或本章节的语言、语法或风格需要改进。参考：帮助:风格

原因：Merged from Power saving, needs reorganization to fit into this page.（在Talk:电源管理讨论）

使用脚本和 udev 规则

本文或本章节可能需要合并到 Laptop#Power management。

附注： 可能更适合放在笔记本电脑专用页面。（在 Talk:电源管理 中讨论）

由于 systemd 用户可以通过 `systemctl suspend` 或 `systemctl hibernate` 来挂起和休眠，并能使用 `/etc/systemd/logind.conf` 处理 acpi 事件，因此删除 *pm-utils* 和 acpid 可能会很具吸引力。只有一件事 systemd 不能做（自 systemd-204）：电源管理取决于系统是运行在交流电还是电池上。为了填补这个空白，您可以创建一个 udev 规则，在电源适配器被插入和拔出时运行一个脚本：

```
/etc/udev/rules.d/powersave.rules
SUBSYSTEM=="power_supply", ATTR{online}=="0", RUN+="/path/to/your/script true"
SUBSYSTEM=="power_supply", ATTR{online}=="1", RUN+="/path/to/your/script false"
```

注意：您可以使用 *pm-powersave* 所用的相同脚本。您只需要将其设置为可执行并放置在其他地方（例如 `/usr/local/bin/`）。

节能脚本示例：

- ftw, 包：ftw-gitAUR
- powersave
- throttlectl，来自 throttlectlAUR

上述 udev 规则应按预期工作，但如果在挂起或休眠周期后未更新电源设置，则应在 `/usr/lib/systemd/system-sleep/` 中添加一个脚本，其中包含以下内容：

```
/usr/lib/systemd/system-sleep/00powersave
#!/bin/sh

case $1 in
    pre) /path/to/your/script false ;;
    post)
       if cat /sys/class/power_supply/AC0/online | grep 0 > /dev/null 2>&1
       then
              /path/to/your/script true
       else
              /path/to/your/script false
       fi
    ;;
esac
exit 0
```

不要忘记使它可执行！

注意：请注意，AC0 可能与您的笔记本电脑不同，如果是这样，请更改它。

允许用户关机

 **本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Merged from Allow users to shutdown, needs reorganization to fit into this page.（在Talk:电源管理讨论）

按钮和笔记本盖事件

按挂起、关机、休眠按钮以及笔记本盖关闭事件由 *logind* 处理，请参考 #ACPI events。

使用 systemd-logind

如果正在使用 polkit包，只要会话未中断，具有非远程会话的用户就可以发出与电源相关的命令。

检查会话是否处于活动状态：

```
$ loginctl show-session $XDG_SESSION_ID --property=Active
```

然后，用户可以在命令行中使用 *systemctl* 命令，或者将它们添加到菜单中：

```
$ systemctl poweroff
$ systemctl reboot
```

也可以使用 `systemctl suspend` 和 `systemctl hibernate` 等其它命令。请参见 systemctl(1) 中的**系统命令**章节。

使用 sudo

安装 sudo包，并授予用户 sudo 权限。用户就能够使用 `sudo systemctl` 命令（例如，`sudo systemctl poweroff`, `sudo systemctl reboot`, `sudo systemctl suspend` 和 `sudo systemctl hibernate`）。请参见 systemctl(1) 中的**系统命令**章节。

没有 sudo 权限的用户

如果只允许用户使用关闭命令，而不允许用户拥有其他 sudo 权限，那么作为 root 用户，使用 `visudo` 命令将以下内容添加到 `/etc/sudoers` 的末尾。用你的用户名代替 *user*，用机器的主机名代替 *hostname*。

```
user hostname =NOPASSWD: /usr/bin/systemctl poweroff,/usr/bin/systemctl halt,/usr/bin/systemctl reboot
```

现在你的用户可以使用 `sudo systemctl poweroff` 来关机，与使用 `sudo systemctl reboot` 来重启。用户想要将系统断电也能使用 `sudo systemctl halt`。仅当你不想被提示输入密码时使用 `NOPASSWD:` 标签。

另见

- ThinkWiki:如何降低功耗
- 如何在 Linux 上获得更长的电池寿命

4.11　电源管理/挂起与休眠

难度 高级 | **篇幅** 42.0 KB | **所属** 电源管理

本节将讲到

- 内核接口 (swsusp)
- 高级接口 (systemd)
- 更改挂起方法
- 休眠
- 睡眠钩子

有多种方法可用于挂起，特别是：

Suspend to idle（挂起到空闲状态）：英特尔称之为 S0ix，微软称之为现代待机（以前是“保持网络连接的待机”），内核称之为 S2Idle。设计用于替代 **S3** 睡眠状态的支持系统，提供相同的节能，但大大减少了唤醒时间。

提示：虽然这种状态会导致 Windows 或 macOS 上的电池消耗问题，因为它们支持在这种状态下唤醒设备进行网络活动，但 Linux 软件生态系统目前没有使用这种特性，应该不会受到影响。

Suspend to RAM（挂起到内存，通称挂起）：ACPI 定义的 **S3** 睡眠状态。通过将机器中大多数和 RAM 不相关的部件断电来工作，RAM 是恢复机器状态所必需的。由于可以节省大量电力，建议笔记本电脑在使用电池运行且盖子关闭时（或用户在一段时间内处于非活动状态）自动进入此模式。 **Suspend to disk（挂起到硬盘，通称休眠）**：ACPI 定义的 **S4** 睡眠状态。将机器状态保存到交换空间并关闭机器。再次开机后，恢复状态。直到开机前，机器都不会有任何待机功耗。 **Hybrid suspend（混合挂起）**：挂起和休眠的混合，有时称为**挂起到两者**。将机器状态保存到交换空间，但并不关闭电脑，而是调用默认的挂起机制，从而使未掉电的电脑能立刻恢复。如果电脑掉电（断电且电池耗尽），系统也可以从硬盘的交换空间中恢复，尽管比从内存中恢复慢一些。

以上三者均由内核提供基础功能，再由高级接口进行了一定调整，以适配一些较为复杂的硬件与内核模块（如显卡唤醒后的重新初始化）

内核接口 (swsusp)

注意：尽管可以直接使用内核接口，这更快，因为不需要额外的用户空间处理，但建议使用高级接口)。它通过执行额外的安全检查以及提供预挂起和后挂起钩子机制来更好地设置硬件时钟、恢复无线等，从而做得更好。

可以通过直接告知内核中的软件挂起代码（swsusp）进入挂起状态，具体的方法和状态取决于硬件支持的程度。在现代内核中，向 `/sys/power/state` 写入特定字符串是触发此挂起的主要机制。

更多信息详见内核文档。

高级接口 (systemd)

注意：高级接口旨在提供可执行的二进制文件/脚本，用于执行挂起/休眠操作，并提供将额外的准备/清理工作挂钩到所述进程的方法。有关电源按钮、菜单点击或笔记本盖事件自动进入睡眠状态，请参阅 Power management#ACPI events。

systemd 提供了用于挂起、休眠和混合挂起的原生命令。这是 Arch Linux 中使用的默认接口。

`systemctl suspend` 开箱即用。要使 `systemctl hibernate` 在你的系统上运行，你可能需要遵循 #休眠中的说明。

还有两种结合挂起和休眠的模式：

- `systemctl hybrid-sleep` 将系统同时挂起到 RAM 和磁盘，因此完全断电不会导致数据丢失。 此模式也称为“同时挂起”。
- `systemctl suspend-then-hibernate` 最初会尽可能将系统挂起到 RAM 中，然后使用 RTC 警报将其唤醒并休眠。RTC 警报由 systemd-sleep.conf(5) 中的 `HibernateDelaySec` 设置。默认值通过估计电池放电率来设置，以使系统保持 5% 的电池电量，或者在没有电池的情况下保持两小时。 估计值是根据 systemd-sleep.conf(5) 中 `SuspendEstimationSec` 指定的时间后电池电量变化获得的，该系统将短暂唤醒进行测量（如果系统从挂起中手动唤醒，也会进行测量）。

提示：systemd v256 支持使用新的 `systemctl sleep` 命令自动选择最合适的睡眠操作。默认情况下，使用 `suspend-then-hibernate`，如果不支持则回退到 `suspend` 和 `hibernate`。更多信息请参见 systemctl(1)。

关于配置睡眠或休眠钩子，详见电源管理#睡眠钩子。又见 systemctl(1)、systemd-sleep(8) 和 systemd.special(7)。

更改挂起方法

在 S0ix 挂起不能提供与常规 S3 睡眠相同的节能的系统上，或者当节能优于快速的恢复时间时，可以更改默认挂起方法。

提示：S0ix 被认为可以提供与 S3 睡眠相同或更好的节能，请参阅以下英特尔博客文章 如何在 Linux 中实现 S0ix 状态，Linux S0ix 故障排除和在 Linux 上高效空闲：一个案例研究，以检查你是否可以使其按预期工作。英特尔用户可以使用 S0ixSelftestTool。AMD 用户可以使用 amd-debug-tools。

运行以下命令以查看硬件公开支持的所有挂起方法（当前方法包裹在方括号中显示）：

```
$ cat /sys/power/mem_sleep
[s2idle] shallow deep
```

<table class="wikitable sortable"><caption>mem_sleep 中支持的睡眠状态
</caption><tbody><tr><th>mem_sleep 字符串</th><th>睡眠状态
</th></tr><tr><td>s2idle</td><td><a rel="nofollow" class="external text"
href="https://docs.kernel.org/admin-guide/pm/sleep-states.html#suspend-to-
idle">suspend-to-idle</td></tr><tr><td>shallow</td><td><a rel="nofollow"
class="external text" href="https://docs.kernel.org/admin-guide/pm/sleep-
states.html#standby">standby</td></tr><tr><td>deep</td><td><a rel="nofollow"
class="external text" href="https://docs.kernel.org/admin-guide/pm/sleep-
states.html#suspend-to-ram">suspend-to-RAM</td></tr></tbody></table>

如果你的硬件没有公布 `deep` 睡眠状态，请首先检查 UEFI 是否公布了它的某些设置，通常是在“电源（*Power*）”或“睡眠状态（*Sleep state*）”或类似的用词下，并使用名为 *Windows 10*, *Windows and Linux* 或 *S3/Modern standby support* 关于 S0ix 的选项，以及 *Legacy*, *Linux*, *Linux S3* 或 *S3 enabled* 关于 S3 睡眠的选项。否则，你可以继续使用 `s2idle`，考虑使用休眠或尝试给 DSDT 表打补丁（或在线查找补丁版本）。

注意：最后一个解决方案很可能会引起问题。自从 Windows 系统默认使用 “现代待机 ”后，制造商已经停止修复 ACPI S3 状态的错误：如果它们主动地不公布它，它可能会在某种程度上出问题。

通过在更改睡眠方法后测试几个睡眠周期，验证你的硬件没有 S3 睡眠问题：

```
# echo deep > /sys/power/mem_sleep
```

如果没有发现任何问题，可以通过 systemd-sleep.conf(5) 中的 `MemorySleepMode` 选项使更改永久化。

```
/etc/systemd/sleep.conf.d/mem-deep.conf
[Sleep]
MemorySleepMode=deep
```

或者通过`mem_sleep_default=deep` 内核参数。

在一些相反的情况下，有缺陷的固件公布支持 `deep` 睡眠，但只支持 `s2idle`。在这种情况下，通过选项中的 `SuspendState` 可以以另一种方法使用 `s2idle` ：

```
/etc/systemd/sleep.conf.d/freeze.conf
[Sleep]
SuspendState=freeze
```

休眠

为了使用休眠功能，你必须创建一个 swap 分区或文件、#配置 initramfs 以便在早期用户空间启动恢复过程，并指定交换空间的位置以便 initramfs 可以访问，例如：由 systemd 定义的 `HibernateLocation` EFI 变量或 `resume=` 内核参数。下面详细描述了这三个步骤。

注意：

- 使用 encryption 时，请参阅 Dm-crypt/交换分区加密#需要休眠支持。
- linux-hardened包 不支持休眠，请参阅 FS#63648。
- 即使 zram 配置了永久存储上的后端设备，也不支持休眠到 zram 上的交换分区。虽然 *logind* 会阻止尝试将休眠到 zram 上的交换空间，但作为替代，你可以创建多个交换空间。内存将存储到交换文件中，而另一个可用交换空间将保留给 zram。有关详细信息，请参阅 #Maintaining swap file for hibernation with zram。

swap分区/文件的大小

即使交换分区比内存小，成功休眠的可能性仍然不小。参见内核文档中的“image_size”部分以获取关于 `image_size` sysfs(5) 伪文件的更多信息。

你可以减小 `/sys/power/image_size` 的设定值以使休眠镜像尽可能小（对于小交换分区），或者增大它以加快休眠过程（也许）。对于 RAM 很大的系统，较小的值可能会显著提高系统从休眠中恢复的速度。Systemd#systemd-tmpfiles_-_临时文件可以用于固定你的修改。

```
/etc/tmpfiles.d/hibernation_image_size.conf
#    Path                    Mode UID  GID  Age Argument

w    /sys/power/image_size   -    -    -    -   0
```

休眠镜像不能跨多个交换分区和/或交换文件。它必须完全适配一个交换分区或一个交换文件

配置 initramfs

- 当使用一个 busybox-based 的 initramfs 时（这是默认值），`/etc/mkinitcpio.conf` 中需要 `resume` 钩子。无论是通过卷标还是通过 UUID，交换分区都是通过 udev 设备节点来引用的，因此 `resume` 钩子必须在 `udev` 钩子之后。这是一个使用默认钩子配置的示例：

: ``` HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block filesystems resume fsck)

: 编辑后记得[重新生成 initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md#手动生成)。

: > **注意：**如果使用堆叠存储作为交换空间，例如 [dm-crypt](../../08_安全与加密/静态数据加密/Dm-crypt.md)、[RAID](../存储虚拟化/RAID.md) 或 [LVM](../存储虚拟化/LVM.md)，则最终映射的设备必须在早期用户空间中可用，并且在恢复过程启动之前。也就是说，在这样的设置中，`resume` 钩子必须放在诸如 `encrypt`、`lvm2` 等钩子之后。

- 当使用带有 `systemd` 钩子的 initramfs 时，已经提供了恢复机制，不需要额外添加钩子。

传递休眠位置到 initramfs

当系统休眠时，内存映像被转储到交换空间，其中也包括挂载文件系统的状态。因此，休眠位置必须提供给 initramfs，即在挂载根文件系统之前，以便从休眠状态恢复。

从 [systemd](https://archlinux.org/packages/?name=systemd)包 v255 和 [mkinitcpio](https://archlinux.org/packages/?name=mkinitcpio)包 v38 开始，当系统运行在 [UEFI](../../01_安装与包管理/启动过程/UEFI.md) 上时，[systemd-sleep(8)](https://man.archlinux.org/man/systemd-sleep.8) 将自动选择合适的交换空间进行休眠，并且所用交换空间的信息存储在 `HibernateLocation` EFI 变量中。在下次启动时，[systemd-hibernate-resume(8)](https://man.archlinux.org/man/systemd-hibernate-resume.8) 从 EFI 变量读取位置，系统恢复。这意味着除非系统使用遗留 [BIOS](../../01_安装与包管理/启动过程/Arch%20的启动流程.md#BIOS) 或你想选择与自动选择的不同的交换空间，否则以下步骤不是必需的。

手动指定休眠位置

可以传递[内核参数](../内核/内核参数.md) `resume=*swap_device*`。其中 *swap_device* 遵循 [persistent block device naming](../../04_硬件与外设/硬件检测和故障排除/块设备持久化命名.md)。例如：

- `resume=UUID=4209c845-f495-4c43-8a03-5363dd433153`
- `resume="PARTLABEL=swap分区卷标"`
- `resume=/dev/archVolumeGroup/archLogicalVolume`—如果swap在[LVM](../存储虚拟化/LVM.md)逻辑卷上（UUID 和 标签应该同样可以工作）

内核参数只有在重新启动后才会生效。要想不重启而马上休眠，请从[lsblk](../系统管理/设备文件.md#lsblk)获取卷的主要和次要设备号，并 echo `*major*:*minor*`到`/sys/power/resume`。

例如，如果交换设备是`8:3`：
如果使用交换文件，请继续查看[#获取交换文件偏移量](#获取交换文件偏移量)。

获取交换文件偏移量

使用交换文件进行休眠时，文件系统所在的块设备应该在 `resume=` 中指定，并且还需要通过 `resume_offset=` [内核参数](../内核/内核参数.md)指定交换文件的物理偏移量。

在除 [Btrfs](../文件系统/Btrfs.md) 之外的文件系统上，可以通过运行 `filefrag -v *交换文件*` 来获取 `resume_offset=` 的值。输出为表格格式，所需的值位于第一行 `physical_offset` 列中。

例如：
Filesystem type is: ef53 File size of /swapfile is 4294967296 (1048576 blocks of 4096 bytes) ext: logical_offset: physical_offset: length: expected: flags: 0: 0.. 0:

38912.. 38912: 1: 1: 1.. 22527: 38913.. 61439: 22527: unwritten 2: 22528.. 53247: 899072.. 929791: 30720: 61440: unwritten ...

在本例中，`resume_offset=` 的值是首先出现的那个 `38912`。

或者，为了直接获取偏移量：
对于 [Btrfs](../文件系统/Btrfs.md)，请不要尝试使用 *filefrag* 工具，因为 *filefrag* 获取到的"物理"偏移量并不是真正在硬盘上的偏移量；这是一个虚拟的硬盘地址空间，用于支持多设备。请使用 [btrfs-inspect-internal(8)](https://man.archlinux.org/man/btrfs-inspect-internal.8) 命令作为代替，例如：
198122980

在本例中，设置偏移量的内核参数是 `resume_offset=198122980`。

为使该更改立刻生效（无需重启），使用 `echo` 将偏移量改动至 `/sys/power/resume_offset`。例如，如果偏移量是`38912`：
> **提示：**这个命令也许可以用于查找交换文件所在设备：`findmnt -no UUID -T *swap_file*`

> **注意：**对于一个堆叠块设备，比如一个加密容器（LUKS），RAID 或者 LVM，`resume` 参数必须指向交换文件所在的已解锁或映射的设备。

更改休眠镜像的压缩算法

从 Linux 6.9 开始，可以更改休眠镜像的压缩算法。默认的压缩算法基于编译时选项 `CONFIG_HIBERNATION_DEF_COMP` 选择，但可以在启动时和运行时覆盖。

不同的压缩算法具有不同的特性，休眠可能会受益于使用其中任何一种算法，尤其是当次要算法（LZ4）在默认算法（LZO）上提供更好的解压缩速度时，这反过来会减少休眠镜像的恢复时间。

你可以通过两种方式覆盖默认算法：

1) 传递 `hibernate.compressor` 作为[内核参数](../内核/内核参数.md)：
hibernate.compressor=lzo hibernate.compressor=lz4

2) 在运行时指定算法：
目前 `lzo` 和 `lz4` 是支持的算法，LZO 是默认算法。

使用 zram 维护休眠的交换文件

> **提示：**与以下涉及多个交换空间的设置不同，[可以使用 zswap](#禁用_zswap_写回以仅将交换空间用于休眠) 来建立类似的行为。

通过同时维护两个或更多的交换空间，可以解决使用 zram 仅内存交换时的休眠问题。[systemd](../../01_安装与包管理/初始化/Systemd.md) 在触发休眠前会始终忽略 zram 块设备 ，因此保持两个交换空间启用应该可以在无需进一步干预的情况下正常工作。

在[配置交换文件](#获取交换文件偏移量)后，请按照 [zram](../内核/Zram.md) 页面的说明进行操作。确保 zram 具有**更高的交换优先级**（例如 `pri=100`）。

> **注意：**
>
> - **不要**为休眠创建按需交换单元，因为这不被官方支持。请参阅 systemd 问题 [#16708](https://github.com/systemd/systemd/issues/16708) 和 [#30083](https://github.com/systemd/systemd/issues/30083)。
> - 内核单独负责回收页面的匿名内存并进行交换；不使用交换空间实际上可能导致内存使用不良。用户可以通过 control groups 中的 `memory.low` 来管理某些应用程序的内存回收优先级。总的来说，这比设置 [swappiness](../文件系统/Swap.md) 参数更有效。

> - 阅读 [内核文档中的交换管理](https://www.kernel.org/doc/gorman/html/understand/understand014.html) 和 [Chris Down 的文章 - 为交换辩护：常见误解](https://chrisdown.name/2018/01/02/in-defence-of-swap.html) 以获取更多详细信息。

休眠到精简配置的LVM卷中

在精简配置的[LVM](../存储虚拟化/LVM.md)卷中休眠是可能的，但必须确保该卷已完全分配。否则系统将无法从中恢复，参见 [FS#50703](https://bugs.archlinux.org/task/50703)。

可以通过简单地用零填充来完全分配LVM卷。例如：
想确认卷是否已完全被分配，使用：
LV VG Attr LSize Pool Origin Data% Meta% Move Log Cpy%Sync Convert swap vg0 Vwi-aot--- 10.00g pool 100

完全分配的卷将显示为具有100%的Data%

> **警告：**不要在用于休眠的精简配置的swap分区上使用[TRIM](../存储/固态硬盘.md#TRIM)，即：不要在`/etc/fstab`使用`discard`，也不要在*swapon*时加入`-d`或`--discard`参数。否则已被使用的空间会被重新去分配

禁用 zswap 写回以仅将交换空间用于休眠

在 Linux 6.8 中，[zswap](../内核/Zswap.md) 增加了[一个按 cgroup 禁用写回的选项](https://git.kernel.org/linus/501a06fe8e4c185bbda371b8cedbdf1b23a633d8)。通过在所有可能的单元类型中使用 systemd 单元设置 `MemoryZSwapWriteback`（参见 [systemd.resource-control(5) § Memory Accounting and Control](https://man.archlinux.org/man/systemd.resource-control.5#Memory_Accounting_and_Control)），可以有效地完全禁用 zswap 写回。这使得 zswap 可以像 zram 一样使用，同时还支持休眠。

为了避免手动创建十二个顶级按类型的 drop-in 文件（用于系统和用户的 `scope`、`service`、`slice`、`socket`、`mount`、`swap` 单元类型），请安装 [zswap-disable-writeback](https://aur.archlinux.org/packages/zswap-disable-writeback/)AUR。[启用 zswap](../内核/Zswap.md#开关_zswap) 并重新启动以使设置生效。

尝试执行内存密集型任务并确认 zswap 没有将任何内容写入磁盘：
0

睡眠钩子

自定义 systemd 单元

[systemd](../../01_安装与包管理/初始化/Systemd.md) 会分别为每个睡眠状态启动 `suspend.target`、`hibernate.target`、`hybrid-sleep.target` 或 `suspend-then-hibernate.target`。所有上述目标都会拉取 `sleep.target`。可以使用这些目标中的任何一个来在挂起/休眠之前或之后调用[自定义单元](../../01_安装与包管理/初始化/Systemd.md#Writing_unit_files)。应为用户操作和 root/系统操作分别创建文件。示例：
/etc/systemd/system/user-suspend@.service

[Unit] Description=User suspend actions Before=sleep.target

[Service] User=%I Type=forking Environment=DISPLAY=:0 ExecStartPre= -/usr/bin/pkill -u %u unison ; /usr/local/bin/music.sh stop ExecStart=/usr/bin/sflock ExecStartPost=/usr/bin/sleep 1

[Install] WantedBy=sleep.target

/etc/systemd/system/user-resume@.service

[Unit] Description=User resume actions After=suspend.target hibernate.target

[Service] User=%I Type=simple ExecStart=/usr/local/bin/ssh-connect.sh

[Install] WantedBy=suspend.target hibernate.target

启用 `user-suspend@*user*.service` 和/或 `user-resume@*user*.service` 以使更改生效。

> **注意：**由于屏幕锁定器可能会在屏幕"锁定"之前返回，因此在从挂起状态恢复时屏幕可能会闪烁。通过 `ExecStartPost=/usr/bin/sleep 1` 添加一个小延迟有助于防止这种情况。

对于 root/系统操作：
/etc/systemd/system/root-suspend.service

[Unit] Description=Local system suspend actions Before=sleep.target

[Service] Type=simple ExecStart=-/usr/bin/pkill sshfs

[Install] WantedBy=sleep.target

/etc/systemd/system/root-resume.service

[Unit] Description=Local system resume actions After=suspend.target hibernate.target

[Service] Type=simple ExecStart=/usr/bin/systemctl restart mnt-media.automount

[Install] WantedBy=suspend.target hibernate.target

组合睡眠/恢复单元

使用组合单元文件，单个钩子可以为不同的阶段（睡眠/恢复）和不同的目标完成所有工作。

示例和解释：
/etc/systemd/system/wicd-sleep.service

[Unit] Description=Wicd sleep hook Before=sleep.target StopWhenUnneeded=yes

[Service] Type=oneshot RemainAfterExit=yes
ExecStart=-/usr/share/wicd/daemon/suspend.py
ExecStop=-/usr/share/wicd/daemon/autoconnect.py

[Install] WantedBy=sleep.target

- `RemainAfterExit=yes`：启动后，服务被视为活动状态，直到显式停止。
- `StopWhenUnneeded=yes`：当活动时，如果没有其他活动服务需要它，服务将被停止。在这个特定示例中，它将在 `sleep.target` 停止后被停止。
- 因为 `sleep.target` 有 `StopWhenUnneeded=yes`，所以钩子保证会为不同的任务正确启动/停止。

位于 /usr/lib/systemd/system-sleep 的钩子

> **注意：**根据 [systemd-sleep(8)](https://man.archlinux.org/man/systemd-sleep.8)，此方法被视为一种 hack。*systemd-sleep* 会并行运行这些钩子，而不是一个接一个地运行。有关支持排序的更明确接口，请参见[#Custom systemd units](#Custom_systemd_units)。

systemd-sleep 会运行 `/usr/lib/systemd/system-sleep/` 中的所有可执行文件，并向每个文件传递两个参数：

1. 要么是 `pre`，要么是 `post`，取决于机器是进入睡眠还是唤醒。
2. `suspend`、`hibernate`、`hybrid-sleep` 或 `suspend-then-hibernate`，取决于正在调用的是哪个。

任何自定义脚本的输出都将由 *systemd-suspend.service*、*systemd-hibernate.service* 或 *systemd-hybrid-sleep.service* 记录。你可以在 *systemd*'s [journalctl](../日志/Systemd_Journal.md) 中查看其输出：
自定义睡眠脚本的示例：
/usr/lib/systemd/system-sleep/example.sh

case $1/$2 in pre/*) echo "Going to $2..." ;; post/*) echo "Waking up from $2..." ;; esac

不要忘记使你的脚本可执行。

一个名为 `SYSTEMD_SLEEP_ACTION` 的环境变量将被设置，并包含正在处理的睡眠操作。这对于 *suspend-then-hibernate* 特别有用，其中变量的值将是 `suspend`、`hibernate` 或 `suspend-after-failed-hibernate`（在休眠失败的情况下）。

提示与技巧

在受信任的位置自动解锁

在恢复时，如果系统连接到某些设备或受信任的 Wi-Fi 网络，你可以自动解锁系统。
/etc/local-scripts/resume-unlock.sh

function trusted() {

[[$(iwgetid -r) == your_home_ssid]] \ && return 0

return 1 # 不受信任 }

for ((i=0; i < 10; i++)); do if trusted; then loginctl unlock-sessions exit fi sleep 0.5 done

配置你的桌面环境，使其在恢复时锁定，然后创建一个[睡眠钩子](#睡眠钩子)，在恢复后运行上述脚本。你还需要安装 [wireless_tools](https://archlinux.org/packages/?name=wireless_tools)包 以读取连接的 Wi-Fi SSID。如果你还想测试连接的 USB 设备，请取消脚本中的 `lsusb -d ...` 行的注释，并填写你受信任设备的 ID。你可以通过运行 `lsusb` 获取设备的 ID。

完全禁用睡眠

当使用设备作为服务器时，可能不需要或甚至不希望挂起/休眠。可以通过 [systemd-sleep.conf(5)](https://man.archlinux.org/man/systemd-sleep.conf.5) 禁用每个睡眠状态：
/etc/systemd/sleep.conf.d/disable-sleep.conf

[Sleep] AllowSuspend=no AllowHibernation=no AllowHybridSleep=no AllowSuspendThenHibernate=no

英特尔快速启动技术（Intel Rapid Start Technology，IRST）

英特尔快速启动技术是一种基于固件的休眠方法，允许在预定义的时间间隔或根据电池状态从睡眠中休眠。这应该比常规休眠更快更可靠，因为它是由固件而不是在操作系统级别完成的。通常，它必须在固件中启用，固件还支持设置暂

停/电池事件触发休眠后的持续时间。然而，尽管固件中支持 IRST，但某些设备仅允许通过英特尔的 Windows 驱动程序进行配置。在这种情况下，下面描述的 intel-rst 内核模块应该能够在 Linux 下配置事件。

启用英特尔快速启动技术（IRST）后，从深度睡眠中恢复需要"[比从 S3 恢复长几秒，但比从休眠中恢复快得多](https://mjg59.dreamwidth.org/26022.html)"。

许多基于英特尔的系统都支持 IRST 固件，但需要在 SSD（而不是 HDD）上使用特殊分区。Windows 的 OEM 部署可能已经有一个预先存在的 IRST 分区，可以在 Arch Linux 安装过程中保留（而不是擦除和重新分区整个 SSD）。它应该显示为与系统 RAM 大小相等的未格式化分区。

> **警告：**英特尔快速启动分区未加密；"如果你使用基于软件的磁盘加密，英特尔建议禁用英特尔快速启动技术"。

但是，如果你打算擦除并重新分区整个驱动器（或者已经这样做了），那么如果你也计划使用该技术，则必须重新创建IRST分区。这可以通过创建一个与系统 RAM 大小相等的空分区，并将其分区类型设置为 [GUID](https://en.wikipedia.org/wiki/GUID_Partition_Table#Partition_type_GUIDs) `D3BFE2DE-3DAF-11DF-BA40-E3A556D89593`（对于 [GPT](../系统恢复/分区.md#GUID_分区表) 分区）或 [ID](https://en.wikipedia.org/wiki/Partition_type) `0x84`（对于 [MBR](../系统恢复/分区.md#主引导记录) 分区）来实现。你可能还需要在系统的固件设置中启用对IRST的支持。

> **提示：**可以在系统的固件设置中调整 IRST 启动前（挂起后）的持续时间。

IRST 休眠过程的持续时间（即将RAM的全部内容复制到特殊分区）取决于系统的 RAM 大小和 SSD 速度，因此可能需要20–60秒。一些系统可以用 LED 指示灯指示过程的完成，例如停止闪烁。

在 Linux 内核中配置 IRST 休眠事件需要 `CONFIG_INTEL_RST` 内置或作为模块。一旦通过 `modprobe intel_rst` 加载，它应该在 `/sys/bus/acpi/drivers/intel_rapid_start/*/` 下创建 `wakeup_events` 和 `wakeup_time` 文件，这些文件可用于进一步配置。该模块的文档较少，请参阅源代码 [drivers/platform/x86/intel/rst.c](https://github.com/torvalds/linux/blob/143a6252e1b8ab424b4b293512a97cca7295c182/drivers/platform/x86/intel/rst.c) 以获取更多详细信息。

另请参阅英特尔快速启动技术的[一般问答](https://www.intel.com/content/www/us/en/support/articles/000024078/technologies.html)和[用户手册](https://www.intel.com/content/www/us/en/support/articles/000024273/technologies/intel-vpro-platform.html?wapkw=user%20guide%20Intel%20rapid%20start%20)。

跟踪笔记本电脑电池在睡眠状态下的能量变化

要测量挂起状态下的功耗，请使用 [Batenergy](https://github.com/equaeghe/batenergy) 脚本将电池变化记录到系统日志中。这允许比较 S3 / S0x 状态下的功耗，或在 BIOS 和内核更新后检查回归和修复。该脚本需要安装 [bc](https://archlinux.org/packages/?name=bc)包 进行计算。

故障排除

ACPI_OS_NAME

你也许需要调整你的 **DSDT table** 来让它工作。参阅 [DSDT](DSDT.md) 词条。

睡眠/休眠无法进行，或无法稳定进行

> [](https://wiki.archlinuxcn.org/wzh/File:Tango-inaccurate.png)**本文或本章节的事实准确性存在争议。**
>
> **原因：** 本节混合了几个无关的原因。（在 [Talk:电源管理/挂起与休眠](https://wiki.archlinuxcn.org/wiki/Talk:%E7%94%B5%E6%BA%90%E7%AE%A1%E7%90%86/%E6%8C%82%E8%B5%B7%E4%B8%8E%E4%BC%91%E7%9C%A0) 中讨论）

有相当多的错误报告指出，他们的屏幕在给出可读的错误信息前就关闭了，有时屏幕关闭了，却再也无法唤醒。这些故障在笔记本电脑和台式机上都曾发生。这不是个官方解决方案，但切换到旧的内核，尤其是LTS内核，或许可以修复这个问题。

使用硬件看门狗定时器（默认禁用，参见`RuntimeWatchdogSec=` in [systemd-system.conf(5) § OPTIONS](https://man.archlinux.org/man/systemd-system.conf.5#OPTIONS)）时可能会出现一个问题。一个有问题的看门狗定时器可能会在系统完成创建休眠映像之前重置计算机。

有时屏幕会变黑，这是由于从initramfs内部初始化设备引起的。移除[Mkinitcpio#MODULES](../../01_安装与包管理/Initramfs/Mkinitcpio.md#MODULES)中可能包含的任何模块，移除`kms`钩子并重新生成initramfs可能会解决这个问题，特别是对于[早期KMS](../内核/内核级显示模式设置.md#Early_KMS_start)的图形驱动程序。在恢复之前初始化这些设备可能会导致系统无法从休眠中恢复的不一致情况。这不影响从RAM中恢复。此外，请查看博客文章[调试挂起问题的最佳实践](https://web.archive.org/web/20230502010825/https://01.org/blogs/rzhang/2015/best-practice-debug-linux-suspend/hibernate-issues)。

从[ATI](../../05_桌面与图形/图形/ATI.md)视频驱动程序切换到较新的[AMDGPU](../../05_桌面与图形/图形/AMDGPU.md)驱动程序也可能有助于使休眠和唤醒过程成功。

对于[NVIDIA](../../05_桌面与图形/图形/NVIDIA.md)用户，[禁用](../内核/内核模块.md)模块`nvidiafb`可能会有所帮助。

带有Intel CPU的笔记本电脑如果加载了`intel_lpss_pci`模块用于触摸板，可能会在恢复时遇到内核恐慌（大写锁定键闪烁）。需要将该模块添加到[initramfs](../../01_安装与包管理/启动过程/Arch%20的启动流程.md)中：

/etc/mkinitcpio.conf

MODULES=(... intel_lpss_pci ...)

然后[重新生成initramfs](../../01_安装与包管理/Initramfs/Mkinitcpio.md#手动生成)。

USB 设备错误

系统可能因为USB设备而无法挂起。你可能会看到以下错误：

PM: Some devices failed to suspend, or early wake event detected ... xhci_hcd 0000:02:00.0: PM: failed to suspend async: error -16

*lspci*可能会给你更多关于故障设备的信息：

$ lspci -s 02:00.0

02:00.0 USB controller: Advanced Micro Devices, Inc. [AMD] 500 Series Chipset USB 3.1 XHCI Controller

尝试断开该端口上的设备。

网络唤醒（WoL, Wake-on-LAN）

如果[网络唤醒](Wake-on-LAN.md)被启用，网卡可能会消耗电力，即使电脑处于休眠状态。

挂起后被立即唤醒

参见[电源管理/唤醒触发器#挂起后被立即唤醒](电源管理_唤醒触发器.md#挂起后被立即唤醒)。

系统在休眠后没有断电

当你休眠系统时，系统应该会断电（在将状态保存到磁盘后）。在某些固件上，S4 睡眠状态可能无法可靠工作。例如，系统可能会重启或保持开启但无响应，而不是断电。如果发生这种情况，可以在 [sleep.conf.d(5)](https://man.archlinux.org/man/sleep.conf.d.5) 中将 `HibernateMode` 设置为 `shutdown`：

/etc/systemd/sleep.conf.d/hibernatemode.conf

[Sleep] HibernateMode=shutdown

使用上述配置，如果其他设置都正确，在调用 `systemctl hibernate` 时，机器将会关闭，并在关闭过程中将状态保存到磁盘。

休眠后启动时未找到操作系统（或启动错误的操作系统）

当引导磁盘是外部磁盘时，可能会发生这种情况，这似乎是由 BIOS/固件限制引起的。BIOS/固件尝试从内部磁盘启动，而休眠是从外部（或其他）磁盘上的操作系统完成的。

类似于[#系统在休眠后没有断电](#系统在休眠后没有断电)，设置 `HibernateMode=shutdown` 以永久解决问题。如果你已经将自己锁定在外，你可以尝试重新启动系统4次（每次等待错误出现），这在某些BIOS上强制执行正常的引导过程。

Swap file in /home

如果交换文件位于 `/home/` 目录下，*systemd-logind* 将无法访问它，从而导致 `Call to Hibernate failed: No such file or directory` 警告信息，并且在执行 `systemctl hibernate` 时需要身份验证。应避免这种设置，因为它被认为是[上游不支持的](https://github.com/systemd/systemd/issues/31100)。有关两种解决方法，请参阅 [systemd issue 15354](https://github.com/systemd/systemd/issues/15354#issuecomment-611077881)。

PC 无法从 A520I 和 B550I 主板的睡眠中唤醒

在某些使用 A520i 和 B550i 芯片组的主板上，系统无法完全进入睡眠状态或从睡眠状态中恢复。症状包括系统进入睡眠状态，显示器关闭，但主板上的内部 LED 或电源 LED 仍然亮着。随后，系统无法从该状态恢复，需要强制关机。如果你在使用 AMD 时遇到类似问题，请首先确保你的系统已完全更新，并检查是否安装了 AMD [microcode](../../04_硬件与外设/CPU/微码.md) 软件包。

验证以 `GPP0` 开头的行是否处于启用状态：

$ cat /proc/acpi/wakeup

Device S-state Status Sysfs node GP12 S4 *enabled pci:0000:00:07.1 GP13 S4* enabled pci:0000:00:08.1 XHC0 S4 *enabled pci:0000:0b:00.3 GP30 S4* disabled GP31 S4 *disabled PS2K S3* disabled GPP0 S4 *enabled pci:0000:00:01.1 GPP8 S4* enabled pci:0000:00:03.1 PTXH S4 *enabled pci:0000:05:00.0 PT20 S4* disabled PT24 S4 *disabled PT26 S4* disabled PT27 S4 *disabled PT28 S4* enabled pci:0000:06:08.0 PT29 S4 *enabled pci:0000:06:09.0

如果该行已启用，你可以运行以下命令禁用它：
现在通过运行 `systemctl suspend` 进行测试，让系统进入睡眠状态。然后尝试在几秒钟后唤醒系统。如果有效，你可以使此解决方法永久生效。创建一个 systemd [单元文件](../../01_安装与包管理/初始化/Systemd.md#Writing_unit_files)：

/etc/systemd/system/toggle-gpp0-to-fix-wakeup.service

[Unit] Description="禁用 GPP0 以修复睡眠问题"

[Service] ExecStart=/bin/sh -c "/bin/echo GPP0 > /proc/acpi/wakeup"

[Install] WantedBy=multi-user.target

执行 daemon-reload 并 start/enable 新创建的单元。

或者，你可以创建一个 [udev](../../04_硬件与外设/硬件检测和故障排除/Udev.md) 规则。假设 `GPP0` 的 sysfs 节点为 `pci:0000:00:01.1`，如示例所示，运行 `udevadm info -a -p /sys/bus/pci/devices/0000\:00\:01.1` 以获取相关信息，并创建如下 udev 规则：
/etc/udev/rules.d/10-gpp0-acpi-fix.rules

KERNEL=="0000:00:01.1", SUBSYSTEM=="pci", DRIVERS=="pcieport", ATTR{vendor}=="0x1022", ATTR{device}=="0x1483", ATTR{power/wakeup}="disabled"

默认情况下，udev 守护程序已经在监视系统中的更改。如果需要，你可以[手动重新加载规则](../../04_硬件与外设/硬件检测和故障排除/Udev.md#Loading_new_rules)。

Suspend from corresponding laptop Fn key not working

如果无论 \`logind.conf\` 中的设置如何，睡眠按钮都不起作用（按下它甚至不会在 \`syslog\` 中产生消息），那么 \`logind\` 可能没有监视键盘设备。 请执行以下操作：
你可能会看到类似以下的内容：
May 25 21:28:19 vmarch.lan systemd-logind: Watching system buttons on /dev/input/event2 (Power Button) May 25 21:28:19 vmarch.lan systemd-logind: Watching system buttons on /dev/input/event3 (Sleep Button) May 25 21:28:19 vmarch.lan systemd-logind: Watching system buttons on /dev/input/event4 (Video Bus)

注意没有键盘设备。按如下方式列出键盘设备：
$ stat -c%N /dev/input/by-id/*-kbd

... /dev/input/by-id/usb-SIGMACHIP_USB_Keyboard-event-kbd -> ../event6 ...

现在获取父键盘设备的 `ATTRS{name}` 。例如，在上面的列表中，此键盘设备的设备输入事件为 `**event6**`，可以用于搜索其相应的属性名称：
... KERNEL=="event6" ... ATTRS{name}=="SIGMACHIP USB Keyboard"

现在编写一个自定义的 \`udev\` 规则来添加 "power-switch" 标签：
/etc/udev/rules.d/70-power-switch-my.rules

ACTION=="remove", GOTO="power_switch_my_end" SUBSYSTEM=="input", KERNEL=="event*", ATTRS{name}=="SIGMACHIP USB Keyboard", TAG+="power-switch" LABEL="power_switch_my_end"

在[重新加载 udev 规则](../../04_硬件与外设/硬件检测和故障排除/Udev.md#Loading_new_rules)并重启 `systemd-logind.service` 后，你应该会在 \`logind\` 的日志中看到 `Watching system buttons on /dev/input/event6`。

System freezes for 60 seconds and then wakes back up or hangs after waking up

自 systemd v256 起，systemd 在休眠前会冻结 `user.slice`。由于内核错误，特别是当使用 KVM 时，此过程可能会失败。

日志中的消息会在休眠前包含 `Failed to freeze unit 'user.slice'`。当发生此类问题时，尝试登录（启动另一个会话）将失败，并显示：
pam_systemd(process:session): Failed to create session: Job 9876 for unit 'session-6.scope' failed with 'frozen'

要暂时恢复旧行为，请edit `systemd-suspend.service`、`systemd-hibernate.service`、`systemd-hybrid-sleep.service` 和 `systemd-suspend-then-hibernate.service`，并添加以下 drop-in：

[Service] Environment=SYSTEMD_SLEEP_FREEZE_USER_SESSIONS=false

系统恢复

系统起不来、root 密码丢失、分区表被改坏、文件误删、换新硬件，这类故障来得突然，有一套可复现的流程，才不至于直接重装。

建议先读《常规故障排除》，理清从启动失败到软件包异常的排查路径，再读《重置 root 密码》与《Chroot》，最后用《文件恢复》兜底，这四节必读。《分区》《Fdisk》《Parted》《GPT fdisk》属手册型内容，按需现查即可；备份、快照与迁移几节是事前预防，可先略过。最易踩的坑是改分区前不备份分区表——这些工具都能备份与恢复分区表，务必先用起来。

5.1　System-tar-and-restore

难度 入门 | **篇幅** 1.6 KB | **所属** 系统恢复

本节将讲到

- 安装
- 使用

System-tar-and-restore 包含两个 bash 脚本，即主程序 **star.sh** 和 gui 包装器 **star-gui.sh**。三种模式可用：

- **Backup**：使用此模式，您可以为系统创建 tar 备份档案。
- **Restore/Transfer**：还原模式使用以上创建的存档将其提取到所需分区中。传输模式使用 rsync包 在所需分区中传输系统。然后，在两种情况下，脚本都会生成目标系统的 fstab，为每个可用内核重建 initramfs，生成语言环境，最后安装并配置选定的引导加载程序。

如果您打算使用 gui，请安装 gtkdialogAUR。

安装

安装 system-tar-and-restoreAUR。

使用

请参阅自述文件。

5.2　重置 root 密码

难度 入门 | **篇幅** 3.2 KB | **所属** 系统恢复

本节将讲到

- 使用 sudo

- 使用 debug shell
- 用 Bash 作为 Init
- 使用 LiveCD
- 参阅

本指南介绍了在遗忘 root 密码后，重置密码的几种方法。

警告：攻击者可以使用以下部分方法攻入系统。除非使用静态数据加密，否则无论系统和密码有多安全可靠，只要攻击者可以物理接触设备，就能启动到其它系统并导出数据。

使用 sudo

如果你安装了 sudo，并对 `wheel` 用户组或其它你记得密码的用户授予了权限，那可以使用 `sudo passwd root` 命令修改 root 密码。

使用 debug shell

1. 将 `systemd.debug_shell` 附加到内核参数。
2. 系统将会正常启动，但同时会启动 `debug-shell.service` 服务，并在 `tty9` 开启一个 root shell（`/bin/sh`），按下 `Ctrl+Alt+F9` 可以进行访问。
3. 使用 passwd 命令为 root 用户创建新密码。
4. 完成后，停止 `debug-shell.service`。

用 Bash 作为 Init

1. 将 `init=/bin/bash` 内核参数附加到启动加载器的启动项。
2. 根文件系统应该是只读挂载，需要以可读写模式重新挂载：`mount -n -o remount,rw /`
3. 用 passwd 创建新的管理员密码。
4. 通过 `reboot -f` 重启，不要再次忘记你的密码。

注意：使用此法时有的键盘不能被初始系统正确加载，可能无法在 bash 提示符后进行输入。如果出现这种情况，你只能使用其他方法。

使用 LiveCD

通过 LiveCD 可以使用好几种方法：chroot 并使用 `passwd` 命令，或者编辑密码文件直接删掉密码项。除使用 chroot 时架构必须与已安装的系统匹配外，可以使用任何 Linux 的 LiveCD。这里仅介绍 chroot 方式，因为这个方法更不容易出错。

Change Root

1. 启动 LiveCD, 挂载主系统的根文件系统。
2. 然后通过下列命令重置密码（不会出现要求输入旧密码的提示）：

```
passwd --root 根文件系统挂载点 用户名
```

1. 卸载根文件系统。
2. 重启并输入你的新密码。

参阅

- 如何用 Knoppix 重置忘记的密码
- 启动早期 Debug Shell

5.3 使用 LVM 为根文件系统创建快照

难度 入门 | **篇幅** 5.2 KB | **所属** 系统恢复

本节将讲到

- 前提条件
- 安装
- 用法

本文描述如何在系统启动时为root文件系统做LVM快照，这些快照可以用来在最短时间内进行全系统备份，或者是测试系统的更新以便于按需回滚。

前提条件

你需要一个root文件系统为LVM分区并且利用systemd进行引导的系统，确认LVM快照相关的前提条件已经正确安装。

安装

在系统启动过程中，利用一个新的systemd service创建一个干净的root卷的快照，创建`/etc/systemd/system/mk-lvm-snapshots.service`包含以下内容:

```
[Unit]
Description=make LVM snapshots
Requires=local-fs-pre.target
DefaultDependencies=no
Conflicts=shutdown.target
After=local-fs-pre.target
Before=local-fs.target

[Install]
WantedBy=make-snapshots.target

[Service]
Type=oneshot
ExecStart=/usr/sbin/lvcreate -L10G -n snap-root -s lvmvolume/root
```

将上述`lvcreate`命令中的root卷的卷组名和卷名改成你系统中的相关名字，如有必要修改上述快照的大小。如果还有其它的文件系统需要进行快照，则将上述`ExecStart`属性扩展为多条命令，用分号隔开(注意在分号前后均要留有一个空格，详见systemd service manual)。

注意：你可以在运行的系统上测试一下`# lvcreate`命令直到得到的结果如你所需。可以用`# lvremove`删除此快照。在运行的系统上做的快照不如在单用户模式下或是在系统启动时做的快照一致性更好。

创建一个新的systemd target `/etc/systemd/system/make-snapshots.target`:

```
[Unit]
Description=Make Snapshots
Requires=multi-user.target
```

如果`multi-user.target`不是你的缺省target，则进行相应的修改。

将此新的服务生效：`# systemctl enable mk-lvm-snapshots.service`。

如果系统启动一个新的target，LVM快照会在挂载完本地文件系统后进行创建，为此，创建一个GRUB menu entry来启动这个新的target，基于正常启动的`grub.cfg`创建`/boot/grub/custom.cfg`，并在其中传递给内核启动`make-snapshots.target`的参数。

```
### make snapshots ###
menuentry 'Arch GNU/Linux, make snapshots' --class arch --class gnu-linux --class gnu
--class os {
...
        echo    'Loading Linux core repo kernel ...'
        linux   /boot/vmlinuz-linux root=/dev/mapper/lvmvolume-root ro
systemd.unit=make-snapshots.target
        echo    'Loading initial ramdisk ...'
        initrd  /boot/initramfs-linux.img
}
```

如果{{ic|grub.cfg}发生了变更，记着相应修正`custom.cfg`。

系统通过此grub entry启动后`# lvs`应该可以看到新创建的快照。

提示：利用`# journalctl -u mk-lvm-snapshots.service`命令可以看到新的服务启动过程的相关输出。

用法

备份

如要进行全系统的备份，首先以上述创建快照的target重启系统，挂载快照卷（and further volumes, if required），最好以只读的方式 (`-o`) 挂载，便可以进行全系统备份，例如可以采用Full system backup with tar文章中的办法进行备份。

在备份的过程中并不影响现有系统的使用，因为所有正常卷的修改都反应在了快照里了。备份完后不要忘记删除这个快照，copy-on-write的快照模式会使正常root卷的所有修改用光你的快照卷空间。如果快照卷空间用光，LVM并不能自动增长快照空间，则LVM会拒绝进一步向正常卷中写入数据或者直接丢掉快照，都应该避免。

更新回滚

另外一个LVM快照的用法是进行系统更新的测试和回滚，即为已知正常使用的系统先做一个快照，再进行更新。

如果想要固化更新的内容，则可以通过`# lvremove`删除快照的方式；如果想要回滚到更新前的状态，可以通过`# lvconvert --merge`恢复快照，在下次系统启动时快照会被合并到正常卷中，创建快照之后所有对正常卷的修改均被撤消。

注意：快照合并会快照卷也将不复存在，如果想进一步测试则必须再创建一个快照卷。

5.4 Chroot

难度 进阶 | **篇幅** 8.1 KB | **所属** 系统恢复

本节将讲到

- 原因
- 必要条件
- 用法
- 在 chroot 中运行图形程序
- 不使用 root 权限

Chroot 是一种修改当前进程及其子进程的可见根目录的操作。修改后，进程将不能访问该环境目录树以外的任何文件和命令，这种修改后的环境叫作 *chroot jail*（直译为 chroot 监狱）。

原因

改变根目录通常是为了在无法启动或登录的系统上进行系统维护，例如：

- 重新安装 bootloader。
- 重建 initramfs 镜像。
- 升级或降级软件包。
- 重置忘记的密码。
- 在在干净的 chroot 环境中构建中构建软件包。

另见 Wikipedia:Chroot#Limitations。

必要条件

- root 权限
- 另一个 linux 环境，例如 liveCD、USB 闪存介质或者一个已经安装的另一个 linux 发行版。

- 匹配的架构，chroot 前后的环境架构要一致(例如，都是 i686 或 x86_64)。可以用以下命令查看当前环境的架构

```
uname -m
```

- 提前加载 chroot 环境需要的内核模块
- 如果需要 swap， chroot 前先启用 swap （`swapon /dev/sdxY`）
- 如果需要网络，chroot 之前先建立好网络连接。

用法

注意：

- 有些systemd 工具无法在 chroot 中运行，例如 *hostnamectl*、*localectl* 和 *timedatectl*，因为这些程序需要可用的 dbus 连接。
- 新的 root (`/`) 所在的文件系统必须是可用访问的状态(提前解密、挂载)。

有两种使用 chroot 的方式：

使用 arch-chroot

`arch-chroot` bash 脚本是软件包 arch-install-scripts包 的一部分，在运行 `/usr/bin/chroot` 前，这个脚本会挂载类似 `/proc` 的 API 文件系统，建立可用的 `/etc/resolv.conf`。

进入 chroot

```
# arch-chroot /location/of/new/root
```

例如在安装指南中，chroot 到 `/mnt`:

```
# arch-chroot /mnt
```

退出 chroot:

```
# exit
```

运行一个命令并退出

用下面命令在 chroot 中运行一个命令并退出：

```
# arch-chroot /location/of/new/root mycommand
```

例如要在 `/mnt/arch` 中运行 `mkinitcpio -p linux` 并退出:

```
# arch-chroot /mnt/arch mkinitcpio -p linux
```

使用 chroot

警告：使用 `--rbind` 选项时，将无法卸载某些 `dev/` 和 `sys/` 的子目录，用 `umount -l` 卸载将会破坏会话并需要重启，所以请尽可能使用 `-o bind`。

在下面的例子中，`*/location/of/new/root*` 代指新的根目录所在的文件夹。 首先，临时挂载 API 文件系统：

```
# cd /location/of/new/root
```

```
# mount -t proc /proc proc/
# mount -t sysfs /sys sys/
# mount --rbind /dev dev/
```

可选挂载:

```
# mount --rbind /run run/
```

如果正运行在 UEFI 系统，则还需要能够访问 EFI 变量，不然在安装 GRUB 时会得到类似这样的消息：`UEFI variables not supported on this machine`：

```
# mount --rbind /sys/firmware/efi/efivars sys/firmware/efi/efivars/
```

如果已经建立了一个网络连接并且想在 chroot 环境中继续使用，将 DNS 服务器配置复制到新环境：

```
# cp -L /etc/resolv.conf etc/resolv.conf
```

chroot 到新环境中并启用指定 shell

```
# chroot /mnt/arch /usr/bin/bash
```

注意：

- 如果遇到错误 `chroot: cannot run command '/bin/bash': Exec format error`，很可能是因为两个环境架构不匹配。
- 如果遇到错误 `chroot: '/usr/bin/bash': permission denied`,用执行权限重新挂载: `mount -o remount,exec */location/of/new/root*`。
- 如果以上方式无效，那么请确保新环境的基础组件完整（如果是 Arch 根目录，请尝试 `paccheck --root=*/location/of/new/root* --files --file-properties --md5sum glibc filesystem`，来自 pacutils包）。

（可选）加载 Bash 配置文件(`~/.bashrc` 和 `/etc/bash.bashrc`)，运行：

```
# source ~/.bashrc
# source /etc/profile
```

或创建一个独特的提示符来区别你的chroot环境：

```
# export PS1="(chroot) $PS1"
```

退出 chroot 环境：

```
# exit
```

然后卸载临时文件系统:

```
# cd /
# umount --recursive /location/of/new/root
```

如果出现 `/mnt`(或其它任何分区) is busy, 这可能意味着：

- chroot环境中残留了一个运行的程序或者还有分区没有被卸载，退出程序并用 `findmnt -R */location/of/new/root*` 查找然后卸载未卸载的分区。
- 如果你仍然不能卸载分区，使用`--force`选项：

```
    # umount -f /mnt
```

，或使用 `umount --lazy` 直接释放挂载。这是，请立即重启系统以避免不一致的状态导致冲突。

在 chroot 中运行图形程序

如果系统上运行了X，可以在 chroot 环境启动图形应用。

为了chroot环境能连接到你的X服务器，在X服务器中打开一个终端(例如，在用户当前登录的桌面中)，然后运行如下命令给任何人连接到用户X服务器的权限(另见 Xhost)：

```
$ xhost +local:
```

然后，从chroot环境中将应用指向你的X服务器，将chroot中的DISPLAY环境变量设定成和拥有X服务器的用户DISPLAY变量相匹配。例如运行：

```
$ echo $DISPLAY
```

作为拥有X服务器的用户查看DISPLAY的值。如果是":0"(例如是)，然后在chroot环境中运行

```
# export DISPLAY=:0
```

现在就可以从chroot命令行启动图形界面应用

不使用 root 权限

Chroot 需要 root 权限，有时用户并没有这个权限，下面工具可用实现类似的功能：

PRoot

PRoot 可用在没有 root 权限的情况下，用 `mount --bind` 设置可见根目录，这样可用为不同的 CPU 架构编译程序。这个程序的缺点是文件属于主机系统。可用用 `--root-id` 选项解决一部分问题。

Fakechroot

fakechroot包 是一个拦截 chroot 调用并伪造结果的程序。用 fakeroot包 可用为普通用户伪造一个 chroot 环境：

```
$ fakechroot fakeroot chroot ~/my-chroot bash
```

Unshare

Unshare 是 util-linux包 的一部分，可以用来创建新的内核命名空间。它使用常规的 chroot 命令运行，例如：

```
$ unshare --map-root-user chroot ~/namespace /bin/sh
```

疑难解答

arch-chroot: /location/of/new/root is not a mountpoint. This may have undesirable side effects.

执行 `arch-chroot */location/of/new/root*` 命令后，出现以下警告：

```
==> WARNING: /location/of/new/root is not a mountpoint. This may have undesirable side effects.
```

见 arch-chroot(8) 中的解释和使用绑定挂载让 chroot 目录变成挂载点的例子。

另见

- 基础 Chroot

5.5 迁移到新硬件

难度 进阶 | **篇幅** 10.3 KB | **所属** 系统恢复

本节将讲到

- 适配新硬件
- 自下而上
- 自上而下
- 另见

本文讨论将Arch Linux系统迁移到新硬件所需的步骤。 我们的目标是实现相同的Arch Linux安装过程，对于硬件无关的软件和配置。

警告：以下某些操作可能有危险：建议您在继续操作之前，先备份旧系统上的所有重要数据。

移植已安装的系统有两种不同的方法：

1. “自下而上”：在新硬件上安装全新的Arch Linux系统，然后恢复已安装的软件包和配置文件，如以点为开头的文件中所述。
2. “自上而下”：将旧硬盘克隆到新硬盘，或将旧硬盘放入新系统；在需要的地方修改配置文件。

“自上而下”的方法比“自下而上”的方法更准确地再现了原始系统。

适配新硬件

警告：对于这两种方法，我们必须考虑新旧硬件之间的差异，并相应地修改安装的驱动程序和配置。

在开始之前，先研究一下新硬件的各个方面，并列出不同之处。一般的区别有

机械硬盘与固态硬盘

参见文章Solid state drive。

CPU供应商

如果将CPU切换到其他供应商的CPU（例如从Intel更换为AMD），请更改微码配置。

GPU供应商

如果您将GPU更换为其他供应商（例如，从AMD更换为NVIDIA），请更换图形驱动程序。

UEFI与MBR引导代码引导

如果您更换到较新的支持UEFI的主板，则可能更可取或需要从“MBR引导代码”引导切换到UEFI引导。在这种情况下，需要一个新的EFI分区。

自下而上

在旧系统上

我们在这里定义了从旧系统迁移到新系统的最小配置，这是此方法与安装指南的不同之处。考虑要复制到新系统的`/etc`中的配置文件和`/home`中的“.”文件，以及用户数据文件。如果您无法从新系统访问旧系统，则备份要复制的所有文件。

列出已安装的软件包

```
$ pacman -Qqen > pkglist.txt
$ pacman -Qqem > pkglist_aur.txt
```

为您提供从软件仓库和AUR显式安装的软件包的列表。如果您创建了备份，请将其包含在备份中。

您还可以使用以下脚本查找pacman感知不到的二进制文件和库（例如，g. 通过Steam、Desura或使用自己的安装方法安装的软件）：

```
find / -regextype posix-extended -regex "/(sys|srv|proc)|.*/\.ccache/.*" -prune -o -
type f \
-exec bash -c 'file "{}" | grep -E "(32|64)-bit"' \; | \
awk -F: '{print $1}' | \
while read -r bin; \
do pacman -Qo "$bin" &>/dev/null || echo "$bin"; \
done
```

pacman 缓存

如果不更改体系结构（例如，从x86更改为x86_64），请考虑备份`/var/cache/pacman/pkg/`。

在新系统上

安装指南的前半部分

有关安装新系统的基本信息，请参阅Installation guide。按照安装指南的前半部分操作，但不包括pacstrap命令。

复制pacman缓存

将位于`/var/cache/pacman/pkg/`的pacman缓存从旧系统复制到新系统，或从备份复制到新系统。

安装指南的后半部分

继续执行中的安装指南（包括pacstrap命令），直到最后，但不要重新启动。不要跳过pacstrap命令，因为它除了安装软件包之外还会执行其他工作。

安装以前安装的软件

编辑pkglist.txt(和pkglist_aur.txt)并删除新系统上不需要的驱动程序。然后使用下述命令安装之前安装过的软件

```
# pacman -S --needed - < pkglist.txt
```

自上而下

自上而下方法有两个选项，您可以保留已安装系统的驱动器并修改其内容，也可以将系统复制到新驱动器。如果保留驱动器并对其进行修改，然后将其放回旧系统中，则修改可能会导致旧系统无法启动。

清理旧系统

在克隆系统之前清理系统也是值得的，如System maintenance#清理文件系统中所述。在继续之前，请确保旧系统在清理后仍按预期工作。

将系统复制到新驱动器

注意：如果您计划保留已安装系统的硬盘，则可以跳过此部分。

将系统复制到新驱动器有两种基本方法：硬盘克隆和文件复制。

硬盘克隆

需要使用Live系统，而不是旧的Arch Linux系统；例如，您可以使用Arch Linux U 盘安装介质。将复制旧系统的分区布局和文件系统。

文件复制

- 在新驱动器上创建新的分区和文件系统。您可以利用此机会选择与以前不同的分区布局和/或文件系统。
- 对于每种文件系统，使用rsync或其他可以保留文件权限、属性等的工具将文件从旧驱动器复制到新驱动器。更多详细信息，参见rsync#Full system backup, Rsync#作为备份工具。

传输方法的选择

如何在两个驱动器之间传输数据有许多不同的方法：

- 将源硬盘和目标硬盘连接到同一台计算机，无论是旧盘还是新盘。数据链路：旧硬盘->电脑->新硬盘。
- 使用外部硬盘或云备份等临时存储设备。数据链路：旧硬盘->旧电脑->存储->新电脑->新硬盘。有关概述，请参阅文章系统备份。
- 通过网络传输数据，例如使用rsync。数据链路：旧硬盘->旧电脑->网络->新电脑->新硬盘。

对于前两个选项，请考虑您可能需要适配器来连接硬盘（PATA->SATA转接、USB移动硬盘盒等），并选择足够快的连接。

后两个选项要求您在新计算机上使用Live系统，因为此时尚无法从新硬盘启动。

更新fstab

警告：在执行此步骤之前，请确保您不希望在旧系统中使用此驱动器，因为更改fstab文件可能会导致系统无法在旧配置中启动。

如果您使用的是Arch Linux安装映像，请将新的根分区挂载到`/mnt`，并挂载好其他需要的分区，就像您在正常安装中所做的那样(请参阅挂载分区)。

在`/mnt/etc/fstab`的末尾插入任意注释，如`#end of old fstab`。按照安装指南#Fstab中的说明生成一个新的fstab文件，并将其附加到当前的fstab文件中。一定要检查genfstab创建的fstab文件。在这种情况下，请在注释之前检查较旧的fstab条目，删除过期项和重复项；如果旧条目仍然有用，则保留它们。例如，可以保留网络驱动器的挂载条目。一般推荐使用持久化命名。

重新安装引导加载程序

出于以下原因，您可能需要重新安装和/或重新配置启动加载器：

- 不同的磁盘、分区布局或文件系统
- 将UEFI引导条目添加到新的主板NVRAM
- 从“MBR引导代码”引导迁移到UEFI引导
- 从USB迁移到SATA/NVMe
- 更新内核命令行（commandline）
- 如果使用不同的GPU，请更新帧缓冲区模式
- 更新微码和initramfs镜像

如果您使用的是Arch Linux live环境，则在重新安装引导加载程序之前，请更改根路径进入新系统：

```
# arch-chroot /mnt
```

有关如何安装/重新安装引导程序的说明，请参阅启动加载器。

重新生成内核映像

建议使用mkinitcpio重新生成initramfs映像，尽管最初可以使用备用initramfs映像。

重新配置音频

- alsamixer volume
- 保存设置

重新配置网络

如果旧系统和迁移后的系统在同一网络中共存，请使用hostnamectl设置新主机名。

另请考虑主机名更改后所需的配置更改：

- /etc/hosts
- 使用主机名的其他应用程序：synergy，nut（网络 ups 工具）
- `grep -Ri 'hostname' /etc`（以root身份运行）也会对需要更新的文件提供一些提示

将dhcpcd配合已命名的网络接口使用时，网络接口名称可能会有变化。

- 使用命令`$ dmesg | grep 'renamed from eth'`可帮助查找新接口名称
- 删除旧接口：disable `dhcpcd@enp*X*s0.service`
- 激活新接口：enable `dhcpcd@enp*X*s0.service`

另见

- Howto clone a linux system

-

5.6 Fdisk

难度 进阶 | **篇幅** 11.2 KB | **所属** 系统恢复

本节将讲到

- 安装
- 显示分区
- 备份和恢复分区表
- 创建分区表和分区
- 移动分区

util-linux fdisk 是基于命令行界面的分区表创建和编辑工具。一个硬盘需要分为一个或多个分区，这个信息被记录在分区表里面。

本文介绍 3 种有着不同交互界面的分区工具：

- fdisk(8)——常用的命令行分区工具（对话式交互）。
- sfdisk(8)——支持脚本的分区工具，可用于自动配置和备份、恢复分区。
- cfdisk(8)——操作简单方便的 TUI 分区工具。

安装

要使用 *fdisk* 及相关工具，请使用 util-linux包 软件包，这个软件包是 base包 元软件包 的依赖。

显示分区

要列出块设备上的分区表和分区，可以运行以下命令，其中设备的名称类似于 `/dev/sda`、`/dev/nvme0n1`、`/dev/mmcblk0` 等：

```
# fdisk -l /dev/sda
```

注意：如果未指定设备，*fdisk* 将列出 `/proc/partitions` 中的所有分区。

备份和恢复分区表

在对硬盘做出更改之前，您可能需要备份硬盘的分区表和分区方案。您还可以使用备份将同一分区布局复制到多个硬盘上。

对于GPT和MBR，您可以使用“sfdisk”将设备的分区布局保存到具有`-d`/`--dump` 选项的文件中. 对设备 `/dev/sda`运行以下命令:

```
# sfdisk -d /dev/sda > sda.dump
```

对于大小为 1 GiB 的单个 ext4 分区，该文件应如下所示：

```
sda.dump
label: gpt
label-id: AAAAAAAA-BBBB-CCCC-DDDD-EEEEEEEEEEEE
device: /dev/sda
unit: sectors
first-lba: 34
last-lba: 1048576
sector-size: 512

/dev/sda1 : start=2048, size=1048576, type=0FC63DAF-8483-4772-8E79-3D69D8477DE4,
uuid=BBF1CD36-9262-463E-A4FB-81E32C12BDE7
```

要稍后恢复此布局，可以运行：

```
# sfdisk /dev/sda < sda.dump
```

创建分区表和分区

硬盘分区的第一步是创建分区表。然后，根据所需的分区方案创建实际分区。参见分区表一文来帮助你选择使用 MBR 或是 GPT。

在开始之前，您可能希望备份您当前的分区表和方案。

fdisk 会自动在 2048 个 512 字节扇区（1 MiB）块大小基础上自动进行分区对齐，这应该与所有使用先进格式化的 HDD 和绝大多数 SSD 兼容。这意味着默认设置会自动使用正确的对齐方式。

To use *fdisk*, run the program with the name of the block device you want to change/edit. This example uses `/dev/sda`:

要使用 *fdisk*，需带上你想要更改/编辑的块设备的名字来运行程序。在本例中我们使用 `/dev/sda`：

```
# fdisk /dev/sda
```

这将启动 *fdisk* 程序，您可以在其中键入命令。

创建新的分区表

警告：如果在存有数据的硬盘上创建新分区表，它将擦除硬盘上的所有数据。请务必确保这是你想要做的

提示：

- 分区前，请检查 NVMe 硬盘和高级格式硬盘是否使用了最佳逻辑扇区大小。
- 在固态硬盘（SSD）上分区前应考虑清空 SSD 存储单元。

要创建一个新的分区表并清除当前所有的分区信息，请在提示符下输入 `g` 以新建一个 GUID 分区表（GPT）或输入 `o` 新建一个 MBR 分区表。若已创建所需的分区表，请跳过此步骤。

创建分区

使用 `n` 命令创建分区。需要输入分区编号，起始扇区和结束扇区。对于 MBR 类型分区，还可能需要输入分区类型。

注意：分区大小和位置的选择请参考 分区#分区方案。

分区类型

使用 MBR 时，*fdisk* 会询问 MBR 分区的类型。输入 `p` 创建主分区，输入 `e` 创建扩展分区，最多可以创建四个主分区。

fdisk 默认不询问分区类型 ID，而是直接使用 'Linux filesystem'，稍后可修改该 ID。

分区编号

分区编号是分区在硬盘中的编号，例如硬盘 `/dev/sda` 的 `1` 号分区为 `/dev/sda1`，对于 `/dev/nvme0n1` 是 `/dev/nvme0n1p1`，而 `/dev/mmcblk0` 则是 `/dev/mmcblk0p1`。有关命名方案的详细信息请参阅设备分区名。分区的编号可能不与硬盘上的分区顺序一致，请参考分区排序。

建议使用 *fdisk* 推荐的默认编号。

起始和结束扇区

起始扇区必须使用绝对扇区号。结束扇区可使用绝对扇区号或以这些单位指定：`K`、`M`、`G`、`T` 或 `P`。

结束扇区的位置可以是：

- 从硬盘起始位置开始的绝对位置。例如，`40M` 作为起始扇区，指定了距硬盘开头 40 MiB 的位置。
- 在 `**+***size*` 或 `**-***size*` 后接大小的相对位置。例如，使用 `+2G` 以指定位于起始扇区之后 2 GiB 的位置，或使用 `-200M` 以指定位于最后一个可用扇区之前 200 MiB 的位置。

未输入扇区位置的指定值而直接按 `Enter` 键将使用默认值，对于起始扇区是最大可用块的开始位置，对于结束扇区则是最大可用块的结束位置。

注意：

- 分区时，使用起始分区的默认值总是个不错的选择。此外，确保指定分区大小时使用 `+*size{M,G,T,P}*`，不要使用小于 1 MiB 的大小。此类分区始终根据设备属性对齐。
- EFI 系统分区需要 `EFI System` 类型的分区。
- 将 GRUB 安装到基于 BIOS 系统的 GPT 分区的设备上时，GRUB 需要一个 `BIOS boot` 类型的 BIOS 启动分区_特殊操作)。

提示：在使用 MBR 分区表的硬盘上，需在硬盘末尾保留至少 33 个 512 字节扇区大小（16.5 KiB）的未分区空间以便在 MBR 与 GPT 分区表之间转换。

重复此过程，直到获得所需的分区。

修改分区类型

每个分区都有类型，MBR 使用分区 ID 进行定义; GPT 使用分区类型 GUID 进行定义。

按下 `t` 修改分区类型，可以使用以下别名指定常用分区类型：

- `uefi`： ESP
- `xbootldr`： XBOOTLDR 分区
- `home`：home 分区
- `swap`： swap 交换分区
- `linux`：用于其它分区

提示：

- 按下 `L` 可以显示 fdisk 的内部码表。
- 在使用 GPT 时，建议遵循 可侦测分区标准，以使用 GPT 自动挂载功能。

将 MBR 分区设为可启动

输入 `a` 可以将一个 MBR 分区设置为可启动（即活动）分区。

检查并写入分区表更改

- 使用 `p` 显示更改内容。
- 使用 `q` 放弃更改。
- 使用 `w` 将更改写入硬盘并退出程序。

移动分区

警告：因为移动分区时需要重写全部的分区数据，所以要移动的分区必须先下线。移动分区的动作很慢且存在风险，强烈建议操作前进行备份。sfdisk(8) § OPTIONS 手册中有说明："这个操作存在风险而且是非原子操作。"

要移动分区，先要准备足够的空余硬盘空间。如果需要，可以缩小分区及文件系统，请参考 Parted#缩小分区。要移动分区：

```
# echo '+sectors,' | sfdisk --move-data device -N number
```

其中 `*sectors*` 是要偏移的扇区数，`*+*` 表示增加，`*device*` 是分区所在设备，`*number*` 是分区编号。如果在硬盘的开始或中间增加了分区，想重新编号，可以参考 #分区排序或 *fdisk* 的"额外功能"模式。

提示和技巧

分区排序

这适用于在两个分区之间的空间中创建或删除了分区的情况。本例中使用了 `/dev/sda`。

```
# sfdisk -r /dev/sda
```

在调整分区顺序后，如果没有使用块设备持久化命名，则需要调整 `/etc/fstab`、`/etc/crypttab` 等使用分区编号的配置文件。

注意：内核必须读取新分区表才能让分区（例如 `/dev/sda1`）处于可访问状态，可以重启系统或让内核重新读取分区表（即执行 `partprobe /dev/sda`）。

cfdisk

cfdisk(8) 提供了基于 curses) 的 TUI，相比 fdisk 操作更简单方便，但也不支持 fdisk 在 extra/expert 下的大多数特性和功能。 cfdisk 的使用方式如下：

```
# cfdisk /dev/sda
```

- 按下 `?` 显示帮助页面。
- cfdisk 启动后，界面顶部会显示目前的分区情况。有着 >> 游标和反色背景的分区是被选中的分区。用户可以使用上下方向键来选择不同的分区。

- 界面底部显示已被选中的分区的详细信息以及可用的命令列表。命令的帮助说明显示在命令的下方。使用左右方向键选择某个命令，按下 `Enter` 键执行。
- 类似 fdisk，所有的操作都会按顺序列队并等待用户运行 `Write` 命令后才会真正执行并写入硬盘。用户可以随时丢弃任何修改并安全地退出程序。
- 如果磁盘上还没有分区，cfdisk 会询问是否需要创建一个分区。如需要修改已有的分区表格式，则必须擦除硬盘内容或改用 fdisk 重建分区表。

另见

- Managing partitions in Linux with fdisk

5.7 文件恢复

难度 进阶 | **篇幅** 14.4 KB | **所属** 系统恢复

本节将讲到

- 特别提示
- 工具清单
- TestDisk 和 PhotoRec
- e2fsck
- 操作原始（raw）硬盘镜像

翻译状态：

- 本文（或部分内容）译自 File recovery，最近一次同步于 2025-04-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

这篇文章列举了一系列在 Linux 下恢复数据的方法。

特别提示

三思而后行

本文主要用于教学目的。如果你意外地删除或损坏了**有价值且不可替代的**数据，并且毫无数据恢复的经验的话，请立刻关闭你的电脑（只要长按电源键或直接拔插头即可，不要使用系统的关机功能），然后寻求专业人员的帮助。

警告：如果在没有完全理解下文内容的情况下就进行任何操作，很有可能会导致情况恶化。

故障的磁盘

在数据恢复的领域里，操作一个磁盘的镜像会比直接操作物理磁盘会更好些。总体上来说，一个坏掉的硬盘的情况会随着时间恶化。我们的目标应当是首先尽快地抢救出尽可能多的数据，并且抛弃故障的磁盘。ddrescue包 和 dd_rescue包 工具与 `dd` 不

同，它们会反复尝试从错误中恢复，并且从硬盘往返读取以抢救出数据。它们会保留日志文件，所以恢复过程中的暂停并不会损失进度。

参阅硬盘克隆。

由类如 ddrescue 这样的工具创建的镜像文件可以像物理设备一样被挂载，并且安全地对其进行操作。**请务必**做好原始镜像的备份以便在恢复工作黄掉的时候重新做人。

**本文或本章节的事实准确性存在争议。**

原因： 虽然这是数据恢复公司博文中提出的内容，但过去 10 年中已有人对降温技巧提出反对。该段落自 2009 年起就基本未作修改，可能不适用于现代硬盘。（在 Talk:文件恢复 中讨论）

改善故障硬盘读取的一个久经考验的方法是保持硬盘清凉。在冰箱里放一会儿是个好主意，但不要让硬盘升温太快，因为这样会产生冷凝水。如果能在进行恢复工作的时候把硬盘放在冰箱里，并用线材连接电脑就再好不过了.

不要尝试对故障硬盘进行文件系统检查，这很可能会使情况变得**更糟糕**。请务必把它挂载为只读。

备份闪存介质或者小分区

在出问题的文件系统不大或硬盘空间足够的情况下，优先选择在镜像上进行操作，而不是在活跃分区上进行（与是否挂载无关）。类似 U 盘，相机，便携播放器，手机等闪存介质的设备通常镜像都足够小。

请务必阅读下列工具的 man 手册，以确保这些工具能够处理映像文件。

可以使用如下命令通过 `dd` 创建镜像：

```
# dd if=/dev/target_partition of=/home/user/partition.image
```

恢复数码相机

为使下一节提到的工具能正常在闪存设备上工作，需要将设备挂载为块设备（即出现在 /dev 下）。在图片传输协议（Picture Transfer，PTP）模式下工作的数码相机不会以该形式出现，而是由 libgphoto 和/或 libptp 透明处理。在这种情况下，“透明”意味着 PTP 设备不会被获取为块设备。不是所有相机都支持作为 PTP 模式替代的 USB 大容量存储设备（USB Mass Storage，UMS）模式。有些相机可以在菜单中切换这两种模式，具体信息请参考相机的用户手册。如果你的相机不支持 UMS 模式，就无法被作为块设备读取，唯一解决方法是将介质从相机中取出，然后使用读卡器连到电脑。

工具清单

See also Wikipedia:List of data recovery software#File Recovery

- **ddrutility** — Compliment to GNU ddrescue包. Find what files are related to the bad sectors and some special tools for NTFS. No longer actively supported.

: https://sourceforge.net/projects/ddrutility/ || ddrutilityAUR

- **dvdisaster** — Additional error protection for CD/DVD media.

: https://sourceforge.net/projects/dvdisaster/ || dvdisasterAUR

- **Foremost** — Console program to recover files based on their headers, footers, and internal data structures. This process is commonly referred to as data carving. The headers and footers can be specified by a configuration file or command line switches can be used to specify built-in file types.

: https://foremost.sourceforge.net/ || foremost包

- **PhotoRec** — File data recovery software designed to recover lost files including video, documents and archives from hard disks, CD-ROMs, and lost pictures (thus the Photo Recovery name) from digital camera memory.

: https://www.cgsecurity.org/ || testdisk包

- **R-Linux** — A free file recovery utility for the Ext2/Ext3/Ext4 file systems.

: https://www.r-studio.com/free-linux-recovery/ || r-linuxAUR

- **Scalpel** — File carving and indexing application originally based on Foremost, although significantly more efficient. It allows an examiner to specify a number of headers and footers to recover filetypes from a piece of media.

: https://github.com/sleuthkit/scalpel || scalpel-gitAUR

- **TestDisk** — Data recovery software primarily designed to help recover lost partitions and/or make non-booting disks bootable again when these symptoms are caused by faulty software: certain types of viruses or human error (such as accidentally deleting a Partition Table).

: https://www.cgsecurity.org/ || testdisk包

- **xfs_undelete** — Traverses the inode B+trees of each allocation group and tries to recover all files on an XFS filesystem marked as deleted.

: https://github.com/ianka/xfs_undelete || xfs_undelete-gitAUR

TestDisk 和 PhotoRec

TestDisk 和 Photorec 都是在 GNU 通用公共许可证（GPL）下发布的开源数据恢复工具。

TestDisk 设计用于在出现软件错误，特定类别的病毒或人为原因导致分区表损坏等情况时，帮助恢复丢失的分区并/或使得硬盘可再起启动。TestDisk 可以检测包括 NTFS, FAT12, FAT16, FAT32, exFAT, ext2, ext3, ext4, btrfs, BeFS, CramFS, HFS, JFS, Linux Raid, Linux Swap, LVM, LVM2, NSS, ReiserFS, UFS, XFS 在内的多种文件系统。它还可以从 FAT，NTFS，exFAT 和 ext2 文件系统中恢复被删除的文件。

TestDisk 可以修复分区表，恢复被删除的分区，从备份恢复 FAT32 启动扇区，重建 FAT12/FAT16/FAT32 启动扇区，修复 FAT 表，重建 NTFS 启动扇区等等。

PhotoRec 是一个文件恢复软件，设计用于从硬盘和 CD-ROM 找回图片（也是名称来源：**Photo**graph**Rec**overy），视频，文档，压缩包等文件。PhotoRec 会无视文件系统并直接读取底层数据，使得其可以在被重新格式化或出现严重损坏的文件系统和/或分区表上运行。

安装

安装 testdisk包，其提供了 TestDisk 和 PhotoRec。

用法

在执行类似 ddrescue包 的工具创建 image.img 后，使用 `photorec image.img` 会打开一个命令行交互界面，以供选择要搜寻的文件类型和被恢复文件的存放位置。更详细的内容请参考 wiki。

通过 photorec 恢复的文件

photorec 会将恢复的文件存放在编号目录下，多数文件都是随机名称，例如 `./recup_dir.1/f872690288.jpg`，`./recup_dir.1/f864563104_wmclockmon-0.1.0.tar.gz`。

参考

- 如何获取原始文件名：PhotoRec FAQ
- 如何添加自定义文件特征：CGSecurity Wiki
- Wiki（TestDisk）：https://www.cgsecurity.org/wiki/TestDisk
- Wiki（Photorec）：https://www.cgsecurity.org/wiki/PhotoRec
- 主页：https://www.cgsecurity.org/

e2fsck

e2fsck 是一个 ext2/ext3 文件系统检查器，已包含在 Arch 的基本安装环境中。e2fsck 依赖于有效的超级块（superblock），其中描述了整个文件系统的参数。由于该数据

非常重要，其内容被多次复制到了分区的多个分散位置中。使用 `-b` 选项可以向 e2fsck 传入替代 superblock 参数，适用于主（即首个）超级块出现损坏的情况。

对未挂载的目标执行 `dumpe2fs -h` 可找到超级块的位置。取决于创建文件系统时块大小的设置不同，超级块的间距也会变动。

另一种确定超级块位置的方法是向 mke2fs 传入 `-n` 选项。**务必确保**使用了 `-n` 选项，根据 mke2fs(8) 所说，它会让 mke2fs 不要实际创建文件系统，而是显示在创建文件系统时会进行的更改。只要传入和创建文件系统时一致的参数（另外要加上 -n），就可以用它来确定特定文件系统中备份超级块的位置。

安装

`e2fsck` 和 `dumpe2fs` 已作为 e2fsprogs包 的一部分包含着默认 Arch 安装中。

另请参考 e2fsck(8) 和 dumpe2fs(8)。

操作原始（raw）硬盘镜像

**本文或本章节可能需要合并到QEMU。**

该部分适用于已通过 ddrescue 或 dd 将硬盘进行备份，并需要将镜像挂载为物理硬盘的情况。

挂载整个硬盘

要将整个硬盘挂载到下个可用回环设备上，请使用 `losetup` 命令：

```
# losetup -f -P /path/to/image
```

提示：

- `-f` 标志会将硬盘挂载到下一个可用回环设备上。
- `-P` 标志会为每个分区创建一个额外的设备。

另请参考 QEMU#loop模块自动检测分区.

挂载分区

为挂载整个硬盘镜像中的单个分区，请先参考以上步骤。

在挂载了整个硬盘镜像后，可以对回环设备使用正常的 `mount` 命令：

```
# mount /dev/loop0p1 /mnt/example
```

该命令会将 loop0 中的镜像的首个分区挂载到 `/mnt/example` 下。记住，指定的挂载点文件夹必须已经存在！

获取硬盘布局

在将硬盘镜像挂载为回环设备后，就可以检查其硬盘布局。

使用 QEMU 来修复 NTFS

由于 Linux 下没有好的 NTFS 检查工具，包含一个或多个 NTFS 分区的硬盘需要使用 Windows 进行 `chkdsk`。 QEMU 可以在虚拟机中将 raw 原始磁盘映像作为物理硬盘使用：

```
# qemu -hda /path/to/primary.img -hdb /path/to/DamagedDisk.img
```

然后, 假设 Windows 被安装在 `*primary*.img`，它就可以用来检查 `*/path/to/DamagedDisk*.img` 上的分区。

警告：不要在低版本的 Windows 上检查高版本 Windows 创建的 NTFS 分区，例如对于 Windows 8 创建的 NTFS 分区， Windows XP 会通过“修复”其不支持的元数据配置，导致这些配置项损坏或被移除。

文本文件恢复

在块设备上有可能通过直接搜索找出被删掉的纯文本文件。你需要一段来自待恢复的文件中的比较独一无二的字符串。

使用 `grep` 在分区上直接筛选固定字符串（`-F`）：

```
$ grep -a -C 200 -F 'Unique string in text file' /dev/sdXN > OutputFile
```

如果运气好的话，被删除文件的内容会出现在 *OutputFile* 中，可直接从文本的上下文中提取。

注意：`-C -200` 选项告诉 grep 打印每个匹配字符串的前后 200 行。也可以使用 `-A` 和 `-B` 参数，分别会输出匹配字符串之后和之前的内容。如果你在寻找的文本很长，则需要视情况调整。

参阅

- Data Recovery on the Ubuntu wiki

5.8 Parted

难度 进阶 | **篇幅** 15.0 KB | **所属** 系统恢复

本节将讲到

- 安装
- 使用
- 数值设定
- 分区
- 已知问题

GNU Parted 是创建和处理分区表的程序。GParted 是 GUI 前端。

安装

安装下列任一软件包：

- parted包 – 仅命令行版本
- gparted包 – *parted* 的图形化前端

注意：在 Wayland 下运行 GParted 同时需要安装 xorg-xhost包 可选包。更多信息请参考

使用

Parted 有两种模式：命令行模式和交互模式，请用下面命令启动：

```
# parted device
```

其中 `*device*` 是要编辑的块设备（例如 `/dev/sda`，`/dev/nvme0n1`，`/dev/mmcblk0` 等）。如果你缺省了 `*device*` 参数，*parted* 将尝试猜测要使用的设备。

命令行模式

在命令行模式下，可以同时执行一个或多个命令：

```
# parted /dev/sda mklabel gpt mkpart P1 ext3 1MiB 8MiB
```

注意：`--help` 等参数只有在命令行中才能指定。

交互模式

交互模式简化了分区过程，并自动将所有命令应用到指定的设备上，无需反复指定目标设备。

In order to start operating on a device, execute:

```
# parted /dev/sdx
```

命令提示符会从 (`#`) 变成 `(parted)`：这也意味着无需将新的命令提示符作为命令的一部分进行输入。

要查看可用的命令，输入：

```
(parted) help
```

完成操作后，或是要对另一个设备创建分区表或格式时，用下面命令退出：

```
(parted) quit
```

退出后命令提示符会变回到 `#`。

如果命令没带参数，Parted 会进行询问：

```
(parted) mklabel
New disk label type? gpt
```

数值设定

很多分区系统有复杂的限制，Parted 可能会对输入的数值进行稍微的修改。例如设定了 10.4Mb，实际会使用 10.352Mb。如果修正后的数值差异太大，Parted 会进行提示

确认。如果你有需要指定数值，或是想看看 Parted 所做的操作，可以用扇区值加上“s”后缀指定数值，并使用“unit s”命令让分区信息以扇区值为单位显示。

从 parted-2.4 开始，当使用 “MiB”, “GiB”, “TiB” 等 IEC 单位时，parted 会使用与对应的字节值（即“B”后缀）相同的精确数值，不进行任何修正。而使用“4GB”这样的值时，可能会落到最大前后相差 500MB 的位置上。因此，在创建分区时，建议使用字节（“B”），扇区（“s”）或是像“MiB”这样的 IEC 二进制单位，而不是“MB”，“GB”等。

分区

创建新分区表

警告：如果你在存有数据的硬盘上创建新分区表，硬盘上所有数据都会被清除。因此，在进行操作前请再次进行确认。

提示：

- 在分区前，检查你的 NVMe 硬盘和先进格式硬盘是否使用了最佳逻辑扇区大小。
- 在对固态硬盘进行分区之前，考虑执行存储单元清理（英语：Solid state drive/Memory cell clearing）。

如果一个硬盘没有做过分区，或者是你想要修改分区表的类型，你需要为对应的设备创建/重建分区表。另外，对于分区方案要完全重建的设备来说，重建分区表也很有用。

用下面的命令打开要创建/重建分区表的设备：

```
# parted /dev/sdx
```

使用以下命令创建 GUID 分区表：

```
(parted) mklabel gpt
```

使用下列命令创建 MBR/MS-DOS 分区表：

```
(parted) mklabel msdos
```

分区方案

你可以决定磁盘分区的数量和各自的大小，以及分区应挂载到安装的系统上哪些文件夹（一般称为*挂载点*）。可以在分区#分区方案查看具体需要的分区。

创建分区将用到如下命令：

```
(parted) mkpart part-type-or-part-label fs-type start end
```

- 根据分区表的不同，`*part-type-or-part-label*` 有不同的解释：
- MBR：参数被用作为*分区类型*，可选值为 `primary`，`extended` 或 `logical`。

- GPT：参数被用作为 `*part-label*`，即分区的 PARTLABEL 参数。如果想不设置分区标签，可以使用引上的空字符（""）。

> **注意**：网上的很多教程对 GPT 也使用了 `mkpart primary` 开头的错误命令，这一命令会将分区标签设置为“primary”。

- `*fs-type*` 是 `help mkpart` 命令显示的标识符中最接近使用的文件系统的一项。*mkpart* 命令并不会创建文件系统：`*fs-type*` 参数只是简单地被 *parted* 用于为 GPT 分区设置分区类型 GUID 或是为 MBR 分区设置分区类型 ID。

> **提示**：*parted* 将 Linux 原生文件系统都映射为同一“Linux 文件系统”分区类型（GUID `0FC63DAF-8483-4772-8E79-3D69D8477DE4` 或是 ID `0x83`），所以可以安全地进行例如为 *ext4* 格式的分区使用 `ext2` 类型的操作。

- `*start*` 是相对于设备起始位置的分区开始位置，由一个数字加上单位组成，例如 `1MiB` 值从 1 MiB 的位置开始。
- `*end*` 是相对于设备起始位置的分区结束位置（*不是*从 `*start*` 的值开始）。它的格式与 `*start*` 相同，例如 `100%` 表示到设备的末端（使用所有剩余空间）。

提示：

- 对于使用 MBR 分区表的硬盘，建议在硬盘末端留下至少 33 个 512-byte 扇区（16.5 KiB）的未分区空间，以保证 MBR 与 GPT 间转换的兼容性。
- 为了不留空隙，分区的开始和结束应该首尾相连。

注意：

- *parted* 不支持指定一个分区相对于前一个分区的开始和结束位置。参见 bug #59176 和 bug #55841.
- *parted* 可能会报出以下错误：

```
Warning: The resulting partition is not properly aligned for best performance.
Ignore/Cancel?
```

在这种情况下，请参考分区#分区对齐并参照#对齐进行修复。

下列命令将包含 `/boot` 目录的分区标记为可启动：

```
(parted) set partition boot on
```

- `*partition*` 是要标记的分区编号（参考 `print` 命令的输出）。
- `esp` 是 GPT 上 `boot` 的别名。

UEFI/GPT 示例

在每一个实例上都需要一个特殊的 EFI 系统分区。

首先需要一个 EFI 系统分区.如果是和 Windows 双系统启动，此分区已经存在，不要重新创建。

使用如下命令创建新的 EFI 系统分区（建议至少 1 GiB）：

```
(parted) mkpart "EFI system partition" fat32 1MiB 1025MiB
(parted) set 1 esp on
```

剩下的空间可以随你决定，例如完全用在根分区上：

```
(parted) mkpart "root partition" ext4 1025MiB 100%
(parted) type 2 4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709
```

如果要拆分为 swap （4 GiB）和 / （剩余所有空间）两个分区：

```
(parted) mkpart "swap partition" linux-swap 1025MiB 5121MiB
(parted) mkpart "root partition" ext4 5121MiB 100%
(parted) type 3 4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709
```

如果要拆分为 swap（4 GiB），/（32 GiB）和 /home（剩余所有空间）三个分区：

```
(parted) mkpart "swap partition" linux-swap 1025MiB 5121MiB
(parted) mkpart "root partition" ext4 5121MiB 37889MiB
(parted) type 3 4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709
(parted) mkpart "home partition" ext4 37889MiB 100%
(parted) set 4 linux-home on
```

BIOS/MBR 示例

最简单的方案是使用如下命令创建使用所有空间的单一主分区：

```
(parted) mkpart primary ext4 1MiB 100%
(parted) set 1 boot on
```

在下面的示例中，将创建一个 4 GiB 的交换分区，然后使用所有剩余空间创建一个 / 分区：

```
(parted) mkpart primary linux-swap 1MiB 4097MiB
 parted) mkpart primary ext4 4097MiB 100%
(parted) set 2 boot on
```

在下面的最后一个示例中，将创建独立的 /boot (1 GiB)、swap (4 GiB)、/ (32 GiB) 和 /home（所有剩余空间）分区：

```
(parted) mkpart primary ext4 1MiB 1025MiB
(parted) set 1 boot on
(parted) set 1 bls_boot on
(parted) mkpart primary linux-swap 1025MiB 5121MiB
(parted) mkpart primary ext4 5121MiB 37889MiB
(parted) mkpart primary ext4 37889MiB 100%
```

调整分区

警告：ext2/3/4 分区在调整前需要先卸载并停止使用。编辑使用中的系统的根分区会非常困难和危险，建议使用安装介质/救援系统进行操作。

注意：

- parted 只能移动分区的末端。
- parted v3.2 *resizepart* 可能需要用到#交互模式。
- 这些说明适用于使用 ext2，ext3 或 ext4 文件系统的分区。

如果要扩展分区，必须先调整分区的大小，然后调整其上文件系统的大小；如果要缩小分区，必须先调整文件系统的大小，再调整分区，以避免数据丢失。

扩展分区

在 parted 交互模式下扩展分区，使用：

```
(parted) resizepart number end
```

其中 *number* 是您正在扩展的分区的编号，而 *end* 是该分区的新末端（需要大于旧的末端）。

要扩展分区上的 ext2/3/4 文件系统，使用如下命令（如果没有指定 *size*，默认值为分区的大小）：

```
# resize2fs /dev/sdaX size
```

要扩展 Btrfs 文件系统：

```
# btrfs filesystem resize size /path/to/mount/point
```

其中 */path/to/mount/point* 代表要扩展的分区的挂载点，*size* 是新大小（格式如 16G）或是修改值（格式如 +1G）。使用 max 可以填充分区中的所有剩余空间。

缩小分区

缩小分区上的 ext2/3/4 文件系统：

```
# resize2fs /dev/sdaX size
```

注意：与 parted 不同，resize2fs(8) 使用 K，M，G 和 T 指代 KiB，MiB，GiB 和 TiB。注意，e2fsprogs 包 文档错误地将 kibibytes，mebibytes，gibibytes 和 tebibytes 称为“kilobytes，megabytes，gigabytes，terabytes 的二次方"。

缩小 Btrfs 文件系统：

```
# btrfs filesystem resize size /path/to/mount/point
```

其中 */path/to/mount/point* 代表要缩小的分区的挂载点，而 *size* 是该分区的新大小。

然后在 parted 交互模式下缩小分区：

```
(parted) resizepart number end
```

其中 *number* 是要缩小的分区的编号，而 *end* 是该分区的新末端（需要小于旧末端）。

完成后，使用 util-linux包 中的 *resizepart* 命令告诉内核新的分区大小：

```
# resizepart device number size
```

其中 *device* 是保存分区的设备，*number* 是分区的编号， *size* 是格式为 512 字节扇区的分区新大小。

已知问题

除非命令本身就具有危险性（例如*rm*，*mklabel* 和 *mkpart*），Parted 始终会在进行具有潜在危险性的操作前发出警告。

对齐

警告：*parted* 仅对齐分区起始端，不对齐大小。这一特性不符合 dm-crypt/LUKS 的需求，详细信息请参考 Advanced Format#Partition alignment。

在创建分区时，*parted* 可能会对分区未对齐发出警告，但不会对正确对齐的分区发出提示。例如：

```
(parted) mkpart primary fat16 0 32M
Warning: The resulting partition is not properly aligned for best performance.
Ignore/Cancel?
```

这一警告意味着分区的起始端未对齐。输入“Ignore”来无视警告，以扇区为格式输出分区表来查看起始端位置，以舍入到 2 次方的值递增作为起始端移除并重建分区，直到警告停止。 举个例子，在具有 512B 扇区的闪存盘上，Parted 期望扇区的起始位置为 2048 的乘积，即 1 MiB 对齐。

如果你希望 *parted* 自动计算正确的对齐值，可以使用 0% 而不是特定值作为起始端。例如，如果要创建一个大 ext4 分区，可以使用如下命令：

```
(parted) mkpart primary ext4 0% 100%
```

小技巧

Check alignment

警告：*parted* 仅验证分区起始端是否对齐，而不验证大小。这一特性不符合 dm-crypt/LUKS 的需求，详细信息请参考 Advanced Format#Partition alignment。

在已分区的硬盘上，你可以用 *parted* 验证单个分区是否对齐。例如，要验证 `/dev/sda` 上的分区 1，可以使用如下命令：

```
# parted /dev/sda
(parted) align-check optimal 1
1 aligned
```

参考

- GNU parted - Parted 用户手册
- How to align partitions for best performance using parted
- 调整 ext3/ext4 分区大小
- GParted 官方论坛

5.9 GPT fdisk

难度 进阶 | **篇幅** 16.9 KB | **所属** 系统恢复

本节将讲到

- 安装
- 列出分区
- 备份和恢复分区表
- 创建分区表和分区
- 提示和技巧

GPT fdisk 是由 Rod Smith 编写的一套用于硬盘分区的文本模式工具集，由 *gdisk*, *cgdisk*, *sgdisk* 和 *fixparts* 组成。它适用于 GUID（Globally Unique Identifier）分区表（GPT）硬盘，而不是过去常用的 MBR（Master Boot Record）分区表。

gdisk, *cgdisk* 和 *sgdisk* 在功能上一致，但提供了不同类型的用户界面。*gdisk* 提供文本交互模式，*sgdisk* 提供命令行模式，而 *cgdisk* 有基于 curses) 的用户界面。该篇文章涵盖了 gdisk(8) 和 sgdisk(8) 工具的使用。

提示：

- cgdisk(8) 提供了基本的功能和命令行用户界面。
- GPT fdisk 网站包含 gdisk，cgdisk，sgdisk 和 FixParts 的详细使用步骤。

安装

安装 gptfdisk包。

列出分区

要显示块设备上的分区表和分区信息，可以使用如下命令，其中设备名类似于 `/dev/sda`，`/dev/nvme0n1`，`/dev/mmcblk0` 等：

```
# gdisk -l /dev/sda
```

或者使用 *sgdisk* 命令:

```
# sgdisk -p /dev/sda
```

备份和恢复分区表

修改磁盘前，请先备份磁盘的分区表，可以使用备份将相同的分区布局复制到多个设备上。

sgdisk 可以创建一个二进制备份，包含 MBR, GPT 主表头，GPT 备份表头和分区表副本。下面示例将 `/dev/sda` 的分区表信息备份到 `sgdisk-sda.bin`:

```
# sgdisk -b=sgdisk-sda.bin /dev/sda
```

通过下面命令恢复备份:

```
# sgdisk -l=sgdisk-sda.bin /dev/sda
```

如果要复制分区到其它磁盘，例如从 `/dev/sda` 复制到 `/dev/sdc`:

```
# sgdisk -R=/dev/sdc /dev/sda
```

如果两个磁盘位于同一个计算机，使用下面命令设置随机的分区 GUID:

```
# sgdisk -G /dev/sdc
```

创建分区表和分区

分区操作的第一步是创建一个分区表。创建分区表后，就可以根据目标分区方案来创建分区了。

在开始之前，建议先备份当前的分区表和分区布局。

下面的步骤演示了如何使用 *gdisk* 来创建分区表和分区。另外，你也可以使用基于 curses 的 *cgdisk*，但下面的操作步骤就不再适用于你了。对应的用法可参考 cgdisk(8)。

gdisk 自动将分区按 2048 个 512 字节扇区（1 MiB）对齐，这一设置兼容所有先进格式化 HDD 和大部分的 SSD。

使用 *gdisk*时, 将要编辑的块设备作为命令参数。以 `/dev/sda` 为例：

```
# gdisk /dev/sda
```

创建新分区表

警告：在已经保存数据的磁盘上创建新分区，会删除磁盘上的所有数据。

提示：在分区前，先确认你的 NVMe 硬盘和先进格式化硬盘是否使用了最佳逻辑扇区大小。

在提示中输入 `o` 来创建新的 GPT 分区表并清除当前所有的分区数据。如果分区已经创建了你需要的分区表，请跳过此步骤。

创建分区

在提示中输入 `n` 命令来创建新分区。你需要提供分区编号，起始扇区位置，末端扇区位置及分区类型。

注意：对于分区大小和位置的相关问题可参考分区#分区方案。

分区编号

分区编号是分配给分区的一个数字，例如：磁盘 `/dev/sda` 上编号为 `1` 的分区是 `/dev/sda1`，类似的有 `/dev/nvme0n1` 上的 `/dev/nvme0n1p1` 和 `/dev/mmcblk0` 上的 `/dev/mmcblk0p1`。命名方案的细节可参考 Device file#Partition。分区编号不一定与分区在硬盘上的顺序匹配，在这种情况下你可以对它们进行排序。

建议使用 *gdisk* 提出的默认编号。

起始和末端扇区

分区的起始和末端扇区可以用扇区数或是以千字节（K），兆字节（M），吉字节（G），太字节（T）或是皮字节（P）为单位的位置进行指定。

位置可以以用下方式进行指定：

- 从硬盘起始端开始计算的绝对值，例如：`40M` 起始端意味着扇区从硬盘的第 40 MiB 开始。
- 以 `**+***size*` 或 `**-***size*` 为格式的相对位置，例如：`+2G` 代指默认起始扇区后 2 GiB 的位置，而 `-200M` 代指最后一个可用扇区往前 200 MiB 的位置。

Pressing the `Enter` key with no input specifies the default value, which is the start of the largest available block for the first sector and the end of the same block for the last sector.

注意：确保使用 `+*size{M,G,T,P}*` 格式的相对值指定分区大小，且不使用小于 1 MiB 的值，这样分区就能始终和设备属性对齐。

提示：为了防备需要创建 BIOS 启动分区_特殊操作)的情况发生，建议在硬盘前 2 TiB 的任意位置保留一个 1 MiB 大小的空余空间（例如使用 `+1M` 参数创建首个分区）。

分区类型

可以通过输入 gdisk 内置类型编码或手动指定分区类型 GUID 来选择分区类型。默认的 `Linux filesystem`（GUID `0FC63DAF-8483-4772-8E79-3D69D8477DE4`，gdisk 内置编码 `8300`）类型适用于大多数用例。

提示：

- 输入 `L` 可显示 gdisk 内置编码列表。
- 建议遵循分区发现规范以让 systemd-gpt-auto-generator(8) 进行自动挂载。如果你想禁用自动挂载，可参考#Prevent GPT partition automounting。

<table class="wikitable"><caption>常见分区类型</caption><tbody><tr><th>分区类型</th><th>挂载点</th><th>gdisk 编码</th><th>分区类型 GUID</th></tr><tr><td>Linux 文件系统</td><td>Any</td><td><code>8300</code></td><td><code>0FC63DAF-8483-4772-8E79-3D69D8477DE4</code></td></tr><tr><td>EFI 系统分区</td><td>Any¹</td><td><code>ef00</code></td><td><code>C12

A7328-F81F-11D2-BA4B-00A0C93EC93B</code></td></tr><tr><td>BIOS 启动分区</td><td>None</td><td><code>ef02</code></td><td><code>21686148-6449-6E6F-744E-656564454649</code></td></tr><tr><td>Linux x86-64 根分区 (/)</td><td><code>/</code></td><td><code>8304</code></td><td><code>4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709</code></td></tr><tr><td>Linux swap</td><td><code>[SWAP]</code></td><td><code>8200</code></td><td><code>0657FD6D-A4AB-43C4-84E5-0933C84B4F4F</code></td></tr><tr><td>Linux /home</td><td><code>/home</code></td><td><code>8302</code></td><td><code>933AC7E1-2EB4-4F13-B844-0E14E2AEF915</code></td></tr><tr><td>Linux /srv</td><td><code>/srv</code></td><td><code>8306</code></td><td><code>3B8F8425-20E0-4F3B-907F-1A25A76F98E8</code></td></tr><tr><td>Linux /var</td><td><code>/var</code>¹</td><td><code>8310</code></td><td><code>4D21B016-B534-45C2-A9FB-5C16E091FD2D</code></td></tr><tr><td>Linux /var/tmp</td><td><code>/var/tmp</code>¹</td><td><code>8311</code></td><td><code>7EC6F557-3BC5-4ACA-B293-16EF5DF639D1</code></td></tr><tr><td>Linux LVM</td><td>Any</td><td><code>8e00</code></td><td><code>E6D6D379-F507-44C2-A23C-238F2A3DF928</code></td></tr><tr><td>Linux RAID</td><td>Any</td><td><code>fd00</code></td><td><code>A19D880F-05FC-4D3B-A006-743F0F84911E</code></td></tr><tr><td>Linux LUKS</td><td>Any</td><td><code>8309</code></td><td><code>CA7D7CCB-63ED-4C53-861C-1742536059CC</code></td></tr><tr><td>Linux dm-

crypt</td><td>Any</td><td><code>8308</code></td><td><code>7FFEC5C9-2D00-49B7-8941-3EA10A5586B7</code></td></tr></tbody></table>

1. systemd-gpt-auto-generator(8) 只会在满足特定条件的情况下自动挂载分区，详细信息可参考 systemd#GPT分区自动挂载。

重复这一步骤，直到创建完所有需要的分区。

写入更改到硬盘

提示：为了方便区分，可以使用 `c` 命令修改分区名称（分区标签）。

使用 `w` 命令来向硬盘写入分区表并退出。

提示和技巧

在 MBR 和 GPT 之间转换

提示：更多信息请阅读 Rod Smith 的 MBR 与 GPT 互转换说明。

gdisk、*sgdisk* 和 *cgdisk* 能将 MBR 和 BSD 盘符无损转换为 GPT。转换时，所有 MBR 主分区和逻辑分区都会变成 GPT 分区，并为每个分区生成正确的分区类型 GUID 和唯一分区 GUID。

转换之后，需要重新安装并配置引导加载程序才能正常从 GPT 启动。

警告：

- GPT 在硬盘末端存放了第二分区表。该数据结构默认消耗 33 个 512 字节（16.5 KiB）扇区大小。MBR 在硬盘末端没有类似的数据结构，也意味着如果 MBR 硬盘上的最后一个分区占用了硬盘的最末端，会导致转换无法完成。如果你遇到了这种情况，需要放弃转换，并调整最后一个分区的大小。

要用 *sgdisk* 将 MBR 分区表转换为 GPT，使用 `-g`/`--mbrtogpt` 选项:

```
# sgdisk -g /dev/sda
```

要将 GPT 转换为 MBR，使用 `-m`/`--gpttombr` 选项，注意最多只能转换四个分区。

```
# sgdisk -m /dev/sda
```

Sort partitions

This applies for when a new partition is created in the space between two partitions or a partition is deleted. `/dev/sda` is used in this example.

```
# sgdisk -s /dev/sda
```

After sorting the partitions if you are not using Persistent block device naming, it might be required to adjust the `/etc/fstab` and/or the `/etc/crypttab` configuration files.

注意：The kernel must read the new partition table for the partitions (e.g. `/dev/sda1`) to be usable. Reboot the system or tell the kernel to reread the partition table.

Recover GPT header

In case main GPT header or backup GPT header gets damaged, you can recover one from the other with *gdisk*. `/dev/sda` is used in this example.

```
# gdisk /dev/sda
```

choose `r` for recovery and transformation options (experts only). From there choose either

- `b`: use backup GPT header (rebuilding main)
- `d`: use main GPT header (rebuilding backup)

When done write the table to disk and exit via the `w` command.

Expand a GPT disk

After enlarging a disk (e.g. in hardware RAID or a virtual machine disk) the newly added free space will not be immediately usable since GPT keeps data at the end of the disk. You must relocate the backup GPT header to the new end of the disk.

Run *sgdisk* with the option `-e`/`--move-second-header`, e.g.:

```
# sgdisk -e /dev/sda
```

Afterwards print the partition table; the total free space should now be increased.

Prevent GPT partition automounting

systemd-gpt-auto-generator(8) will automount partitions following the Discoverable Partitions Specification. Sometimes that may not be desirable.

The automounting can be disabled by setting the partition attribute `63` "do not automount" on the partition.

Start *gdisk*, e.g.:

```
# gdisk /dev/sda
```

Press `p` to print the partition table and take note of the partition number(s) of the for which you want to disable automounting.

Press `x` *extra functionality (experts only)*.

Press `a` *set attributes*. Input the partition number and set the attribute `63`. Under `Set fields are:` it should now show `63 (do not automount)`. Press `Enter` to end attribute changing. Repeat this for all partitions you want to prevent from automounting.

When done write the table to disk and exit via the `w` command.

Alternatively using *sgdisk*, the attribute can be set using the `-A`/`--attributes=` option; see sgdisk(8) for usage. For example, to set partition attribute `63` "do not automount" on `/dev/sda2` run:

```
# sgdisk -A 2:set:63 /dev/sda
```

gdisk EFI application

There is no package for the EFI version of gdisk, but Rod Smith provides a prebuilt gdisk-1.04 EFI binary on SourceForge. Download `gdisk-efi-*.zip` and extract the archive. To use it, copy `gdisk_x64.efi` to the EFI system partition and launch it from your boot loader or UEFI Shell.

gdisk_x64.efi allows you to edit the partition table before the operating system is even booted. It is used the same way as the *gdisk* binary on Linux.

注意：*gdisk_x64.efi* cannot access the file system, thus it cannot backup the partition table to a file or restore it from a backup file.

See README-efi.txt for more information.

参阅

- GPT fdisk 教程 - 带有详细演示步骤的 GPT fdisk 官方页面
- GPT fdisk SourceForge 页面

5.10 常规故障排除

难度 高级 | **篇幅** 26.0 KB | **所属** 系统恢复

本节将讲到

- 常用手段
- 系统启动问题
- 调试系统卡死问题
- 处理软件倒退问题
- 内核升级后部分外设无法使用

本文介绍了一些常规的故障排除方法。有关特定应用程序的问题，请参阅该特定程序的 wiki 页面。

常用手段

出现的错误消息请*务必*阅读，这一点很重要。有时可能很难获得正确的错误消息（比如出错的是图形应用程序）。

1. 在终端中运行应用程序，这样可以查看输出。
2. 如果仍然没有足够的信息进行调试，请增加输出的详细度（通常加上参数 `--verbose`/`-v`/`-V` 或 `--debug`/`-d`）。
3. 如果没有这样的参数，可能是要在应用程序的配置文件中指定调试模式。
4. 应用程序可能会有日志文件，日志文件通常位于 `/var/log`、`$HOME/.cache` 或 `$HOME/.local`。

5. 如果没有办法增加输出的详细度，那么运行 strace 和类似的命令总是可行的。
6. 检查日志。错误也可能在日志中留下痕迹，特别是当这个程序依赖于其他程序时。
7. *dmesg* 从内核环形缓冲区读取日志。当磁盘由于某种原因无法访问时这很有用，但这也可能导致日志不完整，因为内核环形缓冲区的大小是有限的。如果可以的话尽量使用 *journalctl*。
8. *journalctl* 比 *dmesg* 有着更多的过滤选项，且默认使用人类可读的时间戳。
9. 建议检查相关应用的问题跟踪列表，看看这是不是已知问题，以及是否存在解决方案。
10. 应用通常是有问题跟踪列表的，具体取决于上游的选择，有时应用还有论坛，甚至有 IRC 频道。
11. Arch Linux bugtracker 主要用于打包 bug。

额外支援

如果需要额外支援，你可以在 the forums 或 IRC 上提问。

注意：技术支持 *仅限于* Arch Linux 而非 基于 Arch 的发行版。

当要求贴出**完整**的 输出 / 日志 时，不能仅仅贴出你认为的重要部分。信息来源应该包括：

- 所有涉及到的命令的完整输出，不要只选择你认为相关的东西。
- systemd 的日志。
- 要获得更广泛的输出，请使用 `systemd.log_level=debug` 引导参数，但这会产生大量的输出，因此仅在确实需要时才启用它。
- 不要使用 `-x` 参数，因为这会使输出混乱，让它难以阅读。
- 除非需要上次启动的日志，否则请使用 `-b` 参数。不指定这个参数可能会导致日志很大，甚至超过在线剪贴板的允许大小。
- 相关的配置文件
- 相关的驱动程序
- 相关软件包的版本
- 内核日志：`journalctl -k` 或 `dmesg`（都需要 root 权限）。
- Xorg：所用的显示管理器的设置也决定了 Xorg 的日志位置。
- `Xorg.log` 可能在以下位置之一：系统日志、`/var/log/` 或 `$HOME/.local/share/xorg/`。
- 一些显示管理器，如 LightDM，可能会在它自己的日志目录里面也放一份 `Xorg.log`。

- Pacman：如果最近一次更新导致了问题，可以查看 `/var/log/pacman.log`。
- 使用 *pacman*的 `--debug` 参数也很有效。

最好将上述收集到的信息放到一个在线剪贴板里。

在线剪贴板将返回一个链接，你可以把它贴到论坛或 IRC。

此外，在提问前最好复习一下如何正确报告问题。

系统启动问题

诊断系统启动问题时，了解问题出在启动的哪一阶段很重要。

1. 固件（UEFI 或 BIOS）
2. 往往只有非常基础的调试工具。
3. 确保安全启动已关闭。
4. 引导加载器
5. 此处最容易引发故障的因素是修改了内核参数。
6. initramfs
7. 通常能提供一个 emergency shell。
8. 在 shell 中可以用 dmesg 或 journal，取决于构建时选择的钩子。
9. 实际系统
10. 根据故障的严重程度，可能只需要调用调试控制台就行了。

如果某一阶段的调试工具无法修复损坏的组件，请尝试使用包含最新 Arch Linux ISO 的U盘。

控制台的输出信息

在启动过程完成以后，屏幕会被清空并显示登录提示符，这使得用户无法看到初始化过程中的输出和其中的错误信息。这一默认特性可以使用接下来几节中的方法进行修改。

请注意，无论选择下面哪个方法，在启动后通过 `journalctl -k` 或 `dmesg` 都可以显示内核消息，用于检查错误。要显示本次启动的所有日志，请使用 `journalctl -b`。

输出流控制

以下是适用于大多数终端模拟器的基本操作，包括虚拟终端 (VC)：

- 按 `Ctrl+s` 暂停输出。

- 按 `Ctrl+q` 继续输出。

这样不仅会暂停输出，而且会暂停尝试打印到终端的程序，即暂停输出时会阻塞 `write()` 调用。 如果你的 *init* 进程出现冻结，请确保系统控制台没有暂停。

要查看已经显示过的错误信息，参见 Getty#将引导消息保留在 tty1 上。

打印更多内核消息

在启动时大部分内核消息是隐藏的，添加不同的内核参数即可打印出更多消息，最简单的有这些参数：

- `debug`，有如下效果：
- 内核会提高它的控制台日志级别，这样内核日志缓冲区中的所有消息都将打印到控制台。
- systemd 会提高它的日志级别，这会记录下调试信息，这些信息在其他地方是不会生成的。
- `ignore_loglevel`，对于内核来说它和 `debug` 或 `loglevel=8` （debug 对应的级别是 `7`）有相同的效果，但在启动后期日志级别不会提高。

在特定情况下可以添加的参数还有这些：

- `earlyprintk=vga,keep` 打印启动过程中最早期的内核消息，用在还没显示输出就崩溃的情况下。在 UEFI 系统上需要把 `vga` 改成 `efi`。
- `log_buf_len=16M` 将分配一个更大的 (16 MiB) 内核日志缓冲区，确保调试输出不会被覆盖。

生成更多内核调试信息

#打印更多内核消息描述了如何将内核日志缓冲区中的内容打印到控制台，但内核日志缓冲区也不是所有消息都有（除了 systemd 的 debug 输出）。本节讨论如何获取比内核日志更详细的信息。

动态调试

由函数 pr_debug 或类似函数 `dev_dbg()`, `drm_dbg()` 和 `bt_dev_dbg()` 生成的消息不在内核日志里，除非：

- 修改内核源代码，在需要的地方定义 `DEBUG`。
- 利用内核的动态调试功能来启用调试消息。

本节讨论动态调试的用法，当你已经查看了所有内核日志，甚至是 information 级别的日志，但还是想从特定位置获取更多调试信息，那么动态调试很有用。

首先，你用的内核必须是用 CONFIG_DYNAMIC_DEBUG 内核配置选项编译出来的，linux包 包的内核已经满足了要求，如果用这个内核就无需执行任何操作。

然后需要确定所需调试信息在哪，有几种选择：

- 如果问题与特定模块有关，就找到内核模块的名称。比如要排除 Intel 图形处理器的故障，就需要关注 i915 DRM 内核模块.
- 找到内核中相关功能对应的目录，这需要查看（或在线导航）内核源代码以了解其结构。例如，要查看所有 DRM 内核模块的调试消息，可以使用路径 drivers/gpu/drm。

要使用该消息“源”，就必须发起一次动态调试查询，指明要启用哪些调试消息，格式如下：

```
match_type match_parameter flags
```

其中：

- *match_type* 是匹配类型，与上述 2 种选择相对应，可取值为 module 或 file。
- *match_parameter* 是要监控的模块或文件路径，后一种情况允许使用星号作为通配符。
- *flags* 表示如何处理匹配结果，可以取 +p 来开始打印其消息，或者 -p 来撤消打印。

一些查询示例：

- module i915 +p 打印来自 i915 内核模块的调试信息。
- file drivers/gpu/drm/* +p 打印来自 DRM 驱动的调试信息。
- file * +p 打印调试信息。

最后，要实际执行上述查询，可以这样：

- 在运行时执行查询，运行命令：

```
# echo "查询" > /sys/kernel/debug/dynamic_debug/control
```

: 上述命令假设 debugfs 挂载到了 /sys/kernel/debug/，你可以用 mount 来确认。

- 在系统启动时执行查询，添加 dyndbg="*query*" 内核参数。

以上是对动态调试功能非常简略的概述，更多详细信息请参阅文档。

特定子系统的调试

在特定的子系统中，还有许多单独的参数可用于调试，例如 `bootmem_debug`、`sched_debug`。此外 `initcall_debug` 用来检查启动卡死很有用（找到那些没有返回的调用）。特定的信息请查询内核参数文档。

netconsole

netconsole 是一个内核模块，它将所有内核日志消息（即 dmesg）通过网络发送到另一台计算机，不涉及到用户空间（如 syslogd）。 "netconsole" 这个名称不是很准确，因为它并不是真正的控制台，更像是远程日志记录服务。

它可以内置使用或作为模块使用。内置使用的 *netconsole* 在网卡之后就被初始化，并马上启动特定的接口。作为模块使用，主要用于捕获无显示器的计算机的内核崩溃输出，或者另一种情况就是用户空间有故障。

故障恢复控制台

在启动过程中的某个阶段获取一个交互式 shell 可以帮助你准确找出问题出在哪里，以及为何失败。有几个内核参数可以做到这一点，但它们都启动了一个正常的shell，可以随时 `exit` 让内核恢复正在执行的操作：

- `rescue` 在根文件系统刚刚被挂载为读写模式的时候启动一个 shell
- `emergency` 可以在更早的时候启动 shell，早于大部分文件系统挂载之前
- `init=/bin/sh`（作为最后的选择）把 init 程序改成 root shell。因为 `rescue` 和 `emergency` 都依赖于 systemd，而这可以在 *systemd* 坏掉的时候工作

还有一个选择，那就是 systemd 的 debug-shell，它在 `tty9` 上新增了一个 root shell，可以按 Ctrl+Alt+F9 来使用。这个功能可以通过在内核参数中添加 `systemd.debug-shell`，或者是启用 `debug-shell.service` 来打开。

警告：使用完后要记得禁用该服务，避免在每次启动时都打开 root shell 而带来安全风险。

调试内核模块

参阅内核模块#获取信息。

调试硬件问题

- 按照 udev#Debug output 的说明可以显示额外硬件调试信息。
- 确保你的系统已经安装了微码更新。
- 要测试内存请参阅 MemTest86+。
- 要排查系统是否超温请使用 lm_sensors。
- 要检查存储器的健康状况，请参阅 S.M.A.R.T.。

调试系统卡死问题

不幸的是，系统卡死通常很难调试，并且有些需要很长时间才能复现。有些类型的卡死调试起来相对容易一点：

- 声音还在播放？这种情况可能只是显示卡死了，显卡驱动可能有问题。
- 系统还有响应吗？如果切换到其他 TTY 不行的话就试试 SSH。
- 硬盘活动指示灯（如果有的话）是否在指示有大量数据在读写？大量的内存交换也会暂时卡死系统。关于大量写入硬盘时卡死的情况请参阅这个 StackExchange 回答。

如果上述都不行，尝试一次**完整**的关机。*按一次*电源键或许可以使系统有反应，并显示“关机画面”，关机画面包含了所有正在停止的单元。或者使用神奇的 SysRq 键也可以进行完整关机。完整关机很重要，因为日志可能提示了系统卡死的原因。在非正常关闭时日志可能不会写入磁盘。系统毫无反应的情况更难调试，因为日志无法及时写入磁盘。

如果卡死以后没法将任何内容写入磁盘，可以尝试远程日志。以下是一个粗略的远程日志方案，需要从另一个设备发起，可用于基本调试：

```
$ ssh freezing_host journalctl -f
```

许多致命卡死（整个系统无响应只能强制关机）可能与固件、驱动程序或硬件存在缺陷有关。尝试不同的内核（请参阅内核#调试回退），甚至换个 Linux 发行版或操作系统，还有更新固件并运行硬件诊断可能有助于发现问题。

提示：建议尝试更新设备的固件，因为这些更新可能会修复奇怪的问题。

如果卡死以后无法收集用来调试的日志或信息，请尝试在 live 环境中重现卡死。如果需要图形环境来重现，或者如果在 archiso 上成功重现了卡死，请更换不同发行版的 live 环境，最好不是基于 Arch Linux 的发行版，以排除卡死与版本或内核补丁相关的可能性。如果在 live 环境中仍然卡死，则*很可能*与硬件有关。如果不再发生卡死，那就需要了解两个系统的差异。配置、版本和内核参数的不同，以及其他类似的更改可能已经解决了卡死问题。

但是，大写锁定指示灯闪烁可能表示内核崩溃。当发生内核崩溃时，因为某些设置，TTY 可能不会显示，导致内核崩溃被误以为是另一种卡死。

处理软件倒退问题

警告：这样做往往会导致 部分升级，在特殊情况下这是不可避免的。请谨慎操作并准备好 恢复系统的方法，以防部分升级导致无法启动。

如果是更新导致的问题，且降级特定包可以解决问题，则可能是软件倒退。 如果在正常的完整系统升级后发生这种情况，请检查 `pacman.log` 来确定哪个软件包可能导致了该问题。处理软件倒退最重要的是检查新版本是否已解决问题，这样可以节省大量时间。为此，首先确保应用程序已完全更新（确保程序与官方仓库中的版本相同）。如果已经最新或者更新不能解决问题，请尝试使用实际的最新版本，通常是 \-git 版本，它可能已经打包在 AUR 中。如果这样能解决问题，并且已修复的版本还不在官方仓库中，请等待新版本打包，然后切换回官方仓库的版本。

如果问题仍然存在，请调试程序或二分查找程序的代码提交历史，并在上游的问题跟踪系统里报告错误，以便修复问题。

注意：处理内核的倒退问题需要 稍微不同的方法。

内核升级后部分外设无法使用

通常（但可能不仅仅）表现为：

- 新插入的 USB 设备显示在 *dmesg* 中，但不在 `/dev/` 中，
- 如果文件系统在内核更新前没有在使用，就无法挂载，
- 如果笔记本上的有线/无线连接在内核更新前没有在使用，就无法使用，
- 用 *modprobe* 加载一个内核更新前没有加载的模块时提示 `FATAL: Module *module* not found in directory /lib/module/*kernelversion*`。

正如系统维护#在更新后重启程序和系统所述，更新软件包时不会更新内核，内核在重新启动时才会更新。同时，安装新内核时，位于 `/usr/lib/modules/*kernelversion*/` 中的内核模块会被 pacman 删除。 根据 FS#16702 中的解释，这种方法可以避免在系统上留下包管理器管理范围外的文件，但会导致上述症状。要修复这类问题，请在更新内核后重新启动系统。远期目标（尚未实现）是使用版本化内核包：主要障碍是如何删除无用的老版本内核。

另一种解决方案是 kernel-modules-hook包，其中包含两个 pacman 钩子，在内核更新后使用 rsync 将内核模块保留在文件系统上，并启用 `linux-modules-cleanup.service` 在四周后删除旧模块。

软件包管理

参阅适用于一般主题的 Pacman#疑难解答，以及适用于 PGP 密钥问题的 pacman/软件包签名#问题解决。

修复受损的系统

如果进行了部分升级，请尝试升级整个系统，可能还需要重启一次。

```
# pacman -Syu
```

如果以往是启动到图形界面，但这次失败了，可以尝试按 `Ctrl+Alt+F1` 到 `Ctrl+Alt+F6` 来获得一个能运行 *pacman* 的控制台。

如果系统损坏严重无法运行 *pacman*，使用U盘、光盘或带 PXE 的网络上的 Arch ISO 来启动电脑。（不要执行安装指南里的其余操作）

挂载根文件系统：

```
[ISO] # mount /dev/rootFileSystemDevice /mnt
```

挂载其余的独立分区，在它们前面加上前缀 `/mnt`，例如：

```
[ISO] # mount /dev/bootDevice /mnt/boot
```

尝试使用原系统的 *pacman*：

```
[ISO] # arch-chroot /mnt
[chroot] # pacman -Syu
```

如果失败了，退出 *chroot* 并尝试：

```
[ISO] # pacman -Syu --sysroot /mnt
```

如果也失败了，尝试：

```
[ISO] # pacman -Syu --root /mnt --cachedir /mnt/var/cache/pacman/pkg
```

fuser

fuser 是一个用于识别进程占用的资源（如打开的文件、文件系统和 TCP/UDP 端口）的命令行工具。

fuser 由软件包 psmisc包 提供，已经作为 base包 元包的依赖安装。更多信息请查看 fuser(1)。

会话权限

注意：你必须使用 systemd 作为你的 init 进程，本地会话才能正常工作。 它是各种设备的 polkit 权限和 ACL 所必需的 （参见 `/usr/lib/udev/rules.d/70-uaccess.rules` 和 ）

首先，确保你有一个带 X 的可用本地会话：

```
$ loginctl show-session $XDG_SESSION_ID
```

在输出中需要带有 `Remote=no` and `Active=yes` 字样。如果没有，确保 X 运行在和登录时一样的 tty 里面。这是保留登录会话所必须的。

基本 polkit 操作不需要额外的配置。但有一些 polkit 操作需要请求额外的身份认证，即使是本地会话也是如此。为了达成这项工作，必须运行一个 polkit 身份认证组件。更多信息可参见 polkit#身份认证组件。

错误信息： "error while loading shared libraries"

如果在运行程序时遇到类似于这样的错误：

```
error while loading shared libraries: libusb-0.1.so.4: cannot open shared object file: No such file or directory
```

使用 pacman 或 pkgfile 来查找包含丢失共享库的软件包：

```
$ pacman -F libusb-0.1.so.4
extra/libusb-compat 0.1.5-1
    usr/lib/libusb-0.1.so.4
```

在上述例子中，需要安装软件包 libusb-compat包。也可能是依赖于该共享库的程序需要按照 soname 增加后的库重新编译。

这个错误也有可能意味着你用来安装这个软件的 PKGBUILD 里没有将这个共享库作为它的依赖库：如果来自官方源，请报告一个 bug；如果来自 AUR，请在 AUR 网站相关页面上把它报告给维护者。

参阅

- A how-to in troubleshooting for newcomers
- List of Tools for UBCD - Memtest-like tools to add to grub.cfg on UltimateBootCD.com
- Wikipedia:BIOS Boot partition
- Arch:REISUB
- Debug Logging to a Serial Console on Freedesktop.org
- How to Isolate Linux ACPI Issues on Archive.org

5.11 分区

难度 高级 | **篇幅** 34.2 KB | **所属** 系统恢复

本节将讲到

- 分区表
- 分区方案
- 分区工具
- 分区对齐
- 疑难解答

翻译状态：

- 本文（或部分内容）译自 Partitioning，最近一次同步于 2024-07-06，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

根据维基百科:

: 分区是在外存上创建一个或多个区域的过程，分区的目的是单独管理各个部分。

一个硬盘可以被划分为一个或者多个分区。一些场景需要使用多个分区，例如双重或多重启动、使用 Swap 分区等。此外，分区也可以从逻辑上隔离数据，例如为音频和视频数据创建单独的分区。分区方案存储于#分区表中，例如主引导记录（MBR）或 GUID 分区表（GPT）。

可用分区工具创建和修改分区表。#分区工具中列出了在 Arch Linux 中可用的分区工具。

分区通常直接包含一个文件系统，这是在分区上直接创建的（即格式化）。或者，分区可能包含LVM（逻辑分卷管理器）、加密块设备、RAID（独立磁盘冗余阵列），它们最终提供可存放文件系统的设备文件。

直接包含可挂载文件系统的块设备（硬盘、分区、LUKS 设备、LVM 逻辑卷、RAID 阵列等）称为卷)（也有卷标、宗卷等称谓）。

分区表

分区信息被存放在分区表中。目前有两种主流的模式：传统的#主引导记录（MBR）和新的#GUID 分区表（GPT）。后者功能更强大，解决了许多 MBR 的限制。第三个不常用的方式是使用无分区磁盘，后面也会介绍。

可使用#分区工具查看块设备的分区表。

提示：可以用 `parted */dev/sda* print` 或 `fdisk -l */dev/sda*` 查看当前分区信息。

`/dev/sdX` 是块设备，依不同物理设备有不同的格式：

- `/dev/sda` 是 SATA 设备
- `/dev/nvme0n1` 是 NVMe 设备
- `/dev/mmcblk0` 是 eMMC 设备

更多信息请参看块设备名称

主引导记录

主引导记录（Master boot record，MBR）是存储设备最开始的 512 字节。其包含操作系统启动加载器和分区表，在 BIOS 系统的启动流程中扮演重要角色。可参阅上述维基页面了解 MBR 的结构。

注意：MBR 并不位于某个分区中。其位于分区前，设备的最开始的扇区（物理偏移 0）。 无分区设备的启动扇区或分区中的启动扇区被称为 Volume boot record。

MBR（引导代码）

MBR 的前 440 字节是引导（bootstrap）代码。在 BIOS 系统上，它通常包含引导程序的第一阶段。可以用 dd 备份、恢复、抹除引导代码。

MBR（分区表）

MBR 分区表（或称为 DOS 分区表、MS-DOS 分区表）有三种分区类型：

- 主分区（Primary）
- 扩展分区（Extended）
- 逻辑分区（Logical）

主分区每个磁盘或者 RAID 卷上不能超过 4 个，可设置为可启动状态。如果分区方案要求使用多于 4 个的分区，就需将至少一个分区设置为**扩展分区**，并在上面建立**逻辑分区**。

扩展分区可以被看作是容纳逻辑分区的容器。硬盘上最多只能有1个扩展分区。如果磁盘上有 1 个扩展分区，它也被看作是 1 个主分区。因此只能另外再建立 3 个主分区（例如 3 个主分区加 1 个扩展分区）。扩展分区内所包含的逻辑分区数量没有限制。如果在双重启动中有 Windows，Windows 需要占据一个主分区。

通常习惯是创建主分区 sda1 到 sda3，然后建立一个扩展分区 sda4。sda4 中包含的逻辑分区编号为 sda5、sda6 等。

提示：在进行 MBR 分区时，请考虑在硬盘末尾留下至少 33 个 512 字节扇区（总共 16.5 KiB）的不分区空间。这样是为了方便以后 转换到 GPT 方案，会用于备份 GPT 头。

GUID 分区表

全局唯一标识分区表（GUID Partition Table，GPT）是 UEFI 标准定义的分区规范。使用通用唯一识别码（globally unique identifier，GUID；又称为 universally unique identifier，UUID）定义分区和分区类型。设计上是为了替换#主引导记录。

GUID 分区表的磁盘开始位置有一个保护 MBR（protective Master Boot Record，PMBR），用以处理不支持 GPT 软件的访问。这段 MBR 和真正的 MBR 一样，具备一段#引导代码，后者可以用在支持的启动管理器中进行 BIOS/GPT 启动。

选择 GPT 还是 MBR

#GUID 分区表（GPT）是一种更灵活的分区方式。它正在逐步取代#主引导记录（MBR）系统。GPT 相对于诞生于 MS-DOS 时代的 MBR 而言，有许多优点。新版的分区工具可以让 GPT 和 MBR 都获得不错的可靠性和性能。

在做出选择前，需要考虑如下内容：

- 如果使用 GRUB legacy 作为引导程序，必须使用 MBR。
- 如果使用传统的 BIOS，并且双启动中包含 Windows （无论是 32位版还是 64 位版），必须使用 MBR。
- 如果使用 UEFI 而不是 BIOS，并且双启动中包含 Windows 64 位版，必须使用 GPT。
- 非常老的机器需要使用 MBR，因为 BIOS 可能不支持 GPT。
- 如果不属于上述任何一种情况，可以随意选择使用 GPT 还是 MBR。由于 GPT 更先进，建议选择 GPT。
- 建议在使用 UEFI 的情况下选择 GPT，因为有些 UEFI 固件不支持从 MBR 启动。

注意：为了使 GRUB 从一台有 GPT 分区的基于 BIOS 的系统上启动，需要创建一个 BIOS 启动分区_特殊操作)，这个分区和 /boot 没关系，仅仅是 GRUB 使用，不要建立文件系统和挂载。

GPT 相对于 MBR 的一些优势：

- 为每个磁盘和每个分区提供相应的磁盘 UUID 和分区 UUID（partuuid），这样对分区和磁盘的命名就与其文件系统无关。UUID 同时是 systemd 的可发现分区规范的前提，可以用于启用了 systemd 的 initramfs。
- 提供与文件系统无关的分区名（partlabel）。
- 任意多个分区（取决于为分区表本身分配的空间）使扩展分区和逻辑分区不再必要。GPT 表的默认空间可供 128 个分区使用，但若需要更多分区，也可为分区表分配更多空间。目前仅 gdisk 支持该功能。
- 使用 64 位逻辑块地址（logical block address），可寻址的最大硬盘空间是 2 ZiB。MBR 只支持 2 TiB 的最大硬盘。
- 在磁盘末尾储存头部和分区表，有助于恢复分区表。
- CRC32 校验码可用于侦测头部或分区表的错误。

#分区工具一节列出了用于创建和修改 GPT 或 MBR 表的工具。

提示：GPT 和 MBR 之间可以互相转换。请参阅在 MBR 和 GPT 之间互相转换\[损坏的链接：无效的章节\]。

无分区磁盘

无分区磁盘，即superfloppy 是指无分区表的存储设备，一个文件系统占据整个存储空间。无分区设备上的启动扇区被称做 volume boot record (VBR)。

Btrfs 分区

Btrfs可以独占整个存储设备并替代 #MBR 和 #GPT 分区方案。请参考创建 Btrfs 文件系统以获取更多信息。

分区方案

虽然有一些关于分区方案的通用建议，但没有严格的准则。有许多影响分区方案的因素，例如对灵活性的期望、访问速度、安全性以及可用磁盘空间的硬性限制。实际上就是个人取舍的问题。

如果你想双启动 Arch Linux 和 Windows，请参考 Dual boot with Windows。

警告：请别忘记为引导程序预留空间。这对于 MBR 和 GRUB-Legacy 来说不是问题，但是许多新方案可能要求占用一个特殊的小分区。

- UEFI 系统通常需要有 EFI 系统分区。
- 在 BIOS 系统上使用 #GPT 进行分区后，安装 GRUB 时会需要一个额外的 BIOS 启动分区_特殊操作)。
- 如果使用的是 Btrfs，可以用子卷模拟分区，请参考 Btrfs 子卷。

单root分区

这种是最简单，同时也能满足大部分应用场景的方案。如果需要的话，可以建立一个交换文件。通常刚开始的时候建议一个单独的 / 分区，然后根据应用场景的需要，例如 RAID，加密，独立的多媒体分区等建立其他的分区。

单root分区的大小最小不建议低于23–32 GiB。用户文件或swap文件可能需要更多的空间。

警告：A separate physical (in the main partition table of the disk, not under LVM, software RAID or in a file system subvolume etc.) /boot partition is needed unless your boot loader is capable of accessing the `/boot` directory that resides in `/`. That means the boot loader must have support for everything starting from the block devices, stacked block devices (LVM, RAID, dm-crypt, LUKS, etc.) and ending with the file system on which the kernel(s) and initramfs image(s) reside.

使用 GPT 分区的话其类型 GUID 应当是 `4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709` （“Linux root (x86-64)”，“Linux x86-64 root (/)”）。 使用MBR分区表的话则其类型ID应当是 `83`（“Linux”）。

多分区

将某个路径挂载为独立分区可以使其拥有不同的文件系统和挂载参数。某些情况下（例如多媒体文件分区），可以被多个操作系统共享。

下面这些路径可以作为独立分区的挂载点，你也可以根据实际需要做出其他决定。file-hierarchy(7) 介绍了各个目录的作用。

根分区

根目录是目录树的顶层，这里是主文件系统挂载和其他文件系统挂靠的地方。所有文件和目录都在根目录 / 显示，即使它们实际上存储在其他的物理设备上。根文件系统中的内容必须足以启动、恢复、修复系统。因此 / 目录下的特定目录是不能作为独立分区的。

/ 分区或叫根分区是最重要而且必需的，需要最先挂载，其他其他分区可以被它取代。

警告：与系统启动相关的特定目录（除了 `/boot`） **必须** 与 / 在同一个分区，或在系统刚进入用户态的时候通过 initramfs 挂载。这些特定的目录包括：`/etc` 和 `/usr` 。

因为它通常包括 `/usr` 目录，根据安装的软件数量，会产生非常明显的增长。15-20 GiB 对于大多数用户来说是一个比较合适的取值。如果你打算在这里放一个交换文件或者不建立单独的 /var，需要适当调大取值(增加休眠空间和额外的 8–12 GiB 给 `/var`)。

使用 GPT 分区的话其类型 GUID 应当是 `4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709` （“Linux root (x86-64)”，“Linux x86-64 root (/)”）。使用MBR分区表的话则其类型ID应当是 `83`（“Linux”）。

/boot

`/boot` 分区包含vmlinuz、 initramfs 以及引导加载程序配置文件和引导加载程序阶段。它也可以存放内核在执行用户态程序之前所使用的其他数据。`/boot` 在日常系统运行中并不需要，只在启动和内核升级（重建 initial ramdisk 时）的时候用到。

警告：

- 引导加载程序需要能够访问到 `/boot` 目录。所以引导加载程序需要视情况，支持块设备、堆叠块设备（如软 RAID、加密卷或 LVM 卷）以及存放内核、initramfs 的文件系统。
- 有些文件系统能包含引导加载程序尚不支持的新特性，除非将不兼容特性禁用，否则不适于 `/boot` 分区。通常可以使用 FAT32 以避免此问题，因为 FAT32 包含所有需要的功能，而且不会再增加新的功能。

有关启动加载器的需求和能力的更多信息，请参考 Arch boot process#启动加载器。

注意：在 UEFI 系统中，可以将 EFI 系统分区挂载到`/boot`，这样就可以省去一个单独的 boot 分区。

In other cases, it is recommended to set the partition type to Extended Boot Loader (XBOOTLDR) Partition which is GPT partition type GUID `BC13C2FF-59E6-4262-A352-B275FD6F7172` (`ea00` type for gdisk) or MBR partition type ID `ea`.

在两种情况下，推荐的分区大小都是 1 GiB，这样就可以有足够的空间存放多个内核。如果怕不够，可以分配 4 GiB 空间。

/home

`/home`目录包含用户定义的配置文件、缓存、应用程序数据和媒体文件。

将`/home`目录独立使得/分区可以单独重新划分。但是请注意，在 `/home` 没有独立分区的情况下，仍然可以在不修改 `/home` 目录内容的情况下重装 Arch——删除其他顶级目录，然后执行 pacstrap。

不能与使用其他发行版的用户共享同一个 home 目录，因为不同的发行版可能使用不兼容的软件版本和补丁。可以共享媒体目录，或至少使用 `/home` 分区下的不同 home 目录。分区大小跟着存储的内容而定。

使用 GPT 分区的话其类型 GUID 应当是 `933AC7E1-2EB4-4F13-B844-0E14E2AEF915` （“Linux home”，“Linux /home”）。使用MBR分区表的话则其类型ID应当是 `83`（“Linux”）。

Swap

交换文件或分区提供的硬盘空间作为交换空间使用。它们有同样的功能，没有明显的性能区别，但是前者更易于根据需要调整大小。如果没有使用休眠功能的话，交换分区可以被多个系统分时使用。

以前，在拥有不足512 MiB内存的机器上，通常为交换分区分配两倍内存大小的空间。如果有更大的内存（大于1024 MiB），可以分配较少的空间，甚至不需要交换空间。推荐的默认大小是4 GiB.

要使用休眠（挂起到硬盘）功能，最好分配与内存同样大的交换分区。虽然内核会尝试压缩休眠文件到交换分区的大小，但如果交换空间过于小则不一定成功。请参见休眠。

使用GPT分区的话其类型 GUID应当是 `0657FD6D-A4AB-43C4-84E5-0933C84B4F4F` （“Linux swap”）。使用MBR分区表的话则其类型 ID 应当是 `82` （“Linux swap”）。

/data

可以为需要多用户共享的文件建立一个“data”分区。也可以使用 `/home` 分区用于这一目的。

使用 GPT 分区的话其类型 GUID 应当是 `0FC63DAF-8483-4772-8E79-3D69D8477DE4` （“Linux filesystem”）。 使用MBR分区表的话则其类型ID应当是 `83` （“Linux”）。

/var

`/var` 目录存储变量数据例如 spool 目录和文件，管理和登录数据，pacman 的缓存，ABS 树等等。它通常被用作缓存或者日志记录，因此读写频繁。将它独立出来可以避免由于大量日志写入造成的磁盘空间耗尽等问题。

它的存在是方便将 `/usr` 设置为只读挂载。所有在操作系统运行过程（而非安装或软件维护）中，从前要写入 `/usr` 的东西，放到 `/var` 下。

注意：`/var` 包含许多小文件。如果将其作为独立分区，在文件系统的选择上需要考虑这一点。

值得一提的是，`/var` 包含 pacman 缓存。保留缓存的包提供了软件包降级的能力，因此非常有用。也正因为这样，`/var` 的大小会随着时间推移而增长。尤其是 pacman 缓存将会随着新软件的安装、系统的升级而增长。在磁盘空间不足的时候，可以安全地清理这个目录。

`/var` 分配 8~12 GiB 对于桌面系统来说是比较合适的取值，具体取值取决于安装的软件数量。

使用 GPT 分区的话其类型 GUID 应当是 `4D21B016-B534-45C2-A9FB-5C16E091FD2D` （“Linux variable data”，“Linux /var”）。 使用MBR分区表的话则其类型ID应当是 `83` （“Linux”）。

布局示例

下面的例子使用 `/dev/sda` 作为示例磁盘，其中 `/dev/sda1` 是第一块分区。 The block device naming scheme will differ if you are partitioning a NVMe disk (e.g. `/dev/nvme0n1` with partitions starting from `/dev/nvme0n1p1`) or an SD card or eMMC disk (e.g. `/dev/mmcblk0` with partitions starting from `/dev/mmcblk0p1`). See Device file#Block device names for more information.

注意：

- UEFI/GPT 并没有真正的“boot”标志。启动仅依赖于设备上的相应记录。某些 GPT 分区工具标记的“boot”仅意味着它是一个 EFI 系统分区。
- 不要求所有的分区都在同一个硬盘上。

- 不要求所有的硬盘使用相同的分区表类型。

UEFI/GPT 布局示例

挂载点	分区	分区类型 GUID	推荐大小
`/boot` or `/efi` 1	`/dev/sda1`	`C12A7328-F81F-11D2-BA4B-00A0C93EC93B`: EFI system partition	1 GiB
`[SWAP]`	`/dev/sda2`	`0657FD6D-A4AB-43C4-84E5-0933C84B4F4F`: Linux swap	至少 4 GiB，使用 休眠 时等于内存大小
`/`	`/dev/sda3`	`4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709`: Linux x86-64 root (/)	剩余空间，至少 23 - 32 GiB

MBR/BIOS 示例

挂载点	分区类型	分区类型 ID	Boot flag	Suggested size
`[SWAP]`	`/dev/sda1`	`82`: Linux swap	否	至少 4 GiB，使用 休眠 时等于内存大小
`/`	`/dev/sda2`	`83`: Linux	是	剩余空间，至少 23 - 32 GiB
N/A	*Unallocated space*2	N/A	N/A	At least 16.5 KiB at the end of the disk

BIOS/GPT 布局示例

挂载点	分区	分区类型 GUID	推荐大小	
None	`/dev/sda1`	`21686148-6449-6E6F-744E-656564454649`: BIOS boot partition 3	1 MiB	
`[SWAP]`	`/dev/sda2`	`0657FD6D-A4AB-43C4-84E5-0933C84B4F4F`: Linux swap	至少 4 GiB，使用 休眠 时等于内存大小	
`/`	`/dev/sda3`	`4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709`: Linux x86-64 root (/)	`2`: Legacy BIOS bootable	23 - 32 GiB

1. The ESP can be mounted to `/efi` if the used boot loader is capable of accessing the file system (and everything above it) on which the kernel and initramfs images are located. See EFI system partition#Typical mount points and the warning in Arch boot process#Boot loader for details.
2. An unpartitioned space of at least 33 512-byte sectors (16.5 KiB) at the end of the disk to allow converting to GPT in the future. The space will be required for the backup GPT header. The recommendation to preserve an unpartitioned space applies to all MBR partitioned disks.
3. A BIOS boot partition is only required when using GRUB for BIOS booting from a GPT disk, it is not needed when using other boot loaders. The partition has nothing to do with `/boot`, and it must not be formatted with a file system or mounted.

分区工具

下面表格可以帮助选择。

警告：请选择与分区表类型相兼容的工具。不兼容的工具可能损坏分区表，以至于损坏数据。

GPT	CLI	TUI	Scripting utility			
fdisk	util-linux包	是	是	fdisk(8)	cfdisk(8)	sfdisk(8)
GPT fdisk	gptfdisk包	否	是	gdisk(8)	cgdisk(8)	sgdisk(8)
Parted	parted包	是	是	parted(8)	否	parted(8)

fdisk/gdisk

请参考 fdisk。

- **fdisk** — Linux 自带的命令行分区工具。

: https://www.kernel.org/ || util-linux包

- **cfdisk** — 使用 ncurses 库编写的具有伪图形界面的命令行分区工具。

: https://www.kernel.org/ || util-linux包

- **sfdisk** — 支持脚本的 fdisk.

: https://www.kernel.org/pub/linux/utils/util-linux/ || util-linux包

- **gdisk** — GPT 版的 fdisk。

: https://www.rodsbooks.com/gdisk/ || gptfdisk包

- **cgdisk** — GPT 版的 cfdisk。

: https://www.rodsbooks.com/gdisk/ || gptfdisk包

- **sgdisk** — Scriptable version of gdisk.

: https://www.rodsbooks.com/gdisk/sgdisk-walkthrough.html || gptfdisk包

GNU Parted

下面工具在 GNU Parted 中有介绍。

- **GNU Parted** — 命令行分区工具。

: https://www.gnu.org/software/parted/parted.html || parted包

- **GParted** — GTK 图形界面的分区工具。

: http://gparted.sourceforge.net/ || gparted包

- **GNOME Disks** — GTK 写的图形分区工具.

: https://wiki.gnome.org/Apps/Disks || gnome-disk-utility包

- **Partitionmanager** — QT 图形界面的分区工具。

: https://sourceforge.net/projects/partitionman/ || partitionmanager包

备份

请参阅 fdisk#Backup and restore partition table 或 gdisk#Backup and restore partition table.

恢复

- **gpart** — 可猜测被破坏的 MBR 分区表内容。用法在gpart(8)。

: https://github.com/baruch/gpart || gpart包

- **GPT fdisk\[损坏的链接：无效的章节\]** — 该工具可从副 GPT 分区表（存储于磁盘末尾）恢复主 GPT 分区表（存储于磁盘开头），或是反过来。

: https://www.rodsbooks.com/gdisk/ || gptfdisk包

- **TestDisk** — 该工具可恢复 MBR 或 GPT 丢失分区。

: https://www.cgsecurity.org/index.html?testdisk.html || testdisk包

分区对齐

恰当的分区对齐有助于提升性能和使用寿命。这是由硬件层面和文件系统层面的每次块) I/O 操作特性决定的。对齐的关键是分区大小（至少）是*块大小*的倍数，*块大小*取决于选用的硬件设备。如果分区没有以*块大小*的整数倍对齐，对齐文件系统就失去意义了，因为从分区的起始偏移开始就是有偏差的。

fdisk/gdisk 和 parted 会自动处理分区对齐。GNU Parted#Check alignment 介绍了如何验证分区是否对齐。

有些硬盘使用高级格式会提供更好的读写效率。

GPT 内核支持

Linux 内核中 `CONFIG_EFI_PARTITION` 选项配置是否启用内核 GPT 支持（即使名称为 EFI PARTITION）。此选项必须内置于内核，不能作为可加载模块。即使 GPT 磁盘只用作数据存储而不作为启动盘，此选项也是必须的。在 Arch \[core\] 仓库中的 linux包 和 linux-lts包 内核默认启用此选项。如果自定内核，设置 `CONFIG_EFI_PARTITION=y` 启用此选项。

疑难解答

Tricking old BIOS into booting from GPT

Some old BIOSes (from before year 2010) attempt to parse the boot sector and refuse to boot it if it does not contain a bootable MBR partition. This is a problem if one wants to use GPT on this disk, because, from the BIOS viewpoint, it contains only one, non-bootable, MBR partition of type `ee` (i.e., the protective MBR partition). One can mark the protective MBR entry as bootable using `fdisk -t mbr /dev/sda`, and it will work on some BIOSes. However, the UEFI specification prohibits the protective MBR partition entry from being bootable, and UEFI-based boards do care about this, even in the legacy boot mode. So, this matters if one wants to create a GPT-based USB flash drive that is supposed to boot both on modern UEFI-based boards and also on old BIOSes that insist on finding a bootable MBR partition. It is not possible to solve this problem using traditional tools such as fdisk or gdisk, but it is possible to create a fake MBR partition entry suitable for both kinds of BIOSes manually as a sequence of bytes.

The command below will overwrite the second MBR partition slot and add a bootable partition there of type 0 (i.e. unused), covering only the first sector of the device. It will not interfere with the GPT or with the first MBR partition entry which normally contains a protective MBR partition.

```
# printf '\200\0\0\0\0\0\0\0\0\0\0\0\001\0\0\0' | dd of=/dev/sda bs=1 seek=462
```

最终结果看起来是这样的:

```
# fdisk -t mbr -l /dev/sda
Disk /dev/sda: 232.9 GiB, 250059350016 bytes, 488397168 sectors
Disk model: ST3250820AS
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

Device     Boot Start       End   Sectors   Size Id Type
/dev/sda1           1 488397167 488397167 232.9G ee GPT
/dev/sda2   *       0         0         1   512B  0 Empty

Partition table entries are not in disk order.
```

启用固件 RAID 时无法找到硬盘

如果在固件配置界面可以看到 SATA 或 NVMe 硬盘，但在 Linux 中找不到（例如 `fdisk -l` 中没有），有可能控制器正处于固件 RAID 模式。

对于 NVMe，journal 会显示如下信息：

```
kernel: ahci 0000:00:17.0: Found 1 remapped NVMe devices.
kernel: ahci 0000:00:17.0: Switch your BIOS from RAID to AHCI mode to use them.
```

解决方案是进入固件配置界面，禁用 NVMe RAID 模式并将 SATA 控制器运行模式从 *RAID* 修改为 *AHCI*。注意，该设置的名称可能会有所不同（例如“Intel Rapid Storage

Technology”，“Intel RST”，“Intel VMD controller”或“VMD”），且可能会是控制器或端口独立的。

警告：在使用 Arch + Windows 双系统的情况下，在更改控制器模式前需要进行些准备，具体请参考完成安装后如何在 Windows 8 和 Windows 10 中启用 AHCI。

注意：有些固件没有单独的 NVMe RAID 模式配置项，并会将 SATA 控制器运行模式应用到 NVMe 控制器上。在这些固件上，将“SATA 运行模式”设为“AHCI”会将 NVMe 控制器设为“不使用固件 RAID，使用原生运行模式”。

参考资料

- Wikipedia:Disk partitioning
- Wikipedia:Binary prefix
- Understanding Disk Drive Terminology
- What is a Master Boot Record (MBR)?
- Rod Smith's page on What's a GPT? and Booting OSes from GPT
- Make the most of large drives with GPT and Linux - IBM Developer Works
- Microsoft's Windows and GPT FAQ
- Partition Alignment (with examples)

虚拟化

虚拟化要回答的只有一个问题：怎样把一台机器切成多台，或者把同一套 Arch 安装流程固化下来反复执行。本章按隔离层级由轻到重铺开：先是 User-mode Linux，以及把 Arch 搬上云与做成自动化模板的 VPS、AWS AMI、Vagrant、Cloud-init，再是控制组、Incus 与 Libvirt，最后是 OVMF 直通 PCI。

必读控制组、Libvirt 与使用 OVMF 直通 PCI：前者定下资源边界，中间是全章中心，最后一节实操最重。Kubernetes 与 Intel GVT-g 场景很窄，读完 Libvirt 再按需回看即可。最容易踩的坑是把控制组当成容器工具——它只限制资源，不管容器生命周期；另一个误解是以为直通 PCI 只需在虚拟机里加一块显卡，其实设置 IOMMU 与隔离 GPU 才是第一步。

6.1　User-mode Linux

难度 入门 | **篇幅** 4.4 KB | **所属** 虚拟化

本节将讲到

- 什么是 user-mode-linux?
- 为什么使用 UML?
- HOWTO
- 运行所需:
- 现在开始

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 这篇文章或章节的内容已经过期。（在 Talk:User-mode Linux# 中讨论）

什么是 user-mode-linux?

User-mode-linux (UML) 是让一个linux作为一个独立进程运行在另一个linux上。 请在查找 uml 的详细安装使用指南。 这篇 wiki 文章是这帖\[失效链接 2020-08-06 ⓘ\] 概要。 如果你有任何意见和建议可以在这回帖。

为什么使用 UML?

UML 是一种在同一时间运行多 (arch-)linux 的安全方式。 每个进程都是独立去其它的， 这非常安全，例如在同一机器上进行多种测试和开发而不互相干扰。如果一些测试进程损坏并不会影响宿主系统或者开发用进程。

HOWTO

运行所需:

- User-mode-linux 内核(安装 linux-usermodeAUR)

- uml_工具, 特别使 tunctl 程序 (安装 uml_utilitiesAUR)
- archbootstrap 脚本. 在这个主题进行讨论.

现在开始

1.) 首先要创建一个独立、大的文件用来安装 arch。 如下命令会创建一个独立的空的 1GB 文件 - 应该足够安装一个基 arch 系统。

```
dd if=/dev/zero of=archRootFs bs=1MB count=1000
```

2.) 当创建完成后格式化。 如下命令将会提示 archRootFs 不是一个 block 设备。 你可以完全忽略或者加入 *\-F* 来使 mke2fs 可以屏蔽提示信息。

```
mke2fs archRootFS
```

3.) 格式化后加载它。 以root用户执行如下命令:

```
mount -o loop archRootFs /mnt
```

4.) 现在开始基本系统安装。您可以用 archbootstrap 脚本， 这会象光盘安装一样，不同的是这个安装从指定目录开始。这是一个真实的基本系统安装 - 因而会花费一点时间!

```
sh archbootstrap /mnt/ ftp://archlinux-mirror
```

5.) 在系统以 user-mode-linux 模式启动之前, 一些 arch 基本系统文件要求定制。 在 */mnt/etc/fstab* 加入:

```
/dev/ubd0 / ext2 defaults 0 0
```

可以通过禁用 hotplugin 来加快启动时间，在 */mnt/etc/rc.conf* 加入:

```
DAEMONS=(syslog-ng !hotplug !pcmcia network netfs crond)
```

6.) 卸载文件系统。**注意:** 如果你在加载的系统中改变了*任何东西* (例如 /mnt) 而它正在*运行*，则很有可能*毁了*它 !

```
umount /mnt
```

7.) 下一步是配置网络。 因此您要创建 tun 设备 (请阅读 uml howto 得到具体的 tun/tap 信息)， 分配一个 IP 地址。 如下命令将会创建 tun/tap 设备并让普通用户能够使用，然后分配 ip 地址。为了安全起见您最好建立一个 uml 用户组并赋予使用网络设备的权限。

```
modprobe tun
tunctl -u users
chown root.users /dev/net/tun
ip addr add 192.168.0.100/24 dev tap0
```

8.) 现在可以启动镜像。为了使用网络，您要在 uml 内核中声明正确的设备值。 (确保普通用户通过运行 uml 命令有足够的权限使用网络！)

```
vmlinux ubd0=archRootFs eth0=tuntap,,,192.168.0.100
```

",,," 意思是:

```
eth0=transport,tuntap device,MAC adress,ip
```

例如:

```
eth0=tuntap,tap0,3f:2a:bb:00:00:00,192.168.3.23
```

祝您开心使用 uml。

6.2 VPS 上的 Arch Linux

难度 进阶 | **篇幅** 6.1 KB | **所属** 虚拟化

本节将讲到

- Arch Linux 官方云映像

翻译状态：

- 本文（或部分内容）译自 Arch Linux on a VPS，最近一次同步于 2025-04-11，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

摘自 Wikipedia:Virtual private server:

: Virtual private server (VPS) is a term used by Internet hosting services to refer to a virtual machine. The term is used for emphasizing that the virtual machine, although running in software on the same physical computer as other customers' virtual machines, is in many respects functionally equivalent to a separate physical computer, is dedicated to the individual customer's needs, has the privacy of a separate physical computer, and can be configured to run server software.

本文主要讨论 Arch Linux 在 VPS 上的应用，并且包括了一些适用于 VPS 的安装与维护指南.

Arch Linux 官方云映像

作为 arch-boxes 项目的一部分，Arch Linux 提供了官方云映像。映像预装了 Cloud-init，应可用于多数云服务供应商。

可以从镜像源的 `images` 目录下找到该映像，具体使用步骤参考下方。

DigitalOcean

网站：Digital Ocean

位置：全球

创建 Arch Linux VM 的步骤：

1. 在镜像源上找到云映像的位置，例如：https://geo.mirror.pkgbuild.com/images/latest/Arch-Linux-x86_64-cloudimg.qcow2
2. 将其导入为自定义映像
3. 从自定义映像创建新 VM
4. 通过 SSH 连接到 VM：`ssh root@*ip*`

Hetzner Cloud

网站：Hetzner Cloud

位置：纽伦堡（德国），法尔肯施泰因（德国），赫尔辛基（芬兰），新加坡，希尔斯伯勒（美国西部），阿什本（美国东部）

创建 Arch Linux VM 的步骤：

1. 使用该用户数据创建新 VM：

```
    #cloud-configvendor_data: {'enabled': false}
```

Hetzner 的 `vendor_data` 会覆盖掉发行版中设定的值，并在将默认用户设定为 `root` 的同时没有设置 `disable_root: false`，使得你无法登陆到机子上。

1. 以恢复模式启动 VM
2. 通过 SSH 连接到 VM，然后从镜像源下载云映像，例如：`curl -O https://geo.mirror.pkgbuild.com/images/latest/Arch-Linux-x86_64-cloudimg.qcow2`
3. 将映像写入到硬盘：`qemu-img convert -f qcow2 -O raw Arch-Linux-x86_64-cloudimg.qcow2 /dev/sda`
4. 重启 VM
5. 通过 SSH 连接到 VM：`ssh arch@*ip*`

Linode

网站：Linode

位置：全球多地

创建 Arch Linux VM 的步骤：

1. 创建一台新 VM，并选择 Arch 作为发行版（如果要使用 Linode 提供的映像，到这一步就行；如果要使用官方映像，继续下列步骤）
2. 以恢复模式启动 VM
3. 通过 Lish 控制台连接到 VM，然后从镜像源下载基础映像，例如：`curl -O https://geo.mirror.pkgbuild.com/images/latest/Arch-Linux-x86_64-basic.qcow2`

4. 安装 qemu-utils包：`apt update && apt install qemu-utils`
5. 将映像写入到硬盘：`qemu-img convert -f qcow2 -O raw Arch-Linux-x86_64-basic.qcow2 /dev/sda`
6. 在 Linode 控制台上，进入到 VM 配置菜单，然后将内核选项修改为 ”Direct Disk“
7. 重启 VM
8. 通过 SSH 连接到 VM：`ssh arch@*ip*`

OVH Eco

网站：Kimsufi OVH Eco

位置：加拿大，法国

基于官方文档的 Arch Linux VM 创建步骤如下：

1. Navigate to the Dedicated Servers section in your OVH management panel, then select the server you want to deploy Arch Linux to.
2. Click the ... button next to "Last operating system (OS) installed by OVHcloud" and choose "Install"
3. Select "Install from custom image"
4. For "Image URL" put `https://geo.mirror.pkgbuild.com/images/latest/Arch-Linux-x86_64-cloudimg.qcow2`
5. For "Image type" select `qcow2`
6. For "Checksum type" select `sha256`
7. For "Image checksum" put the fingerprint value from https://geo.mirror.pkgbuild.com/images/latest/Arch-Linux-x86_64-cloudimg.qcow2.SHA256
8. Enable "ConfigDrive" to enter "Server host name" and your public "SSH key" (both are mandatory for Arch Cloud Init install)
9. Click "Install the system"
10. Wait (it takes a while) for an email from OVH titled "Installation of your image", it will say "Congratulations! Your dedicated server has just been installed! Connect to your server with ssh key provided during your installation."
11. Use `ssh arch@*ip*` to log in.

Proxmox

网站：Proxmox

位置：N/A

创建 Arch Linux VM 的步骤：

1. 创建一台新 VM

2. 在 OS 选择步骤选中 “Do not use any media”
3. 在 VM 创建完成后从 VM 移除刚创建的硬盘
4. 使用 `qm disk import` 将刚下载好的映像连接到 VM，例如：
5. `qm disk import 100 Arch-Linux-x86_64-cloudimg.qcow2 local.`
6. 在 Options 下，编辑启动顺序，添加刚创建的硬盘
7. 添加 cloudinit 盘，在 Cloud-Init 一项中添加你的配置
8. 启动 VM！

6.3 适用于 AWS 的 Arch Linux AMI

难度 进阶 | **篇幅** 6.7 KB | **所属** 虚拟化

本节将讲到

- 开放社区的 Arch AMI
- 构建 Arch AMI

开放社区的 Arch AMI

注意：Arch Linux 目前暂不提供 *A*mazon *M*achine *I*mages 的官方支持。以下 AMI 由社区创建。

AMI

可以在这里找到 Arch Linux AMI：http://arch-ami-list.drzee.net/

AMI 每月构建两次（一号及15号的 2:00am UTC），并在所有默认启用的地区中提供（参见地域列表）。如果需要在目前不支持的地域使用，可以手动将 AMI 复制到目标区域。

这些 AMI 属于 EBS HVM AMI，并提供了两个内核选项：

- **std** - 使用来自默认 Arch 软件源的标准内核，并带有一些在 EC2 上使用所必须的模块。调度器和 I/O 并没有针对云端使用进行优化，性能上可能较低。
- **ec2** - 使用了由 UplinkLabs 从默认 Arch Linux 内核修改而来的，针对 EC2 进行优化的内核，存放于独立的源内：参见 https://git.uplinklabs.net/steven/ec2-packages.git

两个内核都已在多种 EC2 实例上经过测试（t2，t3，t3a，m/r/c5，m/r/c6 以及带有多 GPU 等高级硬件的实例上）并能正常启动。

注意：截止至 2023-06-12，已确认到 UplinkLabs 提供的 EC2 优化内核在使用 XEN 虚拟化管理器的实例（例如 `t2-micro`）上启动时会卡住，原因是 XEN 内核模块默认没有打包进最新的内核里。请确保仅使用新的 Nitro 虚拟化管理器实例 - 参见 Instances build on Nitro。这些实例仍可以使用 EC2 优化内核。

注意：ec2 仓库除 **linux-ec2** 内核外也提供了其它的包。默认情况下，**ec2** 被设置为 `pacman.conf` 中最顶层的镜像，导致 **ec2** 中的包比官方仓库拥有更高的优先级。取决于具体的包版本，这可能会导致一些一致性问题。

未构建使用 LTS 内核的 AMI。

使用 REST API 获取 AMI 列表

可以使用如下 REST API 获取 JSON 格式的可用 AMI 列表：

注意：API 终端已配置为自定义域名，应不会出现更改。

- 获取所有 AMI：`[https://arch-ami-api.drzee.net/](https://arch-ami-api.drzee.net/)`
- 获取各个地域最新 AMI 的清单：`[https://arch-ami-api.drzee.net/latest](https://arch-ami-api.drzee.net/latest)`（与 http://arch-ami-list.drzee.net/ 的内容相同，但格式为 JSON）
- 获取单个地域内的的所有 AMI：`[https://arch-ami-api.drzee.net/region](https://arch-ami-api.drzee.net/region)` - 将 `region` 替换为要查看的地域：例如 `eu-north-1`, `eu-west-1`，`us-east-1` 等等。
- 按照 CPU 架构列出地域中的所有 AMI：`[https://arch-ami-api.drzee.net/region/arch](https://arch-ami-api.drzee.net/region/arch)` - 将 `arch` 替换为 `x86_64`
- 按照 CPU 架构和内核类型列出地域中的所有 AMI：`[https://arch-ami-api.drzee.net/region/arch/type](https://arch-ami-api.drzee.net/region/arch/type)` - 将 `type` 替换为 `std` 或者 `ec2`
- 按照 CPU 架构和内核类型列出地域中的最新 AMI：`[https://arch-ami-api.drzee.net/region/arch/type/latest](https://arch-ami-api.drzee.net/region/arch/type/latest)` - 替换 `region`、`arch` 和 `type`

初次运行

注意：AMI 内置的镜像源清单在镜像构建时生成，使用位于德国的服务器。

启动了 AMI 后，建议/必须执行以下步骤以初始化 pacman 并选择最快的软件源：

```
# pacman-key --init
# pacman-key --populate
# reflector --country "ISO 3166-1 Alpha-2 Country Code" --protocol https,http --score 20 --sort rate --save /etc/pacman.d/mirrorlist
# pacman -Syu
```

Reflector 包已随 AMI 附带。

建议在 `/etc/xdg/reflector/reflector.conf` 配置设置，然后启用定时服务来自动定时更新镜像清单。具体信息请参考 Reflector 文档。

你也可以使用自己的镜像源清单，而不使用 reflector。

构建流程

注意：该部分简单介绍了 AMI 是如何构建的。

整个构建流程位于 AWS 上，并完全自动化进行。

构建流程由 AWS Step Function 控制，并随一个 Amazon EventBridge 定时事件周期运行。

Step function 负责启动构建，在更复杂的功能上结合使用了一系列本地调用及 AWS Lambda 函数来达成。

新的 AMI 构建流程为：使用旧的 AMI 启动一台 EC2 实例作为*工作/构建机器*，该实例上带了一套特殊的构建脚本，使用了 pacstrap 和一些额外步骤来生成镜像。大致步骤在下一节进行介绍。

完成 AMI 构建后，新的 AMI 会在新的 EC2 实例上进行*测试启动*以验证其可以正常启动。成功后，AMI 将会被分发到各个地域，并在一个 DynamoDB 数据库上进行记录。数据库内容可以由 REST API 进行调取。旧 AMI 将从各个地域及 DynamoDB 数据库中删除。

致谢

感谢来自 UplinkLabs 的 **Steven** 协助解析构建流程，并测试了镜像的早期质量。同时感谢 **Mathcom** 的脚本（遗憾的是文章已被移除），帮助加快了构建流程的基础搭建。

你可以发送评论和建议（不保证会被查看）到：**arch-ami 'at' drzee.net**

未来工作

- 在默认 Arch Linux 源中加入官方的 EC2/云平台优化内核
- 在默认 Arch Linux 源中加入 AWS CLI v2（v1 已在源内，但未来可能会停止更新）
- 由于 AWS CLI v2 与 Python 3.12 的兼容问题，其已被移出 Extra 源 - 当前镜像将继续使用 aws-cli v1

构建 Arch AMI

你也可以自己构建 Arch Linux AMI，具体步骤可以参考 。

6.4 Vagrant

难度 进阶 | **篇幅** 10.1 KB | **所属** 虚拟化

本节将讲到

- 安装
- 配置
- 插件
- 配置自动化
- Vagrant 基础镜像

翻译状态：

- 本文（或部分内容）译自 Vagrant，最近一次同步于 2026-07-11，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Vagrant 是一款用于管理和配置虚拟化开发环境的工具。

Vagrant 包含“服务商” (providers) 概念。此概念直接映射到虚拟化引擎及其 API。在所有服务商中，VirtualBox 的普及率最高且支持最为完善。如需使用`libvirt`、`lxc` 或 `vmware` 等其他引擎，需安装相应的插件。

Vagrant 主要通过声明式的 `Vagrantfile` 来定义虚拟机。只需编写单个 `Vagrantfile`，即可定义多台虚拟机

安装

安装 vagrantAUR 软件包。

配置

可以通过环境变量来管理 Vagrant。如需查看完整的选项列表，参见官方文档。

例如，若要更改 Vagrant 存储大文件的路径，需将 `VAGRAN_HOME` 设置为合适的目录（默认路径为 `~/.vagrant.d`）。

插件

Vagrant 采用中间件架构，以提供强大的插件支持。

使用 Vagrant 内置的插件管理器安装插件。如需同时安装多个插件，需在命令后依次指定：

```
$ vagrant plugin install vagrant-vbguest vagrant-share
```

vagrant-libvirt

注意：因 Arch 仓库中最新版 vagrantAUR 包附带的 ruby gem 存在兼容性问题，当前可能导致 libvirt 插件报错，例如：`Vagrant failed to properly resolve required dependencies`

为避免此兼容性问题，可根据官方文档的说明，通过 Podman 或 Docker 使用容器镜像作为替代方案。

注意：此问题现已修复（至少自 vagrant 2.4.5-1 版本起已恢复正常）

此插件为 Vagrant 增加了 libvirt 服务商支持。使用此服务商前，必须先安装并配置好 libvirt 及其相关软件包（如 QEMU）。

安装此插件前，务必确认已安装 base-devel包 并已启动 `libvirtd.service`。随后执行：

```
$ vagrant plugin install vagrant-libvirt
```

插件安装完成后，即可使用 `libvirt` 服务商：

```
$ vagrant up --provider=libvirt
```

若遇到依赖版本不匹配的问题，可设置以下环境变量以忽略 gem 版本限制：

```
export VAGRANT_DISABLE_STRICT_DEPENDENCY_ENFORCEMENT=1
```

若遇到 ruby-libvirt 构建失败的问题，可尝试执行以下命令（根据需要将命令中的 lib 替换为 lib64）：

```
$ CONFIGURE_ARGS='with-ldflags=-L/opt/vagrant/embedded/lib with-libvirt-
include=/usr/include/libvirt with-libvirt-lib=/usr/lib' \
   GEM_HOME=~/.vagrant.d/gems \
   GEM_PATH=$GEM_HOME:/opt/vagrant/embedded/gems \
   PATH=/opt/vagrant/embedded/bin:$PATH \
       vagrant plugin install vagrant-libvirt
```

若需查看更多故障排除指南，请参考。

vagrant-lxc

先从官方仓库安装 lxc包，随后执行：

```
$ vagrant plugin install vagrant-lxc
```

接下来，按照官方仓库的指引配置 lxc。配置完成后，可通过如下 `Vagrantfile` 使用此插件：

```
VAGRANTFILE_API_VERSION = "2"

Vagrant.configure("2") do |config|

    config.vm.define "main" do |config|
        config.vm.box = 'http://bit.ly/vagrant-lxc-wheezy64-2013-10-23'

        config.vm.provider :lxc do |lxc|
            lxc.customize 'cgroup.memory.limit_in_bytes', '512M'
        end

        config.vm.provision :shell do |shell|
            shell.path = 'provision.sh'
        end
    end
end
```

`provision.sh` 文件应为放置在 `Vagrantfile` 同级目录下的 Shell 脚本。在此脚本中编写所需的初始化配置。例如，若要卸载上述 box 中自带的 puppet，可编写如下内容：

```
rm /etc/apt/sources.list.d/puppetlabs.list
apt-get purge -y puppet facter hiera puppet-common puppetlabs-release ruby-rgen
```

配置自动化

通过使用“配置器” (*Provisioners*)，可在执行 vagrant up 的过程中自动安装软件、修改并自动化配置。最常见的配置器为 puppet包。

Vagrant 基础镜像

以下是获取各种用途（开发、测试或生产环境） Vagrant 基础镜像 (vagrant base boxes) 的站点列表：

- 维护良好且持续更新的 Arch Linux x86_64 Vagrant 基础镜像。
- Vagrant Cloud - HashiCorp 的官方 Vagrant 镜像站点。在此可浏览用户提交的镜像或上传自制镜像。单个 Vagrant Cloud 镜像可通过版本控制支持多个服务商。
- vagrantbox.es - Vagrant 基础镜像列表。此项目由 Gareth Rushgrove (@garethr) 发起，托管于 Heroku 并使用 Nginx 服务。其背后故事参见：Vagrantbox.es 的故事。
- Bento - 一个封装了用于构建 Vagrant 基础镜像的 Packer 模板的项目。部分模板已构建并发布至 Vagrant Cloud 上的 bento 组织。
- Puppet Labs Vagrant Boxes - 由 Puppet Labs 团队制作、开箱即用的预制 Vagrant 镜像。
- Vagrant Ubuntu Cloud Images - 自 2013 年 1 月起上线。Canonical 尚未正式推广 **（现已停止生命周期 (EOL) 或标准支持 (EOSS)）**
- Github 上的 packer-arch 项目 - 提供配置文件，以便使用 packer包 基于官方 ISO 镜像构建轻量级 Arch Linux Vagrant 镜像。

故障排除

主机与 Vagrant 虚拟机之间无法 Ping 通（仅主机网络模式）

有时会出现仅主机网络 (host-only networking) 无法正常工作的问题。表现为主机在 vboxnet 接口上没有 IP 地址，主机无法 Ping 通 Vagrant 虚拟机，虚拟机也无法 Ping 通主机。根据 kevin1024 在此讨论串中的提示，安装 net-tools包 软件包即可解决。

无法从 Arch 主机系统通过网络访问虚拟机

自 1.8.4 版本起，Vagrant 似乎仍在通过已弃用的 `route` 命令来配置连接至虚拟机的桥接路由。若未安装 `route`，将因缺少相应路由而导致无法从主机系统访问虚拟机。解决方法如前文所述，安装包含 route 命令的 net-tools包 软件包即可。

执行 "vagrant up" 停滞在 NFS 挂载阶段 (Mounting NFS shared folders...)

安装 nfs-utils包 软件包可能会解决此问题。

同时务必确认连接未被 iptables包 或 nftables包 拦截。若同时使用 libvirt 和 firewalld，须为 libvirt 区域（zone）放行 `nfs`（`nfs3`）、`rpc-bind` 和 `mountd` 服务。

挂载 NFS 共享文件夹报错：mount.nfs: requested NFS version or transport protocol is not supported

安装 nfs-utils包 软件包。编辑 `/etc/nfs.conf` 并取消以下几行的注释，以启用（v3 和）UDP 支持：

```
[nfsd]
vers3=y
udp=y
```

随后Restart `nfs-server.service` 以立即应用更改。

启动网络"default"报错：internal error: Failed to initialize a valid firewall backend

通常原因为未安装防火墙依赖。安装 iptables包 和 dnsmasq包 软件包，随后重启 `libvirtd.service` 单元。

无法通过 SSH 连接至 Vagrant 客户机

检查 BIOS 中是否已经开启虚拟化支持。虽然 Vagrant 提示虚拟机已启动，但这并不意味着虚拟化一切正常。部分 Vagrant 镜像（例如 tantegerda1/archlinux）在未开启 CPU 虚拟化时，仍能运行到 SSH 阶段才报错

报错：Could not get preferred machine for domain

```
Error while creating domain: Error saving the server: Call to virDomainDefineXML
failed: could not get preferred machine for domain
```

检查 BIOS 中是否已开启虚拟化支持。

另见

- Vagrant 官方文档
- zhwp:Vagrant

6.5 Cloud-init

难度 进阶 | **篇幅** 11.3 KB | **所属** 虚拟化

本节将讲到

- 安装
- 配置
- Systemd 整合
- 用法
- 排障

Cloud-init 包含多个用于云实例早期初始化的工具，提供给在云上环境（如 OpenStack，AWS 等）运行的 Arch Linux 映像所用。

安装

安装 cloud-init包 包。

如果你需要使用 growpart 模块，须同时安装 cloud-guest-utils包 包。

配置

该部分仅描述了最基础的配置选项，完整选项可参考 cloud-init 文档。

主配置文件为 `/etc/cloud/cloud.cfg`。需要加载的其它 `*.cfg` 配置文件可放置于 `/etc/cloud/cloud.cfg.d` 目录下，所有配置内容都将被整合到一起。

cloud-init 19.3 及更新版本的默认配置可直接用于多数知名云环境中。简单来讲，该配置文件进行了如下操作：

- 禁用根用户，创建用于登录的 `arch` 用户
- 依靠 cloud-init 的内置数据源检测
- 运行所有在 Arch Linux 上已知可用的模块

根据用例不同，可能需要修改默认配置文件。

配置默认用户

默认配置中包含了如下内容（为简洁起见已省略注释）：

```
users:
   - default
```

该配置指定了要添加到系统中的用户。`default` 这一特殊名称意为 `system_info` 一节中的 `default_user`（参考下列内容），但该部分语法可使用多种参数配置任意用户信息。该列表中的第一个用户将被其它模块作为“默认用户”（例如通过云环境传递的信息配置 SSH 的模块）。

```
disable_root: true
```

禁用根用户通过 SSH 进行访问。你也可以在映像中删除根用户密码：

```
# passwd -d root
```

```
system_info:
   default_user:
     name: arch
     lock_passwd: true
     gecos: arch Cloud User
     groups: [wheel, adm]
     sudo: ["ALL=(ALL) NOPASSWD:ALL"]
     shell: /bin/bash
```

以上为分发包中指定的默认用户信息：

- 默认用户名为 `arch`
- 默认用户为密码锁定状态，即你只可以通过启动时配置的 SSH 密钥登录到实例中
- 默认用户将加入到 `adm` 和 `wheel` 组中
- 默认用户可以无密码使用 `sudo`
- 默认用户的终端环境为 `/bin/bash`

注意，只有当上面的 `users:` 一节包含了特殊用户“default”（或这一节被完全移除）时，这里配置的用户信息才会被应用。

配置数据源

数据源指定了启动时如何拉取实例的元数据，取决于实例运行的具体云环境（OpenStack，AWS，OpenNebula 等）。在这背后，这一操作为对应模块实现的部分公用接口的方法。

默认配置文件不指定任何数据源，即 cloud-init 将自动检测云环境。但是，部分云环境无法被自动检测，或是需要特殊配置才能正常工作。在这一情况下，数据源可用于手动进行指定和配置。详细信息可参考文档中的已知数据源列表一节。

如需指定数据源列表，可在 `/etc/cloud/cloud.cfg` 配置文件中加入如下内容：

```
datasource_list: [ NoCloud, ConfigDrive, OpenNebula, Azure, AltCloud, OVF, MAAS, GCE, OpenStack, CloudSigma, Ec2, CloudStack, None ]
```

这一操作将指示 cloud-init 在下载实例元数据时需要加载哪些模块。另外，可以对单个数据源进行更详细的配置：

```
datasource:
  OpenStack:
    metadata_urls: [ 'http://169.254.169.254:80' ]
    dsmode: net
```

上述配置告知 OpenStack 数据源使用 `http://169.254.169.254:80` URL 来下载元数据，并在网络初始化之后再运行。这两项都是默认行为，可以省略不写。

模块

Cloud-init 默认包含了一系列模块，可按需进行启用或禁用。默认配置启用了所有已知在 Arch Linux 上可用的模块。被忽略的模块包含如分发版或操作系统特定模块等。

实际上，模块被启用不代表它们一定会进行任何操作。它们会检查是否有任何相关配置被传入（如云环境通过数据源传入的配置），之后再进行相关操作。因此，启用所有模块通常可以最大化对不同云环境的兼容性。不论如何，你可以在配置文件中移除确定不需要的模块，以达到提升启动速度等效果。你可以在启动的实例上运行 `cloud-init analyze` 命令来检查各模块占用的启动时长。

部分模组会向 cloud-init 宣称它在哪些分发版上进行过验证。除非在 `cloud.cfg` 中指定的分发版与模组已验证的分发版匹配，就算配置文件中进行了指定，它们也不会运行。如果你想强制绕过该行为，可在配置文件的 `unverified_modules` 部分添加对应模块，例如：

```
unverified_modules: ['ssh-import-id']
```

Systemd 整合

cloud-init 包提供了四个 systemd.service(5)，两个 systemd.target(5) 和一个 systemd.generator(7)，并通过依赖关系配置为按如下顺序激活：

- `cloud-init-generator`：判断可用数据源，并启用或禁用 `cloud-init.target`
- `cloud-init-local.service`：需要文件系统可用。将执行 `cloud-init init --local`
- `cloud-init-main.service`：需要网络可用。将执行 `cloud-init init --all-stages`
- `cloud-config.target`：对应 cloud-config 启动事件“to inform third parties that cloud-config is available”
- `cloud-config.service`：执行 `cloud-init modules --mode=config`
- `cloud-final.service`：执行 `cloud-init modules --mode=final`
- `cloud-init.target`：当所有服务启动后激活

Uplink Labs EC2 映像启用了上述的所有内容，但出于依赖关系这显得有点繁杂。在准备映像时，启用 `cloud-init-main.service` 和 `cloud-final.service` 服务即可。注意，这不代表 cloud-init 一定会被运行 - 这取决于启动早期时 generator 是否成功启用了 `cloud-init.target`。

更多信息可参考 cloud-init 启动阶段文档。

用法

Archiso

**本文或本章节可能需要合并到Install Arch Linux via SSH#Installation on a headless server。**

附注： It gives more details and illustrates Archiso's cloud-init usage on bare metal.（在 Talk:Cloud-init 中讨论）

以下演示了如何使用 QEMU 测试带有 cloud-init 的 Archiso（参考 archlinux/archiso#27）：

首先使用 YAML 格式创建包含用户名及 SSH 公钥信息的 `user-data` 配置文件。你可以使用 archiso Merge Request #117 提供的 `create_cloud-init.sh` 脚本，也可以参考如下示例手动编写，并添加任何 cloud-init 文档中的额外选项。注意，`#cloud-config` 不是 YAML 注释，而是 cloud-init 要求包含的内容。

```
#cloud-config
users:
  - name: vorburger
    ssh_authorized_keys:
      - ssh-rsa (...)
```

我们也可以使用 `meta-data`，但这不是强制需求，可仅放置一空文件（但文件本身需存在）：

```
$ touch meta-data
```

然后使用 libisoburn包 的 *xorriso* 编译（仅）包含 `user-data` 和 `meta-data` 的 `cloud-init.iso` 文件：

```
$ xorriso -as genisoimage -output cloud-init.iso -volid CIDATA -joliet -rock user-data meta-data
```

现在将 `cloud-init.iso` 作为额外驱动器加载到虚拟机中，例如 `qemu-system-x86_64 ... -cdrom cloud-init.iso`，或在使用 archiso 时执行 `run_archiso -c`。

你现在应该能使用 `user-data` 中指定的用户和公钥信息通过 SSH 登录到机子上了。注意，由于启动菜单的延迟时间，可能会需要等待一会才能正常登录（例如 `archlinux*.iso` 的默认启动等待时长为 30s）。对于需要反复测试带有新主机签名的新实例的情况，可以考虑使用 `ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null`。

提示： Cloud-init's nocloud documenation explains an alternative approach, based on using `-smbios type=1,serial=ds=nocloud`. This can be accomplished locally on a host running QEMU and an ad hoc Python webserver as described in the cloud-init tutorial. To boot on bare metal you could just "burn" two separate USB sticks, for `archlinux*.iso` and `cloud-init.iso`, and boot from `archlinux*.iso`. When doing this, cloud-init will run using the configuration on `cloud-init.iso`.

排障

FAQ

Cloud-init FAQ 提供了如何对 cloud-init 进行排障的相关资料，其中包括 log 文件的位置，以及如何在开发过程中重新进行数据源检测和重新运行 cloud-init。

映像未挂载

通常先使用 `lsblk` 和 `mount` 检查是否能检测到 cloud-init 映像（ISO）。

"device /dev/sr0 with label cidata not a valid seed"

`user-data` 和 `meta-data` 需要都存在，可以留空。例如，当 nocloud 数据源发现 ISO 只包含 `user-data` 而没有 `meta-data` 时，就会提示 `datasourcenocloud.py warning device /dev/sr0 with label cidata not a valid seed` 报错。

"unhandled non-multipart \[...\] ssh_authorized_keys"

当 `user-data` 没有以 `#cloud-config` 开头时（虽然 `yamllint` 会提示 `missing starting space in comment`，中间不能出现空格），就会出现 `unhandled non-multipart (text-x-not-multipart) userdata 'b'ssh_authorized_keys` 报错。

参考

- cloud-init 文档

6.6 Incus

难度 进阶 | **篇幅** 12.8 KB | **所属** 虚拟化

本节将讲到

- 安装
- 配置
- 使用
- 技巧和窍门
- 故障排除

翻译状态：

- 本文（或部分内容）译自 Incus，最近一次同步于 2026-02-26，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Incus 是一个用于管理容器（通过 LXC）和虚拟机（通过 QEMU）的管理程序。

它是由原维护者从 LXD 分叉而来的。LXD 维基页面中的文档在很大程度上仍然相关，值得阅读。

安装

安装 incus包 软件包，然后启用 `incus.socket`。

或者，如果您希望实例自动启动, 您可以直接启用/启动 `incus.service`。

要将容器创建委托给用户，请启用/启动 `incus-user.socket` 单元。有关组委派，请参阅 #以非特权用户身份访问 Incus。

从 LXD 迁移

如果您希望从现有的 LXD 安装迁移，应在此时进行，因为迁移工具只会针对空的目标 Incus 服务器运行。

在验证 `lxc info` 和 `incus info` 命令都能正确运行后，阅读上游关于该过程的 文档，然后运行迁移工具：

```
# lxd-to-incus
```

配置

非特权容器

Incus 默认启动 非特权容器（关于区别的解释，请参阅 Linux 容器#特权或非特权）。

为此，您需要为 root 用户设置适当的子用户ID和子组ID范围：与podman 不同，Incus 使用需要以 root 身份运行的守护进程。

验证 `/etc/subuid` 和 `/etc/subgid` 的内容，如果需要，为 root 用户添加一个 至少 1000 万个 UID/GID 的连续范围：

```
# usermod -v 1000000-1000999999 -w 1000000-1000999999 root
```

然后重启 `incus`。

关于替代方法，请参阅 LXD#特权容器。

以非特权用户身份访问 Incus

来自 官方文档：

"对 Incus 的访问由两个组控制：

- `incus` 允许基本的用户访问，没有配置权限，所有操作限制在每个用户的项目内。
- `incus-admin` 允许完全控制 Incus。"

要使普通用户能够启动和操作实例，请将用户添加到 `incus` 组。

要给普通用户完全控制 Incus 的权限而无需使用 sudo，请将用户添加到 `incus-admin`。

警告：添加到 `incus-admin` 组的任何人都是 root 等效用户。更多信息，请参阅 和 。

初始化 Incus 配置

在使用之前，Incus 的配置需要进行初始化：

```
# incus admin init
```

注意：如果没有 root 权限或属于 `incus-admin` 组的用户，此命令无法成功运行。 来自 官方文档：

这将在终端启动一个交互式配置向导，涵盖存储、网络等不同主题。您可以在官方 入门指南 中找到概述。

添加 Web 界面

用于浏览器的前端 lxd-ui 已打补丁以适配 Incus。这些补丁可在 debian 软件包源代码中找到。

要使用此 UI，请安装 incus-ui包 软件包。

然后设置 Web 服务器的地址和端口：

```
$ incus config set core.https_address=127.0.0.1:8443
```

并重启 Incus。

另一个可用的选项是使用以下命令运行 Web 服务器：

```
$ incus webui
```

关于此选项，请参阅 incus webui

使用

命令概述

您可以通过输入以下命令获取所有可用命令的概述：

```
$ incus
```

创建容器

容器基于从镜像服务器或远程 LXD 服务器下载的镜像。

您可以使用以下命令查看已添加服务器的列表：

```
$ incus remote list
```

注意：可以通过使用 `NAME` 列左侧显示的名称来引用镜像服务器，例如以下示例中的 `images`。

您可以使用 `incus image list *server-name*`：列出服务器上的所有镜像，例如：

提示：由于可用镜像数量众多，建议将以下命令的输出通过 less包 等分页器管道传输。

```
$ incus image list images:
```

这将向您显示默认服务器之一上的所有镜像：images.linuxcontainers.org

您还可以通过添加诸如发行版名称之类的术语来搜索镜像：

```
$ incus image list images:debian
```
使用特定服务器的镜像启动容器：

```
$ incus launch servername:imagename
```
例如，从默认服务器的 Ubuntu Noble 镜像创建一个随机命名的容器实例：

```
$ incus launch images:ubuntu/noble
```
要为实例指定名称，只需在其后添加，例如：

```
$ incus launch images:archlinux/current/amd64 arch
```
将创建一个名为 `arch` 的 amd64 Arch 容器。

Arch Linux 容器配置

在非特权容器中禁用 udev

运行非特权 Arch Linux 容器时，systemd 的 udev 服务会在启动和软件包更新期间（特别是当 mkinitcpio包 运行时）生成大量“权限被拒绝”错误。这是因为 udev 尝试写入由主机拥有的 `/sys/devices/*/uevent` 文件。

这些错误是由于 Arch 容器镜像针对非特权操作的默认配置不当造成的 —— 容器运行正常。

根据 systemd 容器接口，`systemd-udevd` 不打算在非特权容器中运行。在容器内屏蔽 udev 服务：

```
# systemctl mask \
  systemd-udevd.service \
  systemd-udevd-control.socket \
  systemd-udevd-kernel.socket \
  systemd-udevd-varlink.socket \
  systemd-udev-trigger.service \
  systemd-udev-load-credentials.service
```
注意：这仅适用于非特权容器。具有设备直通功能的特权容器可能需要 udev。

屏蔽后，重启容器：

```
$ incus restart 容器名称
```

技巧和窍门

在主机上通过名称访问容器

这假设您正在使用默认桥接，其名为 `incusbr0`，并且您正在使用 systemd-resolved。

```
# systemd-resolve --interface incusbr0 --set-domain '~incus' --set-dns $(incus network get incusbr0 ipv4.address | cut -d / -f 1)
```
现在您可以通过名称访问容器：

```
$ ping 容器名称.incus
```

要永久保存此更改，请编辑 `incus.service` systemd 单元，以包含一个 `ExecStartPost` 指令，该指令在启动后运行命令：

{{hc/etc/systemd/system/incus.service.d/access_by_name.conf 2= \[Service\]
ExecStartPost=/bin/sh -c 'systemd-resolve --interface incusbr0 --set-domain "~incus" --set-dns $(incus network get incusbr0 ipv4.address | cut -d / -f 1)' }}

故障排除

启动虚拟机失败

如果您看到错误：

```
Error: Couldn't find one of the required UEFI firmware files: [{code:OVMF_CODE.4MB.fd vars:OVMF_VARS.4MB.ms.fd} {code:OVMF_CODE.2MB.fd vars:OVMF_VARS.2MB.ms.fd} {code:OVMF_CODE.fd vars:OVMF_VARS.ms.fd} {code:OVMF_CODE.fd vars:qemu.nvram}]
```

这是因为 Arch Linux 不分发安全启动签名的 ovmf 固件。要启动虚拟机，您需要暂时禁用安全启动：

```
$ incus launch ubuntu:18.04 test-vm --vm -c security.secureboot=false
```

这也可以通过以下方式添加到默认配置文件中：

```
$ incus profile set default security.secureboot=false
```

网络连接问题

官方文档提供了 如何配置防火墙 的说明。

提示：除了专用防火墙外，多个程序可能会影响单个系统上的 nftables，包括 VPN 和其他虚拟化软件。运行 `nft list ruleset` 以显示所有当前活动的规则。

Incus 创建并管理自己的网络接口，而 NetworkManager 在某些情况下可能会干扰。最好添加一个条目来忽略 `lxcbr*` 设备，使用方法参考 NetworkManager#忽略特定设备。

Docker 的默认配置会破坏 Incus 中的网络。如果需要在同一台机器上运行 Docker 和 Incus，请查阅 官方文档。

Incus 不遵守 Shell 的环境代理变量

示例是 `incus launch` 或 `incus image` 命令在下载镜像时不使用 `*_proxy`/`*_PROXY` 变量的值。

Incus 实现了服务器-客户端模式。这意味着操作是由作为 Incus 服务器 的 `incusd` 执行的 —— 通常在后台运行，除非从交互式 shell 调用。而 `incus` 命令行界面用于与作为 Incus 客户端 的 Incus 服务器通信。

这使得 `incusd`（通常作为服务启动）不继承客户端 shell 的环境变量，而是尊重它被调用时所处环境的变量。 在 Arch Linux 中，Incus 服务器由 systemd 启动。

针对此问题可以有很多变通方法，以下提供一些示例。更多信息，请参阅 Incus 的 问题#574。

临时方案

将 Shell 变量导入 systemd 的环境

首先，导出 `*_PROXY` 变量：

```
$ export ALL_PROXY="socks://代理服务器地址:端口/"
```

将它们导入 systemd 的环境：

```
# systemctl import-environment ALL_PROXY
```

重新启动 `incus.service` 单元。

提示：使用 `systemctl unset-environment` 命令取消设置变量并重启服务。

持久方案

编辑 incus 服务单元

如果您希望 Incus 守护进程始终使用一些静态环境变量（如 `*_proxy`）启动，您可以使用 systemd 的 `environment` 指令。`systemctl set-property` 命令无法操作 `environment` 指令。编辑 `incus.service` 并添加带有适当 变量=值 对的 `environment` 键。例如：

```
/etc/systemd/system/incus.service.d/environment.conf
[Service]
Environment=ALL_PROXY="socks://proxy_server_address:port/"
```

使用 Incus core.proxy 选项

可以通过使用 配置 Incus 服务器的 core.proxy 选项，使 Incus 服务器使用所需的代理。例如：

```
# incus config set core.proxy_http "代理地址:代理端口"
```

注意：`core.proxy` 选项具有全局作用域。即它们立即适用于集群成员。

卸载

停止并禁用服务。然后卸载 incus包 软件包。

如果您想删除所有数据：

```
# rm -r /var/lib/incus
```

如果您使用了任何示例网络配置，也应该删除它们。

另请参阅

- 官方 Incus 主页
- 官方文档

• 入门指南

• 官方论坛

• Incus GitHub 仓库

6.7 控制组

难度 高级 | **篇幅** 20.8 KB | **所属** 虚拟化

本节将讲到

- 安装
- 使用 Systemd 管理控制组
- 使用 libcgroup 和 cgroup 虚拟文件系统管理控制组
- 例子
- 文档

翻译状态：

- 本文（或部分内容）译自 Cgroups，最近一次同步于 2026-01-20，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

控制组 (或者通常被简写为 **cgroups**) 是一项 Linux 内核提供的特性， 用于管理，限制和审核一组进程。Cgroups 可以操作一个进程的集合或者子集（例如，集合中由不同用户启动的进程），这使得 cgroups 和其它类似工具，如 nice(1) 命令和 `/etc/security/limits.conf` 相比更为灵活。

可以使用以下方式使用控制组：

- 在 systemd 单元文件中使用指令来制定服务和切片的限制；
- 通过直接访问 `cgroup` 文件系统；
- 通过 `cgcreate`，`cgexec` 和 `cgclassify` （libcgroup包 和 libcgroup-gitAUR 包的一部分） 等工具;
- 使用“规则应用守护程序”来自动移动特定的用户/组/命令到另一个组中（`/etc/cgrules.conf` 和 `cgconfig.service`）（libcgroup包 和 libcgroup-gitAUR 包的一部分）；以及
- 通过其它软件例如 Linux 容器 (LXC) 虚拟化。

对于 Arch Linux 来说，systemd 是首选的也是最简单的调用和配置 cgroups 的方法，因为它是默认安装的一部分。

安装

确保你已经安装了这些用于自动处理 cgroups 的包中的至少一个：

- systemd包——用于控制 systemd 服务的资源使用。
- libcgroup包，libcgroup-gitAUR——一系列独立的工具（`cgcreate`, `cgclassify`，通过 `cgconfig.conf` 实现可持久化配置）。

使用 Systemd 管理控制组

层级

现有的 cgroup 层级可以通过 `systemctl status` 或者 `systemd-cgls` 命令查看。

```
$ systemctl status
● myarchlinux
    State: running
     Jobs: 0 queued
   Failed: 0 units
    Since: Wed 2019-12-04 22:16:28 UTC; 1 day 4h ago
   CGroup: /
           ├─user.slice
           │ └─user-1000.slice
           │   ├─user@1000.service
           │   │ ├─gnome-shell-wayland.service
           │   │ │ ├─ 1129 /usr/bin/gnome-shell
           │   │ ├─gnome-terminal-server.service
           │   │ │ ├─33519 /usr/lib/gnome-terminal-server
           │   │ │ ├─37298 fish
           │   │ │ └─39239 systemctl status
           │   │ ├─init.scope
           │   │ │ ├─1066 /usr/lib/systemd/systemd --user
           │   │ │ └─1067 (sd-pam)
           │   └─session-2.scope
           │     ├─1053 gdm-session-worker [pam/gdm-password]
           │     ├─1078 /usr/bin/gnome-keyring-daemon --daemonize --login
           │     ├─1082 /usr/lib/gdm-wayland-session /usr/bin/gnome-session
           │     ├─1086 /usr/lib/gnome-session-binary
           │     └─3514 /usr/bin/ssh-agent -D -a /run/user/1000/keyring/.ssh
           ├─init.scope
           │ └─1 /sbin/init
           └─system.slice
             ├─systemd-udevd.service
             │ └─285 /usr/lib/systemd/systemd-udevd
             ├─systemd-journald.service
             │ └─272 /usr/lib/systemd/systemd-journald
             ├─NetworkManager.service
             │ └─656 /usr/bin/NetworkManager --no-daemon
             ├─gdm.service
             │ └─668 /usr/bin/gdm
             └─systemd-logind.service
               └─654 /usr/lib/systemd/systemd-logind
```

找到进程的控制组

一个进程所属的 cgroup 组可以在 `/proc/*PID*/cgroup` 找到。

例如，找到 shell 进程的 cgroup:

```
$ cat /proc/self/cgroup
0::/user.slice/user-1000.slice/session-3.scope
```

查看控制组的系统资源使用情况

`systemd-cgtop` 命令可以用于查看控制组的资源使用情况：

```
$ systemd-cgtop
Control Group                              Tasks   %CPU   Memory  Input/s Output/s
user.slice                                   540  152,8     3.3G        -        -
user.slice/user-1000.slice                   540  152,8     3.3G        -        -
user.slice/u…000.slice/session-1.scope       425  149,5     3.1G        -        -
system.slice                                  37      -   215.6M        -        -
```

自定义控制组

systemd.slice(5) systemd 单元文件可以用于自定义一个 cgroup 配置。单元文件必须放在 systemd 目录下，例如 `/etc/systemd/system/`。可以指定的资源控制选项文档可以在 systemd.resource-control(5) 找到。

这是一个只允许使用 CPU 的 30% 的切片单元例子：

```
/etc/systemd/system/my.slice
[Slice]
CPUQuota=30%
```

记得 daemon-reload 来应用 `.slice` 文件的更改。

在 Systemd 服务中使用

单元文件

资源可以直接在服务定义或者 drop-in 文件中指定：

```
[Service]
MemoryMax=1G
```

这个例子将内存使用限制在 1 GB。

使用切片将单元分组

一个服务可以在指定切片下运行：

```
[Service]
Slice=my.slice
```

以 root 用户的身份使用

`systemd-run` 可以用于在特定切片下运行命令。

```
# systemd-run --slice=my.slice command
```

`--uid=*username*` 选项可以以特定用户的身份运行命令。

```
# systemd-run --uid=username --slice=my.slice command
```

`--shell` 选项可以在指定切片下启动一个 shell。

以非特权用户的身份使用

非特权用户可以在特定条件下将提供给他们的服务分成若干 cgroups。

必须使用 Cgroups v2 才能允许非 root 用户管理 cgroup 资源。

控制器种类

并非所有系统资源都可以由用户控制。

Controller	Can be controlled by user	Options
cpu	需要委派	CPUAccounting, CPUWeight, CPUQuota, AllowedCPUs, AllowedMemoryNodes
io	需要委派	IOWeight, IOReadBandwidthMax, IOWriteBandwidthMax, IODeviceLatencyTargetSec
memory	是	MemoryLow, MemoryHigh, MemoryMax, MemorySwapMax
pids	是	TasksMax
rdma	否	?
eBPF	否	IPAddressDeny, DeviceAllow, DevicePolicy

注意：eBPF 在技术上不是控制器，但使用它实现的 systemd 选项只允许 root 设置。

用户委派

为了让用户控制 CPU 和 IO 资源的使用，需要委派给用户。这可以使用 drop-in 文件来完成。

假如你的 UID 是 1000：

```
/etc/systemd/system/user@1000.service.d/delegate.conf
[Service]
Delegate=cpu cpuset io
```

重启并确认用户会话下的切片有了 CPU 和 IO 控制器。

```
$ cat /sys/fs/cgroup/user.slice/user-1000.slice/cgroup.controllers
cpuset cpu io memory pids
```

用户定义的切片

用户切片文件可以放置在 `~/.config/systemd/user/`。

可以这样在特定切片下运行命令：

```
$ systemd-run --user --slice=my.slice command
```

你也可以在切片里运行你的登陆 shell：

```
$ systemd-run --user --slice=my.slice --shell
```

运行时调整

cgroups 资源可以在运行时使用 `systemctl set-property` 命令进行调整。选项语法与 systemd.resource-control(5) 中相同。

警告：除非传递了 `--runntime` 选项，否则调整将**永久性**生效。系统范围的调整保存在 `/etc/systemd/systemd/system.control/`，用户范围内的保存在 `.config/systemd/user.control/`

注意：并非所有资源更改都会立即生效。例如，更改 TaskMax 只会在生成新进程时生效。

例如，切断所有用户会话的 Internet 访问：

```
$ systemctl set-property user.slice IPAddressDeny=any
```

使用 libcgroup 和 cgroup 虚拟文件系统管理控制组

与使用 systemd 管理相比，cgroup 虚拟文件系统要更更接近底层。"libgroup" 提供了一个库和一些使管理更容易的实用程序，因此我们也将在这里使用它们。

使用更接近底层的方式的原因很简单：systemd 不为 *cgroups 中的“每个接口文件”*提供接口，也不应该期望它在未来的任何时间点提供它们。从它们中读取以获取有关 cgroup 资源使用的其他信息是完全无害的。

在使用非 Systemd 工具之前...

一个 cgroup *应该*只由一组程序来写入，以避免竞态条件，即“单一写入规则”。这不是由内核强制执行的，但遵循此建议可以防止难以调试的问题发生。为了让 systemd 停止管理某些子控制组，请参阅 `Delegate=` 属性。否则，systemd 可能覆盖你设置的内容。

创建专用组

警告：手动创建“专用”组不会使其被 systemd 管理。除了测试用途之外，不应该这样做；在生产环境中，应当使用 systemd 创建具有适当 `Delegate=` 设置的组（要委派所有权限，设置 `Delegate=yes`）。

cgroups 允许你创建“专用”组。您甚至可以授予创建自定义组的权限给常规用户。`groupname` 是 cgroup 名称：

```
# cgcreate -a user -t user -g memory,cpu:groupname
```

注意：自 cgroup v2 以来，“memory,cpu”部分是无用的。所有可用的控制器都将包含在内，不会有任何提醒，因为它们都直接处于根控制组之下。要加快打字速度，请使用 `cpu` 或 `\*`。

现在，您的用户可以调整 `groupname` 组中的所有设置：

```
$ ls -l /sys/fs/cgroup/groupname
total 0
-r--r--r-- 1 root root 0 Jun 20 19:38 cgroup.controllers
-r--r--r-- 1 root root 0 Jun 20 19:38 cgroup.events
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.freeze
--w------- 1 root root 0 Jun 20 19:38 cgroup.kill
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.max.depth
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.max.descendants
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.pressure
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.procs
-r--r--r-- 1 root root 0 Jun 20 19:38 cgroup.stat
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.subtree_control
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.threads
-rw-r--r-- 1 root root 0 Jun 20 19:38 cgroup.type
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.idle
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.max
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.max.burst
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.pressure
-r--r--r-- 1 root root 0 Jun 20 19:38 cpu.stat
-r--r--r-- 1 root root 0 Jun 20 19:38 cpu.stat.local
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.uclamp.max
```

```
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.uclamp.min
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.weight
-rw-r--r-- 1 root root 0 Jun 20 19:38 cpu.weight.nice
-rw-r--r-- 1 root root 0 Jun 20 19:38 io.pressure
-rw-r--r-- 1 root root 0 Jun 20 19:38 irq.pressure
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.current
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.events
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.events.local
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.high
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.low
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.max
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.min
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.numa_stat
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.oom.group
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.peak
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.pressure
--w------- 1 root root 0 Jun 20 19:38 memory.reclaim
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.stat
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.swap.current
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.swap.events
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.swap.high
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.swap.max
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.swap.peak
-r--r--r-- 1 root root 0 Jun 20 19:38 memory.zswap.current
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.zswap.max
-rw-r--r-- 1 root root 0 Jun 20 19:38 memory.zswap.writeback
-r--r--r-- 1 root root 0 Jun 20 19:38 pids.current
-r--r--r-- 1 root root 0 Jun 20 19:38 pids.events
-r--r--r-- 1 root root 0 Jun 20 19:38 pids.events.local
-rw-r--r-- 1 root root 0 Jun 20 19:38 pids.max
-r--r--r-- 1 root root 0 Jun 20 19:38 pids.peak
```

cgroups 是有层次的，此您可以创建尽任意多的子组。如果普通用户想要创建名为 `foo` 的新子组并且分配内存和 CPU 控制器，可以运行：

```
$ cgcreate -g memory,cpu:groupname/foo
```

注意： 资源以自上而下的方式分配，且一个 cgroup 仅当从父组获得某项资源后，才能进一步分配该资源。这意味着，只有在父组将控制器分配给某个子组时，该子组才能使用此控制器进行资源限制或将其分配给其子组。

使用控制组

正如前文所提到的，在任何时候,只“应该”有一个程序写入 cgroup。这不会影响非写入操作，包括在组内生成新进程、将进程移动到另一个组或从 cgroup 文件读取属性。

生成和移动进程

注意：在 cgroup v2 中，将组资源分配给子组的 cgroup 的内部不能有进程，反之亦然。这是减少混乱的有意的限制！

libcgroup 包含一个简单的工具用于在 cgroup 中运行新进程。如果普通用户想在我们之前的 `groupname/foo` 下运行一个 `bash` shell：

```
$ cgexec -g cpu:groupname/foo bash
```

在 shell 内部,我们可以确认它属于哪个 cgroup：

```
$ cat /proc/self/cgroup
0::/groupname/foo
```

这会使用 `/proc/$PID/cgroup`，一个存在于每个进程中的文件。手动写入文件也会导致 cgroup 发生变化。

要将所有 'bash' 命令移动到此组：

```
$ pidof bash
13244 13266
$ cgclassify -g cpu:groupname/foo `pidof bash`
$ cat /proc/13244/cgroup
0::/groupname/foo
```

如果不想使用 `cgclassify`，内核提供了在 cgroups 之间移动进程的另外两种方法。这两个是等价的:

```
$ echo 0::/groupname/foo > /proc/13244/cgroup
$ echo 13244 > /sys/fs/cgroup/groupname/foo/cgroup.procs
```

注意：在最后一个命令中，一次只能写入一个 PID,因此必须对需要移动的每个进程重复此操作。

管理组属性

一个新的子目录 `/sys/fs/cgroup/group/foo` 将在 `groupname/foo` 创建时创建。这些文件可以读取和写入以更改组的属性。（再次提醒，除非委派完成，否则不建议写入这些文件!）

让我们试着看看我们组中所有的进程占用了多少内存：

```
$ cat /sys/fs/cgroup/groupname/foo/memory.current
1536000
```

要限制组中所有进程使用的 RAM （不包括交换空间）,请运行以下命令：

```
$ echo 10000000 > /sys/fs/cgroup/groupname/foo/memory.max
```

要更改此组的 CPU 优先级（默认值为 100）：

```
$ echo 10 > /sys/fs/cgroup/groupname/foo/cpu.weight
```

您可以通过列出 cgroup 目录下的文件来查找更多可以调节的设置或统计信息。

可持久化组配置

注意：systemd ≥ 205 提供了在单元文件中管理 cgroups 的更好方法。以下内容仍然有效，但不应用于新设置。

如果您希望在引导时创建 cgroup，则可以在 `/etc/cgconfig.conf` 中定义它们。这会导致在引导时启动一个服务以配置您的 cgroups。请参阅有关此文件语法的相关手册页；我们将不会说明如何使用真正已弃用的机制。

例子

限制进程使用的内存和 CPU

下面的示例显示一个 *cgroup*，它将指定的命令使用的内存限制为 2GB。

```
$ systemd-run --scope -p MemoryMax=2G --user command
```

下面的示例显示一个命令使用的 CPU 限制为一个 CPU 核心的 20%。

```
$ systemd-run --scope -p CPUQuota="20%" --user command
```

Matlab

在 MATLAB 中进行大计算可能会使您的系统崩溃，因为Matlab没有任何保护以防止占用机器的所有内存或 CPU。以下示例显示一个将 Matlab 使用的资源限制为前 6 个 CPU 内核和 5 GB 内存的 *cgroup*。

Systemd 配置

```
~/.config/systemd/user/matlab.slice
[Slice]
AllowedCPUs=0-5
MemoryHigh=6G
```

像这样启动 Matlab（请务必使用正确的路径）：

```
$ systemd-run --user --slice=matlab.slice /opt/MATLAB/2012b/bin/matlab -desktop
```

文档

- 有关控制器以及特定开关和可调参数含义的信息，请参阅内核文档的 v2 版本（或安装 linux-docs包 包并查看 `/usr/src/linux/Documentation/cgroup` 目录）。
- Linux 手册页：cgroups(7)
- 详细完整的资源管理指南可在 Red Hat Enterprise Linux 文档中找到。

有关命令和配置文件，请参阅相关手册页，例如 cgcreate(1) 或 cgrules.conf(5)。

历史：cgroup v1

在 cgroup 的当前版本 v2 之前，存在一个称为 v1 的早期版本。V1 提供了更灵活的选项，包括非统一层级和线程粒度的管理。现在来看，这是个坏主意（参见 v2 的设计理由）：

- 尽管可以存在多个层级，并且进程可以绑定到多个层级，但一个控制器只能用于一个层级。这使得多个层级本质上毫无意义，通常的设置是将每个控制器绑定到一个层级（例如 `/sys/fs/cgroup/memory/`），然后将每个进程绑定到多个层级。这反过来使得像 `cgcreate` 这样的工具对于同步进程在多个层级中的成员关系变得至关重要。
- 线程粒度的管理导致 cgroup 被滥用作进程管理自身的一种方式。正确的方法是使用系统调用，而不是为了支持这种用法而出现的复杂接口。自我管理需要笨拙的字符串处理，并且本质上容易受到竞态条件的影响。

为了避免进一步的混乱，cgroup v2 在移除功能的基础上制定了两条关键设计规则：

- 如果一个 cgroup 拥有子 cgroup，则它不能附加进程（根 cgroup 除外）。这在 v2 中是强制执行的，有助于实现使下一条规则（单一写入规则）。

- 每个 cgroup 在同一时间应该只有一个进程管理它（单一写入规则）。这条规则并未在任何地方强制执行，但在大多数情况下都应遵守，以避免软件因争相管理同一个组而产生的冲突痛苦。
- 在有 systemd 系统上，根 cgroup 由 systemd 管理，任何非 systemd 进行的更改都违反了这条规则（这并没有被未强制执行，因此只是建议），除非在相关的服务或作用域单元上设置了 `Delegate=` 选项，告知 systemd 不要干预其内部内容。

在 systemd v258 之前，可以使用内核参数 `SYSTEMD_CGROUP_ENABLE_LEGACY_FORCE=1` `systemd.unified_cgroup_hierarchy=0` 来强制使用 cgroup-v1 启动（第一个参数在 v256 中加入以增加使用 cgroup-v1 的难度）。然而，此功能现已被移除。了解这一点仍然有价值，因为有些软件喜欢在不告知您的情况下将 `systemd.unified_cgroup_hierarchy=0` 放入您的内核命令行，导致整个系统崩溃。

另请参阅

- systemd cgroups hacker guide
- cgroupv2: Linux's new unified control group system

6.8 Intel GVT-g

难度 高级 | **篇幅** 22.2 KB | **所属** 虚拟化

本节将讲到

- 准备步骤
- 分配虚GPU
- 获取虚GPU的显示内容
- 显示虚GPU输出
- 故障排查

Intel GVT-g 是一项为Intel GPU (Broadwell之后的架构)提供中介设备直通的技术，可以在不妨碍宿主机正常使用GPU的同时，将GPU虚拟化出多个性能接近原生硬件的虚GPU供多个虚拟机使用。这对于硬件加速虚拟机中的Windows图形是很有用的，对于没有独立显卡可用于全设备直通的笔记本来说尤其如此。(英伟达和AMD的GPU也有类似的功能，但只给Quadro、Radeon Pro这类“专业版”GPU提供。)

Intel还有另一名字相似的技术叫做GVT-d，即使用vfio-pci驱动进行全设备直通。若使用GVT-d，宿主机不能在虚拟化后使用GPU。

准备步骤

Intel GVT-g在Intel Broadwell (5代) 到 Comet Lake (10代)上是受支持的，但在Ice Lake (10代移动处理器)、Rocket Lake (11th台式机处理器)缺少i915驱动的支持。参见 Intel Support Post 以及 Github Issue 了解具体细节。

有关Intel显卡对虚拟化支持可以参考官网
https://www.intel.cn/content/www/cn/zh/support/articles/000093216/graphics.html?wapkw=gvt-g

目前Ice Lake只支持 GVT-d。 对于Xe Architecture (Gen12)GPU，则需要SR-IOV特性。参考QEMU/Guest graphics acceleration#SR-IOV了解具体细节。

首先，你需要创建一个虚GPU，然后将它分配给某个虚拟机。客户机（guest）会将虚GPU视为“正常”的GPU，因此直接安装原生驱动即可，不需要使用特殊驱动（但要保证驱动不过时）。

步骤如下：

- 使用Linux 4.16（或更新） 和 QEMU 2.12（或更新）
- 将 `intel_iommu=on` 添加到 kernel parameters以启用IOMMU
- 启用内核模块: `kvmgt`, `vfio-iommu-type1` 和 `mdev`
- 设置 i915 模块启动参数 `enable_gvt=1` 以启用GPU虚拟化
- 把 `i915.enable_guc=0` 添加到 kernel parameters, 参见 Intel graphics#Enable GuC / HuC firmware loading的警告
- 检索GPU的PCI地址和区域号（下文分别记为`$GVT_PCI` 和 `$GVT_DOM`， as it resides in `/sys/bus/pci/devices`。 可以用 `lspci -D -nn`检视含有`VGA compatible controller: Intel Corporation HD Graphics ...`的那一行，左边的地址即为`$GVT_PCI`，大概形同`0000:00:02.0`
- 为虚GPU生成一个GUID（下文记为`$GVT_GUID`），之后将用于创建和分配虚GPU。虚GPU与GUID一一对应，如果要创建多个虚GPU，那么它们的GUID必须不同。可以使用`uuidgen`生成随机的GUID。

警告：一些用户报告在 5.12 或更新版本的内核上，会导致宿主机和客户机崩溃，参见 https://github.com/intel/gvt-linux/issues/188

创建虚GPU

正确设置上文的内核参数和模块参数，重启后即可创建虚GPU。

虚GPU的类型有多种，区别在于分配给他们的资源量。用以下命令查看可用类型（另外，在对应类型的目录下`cat description`可以查看此类型的细节）：

```
# ls /sys/devices/pci${GVT_DOM}/$GVT_PCI/mdev_supported_types
i915-GVTg_V5_1  # Video memory: <512MB, 2048MB>, resolution: up to 1920x1200
i915-GVTg_V5_2  # Video memory: <256MB, 1024MB>, resolution: up to 1920x1200
i915-GVTg_V5_4  # Video memory: <128MB, 512MB>, resolution: up to 1024x768
i915-GVTg_V5_8  # Video memory: <64MB, 384MB>, resolution: up to 1024x768
```

注意：If the directory is presented but contains nothing, you may try to increase the AGP aperture size in your computer firmware.

选择一个类型（下文记为`$GVT_TYPE`），用下面的命令创建指定类型的虚GPU：

```
# echo "$GVT_GUID" > "/sys/devices/pci${GVT_DOM}/$GVT_PCI/mdev_supported_types/
$GVT_TYPE/create"
```

要创建多个虚GPU，则修改GUID，重复上面的指令多次。创建好的虚GPU将出现在 `/sys/bus/pci/devices/$GVT_PCI/`中。

要删除已经创建的虚GPU，执行下面的指令

```
# echo 1 > /sys/bus/pci/devices/$GVT_PCI/$GVT_GUID/remove
```

libvirt qemu 钩子

libvirt qemu 钩子可在对应虚拟机启动的时自动创建虚GPU、关闭时自动删除虚GPU。按照实际情况替换下面变量的值（`DOMAIN name`是对应虚拟机的domain）。

```
/etc/libvirt/hooks/qemu
#!/bin/sh
GVT_PCI=<GVT_PCI>
GVT_GUID=<GVT_GUID>
MDEV_TYPE=<GVT_TYPE>
DOMAIN=<DOMAIN name>
if [ $# -ge 3 ]; then
    if [ "$1" = "$DOMAIN" ] && [ "$2" = "prepare" ] && [ "$3" = "begin" ]; then
        echo "$GVT_GUID" > "/sys/bus/pci/devices/$GVT_PCI/mdev_supported_types/
$MDEV_TYPE/create"
    elif [ "$1" = "$DOMAIN" ] && [ "$2" = "release" ] && [ "$3" = "end" ]; then
        echo 1 > /sys/bus/pci/devices/$GVT_PCI/$GVT_GUID/remove
    fi
fi
```

记得给予此文件executable权限，变量的值记得使用引号，例如`GVT_PCI='0000:00:02.0'`.

注意：

- 如果使用libvirt用户会话, 此脚本需要使用权限提升命令，如 pkexec(1)或者sudo.
- domain的XML将通过标准输入传入此脚本。可以使用`xmllint`和XPath表达式从标准输入提取`GVT_GUID`，如:

```
GVT_GUID="$(xmllint --xpath 'string(/domain/devices/hostdev[@type="mdev"]
[@display="on"]/source/address/@uuid)' -)"
```

使用systemd service

可以使用systemd service在启动时自动创建虚GPU。优点如下：

- 不依赖于 libvirt
- 可不使用权限提升，因为你可以让systemd直接以root身份执行脚本

- 虽然不是按需创建虚GPU，但虚GPU闲置的时候似乎不会影响宿主机的GPU性能

创建一个 bash 脚本，内容即#准备步骤中提到的步骤。给予其可执行权限。确保脚本有修改权限，因为它将在启动的时候被root用于运行。 接下来创建 systemd 服务，使之启动时执行此脚本，并设置下列属性：

```
After=graphical.target
Type=oneshot
User=root
```

分配虚GPU

如果以普通用户身份运行 `qemu` 或 `libvirtd`，可能会报告 `/dev/vfio/*number*` 不可写，那么需要给予此用户写对应目录的权限（使用 `chmod` 或者 `setfacl` 修改权限）

QEMU CLI

要创建一个带有虚GPU的虚拟机，将此参数添加到QEMU命令中:

```
-device vfio-pci,sysfsdev=/sys/bus/mdev/devices/$GVT_GUID
```

注意：KVM 需要用 `-enable-kvm`启用

libvirt

把这一设备添加对应虚拟机XML的 `*devices*` 元素中

```
$ virsh edit vmname
...
    <hostdev mode='subsystem' type='mdev' managed='no' model='vfio-pci' display='off'>
      <source>
        <address uuid=''GVT_GUID''/>
      </source>
    </hostdev>
...
```

把 `*GVT_GUID*` 替换成你虚GPU的UUID。

获取虚GPU的显示内容

有几种不同的方式可以从虚GPU中获取显示内容。

使用 DMA-BUF 显示

警告：根据这一 issue, 此方法不适用于使用(未修改的)OVMF的UEFI虚拟机。使用基于BIOS的虚拟机(比如SeaBISO) 或者参考下文的补丁或者处理方法。

QEMU CLI

把 `display=on,x-igd-opregion=on` 添加到 `-device vfio-pci` 参数的后面，如:

```
-device vfio-pci,sysfsdev=/sys/bus/mdev/devices/$GVT_GUID,display=on,x-igd-opregion=on
```

libvirt

首先，修改虚拟机的XML，以便于之后使用QEMU相关的元素。修改：

```
$ virsh edit vmname
<domain type='kvm'>
```

至

```
$ virsh edit vmname
<domain xmlns:qemu='http://libvirt.org/schemas/domain/qemu/1.0' type='kvm'>
```

然后把本配置文件添加到<domain>元素的的末尾，比如，把这段文本插入到</domain>标签的上面：

```
$ virsh edit vmname
...
  <qemu:override>
    <qemu:device alias="hostdev0">
      <qemu:frontend>
        ...
        <qemu:property name="x-igd-opregion" type="bool" value="true"/>
      </qemu:frontend>
    </qemu:device>
  </qemu:override>
...
```

分辨率配置

由于 SPICE 服务器直接从透传的 Intel vGPU 读取显示输出，它会完全镜像显卡的原始信号，因此无法通过客户机代理（Guest Agent）进行动态缩放。为了强制指定自定义显示分辨率并防止查看器窗口缩小或出现黑边，您需要在 QEMU 覆盖的设备属性中明确定义目标宽度和高度：

```
$ virsh edit vmname
...
<qemu:override>
  <qemu:device alias="hostdev0">
    <qemu:frontend>
      ...
      <qemu:property name="xres" type="unsigned" value="1920"/>
      <qemu:property name="yres" type="unsigned" value="1080"/>
      ...
    </qemu:frontend>
  </qemu:device>
</qemu:override>
...
```

使用带UEFI/OVMF的DMA-BUF

如上文所说，DMA-BUF显示不能与使用（未修改过的）OVMF的UEFI客户机一同工作，原因在于它不会通过QEMU的非标准fw_cfg接口暴露出所需的ACPI OpRegion。参见this OVMF bug。

根据 GitHub上的讨论，OVMF bug报告提出了几种解决方案。可以

- 为OVMF打补丁 (细节见此) 以添加针对Intel的特殊行为 (最直接，但和上游不一致);
- 为宿主机的内核打补丁 (细节见此) 以自动为虚GPU提供一个可选的ROM;

- 从内核补丁中提取OpROM(来源) 供QEMU重载。不需要打补丁。

在此选择最后一种方法。

注意： if the link *and* the archive go down, the OpROM can be extracted from the kernel patch by hand.

下载 `vbios_gvt_uefi.rom` 并将其置与某处。(本例中为`/`)。

libvirt

然后编辑虚拟机的XML定义，把下面这段配置添加到先前创建的`qemu:commandline`元素中。

```
$ virsh edit vmname
...
    <qemu:property name="romfile" type="string" value="/vbios_gvt_uefi.rom"/>
...
```

启用RAMFB显示 (可选)

本操作是上文的DMA-BUF配置的补充，用于显示虚拟机Intel驱动载入前的显示画面（如POST，固件界面，客户机初始化）

QEMU CLI

把`ramfb=on,driver=vfio-pci-nohotplug`添加到`-device vfio-pci`参数的末尾，如：

```
-device vfio-pci,sysfsdev=/sys/bus/mdev/devices/$GVT_GUID,display=on,x-igd-opregion=on,ramfb=on,driver=vfio-pci-nohotplug
```

libvirt

首先，参照这一小节修改虚拟机的XML定义。

然后把下面的配置添加到`<domain>`元素中，即插入到`</domain>`标签前面：

```
$ virsh edit vmname
...
  <qemu:override>
    <qemu:device alias="hostdev0">
      <qemu:frontend>
        ...
        <qemu:property name="driver" type="string" value="vfio-pci-nohotplug"/>
        <qemu:property name="ramfb" type="bool" value="true"/>
      </qemu:frontend>
    </qemu:device>
  </qemu:override>
...
```

显示虚GPU输出

由于spice-gtk相关的问题，不同EGL实现的SPICE客户端的配置方法不同。

使用QEMU GTK显示器输出

在性能较弱的CPU上，本方法的刷新率较高、显示延迟较小，至少对于Windows虚拟机来说是如此。并且相比于Looking Glass，本方法的CPU负载较小。代价是得放弃一些SPICE GPU特性，如:

- 共享剪贴板
- 自动 USB 重定向 (需要在启动虚拟机前手动分配USB)
- 鼠标指针自由进出虚拟机
- 与virt-manager的显示器输入整合 (会在另一个窗口里显示)

只有在虚拟机加载了正确的Intel GPU驱动后才会开始输入显示内容（通常是登录界面）。这意味着：

- 最好预先安装好正确的Intel GPU驱动。安装前，可以暂时使用另一种虚拟显示器适配器与Intel vGPU一起工作(如 \-vga std 或者 \-std-vga（针对libvirt)，安装后移除std视频适配器.
- 无法看到系统的启动过程。如果系统在登录前崩溃，只能暂用另一种虚拟显示器适配器以排查错误。
- 要进入BIOS，得启用RAMFB显示).

提示：在QEMU GTK中, `Ctrl+Alt+G` 可以捕获或释放鼠标指针，`Ctrl+Alt+F`可以在全屏模式和窗口模式间切换。

QEMU CLI

把`-display gtk,gl=on`添加到命令后面。QEMU VGA适配器可以通过添加`-vga none`禁用。或者也可以同时用两个虚拟显示屏，只不过连接到QEMU VGA适配器的那个是空白的。

libvirt

- 确保上面添加的`<hostdev>`设备把`display`属性设为`'off'`。
- 确保已经把 `xmlns:qemu='http://libvirt.org/schemas/domain/qemu/1.0'`添加到`domain`(步骤使用DMA-BUF显示器)。
- 移除所有`<graphics>`和`<video>`设备。

QEMU GTK显示窗口需要你指定运行OpenGL的显示输出。如果使用笔记本电脑，则先把所有外接显示器断开，确保笔记本电脑屏幕是唯一的显示器。使用这行的命令获取显示器编号`echo $DISPLAY`, 形如`:0`。获取编号后即可重连外接显示器。把刚才获取的编号插入到下面`env name='DISPLAY'`的这行中。

- 添加下面的QEMU命令行参数

```
$ virsh edit vmname
...
  <qemu:commandline>
    <qemu:arg value="-display"/>
    <qemu:arg value="gtk,gl=on,zoom-to-fit=off"/>
    <qemu:env name="DISPLAY" value=":0"/>
  </qemu:commandline>
  <qemu:override>
    <qemu:device alias="hostdev0">
      <qemu:frontend>
        <qemu:property name="display" type="string" value="on"/>
        ...
      </qemu:frontend>
    </qemu:device>
  </qemu:override>
...
```

缩放

窗口模式中，`-display gtk,gl=on,zoom-to-fit=off`使GTK显示窗口大小和虚拟机的屏幕的分辨率一致，保证像素纵横比是1:1。不启用这个参数（或缺省）会使虚拟机的显示匹配窗口的大小，不能保持像素纵横比为1:1,这种缩放不太好看。

在全屏模式中，缩放自动启用。在修改客户机的分辨率的时，只有降低分辨率会更新缩放，如果虚拟机的分辨率调得比宿主机要高，则需手动退出重进全屏模式。

GTK显示产生的CPU负载

`gl=es`可能可以降低CPU负载，但2021年11月过后。`gl=on`似乎更有优势。

使用MESA EGL实现的SPICE输出

QEMU CLI

把`-display spice-app,gl=on`添加到命令行。须安装`virt-viewer`。

libvirt

- 确保上面添加的`<hostdev>`设备的`display`属性设为`'on'`。
- 移除所有的{ic|<graphics>}}和`<video>`设备。
- 添加下面的设备：

```
$ virsh edit vmname
...
    <graphics type='spice'>
      <listen type='none'/>
      <gl enable='yes'/>
    </graphics>
    <video>
      <model type='none'/>
    </video>
...
```

`gl`标签中的可选属性`rendernode`可以用于指定渲染器，如：

```
<gl enable='yes' rendernode='/dev/dri/by-path/pci-0000:00:02.0-render'/>
```

使用NVIDIA EGL实现的SPICE或VNC输出

libvirt

1. 确保上面添加的<hostdev> 设备的`display`属性设为`'on'`。
2. 移除所有`<graphics>` and `<video>`设备。
3. 添加下面的设备：

```
$ virsh edit vmname
...
    <graphics type='spice' autoport='yes'>
      <listen type='address'/>
    </graphics>
    <graphics type='egl-headless'/>
    <video>
      <model type='none'/>
    </video>
...
```

要使用VNC，则须将`<graphics type='spice'>`的`type`属性改为`'vnc'`。

`<graphics type='egl-headless'>`标签中的`<gl>`可选属性可以用来指定渲染器（由于前面提到的bug，不要把这一可选属性添加到`spice`图形中）。例如：

```
<graphics type='egl-headless'>
  <gl rendernode='/dev/dri/by-path/pci-0000:00:02.0-render'/>
</graphics>
```

禁用所有输出

如果禁用了所有输出，那么只能使用RDP、VNC、Looking Glass等软件获取显示内容。参见PCI passthrough via OVMF#Using Looking Glass to stream guest screen to the host。

QEMU CLI

在`-device vfio-pci`参数中，将`ramfb=on`改为`display=off`。添加`-vga none`以禁用QEMU VGA适配器。

libvirt

要确保没有加载任何模拟的GPU，可以编辑虚拟机的配置：

1. 移除所有`<graphics>`设备。
2. 把`<video>`设备的类型都改为`'none'`。
3. 确保上面添加的`<hostdev>`设备的`display`属性设为`'off'`。

故障排查

mdev_supported_types目录缺失

如果你按步骤操作，在添加`i915.enable_gvt=1`内核参数后仍然找不到`/sys/bus/pci/devices/0000:02:00.0/mdev_supported_types`目录，请再次检查`kvmgt`模块是否已载入。

然后检查你的硬件是否支持，检视dmesg的输出里是否有这条信息：

```
# dmesg | grep -i gvt
[    4.227468] [drm] Unsupported device. GVT-g is disabled
```

如果都没有问题，检查上游是否有支持计划。如，对于"Coffee Lake" (CFL)平台的支持可以参见https://github.com/intel/gvt-linux/issues/53

Windows提示内存损坏错误（bad memory error）

如果Windows虚拟机由于内存损坏错误卡死，检视宿主机*dmesg*的输出以获取更多细节。如果内核日志中有类似超出内存限制（rlimit memory exceeded）的内容，则可能需要增加Linux分配给QEMU的内存限制。若用户在kvm组中，把下面的内容添加到/etc/security/limits.d/42-intel-gvtg.conf然后重启。

```
# qemu kvm, need high memlock to allocate memory for vga-passthrough
@kvm - memlock 8388608
```

同时使用Intel GVT-G和PRIME render offload

在宿主机上同时使用Intel GVT-G和NVIDIA的PRIME render offload会导致客户机出现一些问题。建议使用bbswitch关闭独立显卡或者与Bumblebee、nvidia-xrun或者optimus-manager一同使用。

无显示器

如果虚拟机使用RAMFB显示器并且没有输出任何显示内容，尝试增加以下选项到<qemu:commandline>标签：

```
$ virsh edit vmname
...
  <qemu:commandline>
    <qemu:arg value="-set"/>
    <qemu:arg value="device.hostdev0.display=on"/>
  </qemu:commandline>
...
```

花屏

如果鼠标移入后虚拟机屏幕花屏，下面的方法可能有效

首先，按照#libvirt 2修改虚拟机的XML定义。

然后，把下面的内容插入到</domain>标签的上面。如果<qemu:commandline>标签已经存在，就直接插入到其中去：

```
$ virsh edit vmname
...
  <qemu:commandline>
    <qemu:env name="MESA_LOADER_DRIVER_OVERRIDE" value="i965"/>
  </qemu:commandline>
...
```

宿主机在挂起时卡死

创建GVT-g虚GPU后，宿主机可能在挂起时卡死。参见github以追踪此bug。

一个可行的解决方法是，在挂起前将GVT-g虚GPU移除，唤醒后才重新创建。你可以安装gvtg_vgpu-gitAUR自动化这个过程。

修改虚GPU的显示分辨率

虚GPU默认使用其支持的最大分辨率。无论虚拟机设置了多大的分辨率，所显示的内容都会被缩放到虚GPU的分辨率，造成显示效果不佳。

要真正改变显示分辨率，将下面的内容添加到XML的`<qemu:commandline>`元素中：

```
$ virsh edit vmname
...
    <qemu:arg value='-set'/>
    <qemu:arg value='device.hostdev0.xres=1440'/>
    <qemu:arg value='-set'/>
    <qemu:arg value='device.hostdev0.yres=900'/>
...
```

参见

- Official GVT-g Setup Guide
- Official GVT-g DMA-BUF User Guide.
- Running Windows via QEMU/KVM and Intel GVT-g
- Blog post about using GVT\[失效链接 2022-09-18 ⓘ\]

6.9 Kubernetes

难度 高级 | **篇幅** 26.0 KB | **所属** 虚拟化

本节将讲到

- 安装
- 配置
- 创建集群
- 加入集群
- 提示与技巧

Kubernetes (简称k8s)是一套用于自动部署，扩展以及管理容器化应用的开源系统。

k8s集群由一个控制平面组件和一个或多个节点组件构成(每个组件代表一台或多台运行容器运行时和 `kubelet.service` 服务的主机机器)。安装 Kubernetes 有两种选择：一种是更为正统的Kubernetes安装方式（此处所描述的方式），另一种则是使用 k3s，kind 或者 minikube包 进行本地安装。

安装

配置一个 Kubernetes 集群的方法有很多，本文将主要介绍如何使用 kubeadm包 进行配置。

部署工具

kubeadm

如果要使用 *kubeadm* 来引导 Kubernetes 集群，请在每一个节点上安装 kubeadm包 和 kubelet包。

手动配置

如你想要手动创建一个 Kubernetes 集群，请安装 etcdAUR，并安装软件包组 kubernetes-control-plane包组 (在控制平面节点) 或 kubernetes-node包组 (在 worker 节点)。

集群管理

要管理一个 Kubernetes 集群，请在控制平面节点和要与集群交互的主机上安装 kubectl包。

容器运行时

控制平面节点和常规 worker 节点都需要一个容器运行时以运行 `kubelet` 实例。 安装 containerd包 或 cri-o包 来提供一个运行时。

containerd 运行时

有两种方式可以安装 *containerd*:

1. 安装 containerd包 软件包。
2. 安装非特权模式(rootless)*containerd*，你需要安装 nerdctl-full-binAUR，其是一个完整的 nerdctl 包（containerd，CNI 插件，和 RootlessKit）。随后你需要运行 `containerd-rootless-setuptool.sh install` 以安装非特权模式 *containerd*。

请注意，由于 Arch Linux 使用 systemd 作为 init 系统，你需要在部署控制平面之前选择 systemd cgroup 驱动。

(可选) 包管理器

helm包 是一个用于管理预配置的 Kubernetes 资源的工具，这可能会有所帮助。

配置

集群中的所有节点（控制平面和工作节点）都需要运行一个 `kubelet.service` 实例。

提示：在启动 `kubelet.service` 或使用 `kubeadm` 之前，请仔细阅读以下小节。

所有提供的 systemd 服务都允许通过环境文件进行命令行参数覆盖：

- kubelet.service: /etc/kubernetes/kubelet.env
- kube-apiserver.service: /etc/kubernetes/kube-apiserver.env
- kube-controller-manager.service: /etc/kubernetes/kube-controller-manager.env
- kube-proxy.service: /etc/kubernetes/kube-proxy.env
- kube-scheduler.service: /etc/kubernetes/kube-scheduler.env

禁用交换分区

Kubernetes 目前不支持在系统上启用交换分区。详情请参阅 KEP-2400: 节点系统交换支持。

有关如何禁用交换分区的说明，请参阅 Swap#关闭交换分区。

为 containerd 选择 cgroup 驱动

如要在使用 `runc` 时，同时也在 `/etc/containerd/config.toml` 中使用 `systemd` cgroup 驱动，请设置：

```
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc]
  ...
  [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]
    SystemdCgroup = true
```

如果 `/etc/containerd/config.toml` 不存在，可以使用以下命令生成默认配置：

```
# mkdir -p /etc/containerd/
# containerd config default > /etc/containerd/config.toml
```

请记住重启 `containerd.service` 以使更改生效。

有关是否保留 `cgroupfs` 驱动或使用 `systemd` cgroup 驱动的深入讨论，请参阅官方文档。

选择容器运行时接口 (CRI)

在使用 `kubelet.service` 之前，必须配置并启动容器运行时。

你需要在创建或加入集群时，将带有容器运行时接口端点的标志 `--cri-socket` 传递给 `kubeadm init` 或 `kubeadm join`。

例如，如果你选择 containerd包 作为 CRI 运行时，则在创建时附带上`--cri-socket`标志：

```
kubeadm init --cri-socket /run/containerd/containerd.sock
```

Containerd

在 Kubernetes 1.27.4 版本之前，当使用 containerd包 作为容器运行时，需要向 `kubeadm init` 或 `kubeadm join` 提供其 CRI 端点。为此，请将其 `--cri-socket` 标志指定为 `/run/containerd/containerd.sock`。

```
kubeadm join --cri-socket=/run/containerd/containerd.sock
```

在 Kubernetes 1.27.4 版本之后，kubeadm 将自动检测此 CRI，只有在安装了多个 CRI 时才需要 `--cri-socket` 标志。

CRI-O

当使用 CRI-O 作为容器运行时，需要向 `kubeadm init` 或 `kubeadm join` 提供其 CRI 端点：`--cri-socket='unix:///run/crio/crio.sock'`

选择集群网络参数

选择 Pod CIDR 范围

必须为相应的容器运行时配置集群的网络设置。这可以使用 cni-plugins包 来完成。

Pod CIDR 地址指的是分配给 Kubernetes 集群中 Pod 的 IP 地址范围。当 Pod 被调度到集群中的节点上运行时，它们会从此 CIDR 范围中分配 IP 地址。

Pod CIDR 范围在部署 Kubernetes 集群时指定，并且仅限于集群网络内。它不应与集群中使用的其他 IP 范围（例如如*service CIDR 范围*）重叠。

你将向 `kubeadm init` 或 `kubeadm join` 传递 `--pod-network-cidr` 标志，并指定虚拟网络的 CIDR 值，以创建或加入集群。

例如：

```
kubeadm init --pod-network-cidr='10.85.0.0/16'
```

会将你的 Kubernetes Pod CIDR 范围设置为 `10.85.0.0/16`。

（可选）选择 API 服务器广播地址

如果你的控制平面节点位于多个子网中（例如，你可能安装了 Tailscale tailnet），在使用 kubeadm init 初始化 Kubernetes 主节点时，你可以使用 `--apiserver-advertise-address` 标志指定 API 服务器将广播的 IP 地址。注意，此 IP 地址应可被集群中的所有节点访问。

（可选）选择其他的节点网络代理提供者

节点代理提供者（如 `kube-proxy`）是在集群中每个节点上运行的网络代理，用于维护节点上的网络规则，以允许从集群内部或外部的网络会话与你的 Pod 进行网络通信。

默认情况下，kubeadm包 选择 `kube-proxy` 作为在集群中每个节点上运行的节点代理。

容器网络接口 (CNI) 插件（如 Cilium）提供了 kube-proxy 的完整替代方案。

如果你想使用 Cilium 的节点网络代理实现以充分利用 Cilium 的网络策略功能，你应向 `kubeadm init` 传递 `--skip-phases=addon/kube-proxy` 标志，以跳过 kube-proxy 的安装。

Cilium 将在其安装过程中安装完整的替代方案。详情请参阅 。

创建集群

在使用 `kubeadm` 创建新的 Kubernetes 集群之前，请先启动并启用 `kubelet.service`。

注意：`kubelet.service` 会失败（但会重启），直到为其提供配置。

不使用配置文件的 kubeadm

在使用 `kubeadm` 创建新的 Kubernetes 集群时，必须先创建一个控制平面，然后其他工作节点才能加入。

提示：

- 如果集群以后要转换为 高可用性 集群（堆叠式 etcd 拓扑），则需要在 `kubeadm init` 时提供 `--control-plane-endpoint=*<IP 或域名>*`（无法事后进行此操作！）。
- 可以使用 配置文件 来代替一组参数进行 `kubeadm init`。

初始化控制平面

要初始化控制平面，你需要向 `kubeadm init` 传递以下必要的标志。

如果成功运行，`kubeadm init` 将在 `/etc/kubernetes/` 和 `/var/lib/kubelet/` 下生成 `kubelet` 和各种控制平面组件的配置。

最后，它将输出可以复制并粘贴的命令，用于设置 kubectl包 并使工作节点加入集群（基于有效期为 24 小时的令牌）。

要将 `kubectl` 与刚创建的控制平面节点一起使用，请设置配置（以 root 或普通用户身份）：

```
$ mkdir -p $HOME/.kube
# cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
# chown $(id -u):$(id -g) $HOME/.kube/config
```

安装 CNI 插件（Pod 网络插件）

注意：你必须部署一个基于 **Container Network Interface (CNI)** 的 Pod 网络插件，以便你的 Pod 可以相互通信。**集群 DNS (CoreDNS)** 在网络安装之前不会启动。

Pod 网络插件（CNI 插件）以不同的方式实现 Kubernetes 网络模型，从简单的解决方案如 flannel 到更复杂的解决方案如 calico。有关更多选项，请参阅此列表。

一个日益被采用的高级 CNI 插件是 cilium，它通过 eBPF 实现了令人印象深刻的性能。要安装 `cilium` 作为 CNI 插件，请使用 cilium-cli包：

```
# cilium-cli install
```

这将创建 `/opt/cni/bin/cilium-cni` 插件、配置文件 `/etc/cni/net.d/05-cilium.conflist`，并在 Kubernetes 集群上部署两个 Pod。

注意：如果你使用 CRI-O，请确保已启用 `/opt/cni/bin/` 插件目录。请参阅 CRI-O#Plugin directories。

使用配置文件的 kubeadm

你很可能会发现，创建控制平面需要多次尝试才能找到适合你特定设置的最佳配置。为了使此过程更容易（并且使 kubeadm包 的过程更具可重复性），你可以使用配置文件运行初始化步骤。

提示：计划使用两个配置文件，一个用于初始化，一个用于重置，以便更快地测试配置。

创建初始化配置文件

你可以在任何地方创建此文件，但在此示例中我们将使用 `/etc/kubeadm`。

```
# mkdir -pv /etc/kubeadm
# cd /etc/kubeadm
# kubeadm config print init-defaults > init.yaml
```

这将生成以下文件。

```
/etc/kubeadm/init.yaml
apiVersion: kubeadm.k8s.io/v1beta3
bootstrapTokens:
- groups:
  - system:bootstrappers:kubeadm:default-node-token
  token: abcdef.0123456789abcdef
  ttl: 24h0m0s
  usages:
  - signing
  - authentication
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 1.2.3.4
  bindPort: 6443
nodeRegistration:
  criSocket: unix:///var/run/containerd/containerd.sock
  imagePullPolicy: IfNotPresent
  name: node
  taints: null
---
apiServer:
  timeoutForControlPlane: 4m0s
```

```
apiVersion: kubeadm.k8s.io/v1beta3
certificatesDir: /etc/kubernetes/pki
clusterName: kubernetes
controllerManager: {}
dns: {}
etcd:
  local:
    dataDir: /var/lib/etcd
imageRepository: registry.k8s.io
kind: ClusterConfiguration
kubernetesVersion: 1.29.0
networking:
  dnsDomain: cluster.local
  serviceSubnet: 10.96.0.0/12
scheduler: {}
```

大多数默认设置应该可以工作，但你需要更新其中的一些设置。

引导令牌

使用 `kubeadm token generate` 创建一个令牌，并在配置中使用它代替 `token: abcdef.0123456789abcdef`。

公告地址

`advertiseAddress: 1.2.3.4` 应该是正在初始化的控制平面的网络接口的 IPv4 地址，可能是 `192.168.0.0/16` 子网中的某个地址。

节点名称

默认节点名称可以保留为 `node` 并添加到本地 DNS 服务器或 hosts 文件中，或者你可以将其更改为本地网络上可路由的地址。它应该是一个 DNS 兼容的主机名，例如 `kcp01.example.com`。这将允许你的控制平面在加入其他节点时在本地网络上被发现。

初始化集群

完成所有这些更改后，我们可以初始化我们的集群。

```
# kubeadm init --config /etc/kubeadm/init.yaml
```

这将产生大量输出，输出提供了有关如何将节点加入集群、更新 kubeconfig 以与新集群交互以及其他任务的说明。

使用 calico 进行 CNI 配置

在开始添加节点和运行工作负载之前，你最后需要的东西是 正确配置的 CNI。此示例将使用 calico 来实现这一点。

```
# cd /etc/cni/net.d
# curl
https://raw.githubusercontent.com/projectcalico/calico/v3.27.2/manifests/calico.yaml -O
# kubectl create -f calico.yaml
```

如果此操作成功完成，你就可以开始添加节点并在集群上运行工作负载啦。

创建重置配置文件

为防止 kubeadm包 第一次初始化不成功，你还可以创建一个用于重置命令的配置文件：

```
# kubeadm config print reset-defaults > /etc/kubeadm/reset.yaml
```

这将创建以下文件：

```
/etc/kubeadm/reset.yaml
apiVersion: kubeadm.k8s.io/v1beta4
certificatesDir: /etc/kubernetes/pki
criSocket: unix:///var/run/containerd/containerd.sock
kind: ResetConfiguration
```

重置集群

要将集群重置为零，请运行以下命令：

```
# kubeadm reset --config /etc/kubeadm/reset.yaml
```

可以根据需要多次执行此操作，以确定集群的理想配置。

创建加入配置文件

很可能在初始化集群后，你可以使用 init 命令输出中列出的命令加入任何节点，但如果你遇到问题，在要加入的节点上准备一个加入配置文件将会很有帮助。你可以在控制平面上创建此文件，或者在要加入集群的节点上运行命令，我们假设你执行了后者。

```
# kubeadm config print join-defaults > /etc/kubeadm/join.yaml
```

这将创建以下文件。

```
/etc/kubeadm/join.yaml
apiVersion: kubeadm.k8s.io/v1beta3
caCertPath: /etc/kubernetes/pki/ca.crt
discovery:
  bootstrapToken:
    apiServerEndpoint: kcp01.example.com:6443
    token: abcdef.0123456789abcdef
    unsafeSkipCAVerification: true
  timeout: 5m0s
  tlsBootstrapToken: abcdef.0123456789abcdef
kind: JoinConfiguration
nodeRegistration:
  criSocket: unix:///var/run/containerd/containerd.sock
  imagePullPolicy: IfNotPresent
  name: node01.example.com
  taints: null
```

注意：你需要为此配置文件创建两个不同的令牌，第一个用于 `discovery.bootstrapToken.token`，第二个用于 `discovery.tlsBootstrapToken` 属性。

加入集群

使用在#创建集群中生成的令牌信息，可以通过命令 `kubeadm join` 使另一台机器作为工作节点加入集群。

请记住，你还需要通过将标志 <SOCKET> 传递给命令 kubeadm join 来为工作节点选择容器运行时接口)。

例如：

```
# kubeadm join <api-server-ip>:<port> --token <token> --discovery-token-ca-cert-hash sha256:<hash> --node-name=<name_of_the_node> --cri-socket=<SOCKET>
```

要生成新的引导令牌，可以使用以下命令：

```
kubeadm token create --print-join-command
```

如果你使用 Cilium 并且发现工作节点仍然处于 NotReady 状态，请使用以下命令检查工作节点的状态：

```
kubectl describe node <node-id> --namespace=kube-system
```

如果你发现以下条件状态：

```
Type                 Status       Reason
----                 ------       ------
NetworkUnavailable   False        CiliumIsUp
Ready                False        KubeletNotReady container runtime network not ready: NetworkReady=false reason:NetworkPluginNotReady message:Network plugin returns error: cni plugin not initialized
```

请在工作节点上重启 containerd.service 和 kubelet.service。

提示与技巧

拆除集群

当需要从头开始重建集群时，可使用 kubectl包 按官方文档拆除集群的步骤操作：

```
kubectl drain <节点名称> --delete-local-data --force --ignore-daemonsets
```

此处 <节点名称> 是需要清空并重置的节点名称，可通过 kubectl get node -A 列出所有节点。

随后重置该节点：

```
# kubeadm reset
```

在代理后方操作

kubeadm 会读取 https_proxy、http_proxy 和 no_proxy 环境变量。需确保 Kubernetes 内部网络包含在 no_proxy 中，例如：

```
export no_proxy="192.168.122.0/24,10.96.0.0/12,192.168.123.0/24"
```

其中第二个 CIDR（10.96.0.0/12）是 Kubernetes 默认的服务网络地址段。

故障排除

无法获取容器统计信息

如果 kubelet.service 输出以下错误：

```
Failed to get system container stats for "/system.slice/kubelet.service": failed to get
cgroup stats for "/system.slice/kubelet.service": failed to get container info for
"/system.slice/kubelet.service": unknown container "/system.slice/kubelet.service"
```

则需要为 kubelet 添加配置（参见相关的上游问题）。

```
/var/lib/kubelet/config.yaml
systemCgroups: '/systemd/system.slice'
kubeletCgroups: '/systemd/system.slice'
```

使用 Flannel CNI 和 systemd-networkd 时 Pod 无法通信

参见上游问题报告。

systemd-networkd 会为每个链接分配一个持久的 MAC 地址。此策略在其默认配置文件 `/usr/lib/systemd/network/99-default.link` 中定义。然而，Flannel 依赖于能够选择自己的 MAC 地址。要覆盖 systemd-networkd 对 `flannel***` 接口的行为，请创建以下配置文件：

```
/etc/systemd/network/50-flannel.link
[Match]
OriginalName=flannel*

[Link]
MACAddressPolicy=none
```

然后重启 `systemd-networkd.service`。

如果集群已经在运行，你可能需要手动删除每个节点上的 `flannel.1` 接口和 `kube-flannel-ds-***` Pod，包括主节点。Pod 会立即重新创建，并且它们自己会重新创建 `flannel.1` 接口。

删除 `flannel.1` 接口：

```
# ip link delete flannel.1
```

删除 `kube-flannel-ds-***` Pod。使用以下命令删除所有节点上的所有 `kube-flannel-ds-***` Pod：

```
$ kubectl -n kube-system delete pod -l="app=flannel"
```

CoreDNS Pod 一直处于 Pending 状态，控制平面节点保持 "NotReady"

当在单台机器上使用 `kubeadm init` 引导 Kubernetes 时，如果没有其他机器 `kubeadm join` 集群，控制平面节点默认会被添加上污点（Taint）。因此，工作负载将不会调度到该机器上。

可以通过以下命令确认控制平面节点是否被添加了污点：

```
kubectl get nodes -o json | jq '.items[].spec.taints
```

要临时允许在控制平面节点上调度，执行：

```
kubectl taint nodes <your-node-name> node-role.kubernetes.io/control-plane:NoSchedule-
```

然后重启 `containerd.service` 和 `kubelet.service` 以应用更新。

\[kubelet-finalize\] 畸形头：缺少 HTTP content-type

你可能忘记了选择 systemd cgroup 驱动。参见 kubeadm 问题 2767 报告此问题。

CoreDNS Pod 由于循环无法启动

当主机节点运行本地 DNS 缓存（如 systemd-resolved）时，CoreDNS 可能会由于检测到转发循环而无法启动。可以通过以下方式检查：

```
# kubectl get pods -n kube-system
NAME                                READY   STATUS             RESTARTS       AGE
cilium-jc98m                        1/1     Running            0              21m
cilium-operator-64664858c8-zjzcq    1/1     Running            0              21m
coredns-7db6d8ff4d-29zfg            0/1     CrashLoopBackOff   6 (41s ago)    21m
coredns-7db6d8ff4d-zlvsm            0/1     CrashLoopBackOff   6 (50s ago)    21m
etcd-k8s                            1/1     Running            19             21m
kube-apiserver-k8s                  1/1     Running            17             21m
kube-controller-manager-k8s         1/1     Running            16             21m
kube-proxy-cvntt                    1/1     Running            0              21m
kube-scheduler-k8s                  1/1     Running            23             21m
# kubectl logs -n kube-system coredns-7db6d8ff4d-29zfg
...
[FATAL] plugin/loop: Loop ([::1]:46171 -> :53) detected for zone ".", see
https://coredns.io/plugins/loop#troubleshooting. Query: "HINFO
6481192106818232 5.3042126689798234092."
```

这是由于 *kubelet* 将主机的 `/etc/resolv.conf` 文件传递给所有使用默认 `dnsPolicy` 的 Pod。CoreDNS 使用此 `/etc/resolv.conf` 作为上游转发请求的列表。由于它包含一个环回地址（如 127.0.0.53），CoreDNS 最终将请求转发给自己。

参见 https://coredns.io/plugins/loop/#troubleshooting 以解决此问题。

参见

- Kubernetes Documentation - The upstream documentation
- Kubernetes Cluster with Kubeadm - Upstream documentation on how to setup a Kubernetes cluster using kubeadm
- Kubernetes Glossary - The official glossary explaining all Kubernetes specific terminology
- Kubernetes Addons - A list of third-party addons
- Kubelet Config File - Documentation on the Kubelet configuration file
- Taints and Tolerations - Documentation on node affinities and taints

6.10 Libvirt

难度 高级 | **篇幅** 49.6 KB | **所属** 虚拟化

本节将讲到

- 安装
- 配置
- 测试
- 管理
- Hooks

Libvirt 是一组软件的汇集，提供了管理虚拟机和其它虚拟化功能（如：存储和网络接口等）的便利途径。这些软件包括：一个长期稳定的 C 语言 API、一个守护进程（libvirtd）和一个命令行工具（virsh）。Libvirt 的主要目标是提供一个单一途径以管理多种不同虚拟化方案以及虚拟化主机，包括：KVM/QEMU，Xen，LXC，OpenVZ 或 VirtualBox hypervisors （详见这里）。

Libvirt 的一些主要功能如下：

- **VM management（虚拟机管理）**：各种虚拟机生命周期的操作，如：启动、停止、暂停、保存、恢复和迁移等；多种不同类型设备的热插拔操作，包括磁盘、网络接口、内存、CPU等。
- **Remote machine support（支持远程连接）**：Libvirt 的所有功能都可以在运行着 libvirt 守护进程的机器上执行，包括远程机器。可以使用最简便且无需额外配置的 SSH 协议，也可以使用受支持的多种网络连接方式。
- **Storage management（存储管理）**：任何运行 libvirt 守护进程的主机都可以用于管理多种类型的存储：创建多种类型的文件镜像（qcow2，vmdk，raw，...），挂载 NFS 共享，枚举现有 LVM 卷组，创建新的 LVM 卷组和逻辑卷，对裸磁盘设备分区，挂载 iSCSI 共享，以及更多.....
- **Network interface management（网络接口管理）**：任何运行 libvirt 守护进程的主机都可以用于管理物理的和逻辑的网络接口，枚举现有接口，配置（和创建）接口、桥接、VLAN、端口绑定。
- **Virtual NAT and Route based networking（虚拟 NAT 和基于路由的网络）**：任何运行 libvirt 守护进程的主机都可以管理和创建虚拟网络。Libvirt 虚拟网络使用防火墙规则实现一个路由器，为虚拟机提供到主机网络的透明访问。

安装

基于守护进程/客户端的架构的 libvirt 只需要安装在需要要实现虚拟化的机器上。注意，服务器和客户端可以是相同的物理机器。

服务端

安装 libvirt包 以及至少一个虚拟运行环境（hypervisor）：

- libvirt 的 KVM/QEMU 驱动是 *libvirt* 的首选驱动，如果 KVM 功能已启用，则支持全虚拟化和硬件加速的客户机。详见 QEMU。
- 其他受支持的虚拟运行环境包括 LXC、VirtualBox 和 Xen。请参见它们各自的安装说明。有关 `libvirtd` 的安装备注：

- Libvirt 的 LXC 驱动并不依赖 lxc包 提供的 LXC 用户空间工具。因此，无需安装该工具也能使用这个驱动。需要 `libvirtd` 处于运行状态才能使用 `libvirt-lxc` 连接。
- Libvirt 能支持 Xen，但默认未内建支持（FS#27356）。需要用 ABS 编辑 libvirt包 的 PKGBUILD，去掉 `-Ddriver_libxl=disabled` 选项后重新构建（built）libvirt。

对于网络连接，需要安装这些包：

- dnsmasq包 用于默认的 NAT/DHCP 网络
- openbsd-netcat包 用于 SSH 远程管理

其他可选依赖项可能提供所需或扩展功能，例如 dmidecode包 用于支持 DMI 系统信息。建议在阅读 libvirt包 的 pacman 输出后，将所需组件作为依赖项安装。

注意：若正在使用 firewalld，从 `libvirt` 5.1.0 和 firewalld 0.7.0 开始，不再需要将防火墙后端改为 iptables。`libvirt` 现在会在 firewalld 中自动创建名为 'libvirt' 的区域，并在该区域管理其所需的网络规则。详见 Firewall and network filtering in libvirt.

客户端

客户端是用于管理和访问虚拟机的用户界面。

- **virsh** — 用于管理和配置域（虚拟机）的命令行程序。

: https://libvirt.org/ || libvirt包

- **Boxes** — 简单的 GNOME 3 程序，可以访问远程虚拟系统。是 gnome-extra 包组的一部分。

: https://apps.gnome.org/Boxes || gnome-boxes包

- **Libvirt Sandbox** — 应用程序沙箱工具包。

: https://sandbox.libvirt.org/ || libvirt-sandboxAUR

- **Virt Viewer** — 简单的远程显示客户端。

: https://gitlab.com/virt-viewer/virt-viewer || virt-viewer包

- **Virt-manager（英语：Virt-manager）** — 用于图形化使用 libvirt 管理 KVM，Xen 或是 LXC。

: https://virt-manager.org/ || virt-manager包

- **Cockpit（英语：Cockpit）** — 基于网页的系统管理工具，可通过插件管理虚拟机。

: https://cockpit-project.org/ || cockpit-machines包

兼容 libvirt 的软件列表见这里。

配置

Libvirt 可通过两种模式管理 QEMU 虚拟机：system（系统模式）和 session（会话模式）：

- **system** URI 连接到以 root 身份运行的 libvirtd daemon（该进程在系统启动时加载）。使用 'system' 模式创建和运行的虚拟机默认以 root 身份启动，除非另行配置（例如通过 `/etc/libvirt/qemu.conf`）。
- **session** URI 会以当前本地用户身份启动 libvirtd 实例，所有虚拟机均以该用户权限运行。

模式对比分析：

- 仅 'system' 模式支持虚拟机随宿主机启动自动运行，且 root 权限的 libvirtd 实例拥有通过 bridge 或虚拟网络配置完整网络连接的权限。`qemu:///system` 通常是 virt-manager 等工具的默认选项。
- `qemu:///session` 存在显著缺陷：由于 libvirtd 实例权限不足，仅支持 qemu 的 usermode networking（用户模式网络），该模式存在隐性限制，因此不建议使用（详见qemu 网络配置）。

`qemu:///session` 的优势在于权限简化：用户可直接在 $HOME 存储磁盘镜像、串行 PTY 设备归属用户自身等。

对于***系统***级别的管理任务（如：全局配置和镜像卷位置），libvirt 要求至少要设置授权和启动守护进程。

对于用户***会话***级别的管理任务，守护进程的安装和设置*不是*必须的。授权总是仅限本地，前台程序将启动一个 **libvirtd** 守护进程的本地实例。

设置授权

来自 libvirt：连接授权：

: Libvirt 守护进程允许管理员分别为客户端连接的每个网络 socket 选择不同授权机制。这主要是通过 libvirt 守护进程的主配置文件 `/etc/libvirt/libvirtd.conf`

来实现的。每个 libvirt socket 可以有独立的授权机制配置。目前的可选项有 `none`、`polkit` 和 `sasl`。

Using libvirt group

把用户加到 `libvirt` user group 用户组是确保该用户可以访问libvirt daemon的最简单方法。

默认情况下，`libvirt` 组的成员对 RW 守护程序套接字具有免密访问权限。

使用 polkit

Because libvirt包 pulls polkit包 as a dependency during installation, polkit is used as the default value for the `unix_sock_auth` parameter (source). File-based permissions remain nevertheless available.

注意：为使 `polkit` 认证工作正常，应该重启一次系统。

libvirt 守护进程在 polkit 策略配置文件（`/usr/share/polkit-1/actions/org.libvirt.unix.policy`）中提供了两种 polkit 操作：

- `org.libvirt.unix.manage` 面向完全的管理访问（读写模式后台 socket），以及
- `org.libvirt.unix.monitor` 面向仅监视察看访问（只读 socket）。

默认的面向读写模式后台 socket 的策略需要认证为管理员。这点类似于 sudo 认证，但它并不要求客户应用最终以 root 身份运行。默认策略下也仍然允许任何应用连接到只读 socket。

Arch Linux 默认 `wheel` 组的所有用户都是管理员身份：定义于 `/etc/polkit-1/rules.d/50-default.rules`（参阅：Polkit#管理员身份认证）。所以**如果用户是 `wheel` 组的成员**，就不必新建组和规则文件：只要连接到了读写模式 socket（例如通过 virt-manager包）就会被提示输入该用户的口令。

注意：口令提示依赖于系统上的身份认证组件。文本控制台默认的认证代理是 `pkttyagent`，它可能因工作不正常而导致各种问题。

提示：如果要配置无口令认证，参阅 Polkit#跳过口令提示。

你可能想要修改授权以读写模式访问 socket 的组。例如，你想授权 `mykvm` 组，可创建下面的文件：

```
/etc/polkit-1/rules.d/50-libvirt.rules
/* Allow users in mykvm group to manage the libvirt
daemon without authentication */
polkit.addRule(function(action, subject) {
```

```
    if (action.id == "org.libvirt.unix.manage" &&
        subject.isInGroup("mykvm")) {
            return polkit.Result.YES;
    }
});
```

然后添加自己到 `mykvm` 组并重新登录。可以将 *mykvm* 替换为你想要的任意组，只需确保该组存在，且用户是该组的成员（详情可参考用户和用户组）。

修改组之后不要忘记重新登录才能生效。

基于文件的权限授权

为了给 *libvirt* 组用户定义基于文件的权限以管理虚拟机，取消下列行的注释：

```
/etc/libvirt/libvirtd.conf
#unix_sock_group = "libvirt"
#unix_sock_ro_perms = "0777"  # set to 0770 to deny non-group libvirt users
#unix_sock_rw_perms = "0770"
#auth_unix_ro = "none"
#auth_unix_rw = "none"
```

有些资料提到可以通过改变某些特定 libvirt 目录的权限以简化管理。需要记住的是：包更新时，这些变更会丢失。如要修改这些系统目录的权限，需要 root 用户权限。

守护进程

注意：Libvirt 正在从单个整体守护进程转变为单独的模块化守护进程，目的是在将来删除整体守护进程。参见 Libvirt daemons 获取更多信息。

`libvirtd.service` 和 `virtlogd.service`这两个服务单元都要启动。可以把 `libvirtd.service` 设置为启用，这时系统将同时启用 `virtlogd.service` 和 `virtlockd.socket` 两个服务单元，因此后二者不必再设置为启用。

另一种方案是仅启动/启用 `libvirtd.socket` 与 `virtlogd.socket`，以实现按需 socket 激活。

非加密的 TCP/IP sockets

警告：这种方法常用于在可信网络中快速连接远程域做协助。这是***最不安全*** 的连接方式，应当仅仅用于测试或用于安全、私密和可信的网络环境。这时 SASL 没有启用，所以所有的 TCP 通讯都是*明文*传输。在正式的应用场合应当*始终* 启用 SASL。

编辑 `/etc/libvirt/libvirtd.conf`：

```
/etc/libvirt/libvirtd.conf
listen_tls = 0
listen_tcp = 1
auth_tcp="none"
```

同时需要编辑 `/etc/conf.d/libvirtd` 以在监听模式下启动服务：

```
/etc/conf.d/libvirtd
LIBVIRTD_ARGS="--listen"
```

用主机名访问虚拟机

在非隔离的、桥接的网络中从宿主机访问客户机，可以通过启用 libvirt包 提供的 `libvirt` 和/或 `libvirt_guest` NSS 模块来实现。For the comparison of the two modules and technical details, see libvirt documentation.

在 nsswitch.conf(5) 中添加需要的模块：

```
/etc/nsswitch.conf
hosts: files libvirt libvirt_guest dns myhostname
```

注意： `ping` 和 `ssh` 这类命令使用虚拟机主机名可以正常工作，但 `host` 和 `nslookup` 这类命令可能会失败或产生非预期结果，因后者依赖 DNS 。应改用 `getent hosts <vm-hostname>` 命令。

测试

测试 libvirt 在*系统*级工作是否正常：

```
$ virsh -c qemu:///system
```

测试 libvirt 在用户*会话*级工作是否正常：

```
$ virsh -c qemu:///session
```

管理

绝大部分的 libvirt 管理可以通过三个工具实现：virt-manager包（图形界面）、`virsh` 和 `guestfish`（它是 libguestfs包 的一部分）。

virsh

Visrsh 用于管理客户*域*（虚拟机），适用于脚本及虚拟环境管理工作。受限于与虚拟化环境通信的通道，绝大部分 virsh 命令需要管理员权限。尽管如此，一些典型的管理操作如域的创建、运行等也可以像 VirtualBox 那样以普通用户身份执行。

Virsh 允许带命令行选项执行。如果不带则进入其内置的交互式终端：`virsh`。交互式终端支持 tab 键命令补全。

从命令行执行：

```
$ virsh [可选项] <命令> [参数]...
```

在交互式终端里运行：

```
virsh # <命令> [参数]...
```

帮助也是可用的：

```
$ virsh help [option*] or [group-keyword*]
```

存储池

存储池是指保存*卷*的位置。Libvirt 中*卷*的定义相当于其他系统中*虚拟磁盘*或*虚拟机镜像*的概念。存储池应该是一个目录、一个网络文件系统或一个分区（这也包括 LVM）。存储池可以在活动与不活动之间切换，可以为其分配存储空间。

在*系统*级别，默认被激活的存储池是 `/var/lib/libvirt/images/`；在用户*会话*级别，`virt-manager` 将存储池创建在 `$XDG_DATA_HOME/images` 目录。

列出活动和不活动的存储池的命令：

```
$ virsh pool-list --all
```

用 virsh 新建存储池

以下示例为*添加*存储池、目录和 LVM 卷的方法：

```
$ virsh pool-define-as name type [source-host] [source-path] [source-dev] [source-name]
[<target>] [--source-format format]
$ virsh pool-define-as poolname dir - - - - /home/username/.local/libvirt/images
$ virsh pool-define-as poolname fs - -  /dev/vg0/images - mntpoint
```

上述示例仅仅定义了存储池的信息，下面创建它：

```
$ virsh pool-build     poolname
$ virsh pool-start     poolname
$ virsh pool-autostart poolname
```

删除它的命令：

```
$ virsh pool-undefine  poolname
```

提示：对于 LVM 存储池而言：

- 最佳实践是仅把一个卷组分配给一个存储池。
- 请为存储池选择一个与 LVM 卷组不同的名字。否则当存储池被删除时，该卷组也将被删除。

用 virt-manager 新建存储池

首先，连接到虚拟运行环境（例如 QEMU/KVM 的系统/用户会话）。然后，右键点击一个**连接**，选择**详情**；切换到**存储**选项卡，点击左下角的**+**，按照向导操作。

存储卷

存储池被创建之后，就可以在存储池中创建存储卷。*如果你想新建一个域（虚拟机），那么这一步可以跳过，因为这一步可以在创建域的过程中完成。*

用 virsh 新建卷

新建卷，列出卷，变更卷大小，删除卷：

```
$ virsh vol-create-as      poolname volumename 10GiB --format aw|bochs|raw|qcow|qcow2|
vmdk
$ virsh vol-upload  --pool poolname volumename volumepath
$ virsh vol-list           poolname
$ virsh vol-resize  --pool poolname volumename 12GiB
$ virsh vol-delete  --pool poolname volumename
$ virsh vol-dumpxml --pool poolname volumename  # for details.
```

域

虚拟机被称作**“域”**。如果你想在命令行下操作，使用 `virsh` 列出，创建，暂停，关闭……域。`virt-viewer` 可以用来查看使用 `virsh` 启动的域。域的创建通常以图形化的 `virt-manager` 或者命令行下的 `virt-install`（一个命令行工具，是 virt-install包 包的一部分）完成。

创建新域通常需要安装媒介，例如存储池中的 `.iso` 文件或是直接从光驱安装。

列出活动的和不活动的域：

```
# virsh list --all
```

注意：SELinux 有内置策略使在 `/var/lib/libvirt/images/` 中的卷可以被访问。如果你使用 SELinux 并且在卷方面有问题，确保卷位于该目录，或是其它存储池的标记正常。

用 virt-install 新建域

**本文或本章节的事实准确性存在争议。**

原因： `/usr/share/libosinfo` is not provided by any official packages, including libosinfo包.（在 Talk:Libvirt#Where_is_'/usr/share/libosinfo/db/oses/os.xml'? 中讨论）

对于很详细的域（虚拟机）配置，可以#用 virt-manager 新建域更简单地完成。但是，基础配置同样可以用 `virt-install` 完成并且同样顺利运行。最小配置包括 `--name`，`--memory`，存储（`--disk`，`--filesystem` 或 `--nodisks`）和安装介质（通常来说是 `.iso` 文件或 CD）。查看 virt-install(1) 得到未列出的选项和更多的详情。

安装 Arch Linux（创建了 2 GiB qcow2 格式卷；用户网络）：

```
$ virt-install  \
  --name arch-linux_testing \
  --memory 1024             \
  --vcpus=2,maxvcpus=4      \
  --cpu host                \
  --cdrom $HOME/Downloads/arch-linux_install.iso \
  --disk size=2,format=qcow2  \
  --network user            \
  --virt-type kvm
```

Fedora testing (Xen, 非默认池,无默认控制台):

```
$ virt-install  \
  --connect xen:///     \
  --name fedora-testing \
  --memory 2048         \
  --vcpus=2             \
  --cpu=host            \
  --cdrom /tmp/fedora20_x84-64.iso      \
  --os-type=linux --os-variant=fedora20 \
  --disk pool=testing,size=4            \
  --network bridge=br0                  \
  --graphics=vnc                        \
  --noautoconsole
$ virt-viewer --connect xen:/// fedora-testing
```

Windows:

```
$ virt-install \
  --name=windows7            \
  --memory 2048              \
  --cdrom /dev/sr0           \
  --os-variant=win7          \
  --disk /mnt/storage/domains/windows7.qcow2,size=20GiB \
  --network network=vm-net   \
  --graphics spice
```

提示：运行 `osinfo-query --fields=name,short-id,version os` 来获得 `--os-variant` 的参数，这可以帮助定制域的一些规格。然而 `--memory` 和 `--disk` 是必须被输入的。如果需要查看这些规格，可以看看 `/usr/share/libosinfo/db/oses/*os*.xml`（译注：此处可能已过时）。在安装后，推荐安装 Spice Guest Tools，其中包括 VirtIO 驱动。Windows 的 VirtIO 网络驱动可通过 virtio-winAUR 获得。要使用 VirtIO，需要在虚拟机 `.xml` 配置中使用 `<model type='virtio' />`。更多的信息可以参考 QEMU 页面.

导入现有的卷：

```
$ virt-install  \
  --name demo  \
  --memory 512 \
  --disk /home/user/VMs/mydisk.img \
  --import
```

用 virt-manager 新建域

首先，连接到虚拟运行环境（例如 QEMU/KVM *system* 或用户 *session*，在连接上右击并选择*新建*，然后跟随向导完成。

- 在**第四步**中取消选中**立即分配全部虚拟磁盘空间**会加快创建过程并节省实际虚拟磁盘空间占用；然而，这将导致将来花费额外的磁盘整理时间。
- 在**第五步**中打开**高级选项**并确认**虚拟化类型**设为 **kvm**（这通常是首选模式）。如果要求附加的硬件配置，选中**安装前定制**选项。

管理域

启动域：

```
$ virsh start domain
$ virt-viewer --connect qemu:///session domain
```

正常关闭域；强制关闭域:

```
$ virsh shutdown domain
$ virsh destroy  domain
```

在 libvirtd 启动时自动启动域:

```
$ virsh autostart domain
$ virsh autostart domain --disable
```

在宿主机关闭时自动关闭域:

: 通过使用 `libvirt-guests.service` Systemd 服务，运行中的域可以在宿主机关闭时自动挂起/关闭。同时这个服务还可以让挂起/休眠的域在宿主机启动的时候自动恢复。可查看 libvirt-guests(8) 了解相关选项。

编辑一个域的 XML 配置：

```
$ virsh edit domain
```

参考 libvirt 维基的 XML 格式一节了解关于 XML 配置文件的信息。

注意：直接被 QEMU 启动的虚拟机不被 libvirt 管理。

网络

这里是有关 libvirt 网络的的概述。

可将以下四种网络连接到域：

- bridge——这是一个虚拟设备，它通过一个物理接口直接共享数据。使用场景为：宿主机有*静态*网络、不需与其它域连接、要占用全部进出流量，并且域运行于*系统*层级。网桥必须要在 libvirt 外配置，详细操作可参考网桥。网桥创建后，需要将它指定到相应客户机的 `.xml` 配置文件中。
- network——这是一个虚拟网络，它可以与其它虚拟机共用。Libvirt 提供多种虚拟网络模式，例如 NAT（Network address translation，网络地址转换）模式，路由模式和隔离模式。使用场景为：宿主机有*动态*网络（例如：NetworkManager）或使用无线网络。
- macvtap——直接连接到宿主机的一个物理网络接口。相较*桥接*更加简单，代价是宿主机无法通过该接口与域通信。Libvirt 可以很方便地配置该类网络。
- user——本地网络，仅用于用户*会话*。

绝大多数用户都可以通过 `virsh` 的各种可选项创建具有各种功能的网络，一般来说通过 GUI 程序（像 `virt-manager` 之类）更方便，也可以按 #用 virt-install 新建域所述实现。

注意：

- libvirt 通过 dnsmasq包 处理 DHCP 和 DNS 请求，为每个虚拟网络创建一个实例。它也会为特定的路由添加 iptables 规则并启用 `ip_forward` 内核参数。这也意味着宿主机上无需预先运行 dnsmasq（并可能干扰到 libvirt 的 dnsmasq 实例）。
- 如果无法启动默认网络，请确保已安装 iptables-nft包 和 dnsmasq包，并在 重启 `libvirtd.service` 前先 启动/启用 `iptables.service`。

注意：

如果你用了 iptables包 而不是 nftables包, 那么有必要在配置文件/etc/libvirt/network.conf中相应地指定它。

示例:

1. default: #firewall_backend = "nftables"

firewall_backend = "iptables"

通过以下命令获取虚拟机的 IP（假设它连接到 *default* 网络并通过 DHCP 获取 IP）：

```
$ virsh net-dhcp-leases default
```

如果 VM 上运行有 *qemu-guest-agent*：

```
$ virsh domifaddr --source agent $vm
```

将 *$vm* 替换为实际的虚拟机名称（或域 ID）。

管理并修改网络

修改网络前建议先阅读 libvirt 维基的用于虚拟网络的基本命令行用法一节。另外也建议通过 libvirt 网络维基了解用法。

IPv6

当通过任何配置工具试图添加 IPv6 地址时，你可能会遇到这样的错误：

```
Check the host setup: enabling IPv6 forwarding with RA routes without accept_ra set to 2 is likely to cause routes loss. Interfaces to look at: eth0
```

要修复这个问题，运行如下命令（将 eth0 改为你的物理接口的名称），并重新启动系统。

```
# sysctl net.ipv6.conf.eth0.accept_ra=2
```

Macvtap

要创建 macvtap 网络，先创建该文件：

```
macvtap.xml
<network>
  <name>macvtap-net</name>
  <forward mode='bridge'>
    <interface dev='eth0'/>
  </forward>
</network>
```

然后定义并启用网络：

```
$ virsh net-define macvtap.xml
$ virsh net-autostart macvtap-net
$ virsh net-start macvtap-net
```

现在已创建好 macvtap-net 网络并已持久化，它通过 eth0 桥接到外部网络。

警告：宿主机无法通过该网络与域通信。宿主机的网络不受影响，域也可以与该网络的其它机子通信，但不能与宿主机本身通信。可参考 libvirt 维基绕过该限制。

nftables 防火墙

当使用 NAT 类型网络配合简单的限制性 nftables 防火墙（例如默认配置）时，可能需要允许以下流量：

虚拟网络接口（默认配置在此丢弃数据包）的转发流量

来自虚拟网络接口的 DNS/DHCP 请求（供 DHCP 客户端与宿主机通信）

以 `virbr0` 作为 libvirt 桥接接口的简化配置示例：

```
/etc/nftables.conf
# ...
table inet filter {
  chain input {
    type filter hook input priority filter
    policy drop
    # ...
    iifname virbr0 udp dport {53, 67} accept comment "allow VM dhcp/dns requests to
host"
    # ...
  }

  chain forward {
    type filter hook forward priority filter
    policy drop

    iifname virbr0 accept
    oifname virbr0 accept
  }
}
```

快照

快照保存某一时刻域的磁盘、内存和设备状态以供将来使用。快照有很多用处，例如在进行可能的破坏性操作时保存一份干净的域状态。快照使用唯一名称进行标识。

快照保存在卷之中，卷必须为 qcow2 或 raw 格式。快照使用增量存储，所以并不会占用很多空间。

创建快照

Once a snapshot is taken it is saved as a new block device and the original snapshot is taken offline. Snapshots can be chosen from and also merged into another (even without shutting down the domain).

Print a running domain's volumes (running domains can be printed with `virsh list`):

```
# virsh domblklist domain
 Target     Source
 ------------------------------------------------
 vda        /vms/domain.img
```

To see a volume's physical properties:

```
# qemu-img info /vms/domain.img
 image: /vms/domain.img
 file format: qcow2
 virtual size: 50G (53687091200 bytes)
```

```
 disk size: 2.1G
 cluster_size: 65536
```

Create a disk-only snapshot (the option `--atomic` will prevent the volume from being modified if snapshot creation fails):

```
# virsh snapshot-create-as domain snapshot1 --disk-only --atomic
```

List snapshots:

```
# virsh snapshot-list domain
 Name                 Creation Time             State
 ------------------------------------------------------------
 snapshot1            2012-10-21 17:12:57 -0700 disk-snapshot
```

One can then copy the original image with `cp --sparse=true` or `rsync -S` and then merge the original back into snapshot:

```
# virsh blockpull --domain domain --path /vms/domain.snapshot1
```

`domain.snapshot1` becomes a new volume. After this is done the original volume (`domain.img` and snapshot metadata can be deleted. The `virsh blockcommit` would work opposite to `blockpull` but it seems to be currently under development (including `snapshot-revert feature`, scheduled to be released sometime next year.

其它管理操作

连接到非默认的虚拟运行环境：

```
$ virsh --connect xen:///
virsh # uri
xen:///
```

通过 SSH 连接到 QEMU 虚拟运行环境，并且以相同方式登录：

```
$ virsh --connect qemu+ssh://username@host/system
$ LIBVIRT_DEBUG=1 virsh --connect qemu+ssh://username@host/system
```

通过 SSH 连接到一个图形控制台：

```
$ virt-viewer  --connect qemu+ssh://username@host/system domain
$ virt-manager --connect qemu+ssh://username@host/system domain
```

注意：如果你在连接 RHEL 服务器（或其他不是 Arch 的服务器）时出现问题，可尝试这里提到的两个方案：FS#30748 和 FS#22068。

连接到 VirtualBox（*libvirt 对 VirtualBox 的支持尚不稳定，可能会导致 libvirtd 崩溃*）：

```
$ virsh --connect vbox:///system
```

网络配置:

```
$ virsh -c qemu:///system net-list --all
$ virsh -c qemu:///system net-dumpxml default
```

Hooks

钩子 (Hooks) 是指在启动和运行 libvirt 守护进程时，由不同事件触发的脚本。 它们可以用来执行启动访客机所需的准备命令，例如设置网络或预留内存。

libvirt 提供以下类型的钩子：

- daemon - 触发时机：启动、关闭、重新加载
- qemu - 触发时机：准备、启动、已启动、已停止、释放、迁移、恢复、重新连接、附加
- lxc - 触发时机：准备、启动、已启动、已停止、释放、重新连接
- libxl - 触发时机：准备、启动、已启动、已停止、释放、迁移、重新连接
- network - 触发时机：启动、已启动、已停止、端口创建、更新、端口删除

有关每个钩子和触发器的详细信息，请参见 libvirt 文档。

新建一个 hook

钩子由位于 `/etc/libvirt/hooks` 的脚本表示。如果该文件夹不存在，则需要创建它。

每个钩子由该文件夹中同名的脚本（例如 `/etc/libvirt/hooks/qemu`）或子文件夹（例如 `/etc/libvirt/hooks/qemu.d/`）表示。后者可以包含不同的脚本，这些脚本在触发点都会运行。脚本像其他任何脚本一样运行，因此需要以要使用的命令解释器的声明开头（例如 `#!/bin/bash`）并可由 libvirt 用户执行。

每当碰到触发点时，脚本就会运行。例如，daemon 脚本在系统的启动/停止周期中至少运行两次，一次在启动时，一次在关闭时。要仅在给定点运行命令，必须在脚本中实现条件。为此，libvirt 传递可用于识别当前触发条件的参数。

根据 libvirt 文档，这些参数定义如下：

- 参数 1：操作涉及的对象的名称
- 参数 2：正在执行的操作的名称
- 参数 3：如果要命名子操作则使用
- 参数 4：如有需要，则为额外参数 如果其中一个参数不适用，则传递一个连字符。

注意：如果创建脚本后钩子不起作用，请尝试重启 libvirt 守护进程。根据新的模块化 daemon 架构，需重启的具体 daemon 取决于对应的 hook（例如针对 `qemu` hook 需操作 `virtqemud`）。

示例

To run an command every time you start an qemu guest, before any resources are allocated, you can use the qemu hook. At this point, libvirt runs the hooks like this: `/etc/libvirt/hooks/qemu <guest_name> prepare begin` - The script for this could like this:

```
/etc/libvirt/hooks/qemu
#!/bin/bash
guest_name="$1"
libvirt_task="$2"
```

```
if [ "$libvirt_task" = "prepare" ]; then
       <run some important code here>
fi
```
If the guest is stopped, the same script would be run, but this time the daemon would start the command like this: `/etc/libvirt/hooks/qemu <guest_name> stopped end -`

宿主机与虚拟机间共享数据

Virtio-FS

注意：Virtio-FS is not supported in QEMU/KVM user sessions.

The description here will use hugepages to enable the usage of shared folders. Sharing files with Virtio-FS lists an overview of the supported options to enable filesharing with the guest.

First you need to enable hugepages which are used by the virtual machine:

```
/etc/sysctl.d/40-hugepage.conf
vm.nr_hugepages = nr_hugepages
```
To determine the number of hugepages needed check the size of the hugepages:

```
$ grep Hugepagesize /proc/meminfo
```
The number of hugepages is *memory size of virtual machine / Hugepagesize*. Add to this value some additional pages. You have to reboot after this step, so that the hugepages are allocated.

Now you have to prepare the configuration of the virtual machine:

```
# virsh edit name_of_virtual_machine
<domain>
...
  <memoryBacking>
    <hugepages/>
  </memoryBacking>
...
  <cpu ...>
    <numa>
      <cell memory='memory size of virtual machine' unit='KiB' memAccess='shared'/>
    </numa>
  </cpu>
...
  <devices>
    ...
    <filesystem type='mount' accessmode='passthrough'>
      <driver type='virtiofs'/>
      <source dir='path to source folder on host'/>
      <target dir='mount_tag'/>
    </filesystem>
    ...
  </devices>
</domain>
```
It is necessary to add the NUMA definition so that the memory access can be declared as shared. id and cpus values for NUMA will be inserted by virsh.

It should now be possible to mount the folder in the shared machine:

```
# mount -t virtiofs mount_tag /mnt/mount/path
```
Add the following fstab entry to mount the folder automatically at boot:

```
/etc/fstab
...
mount_tag /mnt/mount/path virtiofs rw,noatime,_netdev 0 0
```

9p

File system directories can be shared using the 9P protocol). Details are available in QEMU's documentation of 9psetup.

Configure the virtual machine as follows:

```
<domain>
...
  <devices>
    ...
    <filesystem type="mount" accessmode="mapped">

      <target dir="mount_tag"/>
    </filesystem>
  </devices>
</domain>
```

Boot the guest and mount the shared directory from it using:

```
# mount -t 9p -o trans=virtio,version=9p2000.L mount_tag /path/to/mount_point/on/guest
```

See https://docs.kernel.org/filesystems/9p.html for more mount options.

To mount it at boot, add it to the guest's fstab:

```
/etc/fstab
...
mount_tag     /path/to/mount_point/on/guest      9p     trans=virtio,version=9p2000.L
0 0
```

The module for the 9p transport (i.e. `9pnet_virtio` for `trans=virtio`) will not be automatically loaded, so mounting the file system from `/etc/fstab` will fail and you will encounter an error like `9pnet: Could not find request transport: virtio`. The solution is to preload the module during boot:

```
/etc/modules-load.d/9pnet_virtio.conf
9pnet_virtio
```

Samba / SMB

An other easy way to share data between guest and host is to use the smb protocol. While performance and latency may not be as good as in the other described ways, its sufficient for simple tasks like transfering simple files like images or documents from and to the guest.

The smb server can be set up directly on either the host, or the guest, for example using Samba, eliminating the need for a dedicated file server. Windows guests have the ability to create smb shares included right after installation (Microsoft Supportpage).

One possible way to access the share under linux (either from the host, or from the guest, depending, where you have installed your server) is to create an entry in your fstab. The samba包 package is required.

```
/etc/fstab
#Accessing a samba share on my vm from the host
```

```
//my_vm/my_share /home/archuser/my_vm cifs
_netdev,noauto,nofail,user,credentials=/home/archuser/.config/my_vm.key,gid=1000,uid=98
4 0 0
```

`_netdev,noauto,nofail` ensures that the share is only mounted when needed without causing issues if the vm is not booted. `user,credentials=/home/user/.config/my_vm.key,gid=1000,uid=984` gives you the ability to mount the share on the fly while first accessing it, without needing a password.

UEFI 支持

Libvirt 可以通过 QEMU 和 OVMF 来支持 UEFI 虚拟机。

安装 edk2-ovmf包。

重启 `libvirtd`。

你现在就可以创建 UEFI 虚拟机了。通过 virt-manager包 创建一台虚拟机，在到达*新建虚拟机*向导的最后一页时，进行如下操作：

- 勾选**在安装前自定义配置**，之后点击完成。
- 在**概况**屏幕, 将固件改为：

1. - *UEFI x86_64: /usr/share/edk2/x64/OVMF_CODE.4m.fd*：无安全启动支持的 x64 UEFI，
2. *UEFI x86_64: /usr/share/edk2/x64/OVMF_CODE.secboot.4m.fd*：带安全启动支持的 x64 UEFI（无预置证书）。
3. 点击*应用*。

- 点击**开始安装**

参考 Fedora:Using UEFI with QEMU 获得更多信息。

小技巧

在虚拟机中使用整个物理磁盘设备

若宿主机存在包含其他操作系统（如 Windows）的独立物理磁盘，可将其配置为虚拟机中的虚拟磁盘进行启动。由于采用原始磁盘访问，虚拟机内磁盘性能接近原生。

Windows 虚拟机启动前提条件

在尝试启动前，**必须**在该磁盘的 Windows 系统中安装 virtio 驱动：

- **Windows 7**：使用版本 0.1.173-4。新版 virtio 中的部分驱动可通过设备管理器手动安装
- **Windows 10**：支持最新 virtio 版本

配置 Windows 磁盘接口驱动

若启动时遭遇 `0x0000007B` 蓝屏错误，表示 Windows 在早期启动阶段无法访问磁盘（因相关接口驱动未加载或设置为手动启动）。需修改注册表强制驱动随系统启动：

1. 打开注册表编辑器，定位至 `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services`
2. 对以下键值（如存在）执行操作：`aliide`, `amdide`, `atapi`, `cmdide`, `iastor`, `iastorV`, `intelide`, `LSI_SAS`, `msahci`, `pciide`, `viaide`
3. 将每个键中的 **Start** 值改为 **0**
4. 若使用 PCIe NVMe 磁盘，需同样启用对应驱动

获取磁盘唯一标识路径

执行 `ls /dev/disk/by-id/` 查看磁盘 ID（例如，得到 `ata-TS512GMTS930L_C199211383`则 `/dev/disk/by-id/ata-TS512GMTS930L_C199211383` 为磁盘唯一标识）

通过 QEMU CLI 添加磁盘

在 QEMU 命令行中添加参数：`-drive file=/dev/disk/by-id/ata-TS512GMTS930L_C199211383,format=raw,media=disk`

（替换 `file=` 后的路径为实际磁盘路径）

通过 libvirt XML 添加磁盘

在 QEMU 命令行中添加参数：

```
$ virsh edit vmname
...
    <disk type="block" device="disk">
      <driver name="qemu" type="raw" cache="none" io="native"/>
      <source dev="/dev/disk/by-id/ata-TS512GMTS930L_C199211383"/>
      <target dev="sda" bus="sata"/>
      <address type="drive" controller="0" bus="0" target="0" unit="0"/>
    </disk>
...
```

替换 `file=` 后的路径为实际磁盘路径

通过 virt-manager 添加磁盘

- **新建虚拟机**：选择 "导入现有磁盘"，粘贴磁盘唯一路径
- **已有虚拟机**：添加设备 → 存储 → 选择/创建自定义存储 → 粘贴磁盘路径

Python 连接代码

libvirt-python包 在 `/usr/lib/python3.x/site-packages/libvirt.py` 提供了一个 Python API。

常用例子在 `/usr/share/doc/libvirt-python-*your_libvirt_version*/examples/` 给出。

一个使用 qemu-desktop包 和 openssh包 的例子（非官方）：

```
#! /usr/bin/env python3
import socket
import sys
import libvirt

conn = libvirt.open("qemu+ssh://xxx/system")
print("Trying to find node on xxx")
domains = conn.listDomainsID()
for domainID in domains:
    domConnect = conn.lookupByID(domainID)
    if domConnect.name() == 'xxx-node':
        print("Found shared node on xxx with ID {}".format(domainID))
        domServ = domConnect
        break
```

先进格式化 4K 原生硬盘

To turn a disk into an Advanced Format 4Kn disk, both its physical and logical sector size needs to be set to 4 KiB. For virtio-blk and virtio-scsi this can be done by setting the `logical_block_size` and `physical_block_size` options with the <blockio> element. For example:

```
# virsh edit name_of_virtual_machine
<domain>
  ...
  <devices>
    ...
    <disk type='file' device='disk'>
      ..
      <blockio logical_block_size='4096' physical_block_size='4096'/>
    </disk>
    ...
  </devices>
</domain>
```

控制 QEMU

Libvirt is capable of passing on QEMU command line arguments to the underlying QEMU instance running the VM. This functionality is highly useful when libvirt does not provide QEMU features (yet). For examples, see the entire Intel GVT-g article.

modify VM XML schema for QEMU

This serves to enable QEMU-specific elements. Change

```
$ virsh edit vmname
<domain type='kvm'>
```

to

```
$ virsh edit vmname
<domain xmlns:qemu='http://libvirt.org/schemas/domain/qemu/1.0' type='kvm'>
```

QEMU 命令行参数

In libvirt, QEMU command line arguments separated by whitespaces need to be provided separately.

The correct location to insert them is at the end of the `<domain>` element, i. e. right above the closing `</domain>` tag.

```
-display gtk,gl=es,zoom-to-fit=off
```

Becomes

```
$ virsh edit vmname
...
  </devices>
  <qemu:commandline>
    <qemu:arg value="-display"/>
    <qemu:arg value="gtk,gl=es,zoom-to-fit=off"/>
  </qemu:commandline>
</domain>
```

排障

系统实例下的 PulseAudio

PulseAudio 守护进程通常在你的普通用户下运行，并且只接受来自相同用户的连接。然而 libvirt 默认使用 root 运行 QEMU。为了让 QEMU 在普通用户下运行，编辑 `/etc/libvirt/qemu.conf` 并将 `user` 设置为你的用户名。

```
user = "dave"
```

你同样需要告诉 QEMU 使用 PulseAudio 后端并识别要连接的服务器。使用 `virsh edit` 将如下内容添加到你的域配置中：

```
  <audio id="1" type="pulseaudio" serverName="/run/user/1000/pulse/native">
    <input latency="20000"/>
    <output latency="20000"/>
  </audio>
```

`1000` 是你的用户 ID，如有必要可修改。

可省略延迟参数（单位：微秒），但使用默认值可能导致音频出现 crackling（爆音）。

Hypervisor CPU 使用率过高

virt-manager 生成的默认虚拟机配置可能导致 QEMU 进程占用较高 CPU（10-20%）。若以无界面模式运行虚拟机，建议移除非必要设备

Virtual machine cannot be un-paused on virt-manager

If you are using a disk image format such as qcow2 which has a specified virtual capacity, but only stores what is needed, then you need to have space on the host partition for the image to grow. If you see I/O related errors when attempting to start the VM, it's possible that the host partition holding the virtual disk image is full. You can run `df -h` on the host to verify how much free space is available.

If this is the case, see System maintenance#Clean the filesystem for ways to free up space.

Redirect USB Device is greyed out in virt-manager

If the *Redirect USB Device* menu item is greyed out, check that the following hardware is configured for the VM:

- A USB Controller.
- One or more USB Redirectors.

Error starting domain: Requested operation is not valid

When you try to open a virtual machine this error may pop up. This is because when you try to open a existing virtual machine libvirt tries to search for the default network which is not available. To make it available you have to autostart your network interface so that whenever your restart your computer your network interface is always active. See libvirt networking page.

Look at the name of your network interface with the following command:

```
# virsh net-list --all
```

To autostart your network interface:

```
# virsh net-autostart name_of_the_network
```

To start your network interface:

```
# virsh net-start name_of_the_network
```

Virt Manager Error 'Virt Manager doesn't have search permissions'

Ensure the folder containing your virtual machine files and installation ISO are owned by the `libvirt-qemu` group

```
$ sudo chown -R $USER:libvirt-qemu /path/to/virtual/machine
```

启动域时错误：请求的操作无效：网络 'default' 未激活

若默认网络因故处于未激活状态，所有配置使用该网络的虚拟机将无法启动。初步解决措施可尝试通过 virsh 手动启动网络：

```
# virsh net-start default
```

若问题仍未解决，请参考网络故障排查指南

参阅

- libvirt 网站
- Red Hat 虚拟化部署和管理指南
- Red Hat 虚拟化调优指南
- Slackware KVM and libvirt
- IBM KVM
- libvirt 网络手册

6.11 使用 OVMF 直通 PCI

难度 高级 | **篇幅** 71.5 KB | **所属** 虚拟化

本节将讲到

- 硬件要求
- 设置IOMMU
- 隔离GPU
- 设置 OVMF 虚拟机
- 性能调整

开放虚拟机固件（Open Virtual Machine Firmware, OVMF)是一个为虚拟机启用 UEFI 支持的项目。Linux 内核 3.9 之后的版本 和近期版本的 QEMU ，支持将显卡直通给虚拟机，让虚拟机在执行图形密集型任务的时候有接近实体机的图形性能。

如果你可以为宿主机准备一张备用的显卡（无论是集成显卡还是独立显卡，还是旧的 OEM 卡都可以，品牌不需要一致），并且你的硬件支持 PCI 直通（参见 #硬件要求），那么任何操作系统的虚拟机都可以使用一张专用显卡并得到接近于原生的图形性能。请参阅这篇演讲(PDF)了解有关技术的更多信息。

硬件要求

VGA直通依赖许多功能。这些功能尚未成为所有硬件的标配，所以您的硬件可能并不支持。如果要使用VGA直通，您的硬件必须满足如下条件：

- 您的CPU必须支持硬件虚拟化（为了使用 kvm）和 IOMMU（为了使用 VGA 直通）。
- Intel CPU 支持列表（VT-x 和 VT-d）
- 推土机架构及以后的所有的AMD CPU（包括 Zen）都应支持。
- K10架构（2007）的 CPU 不具备 IOMMU，你需要一张890FX或者990FX芯片组的主板来使用 IOMMU（这些主板有自己的IOMMU支持）。
- 您的主板必须支持 IOMMU
- 芯片组和BIOS必须都支持IOMMU。确定是否支持可能不是十分容易（比如，主板说明书一般不会提到这些），但是在 Xen wiki 和 Wikipedia:List of IOMMU-supporting hardware 上有一个相当全面的支持列表。
- 分配给客户机的 GPU 的 ROM 必须支持 UEFI
- 如果你可以在这个列表中找到适用您显卡的 ROM，就代表您的显卡可以支持 UEFI。所有2012年之后的显卡都应该支持 UEFI，因为微软将 UEFI 支持作为 Windows 8 兼容认证的必须条件。

您可能需要一台备用或者支持多个输入的显示器连接到显卡。如果没有连接显示器，那么直通的显卡不会有任何输出，并且使用 Spcie 或是 VNC 连接到虚拟机并不会让直通的显卡工作，换言之，没有任何使用 VGA 直通所带来的性能提升。您可能还需

要准备一套备用的键鼠或是能通过 SSH 连接到宿主机的设备，如果虚拟机的配置出现了什么问题，您至少还可以控制宿主机。

设置IOMMU

注意：

- IOMMU 是 Intel VT-d 和 AMD-Vi 的通用名称。
- VT-d 指的是直接输入/输出虚拟化(Intel Virtualization Technology for Directed I/O)，不应与VT-x(x86平台下的Intel虚拟化技术，Intel Virtualization Technology)混淆。VT-x 可以让一个硬件平台作为多个“虚拟”平台，而 VT-d 提高了虚拟化的安全性、可靠性和 I/O 性能。

打开IOMMU之后可以使用 PCI Passthrough 和对于故障和恶意行为的内存保护。参见：wikipedia:Input-output memory management unit#Advantages 以及 Memory Management (computer programming): Could you explain IOMMU in plain English?。

启用IOMMU

确保您的 CPU 支持 AMD-Vi/Intel Vt-d 并且已经在 BIOS 中打开。通常这个选项会在类似“其他 CPU 特性”的菜单里，也有可能隐藏在超频选项之中。选项可能就叫做 “VT-d” 或者 “AMD-Vi” ，也有可能是更通用的名称，比如“虚拟化技术”之类。有可能您主板的手册并不会解释这些。

设置内核参数以启用 IOMMU，注意不同品牌的 CPU 所需的内核参数并不同。

- 对于 Intel CPU(VT-d)，使用 `intel_iommu=on`。因为内核参数 CONFIG_INTEL_IOMMU_DEFAULT_ON在linux包中并未启用。
- 对于 AMD CPU(AMD-Vi)，当内核检测到硬件支持IOMMU时就会自动启用。

您同时需要设置`iommu=pt`，这将防止Linux试图接触(touching)无法直通的设备。

在重启之后，检查 dmesg 以确认 IOMMU 已经被正确启用：

```
dmesg | grep -e DMAR -e IOMMU
[    0.000000] ACPI: DMAR 0x00000000BDCB1CB0 0000B8 (v01 INTEL  BDW      00000001 INTL
00000001)
[    0.000000] Intel-IOMMU: enabled
[    0.028879] dmar: IOMMU 0: reg_base_addr fed90000 ver 1:0 cap c0000020660462 ecap
f0101a
[    0.028883] dmar: IOMMU 1: reg_base_addr fed91000 ver 1:0 cap d2008c20660462 ecap
f010da
[    0.028950] IOAPIC id 8 under DRHD base  0xfed91000 IOMMU 1
[    0.536212] DMAR: No ATSR found
[    0.536229] IOMMU 0 0xfed90000: using Queued invalidation
[    0.536230] IOMMU 1 0xfed91000: using Queued invalidation
[    0.536231] IOMMU: Setting RMRR:
[    0.536241] IOMMU: Setting identity map for device 0000:00:02.0 [0xbf000000 -
0xcf1fffff]
[    0.537490] IOMMU: Setting identity map for device 0000:00:14.0 [0xbdea8000 -
0xbdeb6fff]
```

```
[    0.537512] IOMMU: Setting identity map for device 0000:00:1a.0 [0xbdea8000 -
0xbdeb6fff]
[    0.537530] IOMMU: Setting identity map for device 0000:00:1d.0 [0xbdea8000 -
0xbdeb6fff]
[    0.537543] IOMMU: Prepare 0-16MiB unity mapping for LPC
[    0.537549] IOMMU: Setting identity map for device 0000:00:1f.0 [0x0 - 0xffffff]
[    2.182790] [drm] DMAR active, disabling use of stolen memory
```

确保组有效

这个脚本将输出您的 PCI 设备是如何被分配到 IOMMU 组之中的。如果这个脚本没有输出任何东西，这代表您并没有启用 IOMMU 支持或者是您的硬件不支持 IOMMU。

```
#!/bin/bash
shopt -s nullglob
for d in /sys/kernel/iommu_groups/*/devices/*; do
    n=${d#*/iommu_groups/*}; n=${n%%/*}
    printf 'IOMMU Group %s ' "$n"
    lspci -nns "${d##*/}"
done
```

例子：

```
IOMMU Group 1:
        00:01.0 PCI bridge: Intel Corporation Xeon E3-1200 v2/3rd Gen Core processor PCI
Express Root Port [8086:0151] (rev 09)
IOMMU Group 2:
        00:14.0 USB controller: Intel Corporation 7 Series/C210 Series Chipset Family
USB xHCI Host Controller [8086:0e31] (rev 04)
IOMMU Group 4:
        00:1a.0 USB controller: Intel Corporation 7 Series/C210 Series Chipset Family
USB Enhanced Host Controller #2 [8086:0e2d] (rev 04)
IOMMU Group 10:
        00:1d.0 USB controller: Intel Corporation 7 Series/C210 Series Chipset Family
USB Enhanced Host Controller #1 [8086:0e26] (rev 04)
IOMMU Group 13:
        06:00.0 VGA compatible controller: NVIDIA Corporation GM204 [GeForce GTX 970]
[10de:13c2] (rev a1)
        06:00.1 Audio device: NVIDIA Corporation GM204 High Definition Audio Controller
[10de:0fbb] (rev a1)
```

一个 IOMMU 组是将物理设备直通给虚拟机的最小单位（这不意味着您必须直通整个 USB 控制器，单独的 USB 设备，如键盘鼠标，可以单独设置直通）。例如，在上面的例子中，在 06:00.0 上的显卡和在6:00.1上的音频控制器被分配到13组，它们必须一起直通给虚拟机。前置 USB 控制器被分在了单独的一组（第2组），并不和 USB 扩展控制器（第10组）和后置USB控制器（第4组）分在一组，这意味着它们都可以在不影响其它设备的情况下直通给虚拟机。

注意

如果客户机所用显卡插在 CPU 提供的 PCI-E 插槽中

并非所有的PCI-E插槽均相同。许多主板同时具有 CPU 提供和 PCH 提供的 PCI-E 插槽。根据 CPU 不同，可能您 CPU 提供的 PCI-E 插槽并不能正确隔离。在这种情况下，PCI-E 插槽本身和所连接设备将分在同一组。

```
IOMMU Group 1 00:01.0 PCI bridge: Intel Corporation Xeon E3-1200 v2/3rd Gen Core
processor PCI Express Root Port (rev 09)
IOMMU Group 1 01:00.0 VGA compatible controller: NVIDIA Corporation GM107 [GeForce GTX
750] (rev a2)
IOMMU Group 1 01:00.1 Audio device: NVIDIA Corporation Device 0fbc (rev a1)
```

如果上一节的脚本输出类似这样，那么您的 PCI-E 插槽就属于上面所述的情况。根据您在 PCI-E 插槽上连接的设备和PCI-E插槽是由PCH还是CPU提供的，您可能发现在需要被直通的组中还有其它的设备，您必须将整个组中的设备全部直通给虚拟机。如果您愿意这样做的话，那么就可以继续。否则，您需要尝试将显卡插入其他的 PCI-E 插槽（如果有的话）并查看这些插槽是否能正确隔离。您也可以安装 ACS 补丁来绕过这个限制，不过这也有相应的缺点。要获得关于 ACS 补丁的更多信息，请参阅#分离IOMMU组(ACS补丁))

隔离GPU

为了将设备分配给虚拟机，此设备和同在一个IOMMU组的所有设备必须将驱动程序更换为 stub 或是 vfio ，以防止宿主机尝试与其交互。对于大多数设备，在虚拟机启动时就可以完成这项工作。

但是，由于它们的大小和复杂性，GPU 驱动程序并不倾向于支持动态的重新绑定。所以一般您不能将宿主机所使用的显卡直通给虚拟机，这会导致驱动程序之间的冲突。因此，通常情况下建议在启动虚拟机之前手动绑定这些占位 (placeholder) 驱动，防止其他驱动程序尝试认领 (claim) 设备。

以下部分详细地介绍了如何配置GPU，以便在启动过程中尽早绑定这些占位 (placeholder) 驱动程序，这会使设备处于非活动状态，直到虚拟机认领 (claim) 设备或是切换到其他驱动程序。这是首选方法，因为相比于系统完全启动之后再切换驱动程序，这种方法要简单许多。

警告：在配置完成并重新启动之后，无论您配置的 GPU 是什么，它都将无法在宿主机上使用，直到您取消配置。在执行操作之前，确保您要在宿主机上使用的GPU 已经被正确配置，并且您也应该在BIOS中将宿主机所使用的GPU设为主要GPU。

从 Linux 4.1 开始，内核包括了 `vfio-pci` 。这是一个 VFIO 驱动程序，与 `pci-stub` 的作用相同。但它可以在一定程度上控制设备，例如在不使用设备的时候将它们切换到 D3 状态。

通过设备ID绑定vfio-pci

`vfio-pci`通常通过 ID 来定位 PCI 设备，这意味着您只需要指定设备的ID就可以完成绑定。对于下面的 IOMMU 组，您可能希望将`vfio-pci`与 `10de:13c2` 和 `10de:0fbb` 绑定，这将作为本章节其余部分的示例值。

IOMMU Group 13 06:00.0 VGA compatible controller: NVIDIA Corporation GM204 \[GeForce GTX 970\] \[10de:13c2\] (rev a1) IOMMU Group 13 06:00.1 Audio device: NVIDIA Corporation GM204 High Definition Audio Controller \[10de:0fbb\] (rev a1)}}

注意：

- 如果宿主机和客户机所使用的GPU设备ID相同（例如是同一型号）那么就不能使用供应商-设备ID来指定需要隔离的设备。如果是这种情况，参见 #宿主机和客户机使用相同的GPU。
- 音频设备（例如上面的10de:0fbb）不是必须绑定的，Libvrt能够自动将其从`snd_hda_intel`解绑

有两种方法可以实现设备ID与vfio-pci绑定。

方法一：在内核启动参数中进行绑定。

通过内核启动参数进行绑定，能够方便地编辑、删除、撤销这类具有破坏性的配置。参考内核参数加入以下命令：

```
vfio-pci.ids=10de:13c2,10de:0fbb
```

注意：

- 如果你使用nvidia驱动，并且对其进行了modesetting，则`vfio-pci.ids`必须通第二种方式进行绑定。第一种方法会导致参数生效过晚从而绑定失败。

方法二：通过载入模块进行绑定。

您需要在`/etc/modprobe.d/`创建`vfio.conf`，并将编辑内容如下：

```
/etc/modprobe.d/vfio.conf
options vfio-pci ids=10de:13c2,10de:0fbb
```

更改过后需要重新生成initramfs

注意：

- 像上面提到过的一样，#如果客户机所用显卡插在 CPU 提供的 PCI-E 插槽中，并且 PCI 根设备是 IOMMU 组的一部分，不要将根设备的 ID 传递给`vfio-pci`，因为他需要连接到宿主机以保持正常运行。但是，该组中的任何其它设备都应该绑定到`vfio-pci`。

提前加载vfio-pci

mkinitcpio

因为Arch的linux包内核将vfio-pci作为一个模块，我们需要强迫vfio-pci在显卡驱动抢占硬件之前加载。为了达成这一目的，我们向mkinitcpio中添加 `vfio_pci`, `vfio`, `vfio_iommu_type1`, `vfio_virqfd`：

注意：

- 从内核 6.2 版本开始，vfio_virqfd 的功能已经并入 vfio 模块

```
/etc/mkinitcpio.conf
MODULES=(... vfio_pci vfio vfio_iommu_type1 vfio_virqfd ...)
```

注意：

- 如果你在其中加载了其他驱动来进行modesetting（例如`nouveau`, `radeon`, `amdgpu`, `i915`等）则上述vfio模组必须在他们之前加载

另外，确保 modconf hook 在`mkinitcpio.conf`的 HOOKS 列表中：

```
/etc/mkinitcpio.conf
HOOKS=(... modconf ...)
```

由于新模块已添加到 initramfs 配置中，因此必须重新生成initramfs。若在`/etc/modprobe.d/vfio.conf`修改了设备ID，您同样需要重新生成它。之后重启系统。

booster

与mkinitcpio类似，你需要自定义模组来提前加载：

```
/etc/booster.yaml
modules_force_load: vfio_pci,vfio,vfio_iommu_type1,vfio_virqfd
```

之后重新生成initramfs

dracut

与上面的思路类似， 在 `/etc/dracut.conf.d`添加以下文件：

```
10-vfio.conf
force_drivers+=" vfio_pci vfio vfio_iommu_type1 vfio_virqfd "
```

注意到我们使用`force_drivers` 而不是常规的 `add_drivers` 选项, 这能保证驱动程序能通过modprobe提前加载 (Dracut#Early kernel module loading).

与mkinitcpio类似的，你需要重新生成initramfs. 详见dracut。

验证`vfio-pci`是否已经正确加载并绑定到正确的设备

```
$ dmesg | grep -i vfio
[    0.329224] VFIO - User Level meta-driver version: 0.3
[    0.341372] vfio_pci: add [10de:13c2[ffff:ffff]] class 0x000000/00000000
[    0.354704] vfio_pci: add [10de:0fbb[ffff:ffff]] class 0x000000/00000000
[    2.061326] vfio-pci 0000:06:00.0: enabling device (0100 -> 0103)
```

`vfio.conf`中所绑定的设备（甚至是需要直通的设备）并不一定会出现在 dmesg 的输出之中。有时尽管设备没有出现在 dmesg 的输出中，但实际上在虚拟机中可以被使用。

```
$ lspci -nnk -d 10de:13c2
06:00.0 VGA compatible controller: NVIDIA Corporation GM204 [GeForce GTX 970]
[10de:13c2] (rev a1)
        Kernel driver in use: vfio-pci
        Kernel modules: nouveau nvidia
$ lspci -nnk -d 10de:0fbb
06:00.1 Audio device: NVIDIA Corporation GM204 High Definition Audio Controller
[10de:0fbb] (rev a1)
        Kernel driver in use: vfio-pci
        Kernel modules: snd_hda_intel
```

设置 OVMF 虚拟机

OVMF是一个为QEMU虚拟机开发的开源UEFI固件，虽然您可以使用 SeaBIOS 来获得和 PCI 直通类似的结果，但设置过程并不相同。如果您的硬件支持，则最好使用EFI方法。

配置libvirt

Libvirt 是许多虚拟化实用程序的包装，可以极大的简化虚拟机的配置和部署过程。对于 KVM 和 QEMU ，它提供的前端让我们避免处理 QEMU 的权限，并使得在现有的虚拟机上添加和删除设备更加容易。但是，作为一个包装，它可能并不总是支持最新的 QEMU 功能，最终可能需要一些提供的脚本为 QEMU 传递一些额外的参数。

安装qemu包\[损坏的链接：package not found\]、libvirt包、ovmf包\[损坏的链接：replaced by edk2-ovmf包\]和virt-manager包之后，将您的 OVMF 固件映像和运行时变量模板添加到 libvirt 配置中，以便让 `virt-install`或是`virt-manager` 可以找到它们。

```
/etc/libvirt/qemu.conf
nvram = [
       "/usr/share/ovmf/x64/OVMF_CODE.fd:/usr/share/ovmf/x64/OVMF_VARS.fd"
]
```

您现在可以启用并启动`libvirtd.service`和它的日志记录组件`virtlogd.socket`。

安装客户机系统

使用 `virt-manager` 配置虚拟机的大部分过程都无需指导，只要按照屏幕上的提示即可。

如果使用`virt-manager`，则必须将用户添加到 `libvirt` 组。

但是，您仍应该特别注意如下步骤：

- 在虚拟机创建向导要求您命名虚拟机时（点击“完成”前的最后一步），勾选“在安装前自定义配置”。
- 在“概况”屏幕，将“固件”选为"UEFI"，如果您不能选择它，检查是否在`/etc/libvirt/qemu.conf`正确配置了固件的路径并且重新启动了`libvirtd.service`。
- 在“CPUs”屏幕，将CPU型号改为"`host-passthrough`"。如果它不在列表中，则您必须手动输入。这确保您的CPU会被正确检测，从而能使 libvirt 完全暴露您 CPU 的功能，而不是仅仅是它所识别到的那些（这会让 CPU 按默认行为运作）。如果不进行这项设置，某些程序可能会抱怨说您的 CPU 型号是未知的。
- 如果要最小化IO开销，请点击“添加硬件”，并在“控制器：中选择“SCSI”类型，型号为 "VirtIO SCSI"。你可以将默认的 IDE 磁盘改为 SCSI 磁盘，并绑定到上述的控制器。

- Windows 不包含VirtIO驱动程序，所以你需要从这里下载包含驱动程序的 ISO 并且添加一个IDE CDROM（Windows 7之后可以使用SATA）并且连接到刚才的 ISO 。否则在安装过程中 Windows 无法识别 VirtIO 控制器。当 Windows 安装程序要求您选择要安装的磁盘时，加载 CD-ROM 下 visscsi 目录下的驱动程序。

其余的安装过程将使用在窗口中运行的标准QXL图形适配器进行。此时，无需为其余虚拟设备安装驱动程序，因为一些虚拟设备将被删除。客户机操作系统安装完成之后，只需要关闭虚拟机即可。您还可能会直接进入UEFI菜单，而不是自动从安装程序引导。客户机在启动的时候可能并未检测到正确的ISO文件，您需要手动指定引导顺序。点击“exit”并选择“boot manager”，您将会进入一个选择引导设备的菜单。

附加PCI设备

当安装完成之后，就可以编辑libvirt中的硬件详情并且删除一些虚拟设备。例如spcie通道和虚拟显示器，QXL图形适配器，虚拟鼠标键盘以及数位板设备。由于您没有可用的输入设备，您可能还想将几个主机上的USB设备附加到虚拟机。但请记住，**至少为宿主机留下一个鼠标和/或键盘**，防止客户机出现问题的时候无法操作宿主机。此时，可以添加先前隔离的PCI设备。只需单击“添加硬件”然后选择需要使用的PCI主机设备。如果一切顺利，连接到GPU的显示器上应该会显示OVMF启动画面，并可以正常启动。您可以在这时安装其他的驱动程序。

注意

OVMF虚拟机不能引导非EFI镜像

OVMF固件不支持启动非EFI介质。如果启动虚拟机之后您直接看到了UEFI Shell，可能是您的引导介质无效。尝试使用其他的引导映像来确定问题。

性能调整

使用PCI直通多数涉及性能密集型领域，例如游戏或是GPU加速任务。虽然PCI直通本身就是实现原生性能的一步，但宿主机和客户机仍可以通过一些调整来提升性能。

CPU核心固定

默认情况下，KVM将虚拟机的操作作为虚拟处理器的多个线程运行（The default behavior for KVM guests is to run operations coming from the guest as a number of threads representing virtual processors）。这些线程由Linux调度程序管理，如同其他线程一样。并根据 niceness 和 priority 分配给任何可用的CPU核心。因此，当线程切换到另一个核心时，核心的高速缓存将无法发挥作用，这可能会显著影响虚拟机的性能。CPU核心固定旨在解决这些问题，因为它会忽略Linux的线程调度并确保虚拟机

的线程始终在特定的内核上运行。例如客户机的核心 0,1,2,3 分别映射到宿主机的 4,5,6,7 核心。

注意：某些启用CPU核心固定的用户可能会遇到卡顿和短暂挂起的问题。尤其是在使用MuQSS调度程序的情况下（存在与linux-ck内核和linux-zen内核）。如果遇到类似的问题，您可能需要首先禁用固定，保证始终具有最大的即时响应性能。

CPU 拓扑

大多数现代CPU都支持硬件多任务处理，即 Intel CPU 上的超线程或 AMD CPU 上的 SMT。 超线程/SMT 让一个物理核心具有两个虚拟线程。您需要根据虚拟机和宿主机的用途来设置 CPU 核心固定。

要查看您的 CPU 拓扑，运行 `lscpu -e`：

注意：需要特别注意 **"CORE"** 栏，它表明了虚拟核心和物理核心的对应关系。

6c/12t Ryzen 5 1600 上的 `lscpu -e` 输出：

```
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE MAXMHZ    MINMHZ
0   0    0      0    0:0:0:0       yes    3800.0000 1550.0000
1   0    0      0    0:0:0:0       yes    3800.0000 1550.0000
2   0    0      1    1:1:1:0       yes    3800.0000 1550.0000
3   0    0      1    1:1:1:0       yes    3800.0000 1550.0000
4   0    0      2    2:2:2:0       yes    3800.0000 1550.0000
5   0    0      2    2:2:2:0       yes    3800.0000 1550.0000
6   0    0      3    3:3:3:1       yes    3800.0000 1550.0000
7   0    0      3    3:3:3:1       yes    3800.0000 1550.0000
8   0    0      4    4:4:4:1       yes    3800.0000 1550.0000
9   0    0      4    4:4:4:1       yes    3800.0000 1550.0000
10  0    0      5    5:5:5:1       yes    3800.0000 1550.0000
11  0    0      5    5:5:5:1       yes    3800.0000 1550.0000
```

6c/12t Intel 8700k 上的 `lscpu -e` 输出：

```
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE MAXMHZ    MINMHZ
0   0    0      0    0:0:0:0       yes    4600.0000 800.0000
1   0    0      1    1:1:1:0       yes    4600.0000 800.0000
2   0    0      2    2:2:2:0       yes    4600.0000 800.0000
3   0    0      3    3:3:3:0       yes    4600.0000 800.0000
4   0    0      4    4:4:4:0       yes    4600.0000 800.0000
5   0    0      5    5:5:5:0       yes    4600.0000 800.0000
6   0    0      0    0:0:0:0       yes    4600.0000 800.0000
7   0    0      1    1:1:1:0       yes    4600.0000 800.0000
8   0    0      2    2:2:2:0       yes    4600.0000 800.0000
9   0    0      3    3:3:3:0       yes    4600.0000 800.0000
10  0    0      4    4:4:4:0       yes    4600.0000 800.0000
11  0    0      5    5:5:5:0       yes    4600.0000 800.0000
```

如上所示，对于上面的 AMD CPU，CPU 0 和 CPU 1 对应 CORE 0，对于上面的英特尔 CPU，CPU 0 和 CPU 6 对应 CORE 0。

如果您不需要虚拟机使用所有核心，那么最好给宿主机留下一个核心。选择要用于宿主机或虚拟机的核心应该基于CPU的特定硬件特性，但在大多数情况下，“CORE0”是宿主机的不错选择。如果为宿主机保留了任何核心，建议将模拟器和 iothreads（如果使用）固定到宿主机核心而不是虚拟 CPU 上。这可以提高性能并减少虚拟机的延

迟，因为这些线程不会污染缓存或争抢虚拟 CPU 线程调度。如果虚拟机需要使用所有核心，那么固定模拟器或 iothreads 是没有必要的。

XML 示例

注意：如果你的磁盘控制器没有启用 **iothread**，那就不要使用下面例子中的 **iothread**。**iothread**只在 **virtio-scsi** 或是 **virtio-blk** 设备上工作。

4c/1t 无超线程的例子

```
$ virsh edit [vmname]
...
<vcpu placement='static'>4</vcpu>
<cputune>
    <vcpupin vcpu='0' cpuset='0'/>
    <vcpupin vcpu='1' cpuset='1'/>
    <vcpupin vcpu='2' cpuset='2'/>
    <vcpupin vcpu='3' cpuset='3'/>
</cputune>
...
```

6c/2t Intel CPU 固定的例子

```
$ virsh edit [vmname]
...
<vcpu placement='static'>8</vcpu>
<iothreads>1</iothreads>
<cputune>
    <vcpupin vcpu='0' cpuset='2'/>
    <vcpupin vcpu='1' cpuset='8'/>
    <vcpupin vcpu='2' cpuset='3'/>
    <vcpupin vcpu='3' cpuset='9'/>
    <vcpupin vcpu='4' cpuset='4'/>
    <vcpupin vcpu='5' cpuset='10'/>
    <vcpupin vcpu='6' cpuset='5'/>
    <vcpupin vcpu='7' cpuset='11'/>
    <emulatorpin cpuset='0,6'/>
    <iothreadpin iothread='1' cpuset='0,6'/>
</cputune>
    ...
    <topology sockets='1' cores='4' threads='2'/>
    ...
```

4c/2t AMD CPU 的例子

```
$ virsh edit [vmname]
...
<vcpu placement='static'>8</vcpu>
<iothreads>1</iothreads>
<cputune>
  <vcpupin vcpu='0' cpuset='2'/>
  <vcpupin vcpu='1' cpuset='3'/>
  <vcpupin vcpu='2' cpuset='4'/>
  <vcpupin vcpu='3' cpuset='5'/>
  <vcpupin vcpu='4' cpuset='6'/>
  <vcpupin vcpu='5' cpuset='7'/>
  <vcpupin vcpu='6' cpuset='8'/>
  <vcpupin vcpu='7' cpuset='9'/>
  <emulatorpin cpuset='0-1'/>
  <iothreadpin iothread='1' cpuset='0-1'/>
</cputune>
    ...
    <topology sockets='1' cores='4' threads='2'/>
    ...
```

注意：如果你需要进一步的 CPU 隔离，考虑在未使用的物理/虚拟核心上使用 **isolcpus** 内核选项。

如果您不打算在使用虚拟机时在宿主机上进行计算繁重的工作（或是根本不打算在宿主机上做任何事情），您可以将虚拟线程固定在所有核心上以便充分使用 CPU 时间。不过，固定所有物理和虚拟核心可能会导致虚拟机出现延迟。

内存大分页

在处理需要大量内存的应用程序时，内存延迟可能会是一个问题，因为使用的内存分页越多，程序尝试跨分页访问信息的可能性就越高（页是基本的内存分配单位）。将分页解析为实际的内存地址需要多个步骤，因此CPU通常将信息缓存在最近使用的分页上，以加快后续的内存使用。使用大量内存的应用程序可能会遇到内存性能问题：例如，虚拟机使用 4GB 内存，分页大小为 4KB（这是普通分页的默认大小），总共104万页，这意味着缓存命中率可能会大幅降低并大幅增加内存延迟。大分页通过向应用程序提供更大的单个分页来缓解这个问题，从而增加了多个操作中连续位于同一分页的几率。

透明大分页

QEMU 将在 QEMU 或 Libvirt 中自动使用 2 MiB 大小的透明大分页，但这可能并不那么顺利，在使用 VFIO 时，页面会在启动时锁定，并且在虚拟机首次启动时会预先分配透明的大分页。如果内存高度碎片化，或者虚拟机正在使用大部分剩余内存，则内核可能没有足够的2 MiB 页面来完全满足分配。在这种情况下，将会混合使用使用 2 MiB 和 4 KiB 分页，从而无法得到足够的性能提升。由于分页在 VFIO 模式下被锁定，因此内核无法在虚拟机启动后将这些 4 KiB 分页转换为大分页。THP（透明大分页）可用的 2 MiB 大页面数量与通过以下各节中描述的#内存大分页机制相同。

要查看全局使用的 THP 内存量：

```
$ grep AnonHugePages /proc/meminfo
AnonHugePages:   8091648 kB
```

要查看特定 QEMU 实例所使用的 THP ，需要指定 QEMU 的 PID：

```
$ grep -P 'AnonHugePages:\s+(?!0)\d+' /proc/[PID]/smaps
AnonHugePages:   8087552 kB
```

在这个例子中，虚拟机分配了 8388608 KiB 的内存，但只有 8087552 KiB 可通过 THP 获得。 剩下的301056KiB被分配为 4 KiB 分页。没有任何警告提示使用了 4 KiB 分页。因此，THP 的有效性在很大程度上取决于虚拟机启动时宿主机系统的内存碎片。如果这是不可接受的，建议使用#内存大分页。

Arch 内核已经编译并默认启用了 THP，默认为 `madvise` 模式。可在 `/sys/kernel/mm/transparent_hugepage/enabled` 查看。

Static huge pages

While transparent huge pages should work in the vast majority of cases, they can also be allocated statically during boot. This should only be needed to make use 1 GiB hugepages on machines that support it, since transparent huge pages normally only go up to 2 MiB.

警告：Static huge pages lock down the allocated amount of memory, making it unavailable for applications that are not configured to use them. Allocating 4 GiBs worth of huge pages on a machine with 8 GiB of memory will only leave you with 4 GiB of available memory on the host **even when the VM is not running**.

To allocate huge pages at boot, one must simply specify the desired amount on their kernel command line with `hugepages=*x*`. For instance, reserving 1024 pages with `hugepages=1024` and the default size of 2048 KiB per huge page creates 2 GiB worth of memory for the virtual machine to use.

If supported by CPU page size could be set manually. 1 GiB huge page support could be verified by `grep pdpe1gb /proc/cpuinfo`. Setting 1 GiB huge page size via kernel parameters : `default_hugepagesz=1G hugepagesz=1G hugepages=X`.

Also, since static huge pages can only be used by applications that specifically request it, you must add this section in your libvirt domain configuration to allow kvm to benefit from them :

```
$ virsh edit [vmname]
...
<memoryBacking>
      <hugepages/>
</memoryBacking>
...
```

Dynamic huge pages

**本文或本章节的事实准确性存在争议。**

原因： Need futher testing if this variant as effective as static one（在 Talk:使用 OVMF 直通 PCI 中讨论）

Hugepages could be allocated manually via `vm.nr_overcommit_hugepages` sysctl parameter.

```
/etc/sysctl.d/10-kvm.conf
vm.nr_hugepages = 0
vm.nr_overcommit_hugepages = num
```

Where `*num*` - is the number of huge pages, which default size if 2 MiB. Pages will be automatically allocated, and freed after VM stops.

More manual way:

```
# echo num > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
# echo num > /sys/kernel/mm/hugepages/hugepages-1048576kB/nr_hugepages
```

For 2 MiB and 1 GiB page size respectively. And they should be manually freed in the same way.

It is hardly recommended to drop caches, compact memory and wait couple of seconds before starting VM, as there could be not enough free contiguous memory for required huge pages blocks. Especially after some uptime of the host system.

```
# echo 3 > /proc/sys/vm/drop_caches
# echo 1 > /proc/sys/vm/compact_memory
```

Theoretically, 1 GiB pages works as 2 MiB. But practically - no guaranteed way was found to get contiguous 1 GiB memory blocks. Each consequent request of 1 GiB blocks lead to lesser and lesser dynamically allocated count.

CPU调速器

根据您CPU 调速器的配置方式，虚拟机的线程可能无法达到负载阈值以提升频率。实际上，KVM 自身无法更改 CPU 频率，如果 CPU 频率并不能随着虚拟 CPU 的负载使用而提升，这可能是一个严重的性能问题。检查此问题最简单的方法是在虚拟机上运行 CPU 密集型任务的同时在宿主机上运行`watch lscpu`检查CPU频率是否上升。如果您的 CPU 频率无法正常上升且无法达到所报告的最大值，可能是由于主机操作系统限制了 CPU 升频。这种情况下，尝试手动将所有CPU内核设为最大频率查看是否能够提升性能。请注意，如果您使用带有默认pstate驱动程序的现代Intel芯片，则cpupower命令无效\[损坏的链接：无效的章节\]，因此请监视`/proc/cpuinfo`确保您的CPU处于最大频率。

高DPC延迟

**本文或本章节的事实准确性存在争议。**

原因： As far as I can tell all virtio modules listed here are for virtual devices used when Linux runs as a guest. Loading them on the host serves no purpose.（在 Talk:使用 OVMF 直通 PCI 中讨论）

如果您在虚拟中遇到了高DPC和/或中断延迟，请确保已在主机内核上加载了所需的VirtIO内核模块，可加载的VirtIO内核模块包括：`virtio-pci`，`virtio-net`，`virtio-blk`，`virtio-balloon`，`virtio-ring`和`virtio`。

加载一个或多个这些模块后，在宿主机上执行`lsmod | grep virtio`不应返回空。

使用isolcpus固定CPU核心

此外，确保您已正确隔离CPU。 在本例中，我们假设您使用的是CPU 4-7。 使用内核参数`isolcpus nohz_full rcu_nocbs`将CPU与内核完全隔离。

```
sudo vim /etc/defaults/grub
...
GRUB_CMDLINE_LINUX="..your other params.. isolcpus=4-7 nohz_full=4-7 rcu_nocbs=4-7"
...
```

然后，用taskset和chrt运行`qemu-system-x86_64`：

```
# chrt -r 1 taskset -c 4-7 qemu-system-x86_64 ...
```

`chrt`命令将确保任务调度程序将循环分配工作（否则它将全部停留在第一个cpu上）。 对于`taskset`，CPU编号可以用“，”和“/”或"-"分隔，
如“0,1,2,3”或“0-4”或“1,7-8,10”等。

参考reddit上的这篇帖子了解详情。

提高AMD CPU的性能

以前，由于一个非常老的bug，必须在运行 KVM 的 AMD 系统上禁用嵌套页表（NPT）以提高 GPU 性能，但这将降低 CPU 性能，导致卡顿。

有一个内核补丁可以解决这个问题，它已经在4.14-stable和4.9-stable 版本被接受了。 如果你正在运行官方的linux包或linux-lts包内核，那么该补丁被已经应用了（请确保你使用的是最新内核）。如果您正在运行另一个内核，您可能需要手动应用补丁。

注意：几个Ryzen用户（请参阅这个Reddit帖子）已经测试了该补丁，并且可以确认它有效，使GPU直通性能接近原生性能。

进一步调整

Red Hat的虚拟化调试和优化指南（中文）中提供了更专业的VM调优技巧。 如果您遇到以下情况，本指南可能特别有用：

- 在从虚拟中下载/上传期间，主机CPU负载较高，请参阅“桥接零复制传输”了解可能的修复。
- 尽管使用了virtio-net，但是在下载/上传期间，访客网络速度有限制。请参阅“多队列virtio-net”以获得可能的修复。
- 在没有达到负载极限的情况下，虚拟机在高I/O下卡顿。请参阅“多队列virtio-scsi”以获得获得可能的修复。

特殊步骤

某些机器需要特定的配置调整才能正常工作。 如果您在的宿主机或虚拟机不能正常工作，请查看您的系统是否符合以下情况之一，并尝试相应地调整配置。

宿主机和客户机使用相同的型号的GPU

由于`vfio-pci`如何使用您的供应商和设备ID对来识别需要在启动时绑定哪个设备，如果您的两个GPU 具有相同的ID，您将无法让您的直通驱动程序只与一个绑定。这种情况下必须使用脚本完成配置，无论您使用哪个驱动程序，都需要使用`driver_override`机制由pci总线地址完成分配。

脚本示例

直通除了启动时所用GPU之外的所有GPU

在/usr/bin/vfio-pci-override.sh，创建一个脚本让vfio-pci绑定除了启动时所用GPU之外的所有GPU。

```
#!/bin/sh

for i in /sys/devices/pci*/*/boot_vga; do
	if [ $(cat "$i") -eq 0 ]; then
		GPU="${i%/boot_vga}"
		AUDIO="$(echo "$GPU" | sed -e "s/0$/1/")"
		echo "vfio-pci" > "$GPU/driver_override"
		if [ -d "$AUDIO" ]; then
			echo "vfio-pci" > "$AUDIO/driver_override"
		fi
	fi
done

modprobe -i vfio-pci
```

直通选定的GPU

手动指定需要绑定的GPU。

```
#!/bin/sh

GROUP="0000:00:03.0"
DEVS="0000:03:00.0 0000:03:00.1 ."

if [ ! -z "$(ls -A /sys/class/iommu)" ]; then
	for DEV in $DEVS; do
		echo "vfio-pci" > /sys/bus/pci/devices/$GROUP/$DEV/driver_override
	done
fi

modprobe -i vfio-pci
```

脚本安装

创建/etc/modprobe.d/vfio.conf，内容如下:

```
install vfio-pci /usr/bin/vfio-pci-override.sh
```

编辑/etc/mkinitcpio.conf:

从MODULES中移除所有图形驱动程序，并添加vfio-pci和vfio_iommu_type1

```
MODULES=(ext4 vfat vfio-pci vfio_iommu_type1)
```

将/etc/modprobe.d/vfio.conf和/usr/bin/vfio-pci-override.sh添加到FILES

```
FILES=(/etc/modprobe.d/vfio.conf /usr/bin/vfio-pci-override.sh)
```

重新生成initramfs并重新启动。

直通启动时所用的GPU

标记为boot_vga的GPU在进行PCI直通时比较特殊，BIOS需要使用它才能显示启动消息或BIOS配置菜单等内容。为了完成达到这个目的，需要创建一个可以被自由修改的VGA boot ROM 副本。这个副本可以让系统得知，但直通驱动程序可能会认为这是

一个非法的版本。如果可能的话，从BIOS设置中调整启动GPU。如果没有相关选项，调换宿主机和客户机GPU的位置。

使用Looking Glass将客户机画面流式传输到宿主机

通过使用Looking Glass，可以让宿主机和客户机共享显示器，同时将键盘鼠标动作传递给虚拟机。

将IVSHMEM设备添加到虚拟机

Looking glass通过在主机和来宾之间创建共享内存缓冲区来完成传输。这比通过localhost流式传输帧快得多，但需要额外的设置。

关闭你的虚拟机，修改配置：

```
$ virsh edit [vmname]
...
<devices>
    ...
  <shmem name='looking-glass'>
    <model type='ivshmem-plain'/>
    <size unit='M'>32</size>
  </shmem>
</devices>
...
```

您应该根据您要传输的分辨率将32替换为您自己所需要的值。计算方法如下：

```
宽 x 高 x 4 x 2 = 总比特数
总比特数 / 1024 / 1024 = 兆比特数 + 2
```

例如，分辨率为1920x1080

```
1920 x 1080 x 4 x 2 = 16,588,800 比特
16,588,800 / 1024 / 1024 = 15.82 MB + 2 = 17.82
```

实际上的数字应该是大于计算的得数的一个**2的幂**，因为17.82大于16，所以应该选择32。

接下来使用一个脚本创建共享内存文件。

```
/usr/local/bin/looking-glass-init.sh
#!/bin/sh

touch /dev/shm/looking-glass
chown user:kvm /dev/shm/looking-glass
chmod 660 /dev/shm/looking-glass
```

将`user`改为你的用户名。

记得为脚本授予可执行权限。

创建一个systemd单元文件，以便在开机时创建这个文件。

```
/etc/systemd/system/looking-glass-init.service
[Unit]
Description=Create shared memory for looking glass

[Service]
Type=oneshot
```

```
ExecStart=/usr/local/bin/looking-glass-init.sh

[Install]
WantedBy=multi-user.target
```

启动并启用 `looking-glass-init.service`。

将IVSHMEM主机安装到Windows客户机

目前Windows不会通知用户发现新的IVSHMEM设备，它会默默安装虚拟驱动程序。要实际启用该设备，您必须进入设备管理器并在“系统设备”下为“PCI标准RAM控制器”更新设备的驱动程序。因为它在其他地方尚未提供，所以必须从issue 217下载签名的驱动程序。

安装驱动程序后，您必须下载最新的looking-glass-host，然后在您的客户机上启动它。为了运行它，您还需要安装Microsoft Visual C ++ Redistributable。可以通过编辑`HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`注册表并添加路径来自动启动它。

安装客户端

Looking glass 客户端可以通过 looking-glassAUR或looking-glass-gitAUR 安装。

警告：如果你在使用looking-glass-gitAUR时出现问题，请选择looking-glassAUR。git版本并不受上游支持，可能在任何时候破损。只有带有版本号的发布受到支持，并且保证能与Windows客户机上的主机程序一起使用。

当客户机配置完成之后，您可以通过如下方式运行运行客户端：

```
$ looking-glass-client
```

如果您不想通过spcie传递键盘和鼠标的话，你可以禁用它：

```
$ looking-glass-client -s
```

您也可能希望全屏运行客户端，因为图像缩放会导致质量下降：

```
$ looking-glass-client -F
```

使用`--help`选项查看更多信息。

热切换外围设备

Looking Glass 包含一个spcie客户端，以便控制Windows客户机。然而，对于某些应用程序，例如游戏，可能延迟太高了。另一种方法是直通特定的USB设备以尽可能地降低延迟。可以在主机和客户机之间热切换设备。

首先为您想要直通的设备创建一个`.xml`文件，libvirt使用该文件来识别设备。

```
/home/$USER/.VFIOinput/input_1.xml
<hostdev mode='subsystem' type='usb' managed='no'>
<vendor id='0x[Before Colon]'/>
<product id='0x[After Colon]'/>
</hostdev>
```

将[before / After Colon]替换为lsusb命令的内容，取决于于您要传递的设备。

例如，我的鼠标是Bus 005 Device 002: ID 1532:0037 Razer USA, Ltd，所以我的vendor id 是 1532，product id 是 1037。

重复以上步骤以添加所有想要直通的USB设备。如果您的键盘/鼠标在lsusb中有多个条目，那么请为每个条目创建.xml文件。

注意：不要忘记更改上方和下方脚本的路径和名称，以匹配您的环境。

接下来需要创建一个bash脚本来告诉libvirt附加/分离设备。

```
/home/$USER/.VFIOinput/input_attach.sh
#!/bin/sh

virsh attach-device [VM-Name] [USBdevice]
```

将[VM-Name]替换为您的虚拟机名称，名称可以在virt-manager下看到。另外，将[USBdevice]替换为您希望传递的设备的.xml文件的完整路径。为多个设备添加额外的行。例如，这是我的脚本：

```
/home/ajmar/.VFIOinput/input_attach.sh
#!/bin/sh

virsh attach-device win10 /home/ajmar/.VFIOinput/input_mouse.xml
virsh attach-device win10 /home/ajmar/.VFIOinput/input_keyboard.xml
```

接下来复制脚本文件并用detach-device替换attach-device。并为每个脚本设置可执行权限。

现在可以执行这2个脚本，以将USB设备附加/分离到客户机。注意它们可能需要以root身份执行。要从Windows客户机运行脚本，您可以使用PuTTY SSH连接到主机，然后执行脚本。在Windows上，PuTTY附带了plink.exe，它可以通过SSH执行单一命令，然后自动关闭，而不是打开SSH终端。

```
detach_devices.bat
"C:\Program Files\PuTTY\plink.exe" root@$HOST_IP -pw $ROOTPASSWORD /home/
$USER/.VFIOinput/input_detach.sh
```

用宿主机IP 地址替换$HOST_IP，用root密码替换$ROOTPASSWORD。

警告：如果有人可以访问您的虚拟机，则此方法并不安全，因为他们可以打开文件并读取您的ROOT密码。建议使用SSH keys！

您可能还希望使用快捷键来执行脚本文件。在Windows上，您可以使用Autohotkey，在宿主机上可以使用Xbindkeys，您的桌面环境也可能提供快捷键设置。 由于需要以root身份运行脚本，您可能还需要使用Polkit，从而无需输入密码。

提示：

- （译注）如果您配置了Polkit无密码授权，那么这个脚本也可以使用您自己的用户来执行。参见Libvirt#设置授权和Polkit#跳过口令提示.
- （译注）您也可以使用Windows 10 自带的 OpenSSH，使用方法和Linux下相同，参见OpenSSH。

分离IOMMU组(ACS补丁)

如果您发现您的PCI设备与其他不希望直通的设备分在一组，您可以使用Alex Williamson的ACS补丁将它们分离。使用前请确保了解其中的潜在风险。

您需要一个应用了这个补丁的内核，最简单的办法就是安装linux-vfioAUR包。

此外，需要使用内核命令行选项启用ACS补丁。补丁文件添加以下文档：

```
pcie_acs_override =
        [PCIE] Override missing PCIe ACS support for:
    downstream
        All downstream ports - full ACS capabilties
    multifunction
        All multifunction devices - multifunction ACS subset
    id:nnnn:nnnn
        Specfic device - full ACS capabilities
        Specified as vid:did (vendor/device ID) in hex
```

使用选项`pcie_acs_override=downstream`通常就足够了。

安装和配置后，设置内核参数以在启用`pcie_acs_override=`选项的情况下加载新内核。

仅使用QEMU不使用libvirt

如果不想在libvirt的帮助下设置虚拟机，可以使用具有自定义参数的普通QEMU命令来运行要使用PCI直通的虚拟机，这对于脚本等一些用例来说是十分理想的，可以灵活地搭配其他脚本。

先完成#设置IOMMU和#隔离GPU，接着配置好QEMU虚拟化环境,启用KVM并使用`-device vfio-pci,host=07:00.0`选项，将`07:00.0`替换为您先前隔离的设备ID。

要使用OVMF固件，确保已安装ovmf包\[损坏的链接：replaced by edk2-ovmf包\]，将UEFI固件从`/usr/share/ovmf/x64/OVMF_VARS.fd`复制到临时位置，如`/tmp/MY_VARS.fd`。最后指定OVMF路径。使用如下的参数（注意顺序）：

- `-drive if=pflash,format=raw,readonly,file=/usr/share/ovmf/x64/OVMF_CODE.fd` 指向实际的位置，注意这是只读的
- `-drive if=pflash,format=raw,file=/tmp/MY_VARS.fd` 指向临时位置

注意：可以使用QEMU的默认SeaBIOS而不是OVMF，但不推荐使用它，因为它可能会导致直通设置出现问题。

建议通过使用virtio驱动程序并进一步配置，以增强性能,参阅QEMU。

您还可能必须使用`-cpu host,kvm=off`参数将主机的CPU型号信息转发给虚拟机，并欺骗Nvidia和其他制造商的设备驱动程序使用的虚拟化检测，防止驱动程序拒绝在虚拟机中运行。

直通其它设备

USB控制器

如果你的主板具有多个USB控制器并映射到不同的组，则可以直通这些控制器而不是单个USB设备。直通整个控制器有如下的优势：

- 如果设备在某些操作（例如正在进行更新的手机）的过程中断开或更改ID，虚拟机不会突然丢失设备。
- 由该控制器管理的任何USB端口都由虚拟机直接处理，并且可以将其设备拔出，重新插入和更改，而无需通知管理程序。
- 如果启动VM时通常直通给虚拟机的其中一个USB设备丢失，Libvirt将不会抱怨。

与GPU不同，大多数USB控制器的驱动程序不需要任何特定配置即可在VM上运行，并且通常可以在主机和客户机系统之间来回传递控制而不会产生任何副作用。

警告：确保您的USB控制器支持Reset。#直通不支持Reset的设备

您可以使用以下命令找出哪个PCI设备对应于哪个控制器以及每个端口和设备对应哪个控制器：

```
$ for usb_ctrl in $(find /sys/bus/usb/devices/usb* -maxdepth 0 -type l); do
pci_path="$(dirname "$(realpath "${usb_ctrl}")")"; echo "Bus $(cat
"${usb_ctrl}/busnum") --> $(basename $pci_path) (IOMMU group $(basename $(realpath
$pci_path/iommu_group)))"; lsusb -s "$(cat "${usb_ctrl}/busnum"):"; echo; done
Bus 1 --> 0000:00:1a.0 (IOMMU group 4)
Bus 001 Device 004: ID 04f2:b217 Chicony Electronics Co., Ltd Lenovo Integrated Camera
(0.3MP)
Bus 001 Device 007: ID 0a5c:21e6 Broadcom Corp. BCM20702 Bluetooth 4.0 [ThinkPad]
Bus 001 Device 008: ID 0781:5530 SanDisk Corp. Cruzer
Bus 001 Device 002: ID 8087:0024 Intel Corp. Integrated Rate Matching Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub

Bus 2 --> 0000:00:1d.0 (IOMMU group 9)
Bus 002 Device 006: ID 0451:e012 Texas Instruments, Inc. TI-Nspire Calculator
Bus 002 Device 002: ID 8087:0024 Intel Corp. Integrated Rate Matching Hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

款笔记本电脑有3个USB端口，由2个USB控制器管理，每个控制器都有自己的IOMMU组。 在此示例中，总线001管理单个USB端口（其中插入了SanDisk USB记忆

棒，因此它显示在列表中），还有许多内部设备，例如内部网络摄像头和蓝牙卡。 另一方面，除了插入的计算器之外，总线002不会管理其他东西。第三个端口是空的，这就是它没有出现在列表中的原因，但实际上是由总线002管理的。

确定需要直通的控制器之后，只需将其添加到虚拟机控制的PCI主机设备列表中即可。不需要其他配置。

注意：如果您的USB控制器不支持Reset，不在单独的组中，或者无法直通，您仍然可以通过udev规则达到类似的目的。参见，它允许连接到指定USB端口的任何设备自动连接到虚拟机。

使用PulseAudio将虚拟机音频传递给宿主机

可以使用libvirt将虚拟机的音频作为应用程序传输到宿主机。 这具有以下优点：多个音频流可传输到一个主机输出，并且与无需考虑音频设备是否支持直通。PulseAudio是必需的。

PulseAudio守护进程通常在你的普通用户下运行，并且只接受来自相同用户的连接。然而libvirt默认使用root运行QEMU。为了让QEMU在普通用户下运行，编辑`/etc/libvirt/qemu.conf`并将`user`设置为你的用户名。

```
user = "dave"
```

你同样需要告诉QEMU使用PulseAudio后端并识别要连接的服务器，首先将QEMU命名空间添加到你的域。 编辑域的`.xml`文件（使用`virsh edit domain`），将`domain type`行改为：

```
<domain type='kvm' xmlns:qemu='http://libvirt.org/schemas/domain/qemu/1.0'[失效链接 2020-08-04 ⓘ]>
```

并且加入如下的配置（在最后一个`</devices>`之后，`</domain>`之前）：

```
 <qemu:commandline>
   <qemu:env name='QEMU_AUDIO_DRV' value='pa'/>
   <qemu:env name='QEMU_PA_SERVER' value='/run/user/1000/pulse/native'/>
 </qemu:commandline>
```

`1000`是你的用户ID，如果必要的话改为你的用户ID，用户ID可以通过`id`命令查看。

请记住在继续之前保存文件并退出编辑器，否则更改将不会保存。如果您看到：“已更改域 \[名称\] XML配置”或是类似的消息，这表示您的更改已保存。

重启 `libvirtd.service` 和用户服务 `pulseaudio.service`。

现在，虚拟机音频将作为应用程序传递给宿主机。pavucontrol包可用于控制输出设备。请注意，在Windows客户机上，不使用消息信号中断可能会导致出现噼啪声。

物理磁盘/分区

可以通过向XML添加条目来使用整个磁盘或分区来提高I/O性能。

```
$ virsh edit [vmname]
```

```
<devices>
...
  <disk type='block' device='disk'>
    <driver name='qemu' type='raw' cache='none' io='native'/>
    <source dev='/dev/sdXX'/>
    <target dev='vda' bus='virtio'/>
    <address type='pci' domain='0x0000' bus='0x02' slot='0x0a' function='0x0'/>
  </disk>
...
</devices>
```

在Windows客户机上需要安装驱动程序才能工作，请参阅#安装客户机系统。

注意

直通不支持Reset的设备

当虚拟机关闭时，虚拟机使用的所有设备都会被其操作系统取消初始化，以准备关闭。在这种状态下，这些设备不再能使用，必须先重新上电才能恢复正常运行。Linux可以自己处理这种电源循环，但是当设备没有已知的Reset方法时，它将于禁用状态并不再可用。Libvirt和Qemu都希望所有宿主机的PCI设备在完全停止虚拟机之前准备好重新连接到宿主机，当遇到不会重置的设备时，虚拟机将挂起“关闭”状态，并将无法重新启动，除非宿主机系统重新启动。 因此，推荐仅直通支持重置的PCI设备，这可以通过PCI设备sysfs节点中存在`reset`文件来确定，例如`/sys/bus/pci/devices/0000:00:1a.0/reset`。

以下bash命令显示设备是否可以被Reset。

```
for iommu_group in $(find /sys/kernel/iommu_groups/ -maxdepth 1 -mindepth 1 -type d);do
echo "IOMMU group $(basename "$iommu_group")"; for device in $(\ls -1
"$iommu_group"/devices/); do if [[ -e "$iommu_group"/devices/"$device"/reset ]]; then
echo -n "[RESET]"; fi; echo -n $'\t';lspci -nns "$device"; done; done
IOMMU group 0
       00:00.0 Host bridge : Intel Corporation Xeon E3-1200 v2/Ivy Bridge DRAM
Controller [8086:0158] (rev 09)
IOMMU group 1
       00:01.0 PCI bridge : Intel Corporation Xeon E3-1200 v2/3rd Gen Core processor
PCI Express Root Port [8086:0151] (rev 09)
       01:00.0 VGA compatible controller : NVIDIA Corporation GK208 [GeForce GT 720]
[10de:1288] (rev a1)
       01:00.1 Audio device : NVIDIA Corporation GK208 HDMI/DP Audio Controller
[10de:0e0f] (rev a1)
IOMMU group 2
       00:14.0 USB controller [0c03]: Intel Corporation 7 Series/C210 Series Chipset
Family USB xHCI Host Controller [8086:1e31] (rev 04)
IOMMU group 4
[RESET]       00:1a.0 USB controller [0c03]: Intel Corporation 7 Series/C210 Series
Chipset Family USB Enhanced Host Controller #2 [8086:1e2d] (rev 04)
IOMMU group 5
[RESET]       00:1b.0 Audio device : Intel Corporation 7 Series/C210 Series Chipset
Family High Definition Audio Controller [8086:1e20] (rev 04)
IOMMU group 10
[RESET]       00:1d.0 USB controller [0c03]: Intel Corporation 7 Series/C210 Series
Chipset Family USB Enhanced Host Controller #1 [8086:1e26] (rev 04)
IOMMU group 13
       06:00.0 VGA compatible controller : NVIDIA Corporation GM204 [GeForce GTX 970]
[10de:13c2] (rev a1)
       06:00.1 Audio device : NVIDIA Corporation GM204 High Definition Audio Controller
[10de:0fbb] (rev a1)
```

这表示00：14.0中的xHCI USB控制器无法Reset，将导致虚拟机无法正常关闭，而在00：1b.0中的集成声卡和在00：1a.0和00:1d.0中的其他两个控制器没有这个问题，可以毫无问题地直通。

注意：如果你的AMD显卡无法正常Reset，参阅#AMD显卡直通后无法重启虚拟机。

完整的例子

如果您在配置的时候遇到了麻烦，您可以参考完整的配置示例。一些用户在那里描述了它们的配置步骤，或许您可以从中发现一些技巧。

疑难解答

如果您遇到的问题并没有在下面列出，您也可以参考QEMU#Troubleshooting。

Nvidia GPU直通到Windows 虚拟机时发生"错误43:驱动程序加载失败”

注意：

- 这也可能修复与Nvidia驱动程序相关的SYSTEM_THREAD_EXCEPTION_NOT_HANDLED引导失败。
- 这可能也会修复Linux虚拟机的问题。

从337.88开始，Windows上的Nvidia驱动程序将会检查虚拟机管理程序是否正在运行，如果检测到虚拟机管理程序，则会拒绝加载，这会导致Windows设备管理器出现错误43。从QEMU 2.5.0和libvirt 1.3.3开始，可以设置虚假的vendor_id，这足以欺骗Nvidia驱动程序。所需的步骤是将hv_vendor_id=随便什么的添加到QEMU命令行中的cpu参数，或者将以下行添加到libvirt的域配置中。 ID必须设置为12个字符的字母数字（例如'1234567890ab'）。

```
$ virsh edit [vmname]
...
<features>
       <hyperv>
              ...
              <vendor_id state='on' value='whatever'/>
              ...
       </hyperv>
       ...
       <kvm>
       <hidden state='on'/>
       </kvm>
</features>
...
```

使用旧版QEMU和/或libvirt的用户将必须禁用一些虚拟机管理程序扩展，这会大大降低性能。如果必须这样做的话，请在libvirt域配置文件中执行以下替换。

```
$ virsh edit [vmname]
...
<features>
       <hyperv>
              <relaxed state='on'/>
              <vapic state='on'/>
```

```
                <spinlocks state='on' retries='8191'/>
        </hyperv>
        ...
</features>
...
<clock offset='localtime'>
        <timer name='hypervclock' present='yes'/>
</clock>
...
...
<clock offset='localtime'>
        <timer name='hypervclock' present='no'/>
</clock>
...
<features>
        <kvm>
        <hidden state='on'/>
        </kvm>
        ...
        <hyperv>
                <relaxed state='off'/>
                <vapic state='off'/>
                <spinlocks state='off'/>
        </hyperv>
        ...
</features>
...
```

在启动虚拟机后在dmesg中看到"BAR 3: cannot reserve \[mem\]"错误

查看这篇文章:

如果你仍然遇到错误43，请在启动虚拟机后检查dmesg是否存在内存保留错误，如果您有类似消息，则可能是这种情况：

```
vfio-pci 0000:09:00.0: BAR 3: cannot reserve [mem 0xf0000000-0xf1ffffff 64bit pref]
```

找出您的GPU所连接的PCI Bridge。这条命令将给出设备的实际层次结构：

```
$ lspci -t
```

在启动VM之前运行以下命令，将ID替换为来自先前输出的实际ID。

```
# echo 1 > /sys/bus/pci/devices/0000\:00\:03.1/remove
# echo 1 > /sys/bus/pci/rescan
```

注意：可能需要同时设置内核参数 `video=efifb:off`。

VBIOS的UEFI (OVMF) 兼容

With respect to this article:

Error 43 can be caused by the GPU's VBIOS without UEFI support. To check whenever your VBIOS supports it, you will have to use `rom-parser`:

```
$ git clone https://github.com/awilliam/rom-parser
$ cd rom-parser && make
```

Dump the GPU VBIOS:

```
# echo 1 > /sys/bus/pci/devices/0000:01:00.0/rom
# cat /sys/bus/pci/devices/0000:01:00.0/rom > /tmp/image.rom
# echo 0 > /sys/bus/pci/devices/0000:01:00.0/rom
```

And test it for compatibility:

```
$ ./rom-parser /tmp/image.rom
Valid ROM signature found @600h, PCIR offset 190h
       PCIR: type 0 (x86 PC-AT), vendor: 10de, device: 1184, class: 030000
```

```
       PCIR: revision 0, vendor revision: 1
Valid ROM signature found @fa00h, PCIR offset 1ch
       PCIR: type 3 (EFI), vendor: 10de, device: 1184, class: 030000
       PCIR: revision 3, vendor revision: 0
              EFI: Signature Valid, Subsystem: Boot, Machine: X64
       Last image
```

To be UEFI compatible, you need a "type 3 (EFI)" in the result. If it's not there, try updating your GPU VBIOS. GPU manufacturers often share VBIOS upgrades on their support pages. A large database of known compatible and working VBIOSes (along with their UEFI compatibility status!) is available on TechPowerUp.

Updated VBIOS can be used in the VM without flashing. To load it in QEMU:

```
-device vfio-pci,host=07:00.0,.....,romfile=/path/to/your/gpu/bios.bin \
```

And in libvirt:

```
<hostdev>
     ...
     <rom file='/path/to/your/gpu/bios.bin'/>
     ...
   </hostdev>
```

One should compare VBIOS versions between host and guest systems using nvflash (Linux versions under *Show more versions*) or GPU-Z (in Windows guest). To check the currently loaded VBIOS:

```
$ ./nvflash --version
...
Version                : 80.04.XX.00.97
...
UEFI Support           : No
UEFI Version           : N/A
UEFI Variant Id        : N/A ( Unknown )
UEFI Signer(s)         : Unsigned
...
```

And to check a given VBIOS file:

```
$ ./nvflash --version NV299MH.rom
...
Version                : 80.04.XX.00.95
...
UEFI Support           : Yes
UEFI Version           : 0x10022 (Jul  2 2013 @ 16377903 )
UEFI Variant Id        : 0x0000000000000004 ( GK1xx )
UEFI Signer(s)         : Microsoft Corporation UEFI CA 2011
...
```

If the external ROM did not work as it should in the guest, you will have to flash the newer VBIOS image to the GPU. In some cases it is possible to create your own VBIOS image with UEFI support using GOPUpd tool, however this is risky and may result in GPU brick.

警告：Failure during flashing may "brick" your GPU - recovery may be possible, but rarely easy and often requires additional hardware. **DO NOT** flash VBIOS images for other GPU models (different boards may use different VBIOSes, clocks, fan configuration). If it breaks, you get to keep all the pieces.

In order to avoid the irreparable damage to your graphics adapter it is necessary to unload the NVIDIA kernel driver first:

```
# modprobe -r nvidia_modeset nvidia
```

Flashing the VBIOS can be done with:

```
# ./nvflash romfile.bin
```

警告：**DO NOT** interrupt the flashing process, even if it looks like it's stuck. Flashing should take about a minute on most GPUs, but may take longer.

HDMI音频不同步或是有噼啪声

对于一些用户来说，虚拟机的音频会在显卡上通过HDMI输出一段时间后减慢/卡顿。这通常也会降低图形处理速度。可能的解决方案包括启用MSI（基于消息信号的中断）而不是默认（基于线路的中断）。

要检查是否支持MSI以及是否被启用，请以root身份运行以下命令：

```
# lspci -vs $device | grep 'MSI:'
```

\$device\是设备位置 (例如 \01:00.0\)。

输出应该类似于：

```
Capabilities:  MSI: Enable- Count=1/1 Maskable- 64bit+
```

Enable后的-代表支持MSI，但虚拟机并没有启用。+代表虚拟机正在使用MSI。

启用它的过程有些复杂，可以在此处找到说明。

其他的提示可以从lime-technology's wiki上找到， 或参考这篇帖子VFIO tips and tricks。

网上有一些一些工具，例如 MSI_util，不过它们在 Windows 10 64bit下无效。

要解决这个问题，仅仅在function 0(01:00.0 VGA compatible controller: NVIDIA Corporation GM206 [GeForce GTX 960] (rev a1) (prog-if 00 [VGA controller]))上启用MSI是不够的，另一个function(01:00.1 Audio device: NVIDIA Corporation Device 0fba (rev a1))同样需要启用。

intel_iommu启用之后主机没有HDMI音频输出

如果启用 intel_iommu后，Intel GPU的HDMI输出设备在宿主机上无法使用，那么设置选项igfx_off（即intel_iommu=on,igfx_off）可能会恢复音频，有关设置igfx_off的详细信息，请阅读iommu.html。

启用vfio_pci后X无法启动

这与主机GPU被检测为额外GPU有关，当它尝试加载虚拟机所用的GPU的驱动程序时，会导致X崩溃。为了避免这种情况，需要一个指定主机GPU的BusID的Xorg配置文件。 可以从lspci或Xorg日志获取正确的BusID。来源 →

```
/etc/X11/xorg.conf.d/10-intel.conf
Section "Device"
        Identifier "Intel GPU"
        Driver "modesetting"
        BusID  "PCI:0:2:0"
EndSection
```

Chromium忽略集成图形加速

Chromium和它的朋友们将尝试在系统中检测尽可能多的GPU，并选择哪一个作为首选（通常是独立的的NVIDIA/AMD显卡）。它试图通过查看PCI设备来选择GPU，而不是系统中可用的OpenGL渲染器 - 结果是Chromium可能会忽略可用于渲染的集成GPU并尝试使用绑定到无法在宿主机上使用的虚拟机专用GPU。这导致使用软件渲染（导致更高的CPU负载，也可能导致视频播放不稳定，滚动不平滑）。

这可以通过告诉Chromium使用哪个GPU来解决。

虚拟机只使用一个核心

对于某些用户，即使启用了IOMMU并且核心计数设置为大于1，VM仍然只使用一个CPU核心和线程。 要解决此问题，请在`virt-manager`中启用“手动设置CPU拓扑”，并将其设置为所需的CPU，内核和线程数量。 请记住，“线程”是指每个CPU的线程数，而不是总数。

直通貌似工作但没有输出

确保您为您的虚拟机选择了UEFI固件。此外，请确保已将正确的设备直通给虚拟机。

virt-manager 遇到权限问题

如果你在virt-manager中遇到权限问题，将以下内容添加到`/etc/libvirt/qemu.conf`:

```
group="kvm"
user="user"
```

如果仍然不能工作，确保你的用户已经加入了kvm和libvirt组。

虚拟机关闭之后宿主机核心无响应

此问题似乎主要影响运行Windows 10 guest虚拟机的用户，并且通常在虚拟机运行很长一段时间后发生：主机的多个CPU核心被锁定（请参阅）。要解决此问题，请尝试在直通的GPU上启用消息信号中断。有关如何执行此操作的指南，请参见。

客户机在宿主机休眠眠情况下运行导致死机

启用VFIO的虚拟机如果在睡眠/唤醒周期中运行时会变得不稳定，并且在尝试关闭它们时会导致主机死机。为了避免这种情况，可以使用以下libvirt hook脚本和systemd单元在虚拟机运行时阻止主机进入休眠状态。hook文件需要可执行权限才能工作。

```
/etc/libvirt/hooks/qemu
#!/bin/sh

OBJECT="$1"
OPERATION="$2"
SUBOPERATION="$3"
EXTRA_ARG="$4"
```

```
case "$OPERATION" in
        "prepare")
                systemctl start libvirt-nosleep@"$OBJECT"
                ;;
        "release")
                systemctl stop libvirt-nosleep@"$OBJECT"
                ;;
esac
/etc/systemd/system/libvirt-nosleep@.service
[Unit]
Description=Preventing sleep while libvirt domain "%i" is running

[Service]
Type=simple
ExecStart=/usr/bin/systemd-inhibit --what=sleep --why="Libvirt domain \"%i\" is
running" --who=%U --mode=block sleep infinity
```

AMD显卡直通后无法重启虚拟机

某些AMD显卡在Windows虚拟机中可能无法正常Reset，导致虚拟机无法重新启动。

您可以通过在关闭虚拟机之前手动安全移除显卡（就像移除U盘那样）来避免这个问题。当您安装VirtIO驱动程序映像中的`guest-agent`之后，您就可以在安全移除菜单中看到相关的设备。

如果您希望自动完成这项任务，请参阅。

另请参阅

- Discussion on Arch Linux forums | Archived link
- User contributed hardware compatibility list
- Example script from https://www.youtube.com/watch?v=37D2bRsthfI
- Complete tutorial for PCI passthrough
- VFIO users mailing list
- YouTube: Level1Linux - GPU Passthrough for Virtualization with Ryzen

存储

块设备从接入到退役要过几道关：USB 盘挂载，远程盘经 iSCSI 与 USB/IP 接入，固态盘擦除与 4K 扇区又怎么应付。

《固态硬盘》与《先进格式化》必读，决定新盘的对齐与加密怎么设；再读《固态硬盘/存储单元擦除》弄清其用处。只接本地盘的看《USB 存储设备》与《Hdparm》；网络盘读《ISCSI/LIO》与《Open-iSCSI》，《ISCSI》只是指路页。最易踩的坑是拿 512 字节扇区的老经验套新盘：没对齐就掉速，文件系统也遭殃；擦除固态盘不等于删文件，不同接口的做法并不一样。

7.1　ISCSI

难度 入门 | **篇幅** 1.1 KB | **所属** 存储

本节将讲到

- 参见

使用 iSCSI 您可以通过基于 IP 的网络访问存储。

导出的存储实体是 **target** 导入的实体是 **initiator**。

从 Linux 2.6.38 开始，LIO 是内核内 iSCSI 目标。 SCST 是另一个包含内核的候选对象。

当前的启动器是 Open-iSCSI。

可以通过 iSCSI 引导 Arch Linux。

参见

- Gentoo:iSCSI

7.2　ISCSI/LIO

难度 入门 | **篇幅** 5.7 KB | **所属** 存储

本节将讲到

- 安装
- targetcli
- 小技巧
- 参考

LIO（LinuxIO）是从 Linux 2.6.38 开始内置在内核的 iSCSI 目标。

安装

Linux 从 3.1 版本开始自带 iSCSI 目标框架。

需要用到的两个内核模块为 *target_core_mod* 和 *iscsi_target_mod*，应已内置在内核中，并会自动加载。

强烈建议使用开放分支版本的包：targetcli-fbAUR，python-rtslib-fbAUR 和 python-configshell-fb包。

启动/启用随 python-rtslib-fbAUR 安装的 `target.service` 来加载需要的模块，载入配置并加载之前保存的 iSCSI 目标配置。

targetcli

以 root 身份运行 `targetcli status` 来查看使用中配置的相关信息。

你可以使用 **targetcli** 来创建完整配置文件，具体信息可参考 targetcli(8)。

交互式配置终端会自动为你填充大部分名称和参数，但你也可以自行指定配置信息。在终端内的任何时候，你都可以输入 `help` 来查看当前的可用命令。

提示：在该终端内，你可以使用 tab 进行命令补全，也可以使用 `cd` 来查看并选择路径。

在启动目标后（步骤参考上方），你可以通过

```
# targetcli
```

进入到交互式配置终端。

在该终端下，选定一个块设备（此处以 `/dev/disk/by-id/md-name-nas:iscsi` 为例）：

```
/backstores/block> create md_block0 /dev/disk/by-id/md-name-nas:iscsi
```

注意：你可以使用任意块设备，也可以使用 RAID 和 LVM 卷。如果你打算使用 fileio 而不是块存储对象，也可以指定一个文件。

然后创建一个 iSCSI 限定名称（IQN）和目标门户组（TPG）：

```
...> cd /iscsi
/iscsi> create
```

注意：如果要自行指定 IQN，可在 `create` 后加上想要的名称。

接下来让 LIO 使用你指定的块设备作为目标的后端存储（*backstore*）：

注意：你可以通过 `cd` 进入到你环境下的 <iqn>/tpg1 路径。

```
.../tpg1> cd luns
.../tpg1/luns> create /backstores/block/md_block0
```

然后创建一个门户（*portal*），这将创建一个守护进程来监听上行连接：

```
.../luns/lun0> cd ../../portals
```

```
.../portals> create
```
Targetcli 将汇报 LIO 监听的 IP 及端口（默认为 0.0.0.0（所有地址））。 你至少需要将 IP 提供给客户端，端口应为标准的 3260 端口。

为了让客户端/发起方能够连接到目标，你需要将发起端的 IQN 加入到目标的配置中：

```
...> cd ../../acls
.../acls> create iqn.2005-03.org.open-iscsi:SERIAL
```
你应使用发起端的 IQN，而不是照抄 `iqn.2005-03.org.open-iscsi:SERIAL` 到命令中。 一般可以在 `/etc/iscsi/initiatorname.iscsi` 找到 IQN。 你需要对每一个用到该目标的发起端都进行该操作。 Targetcli 会自动将创建的 LUN 映射到最近创建的 ACL 下。

注意：你可以修改自动映射的 LUN，也可以修改其读写权限。具体信息可在该步骤时输入 `help create` 来查看。

确认配置起效后，最后一步是保存配置：

```
...> cd /
/> saveconfig
```
这一步将把配置保存在 `/etc/target/saveconfig.json` 文件内。 你现在就可以安全地启动和停止 `target.service` 了，不用担心丢失配置。

提示：你可以在使用 `saveconfig` 时将文件名作为参数传入，也可以使用 `clearconfig` 清空所有配置。

认证

目标默认启用 CHAP 认证。 你可以选择配置一个密码，也可以禁用认证。

禁用认证

在 targetcli 中进入到目标的路径下（例如 /iscsi/iqn.../tpg1），然后：

```
.../tpg1> set attribute authentication=0
```
警告：设置该参数后，知道任一发起方 IQN 的人都可以连接到该目标。建议仅用于测试或家庭环境中。

配置凭据

在 targetcli 中进入到目标的具体 ACL 路径下（例如 /iscsi/iqn.../tpg1/acls/iqn.../），然后：

```
...> get auth
```
这将显示当前的认证凭据。

```
...> set auth userid=<username in target>
...> set auth password=<password in target>
...> set auth mutual_userid=<username in initiator>  (optional)
...> set auth mutual_password=<password in initiator>  (optional)
```

前两个字段是目标的用户名和密码，发起方会使用该信息登录到目标。后两个字段（以“mutual_”开头）是发起方的用户名和密码（注意，所有发起方都使用同一个用户名和密码）。这两个是可选参数，用于确保发起方仅接受已授权的目标的连接。

小技巧

- 可以使用 `targetcli sessions` 查看当前开启的会话。

参考

- targetcli
- Persistent block device naming in order to use the correct block device for a target

7.3 USB 存储设备

难度 进阶 | **篇幅** 6.1 KB | **所属** 存储

本节将讲到

- 使用 udisks 自动安装
- 手动挂载
- 问题解决

翻译状态：

- 本文（或部分内容）译自 USB storage devices，最近一次同步于 2024-08-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本文介绍了如何在 Linux 下使用流行的 USB 闪存盘。不过，它也适用于其他设备，如数码相机，这些设备就像 USB 存储设备一样。

如果你的系统是最新的 Arch 内核，并使用现代桌面环境，你的设备就会直接显示在桌面上，无需打开控制台。

使用 udisks 自动安装

这是最简单、最常用的方法。许多桌面环境都使用这种方法，但也可以单独使用。

详细信息，包括挂载助手列表，请参阅 Udisks。

手动挂载

注意：在确定 Arch Linux 无法挂载 USB 设备之前，请务必检查所有可用端口。有些端口可能不共享同一个控制器，从而导致无法挂载设备。

获取支持 usb_storage 的内核

如果不使用定制的内核，就可以使用，因为所有 Arch Linux 内核都已正确配置。如果使用定制内核，请确保其编译了 SCSI-Support、SCSI-Disk-Support 和 usb_storage。如果使用最新的 udev，只需插入设备，系统就会自动加载所有必要的内核模块。

识别设备

访问存储设备首先需要内核分配的标识符。详见文件系统#查看现有文件系统。

新插入的设备通常会显示在 Journal 中。

挂载 USB 存储器

请参阅文件系统#挂载文件系统。

如果 `mount` 无法识别设备的文件系统，可以尝试使用 `-t` 参数，详情请参见 mount(8) 。如果挂载失败，可以尝试重新创建文件系统，甚至重新分区磁盘。

注意：有关使用 sudo 的挂载/卸载脚本示例，请参阅 。

允许普通用户写入

如果想让非 root 用户写入 U 盘，可以发出以下命令：

```
# mount -o gid=users,fmask=113,dmask=002 /dev/sda1 /mnt/usbstick
```

如果不起作用，请确保文件系统可以以根用户身份挂载和写入，详见上一节。

使用 fstab 以作为普通用户操作

如果想让普通用户执行挂载/卸载操作，请参阅 FAT#以普通用户身份写入 FAT32。

挂载工具

多个挂载工具便于以普通用户身份挂载。

问题解决

未检测到 USB 存储设备

如果您已连接 USB 存储设备，但它未被 lsblk 列出，而是出现在 journal 中，但未被分配块设备，请参阅常规故障排除#内核升级后部分外设无法使用。

此外，请确保您的 BIOS 已启用 XHCI Handoff 和 EHCI Handoff，但对于大多数现代设备来说，这通常不是问题。

卸载所有分区后设备未关闭

如果不关闭设备电源，可能会导致：

- 硬盘驱动器没有停放磁头，在旋转时发出微弱的刮擦声并降低设备性能 ，或者
- 固态硬盘（尤其是老式固态硬盘）不刷新缓存缓冲区或更新映射表，导致数据丢失 。

卸载分区时，设备仍处于开机状态。为了安全地卸载，您应该要求系统先将其关闭：

```
# echo 1 > /sys/block/disk_name/device/delete
```

如果使用 udisks，则可以使用这些命令：

```
$ udisksctl unmount -b /dev/sdXY
$ udisksctl power-off -b /dev/sdX
```

设备已检测到，但未注册为可安装设备

有一套 udev 规则，其中包括一条出于各种原因忽略某些特定设备的规则。某些硬件设备，如数码相机、便携式录音机等，可能会以某种方式格式化 usb 存储，从而触发忽略规则。你可以使用以下命令检查是否应用了这些规则，然后在默认设置中找到相应的条件：

```
$ udevadm info --attribute-walk --name=device_name | grep UDISKS_IGNORE
```

如果是这种情况您的设备已将此属性设置为 *1*，您可以按照。udev 页面使用自定义规则来覆盖它。

7.4 USB/IP

难度 进阶 | **篇幅** 6.8 KB | **所属** 存储

本节将讲到

- 安装
- 使用
- 提示和技巧
- 参见

USB-IP 是一个将通用的USB设备通过IP网络进行共享的系统。为了在计算机之间共享具有全部功能的USB设备，USB-IP将“USB I/O消息”封装到 TCP/IP 中，并在计算机之间传输，以实现远程共享或访问USB设备。

安装

安装 usbip包 包。

使用

服务端配置

服务端为提供USB设备映射端，所以请确保已连接上物理USB设备，并且加载 `usbip_host` USB-IP内核模块，下面命令为加载模块：

```
# modprobe usbip_host
```

然后使用systemctl来启用 `usbipd.service` 服务，服务默认监听TCP 3240端口。

注意：默认情况下，USB-IP 需要监听端口 3240。如果防火墙正在运行，请确保允许这个端口访问。有关配置防火墙的详细说明，请转到 Category:防火墙。

```
# systemctl enable --now usbipd.service
```

查看当前主机已连接的USB设备总线ID（busid）：

```
$ usbip list -l
```

下面命令将USB设备 *busid* 为 1-1.5 绑定到服务中：

注意：系统在休眠后需要重新进行绑定。

```
# usbip bind --busid=1-1.5
```

解除绑定:

```
# usbip unbind --busid=1-1.5
```

解除绑定后，客户端将无法访问该USB设备

使用systemd进行绑定

如果想让系统开机后自动挂载，可以使用systemd模块单元文件，添加下面配置文件到系统中：

```
/etc/systemd/system/usbip-bind@.service
[Unit]
Description=USB-IP Binding on bus id %I
After=network-online.target usbipd.service
Wants=network-online.target
Requires=usbipd.service
#DefaultInstance=1-1.5

[Service]
Type=simple
ExecStart=/usr/bin/usbip bind --busid=%i
RemainAfterExit=yes
ExecStop=/usr/bin/usbip unbind --busid=%i
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

然后启用配置，如想将USB设备*busid*为1-1.5 自动绑定到服务中,只需使用systemclt命令启动/启用 `usbip-bind@1-1.service`：

```
# systemctl enable --now usbip-bind@1-1.5.service
```

客户端配置

客户端为加载远程USB设备端，需要加载 `vhci-hcd` 内核模块命令如下：

```
# modprobe vhci-hcd
```

查看远程服务端可用的USB设备，如果一切顺利将列出远程服务已绑定服务的所有USB设备:

```
$ usbip list --remote=服务端IP
```

记录下需要连接USB设备，例如连接到*busid*为 1-1.5 的USB设备:

```
$ usbip attach --remote=服务端IP -b 1-1.5
```
提示：如果服务器监听的不是默认端口:3240，可使用参数指定端口 `--tcp-port *端口号*`.

取消绑定设备

只有在客户端上断开设备后才能取消绑定

在客户端上查看已映射的USB端口:

```
$ usbip port
```
在客户端上断开某个已连接的USB端口:

```
$ usbip detach --port=端口号
```
在服务端上取消绑定USB设备r:

```
$ usbip unbind --busid=busid
```
提示和技巧

通过厂商/设备ID进行绑定

如果总线ID(busid)在每次系统重启后都不一样，则可以使用 厂商/设备ID 进行绑定，在服务端将下面配置文件配置保存到系统中：

```
/etc/systemd/system/usbip-bind@.service
[Unit]
Description=USB-IP Binding device id %I
After=network-online.target usbipd.service
Wants=network-online.target
Requires=usbipd.service

[Service]
Type=simple
ExecStart=/bin/sh -c "/usr/sbin/usbip bind --$(/usr/sbin/usbip list -p -l | grep
'#usbid=%i#' | cut '-d#' -f1)"
RemainAfterExit=yes
ExecStop=/bin/sh -c "/usr/sbin/usbip unbind --$(/usr/sbin/usbip list -p -l | grep
'#usbid=%i#' | cut '-d#' -f1)"
Restart=on-failure

[Install]
WantedBy=multi-user.target
```
下面示例是将配置是将厂商/设备ID为 *0924:3d68* 设置开机自动绑定，

```
# systemctl start --now usbip-bind@0924:3d68.service
```
注意：如果系统上存在多个同一个厂商同类型设备，即多个 `vendor:product` ID 重复的设备将无法正常工作。

在客户端的配置如下所示:

- Linux客户端

```
$ usbip attach -r 服务端IP --$(usbip list -p -r 服务端IP | grep '#usbid=0924:3d68#' | cut
-d# -f1)
```
注意：如果上面命令执行失败, 使用 *\-p* 参数测试是否正常，如： `usbip list -p -r *服务端IP*`，如果不正常请改用下面命令:

```
$ usbip attach -r 服务端IP -b $(usbip list -p -r 服务端IP | grep '0924:3d68' | cut '-d:' -f1 | awk '{print $1}')
```

- Windows客户端

```
c:\> for /f "tokens=1 delims=:, " %a in ('usbip list -r 服务端IP ^| findstr /r /c:"0924:3d68"') do start usbip attach -r 服务端IP -b %a
```

将配置存放至 /etc/..

usbip-systemdAUR 提供 systemd 服务文件，用于按供应商/设备 ID 进行绑定，以及按主机名和供应商/设备 ID 进行连接。

服务端配置

将USB设备的`vendor/product`ID按照下面格式编写配置文件，并存放在`/etc/usbip/bind-devices/` 目录下面。

```
/etc/usbip/bind-devices/example-device.conf
USBIP_DEVICE=0924:3d68
```

之后使用下命令启用服务：

```
# systemctl enable --now usbip-bind@example-device.service
```

客户端配置

将服务端USB设备的`vendor/product`ID按照下面格式编写配置文件，并存放在`/etc/usbip/remote-devices/` 目录下面。

```
/etc/usbip/remote-devices/example-device.conf
USBIP_HOST=example-host
USBIP_DEVICE=0924:3d68
```

确保服务端守护进程正在运行并且配置的设备已绑定, 然后下面命令启用服务：

```
# systemctl enable --now usbip@example-device.service
```

参见

- USB-IP项目官方网站
- Linux Kernel中 "usbip-utils自述"
- "如何使用USB/IP"
- "Windows版本的USB/IP"
- usbip(8)
- usbipd(8)

7.5 Open-iSCSI

难度 进阶 | **篇幅** 7.2 KB | **所属** 存储

本节将讲到

- 安装
- 概览
- 配置

- 提示
- 排障

本文讲述如何通过 Open-iSCSI initiator 访问 iSCSI 目标。

注意：iSCSI 是**非加密的**，不建议通过不安全的信道进行数据传输。

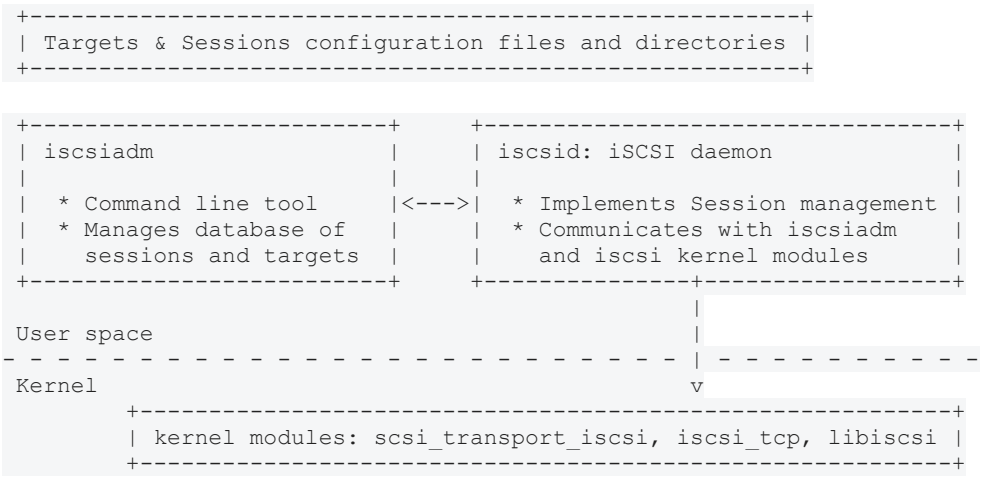
安装

安装 open-iscsi包 包。

注意：旧版的 Linux-iSCSI initiator 已经于2005年4月合并进 Open-iSCSI 。不要和 linux-iscsi.org 搞混了，这个是 LIO 目标的网站。

概览

下图显示各组件是如何协同工作的。更详细的版本参见：Open-iSCSI 模块（已过时）。



来自 Open-iSCSI README：

配置持久化由一套文件及目录树实现，一共包括在两个目录中：

注意：从 open-iscsi包 2.1.9-2 开始，数据库主目录由 `/etc/iscsi/` 变更为 `/var/lib/iscsi/`。这一迁移过程不是自动的，需要先断开所有目标并停止服务，手动迁移后再重新连接。另外，需检查文件系统权限。

`/etc/iscsi/` 应只包含 `initiatorname.iscsi` 和 `iscsid.conf`

- 发现目录 `/var/lib/iscsi/send_targets`：包含由目标地址命名的目录
- 节点目录 `/var/lib/iscsi/nodes`：包含由特定设备的 ION（ISCSI Unique Name）命名的目录

配置

启动服务

`iscsid` 由一个 systemd 单元来管理。

启动 `iscsid.service` 或是 `iscsid.socket`。

ISCSI Qualified Name (IQN)

IQN 被用于识别每个设备。

Open-ISCSI 将它的 initiator IQN 存放于 `/etc/iscsi/initiatorname.iscsi` 文件中，格式为 `InitiatorName=*iqn*`

在安装过程中会生成初始 IQN，如果你想生成新的 IQN，可以使用 `iscsi-iname` 工具来输出新的 IQN。

认证

如果 ISCSI 目标要求 initiator 进行认证，则需要更新配置文件（`/etc/iscsi/iscsid.conf`）的内容。

下列参数用于验证从 initiator 到目标的登录会话：

```
node.session.auth.authmethod = CHAP
node.session.auth.username = initiators_username
node.session.auth.password = initiators_password
```

如果目标启用了双向验证，则需要额外修改下列参数：

```
node.session.auth.username_in = targets_username
node.session.auth.password_in = targets_password
```

如果目标需要验证才能获取节点清单（少数情况下出现），则需要额外修改下列参数：

```
discovery.sendtargets.auth.authmethod = CHAP
discovery.sendtargets.auth.username = initiators_username
discovery.sendtargets.auth.password = initiators_password
```

如果目标启用了双向验证，则需要额外修改下列参数：

```
discovery.sendtargets.auth.username_in = targets_username
discovery.sendtargets.auth.password_in = targets_password
```

警告：不能出现两个一样的密码，意味着上述配置中需要四个不同的密码。

注意：验证数据存放在按节点区分的配置文件中。要更新这些数据，可以打开 `/var/lib/iscsi/nodes/iqn.*node-name*/*node-ip-address*,*port*,1/default` 并按需添加或编辑选项。

发现目标

向目标请求节点清单：

```
# iscsiadm --mode discovery --portal target_ip --type sendtargets
```

成功后，节点和目标的信息会保存到你的 initiator 中。

手动添加目标

```
# iscsiadm -m node --target targetname --portal target_ip -o new
```

可以在服务器不允许发现时使用该方法。

删除废旧目标

```
# iscsiadm -m discovery -p target_ip -o delete
```

登录到有效的目标

```
# iscsiadm -m node -L all
```

或者，登录到指定目标

```
# iscsiadm -m node --targetname=targetname --login
```

登出：

```
# iscsiadm -m node -U all
```

信息

对于运行中的会话

```
# iscsiadm -m session -P 3
```

上面命令输出的最后一行会显示连接到的设备名，比如

```
Attached scsi disk sdd State: running
```

对于已知节点

```
# iscsiadm -m node
```

在线修改卷大小

如果 iscsi 块设备包含一个分区表，则不能在线修改卷大小。这种情况下必须首先卸载文件系统，然后再调整相关分区的大小。

1. 重新扫描当前会话中的活动节点。

```
    # iscsiadm -m node -R
```

1. 在多路径环境中，也必须重新扫描多路径下的卷信息。

```
    # multipathd -k"resize map sdx"
```

1. 完成后再调整文件系统大小。

```
    # resize2fs /dev/sdx
```

提示

检查已连接的 iSCSI 设备

可以用下列命令检查已连接的 iSCSI 设备在 `/dev/` 设备树中的位置：

```
$ ls -l /dev/disk/by-path/ip-*
```

启动时登录到目标

要在启动时登录到目标，需启用 `iscsi.service` 服务，并确保节点的配置（`/var/lib/iscsi/nodes/iqn.*node-name*/*node-ip-address*,*port*`）中包含 `node.startup = automatic` 一项。

注意：systemd 单元名称为 `iscsi.service`，而不是 `iscsi**d**.service`。

排障

客户端 IQN

你可能需要在服务端（target）的账户配置中添加客户端的 IQN（位于 `/etc/iscsi/initiatorname.iscsi`）。

调试 iSCSI daemon

可以使用如下命令以调试模式运行 iSCSI daemon（确保已停止 `iscsid.service`）：

```
# iscsid -d 8 -c /etc/iscsi/iscsid.conf -i /etc/iscsi/initiatorname.iscsi -f
```

7.6 Hdparm

难度 进阶 | **篇幅** 11.8 KB | **所属** 存储

本节将讲到

- 安装
- 使用
- 提示与技巧
- 疑难解答
- 另请参见

翻译状态：

- 本文（或部分内容）译自 Hdparm，最近一次同步于 2025-02-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

hdparm 和 *sdparm* 是用于设置和查看硬盘参数的命令行实用程序。*hdparm* 也可以用作简单的基准测试工具。

最初 *hdparm* 被设计用于 IDE 硬盘，而 *sdparm* 对应的是 SCSI 硬盘。大约从 2010 年开始，存储设备接口变成了 IDE 和 SCSI 的增强组合，*hdparm* 和 *sdparm* 在功能上也形成互补。

警告：更改磁盘的默认参数可能会导致系统冻结甚至对磁盘造成不可逆转的损坏。

安装

安装 hdparm包 软件包。对于 SCSI 设备，安装 sdparm包 软件包。

使用

磁盘信息

运行下面命令获取磁盘信息：

```
# hdparm -I /dev/sda
```

基准测试

hdparm 可以被用于进行基准测试。

电源管理配置

现代硬盘驱动器支持许多电源管理功能，下表总结了最常见的功能。完整列表见 hdparm(8)。

警告：过于激进的电源管理会因频繁停车和降速而缩短硬盘驱动器的使用寿命。

参数	描述
`-B`	设置 Advanced Power Management（APM 高级电源管理）功能。可设定的值介于 1 和 255 之间，值越低意味着更积极的电源管理，值越高意味着更好的性能。从 1 到 127 的值允许降速，从 128 到 254 的值不允许。值 255 完全禁用该功能。
`-S`	设置驱动器待机（减速）前的超时时间。指定空闲状态（没有磁盘活动）的磁盘等待多长时间后关闭电机以节省电力。值设定为 0 禁用降速，从 1 到 240 的值指定 5 秒的倍数，从 241 到 251 的值指定 30 分钟的倍数。
`-M`	设置 Automatic Acoustic Management（自动声学管理）功能。大多数现代硬盘驱动器都能够降低磁头移动速度以减少其噪音输出。可设定的值取决于磁盘，某些磁盘可能不支持此功能。

注意：

- 当同时设置 `-B` 和 `-S` 参数时，APM 的值小于 128 可能会导致驱动器在 `-S` 超时值之前减速。详见 。
- 现在还无法读取 `-S` 的值。

要查询当前值，请传递不带值的参数。以 `-B` 为例：

```
# hdparm -B /dev/sda
```

要应用不同的值，例如将 APM 设置为 127：

```
# hdparm -B 127 /dev/sda
```

写入缓存

写入缓存是指在将数据写入硬盘前暂时将文件缓存到硬盘板载内存的行为，同时可以提升性能。大多数硬盘都提供了写入缓存并默认启用。使用以下命令可以进行检查：

```
$ hdparm -W /dev/sdX
```

警告：写入缓存可以提升性能，但也提升了突然断电时丢失数据的风险。

如果显示被禁用，可以使用如下命令进行启用：

```
$ hdparm -W 1 /dev/sdX
```

另外也可以将其禁用：

```
$ hdparm -W 0 /dev/sdX
```

关闭硬盘驱动器

这个功能的一个典型使用场景是，磁盘使用的是廉价外接 USB/SATA/火线硬盘盒或者桥接设备进行连接。如果在关闭电源开关时没有正确向驱动器发出停止命令，驱动器将被强制执行紧急磁头缩回。经常这样做迟早会破坏驱动器。一种解决方案是，在确定数据已写入介质后，运行命令关闭驱动器电源：

```
# hdparm -Y /dev/sdX
```

警告：绝对要确保

1. 数据已实际完成写入。还建议等待一段时间，以待驱动器空闲。
2. 你要关闭电源的目标驱动器正确（案例中为 `/dev/sd*X*`）

提示与技巧

在不唤醒磁盘的情况下查询磁盘状态

已知调用 hdparm 的查询选项会唤醒某些驱动器。在这种情况下，用 smartmontools 提供的 `smartctl` 来查询将不会唤醒睡眠的磁盘。例如：

```
# smartctl -i -n standby /dev/sda
smartctl 6.5 2016-05-07 r4318 [x86_64-linux-4.10.13-1-ARCH] (local build)
Copyright (C) 2002-16, Bruce Allen, Christian Franke, www.smartmontools.org

Device is in STANDBY mode, exit(2)
```

使用 udev 规则使配置持久化

要使设置在重新启动后保持不变，可以使用 udev 规则：

```
/etc/udev/rules.d/69-hdparm.rules
ACTION=="add", SUBSYSTEM=="block", KERNEL=="sda", RUN+="/usr/bin/hdparm -B 254 -S
0 /dev/sda"
```

由于硬盘可能会被随机分配新的 `/dev/sd*X*`，可以参考 Udev#Identifying a disk by its serial 通过其序列号进行识别。

具有多个硬盘驱动器的系统可以根据某些条件灵活应用该规则。例如，要将省电设置应用于所有旋转驱动器（带旋转磁头的硬盘，尤其不包括固态硬盘），请使用以下规则：

```
/etc/udev/rules.d/69-hdparm.rules
ACTION=="add|change", KERNEL=="sd[a-z]", ATTRS{queue/rotational}=="1",
RUN+="/usr/bin/hdparm -B 127 /dev/%k"
```

唤醒后重新应用配置

如果系统挂起/休眠后配置丢失，可以使用 systemd-sleep 重新应用。

将脚本放入 `/usr/lib/systemd/system-sleep/` 并设置为可执行：

```
/usr/lib/systemd/system-sleep/hdparm
#!/bin/sh

case $1 in post)
        /usr/bin/hdparm -B 254 -S 0 /dev/sda
        ;;
esac
```

启动后直接使驱动器进入睡眠状态

很少使用的设备可以在引导过程结束时直接进入睡眠状态。这不适用于上述 udev 规则，因为它发生得太早了。为了在启动完成时发出命令，只需创建一个 systemd 服务并启用它：

```
/etc/systemd/system/hdparm.service
[Unit]
Description=hdparm sleep

[Service]
Type=oneshot
ExecStart=/usr/bin/hdparm -q -S 120 -y /dev/sdb

[Install]
WantedBy=multi-user.target
```

使用不受支持的硬件

某些驱动器不支持通过 hdparm 降速。类似以下诊断错误消息很好地表明了这种情况：

```
# hdparm -S 240 /dev/sda
/dev/sda:
setting standby to 240 (20 minutes)
HDIO_DRIVE_CMD(setidle) failed: Invalid argument
```

其他一些驱动器会确认 hdparm 命令，但不遵守参数（APM 或降速计时器）。使用 Toshiba P300（型号 HDWD120）HDD 时会观察到这一点。

此类驱动器可以使用 hd-idleAUR 附带的 systemd 服务进行降速。需要编辑 `/etc/conf.d/hd-idle` 和 `HD_IDLE_OPTS` 值，然后启动和启用 `hd-idle.service`。

例如为 `/dev/sda` 设置 10 分钟空闲时间和为 `/dev/disk/by-uuid/01CF0AC9AA5EAF70` 设置 1 分钟空闲时间：

```
HD_IDLE_OPTS="-i 0 -a /dev/sda -i 600 -a /dev/disk/by-uuid/01CF0AC9AA5EAF70 -i 60"
```

开头的 `-i 0` 参数表示在其他驱动器上禁用 hd-idle。

西部数据绿盘的电源管理

Western Digital Green 硬盘有一个特殊的 *idle3* 计时器，控制驱动器在将磁头移动到停放位置并进入低功耗状态之前等待多长时间。出厂默认设置为 8 秒，这可能会在短时间内发生数千个磁头加载/卸载周期并最终导致过早发生故障，更不用说在执行例行 I/O 之前经常需要唤醒驱动器对性能的影响。西部数据发布过声明，声称 Linux 没有针对低功耗存储设备进行优化，并建议降低日志记录频率。这里有几种方法可以修改 *idle3* 状态：

1. 西部数据提供了一个 DOS 实用程序 *wdidle3.exe*，可以下载用于调整此设置。此实用程序旨在仅升级以下硬盘的固件： WD1000FYPS，WD7500AYPS 和 WD7501AYPS，但众所周知它也能够更改其他绿盘的 *idle3* 计时器。
2. hdparm 在 `-J` 标志后面具有逆向工程实现，似乎至少能在几个驱动器上工作，但它不像原始官方程序那样完整。对于 Linux 使用，建议设置为 30 秒。值设定为 0 以完全禁用 WD idle3 计时器（**不推荐**）：

```
# hdparm -J 30 --please-destroy-my-drive /dev/sda
```

。请参阅#使用 udev 规则使配置持久化在支持的硬盘驱动器上自动使用此参数。

1. idle3-tools包 软件包提供了另一个非官方实用程序。原始 `*idle3*` 值作为 *idle3ctl* 命令的参数传递。idle3ctl(8) 内的底部表格中提供了此值与超时（以秒为单位）之间的对应关系。以下命令会将计时器设置为 30 秒：

```
# idle3ctl -s 129 /dev/sdc
```

。以下命令完全禁用计时器（**不推荐**）：

```
# idle3ctl -d /dev/sdc
```

注意：

- 无论使用上述哪个程序，任何更改都需要一个完整的电源循环才能生效。这意味着驱动器需要关闭然后再打开，简单的重启是不够的。
- 众所周知，一些西数绿盘对 hparm 的待机超时参数有不同的解释，导致 `-S 1` 会配置计时器为 10 分钟而不是 5 秒。
- 西数绿盘在读/写期间的功耗通常约为 5.3W，空闲模式下为 4.7W，待机模式下为 0.7W。

疑难解答

APM 级别在挂起后被重置

APM 级别可能会在挂起后重置，需要在每次恢复后重新执行。这可以通过以下 systemd 单元（改编自论坛主题）自动完成：

```
/etc/systemd/system/apm.service
[Unit]
Description=Local system resume actions
After=suspend.target hybrid-sleep.target hibernate.target

[Service]
Type=simple
ExecStart=/usr/bin/hdparm -B 254 /dev/sda

[Install]
WantedBy=sleep.target
```

注意：`sleep.target` 被所有 `suspend`、`hybrid-sleep` 和 `hibernate` 目标拉取，但它在系统挂起**之前**就会完成启动，因此必须明确指定三个目标。详见。

也可以在 /usr/lib/systemd/system-sleep 中创建一个钩子。

另请参见

- https://sourceforge.net/projects/hdparm/ - SourceForge 上的 hdparm 页面

7.7 固态硬盘/存储单元擦除

难度 进阶 | **篇幅** 18.9 KB | **所属** 存储

本节将讲到

- SATA 硬盘
- NVMe 硬盘
- 使用blkdiscard的通用方法
- 故障排除
- 另见

有时，可能希望将固态硬盘完全重置到其出厂时的“干净”状态，这可以将其写入性能恢复到出厂时的水平。即使固态硬盘原生支持TRIM，写入性能也会随着时间的推移而下降，这是由于 TRIM 仅在文件删除时起作用，而对文件替换（例如增量保存）无效。

注意，执行安全擦除不能重置固态硬盘单元的损耗状态。在安全擦除后，一个接近使用寿命的硬盘在短时间内**可能**可再次写入，但在不久之后仍会失效。

警告：

- 继续操作前请备份所有重要数据！**此过程将销毁固态硬盘上的所有数据**，即使专业数据恢复服务提供商也无法恢复数据！完成此过程后，需要重新分区设备并将需要的数据重新复制到硬盘上！

- 如果要擦除的硬盘不是直接连接到SATA/NVMe接口，请**不要**继续。在通过USB或SAS/RAID卡连接的硬盘上执行安全擦除/格式化/净化（Sanitize）命令可能会导致硬盘变砖！

提示：清理硬盘后，在分区之前，应当检查其逻辑扇区大小是否已设置为最佳大小。更改逻辑扇区大小需要清空数据，因此最好在还未存入数据时更改。

SATA 硬盘

ATA有两个用于擦除硬盘的命令——`SECURITY ERASE UNIT` 和 `ENHANCED SECURITY ERASE UNIT`。

提示：这些命令也可用于将机械硬盘清零。

确保硬盘未处于冻结模式

执行以下命令：

```
# hdparm -I /dev/sdX | grep frozen
```

在输出的安全（security）部分，应该显示 `not frozen`。若显示 `frozen`，将无法进行下一步操作。详见固态硬盘#冻结模式。

一个可能的解决方法是，挂起系统（注意要使用 S3 而非 S0ix）。唤醒后，冻结状态很可能会被解除。若不成功，且硬盘通过SATA接口连接，可以尝试热插拔硬盘数据线（注意，这可能会导致内核崩溃）。如果热插拔SATA数据线导致内核崩溃，重启系统，在系统完全启动后，快速地热插拔SATA电源线和数据线。如果热插拔SATA线仍然导致内核崩溃，请确保在BIOS中启用了AHCI（AHCI允许进行热插拔操作，不应引起系统崩溃）。如果有支持热插拔的USB-to-SATA适配器，可尝试通过适配器连接设备。此外，还可通过 USB 使用 hdparm包。

Dell 系统

如果命令输出显示"frozen"，或许可以通过以下方式解决：

1. 启动时按 F2，进入 Dell BIOS。
2. 在 BIOS 中设置内置硬盘密码（注意，BIOS中使用的键盘布局是 en_US / qwerty）。
3. 应用更改并重启。
4. 当 Dell Security Manager 提示输入密码时，直接按ESC而不要输入密码。硬盘将保持锁定但非冻结状态。
5. 在下文的步骤中，跳过启用安全功能的部分，直接执行擦除命令。

提示：如果正在使用联想系统，并且无法通过上面提到的方法解除"frozen"状态（例如，联想平板电脑通过M.2接口而不是SATA接口连接固态硬盘），可以使用**联想提供的专有工具**完成闪存单元清理，而不是遵循本文的步骤。另见

通过设置用户密码来启用安全功能

提示：设置用户密码后，在下一次重启后，硬盘将被锁定，只有使用正确密码解锁后才能正常访问。

警告：执行此步骤后**不要**重启计算机，特别是联想笔记本电脑。某些联想 BIOS 版本可能使用一种有偏差的算法来计算加密密钥。重启后，机器可能识别到固态硬盘。

由于只是临时性地使用密码，因此对密码强度没有要求，可以使用任何密码。安全擦除后，密码将被重置为 NULL。在本例中，假定密码为 `*PasSWorD*`：

```
# hdparm --user-master u --security-set-pass PasSWorD /dev/sdX
security_password="PasSWorD"
/dev/sdX:
Issuing SECURITY_SET_PASS command, password="PasSWorD", user=user, mode=high
```

执行如下命令检查设置是否成功：

```
# hdparm -I /dev/sdX
```

命令输出应为"enabled"：

```
Security:
        Master password revision code = 65534
                supported
                enabled
        not     locked
        not     frozen
        not     expired: security count
                supported: enhanced erase
        Security level high
        2min for SECURITY ERASE UNIT. 2min for ENHANCED SECURITY ERASE UNIT.
```

执行 ATA SECURITY ERASE UNIT 命令

最后一步是执行 ATA SECURITY ERASE UNIT 命令，指示设备固件擦除其内容。注意，对于本例中使用的设备，前面的输出显示：

```
2min for SECURITY ERASE UNIT. 2min for ENHANCED SECURITY ERASE UNIT.
```

根据 ATA 规范，“增强型”安全擦除（`--security-erase-enhanced`）会执行更彻底的擦除。如果两个命令的预计完成时间相等，表明硬盘制造商简化了规范，对两个命令使用了相同的擦除功能。若预计时间较短（如2分钟），表明该设备是自加密的，执行安全擦除操作时，其固件将只擦除内部的加密密钥，而不是覆写所有数据单元。

警告：

- 请再三检查是否使用了正确的硬盘代号。一旦确认命令，将**无法挽回**。
- 确保执行命令时硬盘未被挂载。如果在设备已挂载时执行安全擦除命令，可能无法正常完成擦除。

```
# hdparm --user-master u --security-erase PasSWorD /dev/sdX
```

等待命令完成。对于在本例中使用的Intel X25-M 80GB固态硬盘，该过程耗时约 40 秒。

```
security_password="PasSWorD"
/dev/sdX:
Issuing SECURITY_ERASE command, password="PasSWorD", user=user
0.000u 0.000s 0:39.71 0.0%      0+0k 0+0io 0pf+0w
```

硬盘现已被擦除。成功擦除后，硬盘安全状态应当已经被自动设置为禁用（因此不再需要密码即可访问）。可通过如下命令验证：

```
# hdparm -I /dev/sdX
```

命令输出应显示 "not enabled"：

```
Security:
        Master password revision code = 65534
                supported
        not     enabled
        not     locked
        not     frozen
        not     expired: security count
                supported: enhanced erase
        2min for SECURITY ERASE UNIT. 2min for ENHANCED SECURITY ERASE UNIT.
```

NVMe 硬盘

NVMe 硬盘不使用 SATA 接口，因而不能使用与 SATA 固态硬盘相同的方式进行清理。NVMe 规范定义了一种标准化的格式化 NVMe 硬盘的方法。最初，可通过nvme-format(1) 命令（nvme-cli包 的一部分）进行格式化。在此基础之上，1.3 版规范增加了专用的 nvme-sanitize(1) 命令。NVM Express 联盟对上述命令的描述如下：

: 这些命令用于安全地擦除设备上的用户数据。可在部署新设备、报废旧设备、设备寿命耗尽、将设备用于新应用等场景使用。在 NVMe 1.3 规范中引入净化（Sanitize）之前，NVMe 格式化（Format）专门用于执行安全擦除。虽然格式化与净化都能擦除数据，但净化在确保数据被正确擦除方面更为可靠；格式化则适用于日常使用和测试。

要获取硬盘支持的功能，使用Identify Controller命令：

```
# nvme id-ctrl /dev/nvme0 -H | grep -E 'Format |Crypto Erase|Sanitize'
```

示例输出：

```
  [1:1] : 0x1 Format NVM Supported
  [29:29] : 0 No-Deallocate After Sanitize bit in Sanitize command Supported
    [2:2] : 0 Overwrite Sanitize Operation Not Supported
    [1:1] : 0x1       Block Erase Sanitize Operation Supported
    [0:0] : 0x1       Crypto Erase Sanitize Operation Supported
  [2:2] : 0x1 Crypto Erase Supported as part of Secure Erase
  [1:1] : 0   Crypto Erase Applies to Single Namespace(s)
  [0:0] : 0   Format Applies to Single Namespace(s)
```

根据设备支持情况，选择使用格式化或净化命令。

格式化命令

在概念上，格式化（Format）命令更接近于hdparm和fdisk的结合体。能够设置硬盘的底层参数，并额外发送安全擦除命令。

nvme-format(1)介绍了关于安全擦除选项（`-s`/`--ses`）的如下信息：

: 安全擦除设置：此字段指定是否应在格式化过程中执行安全擦除以及安全擦除操作的类型。擦除操作将作用于所有用户数据，无论其位置如何（例如，在向操作系统暴露的 LBA 内、在缓存内、在已释放的 LBA 内等）。默认为 0。

可能的值：

值	定义
0	不执行安全擦除操作
1	用户数据擦除：所有用户数据都将被擦除，擦除后用户数据的内容是不确定的（例如，用户数据可能被填充为全零、全一等）。如果所有用户数据都已加密，控制器在收到用户数据擦除请求时可能转而选择执行加密擦除。
2	加密擦除：所有用户数据都将通过密码学加密方式擦除。通过擦除加密密钥完成。

格式化命令支持对整个NVMe字符设备（例如`/dev/nvme0`）或特定的命名空间块设备（例如`/dev/nvme0n1`）进行擦除操作。但不同硬盘对命名空间擦除的支持情况不同，在执行格式化操作前，需确保硬盘的支持情况。例如，在本例Identify Controller命令的输出中，`Crypto Erase Applies to Single Namespace(s)`和`Format Applies to Single Namespace(s)`位被设置为零，根据规范，这意味着“控制器支持基于每个命名空间的格式化”（参见“Non-Volatile Memory Host Controller Interface”文档中，图表82“Identify Controller Data Structure”中对第524字节，“Format NVM Attributes”字段的描述）。

例如，要对`/dev/nvme0`设备命名空间1使用加密擦除方式进行格式化：

```
# nvme format /dev/nvme0 -s 2 -n 1
```

使用`-n 0xffffffff`可格式化所有命名空间。

有关设备/命名空间选择的更多信息和注意事项，参见nvme-format(1)。

净化命令

净化（Sanitize）命令用于实现“在功能上等同于 SATA 和 SAS 实现中同名的命令”的功能。前述文章中提到：

: 根据 NVMe 1.4 规范，“净化操作会改变 NVM 子系统中的所有用户数据，使得不可能从任何缓存、非易失性介质或任何控制器内存缓冲区中恢复任何先前的用户数据。” : 净化和格式化之间的最大区别在于，净化确保缓存被擦除，并且在意外断电后重新启动时，该过程会重新开始。对于安全擦除操作，净化还支持使用特定模式覆写（pattern overwrite）。注意，对于NAND存储设备，使用特定模式覆写将损耗闪存寿命。因此，特定模式覆写一般用于使用其它存储方式的设备，但若要进一步降低用户数据恢复的可能性，也可以用于NAND存储设备。

`-a/--sanact`选项的用法和可能取值参见nvme-sanitize(1)。

根据NVMe 1.4 规范中的定义，块擦除和加密擦除的区别在于，加密擦除仅擦除加密密钥：

: 用户数据擦除：所有用户数据都将被擦除，擦除后用户数据的内容是不确定的（例如，用户数据可能被填充为全零、全一等）。如果所有用户数据都已加密，控制器在收到用户数据擦除请求时可能转而选择执行加密擦除。 : 加密擦除：所有用户数据都将通过密码学加密方式擦除。通过擦除加密密钥完成。 : ... : 块擦除净化操作使用依介质而异的低级块擦除方法，更改 NVM 子系统中可能存储用户数据的所有位置的用户数据； : 加密擦除净化操作通过更改 NVM 子系统中可能存储用户数据的所有位置的介质加密密钥来更改用户数据...

可以获取硬盘执行各种擦除操作所需时间的估计值（如果支持）：

```
# nvme sanitize-log /dev/nvme0
...
Estimated Time For Overwrite                    :  4294967295 (No time period reported)
Estimated Time For Block Erase                  :  174
Estimated Time For Crypto Erase                 :  34
```

如果得到如下结果：

```
# nvme sanitize-log /dev/nvme0
...
Estimated Time For Overwrite                    :  4294967295 (No time period reported)
Estimated Time For Block Erase                  :  4294967295 (No time period reported)
Estimated Time For Crypto Erase                 :  4294967295 (No time period reported)
```

说明擦除操作将需要**很长时间**才能完成。例如，在一块报告了上述结果的 Intel 660p 512GB 硬盘上，块擦除大约需要 2-3 小时才能完成。

警告：一旦开始，擦除操作便无法中断（即使将设备断电再重启），并且在完成之前，硬盘将无法使用。擦除可能需要很长时间。

要开始加密擦除净化操作：

```
# nvme sanitize device -a start-crypto-erase
```

注意：`*device*` 参数必须指向整个NVMe字符设备（例如 `/dev/nvme0` 而**不是** `/dev/nvme0n1`）。加密擦除净化操作只适用于**整个设备**。

若要使用块擦除：

```
# nvme sanitize device -a start-block-erase
```

警告：即使硬盘支持，也应当避免使用**块擦除（Overwrite）**操作，因为“由于将损耗设备寿命，不推荐对基于NAND的固态硬盘进行块擦除操作”。

可以使用净化日志（Sanitize Log）跟踪进度：

```
# nvme sanitize-log /dev/nvme0
```

对于正在进行加密擦除的硬盘，输出应形如：

```
Sanitize Progress                       (SPROG) :  655
Sanitize Status                         (SSTAT) :  0x4
Sanitize Command Dword 10 Information (SCDW10) :  0x4
Estimated Time For Overwrite                    :  4294967295 (No time period reported)
Estimated Time For Block Erase                  :  174
Estimated Time For Crypto Erase                 :  34
Estimated Time For Overwrite (No-Deallocate)    :  0
Estimated Time For Block Erase (No-Deallocate) :  0
Estimated Time For Crypto Erase (No-Deallocate):  0
```

当擦除操作完成时，输入应形如：

```
Sanitize Progress                       (SPROG) :  65535
Sanitize Status                         (SSTAT) :  0x101
Sanitize Command Dword 10 Information (SCDW10) :  0x4
Estimated Time For Overwrite                    :  4294967295 (No time period reported)
Estimated Time For Block Erase                  :  174
Estimated Time For Crypto Erase                 :  34
Estimated Time For Overwrite (No-Deallocate)    :  0
Estimated Time For Block Erase (No-Deallocate) :  0
Estimated Time For Crypto Erase (No-Deallocate):  0
```

使用blkdiscard的通用方法

blkdiscard(8)命令（util-linux包软件包的一部分）支持`--secure`选项，用于“执行安全discard操作。安全discard操作与常规discard操作相同。只是，对于正在discard的块，其所有副本（可能在垃圾回收过程中创建）也将被擦除。安全discard操作需要设备的支持”。

要执行安全discard操作：

```
# blkdiscard --secure /dev/device
```

对于不支持安全擦除的设备，可使用`-z`/`--zeroout`选项，在discard设备上的所有块的基础上，还会将设备填充为零。

关于`blkdiscard`安全性的讨论，参见。关于使用`blkdiscard`擦除卷的示例，参见。

故障排除

擦除硬盘后 UEFI 启动项被移除

某些UEFI实现会在系统启动时移除引用了不存在启动文件的启动项。如果计划在擦除设备后从备份中恢复系统，需要使用efibootmgr或通过重新安装引导加载程序的方法恢复启动项。

另见

- Secure Erase HDDs/SSDs (SATA/NVMe) using hdparm & nvme-cli on Linux (2019): good tutorial with images
- Verifying SSD Sanitization

7.8 先进格式化

难度 高级 | **篇幅** 26.1 KB | **所属** 存储

本节将讲到

- 修改扇区大小
- 分区对齐
- dm-crypt
- 文件系统
- 已知问题

机械硬盘的最小物理存储单元为单个扇区。固态硬盘（SSD）相对应为页。 存储设备的固件将设备上的物理扇区抽象为软件可操作的逻辑扇区。该扇区的大小即硬盘上最小可寻址单元的大小。

注意：无论存储设备为何种类型，软件和文档可能将“扇区”与“块”互换使用。

物理扇区大小：这是物理存储设备声称的可原子写入的最小单元。对于机械硬盘来说，这是指碟片上扇区的实际大小。在传统上，机械硬盘的物理扇区大小是 512 字节，这意味着每个扇区可以存放 512 字节的数据。但是，随着先进格式化机械硬盘的出现，物理扇区的扇区提高到了 4096 字节（4 KiB），使得存储密度和错误校正能力都有所提升。固态硬盘（SSD）则不对外暴露 NAND 闪存的实际页大小（通常在 4 KiB 到 16 KiB 之间），而是将物理扇区大小和逻辑扇区大小汇报为同一值。对于 NVMe 固态硬盘，它会在电源故障原子写入单位（Atomic Write Unit Power Fail，AWUPF）参数可用时使用该值。

逻辑扇区大小：逻辑扇区大小（又称为操作系统扇区大小）代表了暴露给操作系统和应用的扇区大小。它是在软件层写入和读取存储设备的扇区大小。逻辑扇区大小可以和物理扇区不同，例如：为了保证旧系统和应用的兼容性，一个有 4096 字节物理扇区大小的先进格式化机械硬盘的逻辑扇区大小可能为 512 字节。

对于不同的“层”而言，即设备，堆叠块设备和文件系统，应保持同一扇区大小。否则，虽然固件的转换层的映射操作是透明的，仍会带来本可避免的开销。

可通过 lsblk 列出当前物理和逻辑扇区大小：

**本文或本章节的事实准确性存在争议。**

原因：下列的两种命令（`lsblk` 和 `cat /sys/class...`）都无法准确提供 NVMe 硬盘的物理扇区大小（可能涉及所有 SSDs），而是会给出逻辑扇区的大小。（在 Talk:先进格式化 中讨论）

```
$ lsblk -td
NAME    ALIGNMENT MIN-IO OPT-IO PHY-SEC LOG-SEC ROTA SCHED       RQ-SIZE  RA WSAME
sda             0   4096      0    4096    4096    1 mq-deadline      64 128    0B
nvme1n1         0   4096      0    4096    4096    0 none           1023 128    0B
```

```
nvme0n1         0    4096       0     4096     4096     0 none             1023 128     0B
```

PHY-SEC 显示了物理扇区大小，LOG-SEC 显示了逻辑扇区大小。

另外，单个特定硬盘的值可以从下列 sysfs 条目读取：

```
$ cat /sys/class/block/drive/queue/physical_block_size
$ cat /sys/class/block/drive/queue/logical_block_size
```

也可以通过 fdisk，smartctl 和 hdparm 的输出查看扇区大小。

修改扇区大小

警告：修改硬盘的扇区大小将不可恢复地抹除所有数据。

部分 NVMe 和“企业级”SATA 硬盘分别支持通过标准的 NVMe（NVMe 命令集标准 1.0 或以上的 Format NVM）或 ATA（ATA 命令集 - 4 或以上的 SET SECTOR CONFIGURATION EXT）命令修改它们汇报的扇区大小。对于机械硬盘而言，这一操作通过修改它们的逻辑扇区大小到与物理扇区大小一致，优化了硬盘性能。而对于 NVMe 固态硬盘而言，逻辑扇区和物理扇区的大小都受到更改。

SATA 固态硬盘通常不支持修改扇区大小。部分 Intel SATA SSD 仅可以被修改汇报的物理扇区大小，无法修改逻辑扇区大小。 可按照 #Intel 中的操作来修改硬盘汇报的物理扇区大小。

修改扇区大小包含了一系列复杂的操作，其中涉及到了低级格式化。作为替代，你可以在硬盘上创建文件系统时手动指定扇区大小来优化性能。详细信息可参考 #dm-crypt 和#文件系统。

先进格式化机械硬盘

使用 hdparm 工具确定先进格式化硬盘的扇区大小是否可被更改：

```
# hdparm -I /dev/sdX | grep 'Sector size:'
```

注意：对于通过 USB 连接的硬盘，对应的 USB 桥接器需支持 SAT（SCSI/ATA 翻译）（ANSI INCITS 431-2007）。

Advanced Format drives whose Sector Configuration Log lists multiple logical sector sizes will show a list of them:

```
        Logical  Sector size:                   512 bytes [ Supported: 512 4096 ]
        Physical Sector size:                  4096 bytes
```

不支持修改逻辑扇区大小的硬盘将只会汇报当前扇区大小。以一块 512e 先进格式化硬盘为例：

```
        Logical  Sector size:                   512 bytes
        Physical Sector size:                  4096 bytes
```

为了达到这一类硬盘的最佳性能，请确保 #dm-crypt 扇区大小或#文件系统块大小至少为 4096 字节，并与其对齐。

对于一块先进格式化的 4Kn 硬盘：

```
        Logical  Sector size:                   4096 bytes
        Physical Sector size:                   4096 bytes
```

4Kn 硬盘在出厂时已经是最佳配置，不需要在分区/格式化是进行特殊操作，可以照常使用。

如果你的 SATA 机械硬盘支持多个逻辑扇区大小和可选的 `SET SECTOR CONFIGURATION EXT` ATA 命令（通常只在企业级机械硬盘上提供），你可以使用 *hdparm* 来在支持的不同逻辑扇区大小间进行切换。可使用以下命令将其设为 4096 字节（即 4Kn）：

```
# hdparm --set-sector-size 4096 --please-destroy-my-drive /dev/sdX
```

之后 *hdparm* 应汇报逻辑扇区大小为 4096 字节：

```
# hdparm -I /dev/sdX | grep 'Sector size:'
        Logical  Sector size:                   4096 bytes [ Supported: 512 4096 ]
        Physical Sector size:                   4096 bytes
```

NVMe 固态硬盘

虽然固态硬盘（SSD）经常使用较大的物理块（通常为 4 KiB，8 KiB，有时会更大），它们通常还是将自己的逻辑块大小汇报为 512 字节。

可以通过 nvme-cli包 使用 *Identify Namespace* 命令来查看 NVMe 硬盘的格式化后逻辑块地址大小（Formatted Logical Block Address Size, FLBAS）：

```
# nvme id-ns -H /dev/nvme0n1 | grep "Relative Performance"
LBA Format  0 : Metadata Size: 0   bytes - Data Size: 512 bytes - Relative Performance:
0x2 Good (in use)
LBA Format  1 : Metadata Size: 0   bytes - Data Size: 4096 bytes - Relative
Performance: 0x1 Better
```

- `Metadata Size` 是每个逻辑块地址（Logical Block Address, LBA）的额外元数据字节大小。这一功能在 Linux 上尚无完善支持，所以最好是使用该值为 0 的格式。
- `Relative Performance` 代表各类格式所提供的性能级别（包括 *degraded*，*good*，*better* 和 *best*）。

smartctl 也可以显示设备支持的逻辑块地址大小，但输出的结果可读性不佳。例如：

```
# smartctl -c /dev/nvme0n1
...
Supported LBA Sizes (NSID 0x1)
Id Fmt  Data  Metadt  Rel_Perf
 0 +     512       0         2
 1 -    4096       0         1
```

警告：特别是在高随机读取负载的情况下，有些显示支持 4K 扇区的 NVMe SSD 有可能会在启用该功能时断续出现稳定性问题。因此在向硬盘存入数据前，请确保已对配置进行了完善的测试。如果出现该问题，只需将其恢复到 512 扇区大小即可。

可以使用 `--lbaf` 参数向 `nvme format` 命令传入目标值来修改逻辑块地址大小：

```
# nvme format --lbaf=1 /dev/nvme0n1
You are about to format nvme0n1, namespace 0x1.
WARNING: Format may irrevocably delete this device's data.
You have 10 seconds to press Ctrl-C to cancel this operation.

Use the force [--force] option to suppress this warning.
Sending format operation ...
Success formatting namespace:1
```

修改需要数秒来完成。

注意：确保分区时参照#分区对齐进行，否则将破坏任何性能优势。

**本文或本章节的事实准确性存在争议。**

原因： 在调整电源状态或是 UEFI 设置前，建议用户使用原生 `nvme` 命令检查操作日志，或是使用类似 `nvme reset` 的命令软重置控制器。（在 Talk:先进格式化 中讨论）

在启动结束阶段会发出非标准的“security freeze”的系统上时，生产于 2020 年前的硬盘可能会拦截 `Format NVM` 命令。 如果 `nvme format` 命令失败，可以先尝试休眠系统（确保使用 S3 而不是 S0ix 休眠），然后在唤醒系统后尝试再次运行 `nvme format` 命令。

使用制造商专用工具

如果上述通用工具无法修改扇区大小，可以试下使用硬盘制造商的专用工具进行修改。

英特尔（Intel）

对于 Intel 硬盘，需要通过 Intel Memory and Storage（MAS）工具（intel-mas-cli-toolAUR）使用 `-set PhysicalSectorSize=4096` 选项进行修改。注意，只有汇报的物理扇区大小会被修改，逻辑扇区大小不会变化。

注意：自从 Intel 固态硬盘业务被 SK 海力士收购为 Solidigm 品牌后，Intel MAS 工具就只能用于傲腾（Optane）产品上，不再能管理固态硬盘。Solidigm 提供了与 Intel MAS 过去功能相同的 solidigm-sst-storage-tool-cliAUR 工具，具体信息可参考 Solid_state_drive/NVMe#Intel/Solidigm。

希捷（Seagate）

对于希捷可以使用 openseachestAUR。

扫描所有硬盘来找到要修改的硬盘，然后输出对应硬盘的相关信息：

```
# openSeaChest_Basics --scan
# openSeaChest_Basics -d /dev/sdX -i
```

应该可以看到硬盘的详细信息了。请确保序列号与修改目标匹配。

通过以下命令检查硬盘支持的逻辑区块大小：

```
# openSeaChest_Format -d /dev/sdX --showSupportedFormats
```

如果其中包含 4096，你可以使用以下命令将逻辑扇区大小修改为对应值：

```
# openSeaChest_Format -d /dev/sdX --setSectorSize=4096 --confirm this-will-erase-data
```
修改需要数分钟，完成后你的硬盘扇区就是 4 KiB 原生大小了。

分区对齐

正确的分区对齐可避免过多的读写操作。对于个人电脑而言，通常建议将各分区的起始端和大小对齐到 1 MiB（1 048 576 bytes）位置。这一操作涵盖了多数常见的页和块大小场景（通常的有 1 MiB，512 KiB，128 KiB，4 KiB 和 512 B）。

警告：分区对齐错误将造成无法在 dm-crypt/LUKS 上使用 4096 字节扇区大小，参见 。

- fdisk，cfdisk 和 sfdisk 会自动处理分区对齐。
- gdisk 和 cgdisk 会自动处理分区对齐。
- *sgdisk* 默认只对齐分区的起始端。可以通过 `-I`/`--align-end` 选项额外启用分区大小和末端对齐。
- Parted 只对齐分区起始端，而不对齐分区大小和末端。在创建分区时，请确保使用 mebibytes 或是更大的 IEC 二进制单位指定分区末端。

可以通过 checkpartitionsalignment.sh bash 脚本来使用 Parted 和 awk 检查对齐。

dm-crypt

从 Cryptsetup 2.4.0开始，`luksFormat`会自动检测 LUKS2 格式的最佳加密扇区大小。

但要实现这一点，需要设备报告正确的默认扇区大小，详情请参考#修改扇区大小。

执行 `cryptsetup luksFormat` 后，你可以使用如下命令确认 LUKS2 卷所使用的扇区大小：

```
# cryptsetup luksDump device | grep sector
```
如果默认扇区大小不正确，可以使用默认选项并强制创建一个扇区大小为 4 KiB 的 LUKS2 容器：

```
# cryptsetup luksFormat --sector-size=4096 device
```
如果请求的大小与设备不匹配，命令会出现报错并终止：

```
# cryptsetup luksFormat --sector-size 4096 device
(...)
Verify passphrase:
Device size is not aligned to requested sector size.
```
注意：参阅 cryptsetup issue 585 来了解在硬盘确实使用 4 KiB 物理扇区的情况下，命令仍可能失败的原因。

如果您使用了错误的扇区大小对设备进行加密，可以使用如下命令重新加密设备：

警告：所包含的文件系统的块大小必须是 4096 字节或其倍数，否则就会崩溃。

```
# cryptsetup reencrypt --sector-size=4096 device
```

文件系统

在 4Kn（物理和逻辑扇区大小为 4096 字节）硬盘上，所有 *mkfs* 工具都会使用 4096 字节块大小。在 512e（4096 字节物理扇区大小，512 字节逻辑扇区大小）及 512n（物理和逻辑扇区大小为 512 字节）硬盘上，不同的 *mkfs* 工具会使用不同的值。

<table class="wikitable"><caption>非 4Kn 硬盘的文件系统块大小（字节）
</caption><tbody><tr><th>mkfs 工具</th><th>512e 硬盘</th><th>512n 硬盘
</th></tr><tr><td>mkfs.bcachefs</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr><tr><td>mkfs.btrfs(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td></tr><tr><td>mkfs.exfat(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr><tr><td>mkfs.ext4(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096¹</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096¹</td></tr><tr><td>mkfs.fat(8)</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">512</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr><tr><td>mkfs.f2fs(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-

align: middle; text-align: center;">4096</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td></tr><tr><td>mkfs.jfs(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td></tr><tr><td>mkfs.nilfs2(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td></tr><tr><td>mkfs.ntfs(8)</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">512</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr><tr><td>mkfs.reiserfs(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td></tr><tr><td>mkfs.udf(8)</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">512</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr><tr><td>mkfs.xfs(8)</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr><tr><td>mkswap(8)</td><td

data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">4096</td></tr><tr><td>zpool-create(8)</td><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">512</td><td data-sort-value="2" class="archwiki-table-cell-background-orange" style="vertical-align: middle; text-align: center;">512</td></tr></tbody></table>

1. mkfs.ext4(8) 默认为小于 512 MiB 的文件系统使用 1024 字节块大小，为 512MiB 及以上的使用 4096 字节大小。

如果存储设备没有汇报正确的扇区大小，你可以手动将分区按照物理扇区大小进行格式化。

特别是对于由固件管理的*叠瓦式磁记录*（SMR）硬盘，例如在物理扇区大小为 4096 字节的设备上使用了 512 字节逻辑扇区大小，它们将受到严重影响。这类磁盘有不同的写入性能区间，且在空闲时会进行重映射和重分配。但是当出现大量写入活动时（例如 RAID resilvering，备份，写入大量小文件，rsync 等），由于高性能写入区被耗尽且叠瓦区会拖慢扇区转换层，文件系统扇区与物理扇区大小不一致可能导致写入速度降到每秒个位数兆字节。

注意：在 x86_64 系统上，Linux 无法挂载扇区大于 4 KiB 的文件系统。具体细节和当前进度可参考 Large block sizes (LBS)。

以下为一些将扇区显式设为 4096 字节大小的命令示例：

- Bcachefs:

```
# bcachefs format --block_size=4096 /dev/device0 /dev/deviceN --replicas=n
```

- exFAT:

```
# mkfs.exfat -s 4096 /dev/device
```

- ext4:

```
# mkfs.ext4 -b 4096 /dev/device
```

- FAT:

```
# mkfs.fat -S 4096 /dev/device
```

- NTFS-3G:

```
# mkfs.ntfs -Q -s 4096 /dev/device
```

- UDF:

```
# mkfs.udf -b 4096 /dev/device
```

- XFS:

```
# mkfs.xfs -s size=4096 /dev/device
```

- ZFS:

```
# zpool create -o ashift=12 poolname raidz device0 … deviceN
```

已知问题

Syslinux 与 FAT

Syslinux 不支持除 512 字节扇区大小外的 FAT 文件系统。

systemd-homed

systemd-homed 不支持 4Kn 硬盘和 4096 字节扇区大小的 LUKS。具体信息请查看 systemd issue 30393 和 30394。

参考

- Western Digital's Advanced Format: The 4K Sector Transition Begins
- White paper entitled "Advanced Format Technology."
- Failure to align one's HDD results in poor read/write performance. See for specific examples.

7.9 固态硬盘

难度 高级 | **篇幅** 29.7 KB | **所属** 存储

本节将讲到

- 使用
- 故障排除
- 固件更新
- 另见

本文介绍如何管理固态硬盘（SSD）等基于闪存的存储设备。

如果要针对特定目的对 SSD 进行分区，请参考针对闪存优化的文件系统清单。

对于一般用途，您只需选择喜好的文件系统并启用 #TRIM。

使用

TRIM

在机械硬盘上，删除操作是在文件系统层级上处理的，而 SSD 会向硬盘主控通知特定区域的空间可以被再次使用。由于每次写入操作都会对闪存单元一定磨损，因此主控会使用算法来将写操作平衡到闪存的所有单元上，这被称作耗损平均技术。在不使用 NVMe DEALLOCATE#NVM_Express)，SAS UNMAP 或 ATA_TRIM 命令（多数 SSD 都支持）的情况下，如果没有空余的存储块，主控就需要反复搬运数据以空出一个最小擦除单元，并擦除该单元以写入数据，导致写入耗时增加（详见：写入放大）。关于 SSD 空间占满前后的性能对比可参考 TechSpot 的评测。

注意：如果要使用 TRIM，**只能**在定期 TRIM 和持续性 TRIM 间二选一。Linux 社区认为持续性 TRIM **不是**首选的 TRIM 方法，例如 Ubuntu 就默认使用定期 TRIM ，Debian 也不建议使用持续性 TRIM，红帽建议尽可能选择使用定期 TRIM 。

自 Linux 内核版本 3.8 开始，对 TRIM 的支持不断被添加到不同的文件系统中。见下表：

文件系统	持续性 TRIM （`discard` 选项）	定期 TRIM （*fstrim*）	参考与备注
Bcachefs	是	否	
Btrfs	是	是	从 6.2 内核版本开始默认启用异步 discard
exFAT	是	是	从 5.13 内核版本开始支持 *fstrim*
ext3	是	是	
ext4	是	是	“discard, nodiscard(*)” #options)
F2FS	是	是	
JFS	是	是	
NILFS2	是	是	
NTFS	是	否	ntfs3 内核驱动仅支持持续性 TRIM
否	是	NTFS-3G 驱动仅支持定期 TRIM	
VFAT	是	是	从 4.19 内核版本开始支持 *fstrim*
XFS	是	是	
Swap	是	否	理论上不是个“文件系统”，但也适用于 TRIM。“once” 选项在启动时提供了定期 TRIM 的功能，具体请参考 swapon(8)

警告：在启用 TRIM 功能前，请确保 SSD 支持 TRIM，否则可能导致数据丢失！

要检查 TRIM 支持，执行：

```
$ lsblk --discard
```

若 DISC-GRAN（discard granularity）和 DISC-MAX（discard max bytes）列上的数值不为零，则表示对应设备支持 TRIM。

对于 SATA SSD，可通过 hdparm包 检测 TRIM 支持：以根用户执行 `hdparm -I /dev/sda | grep TRIM`。注意：`hdparm` 不支持 NVMe SSD。

定期 TRIM

util-linux包 提供了 `fstrim.service` 和 `fstrim.timer` 两个 systemd 单元文件。启用 `fstrim.timer` 计时器会在每周激活服务，在所有已挂载的支持 *discard* 操作的文件系统上执行 fstrim(8)。

该计时器使用 `/var/lib/systemd/timers/stamp-fstrim.timer`（将在服务第一次启动时创建）的时间戳来判断上次运行的时间。因此，不必担心服务被过于频繁地调用（类似 *anacron*）。

单元的状态与活动记录可通过 journalctl 查看。若要修改运行的周期或执行的指令，可编辑单元文件。

持续性 TRIM

除定期执行 TRIM 指令外（若使用 `fstrim.timer` 则默认为每周一次），也可每次在文件被删除后就立即执行 TRIM 指令（这被称为持续性 TRIM）。

警告：在 SATA 3.1 前，所有 TRIM 指令都是非队列化的，因此使用持续性 TRIM 将造成系统频繁卡顿，这种情况下应使用 #定期 TRIM。另外，某些设备（具体请参考 Linux 源码中的 `__ata_dev_quirks` 内带有 `ATA_QUIRK_NO_NCQ_TRIM` 的项）使用队列化 TRIM 命令会造成严重的数据损坏，因此系统会对这些设备强制发送非队列化的 TRIM 命令，具体请参考 Wikipedia:Trim_(computing)#Disadvantages#Disadvantages)。

要使用持续性 TRIM，在 `/etc/fstab` 中对应挂载点指定 `discard` 选项：

```
/dev/sda1  /           ext4  defaults,discard  0  1
```

注意：不能在 `/etc/fstab` 中为 XFS / 分区指定 discard 挂载选项。根据此帖子，必须使用 `rootflags=discard` 内核参数。

对于Ext4文件系统，也可用 *tune2fs* 将 `discard` 设置为一个默认挂载选项：

```
# tune2fs -o discard /dev/sdXY
```

对于可移动设备，由于配置后的分区在其它设备上也会使用默认挂载选项，使用此方式而不是在 `/etc/fstab` 中新增条目尤其有用。这样，在其他计算机上挂载分区时，就不需要每次修改 `/etc/fstab` 了。

注意：此默认挂载选项不会在 `/proc/mounts` 中列出。

Trim 整个设备

当在全新安装或想卖掉你的 SSD 时，你可能想 Trim 整个设备，这时候可以使用 blkdiscard 命令。

LVM

由 LVM 逻辑卷上的文件系统产生的 TRIM 请求将直达对应物理卷，无需额外的配置。

LVM 操作（*lvremove*，*lvreduce*等）默认不会产生 TRIM 请求，以便使用 vgcfgrestore(8) 恢复之前的卷组设定。`/etc/lvm/lvm.conf` 中的 `issue_discards` 设置决定是否在逻辑卷不再占用物理卷空间时将 discard 发送给底层物理卷。

注意：在修改 `issue_discards` 设置前，请仔细阅读`/etc/lvm/lvm.conf`中的注释。`issue_discards` 设置不会影响由逻辑卷文件系统产生的 TRIM 请求（如在文件系统内删除文件）传递到硬盘，也不会影响 thin pool 中的空间管理。

警告：启用 `issue_discards` 后，将不能再使用 *vgcfgrestore* 恢复卷组元数据。一旦执行 LVM 命令，将无法撤销。

dm-crypt

警告：dm-crypt 支持将 discard 请求传递到底层块设备上。这不一定能提升 SSD 的性能，而且还有安全风险，因此默认不被启用。

为 LUKS 和普通 dm-crypt 设备启用 discard 的步骤请参考 dm-crypt/特殊应用#固态硬盘的Discard/TRIM支持。

swap

可以通过将 `discard` 选项添加到 fstab 中的 swap 设备条目或调用 `swapon` 时传递 `--discard` 选项为 swap 启用 discard。

在使用 Systemd#GPT分区自动挂载时，discard *不会*被自动应用到 swap 分区。

See swapon(8) for discussion on when swap is discarded: `discard=once` or `discard=pages`. If `discard` is specified without a specific mode, the default is to enable both.

警告：对于使用 mdadm 的 RAID 配置，为 swapon 启用 discard 会导致系统在启动和运行时锁死。

提升性能

参见性能优化#存储设备。

清空 SSD

有时用户可能会想将 SSD 的存储单元重置回出厂状态，从而恢复到出厂时的写入性能。由于 TRIM 只对文件删除有用，而对替换操作（如增量保存）无能为力，因此就算 SSD 带有原生 TRIM 支持，其写入性能还是会随时间逐渐下降。

无论是 SATA 还是 NVMe SSD，都可以参考固态硬盘/存储单元擦除中的对应步骤进行重置。

注意：如果你想通过清除存储单元来清除数据，靠 SSD 控制器来执行该操作可能并不安全可靠。如果你不信任 SSD 制造商或担心固件中潜在的 bug，可以参考 安全地擦除磁盘#闪存 中的信息和范例进行手动擦除。

安全

冻结（Frozen）模式

有些主板的固件会在初始化时向 SATA 设备发送 ATA SECURITY FREEZE LOCK 命令，将硬盘设为冻结模式（即切换到 SEC2 状态，禁用安全、非锁定并冻结）。同样，有些 SSD 和 HDD 在出厂时也会设成该状态。可以通过 hdparm 和 smartctl 的输出确认该现象：

```
# hdparm -I /dev/sda
Security:
	Master password revision code = 65534
		supported
	not	enabled
	not	locked
		frozen
	not	expired: security count
		supported: enhanced erase
	4min for SECURITY ERASE UNIT. 2min for ENHANCED SECURITY ERASE UNIT.
# smartctl -g security /dev/sda
ATA Security is:  Disabled, frozen [SEC2]
```

格式化或安装系统等操作不受冻结模式的影响。

上面的 *hdparm* 输出显示设备在启动时**未被 HDD 密码锁定**，**frozen** 冻结状态可防止恶意软件在运行时对设备设置密码进行锁定。

如果你想手动为“冻结”的设备设置密码，需要主板 BIOS 支持该功能的。由于硬件加密的需要，很多笔记本都支持该功能，但台式机/服务器主板上则不然。以 Intel DH67CL/BL 主板为例，需要通过板上跳线将主板设为“维护模式”才能访问该设置。

警告：除非你十分清楚自己在干什么，否则不要试图用 *hdparm* 来改变上述的 **lock** 安全设置。

如果你想擦除 SSD，请参考安全地擦除磁盘#hdparm 和 /存储单元擦除。

从睡眠中唤醒时设置 SATA SSD 为冻结模式

当从 S3 睡眠状态唤醒时，SATA SSD 很可能会回到 SEC1 状态（禁用安全、非锁定、非冻结），导致其有可能受 /存储单元擦除中描述的 ATA SECURITY ERASE UNIT 命令攻击。

要防止这一问题，可以在每次唤醒后运行一个脚本：

```
/usr/lib/systemd/system-sleep/ssd-freeze.sh
#!/bin/sh
if [ "$1" = 'post' ]; then
	sleep 1
	if hdparm --security-freeze /dev/disk/by-id/ata-name-of-disk; then
		logger "$0: SSD freeze command executed successfully"
	else
		logger "$0: SSD freeze command failed"
```

```
        fi
fi
```

如果系统有多个存储设备和/或移动 USB 盘，可以通过 Hdparm#使用 udev 规则使配置持久化为所有硬盘（包括机械硬盘）指定 `--security-freeze`。

硬件加密

正如#冻结（Frozen）模式中提到的，在设备支持的情况下，在 BIOS 中为存储设备（SSD/HDD）设置密码可能同时会启用硬件加密。如果设备同时符合 OPAL 标准，即使 BIOS 没有设置密码的功能，硬件加密也可能启动，具体请参考 Self-encrypting drives。

故障排除

你遇到的问题可能是由于 SSD 固件而不是 Linux 产生的。在尝试排除故障前，请检查固件是否有更新：

- SSD 的固件
- 主板的 BIOS/UEFI 固件

即使是固件的 bug，也可能在不更新固件的情况下避免。若没有固件更新可用，或你不想进行固件更新，以下内容可能有所帮助。

处理 NCQ 错误

部分 SSD 和 SATA 芯片组在 Linux 的原生命令队列（NCQ）下不能正常工作。通过 journal 可看到如下错误信息:

```
ata9: exception Emask 0x0 SAct 0xf SErr 0x0 action 0x10 frozen
ata9.00: failed command: READ FPDMA QUEUED
ata9.00: cmd 60/04:00:d4:82:85/00:00:1f:00:00/40 tag 0 ncq 2048 in
res 40/00:18:d3:82:85/00:00:1f:00:00/40 Emask 0x4 (timeout)
```

要在系统启动时禁用 NCQ，需要在引导加载程序配置的内核命令行中添加 `libata.force=noncq`。例如，要仅为硬盘 0 端口 9 禁用 NCQ，可以使用：`libata.force=9.00:noncq`

或者，可通过 sysfs 在不重启的情况下为指定设备禁用 NCQ：

```
# echo 1 > /sys/block/sdX/device/queue_depth
```

如果更新固件并完成操作后问题仍未解决或导致了其他问题，请提交 bug 报告。

处理与 SATA 电源管理有关的错误

某些 SSD（如 Transcend MTS400 或 Crucial M550）搭配特定 SATA 控制器时，在 ALPM 启用时会出现故障。

Linux 从 4.16 版本开始默认启用，也可能会被节能守护进程（例如 TLP 和 Laptop Mode Tools）启用。更多信息请参考电源管理#SATA 活动链路电源管理。

支持 TRIM 的外接 SSD

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Several style and formatting problems, especially too many explicit commands - see Help:Style#File editing requests.（在Talk:固态硬盘讨论）

有多个 USB 转 SATA 桥接芯片（如 VL715，VL716 等）及 USB 转 PCIe 桥接芯片（如 IB-1817M-C31 外置 NVMe 硬盘盒使用的 JMicron JMS583）支持类似 TRIM 的命令，这些命令可通过 USB Attached SCSI 驱动程序（在 Linux 下称为"uas"）发送。

然而内核可能不会自动检测到并启用这一功能。 假设有问题的设备为 /dev/sdX，可以通过 sg3_utils包 使用以下命令确定是否为这种情况：

```
# sg_readcap -l /dev/sdX
```

如果输出中有一行写着“Logical block provisioning: lbpme=0”，就意味着由于没有设置 LBPME 位，内核认为该设备不支持“Logical Block Provisioning Management”。

在这种情况下，就需要找出设备上的“Logical Block Provisioning”中的“Vital Product Data”（VPD）页是否说明了支持的解除数据映射机制。可以使用该命令进行查看：

```
# sg_vpd -a /dev/sdX
```

查看有无类似下面的输出:

```
Unmap command supported (LBPU): 1
Write same (16) with unmap bit supported (LBPWS): 0
Write same (10) with unmap bit supported (LBPWS10): 0
```

在该示例中，设备支持“UNMAP”命令。

此外，查看以下命令的输出：

```
$ cat /sys/block/sdX/device/scsi_disk/*/provisioning_mode
```

如果输出为“full”，则表明内核没有检测到该设备有能力解除数据映射。 除了“full”外，内核的 SCSI 存储驱动目前可以识别以下 provisioning_mode 的值：

```
unmap
writesame_16
writesame_10
writesame_zero
disabled
```

对于以上示例，现在可以通过将“unmap”写入到“provisioning_mode”让内核使用“unmap”模式：

```
# echo "unmap" >/sys/block/sdX/device/scsi_disk/*/provisioning_mode
```

现在就可以在 /dev/sdX 上使用 blkdiscard 等工具，或是对 /dev/sdX 上的文件系统上使用 fstrim。

如果想在特定厂商或产品设备连接时自动启用一个“provisioning_mode”，可以使用 udev。首先找到 USB Vendor 和 Product ID：

```
$ cat /sys/block/sdX/../../../../../../idVendor
$ cat /sys/block/sdX/../../../../../../idProduct
```

然后创建或修改相应的 udev 规则（本例中使用 idVendor 152d 和 idProduct 0583）：

```
# echo 'ACTION=="add|change", ATTRS{idVendor}=="152d", ATTRS{idProduct}=="0583", SUBSYSTEM=="scsi_disk", ATTR{provisioning_mode}="unmap"' >>/etc/udev/rules.d/10-uas-discard.rules
```

（也可以使用 `lsusb` 命令来查看 idVendor/idProduct。）

固件更新

如果设备制造商支持，建议使用 fwupd 工具来更新固件。

检查当前固件版本：

```
# smartctl -i /dev/ssd_device
```

威刚（ADATA）

威刚不支持在 Linux 下更新 SSD 固件。可以在支持页下载仅支持 Windows 的 SSD ToolBox 工具，也可从支持页下载 ADATA XPG 来对 SSD 进行监控、TRIM、基准测试或更新固件。

警告：不建议尝试通过 Wine 来更新固件，Wine 不是被设计来处理硬件接口操作的。使用 Wine 可能使固件更新不完整，导致 SSD 变砖。

英睿达（Crucial）

英睿达提供了以 ISO 镜像文件升级固件的选项。在 SSD 支持页选择产品后，下载“Manual Boot File”可获取 ISO 镜像。

注意：英睿达提供的 ISO 镜像似乎不是 hybrid 格式。如果直接使用 `dd` 命令将镜像复制到设备上，由于没有配置 MBR，将导致无法从该设备启动。要解决这一问题，可安装 syslinux包 并执行 `isohybrid path/to/image.iso`。

Crucial M4 用户可通 过`smartctl` 检查是否有需要的固件更新：

```
$ smartctl --all /dev/sdX
==> WARNING: This drive may hang after 5184 hours of power-on time:
https://www.tomshardware.com/news/Crucial-m4-Firmware-BSOD,14544.html
See the following web page for firmware updates:
https://www.crucial.com/usa/en/support-ssd
```

建议看见这个警告的用户备份所有重要数据并立即更新。参见该指引来使用 ISO 镜像和 grub 更新 Crucial MX100 的固件。

英特尔（Intel）

对于无法使用 Windows 版本 Intel® Solid-State Drive Toolbox 软件的系统，英特尔提供了一个基于 Linux live 系统的固件更新工具。

此外，还可用使用 Intel Memory and Storage (MAS) Tool（intel-mas-cli-toolAUR）命令行工具在 Linux 下刷入固件。（其PDF用户指南）

例如，要检查固件状态：

```
# intelmas show -intelssd 0
DevicePath : /dev/nvme0n1
DeviceStatus : Healthy
Firmware : 002C
FirmwareUpdateAvailable : The selected Intel SSD contains current firmware as of this tool release.
```

如果只有一个 Intel SSD，可以省略 `-intelssd 0`；如果要指定第二块 SSD，则修改为 `-intelssd 1`，依此类推。

如果有可用的更新，可通过 `intelmas load -intelssd 0` 来应用。PDF 用户指南建议进行操作后重启并再进行一次。所有设备的最新固件都作为 MAS 工具的一部分发布，无需单独下载。

金士顿（Kingston）

基于 Sandforce 的硬盘可以使用 KUF 工具，可从 kingston_fw_updaterAUR 获取。

Mushkin

不那么出名的 Mushkin 牌固态硬盘同样使用 Sandforce 控制器，也提供了 Linux 版的升级工具 (和 Kingston 的几乎一样)。

OCZ

OCZ 为 Linux 提供了 Command Line Online Update Tool (CLOUT)。相关软件包可在 AUR 找到：ocz-ssd-utilityAUR、ocztoolboxAUR、oczcloutAUR。

三星（Samsung）

除了使用 Magician 软件外，也有其它方法升级固件（尽管三星认为这是“不受支持”的）。Magician 软件可以创建包含固件更新的启动盘，然而三星不再为消费级 SSD 提供该软件。此外，三星提供的可引导 ISO 镜像也能用来更新固件。另外一种方法是使用 *magician* 工具（samsung_magician-consumer-ssdAUR）。Magician 只支持三星品牌的 SSD，不支持三星为其它 OEM（如联想）制造的 SSD。

注意：对于这些信息，三星提供得并不十分明确。他们似乎有 4 个不同的固件更新页面，每个都介绍了不同的更新方法。

如果你想通过在 Linux 下创建的 Live USB 来更新固件（而不是在 Windows 下使用三星的 Magician 软件），参见。注意，该文章描述的通过 MBR 创建可启动 U 盘的方法在一些新主板上（例如 Intel NUC）不再受支持。

在 Linux 下更新

以下方法可以原生（不使用可启动 U 盘）更新 SSD 固件。首先，访问三星的下载页，进入”Samsung SSD Firmware“一节，然后下载适用于你的 SSD 的最新固件（ISO 镜像）。

注意：若 ISO 镜像中没有下面提到的 `initrd` Linux 镜像，请跳到#较旧的 SSD 一节。

从 ISO 镜像中解压 Linux `initrd` 镜像（将 *samsung_ssd_firmware* 替换为对应的 ISO 镜像文件名）：

```
$ bsdtar xf samsung_ssd_firmware.iso initrd
```

解压 `root/fumagician/`。该目录中包含固件更新文件：

```
$ bsdtar xf initrd root/fumagician
```

最后，以根用户权限运行 `root/fumagician/fumagician`，确认成功更新固件后，重启计算机。

如果重启后固件版本没有变化，可以执行 `root/fumagician/fumagician 2> log` 并检查日志里的报错。例如，如果日志里写着 'unzip is not available'，就说明需要安装 unzip 或从 initrd 中提取 unzip。

较旧的 SSD

有些 SSD 固件 ISO 中包含 FreeDOS 镜像而不是 Linux `initrd` 镜像，故需要采取不同的更新方法。下表展示了这些 SSD 及固件的相对路径：

SSD 型号	FreeDOS 镜像路径	固件包路径
470，830	`BTDSK.IMG`	`SSR/`
840	`isolinux/btdsk.img`	`samsung/DSRD/`
840 EVO（mSATA），Pro	`ISOLINUX/BTDSK.IMG`	

首先，从 ISO 镜像中解压 FreeDOS 镜像（将 *samsung_ssd_firmware* 替换为对应的 ISO 镜像文件名，*freedos_image_path* 替换为 FreeDOS 镜像路径，下同）：

```
$ bsdtar xf samsung_ssd_firmware.iso freedos_image_path
```

将 FreeDOS 镜像挂载到 `/mnt/`：

```
# mount freedos_image_path /mnt
```

从 Magician SSD management utility 的 *Disk Number* 获取 SSD 对应的硬盘编号：

```
# magician --list
```

更新 SSD 固件（将 *firmware_package_path* 替换为对应的固件包路径）：

```
# magician --disk Disk Number --firmware-update --fwpackage-
path /mnt/firmware_package_path
```

最后，以根用户权限执行 `magician --list`，检查输出中 *Firmware* 对应的固件版本是否成功更新，并重启计算机。

闪迪（SanDisk）

对于 SanDisk SSD Toolkit 不支持的操作系统，闪迪提供了 ISO 镜像来进行固件更新。

你必须选择对应 *SSD 型号*和*容量*（例如 60GB 或 256GB）正确的固件。烧录 ISO 镜像后，从创建的 CD/DVD 启动盘启动计算机即可（或许能使用 USB 启动盘）。

此外，由于 ISO 镜像只包含了 Linux 内核及 initrd，也可以将它们提取到 `/boot` 分区下，并用 GRUB 或 Syslinux 启动来更新固件。

另见：

- SanDisk Extreme SSD Manual Firmware update version R211
- SanDisk Ultra SSD Manual Firmware update version 365A13F0
- SanDisk Ultra+ SSD Manual Firmware update version X2316RL - 以根用户权限执行 `smartctl -i dev/disk/by-id/*SanDisk!(*part*)` 检查是否使用了“H2”或“HP”型号。

另见

- Discussion on Reddit about installing Arch on an SSD
- Re: Varying Leafsize and Nodesize in Btrfs
- Re: SSD alignment and Btrfs sector size
- Erase Block (Alignment) Misinformation?
- Is alignment to erase block size needed for modern SSD's?
- Btrfs support for efficient SSD operation (data blocks alignment)

虚拟机管理

本章解决同一台机器上跑多个系统：确认硬件支持虚拟化，把 Arch 装进虚拟机，打通数据与网络。

必读 KVM 与 QEMU：前者给出硬件支持与 Virtio 准虚拟化的判断标准，后者从图形前端、创建与运行讲到数据交互与网络。Xen 的 HVM 与 PV 两类 domU 只在已有宿主机时可看；VirtualBox、VMware 与在 Windows 里装 Arch 的 WSL 三节按需跳读。最容易踩的坑是把“装好虚拟机软件”当成“配好了虚拟化”，硬件支持、准虚拟化驱动、数据与网络通道缺一不可。

8.1　KVM

难度 进阶 | **篇幅** 10.1 KB | **所属** 虚拟机管理

本节将讲到

- 检查 KVM 支持
- Virtio 准虚拟化
- 如何使用 KVM
- 小贴士与小技巧
- 参阅

翻译状态：

- 本文（或部分内容）译自 KVM，最近一次同步于 2024-11-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

KVM，基于内核的虚拟机，是内置于 Linux 内核的 hypervisor。在功能上类似于 Xen，但运行起来更简便。与使用模拟的原生 QEMU 不同的是，KVM 是 QEMU 的一种特殊的运行模式，这种模式通过内核模块使用 CPU 扩展（HVM）进行虚拟化。

通过 KVM，可以运行多台未修改的 GNU/Linux、Windows 或任何其他操作系统的虚拟机（详情请参阅客户机支持状态）。每台虚拟机都有独享的虚拟硬件：网卡、硬盘、显卡等。

有关 KVM 与 Xen、VMware 和 QEMU 的区别，可查看 KVM FAQ。

本页面不包含使用KVM作为后端的其他虚拟机的通用功能。对于这些信息，你应查看相关的文档。

检查 KVM 支持

硬件支持

KVM 需要虚拟机宿主的处理器支持虚拟化（对于 Intel 处理器来说是 VT-x，对于 AMD 处理器来说是 AMD-V）。你可通过以下命令来检查你的处理器是否支持硬件虚拟化：

```
$ LC_ALL=C.UTF-8 lscpu | grep Virtualization
```

或者：

```
$ grep -E --color=auto 'vmx|svm|0xc0f' /proc/cpuinfo
```

如果运行后没有显示，那么你的处理器不支持硬件虚拟化，你不能使用KVM。

注意：你可能需要在 BIOS 中启用虚拟化支持。10 年内所有由 AMD 和 Intel 生产的 x86_64 处理器都支持虚拟化。如果你的处理器看上去不支持虚拟化，几乎可以确定是因为这项功能在 BIOS 中未启用。

内核支持

Arch Linux 内核提供了相应的内核模块来支持KVM。

- 你可以通过以下命令来检查内核中是否已包含必要的模块（`kvm` 以及 `kvm_amd` 和 `kvm_intel` 中的一个）：

```
$ zgrep CONFIG_KVM /proc/config.gz
```

只有当模块设置为 `y` 或 `m` 时才可用。

- 然后，确认这些内核模块已自动加载：

```
$ lsmod | grep kvm
kvm_intel             245760  0
kvmgt                  28672  0
mdev                   20480  2 kvmgt,vfio_mdev
vfio                   32768  3 kvmgt,vfio_mdev,vfio_iommu_type1
kvm                   737280  2 kvmgt,kvm_intel
irqbypass              16384  1 kvm
```

如果运行后没有显示，那么需要手动加载这些模块。

提示：如果 modprobe `kvm_intel` 或 `kvm_amd` 失败，但 modprobe `kvm` 成功，并且 `lscpu` 声称支持硬件加速，检查 BIOS 设置。某些厂商，特别是笔记本电脑厂商，默认禁用这些处理器扩展。modprobe 失败后，dmesg 的输出可以告诉你这些扩展是硬件不支持还是在 BIOS 中禁用。

Virtio 准虚拟化

准虚拟化为客户机提供了一种使用主机上设备的快速有效的通信方式。KVM 使用 **Virtio** API 作为虚拟机管理程序和客户机之间的连接层，为虚拟机提供准虚拟化设备。

所有 Virtio 设备都包括两部分：主机设备和客户机驱动程序。

内核支持

用以下命令检查**虚拟机中**内核的 VIRTIO 模块是否可用：

```
$ zgrep VIRTIO /proc/config.gz
```

然后，检查这些内核模块是否已自动加载：

```
$ lsmod | grep virtio
```

如果运行后没有显示，那么需要手动加载这些模块。

准虚拟化设备列表

- 网络设备 (virtio-net)
- 块设备 (virtio-blk)
- 控制器设备 (virtio-scsi)
- 串口设备 (virtio-serial)
- 气球设备 (virtio-balloon)

如何使用 KVM

请参考QEMU。

小贴士与小技巧

注意：请参考 QEMU#Tips and tricks 和 QEMU#Troubleshooting 获取通用技巧。

嵌套虚拟化

嵌套虚拟化使现有虚拟机能够在第三方虚拟机管理程序和其他虚拟化云平台上运行，而无需对原始虚拟机或其网络进行任何修改。

在宿主机上，启用 `kvm_intel` 模块的嵌套虚拟化功能：

注意：AMD 操作也类似，只需在必要时将 `intel` 替换为 `amd`

```
# modprobe -r kvm_intel
# modprobe kvm_intel nested=1
```

要使嵌套虚拟化永久生效（请参考 Kernel modules#Setting module options）：

```
/etc/modprobe.d/kvm_intel.conf
options kvm_intel nested=1
```

确认嵌套虚拟化功能是否已激活：

```
$ cat /sys/module/kvm_intel/parameters/nested
Y
```

启用主机直通模式以将CPU的所有功能转发给客户操作系统:

1. 如果正在使用QEMU, 可以使用以下命令运行客户操作系统: `qemu-system-x86_64 -enable-kvm -cpu host`.
2. 如果使用virt-manager，请将CPU模式更改为 `host-passthrough`。

3. *如果使用virsh*, 使用`virsh edit *vm-name*`命令并更改 CPU 行为 `<cpu mode='host-passthrough' check='partial'/>`

现在，启动虚拟机并检查VMX标志是否存在

```
$ grep -E --color=auto 'vmx|svm' /proc/cpuinfo
```

开启内存大页

可以启用大页 (Huge Pages) 以提高虚拟机的性能。如果您正在使用最新版本的 Arch Linux 并且正在运行 KVM，那么这基本是开箱即用的。请检查是否存在目录 `/dev/hugepages`。如果不存在，请先创建它。现在，我们需要配置正确的权限。默认目录权限是 root 的 uid 和 gid 及 0755，但我们希望 kvm 组中的任何人都可以访问 hugepages。

在 `/etc/fstab`添加下列内容:

```
/etc/fstab
hugetlbfs       /dev/hugepages  hugetlbfs       mode=01770,gid=kvm        0 0
```

相较于通过 `gid=kvm` 直接指定组名，你也可以使用数字作为 gid，但数字 ID 必须要与 `kvm` 组相一致。`1770` 模式允许所有组成员创建文件，但成员间无法互相取消链接或重命名文件。确保 `/dev/hugepages` 已正确挂载：

```
# umount /dev/hugepages
# mount /dev/hugepages
$ mount | grep huge
hugetlbfs on /dev/hugepages type hugetlbfs (rw,relatime,mode=1770,gid=78)
```

现在可以计算需要多少大页内存了。查看一个大页的大小：

```
$ grep Hugepagesize /proc/meminfo
```

通常会是 2048 kB ≙ 2 MB，加入虚拟机的内存是 1024 MB. 1024 / 2 = 512. 额外增加一点，就变成 550。将此数值写入设置：

```
# echo 550 > /proc/sys/vm/nr_hugepages
```

如果剩余内存足够:

```
$ grep HugePages_Total /proc/meminfo
HugesPages_Total:  550
```

如果数值没有这么大，请关闭其他软件，或减少虚拟机的内存使用(number_of_pages x 2):

```
$ qemu-system-x86_64 -enable-kvm -m 1024 -mem-path /dev/hugepages -hda <disk_image>
[...]
```

注意 `-mem-path` 参数，这将启用大页。 虚拟机运行时，通过下面命令查看使用了多少个页：

```
$ grep HugePages /proc/meminfo
HugePages_Total:     550
HugePages_Free:       48
HugePages_Rsvd:        6
HugePages_Surp:        0
```

如果一切正常，可以默认启动大页了，将下面内容加入 `/etc/sysctl.d/40-hugepage.conf`:

```
/etc/sysctl.d/40-hugepage.conf
vm.nr_hugepages = 550
```

参阅：

- Linux 内核大页支持情况
- Debian Wiki - Hugepages

安全启动

启用 KVM 安全启动前需要满足一下几点：

1. 必须使用编译了安全启动的 UEFI.
2. UEFI 中必须包含加密密钥.

注意：Arch Linux 目前没有像 Fedora 等其他发行版一样提供安全启动密钥，如果要安全启动 Arch Linux, 需要创建自己的签名密钥并在执行下面命令后用密钥签名内核。更多信息请参考 Unified Extensible Firmware Interface/Secure Boot。

要启用 UEFI 安全启动支持，请安装 edk2-ovmf包 并将虚拟机设置为使用启用安全启动的 UEFI。如果是使用 libvirt，可以将如下内容加入虚拟机 XML 配置文件：

```
<os firmware="efi">
  <loader readonly="yes" secure="yes"
type="pflash">/usr/share/edk2/x64/OVMF_CODE.secboot.4m.fd</loader>
</os>
```

下一步需要加入一些密钥，这里我们使用 Microsoft 和 Redhat 的安全启动密钥。安装 virt-firmware包 并执行下面命令，将 `*vm_name*` 替换为虚拟机名称。

```
$ virt-fw-vars --input /var/lib/libvirt/qemu/nvram/vm_name_VARS.fd --
output /var/lib/libvirt/qemu/nvram/vm_name_SECURE_VARS.fd --secure-boot --enroll-redhat
```

编辑 libvirt XML 配置，将其指向新的 VARS 文件。

```
<os firmware="efi">
  <loader readonly="yes" secure="yes"
type="pflash">/usr/share/edk2/x64/OVMF_CODE.secboot.4m.fd</loader>
  <nvram template="/usr/share/edk2/x64/OVMF_VARS.4m.fd">/var/lib/libvirt/qemu/nvram/
{vm-name}_SECURE_VARS.fd</nvram>
</os>
```

这样安全启动应该就会自动启用了，可以通过虚拟机 UEFI 启动时按 `F2` 按键进行确认。

参阅

- KVM Howto
- KVM FAQ

8.2 在 WSL 上安装 Arch Linux

难度 进阶 | **篇幅** 12.8 KB | **所属** 虚拟机管理

本节将讲到

- 安装
- 提示和技巧
- 疑难解答
- 参见

作为 archlinux-wsl 项目的一部分，Arch Linux 提供官方 WSL（Windows Subsystem for Linux，适用于 Linux 的 Windows 子系统）镜像。

镜像每月构建并发布一次，旨在通过最简且完整的系统在 WSL 上提供完整的 Arch Linux 体验。

注意：需要 **WSL 2**。**不支持** WSL 1。以下内容均假定您使用最新稳定版的 WSL 2。

安装

安装 WSL

在 UEFI 设置中启用虚拟化，然后从 Microsoft Store 安装 **Windows Subsystem for Linux（适用于 Linux 的 Windows 子系统）**。

注意：Microsoft Store 内的 WSL 是默认版本。不要启用 "Windows Subsystem for Linux" 组件，也不要安装 WSL 内核或 WSLg 的 MSI 安装包，因为它们不再被需要。相比于 Windows 组件，使用 Microsoft Store 版本的 WSL 可让您更快地获取更新，WSLg 现也已被打包。

更新 WSL

要更新到最新稳定版的 WSL 和 WSLg，在具有管理员权限的 Windows 的命令行 Shell 中执行以下命令：

```
> wsl --update
```

要更新到最新的预发行版本，请换用以下命令：

```
> wsl --update --pre-release
```

在 WSL 上安装 Arch Linux

在已安装了 WSL 2 的 Windows 系统上遵循以下安装方式之一：

自动安装

在 Windows 的命令行 Shell 中执行该命令：

```
> wsl --install archlinux
```

然后您可以在开始菜单中通过 `archlinux` 应用程序在 WSL 中运行 Arch Linux，亦可在 Windows 的命令行 Shell 中执行 `wsl -d archlinux`。

如果您不希望使用 WSL 的默认安装路径和发行版名称，可以使用以下命令：

```
> wsl --install -d archlinux --name 自定义名称 --location 自定义路径
```

其中 `--name` 用于设置名称，`--location` 用于设置安装路径，例如：

```
> wsl --install -d archlinux --name Arrrrch --location D:\WSLOS\Arch
```

执行 `wsl --help` 命令以查看 WSL 的更多操作。

手动安装

下载最新的 Arch Linux C0 镜像，然后双击以进行安装或在 Windows 的命令行 Shell 中执行以下命令：

```
> wsl --install --from-file WSL_image
```

直接安装：

```
> wsl --install --from-file C:\Users\用户名\Downloads\archlinux-2025.04.01.121271.wsl
```

导入式安装（可自定义位置）：

```
> wsl --import archlinux D:\WSL\Arch D:\Download\archlinux-2025.04.01.121271.wsl
```

提示：这会以默认发行版名称 `archlinux` 安装 WSL 镜像，若您欲以不同名称导入，请添加 `--name *名称*` 选项。

然后您可以在开始菜单中通过 `archlinux` 应用程序在 WSL 中运行 Arch Linux，亦可在 Windows 的命令行 Shell 中执行 `wsl -d archlinux`。

提示和技巧

设定默认用户

要设定 `root` 以外的默认用户，首先确保该用户已被创建，然后将以下行添加到 `/etc/wsl.conf`：

```
[user]
default=username
```

关闭会话前记得为 `root` 用户设置密码，如果您被“拒之门外”，请在 Windows 主机的 CMD 中执行：

```
> wsl -u root
```

此更改将在下次会话启动时生效，要终止当前会话，请在 Windows 的命令行 Shell 中执行以下命令：

```
> wsl --terminate archlinux
```

若您使用 WSL 2.4.10 或更新版本，则可以使用以下命令为您的发行版设定默认用户：

```
> wsl --manage archlinux --set-default-user 用户名
```

此更改将在下次启动时生效。

用 WSLg 运行图形界面应用程序

WSLg（Windows Subsystem for Linux GUI）项目致力于让 WSL 支持 Linux 音频（PulseAudio）和图形界面（X11 和 Wayland）。

WSLg 默认启用，您可在 WSL 配置文件中将 `wsl2.guiApplications` 设为 `false` 以将其关闭。

硬件加速渲染

欲在 WSL 中启用 GPU 视频加速渲染，请安装以下软件包：

- mesa包 - 包含 OpenGL 所需的 `d3d12` Gallium 驱动
- vulkan-dzn包 - 包含实验性的 `dzn`（亦作 `microsoft-experimental`）Vulkan 驱动

还需要安装 vulkan-icd-loader包（要运行 32 位应用程序则亦需安装 lib32-vulkan-icd-loader包）。

```
~/.bashrc
export GALLIUM_DRIVER=d3d12

export LIBVA_DRIVER_NAME=d3d12
```

若 openGL 依然在英特尔 GPU 上使用 llvmpipe 软件渲染，则需要为 libedit 创建符号链接：

```
# ln -s /usr/lib/libedit.so /usr/lib/libedit.so.2
```

更多信息参见 Issue 996 和 Gentoo:Gentoo in WSL#OpenGL falling back to llvmpipe software renderer on Intel GPUs。

WSL 互操作

WSL 支持与 Windows 互操作，这让您可以从 WSL 运行 Windows 二进制程序。

其默认启用，您可在 `/etc/wsl.conf` 中将 `interop.enabled` 设定为 `false` 以将其禁用。

有一些工具能让您在 WSL 内使用 Windows 服务和功能。

从 Windows 桥接 SSH 代理服务

wsl2-ssh-agent 工具允许您在 WSL 内使用 Windows SSH 代理。

当您使用需要物理安全密钥甚至是 Windows Hello 的 `*-sk` SSH 密钥时，这会很有用。

安装 wsl2-ssh-agentAUR 并将以下行添加到您的 `~/.bashrc`：

```
eval "$(/usr/sbin/wsl2-ssh-agent)"
```

重启 shell，`SSH_AUTH_SOCK` 环境变量应该已正确设置。

使用 Windows Hello 进行 PAM 认证

WSL-Hello-Sudo 是一个使用 Windows Hello 进行用户身份认证的 PAM 插件。

安装 wsl-hello-sudo-binAUR 并运行 `/opt/wsl-hello-sudo/install.sh`。安装程序会复制一个 Windows 可执行程序到您选定的目录并存储用于身份认证的证书。

提示：WSL-Hello-Sudo 已经 4 年没有更新了，因此 wsl-hello-sudo-binAUR 打包了一个分支，合并了一些依赖更新。

欲使用 Windows Hello 进行认证，将 `auth sufficient pam_wsl_hello.so` 行添加到 `/etc/pam.d` 中相应组件的配置文件，例如 Sudo：

```
/etc/pam.d/sudo
#%PAM-1.0
auth            sufficient      pam_wsl_hello.so
auth            include         system-auth
account         include         system-auth
session         include         system-auth
```

向 WSL 连接设备

WSL 2 是一个 Hyper-V 虚拟机，这使得设备可以从主机（Windows）直连到客户机（WSL 2）。

挂载磁盘

WSL 2 支持附加并挂载 Windows 可用的磁盘。

欲如此做，首先请用以下 PowerShell 命令获取磁盘的 `DeviceID`：

```
> GET-CimInstance -query "SELECT * from Win32_DiskDrive"
```

找到欲挂载的磁盘后，请在 Windows 上执行以下命令（需要管理员权限）：

```
> wsl --mount DeviceID --bare
```

警告：这会使相应磁盘在 Windows 上下线，挂载前确保您已关闭所有正在使用该驱动器的应用程序。

附加后，您应当可以用 `lsblk` 看到该磁盘。

要取消挂载，请运行：

```
> wsl --unmount DeviceID
```

欲获取更多信息，请参阅官方文档（中文文档在此）。

连接 USB 设备

usbipd-win 项目可以将本地连接的 USB 共享到其他设备，包括 WSL 2。

首先请在 Windows 上安装该软件。您可以选择运行最新发布的安装程序（*.msi*）或使用 Windows 包管理器：

```
> winget install usbipd
```

安装完成后，请在 Windows 上执行以下命令，找到欲共享的 USB 设备，特别留意其 bus ID：

```
> usbipd list
```

执行以下命令以准备欲共享的 USB 设备（需要管理员权限）：

```
> usbipd bind --busid busid
```

警告：确保您已关闭了所有使用该 USB 设备的应用程序。

然后执行以下命令将 USB 设备连接到 WSL 2：

```
> usbipd attach --wsl --busid busid
```

连接后，您应当可以用 `lsusb` 看到该设备。

要与 USB 设备断开连接，请执行：

```
> usbipd detach --busid busid
```

欲获取更多信息，请参阅官方文档。

修改区域设置

WSL 默认会尝试将区域设置调整为与 Windows 相匹配。欲手动设置，请执行以下命令：

```
ln -sf /etc/locale.conf /etc/default/locale
```

然后像正常安装一样设定区域。

疑难解答

systemd 支持

Arch Linux WSL 镜像支持 systemd。

然而，有一些已知问题可能需要额外操作才能使 systemd 正常工作。

Docker 容器运行出错

从 WSL 运行 Docker 容器可能会出现如下报错：

```
Error response from daemon: path / is mounted on / but it is not a shared or slave
mount
Error: failed to start containers
```

有可能是 `docker run` 之类的命令一直挂起而不输出。

因为 Docker 需要根目录（`/`）被挂载为可共享。

欲修复，请运行：

```
# mount --make-rshared /
```

欲持久化该更改，您可以创建一个 systemd 服务，使其在启动时就执行命令：

```
/etc/systemd/system/mount-root-rshared.service
[Unit]
Description=Remount / with shared propagation
Requires=-.mount
After=-.mount

[Service]
Type=oneshot
ExecStart=/bin/mount --make-rshared /
```

```
[Install]
WantedBy=local-fs.target
```

然后启动/启用 `mount-root-rshared.service`。

关于该选项的更多信息，请参阅 mount(8) § Shared subtree operations。

参见

- Wikipedia:Windows Subsystem for Linux
- Gentoo:Gentoo in WSL
- 官方文档（英文）
- 官方文档（中文）
- WSLg GitHub Wiki

8.3 VirtualBox/在虚拟机中安装 Arch Linux

难度 进阶 | **篇幅** 15.1 KB | **所属** 虚拟机管理

本节将讲到

- 安装
- 故障排除

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： Partial translation（在 Talk:VirtualBox/在虚拟机中安装 Arch Linux# 中讨论）

本文介绍如何在 VirtualBox 中安装 Arch linux。

在虚拟机的虚拟光驱里加载 Arch Linux 的安装镜像，然后按照安装指南里的步骤继续即可。

安装

以 EFI 模式安装（可选）

如果你想在 VirtualBox 里以 EFI 模式安装 Arch Linux，这需要在虚拟机的设置窗口里，先从左侧选择 *System*，再从右侧选择 *Motherboard* 标签页，最后勾选 *Enable EFI (special OSes only)* 选项。从安装镜像的启动菜单里选好 Arch Linux 的内核之后，这时启动过程要卡顿一两分钟，之后就能正常进入系统，稍安勿躁。

等系统和启动引导程序安装成功之后，VirtualBox 会首先尝试从 ESP 加载 `/EFI/BOOT/BOOTX64.EFI`。如果这个首选的文件加载失败，VirtualBox 会从 ESP 根目录尝试加载 EFI shell 脚本 `startup.nsh`。这样来说，为了能顺利进入系统，你需要从下面几种方案选一个：

- 每次都从 EFI shell 手动选择启动引导程序
- 把启动引导程序的 .EFI 文件移动到默认位置：`/EFI/BOOT/BOOTX64.EFI`
- 自己编写 ESP 分区的 `/startup.nsh` 脚本，用这个脚本加载想使用的引导程序（例如 `\EFI\grub\grubx64.efi`）
- 从 ESP 分区的 startup.nsh 脚本直接启动到 Linux

虽然 VirtualBox 支持按 F2 进入自带的 VirtualBox Boot Manager 来管理 EFI 启动过程，但这部分功能还不完整而且有 bug。它还不能持久保存交互式设置的 EFI 变量。也就是说，开机时按下 F12 之后手动添加的、或者用 efibootmgr包 添加的 EFI 项目，重启后依然可以生效，但关机之后就会失效。

另见：UEFI VirtualBox installation boot problems.

安装客体机插件

VirtualBox 客体机插件为客体系统提供了必要的驱动与应用软件，其作用包括改善图像分辨率与鼠标支持等。在客体机的 Arch Linux 系统里：

- 若需要支持 X，安装 virtualbox-guest-utils包
- 若不需支持 X，安装 virtualbox-guest-utils-nox包

注意：

- 另一个安装方案是从宿主机（如果也是 Arch 系统）安装 virtualbox-guest-iso包，这个软件包里有供客体机安装插件的 ISO 文件。装好这个包之后，启动客体机，点击菜单项目 Devices -> Insert Guest Additions CD Image 即可加载安装镜像。
- 若要手动重新编译客体机的 VirtualBox 内核模块，以 root 权限运行 `rcvboxadd setup` 即可。

客体机里运行的插件和主体机上的 VirtualBox 程序版本需要匹配。否则某些功能（比如剪贴板互通）可能会失效。如果你在客体机里升级系统（比如运行了 `pacman -Syu`），那么宿主机上的 VirtualBox 也要跟进更新到最新版。VirtualBox GUI 菜单里的 "Check for updates" 功能未必够用，可以去官网 VirtualBox.org 再看看。

加载 VirtualBox 内核模块

若要开机自动加载模块，启用 `vboxservice.service` 服务即可。该服务还能起到将客户机时间与宿主机同步的功能。

若要手动加载模块，使用这个命令：

```
# modprobe -a vboxguest vboxsf vboxvideo
```

启动 VirtualBox 客体机服务

安装并加载了 VirtualBox 内核模块之后，如果你在虚拟机系统里使用 X，那么推荐启动这样一个客体机服务：服务的程序名字叫 `VBoxClient`，它与 X 窗口系统沟通来实现其功能。具体功能包括：

- 在宿主机与客体机之间互通剪贴板，实现鼠标拖拽文件
- 无缝窗口模式
- 宿主机的虚拟机窗口缩放之后，使客体机的显示分辨率与之自动匹配
- 检查宿主机的 VirtualBox 的版本

上述功能由专门的命令行参数来各自启用：

```
$ VBoxClient --clipboard --draganddrop --seamless --display --checkhostversion
```

为了方便，`VBoxClient-all` 这个 shell 脚本能取代上面这一整行命令。

virtualbox-guest-utils包 包提供了 XDG 自启动项目 `/etc/xdg/autostart/vboxclient.desktop`，这会在登录 X 时自动运行 `VBoxClient-all`。然而，如果你在用的桌面环境或窗口管理器不支持 XDG 自启动，那么你就需要手动配置。详见自动启动#桌面环境。

至此，你的 Arch Linux 应该能在虚拟机里正常运行了。需要指出的是，某些功能（如剪贴板互通）在 VirtualBox 里是默认禁用的。这需要在各个虚拟机的设置选项里手动开启（*Settings* > *General* > *Advanced* > *Shared Clipboard*）。

显卡加速

从 VirtualBox 选项里可以开启显卡加速。已知 GDM 显示管理器从 3.16 版本起会导致显卡加速失效。 如果你遇到类似问题，可以换另一种显示管理器（比如 LightDM）试试看。

启用共享目录

共享目录的设置在宿主机这边操作。启动 VirtualBox 的图形管理界面，在虚拟机的设置界面的 *Shared Folders* 标签页可以找到相关设置。可以设置的项目包括：目录在宿主机的位置 *Folder Path*，客户机挂载点的名字 *Folder name*，还有 *Read-only*，*Auto-mount*, *Make permanent* 等杂项。还有一种管理方法是使用 `VBoxManage`。详情可以参阅 VirtualBox 手册。

无论用哪一种方法来挂载共享目录，首先都要做些准备工作。

为了避免出现 `/sbin/mount.vboxsf: mounting failed with the error: No such device` 这种错误，首先要确保 `vboxsf` 内核模块已经加载。只要按照前面的步骤在加载客体机里加载内核模块，这一步应该没问题。

为了让挂载之后的目录能让 root 之外的用户也直接读写，还要：

- 安装 virtualbox-guest-utils包 软件包时会创建用户组 `vboxsf`（在前面的步骤就装过了）
- 你的用户需要加入到 `vboxsf` 用户组

手动挂载

在 Arch Linux 客体系统里挂载共享目录的命令是：

```
# mount -t vboxsf <共享目录的名字> <客户机系统的挂载点>
```

这个命令可以查看 vboxsf 的挂载参数：

```
# mount.vboxsf
```

假如用户不在 `vboxsf` 组里，用这个命令可以把挂载点的读写权限授权给他：

```
# mount -t vboxsf -o uid=1000,gid=1000 home /mnt/
```

其中 *uid* 和 *gid* 的值要与接受授权的用户的值相对应。用 `id` 命令可以为该用户找到这两个值。

自动挂载

注意：自动挂载需要启用 `vboxservice` 服务才能生效

为使用自动挂载功能，首先需要在 GUI 的管理界面里勾选自动挂载，或者为命令行工具 `VBoxManage sharedfolder` 加上参数 `--automount`。

此时，共享目录应该已经在 `/media/sf_*<共享目录的名字>*` 挂载上了。如果用户没有读写权限，确认一下该目录的权限是否是 755。如果是 750 的话，确保该目录属于 `vboxsf` 组。

可以用软链接把共享目录链接到方便的位置，这样就不必进入那么远的目录了：

```
$ ln -s /media/sf_共享目录的名字 ~/my_documents
```

按配置于启动时挂载

fstab 可以用来挂载目录。然而，为了避免 Systemd 启动可能带来的问题，要在 `/etc/fstab` 配置里为共享目录加上挂载选项 `noauto,x-systemd.automount`。这样，共享目录会在开机后初次访问时挂载，而不是在系统启动过程里挂载。这可以避免 Systemd 在客体机插件加载之前就按 fstab 挂载目录而导致的出错。

```
sharedFolderName  /path/to/mntPtOnGuestMachine  vboxsf
uid=user,gid=group,rw,dmode=700,fmode=600,noauto,x-systemd.automount
```

- *sharedFolderName*: 在虚拟机设置界面 *Settings* > *SharedFolders* > *Edit* > *FolderName* 里所设置的值。这个值和宿主机里实际的目录名可以不相同。要查看虚拟机的设置，在宿主机的 VirtualBox GUI 管理界面选中虚拟机，然后点击工具栏的 *Settings* 按钮，再从弹出的对话框里选择 *Shared Folders*。
- */path/to/mntPtOnGuestMachine*: 如果这个路径在虚拟机里还不存在，那么需要在挂载之前手动创建（用 mkdir 就可以）。
- dmode/fmode 分别用来指定挂载共享目录之后，下面的子目录与文件的属性。

mount.vboxsf 尚不支持 nofail 挂载参数：

```
desktop  /media/desktop  vboxsf  uid=user,gid=group,rw,dmode=700,fmode=600,nofail  0  0
```

从宿主机 SSH 登录客体机

在虚拟机设置的 Network 标签页 -> 右侧打开 Advanced 折叠 -> 单击 Port Forwarding 按钮，可以设置端口。

假如我们设置了将宿主机的 3022 端口转发到客体机的 22 端口。然后在宿主机执行：

```
user@host$ ssh -p 3022 $USER@localhost
```

即可 SSH 登录客体机。

用 SSHFS 来实现共享目录

配置好了端口转发，再装上 SSHFS，只要在宿主机运行这个命令就可以把客体机的目录挂载到宿主机：

```
user@host$ sshfs -p 3022 $USER@localhost:$HOME ~/shared_folder
```

这样也能实现互传文件。

故障排除

访客操作系统中访问串口

请参阅使用终端仿真器程序连接串口的操作方式.

安装过程中TTY文本太小

在主机上的VirtualBox管理器中，将显示比例设置为2.00或3.00。

启动 Xorg 之后虚拟机卡死

出现这一问题的原因是驱动程序缺失或有错。例如： 或 。试试从菜单项 *Settings* > *Display* 里关闭 3D 加速，并确认要用的 Xorg 驱动都已装好。

全屏模式只能看到黑屏

在某些窗口管理器（i3, awesome）下运行 VirtualBox 时，由于顶层状态栏（overlay bar）的问题，VirtualBox 的全屏模式会出现问题。试试在菜单项 "Guest Settings > User Interface > Mini ToolBar" 里禁用 "Show in Full-screen/Seamless" 有可能绕过这一问题。详情可见上游的问题汇报。

如果屏幕是在某个大小之后开始黑屏，例如大于 2048 像素宽度时，请尝试增加 *Settings* > *Display* > *Screen* > *Video Memory*。

Linux 客体机的声音缓慢 / 扭曲

在 VirtualBox 里运行时，Linux 内核的 AC97 声卡驱动偶尔会猜错时钟频率设定。这就会造成声音播放速度太慢 / 太快。在 `/etc/modprobe.d` 目录里创建一个内容如下的文件即可解决：

```
options snd-intel8x0 ac97_clock=48000
```

Linux 宿主机存在音频卡顿/延迟

在某些情况下，音频可能会出现延迟性能问题（例如在在线视频流媒体中音频与视频不同步）。一个可能的解决方法是在VirtualBox中使用Intel HD音频控制器，并通过在客户操作系统的`/etc/modprobe.d/` 目录下的文件中添加以下行来禁用其节能功能：

```
options snd_hda_intel power_save=0 power_save_controller=N
```

Arch: pacstrap 脚本出错

如果在安装 Arch 系统时，**还没有**启动至新安装到虚拟盘的系统，就用 `pactrap` 直接安装客体机插件，需要以 root 身份运行一次 `umount -l /mnt/dev` 然后再次运行 `pactrap`。否则新装的系统就不可用。

Windows 宿主机: VERR_ACCESS_DENIED

为了能在 Windows 宿主机上读取 RAW 格式的 VMDK 镜像，VirtualBox GUI 需要以管理员权限启动。

在Arch Linux客户机中没有硬件3D加速

virtualbox-guest-utils包 包的版本5.2.16-2缺少文件`VBoxEGL.so`. 这导致Arch Linux虚拟机无法正常使用3D加速。请参阅 FS#49752.

为了解决这个问题，请按照FS#49752#comment152254中提供的补丁集进行修复。需要对补丁集进行一些修复，以使其适用于版本5.2.16-2。

Plasma将客户机的分辨率重置为800×600

See KDE#Cannot change screen resolution when running in a virtual machine.

在Plasma-X11最小安装中出现黑屏

如果您使用的是 **plasma-desktop** 最小安装，而不是 **plasma** (包含Wayland支持), 那么在启动Plasma-X11会话后，可能会出现黑屏但有光标的情况。

要解决此问题，请在VirtualBox窗口中多次调整虚拟机窗口大小，然后通过以下方式手动设置分辨率。

View -> Virtual Screen 1 -> Resize to 1024x768 (or other resolution you like)

然后安装 kscreen包.

Open in KDE launcher **System Settings -> Startup and Shutdown -> Background Services**, stop and unselect **KScreen2** and save settings. Issue should go away forever

8.4 VMware

难度 高级 | **篇幅** 25.5 KB | **所属** 虚拟机管理

本节将讲到

- 安装
- 用法
- 技巧和诀窍
- 解难答疑

翻译状态：

- 本文（或部分内容）译自 VMware，最近一次同步于 2025-07-15，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： need translation（在 Talk:VMware# 中讨论）

本文介绍的是最新的主要 VMware 版本，即 VMware Workstation Pro 和 Player 17、16、15、14 和 12.5。

你可能也会想了解下 /安装 Arch Linux 为虚拟机。

安装

注意：自版本 14 以来，VMware 已放弃对许多 CPU 的支持，包括早期的 Intel Core i7 CPU。请检查主机 CPU 要求，并确保 BIOS/UEFI 固件中启用了 CPU 的虚拟化技术（AMD-V 或 VT-x）。如果较新版本不支持您的 CPU，则可以使用 vmware-workstation12AUR。

安装 vmware-workstationAUR 和已安装内核对应的头文件包（例如 linux-headers包或 linux-lts-headers包）。

然后启动 `vmware-networks-configuration.service` 来生成 `/etc/vmware/networking` 文件。

最后，根据需要启用以下一些服务：

- `vmware-networks.service` 用于虚拟机网络访问（否则您将收到错误 `could no connect 'ethernet 0' to virtual network` 并且您将无法使用 *vmware-netcfg*）
- `vmware-usbarbitrator.service` 用于将 USB 设备连接到虚拟机
- `vmware-hostd.service` 用于共享虚拟机（从版本 16 起不可用）

最后，加载 VMware 模块：

```
# modprobe -a vmw_vmci vmmon
```

如果加载时长时间无响应，请参考 #无法加载 vmmon 模块。

用法

打开 VMware Workstation Pro：

```
$ vmware
```

打开 VMware Player：

```
$ vmplayer
```

技巧和诀窍

输入 Workstation Pro 许可证密钥

从终端输入

```
# /usr/lib/vmware/bin/vmware-vmx-debug --new-sn XXXXX-XXXXX-XXXXX-XXXXX-XXXXX
```

其中 `XXXXX-XXXXX-XXXXX-XXXXX-XXXXX` 是许可证密钥

注意：`-debug` 二进制文件会遇到无效许可证时提示用户。

从 GUI 输入

如果上述方法不起效，您可以尝试：

```
# /usr/lib/vmware/bin/vmware-enter-serial
```

提取 VMware BIOS

```
$ objcopy /usr/lib/vmware/bin/vmware-vmx -O binary -j bios440 --set-section-flags
bios440=a bios440.rom.Z
$ perl -e 'use Compress::Zlib; my $v; read STDIN, $v, '$(stat -c%s "./bios440.rom.Z")';
$v = uncompress($v); print $v;' < bios440.rom.Z > bios440.rom
```

提取安装程序

要查看安装程序 `.bundle` 中的内容：

```
$ sh VMware-edition-version.release.architecture.bundle --extract /tmp/vmware-bundle/
```

使用修改后的 BIOS

如果您决定修改提取的 BIOS，您可以通过将其移动到以下位置让虚拟机使用它：`~/vmware/*`虚拟机名称`*`：

```
$ mv bios440.rom ~/vmware/虚拟机名称/
```

然后将名称添加到 `*`虚拟机名称`*.vmx` 文件:

```
~/vmware/虚拟机名称/虚拟机名称.vmx
bios440.filename = "bios440.rom"
```

在 Intel、Optimus 和 AMD 上启用 3D 图形加速

由于 3D 图形加速的性能或稳定性原因，有些图形驱动默认被禁用。在启用 *Accelerate 3D graphics* 后，可能会出现如下日志：

```
Disabling 3D on this host due to presence of Mesa DRI driver.  Set
mks.gl.allowBlacklistedDrivers = TRUE to override.
```

日志提到的设置对应的配置文件为 `~/.vmware/preferences`。

VMware Workstation 16.2 从 OpenGL 改用到了 Vulkan，因此设置会有些不同。如果你的驱动不受支持，日志会显示：

```
mks Vulkan Renderer: Only the AMDVLK driver is supported at this time.
mks Vulkan Renderer: No supported Vulkan device/driver found (See
mks.vk.allowUnsupportedDevices or mks.vk.forceDevice configuration options).
```

如果你的驱动被禁用，你得把 `mks.vk.allowUnsupportedDevices = "TRUE"` 添加到 `~/.vmware/preferences` 或者换一个支持的驱动 - 如果不确定要进行哪种操作，就看看虚拟机目录里的 `vmware.log`。

注意：You might need to add the `mks.gl.allowBlacklistedDrivers = "TRUE"` inside the `.vmx` file for the specific virtual machine as well, for 3D acceleration with `intel` drivers to be enabled.

在主机挂起/休眠之前挂起虚拟机

创建一个可执行文件：

```
/usr/lib/systemd/system-sleep/vmware_suspend_all.sh
#!/bin/bash

set -eu

if [[ $# -ne 2 ]]; then
    echo "Usage: $0 <period> <action>"
```

```
    exit 1
fi

period=$1
action=$2

echo "vmware system-sleep hook argv: ${period} ${action}"

if ! command -v vmrun &>/dev/null; then
    echo "command not found: vmrun"
fi

if [[ "${period}" = "pre" ]]; then
    readarray -t vms < <(vmrun list | tail -n +2)

    echo "Number of running VMs: ${#vms[@]}"

    if [[ ${#vms[@]} -eq 0 ]]; then
        exit
    fi

    for vm in "${vms[@]}"; do
        echo -n "Suspending ${vm}... "
        vmrun suspend "${vm}"
        echo "done"
    done

    sleep 1
else
    echo "Nothing to do"
fi
```

See also Power management/Suspend and hibernate#Hooks in /usr/lib/systemd/system-sleep, suspend all virtual machines with vmrun and Support for hibernation.

系统提速技巧

参考性能优化。

禁用透明大页

**本文或本章节可能需要合并到Improving performance。**

附注： Not specific to VMware.（在 Talk:VMware 中讨论）

If you notice the guest and/or the host frequently freezing when running a VM, you may want to disable transparent hugepages. To disable them for the current session, run (on the host):

```
# echo never > /sys/kernel/mm/transparent_hugepage/enabled
```

To make the change persistent across boots, add the kernel parameter `transparent_hugepage=never`.

You can also use `madvise` instead of `never` to still allow applications that are optimized for transparent hugepages to obtain the performance benefits. This does the same for vmware as above.

确保直接访问内存

默认情况下，VMware 会将虚拟机系统的内存写入到硬盘上的文件内。如果你确定内存空间足够，可以添加以下设置到虚拟机的 `.vmx` 文件内以确保虚机系统直接写入到内存中：

```
Virtual_machine_name.vmx
MemTrimRate = "0"
sched.mem.pshare.enable = "FALSE"
prefvmx.useRecommendedLockedMemSize = "TRUE"
mainmem.backing = "swap"
```

性能提示

要提升虚拟机的性能，可以参考下列技巧：

准虚拟化 SCSI 适配器

VMware 准虚拟化 SCSI（PVSCSI）适配器是用于 VMware ESXi 的高性能存储适配器，可以达到更高的吞吐量和更低 CPU 占用。PVSCSI 适配器适用于需要高 I/O 吞吐量的软硬件环境。

SCSI 适配器类型 `VMware Paravirtual` 可在虚拟机设置中应用。

如果在虚拟机设置中没有这些选项，也还是可以启用准虚拟化 SCSI 适配器。首先，通过修改 `mkinitcpio.conf` 确保准虚拟化 SCSI 适配器包含在内核镜像中：

```
/etc/mkinitcpio.conf
...
MODULES=(... vmw_pvscsi)
...
```

重新生成 initramfs。

关闭虚拟机，并修改 SCSI 适配器：修改 `.vmx` 中的对应项为如下：

```
scsi0.virtualDev = "pvscsi"
```

准虚拟化网络适配器

VMware 为客户机系统提供了多种网络适配器。默认适配器通常为 `e1000`，它会模拟一张 Intel 82545EM 千兆以太网卡。该 Intel 网卡通常兼容包括 Arch 在内的多数操作系统的内置驱动。

为了获得更高性能和更多特性（例如多队列支持），可以使用 VMware 原生的 `vmxnet3` 网络适配器。

Arch 的默认安装方式已包含了 `vmxnet3` 内核模块。通过 mkinitcpio 启用（或者自动检测启用；可通过执行 `lsmod | grep vmxnet3` 检查是否已加载模块）后，关机并按如下步骤在 *.vmx* 中修改网络适配器类型：

```
ethernet0.virtualDev = "vmxnet3"
```

在修改网络适配器后，需要修改网络和 dhcpcd 设置以使用新网卡名称和 MAC 地址。

```
# dhcpcd new_interface_name
# systemctl enable dhcpcd@new_interface_name.service
```

可通过执行 `ip link` 来获取新的网卡名称。

虚拟机设置

下列设置以主机内存占用为代价降低硬盘 I/O,可以提升虚拟机响应度。Vmware 的 KB1008885 提供了下列优化设置：

```
mainMem.useNamedFile = "FALSE"
MemTrimRate = "0"
prefvmx.useRecommendedLockedMemSize = "TRUE"
MemAllowAutoScaleDown = "FALSE"
sched.mem.pshare.enable = "FALSE"
```

- **mainMem.useNamedFile**：仅适用于 Windows 主机，适用于关闭虚拟机时出现高硬盘占用率的情况，该选项会阻止 VMware 创建 *.vmem* 文件。在 Linux 主机上请使用 *mainmem.backing = "swap"*。
- **MemTrimRate**：该选项阻止虚拟机释放的内存同时被主机释放掉。
- **prefvmx.useRecommendedLockedMemSize**：该选项似乎没有具体功能解释；其作用似乎为防止主机交换虚拟机的部分内存。
- **MemAllowAutoScaleDown**：防止 VMware 在无法分配足够内存的情况下调整虚拟机的内存大小。
- **sched.mem.pshare.enable**：如果同时运行了多个虚拟机，VMware 会试图在多个虚拟机间寻找并共享相同的内存页。可能产生高 I/O 负载。

也可以在 VMware workstation 的配置窗口内添加如下设置（*Edit -> Preferences... -> Memory/Priority*）：

```
prefvmx.minVmMemPct = "100"
mainMem.partialLazySave = "FALSE"
mainMem.partialLazyRestore = "FALSE"
```

- **prefvmx.minVmMemPct**：在主机上为虚拟机预留的内存大小百分比。更低的值会允许为虚拟机分配比主机系统可用大小更多的内存。但请注意，该情况很可能会导致高硬盘占用率。如果主机上的内存足够，建议保留该值为 100。
- **mainMem.partialLazySave** 和 **mainMem.partialLazyRestore**：这两个选项会防止虚拟机在挂起时创建部分快照。在使用这两个选项时，虚拟机挂起耗时会稍微增加，但会降低 VMware 存储这些数据产生的磁盘占用率。

解难答疑

Kernel headers for version x.y-zzzz were not found. If you installed them\[...\]

安装你使用内核所对应的 headers，如 linux包 对应linux-headers包

注意：较常见的错误有：更新内核及头文件后，重启一次才能启动到与头文件版本一致的新内核。

无法加载 vmmon 模块

根据 VMware 社区中的描述，需要禁用安全启动才能加载 `vmmon` 模块。

远程连接 VMware 时提示登录凭据错误

可以通过 `vmware-workstation-server` 服务远程管理 VMware Workstation 已共享的虚拟机，但是，`vmware-authd` 服务中的 PAM 配置问题会导致 `"incorrect username/password"` 报错出现。要修复该问题，需参考如下编辑 `/etc/pam.d/vmware-authd` 文件：

```
/etc/pam.d/vmware-authd
#%PAM-1.0
auth     required       pam_unix.so
account  required       pam_unix.so
password required       pam_permit.so
session  required       pam_unix.so
```

然后重启 `vmware` systemd 服务。

你现在可以使用安装过程提供的凭据连接到服务器了。

注意：启动虚拟机可能需要安装 libxslt包。

ALSA 输出问题

要修复音频质量问题或启用正确的 HD 音频输出，首先执行：

```
$ aplay -L
```

如果要在虚拟机中使用 5.1 声道，请找到 `surround51:CARD=*vendor_name*,DEV=*num*`；如果遇到音频质量问题，请找到 `front:CARD=*vendor_name*,DEV=*num*`。最后，将名称填入到 `.vmx` 内：

```
~/vmware/Virtual_machine_name/Virtual_machine_name.vmx
sound.fileName="surround51:CARD=Live,DEV=0"
sound.autodetect="FALSE"
```

另外，需要禁用 OSS 模拟。

基于内核的虚拟机（KVM）尚在运行

可以通过如下方法在启动时禁用 `KVM`：

```
/etc/modprobe.d/vmware.conf
blacklist kvm
blacklist kvm-amd    # For AMD CPUs
blacklist kvm-intel # For Intel CPUs
```

提示：如果加入黑名单后 `kvm` 模块还会加载，且你使用的设备带有 Intel 集成显卡（i915，i965），那有可能需要同时在内核命令行选项中将 `kvmgt` 模块添加到黑名单。详细信息请查阅内核模块#使用内核命令行_2。

在 HiDPI 显示器上，VMware 应用 UI 上的图标及间距没有正确缩放

参阅 HiDPI#VMware。

Wayland 问题

GNOME 下键盘捕获无效

VMWare Player/Workstation 17.5（及更旧版本）中存在问题：具体信息

要允许用户界面捕获键盘：

```
$ gsettings set org.gnome.mutter.wayland xwayland-allow-grabs "true"
$ gsettings set org.gnome.mutter.wayland xwayland-grab-access-rules
"['vmplayer','vmware-vmx','mksSandbox']"
```

另外，也可以在 Windows 下使用以下快捷键：

- 使用 `Super+Alt+Tab` 取代 `Alt+Tab` 以切换任务
- 使用 `Ctrl+Esc` 显示开始菜单

模块问题

找不到 /dev/vmmon

完整报错如下：

```
Could not open /dev/vmmon: No such file or directory.
Please make sure that the kernel module 'vmmon' is loaded.
```

这表示至少 `vmmon` 模块没有被加载。可参考#安装一节启用自动加载相关的服务。

另一种可能是由 5.18 版本内核以及 11 代和更新 Intel 处理器的间接分支跟踪功能导致。

在这种情况下，将 `ibt=off` 添加到你的内核参数中。具体信息请参考 。

找不到 /dev/vmci

完整报错如下：

```
Failed to open device "/dev/vmci": No such file or directory
Please make sure that the kernel module 'vmci' is loaded.
```

首先尝试手动加载内核：

```
# modprobe -a vmw_vmci
```

也可以试试重新编译 VMware 内核模块：

```
# vmware-modconfig --console --install-all
```

无法启动 VMware

CPUIDEarly 模块上电失败

版本 14 比 12 有更严格的 CPU 需求。如果你尝试在不兼容的 CPU 上启动虚拟机，会出现如下报错：

```
 This host does not support virtualizing real mode.
 The Intel "VMX Unrestricted Guest" feature is necessary to run this virtual machine on
an Intel processor.
```

解决方法为卸载版本 14，并重新安装版本 12（vmware-workstation12AUR）。

如果 VMware 平时可用，但忽然出现该错误，那有可能是在暖/软启动或系统挂起后导致的。请尝试进行一次冷启动（关机并重新开机）。

旧 Intel 微码导致的分段错误

旧 Intel 微码可能会在启动时导致如下分段报错：

```
/usr/bin/vmware: line 31: 4941 Segmentation fault "$BINDIR"/vmware-modconfig --
appname="VMware Workstation" --icon="vmware-workstation"
```

微码更新方法请参考微码一文。

vmplayer/vmware 版本 14 启动失败

在 `librsvg` 版本 2:2.44.0 及以上的系统中，`/tmp/vmware-<id>` 位置下的日志文件会显示数条如下报错：

```
appLoader| I125+ undefined symbol
```

一种临时处理方法为将 `librsvg` 降级到更早版本；更好的方法为强制 VMware 使用其自带的 `librsvg`：

```
# export LD_LIBRARY_PATH=/lib/vmware/lib/librsvg-2.so.2:$LD_LIBRARY_PATH
```

VMware 同时提供了一个 `VMWARE_USE_SHIPPED_LIBS` 变量：

```
$ env VMWARE_USE_SHIPPED_LIBS=1 vmware
```

从 12.5.4 版本开始无法启动 vmplayer/vmware

参考 ，临时解决方法为将 `libpng` 降级到 1.6.28-1 版本，并将其写入到 /etc/pacman.conf 中的 `IgnorePkg` 参数：

更简单的方法为让 VMware 使用系统提供的 zlib，而不是其自带的版本：

```
# cd /usr/lib/vmware/lib/libz.so.1
# mv libz.so.1 libz.so.1.old
# ln -s /usr/lib/libz.so.1 .
```

无法启动 12.5.3 至 12.5.5 版本的 vmplayer/vmware

似乎是 `/usr/lib/vmware/lib/libstdc++.so.6/libstdc++.so.6` 文件缺少 `CXXABI_1.3.8` 导致的问题。

如果系统安装了 gcc-libs包，那就已有了该库文件。因此，可以尝试将其移除，然后 vmplayer 会转而使用 gcc-libs 提供的文件。以根用户权限执行：

```
# mv /usr/lib/vmware/lib/libstdc++.so.6/libstdc++.so.6 /usr/lib/vmware/lib/libstdc+
+.so.6/libstdc++.so.6.bak
```

另一种绕过方法为：

```
# export VMWARE_USE_SHIPPED_LIBS='yes'
```

启动 vmware 12 后进程闪退，未能加载图形界面

Registered bug at Mageia, but it seems that there are no error messages shown in terminal with arch. When inspecting the logs, which are in `/tmp/vmware-<id>`,

there are `VMWARE_SHIPPED_LIBS_LIST is not set`, `VMWARE_SYSTEM_LIBS_LIST is not set`, `VMWARE_USE_SHIPPED_LIBS is not set`, `VMWARE_USE_SYSTEM_LIBS is not set` issues. Process simply terminates with `Unable to execute /usr/lib/vmware/bin/vmware-modconfig`. after vmware or vmplayer is executed. Solution is the same, as root do:

```
# mv /etc/vmware/icu/icudt44l.dat /etc/vmware/icu/icudt44l.dat.bak
```

Also there is a workaround:

```
# export VMWARE_USE_SHIPPED_LIBS='yes'
```

Despite setting the VMWARE_USE_SHIPPED_LIBS variable, VMWare may still fail to find certain libraries. An example is the libfontconfig.so.1 library. Check vmware logs in the tmp directory to see which libraries are still not found. Copy them to the appropriate path with libraries existing on the system:

```
# cp /usr/lib/libfontconfig.so.1 /usr/lib/vmware/lib/libfontconfig.so.1/
```

Instead of copying all these files manually, you may want to try exporting an additional setting:

```
# export VMWARE_USE_SYSTEM_LIBS='yes'
```

On systems with fontconfig version **2.13.0** and above, it may be needed to force VMware to use the shipped libfontconfig file instead of the newer system file. In such case, it is also necessary to provide a shared object library file `libexpat.so.0` for the shipped fontconfig. This applies for at least VMware version **12.5.9**. As root do:

```
# ln -s /usr/lib/libexpat.so /usr/lib/vmware/lib/libfontconfig.so.1/libexpat.so.0
# export LD_LIBRARY_PATH=/usr/lib/vmware/lib/libfontconfig.so.1:$LD_LIBRARY_PATH
```

虚拟机问题

无法为客户机下载 VMware Tools

可以通过访问 VMware 存储库\[失效链接 2025-04-06 ⓘ\]手动下载工具：

访问 "*application name* / *version* / *build ID* / linux / packages/" 路径并下载对应的工具。

使用如下命令解压：

```
$ tar -xvf vmware-tools-name-version-buildID.x86_64.component.tar
```

然后使用 VMware 安装器进行安装：

```
# vmware-installer --install-component=/path/vmware-tools-name-version-buildID.x86_64.component
```

如果以上方法无效，可以尝试安装 ncurses5-compat-libsAUR。

虚拟机的系统时间错误或无法启动："\[...\]timeTracker_user.c:234 bugNr=148722"

**这一章节正在考虑移除。**

原因: `ondemand` is no longer the default governor, does this still apply to the `schedutil` governor? (在 Talk:VMware 讨论)

This is due to incomplete\[失效链接 2024-10-12 ⓘ\] support of power management features (Intel SpeedStep and AMD PowerNow!/Cool'n'Quiet) in VMware Linux that vary the CPU frequency. In March 2012, with the release of linux 3.3-1 the maximum frequency Performance governor was replaced with the dynamic *Ondemand*. When the host CPU frequency changes, the Guest system clock runs too quickly or too slowly, but may also render the whole Guest unbootable.

To prevent this, the maximum host CPU frequency can be specified, and Time Stamp Counter (TSC) disabled, in the global configuration:

```
/etc/vmware/config
host.cpukHz = "X"  # The maximum speed in KHz, e.g. 3GHz is "3000000".
host.noTSC = "TRUE" # Keep the Guest system clock accurate even when
ptsc.noTSC = "TRUE" # the time stamp counter (TSC) is slow.
```

提示：To periodically correct the time (once per minute), in the *Options* tab of VMware Tools, enable: *"Time synchronization between the virtual machine and the host operating system"*.

系统重启后客户机丢失网络连接

该问题很可能由 `vmnet` 未被加载导致 。可参考#安装一节启用自动加载相关的服务。

鼠标问题

超出 5 个后的鼠标按键无法使用

如果你鼠标的拇指或其它额外按钮无法正常工作，可以设置客户机使用高级鼠标。

客户机上的鼠标滚轮表现怪异

This is related to the current Xorg keyboard layout on Host system. Keep primary layout (e.g., English) selected on Host while working on Guest.

嵌套虚拟机没有 IP 地址及网络连接

该问题与混杂模式有关。根据标准 Linux 实践，该模式仅能由根用户启用。为了绕过该限制，需要修改对应网络设备的权限。

对单个用户组给予权限：

```
# chgrp group /dev/vmnetX

# chmod g+rw /dev/vmnetX
```

对所有用户给予权限：

```
# chmod a+rw /dev/vmnetX
```

8.5 Xen

难度 高级 | **篇幅** 28.2 KB | **所属** 虚拟机管理

本节将讲到

- 安装
- Using Xen
- Configuring a hardware virtualized (HVM) Arch domU

- Configuring a paravirtualized (PV) Arch domU
- 常见错误

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

根据Xen 概览:

: Xen 是一套开源的类型一 hypervisor（又称为裸机 hypervisor），可以在单个设备/宿主机上同时运行多个相同或不同的操作系统实例。Xen 是目前唯一的开源类型一 hypervisor。

Xen hypervisor 是一层轻量的软件，它模拟了计算机的架构，使得可以同时运行多个操作系统。Hypervisor 由计算机上的引导加载器启动；当 hypervisor 加载完成后，它会启动 dom0（即“domain 0”的简称，有时被称为宿主或私有域），在当前案例中使用 Arch Linux。在 *dom0* 启动完成后，可以从 *dom0* 启动并控制一个或多个 domU（用户域的简称，有时被称为 VMs 或是客户域）。对于 *domU* 来说，Xen 支持半虚拟化（PV）域，硬件虚拟化域（HVM）及硬件虚拟化包装器中的半虚拟化域（PVH）。更多详细信息可以参考 Xen.org。

Xen hypervisor 需部署在一套安装完整的底层操作系统上。在安装 Xen hypervisor 前，宿主机上需有一套完全可用并更新到最新的 Arch Linux 系统。系统可以是只包含基础软件包的最小化安装，且无需桌面环境或是 Xorg。

如果你在从头开始搭建宿主机，可以参考安装指南来安装 Arch Linux。

警告：不要在使用 Xen hypervisor 时运行如 VirtualBox 等其它虚拟化软件，这可能会导致系统挂起。详细信息可参考 bug 报告（不予修复）。

安装

系统要求

Xen hypervisor 需要包含于较新 Linux 内核中的内核级支持，相关功能已包含在 linux 包 及 linux-lts包 Arch 内核软件包中。如需运行 HVM *domU*，物理硬件也需要包含 VT-x 或 AMD-V（SVM）虚拟化支持功能。可以在 Xen hypervisor 未运行时执行如下命令来进行验证：

```
$ grep -E "(vmx|svm)" --color=always /proc/cpuinfo
```

如果上述命令无结果输出，则说明硬件虚拟化不受支持，即你的硬件无法运行 HVM 类型的 *domU*（或是 Xen hypervisor 正在运行中）。如果你认为你的 CPU 支持上述任一种功能，则可以在启动时查看宿主机的 BIOS 设置菜单，检查虚拟化相关的选项是否被禁用。如果相关的选项存在并被禁用，则将其启用，接着启动到系统中并再次

使用上述命令进行检查。当 PCI 设备支持时，Xen hypervisor 也可以使用 PCI 直通，不需要 *dom0* 的硬件支持也可以将 PCI 设备直接连接到 *domU*。如需使用 PCI 直通，需要 CPU 支持 IOMMU/VT-d 功能。

Installation of the Xen hypervisor

To install the Xen hypervisor, install the xenAUR package. It provides the Xen hypervisor, current xl interface and all configuration and support files, including systemd services. To run most VMs, you will also need to install xen-qemuAUR.

For BIOS support in VMs, install seabios包. For UEFI support, install edk2-ovmf包. To boot VM-local kernels inside of a PVH VM, install xen-pvhgrubAUR.

Building xen

It is recommended that xen and its components are built in a clean environment, either in a VM or a chroot. When building Xen, there are environmental variables that can be passed to *makepkg*.

1. *build_stubdom* -- Build the components to run Xen stubdoms, mainly for dom0 disaggregation. Components for stubdom are broken off into *xen-stubdom* if built. Defaults to false.
2. *boot_dir*\-- Your boot directory. Defaults to */boot*.
3. *efi_dir*, *efi_mountpoint* -- Your EFI directory and mountpoint. Defaults to */boot*.

Pass these arguments to makepkg as variables:

```
$ build_stubdom=true efi_dir="/boot/EFI" makepkg
```

xen-docsAUR will be also built for the man pages and documentation. If you choose to build stubdom support, a *xen-stubdom* package will be built.

注意：The plan for Xen on Arch Linux is to eventually phase stubdom out in favor of PVH domains.

Modification of the bootloader

警告：Never assume your system will boot after changes to the boot system. This might be the most common error new as well as old users do. Make sure you have a alternative way to boot your system like a USB stick or other livemedia **BEFORE** you make changes to your boot system.

The boot loader must be modified to load a special Xen kernel (`xen.gz` or in the case of UEFI `xen.efi`) which is then used to boot the normal kernel. To do this a new bootloader entry is needed.

UEFI

Xen supports booting from UEFI as specified in Xen EFI systems. It also might be necessary to use efibootmgr to set boot order and other parameters.

First, ensure the `xen.efi` file is in the EFI system partition along with your kernel and ramdisk files.

Second, Xen requires an ASCII (no UTF-8, UTC-16, etc) configuration file that specifies what kernel should be booted as dom0. This file must be placed in the same EFI system partition as the binary. Xen looks for several configuration files and uses the first one it finds. The order of search starts with the `.efi` extension of the binary's name replaced by `.cfg`, then drops trailing name components at `.`, `-` and `_` until a match is found. Typically, a single file named `xen.cfg` is used with the system requirements, such as:

```
xen.cfg
[global]
default=xen

[xen]
options=console=vga iommu=force:true,qinval:true,debug:true loglvl=all noreboot=true
reboot=no vga=ask ucode=scan
kernel=vmlinuz-linux root=/dev/sdaX rw add_efi_memmap #earlyprintk=xen
ramdisk=initramfs-linux.img
```

提示：See Xen efi.cfg for additional parameters in this file. For the options line, see Xen Command Line options for a full list of options available such as serial console, limiting dom0 vCPU and memory, scheduling, Intel and AMD microcode and more. For example, Xen Project Best Practices dictates to disabling memory ballooning for dom0. To do that, edit the `xen.cfg` line for `options` to specify the additional parameters.

Systemd-boot

提示：You can continue to boot the dom0 kernel directly even after Xen is installed and configured. This can be useful in the event that an Xen installation becomes unbootable or misconfigured. Therefore, it is recommended to keep the original systemd-boot loader entries configured on the system as rescue boot options and just add additional entries for Xen.

注意：At the time of the system's Systemd-boot installation, the ESP partition should have been mounted to `/boot` as this is where the XenAUR package and EFI binaries were configured and built for, not `/boot/efi`.

Add a new EFI-type loader entry. See Systemd-boot#EFI Shells or other EFI applications for more details. For example:

```
/boot/loader/entries/10-xen.conf
title   Xen Hypervisor
efi     /xen.efi
```

注意：The current systemd-boot and Xen efi binary combination does not allow parameters passed on the `efi` line of the loader's entry. However, the Xen documentation states that `-cfg=file.cfg` can be used as an UEFI Shell parameter which is not true for the efi line option. For now, you can only have one Xen EFI entry which limits you to only one configuration file.

EFISTUB

It is possible to boot an EFI kernel directly from UEFI by using EFISTUB.

Drop to the build-in UEFI shell and call the EFI file directly. For example:

```
Shell> fs0:
FS0:\> xen.efi
```

Note that a `xen.cfg` configuration file in the EFI system partition is still required as outlined above. In addition, a different configuration file may be specified with the `-cfg=file.cfg` parameter. For example:

```
Shell> fs0:
FS0:\> xen.efi -cfg=xen-rescue.cfg
```
These additional configuration files must reside in the same directory as the Xen EFI binary and linux stub files.

BIOS

Xen supports booting from system firmware configured as BIOS.

GRUB

For GRUB users, install the grub-xen-gitAUR package for booting *dom0* as well as building PvGrub2 images for booting user domains.

The file `/etc/default/grub` can be edited to customize the Xen boot commands. For example, to allocate 512 MiB of RAM to *dom0* at boot,
modify `/etc/default/grub` by replacing the line:

```
#GRUB_CMDLINE_XEN_DEFAULT=""
```
with

```
GRUB_CMDLINE_XEN_DEFAULT="dom0_mem=512M"
```
More information on GRUB configuration keys for Xen can be found in the GRUB documentation.

After customizing the options, update the bootloader configuration with the following command:

```
# grub-mkconfig -o /boot/grub/grub.cfg
```
More information on using the GRUB bootloader is available at GRUB.

Building GRUB images for booting guests

Besides the usual platform targets, the grub-xen-gitAUR package builds GRUB for three additional targets that can be used to boot Xen guests: *i386-xen*, *i386-xen_pvh*, and *x86_64-xen*. To create a boot image from one of these targets, first create a GRUB configuration file. Depending on your preference, this file can either locate and load a GRUB configuration file in the guest or it could manage more of the boot process from *dom0*. Assuming all that is needed is to locate and load a configuration file in the guest, add the following to a file,

```
grub.cfg
search -s root -f /boot/grub/grub.cfg
configfile /boot/grub/grub.cfg
```
and then create a GRUB/Tips and tricks#GRUB standalone image that will incorporate that file:

```
 # grub-mkstandalone -O x86_64-xen -o /usr/lib/xen/boot/pv-grub2-x86_64-xen
"/boot/grub/grub.cfg=./grub.cfg"
```
Lastly, add that image as value of the kernel in the domU configuration file (for a 64-bit guest in this example):

```
 kernel = "/usr/lib/xen/boot/pv-grub2-x86_64-xen"
```
More examples of configuring GRUB images for GRUB guests can be found in the Xen Project's PvGrub2 documentation.

Syslinux

For Syslinux users, add a stanza like this to your `/boot/syslinux/syslinux.cfg`:

```
LABEL xen
    MENU LABEL Xen
    KERNEL mboot.c32
    APPEND ../xen-X.Y.Z.gz --- ../vmlinuz-linux console=tty0 root=/dev/sdaX ro
--- ../initramfs-linux.img
```

where `X.Y.Z` is your xen version and `/dev/sdaX` is your root partition.

This also requires `mboot.c32` (and `libcom32.c32`) to be in the same directory as `syslinux.cfg`. If you do not have `mboot.c32` in `/boot/syslinux`, copy it from:

```
# cp /usr/lib/syslinux/bios/mboot.c32 /boot/syslinux
```

Creation of a network bridge

Xen requires that network communications between *domU* and the *dom0* (and beyond) be set up manually. The use of both DHCP and static addressing is possible, and the choice should be determined by the network topology. Complex setups are possible, see the Networking article on the Xen wiki for details and `/etc/xen/scripts` for scripts for various networking configurations. A basic bridged network, in which a virtual switch is created in *dom0* that every *domU* is attached to, can be set up by creating a network bridge with the expected name `xenbr0`.

See Network bridge#Creating a bridge for details.

Systemd-networkd

See Systemd-networkd#Bridge interface for details.

Network Manager

**本文或本章节可能需要合并到 Network_bridge#With_NetworkManager。**

附注： Duplicates the main page.（在 Talk:Xen 中讨论）

Gnome's Network Manager can sometime be troublesome. If following the bridge creation section outlined in the bridges section of the wiki are unclear or do not work, then the following steps may work.

Open the Network Settings and disable the interface you wish to use in your bridge (ex enp5s0). Edit the setting to off and uncheck "connect automatically."

Create a new bridge connection profile by clicking on the "+" symbol in the bottom left of the network settings. Optionally, run:

```
# nm-connection-editor
```

to bring up the window immediately. Once the window opens, select Bridge.

Click "Add" next to the "Bridged Connections" and select the interface you wished to use in your bridge (ex. Ethernet). Select the device mac address that corresponds to the interface you intend to use and save the settings

If your bridge is going to receive an IP address via DHCP, leave the IPv4/IPv6 sections as they are. If DHCP is not running for this particular connection, make sure to give your bridge an IP address. Needless to say, all connections will fail if an IP address is not assigned to the bridge. If you forget to add the IP address when you first create the bridge, it can always be edited later.

Now, as root, run:

```
# nmcli con show
```

You should see a connection that matches the name of the bridge you just created. Highlight and copy the UUID on that connection, and then run (again as root):

```
# nmcli con up <UUID OF CONNECTION>
```

A new connection should appear under the network settings. It may take 30 seconds to a minute. To confirm that it is up and running, run:

```
# brctl show
```

to show a list of active bridges.

Reboot. If everything works properly after a reboot (ie. bridge starts automatically), then you are all set.

<optional> In your network settings, remove the connection profile on your bridge interface that does NOT connect to the bridge. This just keeps things from being confusing later on.

Installation of Xen systemd services

The Xen *dom0* requires the `xenstored.service`, `xenconsoled.service`, `xendomains.service` and `xen-init-dom0.service` to be started and possibly enabled.

Confirming successful installation

Reboot your *dom0* host and ensure that the Xen kernel boots correctly and that all settings survive a reboot. A properly set up *dom0* should report the following when you run `xl list` as root:

```
# xl list
Name                                        ID   Mem VCPUs      State   Time(s)
Domain-0                                     0   511     2      r-----   41652.9
```

Of course, the Mem, VCPUs and Time columns will be different depending on machine configuration and uptime. The important thing is that *dom0* is listed.

In addition to the required steps above, see best practices for running Xen which includes information on allocating a fixed amount of memory and how to dedicate (pin) a CPU core for *dom0* use. It also may be beneficial to create a xenfs filesystem mount point by including in `/etc/fstab`

```
none /proc/xen xenfs defaults 0 0
```

Configure Best Practices

Review Xen Project Best Practices before using Xen.

Using Xen

Xen supports both paravirtualized (PV) and hardware virtualized (HVM) *domU*. In the following sections the steps for creating HVM and PV *domU* running Arch Linux are described. In general, the steps for creating an HVM *domU* are independent of the *domU* OS and HVM *domU* support a wide range of operating systems including Microsoft Windows. To use HVM *domU* the *dom0* hardware must have virtualization support. Paravirtualized *domU* do not require virtualization support, but instead require modifications to the guest operating system making the installation procedure different for each operating system (see the Guest Install page of the Xen wiki for links to instructions). Some operating systems (e.g., Microsoft Windows) cannot be installed as a PV *domU*. In general, HVM *domU* often run slower than PV *domU* since HVMs run on emulated hardware. While there are some common steps involved in setting up PV and HVM *domU*, the processes are substantially different. In both cases, for each *domU*, a "hard disk" will need to be created and a configuration file needs to be written. Additionally, for installation each *domU* will need access to a copy of the installation ISO stored on the *dom0* (see the Download Page to obtain the Arch Linux ISO).

Create a domU "hard disk"

Xen supports a number of different types of "hard disks" including Logical Volumes, raw partitions, and image files. To create a sparse file, that will grow to a maximum of 10GiB, called `domU.img`, use:

```
$ truncate -s 10G domU.img
```

If file IO speed is of greater importance than domain portability, using Logical Volumes or raw partitions may be a better choice.

Xen may present any partition / disk available to the host machine to a domain as either a partition or disk. This means that, for example, an LVM partition on the host can appear as a hard drive (and hold multiple partitions) to a domain. Note that making sub-partitons on a partition will make accessing those partitions on the host machine more difficult. See kpartx(8) for information on how to map out partitions within a partition.

Create a domU configuration

Each *domU* requires a separate configuration file that is used to create the virtual machine. Full details about the configuration files can be found at the Xen Wiki or the xl.cfg(5) man page. Both HVM and PV *domU* share some components of the configuration file. These include

```
name = "domU"
memory = 512
disk = [ "file:/path/to/ISO,sdb,r", "phy:/path/to/partition,sda1,w" ]
vif = [ 'mac=00:16:3e:XX:XX:XX,bridge=xenbr0' ]
```

The `name=` is the name by which the xl tools manage the *domU* and needs to be unique across all *domU*. The `disk=` includes information about both the installation media (`file:`) and the partition created for the *domU* `phy`. If an image file is being used instead of a physical partition, the `phy:` needs to be changed to `file:`. The `vif=` defines a network controller. The `00:16:3e` MAC block is reserved for Xen domains, so the last three digits of the `mac=` must be randomly filled in (hex values 0-9 and a-f only).

Managing a domU

If a *domU* should be started on boot, create a symlink to the configuration file in `/etc/xen/auto` and ensure the `xendomains` service is set up correctly. Some useful commands for managing *domU* are:

```
# xl top
# xl list
# xl console domUname
# xl shutdown domUname
# xl destroy domUname
```

Configuring a hardware virtualized (HVM) Arch domU

In order to use HVM *domU* install the mesa包, numactl包 and bluez-libs包 packages.

A minimal configuration file for a HVM Arch *domU* is:

```
name = 'HVM_domU'
builder = 'hvm'
memory = 512
vcpus = 2
disk = [ 'phy:/dev/vg0/hvm_arch,xvda,w', 'file:/path/to/ISO,hdc:cdrom,r' ]
vif = [ 'mac=00:16:3e:00:00:00,bridge=xenbr0' ]
vnc = 1
vnclisten = '0.0.0.0'
vncdisplay = 1
```

Since HVM machines do not have a console, they can only be connected to via a vncviewer. The configuration file allows for unauthenticated remote access of the *domU* vncserver and is not suitable for unsecured networks. The vncserver will be available on port `590X`, where X is the value of `vncdisplay`, of the *dom0*. The *domU* can be created with:

```
# xl create /path/to/config/file
```

and its status can be checked with

```
# xl list
```

Once the *domU* is created, connect to it via the vncserver and install Arch Linux as described in the Installation guide.

Configuring a paravirtualized (PV) Arch domU

A minimal configuration file for a PV Arch *domU* is:

```
name = "PV_domU"
kernel = "/mnt/arch/boot/x86_64/vmlinuz-linux"
ramdisk = "/mnt/arch/boot/x86_64/initramfs-linux.img"
extra = "archisobasedir=arch archisodevice=UUID=YYYY-mm-dd-HH-MM-SS-00"
memory = 512
disk = [ "phy:/path/to/partition,sda1,w", "file:/path/to/ISO,sdb,r" ]
```

```
vif = [ 'mac=00:16:3e:XX:XX:XX,bridge=xenbr0' ]
```

This file needs to tweaked for your specific use. Most importantly, the `archisodevice=UUID=*YYYY-mm-dd-HH-MM-SS*-00` line must be edited to use the creation date and time of the ISO being used.

Before creating the *domU*, the installation ISO must be loop-mounted. To do this, ensure the directory `/mnt` exists and is empty, then run the following command (being sure to fill in the correct ISO path):

```
# mount -o loop /path/to/iso /mnt
```

Once the ISO is mounted, the *domU* can be created with:

```
# xl create -c /path/to/config/file
```

The "-c" option will enter the *domU*'s console when successfully created. Then you can install Arch Linux as described in the Installation guide, but with the following deviations. The block devices listed in the disks line of the cfg file will show up as `/dev/xvd*`. Use these devices when partitioning the *domU*. After installation and before the *domU* is rebooted, the `xen-blkfront`, `xen-fbfront`, `xen-netfront`, `xen-kbdfront` modules must be added to Mkinitcpio. Without these modules, the *domU* will not boot correctly. For booting, it is not necessary to install Grub. Xen has a Python-based grub emulator, so all that is needed to boot is a `grub.cfg` file: (It may be necessary to create the `/boot/grub` directory)

```
/boot/grub/grub.cfg
menuentry 'Arch GNU/Linux, with Linux core repo kernel' --class arch --class gnu-linux
--class gnu --class os $menuentry_id_option 'gnulinux-core repo kernel-true-__UUID__' {
        insmod gzio
        insmod part_msdos
        insmod ext2
        set root='hd0,msdos1'
        if [ x$feature_platform_search_hint = xy ]; then
          search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-
efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1  __UUID__
        else
          search --no-floppy --fs-uuid --set=root __UUID__
        fi
        echo    'Loading Linux core repo kernel ...'
        linux   /boot/vmlinuz-linux root=UUID=__UUID__ ro
        echo    'Loading initial ramdisk ...'
        initrd  /boot/initramfs-linux.img
}
```

This file must be edited to match the UUID of the root partition. From within the *domU*, run the following command:

```
# blkid
```

Replace all instances of `__UUID__` with the real UUID of the root partition (the one that mounts as `/`).:

```
# sed -i 's/__UUID__/12345678-1234-1234-1234-123456789abcd/g' /boot/grub/grub.cfg
```

Shutdown the *domU* with the `poweroff` command. The console will be returned to the hypervisor when the domain is fully shut down, and the domain will no longer appear in the xl domains list. Now the ISO file may be unmounted:

```
# umount /mnt
```

The *domU* cfg file should now be edited. Delete the `kernel =`, `ramdisk =`, and `extra =` lines and replace them with the following line:

```
bootloader = "pygrub"
```

Also remove the ISO disk from the `disk =` line.

The Arch *domU* is now set up. It may be started with the same line as before:

```
# xl create -c /etc/xen/archdomu.cfg
```

常见错误

"xl list" complains about libxl

Either you have not booted into the Xen system, or xen modules listed in `xencommons` script are not installed.

"xl create" fails

Check the guest's kernel is located correctly, check the `pv-xxx.cfg` file for spelling mistakes (like using `initrd` instead of `ramdisk`).

Creating HVM fails

If creating HVM fails with:

```
libxl: error: libxl_dm.c:3131:device_model_spawn_outcome: Domain 33:domain 33 device
model: spawn failed (rc=-3)
libxl: error: libxl_dm.c:3351:device_model_postconfig_done: Domain 33:Post DM startup
configs failed, rc=-3
libxl: error: libxl_create.c:1837:domcreate_devmodel_started: Domain 33:device model
did not start: -3
libxl: error: libxl_aoutils.c:646:libxl__kill_xs_path: Device Model already exited
```

You have missed to install numactl包.

Arch Linux guest hangs with a ctrl-d message

Press `ctrl-d` until you get back to a prompt, rebuild its initramfs described.

failed to execute '/usr/lib/udev/socket:/org/xen/xend/udev_event' 'socket:/org/xen/xend/udev_event': No such file or directory

This is caused by `/etc/udev/rules.d/xend.rules`. Xend is deprecated and not used, so it is safe to remove that file.

See also

- The homepage at xen.org
- The wiki at xen.org

8.6 QEMU

难度 高级 | **篇幅** 139.3 KB | **所属** 虚拟机管理

本节将讲到

- 安装
- QEMU 的图形前端

- 创建新虚拟系统
- 运行虚拟化的系统
- 宿主机和虚拟机数据交互

翻译状态：

- 本文（或部分内容）译自 QEMU，最近一次同步于 2024-11-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

来自 QEMU 关于页面：“QEMU”是一个广泛使用的开源计算机模拟器和虚拟机。"

当作为架构模拟器时，可以在一种架构（如x86 PC）下运行另一种架构（如ARM）下的操作系统和程序。通过使用动态转换，它可以获得非常好的性能。

作为虚拟机时，QEMU可以使用其他虚拟机管理程序（如 Xen 或 KVM）来使用 CPU 扩展（HVM）进行虚拟化，通过在主机 CPU 上直接执行客户机代码来获得接近于宿主机的性能。

安装

安装 qemu-full包，(或 qemu-base包，一个无GUI支持的基本版本；qemu-desktop包，默认仅支持 x86_64 模拟）

并根据需要安装下面的可选软件包：

- qemu-block-gluster包 - Glusterfs 设备块支持
- qemu-block-iscsi包 - iSCSI 设备块支持
- samba包 - SMB/CIFS 服务器支持

除了这些之外, qemu-user-static包 提供了 user-mode 下运行静态编译程序的支持。

QEMU 变种

QEMU提供了多个变种以供用户在不同场景中使用。

粗略地说，QEMU 有两种运行模式：

全系统模拟模式（full-system emulation）：在该模式下, QEMU 将会模拟一个完整的系统，包含一个或多个处理器以及各种外围设备。这种模式更加贴近真实的系统，且这种模式不要求被模拟的客户机系统是 Linux，但它的速度较慢。：QEMU 中启用 full-system 模式的命令依照如下规则进行命名 `qemu-system-*`目标机器架构*，例如 `qemu-system-x86_64` 用于模拟 x86_64 架构 CPU, `qemu-system-i386` 模拟 Intel 32位 x86 架构 CPU，`qemu-system-arm` 模拟 32 位 ARM 架构，`qemu-`

`system-aarch64` 模拟 64 位 ARM 架构，等等。 : 如果模拟的 CPU 架构与宿主机的 CPU 架构相同, 那么即使在此模式下，QEMU 仍有可能使用 hypervisor（例如KVM 或者 Xen）的技术对模拟机进行加速。 **用户模式（Usermode emulation）**

: 在此模式下, QEMU 能够利用宿主机的系统资源来调用为其他架构编译的 Linux 可执行文件。当然，里面依旧存在一些小问题, 比如说一些功能特性没有被实现, 采用动态链接的可执行文件无法直接在上面使用（参阅#从x86_64环境中 Chroot 至 arm/arm64 环境解决该问题）并且只支持 Linux 程序（尽管我们可以使用wine在 Linux 上运行 Windows 程序）。 : QEMU 中启用用户模式的命令依照如下规则进行命名 `qemu-*`目标机器架构`*`, 例如 `qemu-x86_64` 用于模拟 64 位的 CPU。

QEMU 拥有动态链接和静态链接两个变种：

动态链接 (默认) : `qemu-*` 命令依赖于宿主机上的库文件, 因此整个程序体积较小。 **静态链接** : `qemu-*` 命令则可以在任何架构相同的 Linux 系统上使用。

在 Arch Linux 中，全系统模拟有两个变种：

有界面（Non-Headless）（默认）：这个变种启用了一些 GUI 相关的特性，需要额外的依赖（例如 SDL 或 GTK）。 无界面（Headless）：这个变种不需要 GUI 相关的依赖（适用于服务器场景）。

需要注意的是，这两个版本安装的执行文件名称是一样的（例如 `qemu-system-x86_64`），因此系统上不能同时安装这两个版本。

Arch Linux中相关安装包的详细信息

- qemu-desktop包 包提供了 `x86_64` 架构的模拟器， 可以进行全系统模拟 (`qemu-system-x86_64`)。 qemu-emulators-full包包提供了 `x86_64` 用户模式的模拟 (`qemu-x86_64`)。 对于其他支持的架构，这个包都提供了全系统模拟和用户模拟两个变种 (比如说 `qemu-system-arm` 和 `qemu-arm`)。
- 对应于这些包的无界面版本 (仅适用于全系统模拟模式) 分别是 qemu-base包 (仅`x86_64`) 和 qemu-emulators-full包 (其余架构)。
- 可以用如下独立的安装包中的 QEMU 模块扩展全系统模拟的功能: qemu-block-gluster包, qemu-block-iscsi包, 和 qemu-guest-agent包。
- qemu-user-static包 为所有 QEMU 支持的架构提供了一个带用户模式和静态链接模式的变种。 它的QEMU命令依照 `qemu-*target_architecture*-static`的规则命名, 例如, `qemu-x86_64-static` 代表目标架构为intel 64位 CPU。

注意：目前为止，Arch Linux 无论是在官方仓库还是 AUR 仓库中都不提供全系统模式且使用静态链接的 QEMU 变种，原因是很少有人需要它们。

QEMU 的图形前端

与其他的虚拟化程序如 VirtualBox 和 VMware 不同, QEMU不提供管理虚拟机的 GUI（运行虚拟机时出现的窗口除外），也不提供创建具有已保存设置的持久虚拟机的方法。除非您已创建自定义脚本以启动虚拟机，否则必须在每次启动时在命令行上指定运行虚拟机的所有参数。

Libvirt提供了一种管理 QEMU 虚拟机的便捷方式。有关可用的前端，请参阅 libvirt 客户端列表。

创建新虚拟系统

创建硬盘镜像

**本文或本章节的事实准确性存在争议。**

原因： 如果我没有理解错手册页的话，原始格式只有在文件系统不支持“空洞”或者明确指定了预分配的情况下才会分配全部大小。参见 qemu-img(1) § NOTES。（在 \[主站 Talk:QEMU\] 中讨论）

提示：有关QEMU镜像的更多信息，请参阅 QEMU Wikibook。

除非直接从 CD-ROM 或网络引导（并且不安装系统到本地），运行 QEMU 时都需要硬盘镜像。硬盘镜像是一个文件，存储虚拟机硬盘上的内容。

一个硬盘镜像可能是 *raw*镜像, 和客户机器上看到的内容一模一样，并且将始终使用主机上的来宾硬盘驱动器的全部容量。此方法提供的I / O开销最小，但可能会浪费大量空间，因为guest虚拟机上未使用的空间无法在主机上使用。

另外一种方式是*qcow2* 格式，仅当客户系统实际写入内容的时候，才会分配镜像空间。对客户机器来说，硬盘大小表现为完整大小，即使它可能仅占用主机系统上的非常小的空间。此映像格式还支持QEMU快照功能（有关详细信息，请参阅 #通过 monitor console 创建快照和管理快照）。但是，使用此格式而不是 *raw* 可能会影响性能。

QEMU 提供 `qemu-img`命令创建硬盘镜像.例如创建一个 4 GiB *raw* 格式的镜像:

```
$ qemu-img create -f raw image_file 4G
```

您也可以用 `-f qcow2` 创建一个 *qcow2* 镜像。

用 `dd` 或 `fallocate` 也可以创建一个 *raw* 镜像。

注意：您也可以通过 `dd` 或 `fallocate` 创建一个所需大小的 *raw* 镜像。

警告：如果硬盘镜像存储在 Btrfs 系统上，则应在创建任何映像之前考虑禁用该目录的 写时复制) 功能。可以在创建镜像时为qcow2格式指定选项nocow：

```
$ qemu-img create -f qcow2 image_file -o nocow=on 4G
```

Overlay存储镜像

你可以创建一次存储镜像（backing镜像），然后让 QEMU 只保存对backing镜像的变更到一个overlay镜像中。这能让你恢复到这个存储镜像一开始的状态。你可以在想还原时创建一个新的overlay镜像，基于最初的backing镜像。

要创建一个overlay镜像，执行下面的命令。

```
$ qemu-img create -o backing_file=img1.raw,backing_fmt=raw -f qcow2 img1.cow
```

之后，你可以正常运行 QEMU 虚拟机（参阅 #运行虚拟化的系统）：

```
$ qemu-system-i386 img1.cow
```

基础镜像（指backing镜像）会保持完好，对存储内容的变更将被记录在overlay镜像文件中。

当基础镜像的路径改变时，需要进行修复。

警告：基础镜像的绝对文件系统路径存储在（二进制的）覆写镜像文件中。若改变基础镜像的路径，可能需要做一些调整。

确保原来的基础镜像的路径仍然指向该镜像。如果有必要，在原路径上做一个符号链接到新路径上。然后执行如下命令：

```
$ qemu-img rebase -b /new/img1.raw /new/img1.cow
```

你也可以选择执行一个 “不安全 ”的重定向，即不检查基础镜像的原路径。

```
$ qemu-img rebase -u -b /new/img1.raw /new/img1.cow
```

调整镜像大小

警告：调整包含NTFS引导文件系统的镜像将无法启动已安装的操作系统，推荐在操作之前进行备份

执行 `qemu-img` 带 `resize` 选项调整硬盘驱动镜像的大小.它适用于 *raw* 和 *qcow2*. 例如, 增加镜像 10 GiB 大小, 运行:

```
$ qemu-img resize disk_image +10G
```

在磁盘映像扩容后，必须使用虚拟机内部系统的分区工具对该镜像进行分区并格式化后才能真正开始使用新空间。

减小镜像大小

缩小磁盘映像时，必须首先在虚拟机中使用文件系统和分区工具减少分配的文件系统和分区大小，然后再减小磁盘镜像。对于 Windows 虚拟机，可以从“控制面板-创建并格式化硬盘分区”中执行此操作。

警告：缩小磁盘镜像而不减少虚拟机分区大小将导致数据丢失。

通过以下命令将磁盘镜像减小 10 GiB：

```
$ qemu-img resize --shrink disk_image -10G
```

转换镜像格式

你可以使用 `qemu-img convert` 来调整镜像格式. 例如，你可以使用一下命令来将 *raw* 格式的镜像转换成 *qcow2* 格式的镜像:

```
$ qemu-img convert -f raw -O qcow2 input.img output.qcow2
```

这条命令不会删除 *input*.img

准备安装介质

要将操作系统安装到您的磁盘镜像, 你需要操作系统的安装介质 (例如 光盘, USB设备, 或 ISO 镜像). 不要挂载安装介质，因为 QEMU 要直接访问媒体。

提示：如果使用光盘，最好先将媒体转储到文件中，因为这既可以提高性能，又不需要您直接访问设备（也就是说，您可以将QEMU作为普通用户，而无需更改对媒体设备文件的访问权限）。例如，如果CD-ROM设备节点名为`/dev/cdrom`，则可以使用以下命令将其转储到文件中：

```
$ dd if=/dev/cdrom of=cd_image.iso
```

安装操作系统

这是你第一次需要去启动模拟器的步骤，为了在磁盘镜像上安装操作系统，你必须同时将磁盘镜像与安装介质装载到虚拟机上，从安装介质中启动操作系统。

以i386的客户机为例，为了从CD-ROM内的把可用于启动的ISO文件安装到磁盘镜像上，你需要：

```
$ qemu-system-x86_64 -cdrom iso_image -boot order=d -drive file=disk_image,format=raw
```

参阅 qemu(1) 获得更多关于不同类型安装介质的信息 (例如floppy，磁盘镜像和物理驱动盘)，参阅 #运行虚拟化的系统了解更多有用的选项。

在安装完操作系统后，就可以直接从QEMU镜像内启动了。（参阅 #运行虚拟化的系统）

注意：默认情况下仅分配给虚拟机128MiB的内存， 分配的内存大小可以通过 `-m` 调整， 比如 `-m 512M` 或 `-m 2G`。

提示：

- 相较于指定 `-boot order=x` ，一部分用户感觉使用 `-boot menu=on` 启用boot菜单的体验更舒服些，至少在配置和实验时是这样的。
- 当使用无界面（headless）模式时， 将会默认在本地5900端口启动一个VNC服务器， 可以用 TigerVNC 连接到客户机的系统上: `vncviewer :5900`
- 若你在安装过程中需要替换软盘或CD，可以使用QEMU机器监视器（在虚拟机窗口中按`Ctrl + Alt + 2`）来删除存储设备并将其连接到虚拟机。使用 `info block`查看块设备，然后使用`change`命令换出设备。按下 `Ctrl + Alt + 1`返回虚拟机。

运行虚拟化的系统

`qemu-system-*` 程序 (例如 `qemu-system-i386` 或 `qemu-system-x86_64`, 取决于客户机架构)用来运行虚拟化的客户机。用法是:

```
$ qemu-system-i386 options disk_image
```

所有 `qemu-system-*`的选项是相同的,参见 qemu(1) 查看文档和所有选项

一般来说，如果一个选项有多个可能的值，你可以使用

```
$ qemu-system-x86_64 option help
```

来列出所有可能的值。如果它支持属性，你可以使用

```
$ qemu-system-x86_64 option value,help
```

来列出所有可能的属性。

例如:

```
$ qemu-system-x86_64 -machine help
$ qemu-system-x86_64 -machine q35,help
$ qemu-system-x86_64 -device help
$ qemu-system-x86_64 -device qxl,help
```

你可以使用这些方法和 qemu(1) 的文档来理解接下来几个小节中的相关选项。

默认 QEMU 会在窗口中显示虚拟机的视频输出.有一点要记住:当您单击QEMU窗口,鼠标指针被捕获。要放开，按 `Ctrl+Alt+g`.

警告：QEMU 不应以 root 身份运行. 如果必须以 root 身份在某个脚本中运行 QEMU，那么你需要使用 `-runas` 选项让 QEMU 放弃root 权限

启用 KVM

KVM（基于内核的虚拟机）完全虚拟化必须要您的 Linux 内核和硬件支持，并且必须加载必要的内核模块。更多信息参见 KVM。

要在 KVM 模式中启动 QEMU, 追加 `-accel kvm` 到启动选项。检查运行中的虚拟机是否启用了 KVM，进入 QEMU 监视器，输入 `info kvm`。

注意：

- `-machine` 选项中的 `accel=kvm` 参数与`-enable-kvm` 或 `-accel kvm` 选项是等价的。
- CPU模型 `host` 需要 KVM。
- 如果你使用 GUI 工具去启动 QEMU，但是性能体验极差，那么最好检查一下是否真的开启了 KVM 支持，因为 QEMU 可能选择了备用的软件级模拟。
- 需要启用 KVM 才能正常启动 Windows 7 和 Windows8，否则会出现“蓝屏”.

启用 IOMMU （Intel VT-d/AMD-Vi）的支持

首先，启用 IOMMU。参阅 PCI passthrough via OVMF#Setting up IOMMU.

添加 `-device intel-iommu` 选项创建 IOMMU 设备:

```
$ qemu-system-x86_64 -enable-kvm -machine q35 -device intel-iommu -cpu host ..
```

注意： 在基于 Intel CPU 的系统上，用 `-device intel-iommu` 创建 QEMU 内的 IOMMU 设备将会禁用 PCI 直通， 并返回一个像这样的错误报告：

```
Device at bus pcie.0 addr 09.0 requires iommu notifier which is currently not supported by intel-iommu emulation
```

虽然仍然需要添加内核参数 `intel_iommu=on` 来重新映射 IO（例如，通过 vfio-pci 的 PCI 直通），但如果需要 PCI 直通，则不应设置 `-device intel-iommu`。

以 UEFI 模式启动

QEMU 使用的默认固件是 SeaBIOS，一个传统 BIOS 的实现。QEMU 使用 `/usr/share/qemu/bios-256k.bin` (由 seabios包 提供) 作为默认的只读 (ROM) 镜像。你可以使用参数 `-bios` 来选择另一个固件文件。然而， UEFI 需要可写的内存来正常工作，所以你需要模拟 PC System Flash 。

OVMF 是一个为虚拟机提供 UEFI 支持的 TianoCore 项目。它可以通过 edk2-ovmf包 来安装。

使用 OVMF 作为固件有两种方法。第一种是复制 `/usr/share/edk2/x64/OVMF.4m.fd` 文件，使其可写并用作 pflash 驱动器:

```
-drive if=pflash,format=raw,file=/copy/of/OVMF.4m.fd
```

所有对 UEFI 设置的更改将直接保存到此文件中。

另一种更优选的方法是将 OVMF 分成两个文件。第一个文件将是只读的，并存储固件可执行文件，而第二个文件用作可写的变量存储区。这样做的好处是您可以直接使用固件文件，而无需复制，因此它会由 pacman 自动更新。

使用 `/usr/share/edk2/x64/OVMF_CODE.4m.fd` 作为第一个只读 pflash 驱动器。复制 `/usr/share/edk2/x64/OVMF_VARS.4m.fd` ，使其可写，并用作第二个可写 pflash 驱动器:

```
-drive if=pflash,format=raw,readonly=on,file=/usr/share/edk2/x64/OVMF_CODE.4m.fd \
-drive if=pflash,format=raw,file=/copy/of/OVMF_VARS.4m.fd
```

如果需要启用安全启动，请使用q35机器类型，并将 `/usr/share/edk2/x64/OVMF_CODE.4m.fd` 替换为 `/usr/share/edk2/x64/OVMF_CODE.secboot.fd`。

可信平台模块模拟

QEMU 可以模拟可信平台模块 (Trusted Platform Module) ，这是某些系统（例如需要 TPM 2.0 的 Windows 11）所必需的。

安装 swtpm包 ，来提供一个软件 TPM 实现。创建一个目录来存储 TPM 数据 (这里以 `*/path/to/mytpm*` 为例) 。 运行以下命令启动模拟器r:

```
$ swtpm socket --tpm2 --tpmstate dir=/path/to/mytpm --ctrl
type=unixio,path=/path/to/mytpm/swtpm-sock
```

`*/path/to/mytpm/swtpm-sock*` 将由 *swtpm* 创建: 这是一个 UNIX 套接字，QEMU 将连接到此套接字。您可以将其放在任何目录中。

默认情况下， *swtpm* 启动一个 TPM 1.2 模拟器。 `--tpm2` 选项启用 TPM 2.0 模拟。

最后，将以下选项添加到 QEMU:

```
-chardev socket,id=chrtpm,path=/path/to/mytpm/swtpm-sock \
-tpmdev emulator,id=tpm0,chardev=chrtpm \
-device tpm-tis,tpmdev=tpm0
```

然后 TPM 将在虚拟机内可用。关闭虚拟机后， *swtpm* 将自动终止。

有关更多信息，请参见 the QEMU documentation 。

如果虚拟机操作系统仍然无法识别 TPM 设备，请尝试调整可能会导致该问题的 *CPU 模型和拓扑*选项。

宿主机和虚拟机数据交互

网络

我们可以利用任何支持文件传输的网络协议实现客户机和宿主机之间的数据交互, 例如 NFS, SMB, NBD, HTTP, FTP, 或 SSH, 当然这么做的前提是你已经配置好二者之间的网络，且在系统上启动了相应的服务程序。

在默认情况下，用户模式的客户机能够通过 10.0.2.2 这个IP访问到宿主机。任何运行于宿主机上的服务端程序都可以通过这个地址被访问到，比如说我们可以通过这个IP访问到宿主机上的 SSH 服务器或 SMB 服务器。因此在这种情况下，客户机能够挂载宿主机通过 SMB 或 NFS 暴露出来的目录，也可以访问宿主机上的 HTTP 服务器等。通常情况下宿主机无法访问客户机上的服务，不过你也可以通过一些特殊的网络配置达到这个目的 (参阅#Tap 网络)

QEMU 端口转发

注意：QEMU's port forwarding is IPv4-only. IPv6 port forwarding is not implemented and the last patches were proposed in 2018.

QEMU 能够将宿主机的端口转发到客户机上以实现一些功能，例如从宿主机上访问客户机的 SSH 端口。

举个例子，将宿主机上的 60022 端口与客户机上的22 (SSH) 端口进行绑定，需通过如下命令启动 QEMU：

```
$ qemu-system-x86_64 disk_image -nic user,hostfwd=tcp::60022-:22
```

确认你客户机上的 sshd 程序正在运行，然后可以通过如下命令连接到客户机的 SSH 端口：

```
$ ssh guest-user@127.0.0.1 -p 60022
```

你可以用 SSHFS 把客户机的整个文件系统都挂到宿主机上，这样就可以在宿主机上对客户机的文件系统进行读写了。

想进行多端口转发的话, 只需要在 `-nic` 参数中指定多个 `hostfwd`, 以 VNC 端口为例:

```
$ qemu-system-x86_64 disk_image -nic
user,hostfwd=tcp::60022-:22,hostfwd=tcp::5900-:5900
```

QEMU 的内置SMB服务器

QEMU的文档中指出它有一个内置的 SMB 服务器，但实际上，它只是在宿主机上加载一个自动生成的 `smb.conf` 配置文件 (位于`/tmp/qemu-smb.*random_string*`)，然后启动宿主机上的 Samba，使得客户机能够通过一个IP地址进行访问 (默认的IP地址是10.0.2.4)。这个方法只适用于用户网络，在你不想在宿主机开启通常的 Samba 服务 (客户机同样能访问这类Samba服务) 时这个方法还挺好用的。

选项 `smb=` 仅可以设置共享一个目录，如果 QEMU 的 SMB 配置允许用户使用符号链接，那么即使在虚拟机运行时也很容易新加入更多的目录，只需要通过在共享目录里创建相应的软链接就行。然而就算并没有这么配置，我们可以依照如下进行配置 SMB 服务器

宿主机上必须安装 *Samba*。通过如下QEMU命令启用这项特性:

```
$ qemu-system-x86_64 -nic user,id=nic0,smb=shared_dir_path disk_image
```

`*shared_dir_path*` 就是你想要在宿主机和客户机之间共享的目录。

接着，在客户机内，你应该能够通过 10.0.2.4 访问到名为 qemu 的共享文件夹，例如可以在 Windows 文件浏览器中前往 `\\10.0.2.4\qemu` 这个地址。

注意：

- 如果你像这样多次指定共享选项：`-net user,smb=*shared_dir_path1* -net user,smb=*shared_dir_path2*` 或是 `-net user,smb=*shared_dir_path1*,smb=*shared_dir_path2*`，qemu 只会共享参数中最后的一个目录。
- 如果你不能访问共享文件夹且客户机系统为 Windows, 请检查 NetBIOS 协议是否被启用\[失效链接 2023-05-06 ⓘ\] 并确认防火墙没有屏蔽 NetBIOS 协议的 端口
- 如果你无法访问共享文件夹且客户机系统为 Windows 10 企业版/教育版或 Windows Server 2016, 请启用游客访问.

共享多个文件夹并在运行时增删文件夹的一个方法是：共享一个空目录，然后在其中创建指向其余共享目录的符号链接。可以用下面的脚本修改SMB服务器的配置，这个脚本还能使宿主机上不允许执行的文件在客户机内拥有执行权限。

```
#!/bin/sh
eval $(ps h -C smbd -o pid,args | grep /tmp/qemu-smb | gawk '{print
"pid="$1";conf="$6}')
echo "[global]
allow insecure wide links = yes
[qemu]
follow symlinks = yes
wide links = yes
acl allow execute always = yes" >> "$conf"
# in case the change is not detected automatically:
smbcontrol --configfile="$conf" "$pid" reload-config
```

仅当 qemu 启动的客户机第一次访问到网络磁盘后，才能应用该脚本。共享多文件夹的另一个方法是在配置文件里加入额外的共享路径，就像下面这样

```
echo "[myshare]
path=another_path
read only=no
guest ok=yes
force user=username" >> $conf
```

这个共享文件夹可以在客户机内通过 `\\10.0.2.4\*myshare*` 访问。

使用文件系统直通和 VirtFS

参阅 QEMU 文档.

Host file sharing with virtiofsd

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：See Help:Style/Formatting and punctuation.（在Talk:QEMU讨论）

virtiofsd包 提供了 virtiofsd 工具。有关可用选项的完整列表，请参阅在线文档或 qemu-docs包 内置的 `/usr/share/doc/virtiofsd/README.md` 文件。

由于 qemu 程序需要访问 virtiofsd 套接字，因此请将运行 qemu 的用户添加到 'kvm' 用户组。你可能需要重新登录才能使更改生效。

**本文或本章节的事实准确性存在争议。**

原因： Running services as root is not secure. Also the process should be wrapped in a systemd service.（在 Talk:QEMU 中讨论）

Start as virtiofsd as root:

```
# /usr/lib/virtiofsd --socket-path=/var/run/qemu-vm-001.sock --shared-dir /tmp/vm-001
--cache always
```

where

- `/var/run/qemu-vm-001.sock` is a socket file,

- `/tmp/vm-001` is a shared directory between the host and the guest virtual machine.

The created socket file has root only access permission. Give group kvm access to it with:

```
# chgrp kvm qemu-vm-001.sock; chmod g+rxw qemu-vm-001.sock
```

Add the following configuration options when starting the virtual machine:

```
-object memory-backend-memfd,id=mem,size=4G,share=on \
-numa node,memdev=mem \
-chardev socket,id=char0,path=/var/run/qemu-vm-001.sock \
-device vhost-user-fs-pci,chardev=char0,tag=myfs
```

where

- `size=4G` shall match size specified with `-m 4G` option,
- `/var/run/qemu-vm-001.sock` points to socket file started earlier,

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：The section should not be specific to Windows.（在Talk:QEMU讨论）

Remember, that guest must be configured to enable sharing. For Windows there are instructions. Once configured, Windows will have the `Z:` drive mapped automatically with shared directory content.

Your Windows 10 guest system is properly configured if it has:

- VirtioFSSService windows service,
- WinFsp.Launcher windows service,
- VirtIO FS Device driver under "System devices" in Windows "Device Manager".

If the above installed and `Z:` drive is still not listed, try repairing "Virtio-win-guest-tools" in Windows *Add/Remove programs*.

在宿主机上挂载客户机的分区

在主机系统上挂载驱动器镜像可能很有用，它可以作为在虚拟机和主机之间传输文件的一种方式。这应在虚拟机未运行时进行。

挂载驱动器的过程取决于 qemu 镜像的类型 (*raw* 或 *qcow2*) 。我们将在 #挂载 raw 镜像内的分区和 #挂载 qcow2 镜像内的分区中详细介绍两种格式的挂载步骤。完整文档请参见Wikibooks:QEMU/Images#Mounting an image on the host。

警告：请确保重新运行虚拟机之前，所有挂载的分区都被卸载，否则很可能造成磁盘数据的损坏。

挂载 raw 镜像内的分区

可以通过将 raw 磁盘镜像文件内的分区设置为环回设备来挂载这些分区。

手动指出偏移量

挂载磁盘镜像中分区的一个方法是手动指定挂载的偏移量，你可以使用类似下面的命令完成这个操作：

```
# mount -o loop,offset=32256 disk_image mountpoint
```

`offset=32256` 选项实际上会被传递给 `losetup` 程序，用于设置一个起始地址为 32256 字节处的 loop 设备。接着这个 loop 设备将会被挂载。 你也可以使用 `sizelimit` 选项指定这个分区的具体大小，通常不需要指定该选项。

具体的偏移量取决于你的磁盘镜像，你所需要的分区可能并不以 32256 字节处作为起始地址。运行 `fdisk -l *disk_image*` 查看磁盘镜像中的分区，fdisk 会显示分区的起始地址和结束地址，地址以 512 字节的扇区为单位，因此需要将该地址乘以 512 获得可用于 `mount` 的字节偏移量。

loop模块自动检测分区

Linux 的 loop 驱动支持 loopback 设备的分区，不过默认情况下它是关闭的，可以通过下面的方法启用：

- 卸载所有 loopback 设备 (比如说卸载所有挂载的镜像)。
- 卸下 `loop` 内核模块, 接着以 `max_part=15` 参数重新加载该模块。 此外，你可以用 `max_loop` 参数设置 loop 设备的最大数量。

提示：你可以在 `/etc/modprobe.d` 增加一个条目使得每次加载 loop 模块时带上 `max_part=15` 参数, 或者把 `loop.max_part=15` 加入内核命令行, 这得看你的内核是否带有 `loop.ko` 模块。

将镜像文件设置为 loopback 设备:

```
# losetup -f -P disk_image
```

接着，假设创建的loop设备名称为 `/dev/loop0` ，相应的 `/dev/loop0p*X*` 也会被自动创建， X 代表分区的编号，这些分区可以被直接挂载，例如：

```
# mount /dev/loop0p1 mountpoint
```

如要使用 *udisksctl* 挂载磁盘镜像, 参阅 Udisks#Mount loop devices.

使用 kpartx

multipath-tools包 包内的 *kpartx* 可以读取设备的分区表，然后为每个分区创建一个新设备，举个例子：

```
# kpartx -a disk_image
```

这条命令将会为你设置 loopback 设备，并在 `/dev/mapper/` 下创建必要的分区设备。

挂载 qcow2 镜像内的分区

我们将使用 `qemu-nbd` 完成这一功能, 同时它也能让我们使用 NBD (*network block device*) 协议共享该磁盘镜像。

首先，我们需要加载 *nbd* 模块：

```
# modprobe nbd max_part=16
```

接着，共享该磁盘并创建设备条目：

```
# qemu-nbd -c /dev/nbd0 /path/to/image.qcow2
```

进行分区发现检测：

```
# partprobe /dev/nbd0
```

fdisk 可以获取 `*nbd0*` 内各分区的相关信息：

```
# fdisk -l /dev/nbd0
Disk /dev/nbd0: 25.2 GiB, 27074281472 bytes, 52879456 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa6a4d542

Device       Boot    Start       End   Sectors  Size Id Type
/dev/nbd0p1 *        2048   1026047   1024000  500M  7 HPFS/NTFS/exFAT
/dev/nbd0p2       1026048 52877311 51851264 24.7G  7 HPFS/NTFS/exFAT
```

接下来可以挂载镜像的任意分区了，比如说我们要挂载分区2：

```
# mount /dev/nbd0p2 mountpoint
```

完成任务后，切记卸载镜像文件，然后根据之前的操作一步步还原，即分区并断开与 nbd 设备的连接：

```
# umount mountpoint
# qemu-nbd -d /dev/nbd0
```

将任意分区作为磁盘镜像唯一主分区

有时，你可能想在 QEMU 中使用物理机的一个系统分区。在虚拟机里使用原始分区可以改善读写性能，因为此时 QEMU 不需要经过宿主机的文件系统层完成读写操作，该分区同样可以用于在宿主机和客户机之间进行数据共享。

在 Arch Linux 的默认设置中，代表原始分区的设备文件的所有者为 *root*， 隶属于 *disk* 组。如果你希望使用一个非 root 用户能够对原始分区进行读写，那么请更改对应设备文件的所有者为该用户，将这个用户加入 *disk* 组中，或者使用 ACL 完成更精细的权限管理。

警告：

- 不推荐让虚拟机修改宿主机系统上重要数据，比如说对 root 分区进行修改，尽管你拥有这么做的能力。
- 不要同时挂载一块在宿主机和客户机内都可读写的分区，否则会造成数据的损坏。

在完成上述设置后，你可以将该分区作为一块虚拟磁盘添加到 QEMU 虚拟机内了。

然而，如果你想使得整个虚拟机都包含在单个分区中，事情就开始变得有些复杂了。在这个场景下，由于系统分区本身是作为文件系统完成格式化的，而非一个带有 MBR 的分区设备，我们便无法在这个分区上安装引导加载程序，进而也就没有办法启动虚拟机了。要启动这类虚拟机，你有如下几种选择：#手动指定 kernel 和 initrd，#模拟一块带 MBR 的磁盘，#使用设备映射器，#使用线性 RAID或是#使用网络块设备。

手动指定 kernel 和 initrd

QEMU 支持直接加载 Linux 内核和 init ramdisks，从而绕过了类似 GRUB 这类的引导加载程序。它会把包含根文件系统的物理分区作为虚拟机的虚拟磁盘，然后启动虚拟机。通过类似下面的命令可以完成这些操作：

注意：在示例中， 我们使用的是 **宿主机的** 镜像，而非客户机的。如果你希望使用客户机的镜像，请将 `/dev/sda3` 设置为只读（为了保护宿主机上的文件系统），并且指定 `/full/path/to/images`，或者在客户机内使用一些 kexec 的技巧，重新加载客户机的内核（会延长启动时间）

```
$ qemu-system-x86_64 -kernel /boot/vmlinuz-linux -initrd /boot/initramfs-linux.img -append root=/dev/sda /dev/sda3
```

在上面这个例子中，宿主机上的 `/dev/sda3` 物理分区被用于客户机的根文件系统，但是在虚拟机内部它的名字则是 `/dev/sda`

当然你可以指定任意一个 kernel 和 initrd，而不局限于Arch Linux提供的。

`-append` 中可以传递许多内核参数， 需要用单引号或双引号将他们包起来，比如：

```
... -append 'root=/dev/sda1 console=ttyS0'
```

模拟一块带 MBR 的磁盘

当我们想要既保持物理分区遵循文件系统的格式，又作为客户机的虚拟磁盘包含客户机内的分区时，我们有一个比较复杂的方法，那就是模拟一块带 MBR 的磁盘，这样就使得它可以被类似 GRUB 的引导程序正常启动。

假设你有一块普通的、未挂载的硬盘分区 `/dev/hda*N*`，分区内有些文件系统，若想将其制作为QEMU硬盘镜像，需要在该物理分区内动态地预置一个主引导记录（MBR）。更宽泛地说，该分区可以是更大的模拟磁盘中的一部分，尤其是那种模拟了物理磁盘，但仅向虚拟机暴露 `/dev/hda*N*` 名称的块设备。

这类虚拟磁盘可以用带引用的 VMDK 文件形容，VMDK内的引用指向 MBR 的副本以及实际的分区。 但 QEMU 并不支持这类VMDK格式，例如通过如下命令创建的虚拟机：

```
$ VBoxManage internalcommands createrawvmdk -filename /path/to/file.vmdk -rawdisk /dev/hda
```

将会被QEMU拒绝，并返回一个错误信息

```
Unsupported image type 'partitionedDevice'
```

注意 `VBoxManage` 会创建两个文件：`*file.vmdk*` 和 `*file-pt.vmdk*`, 后一个是 `file.vmdk` MBR 的副本。对目标分区外的数据或MBR本身的读操作将会返回0，而写操作则会被丢弃。

使用设备映射器

与使用 VMDK 描述文件类似的方法就是使用设备映射器，设备映射器会在目标分区内添加一份带有 loop 设备的 MBR 文件。在此种情况下，不要求虚拟磁盘与原始分区拥有同样的大小。首先创建一个包含 MBR 的文件：

```
$ dd if=/dev/zero of=/path/to/mbr count=2048
```

在这里，我们通过这个命令，使用现代化的磁盘分区程序创建了一个符合分区对齐策略的文件，总共大小为 1MiB（2048 * 512 字节）。为了兼容老式的磁盘分区程序，可能要求其为 63 个扇区而非 2048 个扇区。MBR 表仅占一块512字节的块，剩余的空间可以用于创建一个 BIOS 启动分区，或者在创建一个在混合分区方案中需要用到的 GUID 分区表。接着我们将 loop 设备附加到 MBR 文件上：

```
# losetup --show -f /path/to/mbr
/dev/loop0
```

在这个例子里，最终生成的设备为 `/dev/loop0`，设备映射器将会把 MBR 文件与分区结合起来：

```
# echo "0 2048 linear /dev/loop0 0
2048 `blockdev --getsz /dev/hdaN` linear /dev/hdaN 0" | dmsetup create qemu
```

生成的 `/dev/mapper/qemu` 可以被作为 QEMU 原生镜像使用了，在虚拟磁盘上创建一个分区表（可参阅关于线性 RAID 的部分），以及相应的引导加载器代码（代码会存放在 `*/path/to/mbr*`）需要额外的步骤。

下面的步骤中，`/dev/hda*N*` 在虚拟磁盘和物理磁盘上的位置都相同，并且除了MBR（副本）外磁盘的其余部分都是隐藏的：

```
# dd if=/dev/hda count=1 of=/path/to/mbr
# loop=`losetup --show -f /path/to/mbr`
# start=`blockdev --report /dev/hdaN | tail -1 | awk '{print $5}'`
# size=`blockdev --getsz /dev/hdaN`
# disksize=`blockdev --getsz /dev/hda`
# echo "0 1 linear $loop 0
1 $((start-1)) zero
$start $size linear /dev/hdaN 0
$((start+size)) $((disksize-start-size)) zero" | dmsetup create qemu
```

通过管道传入作为 `dmsetup` 标准输入的是一张表，该表的格式与 `VBoxManage` 创建的VMDK描述文件中的表格式相同， 即我们同样能通过 `dmsetup create qemu --table *table_file*` 加载VMDK描述文件中的表。 对于虚拟机来说，仅 `/dev/hda*N*` 能被访问， 而对其余部分， 除了第一个扇区外，对其他扇区的读

操作都只会返回0， 写操作都会被丢弃。 可以用 `dmsetup table qemu` 显示 `/dev/mapper/qemu` 的表（用 `udevadm info -rq name /sys/dev/block/*major*:*minor*` 将 `*major*:*minor*` 转化为类似 `/dev/*blockdevice*` 的名字）。若要删除创建的设备，请使用 `dmsetup remove qemu` 和 `losetup -d $loop`。

设备映射器方法一个可能的应用场景与Windows XP有关，比如说要在Windows XP安装中进行多引导配置，并且有可能要采用混合分区方案（从物理硬件上说，Windows XP可能是唯一使用MBR分区表的系统，更现代化的操作系统能使用GUID分区表）。Windows XP支持硬件配置文件，因此同样的安装可以选择不同的硬件配置完成（在本例中是裸机与虚拟机），而Windows仅需为每个配置文件安装一次新检测到的硬件的驱动即可。注意在这个例子里， 要更新MBR副本中bootloader代码的部分，使得它直接从 `/dev/hda*N*` 加载Windows XP而不是系统内自带的多引导bootloader（比如GRUB）。或者，将包含bootloader安装的启动分区之副本包含于虚拟磁盘中，同样也可起到类似MBR的作用。

使用线性 RAID

你同样能用 线性模式（需要 `linear.ko` 内核驱动）的RAID以及一块loop设备完成这件事。

首先创建一个用于容纳MBR的小文件：

```
$ dd if=/dev/zero of=/path/to/mbr count=32
```

这样就创建了一个 16 KiB（32 * 512字节）的小文件。该文件最好不要太小（尽管 MBR 只需要一个512字节大小的块），过小的话将会限制软 RAID 设备的 chunk 大小，对于性能是有影响的。将 loop 设备设置到 MBR 文件上。

```
# losetup -f /path/to/mbr
```

因为我们尚无法使用其他 loop 设备， 因此就先假设产生的设备名为 `/dev/loop0`。下一步将创建合并式 MBR + 使用软件 RAID 的磁盘镜像 `/dev/hda*N*` ：

```
# modprobe linear
# mdadm --build --verbose /dev/md0 --chunk=16 --level=linear --raid-
devices=2 /dev/loop0 /dev/hdaN
```

这一步产生的 `/dev/md0` 设备将在之后作为一个 QEMU 原始镜像（别忘了给模拟器相应的访问权限）。最后一步（也是比较取巧的一步）是设置磁盘配置，使得 MBR 内主分区的起始地址与 `/dev/md0` 中某一块 `/dev/hda*N*` 的起始地址（在这个例子中是 16 * 512 = 16384 字节的偏移处）。请在宿主机上用 `fdisk` 完成该操作，别在虚拟机里面这么做，因为默认的 QEMU 内的磁盘检测程序常给出无法进行千位舍入的

偏移量（比如 31.5 KiB），软件RAID无法处理这样的情况。因此请在主机上进行下面的操作：

```
# fdisk /dev/md0
```

按下 `X` 进入专家选单， 设置每个磁道上 's'ectors 的数目，使得一个柱面的大小与MBR文件中相符。对于双磁头柱面，且每个扇区为 512 字节的情况， 每个磁道上的扇区数当为 16， 因此我们通过计算 2x16x512=16k 得到柱面的大小。

按下 `R` 返回主界面。

按下 `P` 并检查柱面大小为 16k。

现在，来创建与 `/dev/hda*N*` 对应的主分区。 设置它的起始地址为柱面2， 结束地址在磁盘的末端（注意现在的柱面编号已经与进入 fdisk 时的编号不同了）。

最后，把结果写入：完成～ 现在你拥有了一块可以直接从宿主机进行挂载的分区，该分区同样也是 QEMU 磁盘镜像的一部分。

```
$ qemu-system-x86_64 -hdc /dev/md0 [...]
```

现在，若原始的 `/dev/hda*N*` 分区中包含必要的一些工具，你就可以安全地用 QEMU 在磁盘镜像上设置任何一个引导加载程序。

使用网络块设备

With Network Block Device, Linux can use a remote server as one of its block device. 你也可以用 `nbd-server`（在 nbd包 包中）为 QEMU 创建一个 MBR 封装器。

假设你已经用上面所说的方法创建了一个方法创建了一个MBR封装文件，将其重命名为 `wrapper.img.0`。然后在同一个目录下创建一个名为 `wrapper.img.1`的符号链接，指向你选择的分区。然后，还是在这个文件夹下，创建如下的脚本：

```
#!/bin/sh
dir="$(realpath "$(dirname "$0")")"
cat >wrapper.conf <<EOF
[generic]
allowlist = true
listenaddr = 127.713705
port = 10809

[wrap]
exportname = $dir/wrapper.img
multifile = true
EOF

nbd-server \
    -C wrapper.conf \
    -p wrapper.pid \
    "$@"
```

`.0` and `.1` 这两个后缀名是关键，名字的其他其他部分都可以更改。在运行上面的脚本（有可能需要以root身份运行，保证nbd-server能够去访问该分区）后，你可以用如下命令启动QEMU：

```
qemu-system-x86_64 -drive file=nbd:127.713705:10809:exportname=wrap [...]
```

Using an entire physical disk device inside the virtual machine

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Duplicates #Using any real partition as the single primary partition of a hard disk image, libvirt instructions do not belong to this page.（在Talk:QEMU讨论）

You may have a second disk with a different OS (like Windows) on it and may want to gain the ability to also boot it inside a virtual machine. Since the disk access is raw, the disk will perform quite well inside the virtual machine.

Windows virtual machine boot prerequisites

Be sure to install the virtio drivers inside the OS on that disk before trying to boot it in the virtual machine. For Win 7 use version 0.1.173-4. Some singular drivers from newer virtio builds may be used on Win 7 but you will have to install them manually via device manager. For Win 10 you can use the latest virtio build.

Set up the windows disk interface drivers

You may get a `0x0000007B` bluescreen when trying to boot the virtual machine. This means Windows can not access the drive during the early boot stage because the disk interface driver it would need for that is not loaded / is set to start manually.

The solution is to enable these drivers to start at boot.

In `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services`, find the folders `aliide, amdide, atapi, cmdide, iastor (may not exist), iastorV, intelide, LSI_SAS, msahci, pciide and viaide`. Inside each of those, set all their "start" values to 0 in order to enable them at boot. If your drive is a PCIe NVMe drive, also enable that driver (should it exist).

Find the unique path of your disk

Run `ls /dev/disk/by-id/`: tere you pick out the ID of the drive you want to insert into the virtual machine, for example `ata-TS512GMTS930L_C199211383`. Now add that ID to `/dev/disk/by-id/` so you get `/dev/disk/by-id/ata-TS512GMTS930L_C199211383`. That is the unique path to that disk.

Add the disk in QEMU CLI

In QEMU CLI that would probably be:

```
-drive file=/dev/disk/by-id/ata-TS512GMTS930L_C199211383,format=raw,media=disk
```

Just modify `file=` to be the unique path of your drive.

Add the disk in libvirt

In libvirt XML that translates to

```
$ virsh edit vmname
...
    <disk type="block" device="disk">
      <driver name="qemu" type="raw" cache="none" io="native"/>
      <source dev="/dev/disk/by-id/ata-TS512GMTS930L_C199211383"/>
      <target dev="sda" bus="sata"/>
      <address type="drive" controller="0" bus="0" target="0" unit="0"/>
    </disk>
...
```

Just modify "source dev" to be the unique path of your drive.

Add the disk in virt-manager

When creating a virtual machine, select "import existing drive" and just paste that unique path. If you already have the virtual machine, add a device, storage, then select or create custom storage. Now paste the unique path.

网络

警告：本部分在翻译过程中，英文wiki产生style告警，这代表日后本节的内容将会有相应调整，暂停本节的翻译： 网络拓扑 (#仅主机 网络, #内部网络 节中以及其他节中引用的部分) 不应当根据不同的虚拟网络接口实现进行划分和描述， 例如#用户模式, #Tap 网络, #通过 VDE2 配置网络。

采用TAP设备和网桥的虚拟网络的性能应该会比使用用户模式网络或VDE要好，原因在于TAP设备和网桥是在内核中实现的。

此外，虚拟网络的性能可以通过将 virtio 网络设备直接注册到虚拟机中改善，这比默认情况下模拟 e1000 NIC的性能表现要更好，参阅 #安装 virtio 驱动获得更多相关信息。

关于链路层地址的限制

若在QEMU启动中指定了 `-net nic` 参数，QEMU将会为虚拟机注册一块虚拟网卡，其链路层地址为 `52:54:00:12:34:56` 。然而，当在多台虚拟机之间搭建桥接网络时，每台虚拟机在tap设备的虚拟机端都需要拥有一个独一无二的链路层地址 (MAC)，否则网桥会因为收到多个不同源却拥有相同MAC地址的数据包而无法正常工作。即使你为多个tap设备配置了不同的MAC地址也依旧会出现这个问题，因为当数据包通过tap设备时，tap设备并不会改写包内的链路层地址。

因此请确保每个虚拟机拥有自己独一无二的网卡地址, 并且它们都以 `52:54:` 开头。可以通过如下命令手动设置虚拟机的MAC地址, 下面的'X'可以替换成任何16进制字符:

```
$ qemu-system-x86_64 -net nic,macaddr=52:54:XX:XX:XX:XX -net vde disk_image
```

生成不同的链路层地址有很多方法:

- 手动为每个NIC设置独一无二的链路层地址，这么做的优点在于每次启动虚拟机时，DHCP服务器都会将相同的IP地址分配给对应的MAC地址，但是这个方法在需要大量虚拟机的情况下就不适用了。
- 在每次启动虚拟机时随机生成链路层地址，实际情况下地址冲突的概率可视为0， 不过此方法的缺点在于DHCP服务器每次都会分配一个不同的IP地址。你可以用如下的这些脚本生成随机的链路层地址，并用于 `macaddr` 参数中。

```
printf -v macaddr "52:54:%02x:%02x:%02x:%02x" $(( $RANDOM & 0xff)) $(( $RANDOM & 0xff )) $(( $RANDOM & 0xff)) $(( $RANDOM & 0xff ))
qemu-system-x86_64 -net nic,macaddr="$macaddr" -net vde disk_image
```

- 使用这个脚本 `qemu-mac-hasher.py` 可以根据虚拟机的名字进行Hash得到一个链路层地址。 只要每台虚拟机的名字是独一无二的, 这个方法就结合了上述两种方法的优点: 每次运行脚本生成的链路层地址都是相同的, 且冲突概率在实际应用中仍可视为0.

```
qemu-mac-hasher.py
#!/usr/bin/env python
# usage: qemu-mac-hasher.py <VMName>

import sys
import zlib

crc = str(hex(zlib.crc32(sys.argv.encode("utf-8")))).replace("x", "")[-8:]
print("52:54:%s%s:%s%s:%s%s:%s%s" % tuple(crc))
```

如果要在脚本调用这个方法，你可以参照下面的例子:

```
vm_name="VM Name"
qemu-system-x86_64 -name "$vm_name" -net nic,macaddr=$(qemu-mac-hasher.py "$vm_name") -net vde disk_image
```

用户模式

默认情况下，没有任何`-netdev`参数，QEMU将使用带有内置DHCP服务器的用户模式网络。当您的虚拟机运行其DHCP客户端时，将为其分配IP地址，它们将能够通过QEMU伪装的IP来访问物理主机的网络。

注意：ICMPv6 尚未完成实现，因此不会起效：`Slirp: external icmpv6 not supported yet.` Ping an IPv6 address will not work.

如果主机已连接Internet，则此默认配置可以使您的虚拟机轻松访问Internet。但是如果您同时启动多个虚拟机，则虚拟机将无法在外部网络上直接看到，虚拟机也将无法相互通信。

QEMU的用户模式网络可以提供更多功能，例如内置TFTP或SMB服务器，将主机端口重定向到虚拟机（例如，允许SSH连接到虚拟机）或将虚拟机连接到VLAN，以便它们可以彼此通信。 有关更多详细信息，请参见`-net user`标志上的QEMU文档。

但是，用户模式网络在效用和性能上都有局限性。更高级的网络配置需要使用TAP设备或其他方法。

注意：如果主机系统使用systemd-networkd，请确保按照systemd-networkd#Required services and setup中的描述对`/etc/resolv.conf`文件进行符号链接，否则虚拟机系统中的DNS查找将无法进行。

提示：

- 使用此选项，虚拟机将使用 virtio 网络驱动程序来进行用户模式网络： `-nic user,model=virtio-net-pci`.
- 使用此选项，用户模式网络将被隔离，与主机和外部网络隔离开来：`-net user,restrict=y`.

Tap 网络

Tap devices是一个Linux内核特性，允许您创建作为真实网络接口的虚拟网络接口。发送到tap接口的包将被传递到一个用户空间程序(如QEMU)，该程序将自己绑定到该接口。

QEMU可以为虚拟机使用tap网络，因此发送到tap接口的包将被发送到虚拟机，并显示为来自虚拟机中的网络接口(通常是以太网接口)。相反，虚拟机通过其网络接口发送的所有内容都将出现在tap接口上。

Linux桥接驱动程序支持Tap设备，因此可以将Tap设备彼此桥接在一起，也可以连接其他主机接口，如`eth0`。如果您希望您的虚拟机能够相互通信，或者希望LAN上的其他机器能够与虚拟机通信，那么这是非常理想的方案。

警告：如果您将tap设备和一些主机接口桥接在一起，例如`eth0`，您的虚拟机将直接出现在外部网络上，这将使它们遭受攻击的可能。根据您的虚拟机可以访问的资源，您可能需要采取所有precautions来保护您的虚拟机。如果风险太大,虚拟机没有资源或您设置多个虚拟机,一个更好的解决方案可能是使用host-only networking建立NAT。在这种情况下，您只需要在主机上安装一个防火墙，而不是为每个虚拟机安装多个防火墙。

正如在用户模式网络部分中指出的，tap设备提供比用户模式具有更高的网络性能。如果虚拟机中的操作系统支持virtio网络驱动程序，那么网络性能也会显著提高。假设使用tap0设备，virtio驱动程序在客户端上使用，并且没有使用脚本来帮助启动/停止网络，使用下面的qemu命令：

```
-device virtio-net,netdev=network0 -netdev
tap,id=network0,ifname=tap0,script=no,downscript=no
```

但是，如果已经使用带有virtio网络驱动程序的Tap设备，则甚至可以通过启用vhost来提高网络性能，例如：

```
-device virtio-net,netdev=network0 -netdev
tap,id=network0,ifname=tap0,script=no,downscript=no,vhost=on
```

详情请参考：https://web.archive.org/web/20160222161955/http://www.linux-kvm.com:80/content/how-maximize-virtio-net-performance-vhost-net

仅主机 网络

如果为网桥提供了IP地址，并且使能发往该网桥的流量允许，但没有实际接口（例如`eth0`）连接到网桥，则虚拟机与虚拟机间，虚拟机与主机间能够相互通信。但是，如果您没有在物理主机上设置IP掩蔽，则他们将无法与外部网络进行通信。 此配置被其他虚拟化软件（例如VirtualBox）称为“仅主机网络模式”。

提示：

- 如果你想设置IP掩蔽，例如虚拟机的NAT，请查看Internet sharing#Enable NAT页面。
- 您也许想在网桥接口上运行一个DHCP服务器来服务虚拟网络。例如，使用`172.20.0.1/16`子网， dnsmasq作为DHCP服务器:

```
# ip addr add 172.20.0.1/16 dev br0
# ip link set br0 up
# dnsmasq -C /dev/null --interface=br0 --bind-interfaces --dhcp-
range=172.20.0.2,172.20.255.254
```

内部网络

如果您不为网桥提供IP地址并在iptables添加INPUT规则链，将所有流向网桥中的数据丢弃，则虚拟机将能够彼此通信，但无法与物理主机或外部网络通信。此配置被其他虚拟化软件（例如VirtualBox）称为“内部网络”。您将需要为虚拟机分配静态IP地址，或在其中一个虚拟机上运行DHCP服务器。

在默认情况下，iptables将丢弃桥接网络中的数据包。您可能需要使用这样的iptables规则来允许桥接网络中的数据包:

```
# iptables -I FORWARD -m physdev --physdev-is-bridged -j ACCEPT
```

使用 qemu-bridge-helper 桥接网络

这种方法不需要启动脚本，并且很容易适应多个tap和多个桥。它使用`/usr/lib/qemu/qemu-bridge-helper`，允许在现有桥上创建tap设备。

提示：

- 参见 Network bridge 获取创建网桥的信息
- See https://wiki.qemu.org/Features/HelperNetworking for more information on QEMU's network helper.

首先，创建一个配置文件，包含QEMU使用的所有网桥的名称:

```
/etc/qemu/bridge.conf
allow br0
allow br1
...
```

Make sure `/etc/qemu/` has `755` permissions. QEMU issues and GNS3 issues may arise if this is not the case.

Now start the virtual machine; the most basic usage to run QEMU with the default network helper and default bridge `br0`:

```
$ qemu-system-x86_64 -nic bridge [...]
```

Using the bridge `br1` and the virtio driver:

```
$ qemu-system-x86_64 -nic bridge,br=br1,model=virtio-net-pci [...]
```

手工创建网桥

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：This section needs serious cleanup and may contain out-of-date information.（在Talk:QEMU讨论）

提示：自 QEMU 1.1 起，network bridge helper可以为您设置 tun/tap，而无需其他脚本。 请参阅#使用 qemu-bridge-helper 桥接网络。

下面介绍如何将虚拟机连接到主机接口，如`eth0`，这可能是最常见的配置。这种配置使虚拟机看起来直接位于外部网络，与物理主机位于同一以太网段。

我们将用桥适配器替换普通的以太网适配器，然后将普通的以太网适配器绑定到它。

- 安装 bridge-utils包，它提供 `brctl` 来操作网桥。
- 启用 IPv4 转发:

```
# sysctl -w net.ipv4.ip_forward=1
```

要使更改永久生效，请将 `/etc/sysctl.d/99-sysctl.conf` 中的 `net.ipv4.ip_forward = 0` 更改为 `net.ipv4.ip_forward = 1`。

- 加载 `tun` 模块，并将其配置为在引导时加载。详见Kernel modules。
- 现在创建桥。有关详细信息，请参见Bridge with netctl。请记住网桥的命名，如 `br0`，或将以下脚本更改为网桥的名称。In the `run-qemu` script below, `br0` is set up if not listed, as it is assumed that by default the host is not accessing network via the bridge.
- 创建 QEMU 用于打开 tap 适配器的脚本，该脚本具有 `root:kvm` 750权限:

```
/etc/qemu-ifup
#!/bin/sh

echo "Executing /etc/qemu-ifup"
echo "Bringing up $1 for bridged mode..."
sudo /usr/bin/ip link set $1 up promisc on
echo "Adding $1 to br0..."
sudo /usr/bin/brctl addif br0 $1
sleep 2
```

- 创建QEMU用于在`/etc/qemu-ifdown`中关闭tap适配器的脚本，该脚本具有 `root:kvm` 750权限:

```
/etc/qemu-ifdown
#!/bin/sh

echo "Executing /etc/qemu-ifdown"
sudo /usr/bin/ip link set $1 down
sudo /usr/bin/brctl delif br0 $1
```

```
sudo /usr/bin/ip link delete dev $1
```

- 使用 `visudo` 将以下内容添加到 `sudoers` 文件中:

```
Cmnd_Alias      QEMU=/usr/bin/ip,/usr/bin/modprobe,/usr/bin/brctl
%kvm     ALL=NOPASSWD: QEMU
```

- 您可以使用以下 `run-qemu` 脚本启动QEMU:

```
run-qemu
#!/bin/bash
: '
e.g. with img created via:
qemu-img create -f qcow2 example.img 90G
run-qemu -cdrom archlinux-x86_64.iso -boot order=d -drive file=example.img,format=qcow2
-m 4G -enable-kvm -cpu host -smp 4
run-qemu -drive file=example.img,format=qcow2 -m 4G -enable-kvm -cpu host -smp 4
'

nicbr0() {
    sudo ip link set dev $1 promisc on up &> /dev/null
    sudo ip addr flush dev $1 scope host &>/dev/null
    sudo ip addr flush dev $1 scope site &>/dev/null
    sudo ip addr flush dev $1 scope global &>/dev/null
    sudo ip link set dev $1 master br0 &> /dev/null
}
_nicbr0() {
    sudo ip link set $1 promisc off down &> /dev/null
    sudo ip link set dev $1 nomaster &> /dev/null
}

HASBR0="$( ip link show | grep br0 )"
if [ -z $HASBR0 ] ; then
    ROUTER="192.168.1.1"
    SUBNET="192.168.1."
    NIC=$(ip link show | grep en | grep 'state UP' | head -n 1 | cut -d":" -f 2 |
xargs)
    IPADDR=$(ip addr show | grep -o "inet $SUBNET\([0-9]*\)" | cut -d ' ' -f2)
    sudo ip link add name br0 type bridge &> /dev/null
    sudo ip link set dev br0 up
    sudo ip addr add $IPADDR/24 brd + dev br0
    sudo ip route del default &> /dev/null
    sudo ip route add default via $ROUTER dev br0 onlink
    nicbr0 $NIC
    sudo iptables -I FORWARD -m physdev --physdev-is-bridged -j ACCEPT
fi

USERID=$(whoami)
precreationg=$(ip tuntap list | cut -d: -f1 | sort)
sudo ip tuntap add user $USERID mode tap
postcreation=$(ip tuntap list | cut -d: -f1 | sort)
TAP=$(comm -13 <(echo "$precreationg") <(echo "$postcreation"))
nicbr0 $TAP

printf -v MACADDR "52:54:%02x:%02x:%02x:%02x" $(( $RANDOM & 0xff)) $(( $RANDOM &
0xff )) $(( $RANDOM & 0xff)) $(( $RANDOM & 0xff ))
qemu-system-x86_64 -net nic,macaddr=$MACADDR,model=virtio \
    -net tap,ifname=$TAP,script=no,downscript=no,vhost=on \
    $@

_nicbr0 $TAP
sudo ip link set dev $TAP down &> /dev/null
sudo ip tuntap del $TAP mode tap

if [ -z $HASBR0 ] ; then
    _nicbr0 $NIC
    sudo ip addr del dev br0 $IPADDR/24 &> /dev/null
    sudo ip link set dev br0 down
    sudo ip link delete br0 type bridge &> /dev/null
    sudo ip route del default &> /dev/null
```

```
    sudo ip link set dev $NIC up
    sudo ip route add default via $ROUTER dev $NIC onlink &> /dev/null
fi
```

然后，要启动VM，可以这样做：

```
$ run-qemu -hda myvm.img -m 512
```

- 出于性能和安全原因，建议禁用网桥上的防火墙：

```
/etc/sysctl.d/10-disable-firewall-on-bridge.conf
net.bridge.bridge-nf-call-ip6tables = 0
net.bridge.bridge-nf-call-iptables = 0
net.bridge.bridge-nf-call-arptables = 0
```

In order to apply the parameters described above on boot, you will also need to load the br-netfilter module on boot. Otherwise, the parameters will not exist when sysctl will try to modify them.

```
/etc/modules-load.d/br_netfilter.conf
br_netfilter
```

运行 `sysctl -p /etc/sysctl.d/10-disable-firewall-on-bridge.conf` 立即应用更改。

参见 libvirt wiki 和 Fedora bug 512206。如果在引导过程中 sysctl 报出关于不存在文件的错误，请在引导时加载 `bridge` 模块。参见Kernel module#systemd。

或者，您可以配置 iptables，通过添加类似这样的规则，允许所有流量通过桥进行转发:

```
-I FORWARD -m physdev --physdev-is-bridged -j ACCEPT
```

物理设备和 Tap 设备之间通过 iptables 进行网络共享

**本文或本章节可能需要合并到 Internet_sharing。**

附注： Duplication, not specific to QEMU.（在 Talk:QEMU 中讨论）

桥接网络能在有线接口(例如eth0)之间工作，并且很容易设置。但是，如果主机通过无线设备连接到网络，则无法进行桥接。

参见 Network bridge#Wireless interface on a bridge。

解决这个问题的一种方法是，给tap设备设置一个静态IP，使linux自动处理它的路由，然后通过iptables规则转发tap接口和连接到网络的设备之间的通信。

参见 Internet sharing.

在那里你可以找到在设备之间共享网络所需要的东西，包括tap和tun。下面将进一步介绍所需的一些主机配置。如上所述，需要为静态IP配置客户机，使用分配给tap接口的IP作为网关。需要注意的是，如果客户机上的DNS服务器在从一个连接到网络的主机设备切换到另一个时发生了更改，那么它们可能需要手动编辑。

要在每次启动时允许 IP 转发，需要在 `/etc/sysctl.d` 中，向 sysctl 配置文件添加以下信息:

```
net.ipv4.ip_forward = 1
net.ipv6.conf.default.forwarding = 1
net.ipv6.conf.all.forwarding = 1
```

iptables规则如下：

```
# Forwarding from/to outside
iptables -A FORWARD -i ${INT} -o ${EXT_0} -j ACCEPT
iptables -A FORWARD -i ${INT} -o ${EXT_1} -j ACCEPT
iptables -A FORWARD -i ${INT} -o ${EXT_2} -j ACCEPT
iptables -A FORWARD -i ${EXT_0} -o ${INT} -j ACCEPT
iptables -A FORWARD -i ${EXT_1} -o ${INT} -j ACCEPT
iptables -A FORWARD -i ${EXT_2} -o ${INT} -j ACCEPT
# NAT/Masquerade (network address translation)
iptables -t nat -A POSTROUTING -o ${EXT_0} -j MASQUERADE
iptables -t nat -A POSTROUTING -o ${EXT_1} -j MASQUERADE
iptables -t nat -A POSTROUTING -o ${EXT_2} -j MASQUERADE
```

假设有3个设备连接到一个内部设备的网络共享流量，例如:

```
INT=tap0
EXT_0=eth0
EXT_1=wlan0
EXT_2=tun0
```

前面显示了一个转发，允许与tap设备共享有线和无线连接。

所示的转发规则是无状态的，用于纯转发。可以考虑限制特定的流量，设置防火墙来保护来宾和其他人。然而，这些会降低网络性能，而简单的网桥不包括这些。

好处:不管连接是有线还是无线，如果使用tun设备通过VPN连接到远程站点，假设为该连接打开的tun设备是tun0，并且应用了先前的iptables规则，那么远程连接也将与客户机共享。这避免了客户也需要打开VPN连接。同样，由于来宾网络需要是静态的，因此如果以这种方式远程连接主机，很可能需要编辑来宾网络上的DNS服务器。

通过 VDE2 配置网络

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：This section needs serious cleanup and may contain out-of-date information.（在Talk:QEMU讨论）

何为VDE?

VDE 全称为 Virtual Distributed Ethernet，作为 uml_switch 的一个扩展，是一个用于管理虚拟网络的工具包

其基本的思想是创建一个虚拟的开关，就如插座那样，允许虚拟机和物理机通过"插入"连接彼此。下面的配置非常简单，然而，VDE的功能远比展示的更强大，其能够接入虚拟开关，在不同的主机上运行它们并监听开关上的通信。恳请您阅读该项目的文档获取更多信息。

本方法的优点在于无需sudo特权，普通用户一般没有运行modprobe的权限。

基础操作

可以通过安装 vde2包 包来获得 VDE 支持。

在此处配置中，我们使用tun/tap在主机上创建一块虚拟网卡，用如下命令加载tun模块(参阅Kernel modules获取更多信息)

```
# modprobe tun
```

现在来创建一个虚拟开关：

```
# vde_switch -tap tap0 -daemon -mod 660 -group users
```

这条命令完成了开关创建，tap0创建并将其"插入"，然后允许 users 组内的用户使用之。

该网卡已插入，然而还没有进行配置。需要用下面的命令进行配置：

```
# ip addr add 192.168.100.254/24 dev tap0
```

现在，只需要以普通用户的身份，指定-net参数启动KVM即可：

```
$ qemu-system-x86_64 -net nic -net vde -hda [...]
```

接下来只需同配置物理机网络一般，对客户机网络进行配置就行。

提示：你可能想在tap设备上设置NAT，实现在虚拟机内访问互联网，那就参考Internet sharing#Enable NAT获取更多的帮助吧

启动脚本

启动VDE的一个示例脚本：

```
/etc/systemd/scripts/qemu-network-env
#!/bin/sh
# QEMU/VDE network environment preparation script

# The IP configuration for the tap device that will be used for
# the virtual machine network:

TAP_DEV=tap0
TAP_IP=192.168.100.254
TAP_MASK=24
TAP_NETWORK=192.168.100.0

# Host interface
NIC=eth0

case "$1" in
  start)
        echo -n "Starting VDE network for QEMU: "

        # If you want tun kernel module to be loaded by script uncomment here
       #modprobe tun 2>/dev/null
       ## Wait for the module to be loaded
       #while ! lsmod | grep -q "^tun"; do echo "Waiting for tun device"; sleep 1; done

        # Start tap switch
        vde_switch -tap "$TAP_DEV" -daemon -mod 660 -group users
```

```
        # Bring tap interface up
        ip address add "$TAP_IP"/"$TAP_MASK" dev "$TAP_DEV"
        ip link set "$TAP_DEV" up

        # Start IP Forwarding
        echo "1" > /proc/sys/net/ipv4/ip_forward
        iptables -t nat -A POSTROUTING -s "$TAP_NETWORK"/"$TAP_MASK" -o "$NIC" -j
MASQUERADE
        ;;
  stop)
        echo -n "Stopping VDE network for QEMU: "
        # Delete the NAT rules
        iptables -t nat -D POSTROUTING -s "$TAP_NETWORK"/"$TAP_MASK" -o "$NIC" -j
MASQUERADE

        # Bring tap interface down
        ip link set "$TAP_DEV" down

        # Kill VDE switch
        pgrep vde_switch | xargs kill -TERM
        ;;
  restart|reload)
        $0 stop
        sleep 1
        $0 start
        ;;
  *)
        echo "Usage: $0 {start|stop|restart|reload}"
        exit 1
esac
exit 0
```

使用上面的脚本作为 systemd 服务：

```
/etc/systemd/system/qemu-network-env.service
[Unit]
Description=Manage VDE Switch

[Service]
Type=oneshot
ExecStart=/etc/systemd/scripts/qemu-network-env start
ExecStop=/etc/systemd/scripts/qemu-network-env stop
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

修改 `qemu-network-env` 权限至可执行。

现在可以像其他服务一样，启动 `qemu-network-env.service` 服务。

备用方法

如果上面的方法不起作用，或者说不想折腾内核配置、TUN、dnsmasq 和 iptables，可以采用下面的方法，也可以获得同样的效果，

```
# vde_switch -daemon -mod 660 -group users
# slirpvde --dhcp --daemon
```

然后，启动 VM 并连接至主机网络中：

```
$ qemu-system-x86_64 -net nic,macaddr=52:54:00:00:EE:03 -net vde disk_image
```

VDE2 网桥

根据 quickhowto: qemu networking using vde, tun/tap, and bridge 所描述的. 任何连接到 vde 上的虚拟机都会暴露给外部。举个例子，每台虚拟机都能直接从 ADSL 路由器那收到 DHCP 的配置信息。

基础操作

首先，你需要有 `tun` 模块并安装了 bridge-utils包 包。

创建一个 vde2/tap 设备：

```
# vde_switch -tap tap0 -daemon -mod 660 -group users
# ip link set tap0 up
```

创建一个网桥:

```
# brctl addbr br0
```

添加设备:

```
# brctl addif br0 eth0
# brctl addif br0 tap0
```

配置网桥接口:

```
# dhcpcd br0
```

启动脚本

所有设备应该都设置好了，且只有网桥拥有IP地址。对于网桥上的物理设备(比如 `eth0`)，可以通过netctl使用一个自定义的配置文件完成上面所述设置：

```
/etc/netctl/ethernet-noip
Description='A more versatile static Ethernet connection'
Interface=eth0
Connection=ethernet
IP=no
```

下面的systemd服务用于为`users`用户组内的成员创建和启动VDE2 tap网卡。

```
/etc/systemd/system/vde2@.service
[Unit]
Description=Network Connectivity for %i
Wants=network.target
Before=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/vde_switch -tap %i -daemon -mod 660 -group users
ExecStart=/usr/bin/ip link set dev %i up
ExecStop=/usr/bin/ip addr flush dev %i
ExecStop=/usr/bin/ip link set dev %i down

[Install]
WantedBy=multi-user.target
```

最后，通过 netctl 创建网桥。

简化配置参数

如果你经常需要以不同的网络配置选项运行QEMU，就会发现时常得输入大量的 `-netdev` 和 `-device` 选项组合，这些需要大量重复性的劳动。可以用 `-nic` 选项将二者结合，就如下面这样：

```
-netdev tap,id=network0,ifname=tap0,script=no,downscript=no,vhost=on -device virtio-net-pci,netdev=network0
```

可简化为:

```
-nic tap,script=no,downscript=no,vhost=on,model=virtio-net-pci
```

要注意的是缺失了网络ID，因此将会以`model=`创建这些设备。{ic|-nic}}命令的前半部分参数正是`-netdev`的参数，而后半部分参数（`model=`之后的部分）则与设备有关，原本设备所提供的参数同样可以在此使用（例如，可以指定`smb=`）。若要完全禁用网络，可以用`-nic none`。

参阅 QEMU networking文档了解更多的相关参数。

图形

使用 `-display curses` 命令行选项，QEMU 可以模拟一个标准显卡文本模式。这允许用户可以直接在文本终端中直接输入文本或看见文本输出。 `-nographic` 选项起到了类似的作用。

QEMU 可以根据 `-vga *type*` 参数模拟多种不同的显卡，可用的选项为 `std`,`qxl`,`vmware`,`virtio`,`cirrus` 或 `none`。

std

使用 `-vga std` 你可以得到最高 2560 x 1600 像素的分辨率。从 QEMU 2.2 开始是默认选项。

qxl

QXL是一个支持 2D 的并行虚拟化图形驱动。需要在客户机中安装驱动并在启动 QEMU 时设置 `-vga qxl` 选项。你可能也会想使用 #SPICE 优化 QXL 的图形表现。

在Linux客户机中，需要加载 `qxl` 和 `bochs_drm` 这两个内核模块，以获得一个比较好的效果。

QXL 设备的默认 VGA 内存大小为 16M，这样的内存大小最高支持 QHD (2560x1440) 的分辨率，如果想要一个更高的分辨率，请增加vga_memmb。

vmware

尽管Bug有点多，但相比于std和cirrus它的表现会更好。对于Arch Linux客户机来说可以安装 xf86-video-vmware包 和 xf86-input-vmmouse包 获取VMware驱动。

virtio

`virtio-vga` / `virtio-gpu` 是一个基于virgl的3D并行虚拟化图形驱动。目前它已经成熟；目前它仅支持Linux客户机，且需要以`gallium-drivers=virgl`选项编译mesa包。

若要在客户机上启用3D加速，那么需要用 `-device virtio-vga-gl` 选项选择此vga，并用 `-display sdl,gl=on` 或 `-display gtk,gl=on` 在显示设备上启用OpenGL 上下文，这两个选项分别适用于 SDL 输出和 GTK 输出。如果配置成功了，那么在客户机的 kernel log 里可以看到：

```
# dmesg | grep drm
[drm] pci: virtio-vga detected
[drm] virgl 3d acceleration enabled
```

cirrus

cirrus 是2.2之前默认的图形选项，不应当在现代操作系统中使用它。

none

这就像一台完全没有 VGA 卡的 PC，无法通过 `-vnc` 访问它。另外，这种情况与使用 `-nographic` 选项不同，`-nographic` 会让 QEMU 模拟 VGA 卡，只是关闭了 SDL 输出。

SPICE

SPICE project 旨在为用户提供一种完全开源的方式，无缝地对虚拟机进行远程访问。

在宿主机上启用 SPICE 支持

下面是一个启用SPICE作为远程桌面协议的例子，并支持复制和粘贴操作：

```
$ qemu-system-x86_64 -vga qxl -device virtio-serial-pci -spice port=5930,disable-ticketing=on -device virtserialport,chardev=spicechannel0,name=com.redhat.spice.0 -chardev spicevmc,id=spicechannel0,name=vdagent
```

这些参数的含义如下：

1. `-device virtio-serial-pci` 添加一块 virtio-serial 设备
2. `-spice port=5930,disable-ticketing` 在 TCP `5930` 端口上进行 spice 频道的监听，允许客户端不经验证即可连接。

提示：使用 Unix 套接字 而非 TCP 端口不会涉及宿主机系统的网络栈，也不意味着可以对数据包进行封装和解封以使用网络和其它相关的协议。这些套接字仅通过硬盘上的 inode 进行表示，这么做是出于性能上的考虑。Unix 套接字可以使用 `-spice unix=on,addr=/tmp/vm_spice.socket,disable-ticketing=on` 代替上述参数。

1. `-device virtserialport,chardev=spicechannel0,name=com.redhat.spice.0` 在 virtio-serial 设备上为 spice vdagent 打开一个端口。

2. `-chardev spicevmc,id=spicechannel0,name=vdagent` 为该端口添加一块 spicevmc 字符设备。`virtserialport` 的 `chardev=` 选项需要与 `chardev` 的 `id=` 相符，在本例中是 `spicechannel0`。将端口名字设置为 `com.redhat.spice.0` 也很重要，因为它就是 vdagent 在客户机上所搜索的命名空间。最后，指定 `name=vdagent` 选项使得 spice 知道该频道的服务对象。

通过 SPICE 客户端连接到客户机

若要连接到客户机上必须要有一个 SPICE 客户端。在 Arch 中，有如下可用的客户端：

- **virt-viewer** — 协议开发者所推荐的 SPICE 客户端，是 virt-manager 项目的子集。

: https://virt-manager.org/ || virt-viewer包

- **spice-gtk** — SPICE GTK 客户端，SPICE 项目的一个子集，作为小部件嵌入其它应用中。

: https://www.spice-space.org/ || spice-gtk包

若需要能在智能手机上运行的客户端，或者其他平台的客户端，参照 spice-space download 中的 *Other clients* 章节。

手动开启 SPICE 客户端

连接到一个监听在 Unix 套接字 `/tmp/vm_spice.socket` 上的客户机的方法是用 `$ remote-viewer spice+unix:///tmp/vm_spice.socket` 或 `$ spicy --uri="spice+unix:///tmp/vm_spice.socket"` 命令手动运行 SPICE 客户端，使用的客户端取决于你的喜好。SPICE 模式下的 QEMU 就如一个远程桌面服务器，可能使用 `-daemonize` 参数以daemon 模式运行 QEMU 会更方便一些。

提示：可以用下面的命令，通过 SSH 隧道连接到客户机：

```
$ ssh -fL 5999:localhost:5930 my.domain.org sleep 10; spicy -h 127.0.0.1 -p 5999
```

。这个例子中 *spicy* 连接到了本地的 `5999` 端口，该端口通过 SSH 转发至 *my.domain.org* 上的 SPICE 服务器端口 `5930`。要注意 `-f` 选项让 ssh 在后台执行了 `sleep 10` 命令。这种情况下，在客户端存活时 ssh 会话保持运行，在客户端结束运行时将自动关闭 ssh 会话。

QEMU 运行时启动SPICE

QEMU 可以自动地开启一个 SPICE 客户端并创建一个合适的套接字。如果通过 `-display spice-app` 参数将显示设置为SPICE，那么将会使用系统默认的 SPICE 客户端作为 viewer，默认的客户端取决于你的mimeapps.list文件。

在客户机上开启SPICE的支持

对于**Arch Linux客户机**，如要支持多屏和共享剪贴板，需要安装以下的包：

- spice-vdagent包: Spice的xorg客户端代理，使得用户能够在客户端和X-session之间进行复制和粘贴等。(Refer to this issue, until fixed, for workarounds to get this to work on non-GNOME desktops.)
- xf86-video-qxl包: Xorg X11 qxl视频驱动
- x-resizeAUR: 除 GNOME 之外的桌面环境在 SPICE 客户端窗口大小调整时不会自动反应。此软件包使用 udev 规则和 xrandr 为所有基于 X11 的桌面环境和窗口管理器实现自动调整大小。

对于**其它的操作系统**客户机, 参照spice-space download的*Guest*章节。

开启 SPICE 口令验证

若要启用 SPICE 的口令验证，需要从 `-spice` 的参数中移除 `disable-ticketing`，改为 `password=*yourpassword*`，例如：

```
$ qemu-system-x86_64 -vga qxl -spice port=5900,password=yourpassword -device virtio-serial-pci -device virtserialport,chardev=spicechannel0,name=com.redhat.spice.0 -chardev spicevmc,id=spicechannel0,name=vdagent
```

现在 SPICE 客户端在连接 SPICE 服务器的时候应该就会进行口令询问了。

用 TLS 对与 SPICE 的通信进行加密

同样可以为客户端与 SPICE 服务器的通信配置 TLS 加密。首先，需要有一个包含如下文件的目录（文件名必须与下面保持一致）：

- `ca-cert.pem`: CA 主证书。
- `server-cert.pem`: `ca-cert.pem` 签名后的服务器证书。
- `server-key.pem`: 服务器的私钥。

Spice User Manual还展示了使用服务器自己生成的CA生成自签名证书的例子。

这些完成后，你可以用前文描述的 `-spice` 的参数在 QEMU 启动时自动开启 SPICE：`-spice tls-port=5901,password=*yourpassword*,x509-dir=*/path/to/pki_certs*`，其中 `*/path/to/pki_certs*` 代表包含那三个文件的目录。

现在可以用 virt-viewer包 连接到服务器了：

```
$ remote-viewer spice://hostname?tls-port=5901 --spice-ca-file=/path/to/ca-cert.pem --spice-host-subject="C=XX,L=city,O=organization,CN=hostname" --spice-secure-channels=all
```

要记住的是，`--spice-host-subject` 参数需要根据你的 `server-cert.pem` 中的子条目进行设置。此外还需要将 `ca-cert.pem` 复制到每个客户端上用于验证服务器证书。

提示：使用下面的命令，就可以获得服务器证书中的子条目格式，可用于 `--spice-host-subject` 参数中（以逗号进行分隔）：

```
$ openssl x509 -noout -subject -in server-cert.pem | cut -d' ' -f2- | sed 's/\///' | sed 's/\//,/g'
```

等效的 spice-gtk包 命令为:

```
$ spicy -h hostname -s 5901 --spice-ca-file=ca-cert.pem --spice-host-subject="C=XX,L=city,O=organization,CN=hostname" --spice-secure-channels=all
```

VNC

可以用`-vnc :*X*`选项将QEMU的VGA输出重定向至VNC会话中。将`*X*`替换为输出目标的编号（0代表之后监听在5900，1代表监听在5901...）。

```
$ qemu-system-x86_64 -vnc :0
```

在#开机时启动QEMU虚拟机这一小节中同样提供了一个VNC的示例。

警告：默认的 VNC 服务器没有使用任何验证手段，用户可以从任何主机上连接到 VNC。

基本的口令验证

可以通过使用`password`选项很容易地设置访问口令。必须在QEMU Monitor中指定口令，仅当用户提供口令时才有可能连接到VNC。

```
$ qemu-system-x86_64 -vnc :0,password -monitor stdio
```

在QEMU Monitor中设置口令需使用 `change vnc password` 命令，然后指定一个口令。

底下的命令将在启动 VNC 时直接为其设置口令：

```
$ printf "change vnc password\n%s\n" MYPASSWORD | qemu-system-x86_64 -vnc :0,password -monitor stdio
```

注意：口令被限制在8个字符内，可以用暴力破解的方式猜到口令。因此在公网上推荐使用更细致的保护措施。

音频

创建音频后端

`-audiodev` 标识用于设定后端音频驱动及其相关选项。

To list availabe audio backend drivers:

```
$ qemu-system-x86_64 -audiodev help
```

在 qemu(1)man 页面中详细列出了可用的后端音频驱动以及可选的设置项。

最简单的情况下，你需要选择一个驱动并设置一个 id。以 PulseAudio 为例：

```
-audiodev pa,id=snd0
```

使用音频后端

Intel HD Audio

模拟 Intel HD Audio 需要添加控制器和编解码器设备。可以用如下命令列出可用的 Intel HDA Audio 设备：

```
$ qemu-system-x86_64 -device help | grep hda
```

添加音频控制器：

```
-device ich9-intel-hda
```

添加音频编解码器并将其映射到宿主机的音频后端 id 上。

```
-device hda-output,audiodev=snd0
```

Intel 82801AA AC97

模拟 AC97 需要添加声卡设备并将其映射到宿主机的一个音频后端 id 上。

```
-device AC97,audiodev=snd0
```

注意：

- 如果没有提供 audiodev 后端，QEMU 会自动查找并添加它，这只适用于单个 audiodev。例如，`-device intel-hda -device hda-duplex` 将使用默认的 audiodev 后端在客户机上模拟 `intel-hda`。
- 客户机的显卡模拟驱动也可能导致声音质量的问题，需要逐一测试以使其正常工作。你可以用 `qemu-system-x86_64 -h | grep vga` 列出可能的选项。

VirtIO sound

自 QEMU 8.2.0 版本开始，VirtIO 声卡也可用。使用方式如下：

```
-device virtio-sound-pci,audiodev=my_audiodev -audiodev alsa,id=my_audiodev
```

更多信息可以在 QEMU 文档中找到。

安装 virtio 驱动

QEMU 为用户提供并行虚拟化块设备和网络设备的能力，其是借助 virtio 驱动实现的，拥有更好的性能表现以及更低的开销。

- virtio块设备需要使用 `-drive` 指定一个 disk image 的参数，且需要带上 `if=virtio`参数：

```
$ qemu-system-x86_64 -drive file=disk_image,if=virtio
```

- 网络配置也是类似的：

```
$ qemu-system-x86_64 -nic user,model=virtio-net-pci
```

注意：仅有当客户机有 virtio 设备对应的驱动时该方法才能起效，Linux 是有这方面支持的，而在 Arch Linux 中已经包含所需的驱动了，不过无法保证这些驱动能够兼容其他操作系统。

Arch Linux 客户机

要在Arch Linux客户机中使用virtio设备，必须在客户端加载以下模块：{`virtio`、`virtio_pci`、`virtio_blk`、`virtio_net`、`virtio_ring`。对于32位系统来说，不需要特定的“virtio”模块。

如果希望从 virtio 磁盘引导，ramdisk 必须包含必要的模块。默认情况下，这是由 mkinitcpio 的 `autodetect` 钩子处理的。否则要在 `/etc/mkinitcpio.conf` 中使用 `MODULES` 数组包含必要的模块并重新构建 ramdisk。

```
/etc/mkinitcpio.conf
MODULES=(virtio virtio_blk virtio_pci virtio_net)
```

Virtio 磁盘公认的前缀为 `**v**`（例如：`**v**da`，`**v**db`，等等）。因此，当从 virtio 磁盘启动时，需要在 `/etc/fstab` 和 `/boot/grub/grub.cfg` 中进行更改。

提示：当在 `/etc/fstab` 和引导加载程序中通过 UUID 引用磁盘时，不需要执行任何操作。

关于使用KVM进行半虚拟化的更多信息，可以参考这里。

您可以安装 qemu-guest-agent包 来实现对 QMP 命令的支持，从而增强管理程序的管理能力。安装完成后，您需要启动`qemu-guest-agent.service`。

Windows 客户机

Windows 的 Virtio 驱动

Windows 系统不自带 virtio 驱动程序。最新和稳定版本的驱动程序由 Fedora 定期构建，有关下载驱动程序的详细信息可以在 GitHub 上的 virtio-win 项目中找到。 在以下部分中，我们将主要使用由 virtio-win.iso 提供的稳定 ISO 文件。或者，可以使用 virtio-winAUR。

块设备驱动

安装一个新的Windows

需要在系统安装时通过 ISO 加载驱动，具体操作是在主磁盘设备和 Windows 安装盘外挂载一个额外的 cdrom 设备，将系统镜像与 virtio 驱动一同加载：

```
$ qemu-system-x86_64 ... \
-drive file=disk_image,index=0,media=disk,if=virtio \
-drive file=windows.iso,index=2,media=cdrom \
-drive file=virtio-win.iso,index=3,media=cdrom \
...
```

在安装过程中，Windows Installer 会询问你“Where do you want to install Windows?”，其会返回一个警告表示没有找到任何磁盘设备。接下来跟着如下示例中的步骤进行操作（基于Windows Server 2012 R2 with Update）：

- 选择选项*加载驱动*

- *反选隐藏与这个计算机硬件不兼容的驱动*
- 按 浏览 按钮并选择 virtio iso 的 CDROM ，一般命名为 "virtio-win-XX"
- 然后浏览 `E:\viostor\[your-os]\amd64` ，选择它并确认

现在应该能看到 virtio 磁盘出现在列表中了，等待被选中、格式化并安装。

将现有的 Windows VM 转为使用 virtio

可将现有的 Windows 客户机改为从 virtio 磁盘中启动，前提是客户机中有 virtio 驱动，且该驱动在启动期间就被加载。

然后我们需要让 Windows 在 Windows 能以 virtio 模式启动一个磁盘镜像之前，在启动时加载 virtio 驱动。

先创建一个新的磁盘镜像，用于搜索 virtio 驱动：

```
$ qemu-img create -f qcow2 fake.qcow2 1G
```

挂载 fake 磁盘（处于virtio模式下）和带有驱动的CD-ROM，运行原本的Windows客户机（启动盘依旧是处于IDE模式中）：

```
$ qemu-system-x86_64 -m 4G -drive file=disk_image,if=ide -drive
file=fake.qcow2,if=virtio -cdrom virtio-win.iso
```

Windows会自动检测fake磁盘，并搜索适配的驱动。如果失败了，前往*Device Manager*，找到SCSI驱动器（带有感叹号图标，应处于打开状态），点击*Update driver*并选择虚拟的CD-ROM。不要定位到CD-ROM内的文件夹了，只选择CD-ROM设备就行，Windows会自动找到合适的驱动的。（已在Windows 7 SP1中完成测试）。

可以让Windows从下一次起为以安全模式启动，可以用Windows上的*msconfig.exe*工具完成该配置，在安全模式中所有的驱动都会在boot期间被加载，包括我们新装的virtio驱动。只要Windows知道boot期间需要加载virtio，在未来的boot过程中Windows都会记住该设置的。

将客户机配置为安全模式启动后，可以关机并重新启动它，现在可以以 virtio 模式挂载 boot 磁盘：

```
$ qemu-system-x86_64 -m 4G -drive file=disk_image,if=virtio
```

现在应当能携带 virtio 驱动启动到安全模式中，你可以返回到 *msconfig.exe* 中禁用安全模式并重启 Windows。

注意：如果使用`if=virtio`参数时碰上了蓝屏问题，这代表virtio磁盘驱动可能没有在boot期间被安装，以安全模式重启之，然后检查你的驱动配置。

网络驱动

安装virtio网络驱动程序要容易一些，只需如上所述添加`-net`参数即可。

```
$ qemu-system-i386 -m 4G -vga std -drive file=windows_disk_image,if=virtio -net
nic,model=virtio -cdrom virtio-win-0.1-185.iso
```

Windows将检测网络适配器并尝试为其找到驱动程序。如果失败，请转到“设备管理器”，找到带有感叹号图标的网络适配器（双击打开），切换到驱动程序并单击“更新驱动程序”，然后选择虚拟CD-ROM。别忘了选中显示要递归搜索目录的复选框。

Balloon 驱动

如果想要追踪客户机内存状态（比如通过`virsh`的`dommemstat`命令）或者在运行时改变客户机内存大小（尽管依然无法改变实际的内存大小，不过可以通过inflating balloon驱动限制内存的使用），那么请在客户机上安装balloon驱动吧。

安装该驱动需要前往*Device Manager*，其位于*System devices*内的*PCI standard RAM Controller*中（未识别的PCI控制设备则在*Other devices*中）。选择*Update driver*，在打开的窗口中选择*Browse my computer...*，然后选择CD-ROM（记得勾上*Include subdirectories*选项），安装完成后重启，驱动就安装成功了。现在可以如气球打气一般调整内存限制（例如通过hmp命令`balloon *memory_size*`，该命令会使balloon从客户机中尽可能地夺取内存，将客户机的内存大小限制至*memory_size*）。然而，现在依然无法追踪内存的状态，我们还需要正确地安装*Balloon*服务才行。以管理员身份启动命令行，前往CD-ROM中的*Balloon*目录，然后在目录下找到对应系统和架构的地方。当深入至 *amd64* (*x86*) 目录时，运行`blnsrv.exe -i`进行安装，安装之后`virsh`命令`dommemstat`就会输出命令所支持的各种数据。

FreeBSD客户机

如果你使用的是FreeBSD 8.3之后10.0-CURRENT之间的版本， 那么无论内核中是否包含了virtio-kmod， 都需要安装`emulators/virtio-kmod`端口。安装之后请在`/boot/loader.conf`添加如下内容：

```
virtio_load="YES"
virtio_pci_load="YES"
virtio_blk_load="YES"
if_vtnet_load="YES"
virtio_balloon_load="YES"
```

按照下面修改 `/etc/fstab`：

```
# sed -ibak "s/ada/vtbd/g" /etc/fstab
```

注意确认 `/etc/fstab` 内容与该命令的预期效果相同，如果出错了就通过 rescue CD 启动，然后将 `/etc/fstab.bak` 复制到 `/etc/fstab`。

QEMU 监视器

QEMU运行时会提供一个监视器 console 界面以方便用户同虚拟机进行交互。QEMU 监视器提供了许多有趣的功能，例如获取当前虚拟机的信息，热插拔设备，创建快照等。在 QEMU 监视器 console 中运行 `help` 或 `?` 命令，或者阅读官方 QEMU 文档获得完整的命令列表。

访问 QEMU 监视器 Console

图形化界面

当使用默认的 `std` 图形选项时，可以通过按下 `Ctrl+Alt+2` 组合键或从QEMU窗口上的 *View* > *compatmonitor0* 访问到 QEMU 监视器。若要返回到虚拟机的图形界面，那么按下 `Ctrl+Alt+1` 或者 *View* > *VGA* 就行。

然而，这种标准的访问方式不够方便，而且并不是在QEMU的所有图形化输出方式中都适用。

Telnet

启动 QEMU 时带上 `-monitor telnet:127.0.0.1:*port*,server,nowait` 参数可以启用 telnet。虚拟机启动后可以通过 telnet 访问到监视器：

```
$ telnet 127.0.0.1 port
```

注意：如果指定 `127.0.0.1` 作为监听地址，那么只能在运行 QEMU 的宿主机上连接到该监视器。如果想要远程访问，QEMU需要在 `0.0.0.0` 上进行监听：`-monitor telnet:0.0.0.0:*port*,server,nowait`。还要记住的是，最好对防火墙进行配置，该连接是完全不进行认证和加密的，因此需要通过防火墙确保本地网络环境是可信的。

UNIX 套接字

通过 `-monitor unix:*socketfile*,server,nowait` 参数运行QEMU，之后就可以通过 socat包、nmap包 或 openbsd-netcat包 连接到监视器上。

例如，如果QEMU是通过如下命令启动：

```
$ qemu-system-x86_64 -monitor unix:/tmp/monitor.sock,server,nowait [...]
```

就可以像这样连接到监视器上：

```
$ socat - UNIX-CONNECT:/tmp/monitor.sock
```

或者通过这种方式:

```
$ nc -U /tmp/monitor.sock
```

也可以使用 nmap包：

```
$ ncat -U /tmp/monitor.sock
```

TCP

可以使用 `-monitor tcp:127.0.0.1:*port*,server,nowait` 参数将监视器暴露于TCP端口上，然后用 netcat（openbsd-netcat包 或 gnu-netcat包 都可以）进行连接：

```
$ nc 127.0.0.1 port
```

注意：为了能够从其它设备上通过 TCP 套接字访问到监视器，而不仅仅从运行QEMU的主机上连接，需要像前面Telnet中描述的那样，在`0.0.0.0`地址上进行监听。

标准 I/O

如果以 `-monitor stdio` 参数运行QEMU，那么其实是可以在运行 QEMU 的终端下访问到监视器的。

在 Monitor conosle 下向虚拟机发送按键行为

由于在某些配置下，宿主机可能会拦截一些按键组合另作他用，这导致要在虚拟机中触发一些特定按键组合变得有些困难（一个显然的例子就是`Ctrl+Alt+F*`组合，该组合用于改变当前的tty）。我们采用在monitor console下发送按键组合的方式解决该问题。只需切换到monitor console下，然后使用`sendkey`命令，即可将按键转发至虚拟机中，例如：

```
(qemu) sendkey ctrl-alt-f2
```

通过 monitor console 创建快照和管理快照

注意：该特性只支持 *qcow2* 格式的虚拟机磁盘镜像，对于 *raw* 是无效的。

有时候我们很需要将虚拟机的当前状态进行保存，或是将虚拟机重置到之前的快照状态，而且最好是随时能进行这些操作。QEMU monitor console为用户提供了必要的功能，进行快照创建，快照管理，以及快照恢复。

- Use `savevm *name*` 用于创建一个名为*name*的快照。
- Use `loadvm *name*` 用于将虚拟机状态恢复至快照*name*。
- Use `delvm *name*` 用于删除快照*name*。
- Use `info snapshots` 用于查看保存的快照列表，这些快照由一个自增长的ID和标签名（用户创建快照时赋予）进行标识。

以冻结模式运行虚拟机

QEMU支持以冻结态运行虚拟机（需使用`-snapshot`参数），换句话说，虚拟机关闭时，对于虚拟机的一切修改都会丢弃。当用户对磁盘镜像写入时，这些变动最终写入的位置是`/tmp`目录下的一个临时文件，QEMU关机时将会把他们丢弃。

不过，即使虚拟机运行于冻结状态下，依旧可以通过monitor console将这些变化写入磁盘镜像（如果你想的话）。使用下面的命令：

```
(qemu) commit all
```

另外如果在冻结状态下创建快照，这些快照在QEMU退出时都会被丢弃，除非你显式地commit了他们。

monitor console 中的开机和暂停命令

在QEMU monitor console下也可以模拟对物理机的一些操作：

- `system_powerdown` 会向虚拟机发送ACPI关机信号，效果就类似物理机上按下电源按钮。
- `system_reset` 会重置虚拟机，类似物理机上的重置按钮。该操作可能导致数据丢失或文件系统的损坏，这是因为虚拟机并不是"干净地"重启的。
- `stop` 会暂停虚拟机。
- `cont` 使暂停的虚拟机恢复运行。

虚拟机截屏

可以在monitor console下运行该命令，获取PPM格式的截屏图片：

```
(qemu) screendump file.ppm
```

QEMU 机器协议

QEMU机器协议（QMP）是一个基于JSON格式的协议，使得其他应用程序可以通过该协议控制QEMU实例。类似#QEMU 监视器，其提供了与运行中的虚拟机进行交互的能力，且能够编程进行控制。关于QMP各命令的描述可以在这个qmp-commands链接中找到。

启动 QMP

使用QMP协议来控制虚拟机的通常做法是在启动QEMU时使用`-qmp`打开一个 TCP 套接字。底下是一个使用 TCP 4444 端口的例子：

```
$ qemu-system-x86_64 [...] -qmp tcp:localhost:4444,server,nowait
```

而与QMP代理进行通信的一个选择是使用 netcat:

```
nc localhost 4444
{"QMP": {"version": {"qemu": {"micro": 0, "minor": 1, "major": 3}, "package": ""},
"capabilities": []} }
```

在目前这个阶段，其能识别的命令仅有`qmp_capabilities`，QMP将进入了命令模式。敲下：

```
{"execute": "qmp_capabilities"}
```

现在，QMP可以接收命令了。要查看QMP接受的命令列表，使用：

```
{"execute": "query-commands"}
```

即时将子镜像合并至父镜像中

通过发起一个`block-commit`可以将一个正处于运行态的快照合并到其父结点上。下面的例子是其最简单的一种形式，将把子镜像提交至父镜像中：

```
{"execute": "block-commit", "arguments": {"device": "devicename"}}
```

QMP收到该命令后，处理程序将会寻找基镜像，把该镜像由只读转为可读写模式，然后完成commit的任务。

一旦*block-commit*操作完成，将会触发一个`BLOCK_JOB_READY`事件，发出同步完成的信号。当然也可以用一种更优雅的方式完成该操作，改为使用`block-job-complete`：

```
{"execute": "block-job-complete", "arguments": {"device": "devicename"}}
```

在发出该命令之前，*commit*操作将会保持活动状态。 任务完成后，基镜像将会保持在可读写模式，并变为active层。另一方面，子镜像将变得不可使用，用户有责任将其清除。

提示：执行`query-block`命令并对输出进行解析可以获取设备列表和其对应名字。设备名在 `device` 字段中，例如本例中磁盘设备名就是 `ide0-hd0`：

```
{"execute": "query-block"}
{"return": [{"io-status": "ok", "device": "ide0-hd0", "locked": false, "removable":
false, "inserted": {"iops_rd": 0, "detect_zeroes": "off", "image": {"backing-image":
{"virtual-size": 27074281472, "filename": "parent.qcow2", ... }
```

即时创建一个新的快照

为运行中的镜像创建一个快照：

```
{"execute": "blockdev-snapshot-sync", "arguments": {"device": "devicename","snapshot-
file": "new_snapshot_name.qcow2"}}
```

该命令将会创建一个名为 `*new_snapshot_name*.qcow2` 的堆叠文件，该快照也将成为新的活动层。

技巧

改善虚拟机的性能表现

底下是一些可以改善虚拟机性能表现的技术，例如：

- 启用#启用 KVM：QEMU的启动命令加上 `-enable-kvm` 选项。
- 通过 `-cpu host` 选项让 QEMU 模拟宿主机上的特定 CPU，如果没有该选项 QEMU 尝试模拟的是一个更为通用的 CPU。
- 特别的，如果客户机是 Windows，启用Hyper-V enlightenments可以改善性能：`-cpu`

`host,hv_relaxed,hv_spinlocks=0x1fff,hv_vapic,hv_time`.有关更多信息和标志，请参阅 QEMU 文档。

- 可以使用 `-smp cores=x,threads=y,sockets=1,maxcpus=z` 选项为虚拟机分配多个核心。threads 参数用于分配 SMT 核心。为 QEMU、管理程序和主机系统保留一个物理核心不受干扰是非常有益的。
- 检查是否为虚拟机分配的足够的内存。默认情况下，QEMU仅仅为每台虚拟机分配 128MiB 的内存，可以使用 `-m` 选项分配更多的内存。例如，`-m 1024` 代表启动一台内存为 1024MiB 的虚拟机，`-m 4G`则为4GiB的内存。
- 如果客户机操作系统支持相关的驱动，可以使用virtio创建网络设备或块设备，详情可参考 #Installing virtio drivers。
- 使用 TAP 设备代替 user-mode 网络，参阅#Tap 网络。
- 如果客户机需要进行大量的磁盘写工作，在宿主机文件系统上设置合适的挂载选项可以优化该工作。例如，可以用`barrier=0`选项挂载一个 Ext4 文件系统。在使用这些性能强化选项之前最好阅读相关文档，因为性能上的提升通常伴随着数据完整性下降的代价。
- 如果有一块原始磁盘或分区，你可能会想要禁用缓存：

```
$ qemu-system-x86_64 -drive file=/dev/disk,if=virtio,cache=none
```

- 使用原生的Linux AIO：

```
$ qemu-system-x86_64 -drive file=disk_image,if=virtio,aio=native,cache.direct=on
```

- 如果正同时运行多台虚拟机，而它们拥有同样的操作系统，可以通过启用内核页归并)节省内存。参阅#开启 KSM。
- 在一些情况下，可以在运行时从安装了 balloon 驱动的客户机上回收内存，这需要 QEMU 启动该客户机时使用`-device virtio-balloon`选项。
- 尽管不是很稳定，可以使用 ICH-9 AHCI 控制器的仿真层。AHCI 的仿真模拟支持 NCQ，因此可以同时处理多个读写请求：

```
$ qemu-system-x86_64 -drive id=disk,file=disk_image,if=none -device ich9-ahci,id=ahci -device ide-drive,drive=disk,bus=ahci.0
```

参阅 https://www.linux-kvm.org/page/Tuning_KVM 获取更多信息。

开机时启动QEMU虚拟机

通过 libvirt 实现

如果虚拟机是通过 libvirt 设置的，可以用 `virsh autostart` 将其配置为开机自启，或者通过 *virt-manager* GUI 中虚拟机的 Boot Options，选择 "Start virtual machine on host boot up" 实现开机自启。

通过 systemd 服务实现

可以用如下的 systemd unit 和 config 配置开机时启动 QEMU VM。

```
/etc/systemd/system/qemu@.service
[Unit]
Description=QEMU virtual machine

[Service]
Environment="haltcmd=kill -INT $MAINPID"
EnvironmentFile=/etc/conf.d/qemu.d/%i
ExecStart=/usr/bin/qemu-system-x86_64 -name %i -enable-kvm -m 512 -nographic $args
ExecStop=/usr/bin/bash -c ${haltcmd}
ExecStop=/usr/bin/bash -c 'while nc localhost 7100; do sleep 1; done'

[Install]
WantedBy=multi-user.target
```

注意：为了方便地结束任务，该 service 会等待至 console 端口被释放（这意味着 VM 已被关闭）。

接着按单个 VM 创建配置文件，命名为 `/etc/conf.d/qemu.d/*VM 名称*`，在其中设置好 `args` 和 `haltcmd` 变量，配置示例：

```
/etc/conf.d/qemu.d/one
args="-hda /dev/vg0/vm1 -serial telnet:localhost:7000,server,nowait,nodelay \
 -monitor telnet:localhost:7100,server,nowait,nodelay -vnc :0"

haltcmd="echo 'system_powerdown' | nc localhost 7100" # or netcat/ncat
/etc/conf.d/qemu.d/two
args="-hda /srv/kvm/vm2 -serial telnet:localhost:7001,server,nowait,nodelay -vnc :1"

haltcmd="ssh powermanager@vm2 sudo poweroff"
```

对该变量的描述如下：

- `args` - 使用的 QEMU 命令行参数。
- `haltcmd` - 安全关闭虚拟机的命令，在第一个例子中，QEMU monitor 是通过 `-monitor telnet:..` 选项暴露至 telnet，因而关闭虚拟机是通过 `nc` 命令在 monitor console 中发送 `system_powerdown`，完成 ACPI 关机的工作。在另一个例子里，使用的则是 SSH。

若要设置启动时运行哪个虚拟机，启用 `qemu@*vm_name*.service` 这个 systemd 单元。

鼠标整合

添加 `-usb -device usb-tablet` 选项以避免点击客户机系统的窗口时鼠标被捕获。该选项代表 QEMU 能够在不捕获鼠标的情况下，向系统报告鼠标的位置，该选项启用时还会覆盖 PS/2 鼠标模拟功能。例如：

```
$ qemu-system-x86_64 -hda disk_image -m 512 -usb -device usb-tablet
```

如果该命令不起作用，试试 `-vga qxl` 参数，并看看#鼠标指针抖动或者不稳定的操作指导。

宿主机的USB设备传递至虚拟机

从客户机访问连接到宿主机USB口的设备是可能的，首先需要识别设备连接的位置，可以用`lsusb`命令找到设备连接位置，例如：

```
$ lsusb
...
Bus 003 Device 007: ID 0781:5406 SanDisk Corp. Cruzer Micro U3
```

上面以粗体显示的数字分别用于标识“ host_bus”和“ host_addr”或者“ vendor_id”和“ product_id”。

基本的思想是在 QEMU 中 `-device usb-ehci,id=ehci` 或 `-device qemu-xhci,id=xhci` 分别对 EHCI (USB 2) 或 XHCI (USB 1.1，USB 2，USB 3) 控制器进行模拟，然后将物理设备通过 `-device usb-host,..` 选项进行添加。在本节中的剩余部分，*controller_id*要么是 `ehci`，要么是 `xhci`。

接着，这里有两种方法通过qemu连接到宿主机的USB：

1. 识别出该设备，并将其连接至任一总线以及宿主机上的地址，通用的语法如下：

```
    -device usb-host,bus=controller_id.0,vendorid=0xvendor_id,productid=0xproduct_id
```

。Applied to the device used in the example above, it becomes:

```
    -device usb-ehci,id=ehci -device usb-
host,bus=ehci.0,vendorid=0x0781,productid=0x5406
```

；此外也可以在上面的选项中添加`...,port=*port_number*` 设置，用于指定设备添加至虚拟机控制器上的哪个物理端口。该设置在为 VM 添加多个设备时比很有用。另一个方案是使用 QEMU 5.1.0 之后出现的 `usb-host` 的 `hostdevice`属性，语法为：

```
    -device qemu-xhci,id=xhci -device usb-host,hostdevice=/dev/bus/usb/003/007
```

1. 若要添加一个已连接至特定USB总线和地址的设备，语法为：

```
    -device usb-host,bus=controller_id.0,hostbus=host_bus,host_addr=host_addr
```

，将其中的参数修改至该总线和地址即可，拿上面的例子来说，需修改为：

```
    -device usb-ehci,id=ehci -device usb-host,bus=ehci.0,hostbus=3,hostaddr=7
```

注意：如果运行 QEMU 时遇到了权限方面的错误，可以阅读 udev#About udev rules 获取更多信息，以了解如何为设备设定合适的权限。

使用 SPICE 进行 USB 重定向

使用#SPICE时可以将USB设备从客户端重定向至虚拟机中，无需使用QEMU命令。还支持为配置USB重定向插槽数（插槽数将决定可同时重定向的最大设备数）。相比于前面那种使用`-usbdevice`进行重定向的方法，SPICE方法的优势在于可以在虚拟机启动后USB设备热插拔，移除或添加USB设备时无需停机。这个方法还允许通过网

络将客户端的USB设备重定向至服务端。总之，其是在QEMU虚拟机中使用USB设备最灵活的方法。

我们需要为每个所需的USB重定向插槽添加一个EHCI/UHCI控制器，以及每个插槽添加一个SPICE重定向通道。例如，将下面的选项加入到所使用的QEMU命令中，以SPICE模式启动虚拟机时将会添加三个用于重定向的USB插槽：

```
-device ich9-usb-ehci1,id=usb \
-device ich9-usb-uhci1,masterbus=usb.0,firstport=0,multifunction=on \
-device ich9-usb-uhci2,masterbus=usb.0,firstport=2 \
-device ich9-usb-uhci3,masterbus=usb.0,firstport=4 \
-chardev spicevmc,name=usbredir,id=usbredirchardev1 -device usb-
redir,chardev=usbredirchardev1,id=usbredirdev1 \
-chardev spicevmc,name=usbredir,id=usbredirchardev2 -device usb-
redir,chardev=usbredirchardev2,id=usbredirdev2 \
-chardev spicevmc,name=usbredir,id=usbredirchardev3 -device usb-
redir,chardev=usbredirchardev3,id=usbredirdev3
```

参阅 SPICE/usbredir 获取更多信息。

spice-gtk包中的`spicy`(*Input > Select USB Devices for redirection*)和virt-viewer包中的`remote-viewer`(*File > USB device selection*)都支持该特性。请确保你已在客户机上安装必要的SPICE Guest Tools，以使得该功能正常运作（参见#SPICE节获得更多信息）。

警告：需要牢记一点，当将USB设备从客户端重定向至服务端时，在客户端所在的操作系统上都无法使用该USB了，除非停止重定向。尤其重要的是千万不要将输入设备重定向（即不要将鼠标和键盘重定向），这么做会使得你难以访问到SPICE客户端的菜单而无法撤销该操作，因为在输入设备被重定向至虚拟机后，客户端无法响应它们的操作。

使用 udev 进行自动 USB 转发

通常，转发的设备必须在虚拟机启动时可用才能被转发。如果该设备断开连接，它将不再被转发。

您可以使用 udev rule 在设备上线时自动附加设备。在磁盘上的某个位置创建一个 `hostdev` 条目，并将其所有权 chown 为 root ，以防止其他用户修改它。

```
/usr/local/hostdev-mydevice.xml
<hostdev mode='subsystem' type='usb'>
      <vendor id='0x03f0'/>
    <product id='0x4217'/>
</hostdev>
```

然后创建一个用于附加或脱离设备的 *udev* 规则:

```
/usr/lib/udev/rules.d/90-libvirt-mydevice
ACTION=="add", \
    SUBSYSTEM=="usb", \
    ENV{ID_VENDOR_ID}=="03f0", \
    ENV{ID_MODEL_ID}=="4217", \
    RUN+="/usr/bin/virsh attach-device GUESTNAME /usr/local/hostdev-mydevice.xml"
ACTION=="remove", \
    SUBSYSTEM=="usb", \
    ENV{ID_VENDOR_ID}=="03f0", \
    ENV{ID_MODEL_ID}=="4217", \
```

```
    RUN+="/usr/bin/virsh detach-device GUESTNAME /usr/local/hostdev-mydevice.xml"
```

详阅这篇博客。

开启KSM

Kernel Samepage Merging (KSM) 是Linux内核的一个特性，允许应用程序向内核申请同其他申请页归并的进程进行页归并，KSM机制允许客户虚拟机之间进行页共享。当许多客户机运行相似的操作系统时，这个机制可以节省客观的内存。

注意：尽管KSM可能减小内存占用量，其增加了CPU负担。此外要注意的是使用该机制可能产生一些安全问题，参见Wikipedia:Kernel same-page merging。

启用KSM:

```
# echo 1 > /sys/kernel/mm/ksm/run
```

使之常驻，使用 systemd 的临时文件

```
/etc/tmpfiles.d/ksm.conf
w /sys/kernel/mm/ksm/run - - - - 1
```

当 KSM 启用，而正好有准备进行归并的页，（比如，至少有两个相似的 VM 正在运行），那么 `/sys/kernel/mm/ksm/pages_shared` 应会是非零的。参阅 https://www.kernel.org/doc/html/latest/admin-guide/mm/ksm.html 了解更多信息。

提示：查看KSM性能的一种简单方法是列出此目录下的所有文件内容：

```
$ grep -r . /sys/kernel/mm/ksm/
```

多屏支持

Linux的QXL驱动支持默认支持四头（虚拟屏幕），可以通过`qxl.heads=N`这一内核参数进行变更。

QXL设备的默认VGA内存大小为16M（VRAM大小为64M），在想使用两块1920x1200显示器时（需要2 × 1920 × 4 (色深) × 1200 = 17.6 MiB VGA内存）是不够用的。用`-vga none -device qxl-vga,vgamem_mb=32`选项代替`-vga qxl`可以对该属性进行变更。如果你将vgamem_mb增大至64M以上，则还需要增大`vram_size_mb`选项的值。

自定义显示器分辨率

可以使用 `-device VGA,edid=on,xres=1280,yres=720` 来设置自定义显示器分辨率 (参见 EDID 和 display resolution) 。

复制和粘贴

在宿主机和客户机之间共享剪贴板的方法之一是使用SPICE远程桌面协议，通过SPICE客户端访问客户机，你需要遵照#SPICE节中描述的步骤，通过该方式运行的客户机将支持与宿主机进行复制粘贴的操作。

SPICE

在主机和虚拟机之间共享剪贴板的一种方法是启用 SPICE 远程桌面协议，并使用 SPICE 客户端访问客户端。

请按照 #SPICE 中描述的步骤进行。以这种方式运行的虚拟机将支持与主机的复制粘贴功能。

qemu-vdagent

QEMU 提供了其自己的 spice vdagent 字符设备实现，称为 `qemu-vdagent`。它与 spice-vdagent 客户端服务接口，可以实现虚拟机和主机之间的剪贴板共享。

要使用 QEMU 的 GTK 显示器访问此共享剪贴板，你需要从源码编译 QEMU，并使用 `--enable-gtk-clipboard` 配置参数。这需要替换已安装的 `qemu-ui-gtk` 包。

注意：

- 已提交功能请求 FS#79716 以在官方包中启用此功能。
- 由于 在某些情况下会导致虚拟机冻结，qemu-ui-gtk 中的共享剪贴板功能已推回到实验阶段。上游已提出解决此问题的修复方案。

添加以下的 QEMU 命令行参数:

```
-device virtio-serial,packed=on,ioeventfd=on
-device virtserialport,name=com.redhat.spice.0,chardev=vdagent0
-chardev qemu-vdagent,id=vdagent0,name=vdagent,clipboard=on,mouse=off
```

这些参数如果转换为 libvirt 格式也是有效的。

注意：虽然 spicevmc 字符设备会自动启动虚拟机的 spice-vdagent 服务，但 qemu-vdagent 字符设备可能不会。

在 Linux 虚拟机上，你可以手动启动 `spice-vdagent.service` 用户单元。在 Windows 虚拟机上，请将 spice-agent 的启动类型设置为自动。

Windows专用说明

QEMU可以运行从 Windows 95 至 Windows 11 的任何版本 Windows。

在QEMU中也可以运行 Windows PE。

快速启动设置

注意：改变电源设置需要一个 Administrtor 账户

根据此论坛帖子所解释的那样，对于Windows8及之后的客户机最好在Control Panel的Power Options中禁用"Turn on fast startup (recommended)"，该设置导致客户机在每次启动时卡死。

为了正确应用对 `-smp` 选项进行的更改，可能也需要禁用快速启动。

远程桌面协议

如果你使用一个 MS Windows 客户机，可能会想使用 RDP 连接到客户机 VM。若你正使用 VLAN 或与客户机处于同一个网络中，先使用：

```
$ qemu-system-x86_64 -nographic -nic user,hostfwd=tcp::5555-:3389
```

接着通过 rdesktop包 或 freerdp包 连接到客户机。例如：

```
$ xfreerdp -g 2048x1152 localhost:5555 -z -x lan
```

在物理设备上克隆一个已安装的Linux系统

安装在物理设备上的Linux系统可以克隆至QEMU VM中运行，参阅Clone Linux system from hardware for QEMU virtual machine

从 x86_64 环境中 Chroot 至 arm/arm64 环境

有时候相比于在基于ARM架构的设备上工作，在磁盘镜像上进行工作更容易。可以通过挂载一个*root*分区的SD卡/存储设备，然后chroot到该设备实现该目的。

另一个使用 ARM chroot 的场景是在 x86_64 机器上构建 ARM 包。这种情况下，可以使用来自 Arch Linux ARM 的镜像包来创建 chroot 环境 - 具体操作可查阅 。

无论哪种方式，都应该可以从chroot运行pacman并安装更多软件包，编译大型库等。由于可执行文件是针对ARM体系结构的，因此需要QEMU转换为x86。

在 x86_64 宿主机上安装 qemu-user-static包 和 qemu-user-static-binfmt包，后者会将 qemu 二进制文件注册至 binfmt 服务。

需要使用 *qemu-user-static* 执行其他架构上的程序，这个包与 qemu-emulators-full包 提供的程序很相似，只是对于 chroot 来说必须使用 "static" 变种。比如：

```
qemu-arm-static path_to_sdcard/usr/bin/ls
qemu-aarch64-static path_to_sdcard/usr/bin/ls
```

这两个命令分别用于执行 32 位 ARM 和 64ARM 的 {{ic|ls} }命令。注意如果没有 chroot，这些都不会生效，因为那样的话它们将会试图寻找不存在宿主机系统上的一些二进制文件。

qemu-user-static包 允许自动地用为 ARM 可执行文件添加 `qemu-arm-static` 或 `qemu-aarch64-static` 前缀。

检查一下 ARM 可执行的支持是否被开启了：

```
$ ls /proc/sys/fs/binfmt_misc
qemu-aarch64  qemu-arm        qemu-cris  qemu-microblaze  qemu-mipsel  qemu-ppc64
qemu-riscv64  qemu-sh4    qemu-sparc      qemu-sparc64  status
qemu-alpha    qemu-armeb  qemu-m68k  qemu-mips         qemu-ppc          qemu-
ppc64abi32  qemu-s390x        qemu-sh4eb  qemu-sparc32plus     register
```

每种可执行的文件都须在此处列出。

如果未开启，请重启 `systemd-binfmt.service`。

将SD卡挂载至 `/mnt/sdcard`（设备名可能不同）：

```
# mount --mkdir /dev/mmcblk0p2 /mnt/sdcard
```

如有需要，挂载启动分区（同样，使用合适的设备名）：

```
# mount /dev/mmcblk0p1 /mnt/sdcard/boot
```

最后如Change root#Using chroot所描述的那样，*chroot* 到 SD 卡的 root 中：

```
# chroot /mnt/sdcard /bin/bash
```

此外，可以使用 arch-install-scripts包 提供的 *arch-chroot* 替代 chroot，可以方便地获取网络支持：

```
# arch-chroot /mnt/sdcard /bin/bash
```

还可以用 systemd-nspawn chroot 到 ARM 环境中：

```
# systemd-nspawn -D /mnt/sdcard -M myARMMachine --bind-ro=/etc/resolv.conf
```

`--bind-ro=/etc/resolv.conf` 是可选的，其在 chroot 中提供了一个可用的网络 DNS。

在 chroot 环境中使用 sudo

如果您在 chroot 环境中安装了 sudo，并在尝试使用它时收到以下错误：

```
sudo: effective uid is not 0, is /usr/bin/sudo on a file system with the 'nosuid'
option set or an NFS file system without root privileges?
```

那么您可能需要修改 binfmt 标志，例如对于 aarch64：

```
# cp /usr/lib/binfmt.d/qemu-aarch64-static.conf /etc/binfmt.d/
# vi /etc/binfmt.d/qemu-aarch64-static.conf
```

并在该文件末尾添加一个 `C`：

```
:qemu-aarch64:M::
\x7fELF\x02\x01\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x00\xb7\x00:
\xff\xff\xff\xff\xff\xff\xff\x00\xff\xff\xff\xff\xff\xff\xff\xff\xfe\xff\xff\xff:/usr/b
in/qemu-aarch64-static:FPC
```

然后重新启动 systemd-binfmt.service 并检查更改是否生效（注意标志行上的 C）：

```
# cat /proc/sys/fs/binfmt_misc/qemu-aarch64
enabled
interpreter /usr/bin/qemu-aarch64-static
flags: POCF
offset 0
magic 7f454c460201010000000000000000000200b700
mask ffffffffffffff00fffffffffffffffffeffffff
```

有关更多信息，请参阅内核 binfmt 文档的“flags”部分。

处理常见问题

**本文或本章节可能需要合并到 QEMU/Troubleshooting。**

附注： This section is long enough to be split into a dedicated subpage.（在 Talk:QEMU 中讨论）

Not grabbing mouse input

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：It is not explained what the option actually does. Is it causing or avoiding the side effect?（在 Talk:QEMU讨论）

Tablet mode has side effect of not grabbing mouse input in QEMU window:

```
-usb -device usb-tablet
```

It works with several `-vga` backends one of which is virtio.

鼠标指针抖动或者不稳定

如果QEMU中鼠标指针不受控制地跳来跳去，那么启动QEMU之前在命令进行如下设置或许能解决该问题：

```
$ export SDL_VIDEO_X11_DGAMOUSE=0
```

如果成功了，可以把这条命令加入到你的 `~/.bashrc` 文件中

看不见鼠标指针

使用`-show-cursor`选项启动QEMU以显示指针。

如果这么做不起作用的话，请确保你的显示设备被正确设置了。例如： `-vga qxl`。

还可尝试 #鼠标整合中提到的 `-usb -device usb-tablet`。该选项会覆盖默认采用的PS/2鼠标仿真，将鼠标作为外设在宿主机和客户机之间同步指针位置。

有两个不同的鼠标指针

遵循 #鼠标整合提到的方法。

使用 VNC 时键盘出现问题

使用 VNC 时，您可能会遇到这里所描述的键盘问题。 解决的方法是不使用 QEMU 的 `-k` 选项, 并使用 gtk-vnc包 的 `gvncviewer` 。另请参见在libvirt邮件列表中发布的消息。

键盘像坏了一样或者方向键不起作用

如果发现某些键不起作用或按下一个按键，触发的却是其他按键，那么你可能需要指定键盘布局选项。在 `/usr/share/qemu/keymaps` 中可以找到键盘布局选项。

```
$ qemu-system-x86_64 -k keymap disk_image
```

无法读取键盘映射文件

```
qemu-system-x86_64: -display vnc=0.0.0.0:0: could not read keymap file: 'en'
```

这类问题产生的原因是将非法的键盘映射名作为`-k`的参数传递给qemu。例如，`en`是错误的键盘映射名称，`en-us`才是正确的。相关信息参见`/usr/share/qemu/keymaps`。

客户机在调整窗口大小时一起被拉伸了

按下 `Ctrl+Alt+u` 可以恢复到原来的大小。

ioctl(KVM_CREATE_VM) failed: 16 Device or resource busy

如果在以 `-enable-kvm` 选项启动QEMU时显示了这样一条错误信息：

```
ioctl(KVM_CREATE_VM) failed: 16 Device or resource busy
failed to initialize KVM: Device or resource busy
```

这意味着还有另一个 hypervisor 处于运行状态， 不推荐同时运行多个hypervisor。

libgfapi 的错误信息

启动时若显示如下错误信息:

```
Failed to open module: libgfapi.so.0: cannot open shared object file: No such file or directory
```

安装 glusterfs包 或者直接忽略这个错误信息，GlusterFS 只是一个可选的依赖。

LIVE-environments 上发生内核错误

如果你启动一个 live-environment（或者启动一个系统），可能会遭遇如下错误：

```
[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown block(0,0)
```

或者其他阻碍启动过程的东西（比如无法解包initramfs， 无法启动foo服务之类的），试试启动VM的时候用 `-m VALUE` 分配合适的 RAM 大小，如果内存太小是很有可能遭遇上述的问题的。

Windows 7 客户机的音频质量差

为Windows7使用 `hda` 音频驱动可能造成音频质量降低， 通过QEMU的 `-soundhw ac97` 参数选择 `ac97` 作为音频驱动， 并从Realtek AC'97 Audio Codecs下载AC97驱动，然后在客户机中安装该驱动可能会解决这个问题。参见Red Hat Bugzilla – Bug 1176761以获取更多信息。

Could not access KVM kernel module: Permission denied

如果遇见了如下的错误信息:

```
libvirtError: internal error: process exited while connecting to monitor: Could not access KVM kernel module: Permission denied failed to initialize KVM: Permission denied
```

这是因为 Systemd 234 为 `kvm` 组注册了一个动态ID （参见FS#54943）。可以编辑 `/etc/libvirt/qemu.conf` 将 `group = "78"` 改为 `group = "kvm"` 避免产生这个错误。

启动 Windows VM 时产生 "System Thread Exception Not Handled"

Windows 8或Windows 10 guest虚拟机在启动时可能会引发通用兼容性异常，即“System Thread Exception Not Handled”，这通常是由实体机上旧版驱动程序的异常行为引起的。在KVM机器上，通常可以通过将CPU模型设置为 `core2duo`来解决此问题。

某些 Windows 游戏/程序导致蓝屏出现

有时，一些能在物理机上正常运行的程序在虚拟机中会意外地崩溃。如果在以 root 运行 `dmesg -wH` 时发现有 `MSR` 相关的错误，则导致这些崩溃的原因是 KVM 注入了 GPF（General protection fault），当客户机尝试访问不受支持的寄存器（MSR，Model-specific registers）时会导致客户机上的应用程序崩溃。将 `ignore_msrs=1` 传给 KVM 模块可以使其忽略不支持的 MSR， 从而解决大部分的同类问题。

```
/etc/modprobe.d/kvm.conf
...
options kvm ignore_msrs=1
...
```

一些添加该选项可能会起作用的场景：

- GeForce Experience 反映没有可支持的CPU。
- 星际争霸2和黑色洛城使用 `KMODE_EXCEPTION_NOT_HANDLED` 对 Windows 10 的蓝屏做了一些处理，蓝屏信息中将无法识别驱动。

警告：虽然通常这么做是安全的，对于某些应用程序也可能不起作用，但默默忽略未知的 MSR 仍可能破坏 VM 或其他 VM 中的其他软件。

VM 中的应用程序有很高的延迟，或是需要等待很长的时间才会启动

这可能是由于VM中的可用熵不足所致。考虑通过将 VirtIO RNG 设备添加到 VM，或安装诸如 Haveged 之类的熵生成守护程序。

有趣的是，OpenSSH 需要一段时间才能在熵不足的情况下开始接受连接，而其日志却不会显示原因。

中断时间过长以及运行不流畅

此问题表现为小停顿（断断续续），在图形密集型应用程序（例如游戏）中尤其明显。

- 原因之一是CPU的节能功能，该功能由CPU frequency scaling控制。请对所有处理器核心进行修改，将其更改为`performance`。
- 另一个可能的原因是PS/2输入。从PS/2切换到Virtio输入，请参阅PCI passthrough via OVMF#Passing keyboard/mouse via Evdev。

QXL 导致视频分辨率降低

QEMU 4.1.0版本带来了一个回退问题，当使用SPICE时，会将QXL视频降到低分辨率。 例如，当KMS启动时，文本分辨率可能会低至4x10个字符。尝试提高GUI分辨率时，它可能会达到最低的支持分辨率。

解决方法是，以这种形式创建设备：

```
-device qxl-vga,max_outputs=1...
```

使用启用了安全启动的OVMF时，VM无法启动

edk2-ovmf包 中的 `OVMF_CODE.secboot.4m.fd` 以及 `OVMF_CODE.secboot.fd` 文件是在 SMM 的支持下完成构建的，如果未在 VM 中禁用 S3 支持，则 VM 可能根本无法启动。

在QEMU命令中添加 `-global ICH9-LPC.disable_s3=1` 以解决该问题。

参见 FS#59465 以及 https://github.com/tianocore/edk2/blob/master/OvmfPkg/README 获取 QEMU 使用 Secure Boot 所需要的更多选项。

Virtual machine not booting into Arch ISO

When trying to boot the virtual machine for the first time from an Arch ISO image, the boot process hangs. Adding `console=ttyS0` to kernel boot options by pressing `e` in the boot menu you will get more boot messages and the following error:

```
:: Mounting '/dev/disk/by-label/ARCH_202204' to '/run/archiso/bootmnt'
Waiting 30 seconds for device /dev/disk/by-label/ARCH_202204 ...
ERROR: '/dev/disk/by-label/ARCH_202204' device did not show up after 30 seconds...
   Falling back to interactive prompt
   You can try to fix the problem manually, log out when you are finished
sh: can't access tty; job control turned off
```

The error message does not give a good clue as to what the real issue is. The problem is with the default 128MB of RAM that QEMU allocates to the virtual machine. Increasing the limit to 1024MB with `-m 1024` solves the issue and lets the system boot. You can continue installing Arch Linux as usual after that. Once the installation is complete, the memory allocation for the virtual machine can be decreased. The need for 1024MB is due to RAM disk requirements and size of the installation media. See this message on the arch-releng mailing list and this forum thread.

客户机上的CPU中断没有被触发

如果你是按照 OSDev Wiki 来编写自己的操作系统，或者只是使用 QEMU 的 `gdb` 接口和 `-s` 标志，您需要知道的是，很多仿真器（包括QEMU）通常会实现一些CPU中断，同时还有许多硬件中断没有被实现。了解代码是否触发中断的一种方法是使用：

```
-d int
```

可以在输出中查看 interrupts/exceptions。

如要查看QEMU还提供了什么客户机调试功能，使用如下命令：

```
qemu-system-x86_64 -d help
```

或者将 `x86_64` 替换为你选择的其他架构

KDE with sddm does not start spice-vdagent at login automatically

Remove or comment out `X-GNOME-Autostart-Phase=WindowManager` from `/etc/xdg/autostart/spice-vdagent.desktop`.

Error starting domain: Requested operation is not valid: network 'default' is not active

If for any reason the default network is deactivated, you will not be able to start any guest virtual machines which are configured to use the network. Your first attempt can be simply trying to start the network with virsh.

```
# virsh net-start default
```

For additional troubleshooting steps, see .

参阅

- Official QEMU website
- Official KVM website
- QEMU Emulator User Documentation
- QEMU Wikibook
- Hardware virtualization with QEMU by AlienBOB (last updated in 2008)
- Building a Virtual Army by Falconindy
- QEMU documentation
- QEMU on Windows
- Wikipedia
- Debian Wiki - QEMU
- QEMU Networking on gnome.org\[失效链接 2022-09-22 ⓘ\]
- Networking QEMU Virtual BSD Systems
- QEMU on gnu.org
- QEMU on FreeBSD as host
- Managing Virtual Machines with QEMU - openSUSE documentation
- Managing Virtual Machines with QEMU - openSUSE documentation
- KVM on IBM Knowledge Center

数据压缩与归档

归档、压缩与整盘复制常被混为一谈，其实要分开看：只把文件打成一个包、打包时压缩、或者连整块磁盘一起复制，是不同的事，各有各的工具。归档与压缩一节是必读的地基，先分清仅归档与压缩工具的分界，再比较用法、确定归档格式；硬盘克隆同样必读，dd 与 cp 两节都要过。

7-Zip 与 Rar 可按需查阅，但别把 7z、7za、7zr 当成同一个二进制，rar 与 unrar 也要分开处理。最容易误解的是把归档当压缩，仅归档工具打出的包并不会变小；另一处是选格式只看能不能解开，小众、罕见或过时的工具解得开，不等于适合长期存档。

9.1　7-Zip

难度 入门 | **篇幅** 2.6 KB | **所属** 数据压缩与归档

本节将讲到

- 安装
- 示例
- 二进制文件 7z、7za 与 7zr 的区别
- 参见

7-Zip 是一款高压缩比的文件压缩工具。它曾经由 *p7zip* 包提供。

安装

安装 7zip包 软件包。

程序可通过以下命令运行：

```
$ 7z
```

替代方案：bsdtar(1) 提供了 7z 归档支持，它包含在 libarchive包 包中。

示例

警告：7z 格式不会保存文件的所有者/组，因此不应用于备份目的。

添加文件或目录至已有的归档（或创建一个新的归档）：

```
$ 7z a 归档名 文件名
```

也可以使用 `-p` 设置密码，并使用 `-mhe=on` 隐藏归档的目录结构：

警告：使用 `-p` 要求在命令行中输入密码。从 24.09 版本开始，在 `Enter password:` 后输入的内容不再隐藏，输入的内容可见，并且不会要求确认密码。

```
$ 7z a 归档名 文件名 -p -mhe=on
```

更新归档内已有的文件或添加新文件：

```
$ 7z u 归档名 文件名
```

列出归档的内容：

```
$ 7z l 归档名
```

从归档中解压所有文件到当前文件夹中，不保留存档内的目录结构：

```
$ 7z e 归档名
```

如果需要保留归档内的目录结构，使用：

```
$ 7z x 归档名
```

解压到新的目录中：

```
$ 7z x -o文件夹路径 归档名
```

校验归档完整性：

```
$ 7z t 归档名
```

二进制文件 7z、7za 与 7zr 的区别

软件包中包含了三个二进制文件：

- `7z` 使用插件来处理归档文件。
- `7za` 是一个独立的可执行文件，支持的归档格式比 7z 少。
- `7zr` 是一个独立的可执行文件。它是 7za 的“轻量版”，仅支持 7z 格式的归档文件。与 7za 不同的是，它无法处理加密的归档文件。

参见

- zhwp:7-Zip
- 7-Zip 主页（英文）
- 官方手册（英文）

9.2 Rar

难度 入门 | **篇幅** 5.7 KB | **所属** 数据压缩与归档

本节将讲到

- 主要特点
- 安装
- 配置文件
- RAR 压缩例子
- UNRAR 例子

RAR (and UNRAR) is the Linux port of the commandline-only version of WinRAR available in both the i686 and x86-64 flavors.

主要特点

- Variable amounts of redundancy ("recovery record" or "recovery volumes" both of which are demonstrated below) can be added to an archive, making it more resistant to corruption. Even if parts of an archive are damaged, it is possible to fully recover the stored data if a large enough recovery record exists. On its own, Tar does not have this ability.
- RAR is able to efficiently handle split volumes. Built-in support for multi-volume files enables the unpacking program to simply prompt the user for the next .partXXX RAR file, without the need to manually copy and then rejoin the pieces, or for extracting a file from a single piece without needing all pieces. RAR does not support tapes, as it uses seek and rename operations on its files.
- RAR archives can be of a solid format, in which all of the compressed files are treated as a single data block. Most currently used compression formats (with the exception of the older ZIP) allow solid structuring.
- Strong encryption capabilities. Older versions of the file format used a proprietary algorithm; newer versions use the AES encryption algorithm, a block cipher adopted as an encryption standard by the U.S. government. The only known ways to recover an encrypted file are via dictionary or brute force attacks. In newer versions, password protection can optionally protect filenames too, so that the filenames contained within the archive will not be displayed without the right password.

安装

RAR

可以通过 AUR 来安装 rarAUR（除 UNRAR 外的所有功能）。

UNRAR

官方仓库单独提供了 unrar包 ，可以像平常一样通过 pacman 来安装。

配置文件

Linux 版的 RAR 从 `~/.rarrc` （即用户的 home 目录下）读取配置信息，又或者可以在 /etc 目录下，定义一个全局配置文件，供所有用户使用。

这个文件的语法如下：

```
switches=any RAR switches, separated by spaces
```

举个例子：

```
switches=-m5 -rr5 -ol -msjpg;mp3;avi;zip;rar;tar;gz;jpg
```

要想获得关于 RAR 选项的完整清单及详细解释，可以访问用户手册。

RAR 压缩例子

基本语法

```
$ rar command -switch 1 -switch N archive files.rar @listfiles...
```

要想获得完整的命令及选项，请参阅本文的最后一部分，或者运行 `rar` 。

Recursively compress an entire directory structure

- Task: backup `/home/darkhorse` to `/media/data/darkhorse-backup.rar` using 10 % recovery records:

```
$ rar a -r -rr10 /media/data/darkhorse-backup.rar /home/darkhorse
```

- Explained:

<table class="wikitable"><tbody><tr><td>Switch</td><td>Action</td></tr><tr><td>a</td><td>adds files to archives.</td></tr><tr><td>-r</td><td>recurse subdirectories (includes all dirs/files under the parent directory).</td></tr><tr><td>-rr10</td><td>adds recovery records to the archive. This way up to 10% of the compressed archive can become corrupt or unusable, and it will be able to recover the data through parity.</td></tr></tbody></table>

Mixed-mode archives

You can also use mixed-mode archives which means that file types you specifiy do not get compressed - they simply get stored.

- Task: backup `/home/darkhorse` to `/media/data/darkhorse-backup.rar`:

```
$ rar a -r -rr10 -s -m5 -msjpg;mp3;tar /media/data/darkhorse-backup.rar /home/darkhorse
```

- Explained:

<table class="wikitable"><tbody><tr><td>Switch</td><td>Action</td></tr><tr><td>a</td><td>adds files to archives.</td></tr><tr><td>-r</td><td>recurse subdirectories (includes all dirs/files under the parent directory).</td></tr><tr><td>-rr10</td><td>adds recovery records to the archive. This way up to 10% of the compressed archive can become corrupt or unusable, and it will be able to recover the data through parity.</td></tr><tr><td>-m5</td><td>Use the highest level of compression (m0 = store ... m3 = default ... m5 = maximal level of compression.</td></tr><tr><td>-msjpg;mp3;tar</td><td>ignore the compression option and store all .jpg and .mp3 and .tar files.</td></tr></tbody></table>

Recursively compress many directory structures using a list

- Task: backup `/home/darkhorse` and `/home/palomino` and `/home/seabiscuit` to `/media/data/homes-backup.rar`.

First create a list (simple text file) containing the various targets. In this example, the list will be three lines long. I named it 'home-list' in this example but you can call it anything you want:

```
/home/darkhorse
/home/palomino
/home/seabiscuit
$ rar a -r -rr10 -s /media/data/homes-backup.rar @/path/to/home-list
```

UNRAR 例子

基本语法

```
$ unrar command -switch 1 -switch N archive files... @listfiles... path_to_extract\
```

想了解完整的命令及选项，只需运行：

```
$ unrar --help
```

解压到一个新的目录：

```
$ unrar x /media/data/homes-backup.rar homes-backup/
```

运行以下命令，来解压分卷压缩文件：

```
$ unrar x homes-backup.part1.rar homes-backup/
```

9.3 硬盘克隆

难度 进阶 | **篇幅** 11.8 KB | **所属** 数据压缩与归档

本节将讲到

- 使用dd
- Using cp
- Disk cloning software
- 外部链接

磁盘克隆是制作整个硬盘镜像的过程, 其对于复制整个磁盘到别的电脑和备份/恢复都十分有用. 你可以访问File recovery (简体中文), 这是一个专注于文件恢复的页面.

使用dd

dd命令是一个简单而多功能且强大的工具. 它可以用于在无视文件系统或操作系统的情况下一块一块的将数据从原始地址复制到目的地. 在live环境比如livecd下使用dd是个方便的做法.

警告：在使用这个工具时请对任何操作保持警惕, 因为它可以破坏你的数据. 请牢记输入文件 (if=) 和输出文件 (of=) 的次序并千万不要颠倒他们! 请确保目的地(也就是分区 (of=) 不能小于源地址 (if=).

克隆分区

从物理盘 /dev/sda 的1分区 到物理盘 /dev/sdb 的1分区.

```
dd if=/dev/sda1 of=/dev/sdb1 bs=4096 conv=notrunc,noerror,sync
```

如果输出文件 ***of*** (在这个例子中是sdb1) 不存在, dd 会从这个磁盘的开头开始并且创建他.

克隆整个硬盘

从物理盘 /dev/sda 到物理盘 /dev/sdb

```
dd if=/dev/sda of=/dev/sdb bs=4096 conv=notrunc,noerror,sync
```

这条命令会克隆整个盘, 包括MBR(所以也包括bootloader), 所有的分区、UIUD 以及数据.

- *notrunc* (也就是'不许删') 通过要求 dd 不对数据进行任何删节来保证数据的统一完整.
- *noerror* 要求 dd 无视任何读错并继续工作. dd的初始设定是一旦遇到错误就停下.
- *sync* 对读错误进行清零, 这样数据的开端就可以保持在 sync.
- *bs=4096* 将block size设置为4k. 这样硬盘读写效率不知道高到哪里去了, 显然这个对克隆速度就是最好的.

注意：如果想要重新获得独特的UUID, 请对每个分区都使用 "tune2fs /deb/sdbX -U random".

注意：dd带来的分区表的改变并不会注册到内核. 如果想要不重启即传达这一变化, 请使用类似于 partprobe(GNU parted 的一部分)的工具来达到目的.

备份MBR

MBR存储在磁盘的前512字节中。它由三部分组成：

1. 前446字节包含引导加载程序。
2. 接下来的64个字节包含分区表（4个16字节的条目，每个主分区一个条目）。
3. 最后2个字节包含一个标识符

To save the MBR into the file "mbr.img":

```
  # dd if=/dev/hda of=/mnt/sda1/mbr.img bs=512 count=1
```

To restore (be careful : this could destroy your existing partition table and with it access to all data on the disk):

```
  # dd if=/mnt/sda1/mbr.img of=/dev/hda
```

If you only want to restore the boot loader, but not the primary partition table entries, just restore the first 446 bytes of the MBR:

```
  # dd if=/mnt/sda1/mbr.img of=/dev/hda bs=446 count=1
```

To restore only the partition table, one must use

```
  # dd if=/mnt/sda1/mbr.img of=/dev/hda bs=1 skip=446 count=64
```

You can also get the MBR from a full dd disk image.

```
  #dd if=/path/to/disk.img of=/mnt/sda1/mbr.img bs=512 count=1
```

创建磁盘映像

1\. 从liveCD或liveUSB引导.

2\. 确保没有从源硬盘驱动器装入分区.

3\. 挂载外部硬盘

4\. 备份硬盘.

```
 # dd if=/dev/hda conv=sync,noerror bs=64K | gzip -c  > /mnt/sda1/hda.img.gz
```

5\. Save extra information about the drive geometry necessary in order to interpret the partition table stored within the image. The most important of which is the cylinder size.

```
 # fdisk -l /dev/hda > /mnt/sda1/hda_fdisk.info
```

NOTE: You may wish to use a block size (bs=) that is equal to the amount of cache on the HD you are backing up. For example, bs=8192K works for an 8MB cache. The 64K mentioned in this article is better than the default bs=512 bytes, but it will run faster with a larger bs=.

Restore system

To restore your system:

```
 # gunzip -c /mnt/sda1/hda.img.gz | dd of=/dev/hda
```

Examples with compression

When you need to create the hard drive or a single partition compressed backup image file you must use compression tools which can do backup from a *stdout* and the *dd* command. Those compressed files cannot be mounted by the *mount* command but are useful to know how to create and restore them.

7zip

Install the p7zip包 package from the official repositories. This backup example will split the *dd* command output in the files by up to the 100 megabyte each:

```
dd if=/dev/sdXY | 7z a -v100m -t7z -si image-file.7z
```

Restore with 7zip:

```
7z x -so  image-file.7z | dd of=/dev/sdXY
```

注意：7zip can split only the *7z* compression type files

Zip

Install the zip包 package from the official repositories, which contains *zipsplit* among other utilities for the management of zip archives. It will create a file with "-" name inside the image-file.zip file which will contain data from the *dd* command output. To make a raw output of the file you can use the `-cp` option with *unzip* in stdout for the *dd* command. Backup:

```
dd if=/dev/sdXY | zip --compression-method bzip2  image-file.zip -
```

Restore:

```
unzip -cp image-file.zip | dd of=/dev/sdXY
```

The *zip* tool cannot split files on the fly but you can use the `zipsplit` utility on an already created file.

See also zip(1) for more information.

Rar

Install the rarAUR package from the AUR.

警告： The *rar* examples were made based on the manuals, please confirm!

This should do a backup and split the creating file on the fly in by up to 150 megabyte files each.

```
dd if=/dev/sdXY | rar a -v150m -siimage-file.rar
```

This should restore

```
unrar x -p image-file.rar | dd of=/dev/sdXY
```

or you can use the *rar* instead of the unrar包 utility. The unrar utility is available in the official repositories and can be installed with `pacman -S unrar`.

Bzip2

Creation by using the *dd* is more safe and use to be error free:

```
dd if=/dev/sdXY | bzip2 -f5 > compressedfile.bzip2
937016+0 records in
937016+0 records out
479752192 bytes (480 MB) copied, 94.7002 s, 5.1 MB/s
```

And a safe way of restoring with combination of the *dd*:

```
$ bunzip2 -dc compressedfile.bzip2 | dd of=/dev/sdXY
```

or

```
$ bzcat compressedfile.bzip2 | dd of=/dev/sdXY
```

警告：Never ever use the `bzip2 -kdc imgage.bzip2 > /dev/sdXY` and `bzip2 -kc /dev/sdXY > imgage.bzip2` methods for serious backup of partitions and disks. The errors might be due the end of the device or partition and the restore process gives also errors due the truncated end.

Using cp

The cp program can be used to clone a disk, one partition at a time. An advantage to using cp is that the filesystem type of the destination partition(s) may be the same or different than the source. For safety, perform the process from a live environment.

注意：This method should not be considered in the same category as disk cloning on the level at which dd operates. Also, it has been reported that even with the **\-a** flag, some extended attributes may not be copied. For better results, rsync or tar should be used.

The basic procedure from a live environment will be:

- Create the new destination partition(s) using fdisk, cfdisk or other tools available in the live environment.
- Create a filesystem on each of the newly created partitions. Example:

```
mkfs -t ext3 /dev/sdb1
```

- Mount the source and destination partitions. Example:

```
mount -t ext3 /dev/sda1 /mnt/source
mount -t ext3 /dev/sdb1 /mnt/destination
```

- Copy the files from the source partition to the destination

```
cp -a /mnt/source/* /mnt/destination
```

\-a: preserve all attributes , never follow symbolic links and copy recursively

- Change the mount points of the newly cloned partitions in /etc/fstab accordingly
- Finally, install the GRUB bootloader if necessary. (See GRUB)

Disk cloning software

Disk cloning in Arch

- Partclone provides utilities to save and restore used blocks on a partition and supports ext2, ext3, ext4, hfs+, reiserfs, reiser4, btrfs, vmfs3, vmfs5, xfs, jfs, ufs, ntfs, fat(12/16/32) and exfat. Optionally, a ncurses interface can be used. Partclone is available in the community repository.
- Partimage, an ncurses program, is available in the community repos. Partimage does not currently support ext4 or btrfs filesystems. NTFS is experimental.

Disk cloning outside of Arch

If you wish to backup or propagate your Arch install root, you are probably better off booting into something else and clone the partition from there. Some suggestions:

- PartedMagic has a very nice live cd/usb with PartImage and other recovery tools.
- Mindi is a linux distribution specifically for disk clone backup. It comes with its own cloning program, Mondo Rescue.
- Acronis True Image is a commercial disk cloner for Windows. It allows you to create a live (from within Windows), so you do not need a working Windows install on the actual machine to use it. After regitratinon of the Acronis software on their website, you will be able to download a Linux based Live cd and/or plugins for BartPE for creation of the Windows based live cd.
- FSArchiver allows you to save the contents of a file system to a compressed archive file. Can be found on the System Rescue CD.
- Clonezilla is an enhanced partition imager which can also restore entire disks as well as partitions.
- Redo Backup and Recovery is a Live CD featuring a graphical front-end to partclone.

外部链接

- Wikipedia:List of disk cloning software
- Arch Linux forum thread

9.4 归档与压缩

难度 进阶 | **篇幅** 18.7 KB | **所属** 数据压缩与归档

本节将讲到

- 仅归档
- 压缩工具
- 用法比较
- 便利工具
- 确定归档格式

根据 Unix 哲学,归档与压缩工具应当是分离的:

- 归档工具将多个文件合并为一个归档文件，例如 *tar*。
- 压缩工具压缩和解压缩数据，例如 *gzip*。

这些工具通常按顺序使用，首先创建一个归档文件，然后压缩它。

当然，也有一些工具可以同时实现这两个功能，它们往往还提供加密、错误检测和恢复功能。

仅归档

名称	包	手册	说明
GNU tar)	tar包	tar(1), info	用于处理无处不在的 tar 归档文件(tarball)的核心工具，pacman 和 AUR 使用这些文件。
libarchive	libarchive包	bsdtar(1) bsdcpio(1)	*tar* 和 *cpio* 的实现,同时也提供了一个库。pacman 和 mkinitcpio 使用该库。
ar)	binutils包	ar(1)	在 *tar* 之前的传统 Unix 归档程序。现在只用于创建静态库文件。
cpio	cpio包	cpio(1)	通过 stdin/stdout 进行文件归档，支持 cpio 和 tar 格式。
DAR	darAUR	dar(1)	归档以备份大型活动文件系统，处理硬链接、扩展属性、稀疏文件和 inode 类型。

提示：GNU 和 BSD tar 都自动为 bzip2、compress、gzip、lzip、lzma、lzop、zstd 和 xz 压缩归档执行解压缩。lz4 只有 BSD tar 支持（但是 GNU tar 可以使用`--use-compress-program=lz4`/`-Ilz4`进行等效操作）。当创建归档文件时，两者都支持`-a`开关，以根据文件扩展名通过正确的压缩程序自动处理创建的归档。BSD tar 根据格式识别压缩格式，而 GNU tar 只根据文件扩展名猜测压缩格式。

参见#仅归档用法。

压缩工具

仅压缩

这些压缩程序实现自己的文件格式。

名称	包	手册	扩展名	Tar 扩展名	说明	并行实现
bzip2	bzip2包	bzip2(1)	.bz2, .bz	.tbz2, .tbz	使用 Burrows–Wheeler 算法。	lbzip2包, pbzip2包
bzip3	bzip3包	bzip3(1)	.bz3	.tbz3	使用 Burrows–Wheeler 算法。	
gzip	gzip包	gzip(1)	.gz, .z	.tgz, .taz	GNU zip, 基于 DEFLATE 算法。	pigz包, bgzip(1) (htslibAUR 的一部分), crabzAUR
lrzip（英语：lrzip）	lrzip包	lrzip(1)	.lrz		改良版的 rzip,使用多种算法。	是多线程的
LZ4)	lz4包	lz4(1)	.lz4		用 C 语言编写，专注于压缩和解压缩速度。	可以是多线程的，尽管 lz4包 不是。查看 https://lz4.github.io/lz4/ 获取替代方案。
lzip	lzip包	lzip(1)	.lz		使用 LZMA.	plzipAUR
lzop	lzop包	lzop(1)	.lzo	.tzo	使用 LZO 库 (lzo包).	
xz	xz包	xz(1)	.xz, .lzma	.txz, .tlz	使用 LZMA, GNU coreutils包 和内核归档文件的默认值。	是多线程的, pixz包, pxz-gitAUR
zstd	zstd包	zstd(1)	.zst		使用 Zstandard 算法。	是多线程的

- 并行实现通过使用多个CPU核心提高速度。
- Tar 扩展名是指使用`tar`和压缩工具的压缩归档，例如`.tzo`相当于`.tar.lzo`。
- 参见#仅压缩用法。

归档和压缩

名称	包	手册	扩展名	说明
p7zip	p7zip包	7z(1)	.7z	7-zip 命令行的第三方 POSIX 移植。
7-Zip	7-zipAUR、7-zip-fullAUR 或 7-zip-fullCNRepo	-	一个高压缩比的文件归档器的上游 Linux 版本。	

名称	包	手册	扩展名	说明
RAR)	rarAUR, unrar包	rar(1)	.rar	rar 格式和工具都是专有的。
t2sz	t2szAUR		.tar.zst .tzst	用 C 写得 Tar 归档工具，使用成员对齐的 zstd 压缩
tarlz	tarlzAUR	tarlz(1)	.tar.lz .tlz	用 C++ 写得 Tar 归档工具，使用成员对齐的 lzip 压缩
ZIP)	zip包, unzip包	zip(1), unzip(1)	.zip	在 Linux 世界之外广泛使用。
Unarchiver	unarchiver包	unar(1), lsar(1)	*many*	Mac 应用程序的命令行工具，支持40多种归档格式。
ZPAQ	zpaqAUR	zpaq(1)	.zpaq	用 C++ 编写的高压缩比归档程序，使用多种算法。
LHa)	lhasa包, lhaAUR	lha(1)	.lzh (在 Amiga 上: .lha)	LZH/LHA 归档，支持 lh7 方法。
WinAce	unace包	unace(1)	.ace	ACE) 文件格式和归档工具都是专有的。

参见#归档和压缩用法。

功能图表

上面的一些工具能够处理多种格式，允许安装更少的软件包。

解压缩

名称	gzip	bzip2	ZIP	LHa/LZH	RAR	compress	CAB	ARJ
gzip包	是	否	部分1	否	否	是	否	否
p7zip包	是	是	是	是	是	否	是	是
unarchiver包	是	是	是	是	是	是	是	部分

1. gzip包 的 *gunzip* 只能解压缩单个成员的 ZIP 文件。

用法比较

仅归档用法

名称	创建归档	提取归档	列出内容	
tar(1)	`tar cfv archive.tar file1 file2`	`tar xfv archive.tar`	`tar -tvf archive.tar`	
cpio(1)	`ls file1 file2	cpio -o > archive.cpio`	`cpio -i -vd < archive.cpio`	`cpio -t < archive.cpio`

仅压缩用法

名称	压缩	解压缩	解压缩到 stdout
bzip2(1)	`bzip2 file`	`bzip2 -d file.bz2`	`bzcat file.bz2`
gzip(1)	`gzip file`	`gzip -d file.gz`	`zcat file.gz`
lrzip(1)	`lrzip file lrztar folder`	`lrzip -d file.lrz lrztar -d folder.tar.lrz`	`lrzcat file.lrz`

名称	压缩	解压缩	解压缩到 stdout
xz(1)	`xz file`	`xz -d file.xz`	`xzcat file.xz`

归档和压缩用法

名称	压缩	解压缩	解压缩到 stdout	列出内容
7z(1)	`7z a archive.7z file1 file2`	`7z x archive.7z`	`7z e -so archive.7z file1`	`7z l archive.7z`
rar(1)	`rar a archive.rar file1 file2`	`rar x archive.rar`	`rar p -inul archive.rar file1`	`rar l archive.rar`
zip(1), unzip(1)	`zip archive.zip file1 file2`	`unzip archive.zip`	`unzip -p archive.zip file1`	`unzip -l archive.zip`
lha(1)	`lha ao7 archive.lzh file1 file2`	`lha x archive.lzh`		简单: `lha l archive.lzh` 详细: `lha v archive.lzh`

便利工具

- **atool** — 用于管理各种类型的文件归档的脚本。

: https://www.nongnu.org/atool/ || atool包

- **dtrx** — 一个智能归档提取工具。

: https://github.com/brettcs/dtrx || dtrxAUR

- **J7Z** — 用 Java 写得 Linux GUI 工具，尝试简化数据压缩和备份。它可以创建 7z, BZip2, Zip, GZip, Tar 归档。

: http://j7z.xavion.name/ || j7zAUR

- **unp** — 命令行工具，可以轻松解包归档文件。

: https://github.com/mitsuhiko/unp || python-unpAUR

- **unpack** — 用于处理多种归档格式的包装器脚本。

: https://github.com/githaff/unpack || unpack-gitAUR

- **zutils** — 用 C++ 编写，包含 zcat(1)/zgrep(1)/zdiff(1)，以像 bsdcat(1) 一样透明地透明地处理压缩和未压缩文件的任何组合。支持 bzip2、gzip、lzip、xz和 zstd。通常与常见（解）压缩器的并行变体（如 pixz(1)）结合使用。

: https://www.nongnu.org/zutils/zutils.html || zutilsAUR

- Bash/函数#提取

确定归档格式

要提取归档，需要确定其文件格式。如果文件名正确，则可以从文件扩展名推断其格式。

否则，您可以使用 file包 工具，参见 file(1)。

小众、罕见或过时的工具

名称	包	扩展名	包
ARC)	arcAUR	.arc, .ark	在拨号 BBS 的早期非常流行。被 ZIP 取代。
ARJ	arj包	.arj	1990年代中期在 DOS/Windows 上使用的归档程序，源中包为一个开源克隆。
compress	ncompress包	.Z	在 gzip 出现之前是 UNIX 压缩工具的事实标准，因使用哈夫曼算法而成功。Man 页面：pack(1)。
PAR2	par2cmdline包	.par2	用于提高数据完整性的奇偶校验归档程序。参见 Parchive（英语：Parchive）。
shar	sharutils包	.shar	用于创建自提取归档。这些自提取归档是有效的 shell 脚本。
Zoo)	zooAUR	.zoo	在 PKZIP 流行之前，在 OpenVMS 操作系统上流行。

文件系统压缩

有些文件系统支持动态压缩文件数据:

- 默认情况下，Btrfs可以配置为压缩单个文件、目录或整个卷。
- 在ZFS上，可以在池或文件系统上启用压缩。

设备映射器压缩

目前正在将开源的 VDO 项目主线化(集成到 Linux 内核项目中)，该项目为提高存储效率提供了一个重复数据消除和压缩设备映射器层。以下包可供选择：

- **vdo** — 用于管理 VDO 卷的用户空间工具

: https://github.com/dm-vdo/vdo || vdoAUR

- **kvdo** — 两个内核模块，提供重复数据消除和/或压缩块存储池

: https://github.com/dm-vdo/kvdo || kvdo-dkmsAUR

压缩库

- **Brotli** — 使用LZ77算法、霍夫曼编码和二阶上下文建模的数据流压缩算法。

: https://github.com/google/brotli || brotli包

- **libzip** — 提供ZIP文件的创建和提取。由 KDE 和 Deepin 代替 zip/unzip 工具使用。

: https://libzip.org || libzip包

- **zlib** — 压缩库中实现了 gzip 和 PKZIP 中的 deflate 压缩方法。

: https://www.zlib.net/ || zlib包

- **Zopfli** — Google 提供的高压缩比文件压缩器，使用了一种名为 zopfli 的 deflate 兼容算法。

: https://github.com/google/zopfli || zopfli-gitAUR

故障排除

日语文件名乱码

Windows 的日文版本使用 Shift-JIS 编码 ZIP 归档。默认情况下，这些归档在解压缩时会受到乱码文件名的影响。要正确解压缩，请在命令行中使用 shift-jis 选项解压缩。

```
$ unzip -O shift-jis nihongo.zip
```

或者也可以使用社区提供的 unzip-natspecAUR 包。

参见

- 应用程序列表/工具#Archive_managers
- 应用程序列表/多媒体#Image_compression
- Wikipedia:Comparison of file archivers
- Wikipedia:List of archive formats
- Wikipedia:Comparison of archive formats

FUSE

本章处理非原生文件系统的落地：FUSE 是底座，Gitfs 把 Git 仓库挂成目录，NTFS-3G 负责 NTFS 分区的挂载、格式化与容量调整。《FUSE》必读，先弄清卸载与现成的 FUSE 文件系统清单；《NTFS-3G》必读，它覆盖从安装、手动挂载、格式化到调整分区大小的全链路；《Gitfs》篇幅最短，只处理 NTFS 数据时可整节跳过。最容易踩的坑是轻视卸载：它排在清单之前，挂载成功只是开始，残留挂载点才是麻烦；另一个误解是格式化与调整分区大小能顺手完成，动手前先看疑难解答。

10.1　Gitfs

难度 入门 | **篇幅** 2.7 KB | **所属** FUSE

本节将讲到

- 安装
- 用法
- 问题解决
- 参见

摘自 gitfs:

: gitfs 是一个与 git 完全集成的 FUSE 文件系统。你可以在本地挂载远程仓库的分支，随后对文件所做的任何修改都会自动提交到远程仓库。 : 您可以挂载任何版本库，您所做的所有改动都会自动转化为提交。gitfs 还会通过模拟每次提交的快照，显示您当前工作分支的历史。 : gitfs 对于那些需要跟踪所有文件，但又无法将所有文件整理成文件提交的地方非常有用。一个用于 git 仓库的 FUSE 文件系统，带有本地缓存。

安装

安装 gitfsAUR。

用法

例如，通过 gitfs，用户可以将远程 git 仓库挂载为FUSE文件系统：

```
$ gitfs https://example.com/repository.git /mount/directory
```

请参阅 options 文档。

问题解决

对 /var/lib/gitfs 的写访问权限

`/var/lib/gitfs` 需要存在，但不是自动创建。此外，如果想以普通用户身份挂载 gitfs，请确保该用户可写：

```
# mkdir /var/lib/gitfs
```

```
# chown username:users /var/lib/gitfs
```

对 pygit2 的写访问权限

如果以普通用户身份运行，首次运行时 gitfs 会尝试进行自我检查，但会失败。要解决这个问题，请以根用户身份运行一次。您不需要实际挂载任何东西。以 root 身份显示帮助信息即可：

```
# gitfs -h
```

与 ssh 密钥一起使用的选项

Gitfs（以及它所依赖的 pygit2）似乎正在进行大量开发，选项也在不断变化。 虽然官方文档说可以使用 `-o key=` 选项来更改密钥，但 AUR 的 0.4.1-1 版本却要求使用 `-o ssh_key=`。 注意，如果密钥受密码保护，gitfs 不会要求输入密码。它只会返回错误信息：

```
_pygit2.GitError: Failed to authenticate SSH session: Callback returned error
```

建议为此创建一个单独的密钥，方法是发出

```
ssh-keygen
/home/user/.ssh/gitfs_rsa
<empty passphrase>
<empty passphrase again>
```

参见

- git worktree

10.2 FUSE

难度 进阶 | **篇幅** 7.5 KB | **所属** FUSE

本节将讲到

- 卸载
- FUSE 文件系统列表
- 另请参见

用户空间中的文件系统 (Filesystem in Userspace) (FUSE) 是一种用于类 Unix 操作系统的机制，它使非特权用户可以创建自己的文件系统而无需编辑内核代码。这是通过在用户空间中运行文件系统代码来实现的，而 FUSE 内核模块仅提供了到实际内核接口的“桥梁”。

卸载

可以使用 fuse3包 提供的 fusermount3(1) 或 fuse2包 提供的 fusermount(1) 卸载 FUSE 文件系统。例如：

```
$ fusermount3 -u mountpoint
```

FUSE 文件系统列表

- **adbfs-rootless** — 挂载通过 USB 连接的 Android 设备。

: https://github.com/spion/adbfs-rootless || adbfs-rootless-gitAUR

- **apfs-fuse** — 用于 APFS（Apple 文件系统 (Apple File System)）的 FUSE 驱动程序。

: https://github.com/sgan81/apfs-fuse || apfs-fuse-gitAUR

- **astreamfs** — 异步流式 fuse 文件系统。(A(synchronous) Stream(ing) (fuse) F(ile)S(ystem))

: https://gitlab.com/BylonAkila/astreamfs/tree/master || astreamfs-gitAUR

- **AVFS** — 虚拟文件系统，可浏览压缩文件（档案）。

: https://avf.sourceforge.net || avfs包

- **CurlFtpFS** — 基于 FUSE 和 libcurl 的用于访问 FTP 主机的文件系统。

: https://curlftpfs.sourceforge.net/ || curlftpfs包

- **davfs2** — 文件系统驱动程序，可让您挂载 WebDAV 文件夹。

: https://savannah.nongnu.org/projects/davfs2 || davfs2AUR

- **EncFS** — 用户空间可堆叠加密文件系统。

: https://vgough.github.io/encfs/ || encfs包

- **fuse-archive** — 将存档或压缩文件作为只读 FUSE 文件系统提供服务。

: https://github.com/google/fuse-archive || fuse-archiveAUR

- **fuseiso** — 以普通用户身份挂载 ISO。

: https://sourceforge.net/projects/fuseiso/ || fuseiso包

- **GDriveFS** — 用于 Google Drive 的创新型 FUSE 包装器。

: https://github.com/dsoprea/GDriveFS || gdrivefsAUR

- **gitfs** — gitfs 是与 git 完全集成的 FUSE 文件系统。

: https://www.presslabs.com/gitfs/ || gitfsAUR

- **gocryptfs** — gocryptfs 是一个用户空间可堆叠的加密文件系统。

: https://nuetzlich.net/gocryptfs/ || gocryptfs包

- **google-drive-ocamlfuse** — 使用 OCaml 编写的 Google Drive 支持的基于 FUSE 的文件系统。

: https://astrada.github.io/google-drive-ocamlfuse/ || google-drive-ocamlfuseAUR

- **gphotofs** — 将相机作为文件系统挂载的 FUSE 模块。

: http://www.gphoto.org/proj/gphotofs/ || gphotofsAUR

- **HubicFuse** — 用于访问 HubiC 云存储的 FUSE 文件系统。

: https://github.com/TurboGit/hubicfuse || hubicfuseAUR

- **iFuse** — 用于访问 iPhone 或 iPod Touch 内容的 FUSE 文件系统。

: https://libimobiledevice.org/ || ifuse包

- **NTFS-3G** — 具有扩展功能的 NTFS 驱动程序。

: https://github.com/tuxera/ntfs-3g || ntfs-3g包

- **s3fs** — 由 Amazon S3 支持的基于 FUSE 的文件系统。

: https://github.com/s3fs-fuse/s3fs-fuse || s3fs-fuse包

- **splitviewfuse** — 将文件拆分/合并为段的目录视图。

: https://github.com/seiferma/splitviewfuse || splitviewfuseAUR\[损坏的链接：package not found\]

- **SSHFS** — 基于 FUSE 的文件系统客户端，用于通过 SSH 挂载目录。

: https://github.com/libfuse/sshfs || sshfs包

- **TMSU** — 一种命令行工具，用于标记文件并通过虚拟文件系统访问它们。

: https://tmsu.org/ || tmsuAUR

- **vdfuse** — 挂载 VirtualBox 磁盘映像 (VDI/VMDK/VHD)。

: https://github.com/muflone/virtualbox-includes || vdfuseAUR

- **vramfs** — 基于 VRAM 的文件系统。

: https://github.com/Overv/vramfs || vramfs-gitAUR

- **wimmount** — 挂载 Windows 映像格式 (WIM) 映像。

: https://wimlib.net/ || wimlib包

- **xbfuse** — 挂载 Xbox (360) 的 ISO。

: https://multimedia.cx/xbfuse/ || xbfuse-gitAUR

- **xmlfs** — 将 XML 文件表示为易于访问的目录结构。

: https://github.com/halhen/xmlfs || xmlfsAUR

- 媒体传输协议#FUSE 文件系统

另请参见

- Wikipedia:Filesystem in Userspace#Applications

10.3 NTFS-3G

难度 进阶 | **篇幅** 14.7 KB | **所属** FUSE

本节将讲到

- 安装
- 手动挂载
- 格式化
- 配置
- 调整NTFS分区大小

NTFS-3G 是微软 NTFS 的一个开源实现，同时支持读和写。NTFS-3G 开发者使用 FUSE 文件系统辅助开发并帮助实现可移植性。

安装

安装下列的软件包：

- ntfs-3g包—用于挂载NTFS分区的FUSE驱动。
- ntfsprogs包\-用户空间工具，用于执行#格式化等操作。

手动挂载

有两种手动挂载 NTFS 分区的方法。传统方法是：

```
# mount /dev/你的_NTFS_分区 /mount/point
```

在 Arch 中不需要显式指定挂载类型 `ntfs-3g`。*mount* 命令默认会使用 `/usr/bin/mount.ntfs`（在安装了 ntfs-3g 软件包之后被符号连接到 `/usr/bin/ntfs-3g`）。

第二种方法是直接调用 `ntfs-3g`：

```
# ntfs-3g /dev/你的_NTFS_分区 /mount/point
```

可用选项参见 ntfs-3g(8)。

格式化

警告：与往常一样，仔细检查设备路径。

```
# mkfs.ntfs -Q -L 卷标 /dev/sdXY
```

注意：`-Q` 通过不向驱动器填充零且不检查坏扇区来加速格式化。

配置

你的 NTFS 分区可以被配置成自动挂载，或者预先配置好来安装你想要的方式挂载。配置可以在文件系统配置文件 fstab 中指定或者使用 udev 规则。

默认配置

使用默认配置会在启动时挂载 NTFS 分区。使用这种方法，如果挂载位置的父文件夹有合适的用户或组权限，用户或组就可以读写这个分区。

把以下内容写入`/etc/fstab`:

```
# <file system>  <dir>  <type>  <options>  <dump>  <pass>
/dev/NTFS-part  /mnt/windows  ntfs-3g   defaults          0       0
```

Linux权限兼容

Linux系统通常将目录的权限设为755，将文件的权限设为644。如果您经常使用NTFS分区，建议保留这些权限。下面的示例将上述权限分配给普通用户：

```
# 安装具有 linux 兼容权限的内部 Windows 分区，即权限755用于目录（dmask=022）和权限644用于文件（fmask=133）
/dev/NTFS-partition  /mnt/windows  ntfs-3g uid=username,gid=users,dmask=022,fmask=133 0 0
```

或者，如果Windows上的权限对你很重要，你可以使用 ntfsusermap(8) 命令将 Windows 用户 映射到 Linux 用户上。ntfs-3g将会处理权限的翻译。

提示：你可能希望将多个 NTFS SID 映射到同一个 Linux UID，例如存在 Syncthing 的 `SyncthingServiceAcct` 服务账户时。从 Windows 到 Linux 的映射很简单，但从 Linux 到 Windows 时，你需要确保由 Linux 用户创建的文件显示为属于你的 Windows 用户，而不是任何服务账户。

首先，你需要知道目标 Windows 用户的 SID。你可以使用 `ntfsusermap` 向导根据文件路径推断 SID，也可以在 Windows 系统上执行 `wmic useraccount get`，获取完整且准确的用户列表。然后，调整 `UserMapping` 文件中的条目顺序，确保 Windows 用户的 SID 位于服务账户的 SID 之前。

允许组/用户

在`/etc/fstab`中，您还可以指定其他选项，如允许访问（读取）分区的用户。例如，您允许`users`组中的人员具有访问权限：

```
/dev/NTFS-partition  /mnt/windows  ntfs-3g   gid=users,umask=0022    0       0
```

默认情况下, 上述命令仅为root用户启用写支持。若要为其他用户启用，必须显示指定应授予写入权限的用户。使用`uid`参数加您的用户名以启用用户写支持：

```
/dev/NTFS-partition  /mnt/windows  ntfs-3g   uid=username,gid=groupid,umask=0022    0
0
```

如果您在一个单用户计算机上运行，您可能希望自己拥有该文件系统并授予所有可能的权限：

```
/dev/NTFS-partition  /mnt/windows  ntfs-3g   uid=username,gid=groupid    0       0
```

基本的 ntfs-3g 选项

对大多数人来说，上面的设置已经足够了。这是一些其他的对于不同的Linux文件系统的通用选项。完整列表参见ntfs-3g(8) § OPTIONS

umask : umask 是一个嵌入的 shell 命令，可以自动设置新创建的文件的权限。对于 Arch Linux，对于 root 和 user 默认的 umask 是 0022。设为 0022 将使新目录有目录权限755，新文件有权限644。你可以在这里查看更多关于 umask 权限的信息：。 **fmask and dmask** : 与 `umask` 类似但是分别定义的是文件和目录的权限。 **noauto** : 如果设置了 `noauto`，`/etc/fstab` 中的 NTFS 条目不会在启动时自动挂载。 **uid** : 用户 id 号码。这允许指定用户具有完全的访问权限。你的uid可以用 `id` 命令获得。

下面的选项只对ntfs-3g有用：

windows_names : 阻止创建拥有不被Windows允许的名字的文件、目录以及扩展属性的。

允许用户挂载

默认情况下，如果*ntfs-3g*在挂载一个块设备，*ntfs-3g*需要 root 权限运行, 即使在`/etc/fstab`中有`user`选项。有关详细信息，请参阅 ntfs-3g-faq。即使在fstab里面有了`user`选项，ntfs-3g仍然需要root权限。

注意：

- ntfs-3g包包没有内置FUSE支持。使用ABS重新构建包以启用FUSE支持。
- 完整的解释是， "user" and "users" work via a setuid `mount` not dropping its setuid privilege so that the block device can be used without root. However, ntfs-3g has a hard-coded restriction in ntfs-3g that bails on setuid if an external libfuse is used.
- 并没有好的技术原因 for not allowing setuid for external FUSE besides a mistrust of the library. This patch removes the said restriction.
- 卸载权限似乎存在问题，因此如果需要卸载文件系统，则仍需要 root 权限。您还可以使用`fusermount -u /mnt/*mountpoint*`来卸载文件系统并避免使用root权限。此外, 如果在`/etc/fstab`中使用`*users*`（复数）而不是`user`选项，您可以使用`mount`和`umount`命令装卸载文件系统。

对于非块设备，比如普通的镜像，命令行下的 *ntfs-3g* 应该可以在普通用户权限下照常使用，因为当无法与内核直接交互时，the underlying FUSE calls 会重定向到 setuid-root *fusermount* 。

调整NTFS分区大小

注意：对重要数据请提前做好备份!

大多数已购买的系统已经有Windows安装在其上，有些人希望在进行 Arch Linux 安装时不要完全擦除它。因此，在某些方面，调整现有 Windows 分区的大小以为 Linux 分区腾出空间是很有用的。这经常通过Live CD或可引导的USB闪存驱动器完成。

对于Live CD，典型的创建过程是下载ISO文件，刻录到CD，然后从它启动。InfraRecorder是一个免费（通过GPL3）的Windows上的CD/DVD刻录应用程序，这是很合适的方法。如果您想要使用可引导的USB驱动器，请参阅USB flash installation media中创建可引导的USB驱动器的方法。

有许多可引导的CD/USB映像可用。此列表不是详尽无遗的, 但是是个很好的开始：

- **GParted** — 为x86计算机设计的小型GNU/Linux发行版。允许你使用最新版GParted应用的所有功能。不包括额外的软件包System Rescue CD，并且磁盘加密可能不受支持。

: http://gparted.sourceforge.net/ || gparted包

- **Parted Magic** — gparted包。使用分区编辑器可以调整大小、复制和移动分区。您可以增长或收缩您的 C: 驱动器，为新操作系统创建空间。尝试从丢失的分区中进行数据抢救。

: https://partedmagic.com/ ||

请注意, 调整 NTFS 分区大小的重要程序包括 ntfs-3g 和类似于 (G)parted 或 fdisk 的实用程序，由util-linux包包提供。除非您是高级用户，否则最好使用像 GParted 这样的工具来执行任何调整大小操作，以尽量减少由于用户错误而导致数据丢失的可能性。

如果您的系统上已经安装了 Arch Linux，并且只想调整现有 NTFS 分区的大小，则可以使用 parted 和 ntfs-3g 包来完成。或者，在安装 GParted 包后, 可以使用 GParted GUI，其本质是调用 ntfsresize(8) 命令

疑难解答

不支持的重解析点

若您在挂载一个Windows 10的NTFS文件系统，并且读取其中的文件和文件夹时出现以下情形：

1. 出现链接到“不支持的重解析点”（unsupported reparse point）的损坏的符号链接，或
2. 出现错误信息：`cannot access *some_file*: Input/output error`（此时`Could not load plugin /usr/lib64/ntfs-3g/ntfs-plugin-80000017.so: Success`在日志中)

其原因是NTFS-3G默认不支持某些类型的重解析点。一些插件可用于部分特性的兼容性，如：

- 系统压缩：也称为“Compact OS”，该功能提供了一种比 NTFS 旧有的 LZ77 压缩更强、针对可执行文件优化的压缩方式。可以使用 ntfs-3g-system-compression-gitAUR 插件获得只读支持，或者在 Windows 中运行 `compact.exe /CompactOS:never` 来禁用此功能。
- 重复数据删除文件：这是 Windows Server 2012 引入的一项功能，提供块级离线重复数据删除。目前尚未在 AUR 中打包。
- OneDrive 文件：OneDrive 文件在 Windows 中存储于特殊卷中。ntfs-3g-onedrive-binAUR 插件仅允许对标记为“在本地可用”的文件进行读写访问。

有关更多详细信息，请参阅此页面；有关下载资源，请参阅 archive.org 存档页面。

损坏的NTFS文件系统

如果NTFS文件系统有错，ntfs-3g会以只读方式挂载它。要修复NTFS系统，启动Windows并使用它的磁盘检查程序，chkdsk。考虑到 ntfsfix 只能修复一些错误，如果失败，chkdsk 可能会成功。

想要修复 NTFS 文件系统，该设备必须已经被卸载。例如，想要修复 `/dev/sda2` 中的 NTFS 文件系统：

```
# umount /dev/sda2
# ntfsfix /dev/sda2
Mounting volume... OK
Processing of $MFT and $MFTMirr completed successfully.
NTFS volume version is 3.1.
NTFS partition /dev/sda2 was processed successfully.
# mount /dev/sda2
```

如果顺利的话，该分区已经可以写入了。

WIndows分区下的中文文件乱码

见简体中文本地化#Windows 分区下的中文文件名乱码

元数据保存在Windows中，拒绝挂载

当与 Windows 8 或 10双引导时,试图挂载一个可见的Windows可能会出现如下错误:

```
The disk contains an unclean file system (0, 0).
Metadata kept in Windows cache, refused to mount.
Failed to mount '/dev/sdc1': Operation not permitted
The NTFS partition is in an unsafe state. Please resume and shutdown
Windows fully (no hibernation or fast restarting), or mount the volume
read-only with the 'ro' mount option.
```

问题是因为Windows 8中引入"快速启动"特性。启用快速启动后，所有分区的元数据的一部分被还原到它们在以前关闭的状态。因此，在 Linux 上所做的更改可能会丢失。这会发生在任何选择"关闭"或"休眠" NTFS 分区的Windows 8 或 10 下。然而，通过选择"重新启动"关闭 Windows 是安全的。

要启用对其他操作系统的系统分区写入，请确保禁用快速重启。通过以管理员身份执行命令:

```
powercfg /h off
```

你可以在*控制面板 >硬件与声音> 电源选项 > 系统设置 > 当电源键按下时做什么*， 去掉勾选*启用快速启动*

删除Windows休眠元数据

作为一个以上干净关机方法的替代方法，有一个办法可以彻底删除休眠后保存的 NTFS 元数据。这个方法只适用于当你不能或不想启动至 Windows，并希望它完全关闭。这个办法是在使用 ntfs-3g包 提供的 ***ntfsfix***。

```
# ntfsfix /dev/你的NTFS分区
```

警告：这个方法意味着已保存的 Windows 会话将彻底丢失。使用该选项后果自负。

挂载失败

如果你按本指南内容操作也无法挂载你的 NTFS 分区，可以尝试一下在 `fstab` 中的所有 ntfs 分区里加上 UUID。参见示例.

Windows上挂载失败

Windows 不会识别一个没有相应的分区类型的NTFS分区。创建一个与 Windows 共用的NTFS分区时的常见陷阱就是忘记设置分区类型为 NTFS。 见 fdisk 或者分区工具。

参见

- ntfs-3g(8)

光盘

光盘在 Arch 上已不是插进去就能用：光驱可能只是要手工挂载的设备，镜像要能当盘读，得先有虚拟光驱。本章处理三件事：用 CDemu 把镜像挂进系统，弄清蓝光的原理与回放，以及实体光驱上的刻录、播放与翻录。

必读的是光盘驱动器：刻录、播放、翻录与疑难解答都在此。蓝光篇幅最长，要动手的却只有回放，原理与参考先扫读，跑不通再补；CDemu 较短，照示例做完即可。最容易踩的坑是把 CDemu 当成实体光驱的替代：它给的是镜像，刻录与真盘读取仍归光盘驱动器；另一种误解是蓝光通读一遍就够用，卡人的常在回放之外。

11.1　CDemu

难度 入门 | **篇幅** 3.7 KB | **所属** 光盘

本节将讲到

- 安装
- 示例

CDemu 是一个虚拟光驱，可以挂载.bin/.cue, .nrg, 或 .ccd 等非ISO-9660格式光盘镜像。`mount` 只能挂载包含单一文件系统的 iso。而许多 cd 都带有复杂的信息，比如混合数据和音轨。CDemu 能获得这些CD镜像完整、原始的内容。

CDemu 利用vhba内核模块模拟出一个SCSI CD/DVD设备，由后台运行的cdemud守护进程(`cdemud`)与该模块通信。镜像分析代码被抽象到一个库中(`libmirage`)，需要支持新的镜像格式时，也便于扩充。守护进程响应来自客户端的dbus命令。CDemu软件包提供了两种可选的客户端：基于命令行的(`cdemu-client`)和GNOME的panel applet——(`gcdemu`).

安装

CDemu 可以通过安装 官方软件仓库中的软件包 cdemu-client包 获得。如果使用自定义内核，不要使用正常的 `vhba` 内核软件包 vhba-module包, 必须使用 DKMS 版本的软件包 vhba-module-dkms包.

启动 Systemd 服务:

```
# systemctl enable cdemu-daemon.service
```

systemd 不会自动加载 CD/DVD 驱动，需要手动加载：

```
 # modprobe -a sg sr_mod vhba
```

图形界面

AUR 中包含了多个图形界面：

- GTK/Gnome gcdemuAUR: 提供 GNOME 面板程序。
- KDE kde-cdemu-managerAUR: 独立程序，与 Dolphin 的动作菜单交互，可以在镜像文件上右键。

示例

将单个镜像加载为第一个设备：

```
$ cdemu load 0 ~/image.mds
```

将多文件镜像加载为第一个设备：

```
$ cdemu load 0 ~/session1.toc ~/session2.toc ~/session3.toc
```

设定字符编码:

```
$ cdemu load 0 ~/image.cue --encoding=windows-1250
```

挂载加密镜像:

```
$ cdemu load 0 ~/image.daa --password=seeninplain
```

卸载第一个设备:

```
$ cdemu unload 0
```

显示设备状态:

```
$ cdemu status
```

显示设备挂载信息:

```
$ cdemu device-mapping
```

Setting daemon debug mask for the first device:

```
$ cdemu daemon-debug-mask 0 0x01
```

Obtaining library debug mask for the first device:

```
$ cdemu library-debug-mask 0
```

Disabling DPM emulation on all devices:

```
$ cdemu dpm-emulation all 0
```

Enabling transfer rate emulation on first device:

```
$ cdemu tr-emulation 0 1
```

Changing device ID of first device:

```
$ cdemu device-id 0 "MyVendor" "MyProduct" "1.0.0" "Test device ID"
```

Enumerating supported parsers:

```
$ cdemu enum-supported-parsers
```

Enumerating supported fragments:

```
$ cdemu enum-supported-fragments
```

Enumerating supported daemon debug masks:

```
$ cdemu enum-daemon-debug-masks
```

Enumerating supported library debug masks:

```
$ cdemu enum-library-debug-masks
```
Displaying daemon and library version:

```
$ cdemu version
```

11.2 蓝光

难度 进阶 | **篇幅** 18.6 KB | **所属** 光盘

本节将讲到

- 原理
- 回放
- 参考

翻译状态：

- 本文（或部分内容）译自 Blu-ray，最近一次同步于 2025-01-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本文旨在帮助 Linux 用户在电脑上播放合法购买的蓝光（Blu-ray）光盘。

注意：鉴于 Linux 上缺乏商业性蓝光播放软件，用户必须使用开源库来处理保护光盘内容的 DRM 方案。这在大多数允许互操作性的国家都是合法的。

原理

蓝光光盘 DRM

与 DVD CSS 一旦特定加密密钥泄露就会被攻破不同，蓝光使用了更强的 DRM 手段，使得其更难以被破解。第一，AACS 标准使用了更复杂的加密流程来保护光盘内容，且一旦密钥泄露，业内可将旧密钥吊销，并通过新光盘分发新的密钥。第二，蓝光具有另一层保护方式：BD+。虽然大部分商业光盘都使用 AACS，有一些额外使用了 BD+。在 2007 年，AACS 系统被黑入，解密密钥被流出到网络上，并由此产生了许多解密软件，但 Linux 用户的兴趣主要集中如何于在电脑上播放合法购买的光盘的能力。虽然业内吊销了最初流出的解密密钥，但新密钥还是时不时被流出，基本上已成了猫鼠游戏。

AACS

AACS 标准和解密流程公开于 。大量相关文章和研究报告位于 ， 和 \[失效链接 2024-07-30 ⓘ\]。

libaacs包 是 VideoLAN 开发团队基于高级内容访问系统（Advanced Access Content System，AACS）标准实现并分发的开源库 。该项目不包含任何可被用于解密版权资源的密钥或证书，但只要结合密钥数据库文件，就可以将其用于播放使用 AACS 标准

的蓝光光盘。该文件名为 `KEYDB.cfg`，会被 libaacs 在 `$XDG_CONFIG_DIRS/aacs` 路径下进行访问。该文件的格式位于 。

AACS 解密流程

认证播放器解密受 AACS 保护的光盘内容分为四个阶段：

1. 软件/嵌入式播放器的设备密钥和光盘的媒体密钥块（Media Key Block，MKB）被用于获取“处理密钥”，该密钥加上来自 MKB 的其它数据被用于计算媒体密钥。
2. 该媒体密钥，加上播放器向光盘出示有效主机证书后获取的光盘卷 ID（Volume ID，VID）被用于生成卷唯一密钥（Volume Unique Key，VUK）。
3. 该 VUK 被用于解密光盘的一系列头密钥。
4. 最后，这些头密钥被用于解密受保护的光盘媒体内容。

注意，这些是包含 MKB 的光盘。从 2006 年第一个蓝光产品发布开始，MKB 就一直在不断更新。最新的 MKB 版本是 78，很多光盘其实有完全相同的 MKB。播放器软件提供主机密钥和证书，而光驱包含了一系列被吊销的主机密钥/证书清单。主机密钥/证书吊销在 MKB 版本比之前播放过的盘更高的新光盘被（尝试）解密/播放时发生（将光盘插入播放器时不会），在该情况下，光驱会失去使用旧主机密钥/证书的能力。

通过使用 libaacs包，可以跳过以上部分阶段到最后一步，使得媒体播放器可以播放光盘。该步骤是通过在 `KEYDB.cfg` 中放入以下任一/两者实现的：

- 一个有效（于光盘 MKB 版本对应）处理密钥和一个有效（即未被光驱吊销）主机密钥/证书
- 针对特定光盘的有效 VUK

如果 libaacs 找到了针对光盘 MKB 版本有效的处理密钥及有效主机密钥和证书，它就可以从步骤 2 开始解密流程。然而，主机密钥/证书会随着新蓝光光盘的传播而周期性被吊销。在吊销后，光驱将无法读取再新的和旧的光盘。该操作通常无法逆转，唯一修复方法为提供更新的主机密钥/证书（对于 Windows 用户就是更新对应的播放器软件）。该方法的优势为，只要主机密钥证书/有效且光盘使用处理密钥已知的 MKB，libaacs 就可以为任意光盘计算 VUK。

幸运的是，在无有效处理密钥及/或主机证书被吊销的情况下，libaacs 还是有办法解密光盘内容：在 `KEYDB.cfg` 文件内提供有效 VUK，使得 libaacs 跳到步骤 3 开始处理。与处理密钥不同，VUK 是特定于单个光盘的，但优势是只要被计算出来，VUK 就无法被吊销。如果 libaacs 使用有效主机密钥/证书成功进行了步骤 2，那它会将步

骤 3 生成的 VUK 存放在 `~/.cache/aacs/vuk`。后续对同一光盘的播放，libaacs 会重用之前保存的 VUK。因此，建议备份这些 VUK，或是在网上进行共享。

已经有多次从不同来源汇编 VUK 的尝试，最早的可在如 Doom9.org 的论坛上找到。随着社群不断组织起来， 上创建了一个中心化的 VUK 数据库，其中包括了超过 24,000 个 VUK，但该网站似乎已不再进行维护。由 FindVUK tool 的作者倡导的新库创建在了 http://fvonline-db.bplaced.net/ ，其中包含超过 150,000 条可下载项，使其成为了最完整的公开可用 VUK 来源。

BD+

BD+ 是蓝光 DRM 的一个可选附加组件。在 2013 年 12 月，VideoLAN 发布了 libbdplusAUR，其提供了实验性 BD+ 解密支持。该库不提供 BD+ 解密所需的密钥或证书，需要你手动在其他地方获取并安装。

BD+ 主要工作方式为在视频流中添加错误，虽不到无法观看的地步，但其持续性的伪影足以令人难以忍受。官方播放器通过使用“修复表”来修复这些问题。“修复表”从网上下载，提供了将损坏的视频流转换为正确视频流的映射。

回放

准备工作

1. 安装 libbluray包 和 libaacs包。
2. 从 下载一个 `KEYDB.cfg` 文件（名称区分大小写）， 并复制到 `~/.config/aacs` 或 `/etc/xdg/aacs` 目录下，其中包含解密超过 90,000 张光盘所需的 VUK 数据。顺带一提，不同语言包含的光盘解密信息是一样的，只有光盘名称被翻译过。光盘数据会不断更新，所以你可能会想定期更新下该文件。
3. 你可以将 提供的处理密钥和主机密钥/证书数据复制到 `KEYDB.cfg` 文件的开头，这些数据适用于 MKB v68 及更旧的光盘。该操作仅适用于未在 VUK 文件中列出的光盘，且只对从未读取过 MKB v70 及以上光盘的光驱有用。
4. 如有需要的话（如卷未被自动挂载），可以用如下命令将光盘挂载到目录下：

```
# mount /dev/sr0 /media/blurays
```

调取光盘信息

libbluray包 附带的 `bd_info` 工具可用于识别光盘使用的加密和 DRM 方案，以及当前配置是否可以对其进行解密。例如：

```
# bd_info /dev/sr0
Using libbluray version 1.2.1
aacs.c:597: Error calculating media key. Missing right processing key ?
mmc.c:386: AACS not active or supported by the drive
```

```
bluray.c:1091: WARNING: BluRay profile 6 BD-J menu support is experimental
Volume Identifier   : TITLE
BluRay detected     : yes
First Play supported: yes
Top menu supported  : yes
HDMV titles         : 8
BD-J titles         : 3
UNSUPPORTED titles  : 0

BD-J detected       : yes
Java VM found       : yes
BD-J handled        : yes
BD-J organization ID: XXXXX
BD-J disc ID        : XX00000000000000000000000XXXXX

AACS detected       : yes
libaacs detected    : yes
Disc ID             : XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
AACS MKB version    : 76
AACS handled        : yes

BD+ detected        : no
...
```

注意，`AACS handled: yes` 表示光盘可被解密，且回放应不成问题。

解密流程

打开例如 VLC 的蓝光播放软件，并播放光盘（以 VLC 为例，选择 *Media* > *Open Disc*，然后进入 *Disc* 标签，选择 *Blu-ray*，并确保选中了 *No disc Menus*）。接着播放软件会进行如下解密操作：

1. 用户使用支持 libbluray 和 libaacs 的播放软件打开蓝光盘
2. 如果蓝光盘没有使用 AACS 加扰，跳到步骤 4.1。
3. 如果蓝光盘使用了 AACS 加扰，那么 libaacs 将：
4. 检查 `~/.cache/aacs/vuk/` 是否存在有效 VUK，如果有，就跳到步骤 4.1；如果没有，继续下一步。
5. 读取 `$XDG_CONFIG_DIRS/KEYDB.cfg`：
6. 如果存在有效 VUK，就跳到步骤 4.1，否则继续下一步。
7. 如果存在有效处理密钥（与光盘 MKB 版本对应），且存在有效（未被光驱吊销）的主机密钥/证书，那么 libaacs 将尝试计算 VUK。计算出的 VUK 将被保存在 `~/.cache/aacs/vuk` 供日后使用，并继续到步骤 4.1。如果找不到有效处理密钥或主机密钥/证书，就跳到步骤 4.2。
8. 结果
9. 播放器软件成功读取到光盘内容。
10. 播放器软件读取光盘内容失败。

使用 VUK 进行解密（步骤 3.1 或 3.2.1）

由于 VUK 是蓝光光盘最下游的解密密钥，因此使用光盘专用的 VUK 将始终有效，而且不会被业界撤销。但是，每张光盘只有唯一的一个 VUK 对应一个 VID，因此这种方法依赖于 VUK 列表或数据库。如果满足以下任意一种情况，就可以得出 VUK：

- 使用处理密钥和主机密钥/证书按下文方法解密过一次，并将得出的 VUK 存放到了 `~/.cache/aacs/vuk`；
- 从第三方获取到了光盘的有效 VUK（即 `KEYDB.cfg` 中的数据）。这使得在处理密钥和主机密钥/证书被光盘 MKB 版本注销的情况下，还能读取光盘内容。

如果不满足以上情况，播放软件会尝试通过第二种方法进行解密。

使用处理密钥和主机密钥/证书进行解密（步骤 3.2.2）

该方法使用了 `KEYDB.cfg` 开头的处理密钥和主机密钥/证书，只有在这两者未被光驱吊销的情况下可用。

注意：从 2019 年 8 月开始，最新的蓝光盘都携带了 MKB v70。鉴于最新公开的处理密钥和主机密钥/证书仅对 MKB v68 及更老的版本可用，该方法只适用于读取过不高于 MKB v68 版本光盘的光驱有效。如果你读取过包含 MKB v70 及以上版本的光盘，那所有公开已知的主机密钥/证书都已被光驱注销，你只能使用上面的方法。

如果该方法可用，libaacs 将在播放光盘后将 VUK 保存在 `~/.cache/aacs/vuk` 下，文件名是光盘 ID，内容是 VUK。即使在没有有效 `KEYDB.cfg` 文件的情况下，VLC 还是会复用该 VUK，因此建议备份该文件夹以供后续使用。

BD+ 支持

libbdplusAUR 提供了实验性 BD+ 解密支持。但如果该方法失败，用户只能使用类似 makemkvAUR 或 DVDFab（在 Wine 下使用）的商业方案。

自 0.2.0 版本起，libbdplus 支持用于校正数据流的缓存表，从而避免了完全模拟 BD+ 虚拟机的需要。

为了让 libbdplus 正常工作，需要进行以下准备：

1. 从 Doom9 论坛 下载 BD+ 虚拟机文件及缓存表存档。
2. 将虚拟机文件移动到 `~/.config/bdplus/vm0/`。
3. 解压缓存表，并移动到 `~/.cache/bdplus/convtab/`。

完成后，在播放受 BD+ 保护的光盘时，libbdplus 应能找到相应的表并修复数据流。

媒体播放器

有些媒体播放器可以使用 libbluray 和 libaacs 来播放 AACS 加密过的蓝光盘。

mplayer

用 mplayer 播放蓝光盘的基本命令是：

```
$ mplayer br:////bluray/mount/dir
```

或者是：

```
$ mplayer br://title number -bluray-device /bluray/mount/dir
```

视频卡顿

你可能需要启用硬件加速和多核 CPU 支持，才能流畅播放蓝光盘。

对于 nvidia 显卡用户，可以通过安装 libvdpau 来启用硬件加速，并对 mplayer 使用 `--hwdec=auto` 选项，例如：

```
$ mplayer --hwdec=auto br:////bluray/mount/dir
```

使用 `--vd-lavc-threads=*N*` 选项来启用多核 CPU 支持，其中 `*N*` 是核心数，例如：

```
$ mplayer --vd-lavc-threads=4 br:////bluray/mount/dir
```

音频语言错误

你可以用 `#` 键查看回放语言列表。

音画不同步

你需要从 mplayer 的第一个输出中，找到蓝光使用的编解码器。它位于 ”Selected video codec“ 一行的末尾。

对于 H.264 光盘，使用 `-vc ffh264vdpau` 选项，例如：

```
$ mplayer -vc ffh264vdpau br:////bluray/mount/dir
```

对于 VC-1 光盘，使用 `-vc ffvc1vdpau` 选项，例如：

```
$ mplayer -vc ffvc1vdpau br:////bluray/mount/dir
```

对于 MPEG 光盘，使用 `-vc ffmpeg12vdpau` 选项，例如：

```
$ mplayer -vc ffmpeg12vdpau br:////bluray/mount/dir
```

mpv

可以通过如下命令播放蓝光盘：

```
$ mpv bd://title/device
```

VLC

使用如下命令进行回放：

```
$ vlc bluray:///bluray/mount/dir
```

VLC 长时加载

如果 VLC 一直在加载，同时没有出现报错信息，请参考 VLC#介质无法加载。

xine

使用如下命令进行回放：

```
$ xine bluray:///bluray/mount/dir
```

排障

KEYDB.cfg 文件缺失

如果 `~/.cache/aacs/vuk` 中存在有效 VUK，则 libaacs 无需 `KEYDB.cfg` 来解密光盘内容。但是，`$XDG_CONFIG_DIRS/aacs/` 下仍必须要有一个 `KEYDB.cfg` 文件（与是否为空无关）。

主机密钥/证书被吊销

不幸的是，当尝试播放较新的蓝光盘时，可能会发生主机密钥/证书（即受认证的软件播放器的密钥）被光驱吊销的情况。出现这种情况时，aacskeysAUR 将返回以下信息：

```
The given Host Certficate / Private Key has been revoked by your drive.
```

这是 AACS 保护方案的一部分：编辑人员可以吊销在互联网上泄露的旧软件播放器主机密钥，并在新的商业光盘上分发这些列表。该操作不可逆，即使重新刷写光驱也无法修复。纠正这种情况的方法只有两种：

- 将 `KEYDB.cfg` 中的主机密钥/证书替换为尚未被吊销密钥/证书
- 参考上文在 `KEYDB.cfg` 中添加光盘特定的 VUK。VUK 无法被业内吊销。

使用 mplayer 或 VLC 将光盘解密成功后，libaacs 会将 VUK 保存在 `~/.cache/aacs/vuk` 中。如果 `KEYDB.cfg` 中的主机密钥/证书随后被吊销，VLC 仍能使用已存储的 VUK，建议备份 `~/.cache/aacs` 目录以供后续使用。

使用 aacskeys

安装 aacskeysAUR。你需要在包含有效主机密钥/证书和处理密钥的文件夹中运行 aacskeysAUR:

```
$ cd /usr/share/aacskeys
```

然后运行：

```
$ aacskeys /bluray/mount/dir
```

例如：

```
$ cd /usr/share/aacskeys && aacskeys /media/blurays
```

你也可以将蓝光盘添加到密钥数据库中：编辑 `$XDG_CONFIG_DIRS/KEYDB.cfg` 并按如下格式添加 aacskeys 的输出：

```
0xunit key file hash = Film Title    | V | 0xvolume unique key
```

aacskeys 无法生成密钥

试下 makemkvAUR，它给出的报错信息相对会更清晰些。

在光驱支持的情况下，你可以尝试刷入定制固件来解除读取限制，也就是“LibreDrive”模式。makemkvAUR 附带的 `sdftool` 可被用于在 Linux 下原生刷写光驱固件。具体选项作用可参考 GUI 封装程序的讨论贴，可帮你决定是否需要使用如 `enc` 等选项。

和其它固件刷写操作一样，该操作有可能会损坏设备，需十分小心。

刷入新固件后，光驱将能允许 VLC 等媒体播放器直接读取光驱，无需使用 makemkv。

如果你有对应的 VUK，但还是没法播放蓝光盘

有些光驱需要加载 sg 内核模块。

```
# modprobe sg
```

参考

下面为一些蓝光盘/HD-DVD 盘的解密方式。用户可在合理使用原则的指导下使用它们对商业蓝光电影进行备份：

- aacskeysAUR - 开源
- makemkvAUR - 闭源/功能有限的免费测试版，提供原生 Linux 版本
- vukextract VukExtract 支持+讨论 - 使用私有 DVDFab 来提取 VUK 的开源软件
- AnyDVD HD\[失效链接 2024-07-30 ⓘ\] - 商业软件，需要 Windows 虚拟机来运行
- DVDFab HD Decrypter - 用于 Windows 下的商业软件，但可以用 Wine 来运行
- Aiseesoft Blu-ray Player - Windows 下的商业播放软件，但可以用 Wine 来运行
- 使用 libaacs 和 libbdplus 来播放蓝光盘 - Doom9 论坛讨论贴

11.3 光盘驱动器

难度 高级 | **篇幅** 39.8 KB | **所属** 光盘

本节将讲到

- 刻录
- 播放
- 翻录
- 疑难解答
- 参见

来自维基百科：

: 在计算机领域，Optical disc drive（ODD）是一种光盘驱动器，它使用激光或可见光频谱内或附近的电磁波作为向光盘读写数据过程的一部分。

刻录

警告：有很大数量的光学驱动器和碟片是有很大不同的。通常，在需要可靠性的时候，推荐使用慢速刻录。如果你遇到了未预期的行为时，尝试使用刻录器所支持的最慢速度刻录。

光盘驱动器的刻录过程包括创建或获得一个图像并将其写入光学介质。该镜像原则上可以是任何数据文件。如果你想挂载生成的介质，那么它通常是一个ISO 9660文件系统的镜像文件。音频和多媒体光盘通常是从 *.bin* 文件刻录的，在 *.toc* 文件或 *.cue* 文件的控制下，告诉所需的轨道布局。

安装刻录工具

如果你想要使用 GUI 程序，见 #Burning CD/DVD/BD with a GUI。

这里列出的程序是面向命令行的。它们是大多数用于CD、DVD和BD的免费GUI程序所使用的后端。当涉及到故障排除或编写刻录活动的脚本时，GUI用户可能会用到它们。

你至少需要一个创建文件系统镜像的程序和一个能够将数据刻录到你所需要的媒体类型的程序。

可用于创建ISO9660类型镜像的应用有:

- mkisofs(8) 自 cdrtools包
- xorriso(1) 和 xorrisofs(1) 自 libisoburn包

常见的选择是 *mkisofs*，因为它的资格最老。

可用于刻录媒体文件的应用有:

- cdrdao(1) 自 cdrdao包 (仅 CD, 格式仅 TOC/CUE/BIN)
- cdrecord(1) 自 cdrtools包
- cdrskin(1) 自 libburn包
- growisofs(1) 自 dvd+rw-tools包 (仅 DVD 和 BD)
- xorriso(1) 和 xorrecord(1) 自 libisoburn包

常见的对于 CD 的选择有 *cdrecord*，而对于 DVD 和 BD 来说，*growisofs* 是常见的选择。因为 *cdrecord* 是第一个不需要解释文件就可以刻录 CD 的程序：*growisofs* 是第一个提供 DVD 与 BD 写入而没有烧录程序的人为限制的。为将 TOC/CUE/BIN 烧录到 CD，安装 cdrdao包。

自由的 GUI 烧录程序至少依赖于一个上述包。

在这份文档中，*xorrisofs* 支持 *mkisofs* 选项。

cdrskin 支持 *cdrecord* 选项 ; *xorrecord* 在不处理音频 CD 时也支持它。

从已存在的文件构建一个 ISO 镜像

先将需要的文件拷贝到一个文件夹，是创建一个 ISO 镜像的最简单办法。例如：`./for.iso`。

然后用 *mkisofs* 生成这个镜像：

```
$ mkisofs -V "ARCHIVE_2013_07_27" -J -r -o isoimage.iso ./for_iso
```

这些选项在下列的节中被解释。

基本选项

`-V`：（分配给）文件系统的名称。ISO 9660 标准限定了 32 字节的长度，同样，这些字符被限制为："A"-"Z"，"0"-"9"以及"_"。卷标签可能展示为挂载点，如果这个介质是被自动挂载的。 **`-J`** : 启用 Joliet) 扩展, 允许特定空间用 Unicode 存储文件名称。(最高每个文件 64 UTF-16 字符)。 **`-joliet-long`** : 增加 Joliet 表命名的最大长度，从 64 到 103 字符。 不符合Joliet规范，通常不受支持。 **`-r`** : 启用 Rock Ridge 扩展，POSIX 文件系统到一个镜像，包括 255 字符长度的文件名和 Unix 风格文件拓展名。 **`-o`** : 为输出的 ISO 镜像设置文件路径。

移植点

可以让 "mkisofs" 查找文件和目录从制定路径

```
$ mkisofs -V "BACKUP_2013_07_27" -J -r -o backup_2013_07_27.iso \
  -graft-points \
  /photos=/home/user/photos \
  /mail=/home/user/mail \
  /photos/holidays=/home/user/holidays/photos
```

`-graft-points` 允许识别由ISO文件系统中的目标地址组成的“路径规范” (如 `/photos`) 和源地址 (如 `/home/user/photos`). 使用 "=" 字符分隔。

这个例子将 文件夹 `/home/user/photos/home/user/mail` and `/home/user/holidays/photos` 分别设为 ISO 镜像中的 `/photos`, `/mail` and `/photos/holidays`。

mkisofs 和 *xorrisofs* 使用相同的参数。使用安全备份，为 *xorrisofs* 加上 `--for_backup`，这将记录事件 ACL 并为每个文件存储 MD5 校验值。

见 mkisofs(8) 和 xorrisofs(1) 手册以获得更多关于命令的参数的信息。

挂载一个 ISO 镜像

你可以挂载一个 ISO 镜像，如果你想要浏览它的文件。 我们可以使用下面的指令挂载一个 ISO 镜像：

```
# mount -t iso9660 -o ro,loop /path/to/file.iso /mount-point
```

浏览完毕后，一定记得卸载：

```
# umount /mount-point
```

无 Root 用户挂载镜像，见 Mounting images as user。

转换 img/ccd 到 iso

转换 `img`/`ccd` 镜像, 你可使用 ccd2iso包:

```
$ ccd2iso ~/image.img ~/image.iso
```

知晓你的设备的名称

为剩余部分，假定你设的记录设备的节为 `/dev/sr0`

用下面的指令检查

```
$ cdrecord dev=/dev/sr0 -checkdrive
```

这将报告这个设备的 `Vendor_info` 和 `Indentification`。

如果没有找到设备，检查一切 `/dev/sr*` 是否存在，当他们提供了读写权限 (`wr-`) 到你的组。

如果没有 `/dev/sr*` 存在，尝试手动加载模块 `sr_mod`。

读取 CD/DVD 的卷名

如果你想获得名称或标签，使用*dd*:

```
$ dd if=/dev/sr0 bs=1 skip=32808 count=32
```

从 CD, DVD 或 BD 中创建 ISO 镜像

为从介质中拷贝数据，并不让空块填充它，先确定它的块/扇区数量与大小（多数为每个 2048）。

```
$ isosize -x /dev/sr0
sector count: 2041796, sector size: 2048
```

或

```
$ isoinfo -d -i /dev/sr0 | awk '/block size|Volume size/'
Logical block size is: 2048
Volume size is: 2041796
```

注意：别忘记把`/dev/sr0`换成你的光驱。

然后用dd用获得的数据拷出来：

```
$ dd if=/dev/sr0 of=discmage.iso bs=sector_size count=sector_count status=progress
```

提示：如果介质坏了，建议用类似于 ddrescue包 这样的专门工具，见 Disk cloning#Using ddrescue 。

如果原介质是可启动的，它的拷贝也将是一个可启动镜像。你可能将它用于虚拟机的虚拟光驱或将它烧录到光学介质，这也将是可启动的。

擦除CD-RW和DVD-RW

使用过的CD-RW介质需要在写上以前记录的数据之前被擦除。这可以通过以下方式完成

```
$ cdrecord -v dev=/dev/sr0 blank=fast
```

清空有两个选项：blank=fast 和 blank=all。完全清空的持续时间与完全写操作一样长。它覆盖了光盘上的有效载荷数据。然而这不应该被认为是安全地使这些数据不可读。为了这个目的，建议用随机数据进行几次全写运行。

替代命令有：

```
$ cdrskin -v dev=/dev/sr0 blank=fast
$ xorriso -outdev /dev/sr0 -blank as_needed
```

擦除 DVD-RW 使用 *dvd+rw-format* 工具来自 dvd+rw-tools包:

```
$ dvd+rw-format -blank=fast /dev/sr0
```

替代命令有：

```
$ cdrecord -v dev=/dev/sr0 blank=fast
$ cdrskin -v dev=/dev/sr0 blank=deformat_sequential_quickest
$ xorriso -outdev /dev/sr0 -blank deformat_quickest
```

这种快速消隐的DVD-RW不适合多时段使用，也不能接受未预知长度的输入流。为此，人们必须使用以下方法之一：

```
$ cdrecord -v dev=/dev/sr0 blank=all
$ dvd+rw-format -blank=full /dev/sr0
$ cdrskin -v dev=/dev/sr0 blank=as_needed
$ xorriso -outdev /dev/sr0 -blank as_needed
```

其他媒体类型要么是一次性写入（CD-R、DVD-R、DVD+R、BD-R），要么是无需擦除的可重写（DVD-RAM、DVD+RW、BD-RE）。

格式化 DVD-RW

格式化过的 DVD-RW 介质可以在不提前擦除的情况下重写。所以考虑在其生命周期内格式化一次。

```
$ dvd+rw-format -force /dev/sr0
$ cdrskin -v dev=/dev/sr0 blank=format_if_needed
$ xorriso -outdev /dev/sr0 -format as_needed
```

不像 DVD-RAM, DVD+RW 和 BD-RE，格式化的 DVD-RW 无法被用于（低速）硬盘直写，而是需要驱动 ptcdvd。见 pktsetup(8)。

格式化 BD-RE 和 BD-R

BD-RE 需要在使用前格式化。当检测到未格式化状态时，烧录程序会自动完成它。然而，数据存储区域的大小可能会受到上述DVD-RW格式命令的影响。

BD-R 可以是格式化的或非格式化的。非格式化的写入将会全程普通速度写入，并提供最大的存储空间。格式化的可以在写入操作时检查，坏块会从空闲块中被替换。这可能会把速度降到一半于普通速度。普通的格式化会使存储空间下降 768MiB。

growisofs 默认会格式化 BD-R，其他的不会。growisofs 可不进行格式化。cdrskin 和 xorriso 可以完全速度写入格式化的 BD-RE 或 BD-R:

```
 $ growisofs -use-the-force-luke=spare:none ...growisofs.or.mkisofs.options...
 $ cdrskin stream_recording=on ...cdrecord.options...
 $ xorriso -stream_recording on ...xorriso.commands...
```

烧录一个 ISO 镜像到 CD、DVD 或 BD。

为烧录一个 ISO 镜像文件 `isoimage.iso` 到光学介质， 为CD:

```
$ cdrecord -v -sao dev=/dev/sr0 isoimage.iso
```

为 DVD 或 BD:

```
$ growisofs -dvd-compat -Z /dev/sr0=isoimage.iso
```

为 CD、BD 或 DVD:

```
$ xorriso -as cdrecord -v dev=/dev/sr0 -sao isoimage.iso
```

注意：

- 当你准备写入时，确保介质未挂载。当介质包含一个可读的文件系统时，管挂载可能是自动的。在最好的情况下，它会防止烧录程序使用烧录设备。在最坏的情况下，可能会因为读取操作弄坏设备。所以，在不能确定的情况下，

```
# umount /dev/sr0
```

- *growisofs* 有个关于 BD-R 空白介质的小 bug。它会报出一个错误信息在烧录完成后。像 *k3b* 这样的程序会认为整个烧录过程失败了。为防止这个问题：
- 这样 `dvd+rw-format */dev/sr0*` 格式化在使用 *growisofs* 之前。
- 或用 *growisofs* 选项 `-use-the-force-luke=spare:none`

验证烧录的ISO镜像

你可以验证烧录的介质的完整性，以确保它不包含错误。在验证之前一定要弹出介质并重新插入。这将保证任何内核缓存不会被用来读取数据。

首先计算原始ISO镜像的MD5校验值：

```
$ md5sum isoimage.iso
 e5643e18e05f5646046bb2e4236986d8 isoimage.iso
```

之后在介质上为每个 ISO 文件系统计算 MD5。不过一些类型的介质完全提交给烧录程序相同数量的数据，很多其他的尾随的垃圾也会被读取。所以你需要限制为 ISO 镜像文件读取的大小。

```
$ blocks=$(expr $(du -b isoimage.iso | awk '{print $1}') / 2048)
$ dd if=/dev/sr0 bs=2048 count=$blocks | md5sum
43992+0 records in
43992+0 records out
90095616 bytes (90 MB, 86 MiB) copied, 0.359539 s, 251 MB/s
e5643e18e05f5646046bb2e4236986d8  -
```

每次执行都需要给出相同的 MD5 校验值（这：`e5643e18e05f5646046bb2e4236986d8`）。如果没有，你可能会一同获得一个 I/O 错误自 `dd` 的运行中。当你感兴趣的时候，dmesg 可能告诉你 SCSI 错误和块编号。

ISO 9660 和运行时烧录

没有必要存储一个应急的 ISO 文件系统在硬盘，在它写到光学介质前。在写到一个空设备缓冲区的时候，只有极老的 CD 驱动器在极老的电脑上运行会遭受错误。

如果你省略了 *mkisofs* 的 `-o` 选项，它将写一个 ISO 镜像到标准输出。这可被用管道导到烧录程序。

```
$ mkisofs -V "ARCHIVE_2013_07_27" -J -r ./for_iso | \
  cdrecord -v dev=/dev/sr0 -waiti -
```

选项 `-waiti` 此处并非必须. 它避免 *cdrecord* 在 *mkisofs* 在其输出前写入到介质。这将允许 *mkisofs* 排除干扰的去读取已开始烧录的介质。见关于多会话的下一节。

在 DVD 和 BD 中, 你可能让 *growisofs* 为你操作 *mkisofs* 并在运行烧录它的输出：

```
$ growisofs -Z /dev/sr0 -V "ARCHIVE_2013_07_27" -r -J ./for_iso
```

多会话

ISO-9960 多会话意味着一个介质使用只读文件系统的同时，依然可以在第一个未被使用的块地址写，并且在这个块上会有新的 ISO 树。这个新树协同于新加的或覆写的文件的数据块。数据的块在旧的 ISO 树中的，无法再被写。

Linux 和其他操作系统将挂载在介质上的最新的会话上的文件夹树。最新的树也将正常的显示旧会话的文件。

使用 cdrecord 的多会话

CD-R 和 CD-RW 在 `-multi` 参数使用时保持可写（或者说可追加）

```
$ cdrecord -v -multi dev=/dev/sr0 isoimage.iso
```

之后，可以用这个下一个会话的参数在介质中查询

```
$ m=$(cdrecord dev=/dev/sr0 -msinfo)
```

通过这些参数的帮助，和驱动器中的可读介质，你可以制作一个附加的 ISO 会话

```
$ mkisofs -M /dev/sr0 -C "$m" \
   -V "ARCHIVE_2013_07_28" -J -r -o session2.iso ./more_for_iso
```

最后添加这些会话到介质并使其再次可追加

```
$ cdrecord -v -multi dev=/dev/sr0 session2.iso
```

cdrskin 和 *xorrecord* 也可做这些到 DVD-R, DVD+R, BD-R 和 未格式化的 DVD-RW 上。*cdrecord* 使多会话可被用于最新的 DVD-R 和 DVD-RW。当然，这也可用于 CD-R 和 CD-RW。

growisofs 和 *xorriso* 也可做这些，并且隐藏大部分复杂操作。

使用 growisofs 的多会话

默认情况下，*growisofs* 使用 *mkisofs* 作为多数创建 ISO 镜像的参数的默认后端。见 *mkisofs* 的例子。它不使用 `-o` 和 `-C`。默认它使用 *mkisofs*。如果你要指定使用其他后端程序，需要设定环境变量 `MKISOFS`：

```
$ export MKISOFS="xorrisofs"
```

这应该在光学介质上建立一个新的 ISO 文件系统，被选项 `-Z` 压缩

```
$ growisofs -Z /dev/sr0 -V "ARCHIVE_2013_07_27" -r -J ./for_iso
```

这应为新会话加入更多文件到一个存在的 ISO 文件系统，被 `-M`压缩

```
$ growisofs -M /dev/sr0 -V "ARCHIVE_2013_07_28" -r -J ./more_for_iso
```

更加详细的信息请参阅 growisofs(1) 手册，以及 *mkisofs* 和 *xorrisofs* 的手册。

使用 xorriso 的多会话

xorriso 应从介质的空白区域建立一个 ISO文件系统。所以它将占用区域，并且，如果这个区域存储了数据，则会让其无效。参数 `-blank as_needed` 可应用于所有可重用媒介甚至是 在硬盘中的 ISO 镜像文件。这不会在应用到空白的单次介质时报错。

```
$ xorriso -outdev /dev/sr0 -blank as_needed \
          -volid "ARCHIVE_2013_07_27" -joliet on -add ./for_iso --
```

如果 `-dev` 而不是 `-outdev` 被使用，在无空白可写介质上，*xorriso* 会添加新的碟片文件。当然，`-blank`在此处不能使用。

```
$ xorriso -dev /dev/sr0 \
          -volid "ARCHIVE_2013_07_28" -joliet on -add ./more_for_iso --
```

详见 xorriso(1) 手册页及其例子。

BD 缺陷管理

BD-RE 和被格式化过的 BD-R 媒介通常会在开启了 BD 缺陷管理的情况下写入。这个特性在数据还存在设备缓冲区时读取这些数据块。为避免低读取质量，这些数据块会被写到（或者说重定向到）空区域。

这种检查读取使写速最多降到 BD 介质或驱动器的极限的一半（有时甚至更坏）。大量使用这些空区域也会导致在读取操作中产生大量延迟。所以，BD 缺陷管理并非总是被人想要开启的。

cdrecord 并不格式化 BD-R。但是，在 BD-RE 介质上避免 BD 缺陷管理没有意义。

growisofs 默认格式化 BD-R。使用参数`-use-the-force-luke=spare:none` 可以避免使用 BD 缺陷管理。但是，在 BD-RE 介质上避免 BD 缺陷管理没有意义。

cdrskin、 *xorriso* 和 *xorrecord* 默认并不格式化 BD-R。它们需要分别使用`cdrskin blank=format_if_needed`、`xorriso -format as_needed`、 `xorrecord blank=format_overwrite`。这三个程序可以在 BD-RE 和已格式化的 BD-R 上关闭 BD 缺陷管理，通过分别使用 `cdrskin stream_recording=on`、`xorriso -stream_recording on`、`xorrecord stream_recording=on`

烧录一个音频 CD

创建你的音频轨，并在不压缩的情况下存储他们为 16 bit、44100 Hz 的立体声 WAV 文件。为将 MP3 文件转换为 WAV 文件，确保 lame包 已经安装。*cd* 到你存储 MP3 文件的目录，并执行

```
$ for i in *.mp3; do lame --decode "$i" "$(basename "$i" .mp3)".wav; done
```

为避免你在烧录由 LAME 转换而来的 WAV 文件时报错，尝试使用 mpg123包 解码：

```
$ for i in *.mp3; do mpg123 --rate 44100 --stereo --buffer 3072 --resync -w $(basename "$i" .mp3).wav $i; done
```

为将 AAC 转换至 WAV，确保 faad2包 已经安装，然后执行：

```
$ for i in *.m4a; do faad $i; done
```

为修复一个已存在的 WAV 文件的码率（或者其他的很多格式），尝试使用包 sox包：

```
$ for i in *.wav; do sox "$i" -c 2 -r 44100 "$(basename "$i" .wav)"-ok.wav; done
```

这些音频文件会被按照它们的文件名，以字母表顺序排列在轨中，如 `01.wav`, `02.wav`, `03.wav` 等等。用以下的指令来模拟烧录 WAV 文件到 音频 CD 中。

```
$ cdrecord -dummy -v -pad speed=1 dev=/dev/sr0 -dao -swab *.wav
```

如果以上操作都正常工作，你可以移除 `-dummy` 来实际烧录 CD。

注意：用以上方法烧录出的碟可能无法在所有的 CD 播放器中播放。对于一个仅音频的碟（丢弃一些音频轨元数据），加入 -audio 标志

使用 cdrdao包 包，执行以下指令，亦可创建“内容表”。

```
$ {
    echo "CD_DA"
    for file in *.wav ; do
      echo "TRACK AUDIO"
      echo "FILE \"$file\" 0"
    done
  } > toc
```

这将使轨道之间不存在间隙。或者，如果您想在某些音轨之间插入 *X* 秒的间隙，您可以编辑 toc 文件，并在该音轨的 TRACK AUDIO 和 file 行之间插入以下行:

PREGAP 00:0*X*:00

然后，烧录文件：

```
$ cdrdao write --speed X toc
```

刻录速度可以调节，更低的烧录速度产生更高质量的结果。这是因为音频 CD 格式较数据存储格式的高级错误矫正更少。

用 MPlayer 来测试新的音频 CD：

```
$ mplayer cdda://
```

烧录一个 BIN/CUE

用以下指令烧录一个 BIN 或 CUE 镜像：

```
$ cdrdao write --device /dev/sr0 image.cue
```

混合模式的 TOC/CUE/BIN 碟

ISO 镜像仅存储数据轨。如果你想要创建一个混合模式的碟（数据轨及多音频轨），你需要创建一对 TOC 及 BIN。

```
$ cdrdao read-cd --read-raw --datafile image.bin --driver generic-mmc:0x20000 --device /dev/cdrom image.toc
```

一些软件只接受一对 CUE 及 BIN，你可以使用 *toc2cue*（包含在 cdrdao包 包中） 创建一个 CUE 表：

```
$ toc2cue image.toc image.cue
```

烧录程序后端的问题

如果你使用中出了问题，你可以向 cdwrite@other.debian.org 请求建议，或尝试向程序的 man 页中所列出的邮箱地址求助。

告知他们你尝试的命令及参数、介质类型（如 CD-R、DVD+RW），以及程序运行时的表现（程序信息、非预期表现等），你可能会被告知尝试获得程序最新版本，或者是开发版本来使测试可以通过。

用 GUI 程序烧录 CD/DVD/BD

这里有一些可用的用于烧录 CD 的 GUI 程序。

参见 Wikipedia:Comparison of disc authoring software。

- **AcetoneISO** — All-in-one ISO 工具 (支持 BIN、 MDF、 NRG、 IMG、 DAA、 DMG、 CDI、 B5I、 BWI、 PDI 和 ISO).

: https://sourceforge.net/projects/acetoneiso || acetoneiso2AUR

- **BashBurn** — 轻量的基于终端目录前端的 CD/DVD 烧录工具

: http://bashburn.dose.se/ || bashburn包

- **Brasero)** — 被尽可能设计的简单的为 GNOME 桌面设计的光盘烧录程序

: https://wiki.gnome.org/Apps/Brasero || brasero包

- **cdw** — *cdrecord*、 *mkisofs*、 *growisofs*、 *dvd+rw-mediainfo*、 *dvd+rw-format* 和 *xorriso* 的 Ncurses 前端。

: http://cdw.sourceforge.net/ || cdwAUR

- **Graveman** — 基于 GTK 的 CD/DVD 烧录程序。。它需要配置到真实的烧录设备上。

: https://graveman.tuxfamily.org/ || gravemanAUR

- **isomaster** — ISO 镜像编辑者。

: http://littlesvr.ca/isomaster || isomasterAUR

- **K3b** — 基于 KDElibs 的多特性的及易用的 CD 烧录和翻录程序。

: https://userbase.kde.org/K3b || k3b包

- **X-CD-Roast** — 写入 CD 和 DVD 用的 *cdrtools* 轻量端。

: http://www.xcdroast.org/ || xcdroastAUR

- **Xfburn** — 支持 CD 和 DVD （-RW）、ISO 镜像，且支持 BurnFree 的，基于 libburnia 库的简单前端。

: https://docs.xfce.org/apps/xfburn/start || xfburn包

- **xorriso-tcltk** — xorriso 的图形前端，用于烧录入 ISO 和 CD/DVD/BD 的工具。

: https://www.gnu.org/software/xorriso/ || libisoburn包

播放

CD

CD的音频播放需要libcdio包 软件包。为了使KDE应用程序如Dolphin能够读取CD中的音频，你需要安装audiocd-kio包。

DVD

如果你想播放加密的DVD，你必须安装诸如libdvd*软件包。

- libdvdread包
- libdvdcss包
- libdvdnav包

除此之外，你还需要安装媒体播放器。热门的DVD播放器包括MPlayer, xine 和 VLC。 参见视频播放上的列表和对于 MPlayer 的特殊介绍。

Blu-ray

参见 Blu-ray#Playback。

翻录

翻录是指把音视频内容从可移除介质或媒体流拷到硬盘中的过程。

音频 CD

控制台

- **abcde** — Comprehensive command-line tool for ripping audio CDs.

: https://abcde.einval.com/ || abcdeAUR

- **cdda2wav** — CD-audio extraction program with paranoia support.

: http://www.cdda2wav.de/ || cdrtools包

- **cdparanoia** — Compact Disc Digital Audio (CDDA) Digital Audio Extraction (DAE) tool.

: https://xiph.org/paranoia/index.html || cdparanoia包

- **cyanrip** — Extensive CD ripper with musicbraniz integration, error detection, log generation, many formats, HDCD decoding, covert art automatic download.

: https://github.com/cyanreg/cyanrip/ || cyanripAUR

- **redumper** — An advanced byte perfect disc dumper supporting incremental dumps, advanced SCSI/C2 repair, intelligent audio CD offset detection and a lot of other features.

: https://github.com/superg/redumper/tree/main/ || redumperAUR

- **ripright** — Minimal CD ripper modeled on autorip.

: https://www.mcternan.me.uk/ripright/ || riprightAUR

- **ripit** — Command-line ripper that supports MusicBrainz, freeddb and various codecs.

: https://www.ripit.pl/ || ripitAUR

- **shnsplit** — Splits .wav and .flac files according to a CUE sheet and encodes the resulting pieces. A useful companion to ABCDE.

: http://shnutils.freeshell.org/shntool/ || shntoolAUR

- **whipper** — CD ripper aiming for accuracy over speed. Uses cdparanoia, MusicBrainz, AccurateRip.

: https://github.com/whipper-team/whipper || whipper包

图形化

- **Asunder)** — GTK-based CD ripping program.

: http://littlesvr.ca/asunder/ || asunder包

- **Audex** — Audio grabber tool for CD-ROM drives based on KDE.

: https://userbase.kde.org/Audex || audex包

- **fre:ac** — Audio converter and CD ripper with support for various popular formats and encoders.

: https://freac.org/ || freacAUR

- **Goobox** — CD player and ripper for GNOME.

: https://people.gnome.org/~paobac/goobox/\[失效链接 2022-09-22 ⓘ\] || goobox包

- **Grip)** — Fast and light CD ripper within the GNOME project that resembles Audiograbber.

: https://sourceforge.net/projects/grip/ || gripAUR.

- **K3b** — Feature-rich and easy to handle CD/DVD burning and ripping application based on KDElibs.

: https://userbase.kde.org/K3b || k3b包

- **KAudioCreator** — KDE program for ripping and encoding Audio-CDs. Unmaintained and does not start anymore.

: https://invent.kde.org/unmaintained/kaudiocreator || kaudiocreator-gitAUR

- **ripperX** — GTK program to rip CD audio tracks and encode them to the Ogg, MP3, or FLAC formats.

: https://sourceforge.net/projects/ripperx/ || ripperxAUR

- **rubyripper** — Audio disc ripper that tries to deliver a secure rip through multiple rippings of the same track and corrections of any differences.

: https://github.com/bleskodev/rubyripper || rubyripperAUR

- **Sound Juicer** — CD ripper for GNOME.

: https://wiki.gnome.org/Apps/SoundJuicer || sound-juicer包

- **soundKonverter** — Front-end to various audio converters.

: https://www.linux-apps.com/content/show.php?content=29024 || soundkonverterAUR

视频 DVD

参见 Wikipedia:Comparison of DVD ripper software.

翻录一个 DVD 的过程常常能拆分为两个子项目：

1. **数据解压**——把音频与视频复制到硬盘，
2. 转码——把解压过后的数据转换为适宜的格式。

一些工具可以同时完成两项工作，其他的则聚焦于其中一项。

控制台

- **dvdbackup** — Tool for pure data extraction which does not transcode. It is useful for creating *exact* copies of encrypted DVDs in conjunction with **libdvdcss** or for decrypting video for other utilities unable to read encrypted DVDs.

: http://dvdbackup.sourceforge.net/ || dvdbackup包

- **FFmpeg** — Complete and free Internet live audio and video broadcasting solution for Linux/Unix, capable to do a direct rip in any format (audio/video) from a DVD-Video ISO image, just select the input as the ISO image and proceed with the desired options. It also allows to downmixing, shrinking, spliting, selecting streams among other features.

: https://ffmpeg.org/ || ffmpeg包

- **HandBrake CLI** — Simple yet powerful video transcoder ideal for batch mkv/x264 ripping.

: https://handbrake.fr/ || handbrake-cli包

- **MEncoder** — Free command line video decoding, encoding and filtering tool. It supports ripping and transcoding DVD-Video.

: http://www.mplayerhq.hu/ || mencoder包

- **subtitleripper** — Convert DVD subtitles into text format.

: https://subtitleripper.sourceforge.net/ || subtitleripperAUR

- **Transcode** — Video/DVD ripper and encoder for the terminal/console. It is outdated/no longer actively maintained.

: || transcodeAUR

- **Vobcopy** — Copies DVD *.vob* files to hard disk, decrypting them on the way.

: https://github.com/barak/vobcopy || vobcopyAUR

图形化

- **dvd::rip** — Front-end to transcodeAUR, used to extract DVD's to the hard disk and transcode or extract and transcode on-the-fly.

: https://www.exit1.org/dvdrip/ || dvdripAUR

- **HandBrake** — Simple yet powerful video transcoder ideal for batch mkv/x264 ripping. GTK version.

: https://handbrake.fr/ || handbrake包

- **K3b** — Feature-rich and easy to handle CD/DVD burning and ripping application based on KDElibs.

: https://userbase.kde.org/K3b || k3b包

- **OGMRip** — An application and a set of libraries for ripping and encoding DVD into AVI, OGM, MP4, or Matroska files using a wide variety of codecs. It relies on mplayer, mencoder, ogmtools, mkvtoolnix, mp4box, oggenc, lame, and faac to perform its tasks.

: http://ogmrip.sourceforge.net/ || ogmripAUR

音频 DVD

- **Python Audio Tools** — Includes dvda2track, which is easy to use command line tool to extract DVD-Audio tracks to uncompressed wav files.

: http://audiotools.sourceforge.net/ || audiotoolsAUR

多格式

- **Aaru** — Software for archival ripping of CDs, DVDs, and harddrives

: https://aaru.app || aaruAUR

疑难解答

Brasero 无法使音频 CD 归一化（normalize）

如果你在第一步“归一化”的时候就出错了，一个解决方法是在在*编辑* > *插件*菜单中关闭归一化插件。

VLC: Error "... 无法打开 /dev/dvd"

如 VLC 如下报错

```
vlc dvdread could not open the disc "/dev/dvd"
```

这可能是因为设备节点 `/dev/dvd` 不在你的系统中。Udev 不再创建 `/dev/dvd`，并用 `/dev/sr0` 替代。你需要通过修改 VLC 配置文件 `~/.config/vlc/vlcrc` 来修复这个问题：

```
# DVD device (string)
dvd=/dev/sr0
```

DVD 驱动器很吵

如果系统因为放 DVD 视频变的非常吵，可能是因为碟转的太快了。执行以下指令调节速度：

```
# eject -x 12 /dev/dvd
```

有时要：

```
# hdparm -E12 /dev/dvd
```

任何光驱支持的速度均可使用（0 是最大速度）。调节 CD-ROM 和 DVD-ROM 光驱速度

播放后端无法在新的电脑（新 DVD 驱动器）上工作

如果你有一台新的电脑（新的 DVD 光驱），而你的播放后端无法工作，那么原因可能是因为区域码没有设置。你可以阅读前链所贴出的文章来用 regionsetAUR 包设置地区码。

这些程序无法翻录或编码一个 DVD 到我的硬盘上

确保你的 DVD 光驱已经正确配置地区。否则，你会得到大量莫名其妙的 CSS 相关错误。用 regionsetAUR 包来配置地区。

如果在地区已经配置好后，翻录仍然不能工作，参考 libdvdcss 开发者文档来启动日志信息，以及设置其他的相关选项。

GUI 程序声称后端出了问题

如果你使用一个GUI程序并遇到问题，而该程序的日志将其归咎于一些后台程序，那么请尝试通过记录的后台程序参数来重现该问题。 无论你是否成功重现，你都可以向#烧录程序后端的问题章节中提到的地方报告记录的步骤和你自己的发现。

特例：介质错误或写错误

这里有一些关于驱动器不适配介质的典型信息。这只能通过使用不同的驱动器或不同的介质来解决。使用不同的程序几乎没有帮助。

使用 growisofs 作为后端的Brasero：

```
BraseroGrowisofs stderr: :-[ WRITE@LBA=0h failed with SK=3h/ASC=0Ch/ACQ=00h]: Input/output error
```

使用 libburn 作为后端的 Brasero：

```
BraseroLibburn Libburn reported an error SCSI error on write(16976,16): [3 0C 00] Write error
```

BD-R DL 50GB 在尝试写第二面时出错

使用 dvd+rw-tools包 包中的 *growisofs* 烧录 50GB BD-R DL 光碟时可能返回一个致命的错误，并损伤数据，如下：

```
$ growisofs -Z /dev/sr0 -J -R -V "label" files
Executing 'mkisofs -J -R -V label files | builtin_dd of=/dev/sr0 obs=32k seek=0'
I: -input-charset not specified, using utf-8 (detected in locale settings)
  0.03% done, estimate finish Fri Jan 29 19:50:36 2016
  0.05% done, estimate finish Fri Jan 29 19:50:36 2016
  0.08% done, estimate finish Fri Jan 29 19:50:36 2016
/dev/sr0: pre-formatting blank BD-R for 49.8GB...
/dev/sr0: "Current Write Speed" is 8.2x4390KBps.
  0.11% done, estimate finish Sat Jan 30 03:29:13 2016
  0.13% done, estimate finish Sat Jan 30 02:10:01 2016
...
 63.20% done, estimate finish Fri Jan 29 20:43:45 2016
:-[ WRITE@LBA=b6d820h failed with SK=3h/WRITE ERROR]: Input/output error
:-( write failed: Input/output error
/dev/sr0: flushing cache
/dev/sr0: closing track
/dev/sr0: closing session
:-[ CLOSE SESSION failed with SK=5h/INVALID FIELD IN CDB]: Input/output error
/dev/sr0: reloading tray
```

这在开始写第二面时发生（即 25GB 左右）。使用 cdrtools包 中的 *cdrecord* 写入没有问题。测试光驱：'HL-DT-ST BD-RE WH16NS40' LG。测试光盘： Verbatim BD-R DL 6x discs (#96911)。FS#47797

光盘托盘自动合上

如果用 `eject` 指令，或是按下光驱的按钮来弹出光盘后，托盘又自动合上导致无法取出光盘，可以尝试这个指令：

```
# sysctl -w dev.cdrom.autoclose=0
```

如果问题解决了，可以按照如下方式将此设置持久化：

```
/etc/sysctl.d/60-cdrom-autoclose.conf
dev.cdrom.autoclose = 0
```

如果以上方法没有生效的话，还有个办法是：用以下的指令，从内核中卸载光盘模块：

```
# rmmod sr_mod
```

这个驱动器现在应该正常了，但系统也不会再装载任何光盘。在把光盘放到光驱后，通过以下指令来重新激活该模块：

```
# modprobe sr_mod
```

光盘就应该装载了。

参见

- 在美国，对物理存在的介质备份在这些情况下允许：关于非法复制 - RIAA。
- Libburnia 项目的主页

备份

分区被误格式化、升级翻车后，能依靠的只有事先的备份。本章讲怎样为 Arch 建立备份，以及怎样恢复。

先读《系统备份》，它对比 Btrfs、LVM 快照、rsync、tar、SquashFS 与可启动的备份；再读《使用 tar 备份整个系统》，讲从 LiveCD 启动、change root 到恢复；《Back In Time》只管桌面文件，可跳。用 tar 打包整个系统得在 LiveCD 里 change root 之后做，否则拿到不一致的副本；只备数据不做可启动的备份，也救不回机器。

12.1　Back In Time

难度 入门 | **篇幅** 2.9 KB | **所属** 备份

本节将讲到

- 安装
- 用法
- 启用计划任务
- 参见

摘自上游:

: Back In Time 是一款易于使用的文件和文件夹备份工具。它可在 GNU Linux 上运行（不能在 Windows 或 OS X/macOS 上运行），并提供一个命令行工具 *backintime* 和一个图形用户界面 *backintime-qt*，这两个工具都是用 Python3 编写的。它使用 rsync 手动或计划快照，并通过 SSH 在本地或远程存储。每个快照都在自己的文件夹中，并带有原始文件的副本，但为了节省存储空间，快照之间会硬链接不变的文件。它的灵感来自 FlyBack。

安装

可以安装 GUI 版本 backintimeAUR 或者 CLI-only 版本 backintime-cliAUR 。

用法

对于图形界面 (GUI)，运行 *backintime-qt*。对于命令行界面（CLI），运行 *backintime*。

对于图形用户界面版本，桌面环境中安装了两个启动程序：*Back In Time* 和 *Back In Time (root)*。后者将以根用户权限启动图形用户界面（英语：Running GUI applications as root）（使用 pkexec(1)），可用于备份当前用户在 `/home` 中的文件以外的文件。

启用计划任务

备份工作的计划任务是通过 cron 实现的。

默认情况下，没有自动启用服务：您需要启用与您安装的 cron 提供程序相关的cron 服务。

参见

- 在 Microsoft GitHub 上的上游项目
- 已知问题与解决方案 (README.md)
- FAQ
- 错误与处理办法
- 邮件列表 *bit-dev*
- 用户手册

12.2 使用 tar 备份整个系统

难度 入门 | **篇幅** 5.4 KB | **所属** 备份

本节将讲到

- 从 LiveCD 启动
- Change root
- 装载其他分区
- 排除文件
- 备份脚本

本篇介绍如何使用 tar 进行全系统备份。

使用 tar 进行备份具有压缩的优点，有助于节省磁盘空间，且使用简单。此过程只需要以下几个步骤：

1. 从 LiveCD 启动
2. Change root 到 Linux 系统安装位置
3. 装载其他（如有的话）分区/驱动器
4. 添加排除对象
5. 使用备份脚本进行备份

为最大限度减少停机时间，如果所有文件系统均驻留在 LVM 卷中，则可以使用LVM 快照在正在运行的系统上进行备份。

从 LiveCD 启动

许多 Linux 可启动 CD、U盘等能够 change root 到系统安装位置。虽然不是必须 change root 才能执行备份，但这提供了运行脚本的功能，而无需将其传输到临时驱动器或必须在文件系统中找到它。Live 媒体必须与已安装的 Linux 系统的架构相同（即 i686 或 x86_64）。

Change root

首先，应该在当前的 Linux 安装系统中设置一个脚本环境。如果您不知道这是什么，这表示您可以执行拥有的任意脚本，它们是常规程序一样。如果您的系统未设置好，请参阅这文档了解如何操作。接下来要做的是 change root，要了解有关 change root 的更多信息，请参阅此文档。Change root 时，不需要装载任何临时文件系统（`/proc`、`/sys` 和 `/dev`）。临时文件系统在系统启动时填充，实际上不需要备份它们，因为它们会干扰正常（和必要的）填充过程，该过程会在任何升级时更改。要 change root，需要装载当前的 Linux 安装的 root 分区。示例：

```
# mkdir /mnt/arch
# mount /dev/your-partition-or-drive
```

使用 `fdisk -l` 查询分区和驱动器。然后 chroot：

```
# cd /mnt/arch
# chroot . /bin/bash
```

警告：不要使用 `arch-chroot` 来 chroot 到目标系统 - 备份过程将会失败，因为这将尝试备份临时文件系统、所有系统内存和其他所有东西。请使用普通的 `chroot`。

此示例明显使用 bash，但也可以使用其他可用的 shell。现在，您应处于脚本环境中（前提是您在输入时有获取 `~/.bashrc`）：

```
~/.bash_profile
# 如果使用 bash，获取本地 .bashrc
source ~/.bashrc
```

装载其他分区

要使用的其他分区（如果有的话）需要装载到适当的位置（例如，如果有单独的 `/home` 分区）。

排除文件

tar 能够忽略指定的文件和目录。语法是每行一个定义。*tar* 还可以理解正则表达式（regexp）。示例：

```
# 排除旧的备份
/opt/backup/arch-full*

# 排除临时文件
/tmp/*
```

```
# 排除 pacman 的缓存
/var/cache/pacman/pkg/
...
```

备份脚本

使用 tar 备份是一个直接过程。以下是一个进行备份的基本脚本，并提供了几项检查。需要修改此脚本以定义备份位置，并排除文件（如果有的话），然后在 `chroot` 和装载所有分区之后运行此脚本。

```
#!/bin/bash
# 全系统备份

# 备份目标位置
backdest=/opt/backup

# 备份名称标签
#PC=${HOSTNAME}
pc=pavilion
distro=arch
type=full
date=$(date "+%F")
backupfile="$backdest/$distro-$type-$date.tar.gz"

# 排除文件位置
prog=${0##*/} # 文件名中的程序名称
excdir="/home/<user>/.bin/root/backup"
exclude_file="$excdir/$prog-exc.txt"

# 检查 chroot 提示
echo -n "来自 LiveCD 的第一个 chroot。确定开始备份？ (y/n)："
read executeback

# 检查排除文件是否存在
if [ ! -f $exclude_file ]; then
  echo -n "没有排除文件，是否继续？ (y/n)："
  read continue
  if [ $continue == "n" ]; then exit; fi
fi

if [ $executeback = "y" ]; then
  # -p、--acls 和 --xattrs 存储所有权限、ACL 和扩展属性。如果没有这些属性，许多程序会停止工作！
  # 可以安全地移除详细信息标志（-v）。如果使用的终端速度较慢，这可以加快备份过程。
  tar --exclude-from=$exclude_file --acls --xattrs -cpvf $backupfile /
fi
```

恢复

要恢复以前的备份，需要装载所有相关分区更改当前工作目录为 root，然后执行

```
$ tar --acls --xattrs -xpf backupfile
```

backupfile 替换为备份存档文件。必须手动删除备份以来添加的所有文件。最简单的方法是重新创建文件系统。

并行压缩备份

要使用并行压缩（SMP），可以使用 pbzip2包（Parallel bzip2）：

```
# tar -cvf /path/to/chosen/directory/etc-backup.tar.bz2 -I pbzip2 /etc
```

将 `etc-backup.tar.bz2` 文件存储到一个或多个脱机媒体上，如U盘、外置硬盘或光盘。有时，需要通过比较原始文件和目录与其备份文件来验证备份的完整性。可以维护备份文件的 hash 列表，以使比较过程更快速。

可以通过在临时工作目录中提取 `etc-backup.tar.bz2` 文件，并根据需要复制单个文件和目录来恢复损坏的 `/etc` 文件。要恢复整个 `/etc` 目录及其所有内容，可以以 root 身份执行以下命令：

```
# tar -xvf etc-backup.tar.bz2 -C /
```

12.3 系统备份

难度 入门 | **篇幅** 5.4 KB | **所属** 备份

本节将讲到

- 使用 Btrfs 快照
- 使用 LVM 快照
- 使用 rsync
- 使用 tar
- 使用 SquashFS

系统备份是备份操作系统、文件和特定于系统的有用的/关键的数据的过程。（系统备份）主要确保除了系统中的用户数据，系统的状态或操作条件也已保存。这有助于使用选择的备份数据将系统恢复到之前保存的状态。

使用 Btrfs 快照

详情请看 Btrfs#快照、#快照与 /boot 分区、Snapper、Timeshift 与 Yabsnap。

使用 LVM 快照

详情请看 LVM#Snapshots、Create root filesystem snapshots with LVM 和 #快照与 /boot 分区。

使用 rsync

详情请看 rsync#作为备份工具。

使用 tar

详情请看使用 tar 备份整个系统。

使用 SquashFS

详情请看 Full system backup with SquashFS。

可启动的备份

在文件系统损坏或系统因更新而损坏的情况下，有一个可启动的备份非常有用。备份还可以启用*testing*软件仓库，并用于测试更新。在您将系统迁移到不同的分区或驱动器之后，只需更新备份的 `/etc/fstab` 和引导加载程序的配置文件，就可以从迁移后的系统启动了。

在您在已将系统备份到另一个驱动器或分区，且当前的引导加载程序工作正常的情况下，接下来的操作将指导您如何从备份的系统启动。

更新 fstab

不需要重新启动，可以通过注释掉或删除任何现有条目来编辑备份的 fstab 文件。为包含备份的分区添加一个条目，如下例所示:

```
/dev/sdaX    /              ext4      defaults                0   1
```

记得使用正确的设备名和文件系统类型。

更新引导程序的配置文件

对于 Syslinux，你只需复制当前项目，不过使其指向不同的磁盘或分区。

提示：除了直接编辑 `syslinux.cfg`，您也可以在启动的时候临时编辑启动菜单。当显示启动菜单的时候，按 `Tab` 键来编辑对应的启动项。分区从1数起，而驱动器从0数起。

对于 GRUB，建议你自动重新生成主配置文件。如果你想将 GRUB 文件全新安装到 `/boot` 以外的位置（例如 `/mnt/newroot/boot`），使用 `--boot-directory` 标志。

请注意检查 `/boot/grub/grub.cfg` 中的启动项，确保其中的 UUID 与新分区相匹配，否则设备将仍然从原系统中启动。您可以通过 lsblk 命令来找到正确的UUID：

```
$ lsblk -no NAME,UUID /dev/sdXY
```

`/dev/sd*XY*` 是您备份的目标分区 （比如 `/dev/sdb3`）要列出所有 GRUB 可启动的分区，请使用 *grep* 命令：

```
# grep UUID= /boot/grub/grub.cfg
```

第一次启动

重启计算机并在引导程序中选择正确的项目。系统应该会在第一次启动的时候检测到所有外部设备并填充根目录下的空文件夹。

你现在可以编辑 `/etc/fstab`，将之前删除的分区和挂载点重新加入。

快照与 /boot 分区

如果你的文件系统支持快照（例如 LVM 和 Btrfs），这些快照将很可能不包含 `/boot` 分区或 EFI 系统分区。

进行内核升级时，您可以用 pacman hook 来把升级后的内核自动复制启动分区到 `root` 分区（请确保该钩子文件为 root 所有）：

```
/etc/pacman.d/hooks/95-bootbackup.hook
[Trigger]
Operation = Upgrade
Operation = Install
Operation = Remove
Type = Path
Target = usr/lib/modules/*/vmlinuz

[Action]
Depends = rsync
Description = Backing up /boot...
When = PostTransaction
Exec = /usr/bin/rsync -a --delete /boot /.bootbackup
```

系统监视器

硬盘温度和主板传感器读数都不在系统面板里，本章用三样工具补上：Hddtemp 报硬盘温度，lm sensors 报电压、温度与风扇转速，Conky 把关心的几项常驻桌面。

以 lm sensors 为必读，它是多数机器上唯一的常驻数据源；Hddtemp 只看安装与使用两段，守护进程、监控器可跳过；Conky 属于展示层，只要数据可整节略过，要美化桌面再回来。最容易踩的坑是以为装完就有数字：lm sensors 首次运行前须先探测并写配置，否则读到的是错的传感器；固态硬盘通常不经过 Hddtemp，读不到温度不算工具坏了。

13.1　Hddtemp

难度 入门 | **篇幅** 4.2 KB | **所属** 系统监视器

本节将讲到

- 安装
- 使用
- 守护进程
- 监控器
- 固态硬盘

Hddtemp 是一个小工具（有守护进程），通过 S.M.A.R.T.（对于支持该功能的驱动器）提供硬盘温度。

提示：访问驱动器温度的一个更通用的选项是 Linux 监测传感器，因为它也处理其他数据。

安装

安装 hddtemp包。

使用

Hddtemp 需要根用户权限。命令 `hddtemp` 后面必须至少有一个驱动器的位置。你可以列出几个用空格分隔的驱动器。

```
# hddtemp /dev/disk/by-id/wwn-0x60015ee0000b237f /dev/sdX2 ... /dev/sdXn
```

注意：`/dev/`下的块设备命名，如`/dev/sdX`，是不一致的。更多信息请参见持久化块设备命名。

更多使用信息见 hddtemp(8)。

守护进程

运行守护进程允许普通用户通过 TCP/IP 访问温度信息。这对脚本和系统监控很有帮助。

该守护进程由 hddtemp.service 控制。

要获取温度，请连接该守护进程，其监听端口为7634。

使用 inetutils包：

```
$ telnet localhost 7634
```

使用 gnu-netcat包:

```
$ nc localhost 7634
```

两种输出都类似于：

```
|/dev/sda|ST3500413AS|32|C||/dev/sdb|ST2000DM001-1CH164|36|C|<br>```
```

为了数据更直观:

$ nc localhost 7634 |sed 's/|//m' | sed 's/||/ \n/g' | awk -F'|' '{print $1 " " $3 " " $4}'

/dev/sda 32 C /dev/sdb 36 C

```
### 覆盖默认磁盘
```

默认的 Hddtemp 守护进程只监控 `/dev/sda`。如果有多个磁盘，需要[覆盖](../../01_安装与包管理/初始化/Systemd.md#Editing_provided_units)默认配置来监控。

如果需要查看哪些硬盘支持监控，可以用 [smartmontools](https://archlinux.org/packages/?name=smartmontools)包 检查。编辑 `hddtemp.service`：

[Service] ExecStart= ExecStart=/usr/bin/hddtemp --daemon --foreground /dev/disk/by-id/wwn-0x60015ee0000b237f /dev/sdb --listen=127.0.0.1

将设备名称更改为需要监控的设备。

编辑完毕后，保存文件并退出编辑器。[Systemd](../../01_安装与包管理/初始化/Systemd.md) 会应用更改并自动重新加载 `hddtemp` 服务。

你也可以使用 [auto-generate](https://github.com/AndyCrowd/auto-generate-configuration-files/blob/master/gen-customexec.conf-hddtemp.sh) 脚本，它会使用 {{pkg|smartmontools}} 检测支持的硬盘，并打印到标准输出文件。

```
## 监控器
```

Hddtemp 可以与[系统监视器](../../06_软件与应用/应用程序/应用程序列表.md#System_monitors)集成。[Conky](Conky.md) 在守护进程模式中已内置对 Hddtemp 的支持。只需启用 `hddtemp.service` 并将 `$hddtemp °C` 添加到 Conky 配置文件。

```
## 固态硬盘
```

Hddtemp 通常从驱动器的智能数据中读取字段`194`。在固态硬盘中，温度信息通常存储在`190`字段。要获取这些信息，可以运行：
或
其中 `/dev/sd*x*` 是驱动器（使用 [lsblk](../系统管理/设备文件.md) 来检查）。

或者在 `/etc/hddtemp.db` 中添加新条目。例如：

13.2 Conky

难度 进阶 | **篇幅** 9.0 KB | **所属** 系统监视器

本节将讲到

- 安装
- 配置
- 字体
- 自启动
- 故障排除

Conky 是一个用于X窗口系统的系统监视软件。它可以运行在 GNU/Linux 和 FreeBSD 上，是一个基于 GPL 协议的自由软件。Conky 可以监控许多系统变量，包括 CPU、内存、交换分区、磁盘空间、温度、top、上传、下载、系统消息以及更多。它具有很高的可配置性，但配置有一些难于理解。Conky 是 torsmo 的一个分支。

安装

- 官方软件库中已经包含了conky包，您可以通过pacman来安装它。

除了官方软件仓库上的 conky包 软件包, 在 AUR 上还有很多关于conky的软件包。

- Conky基本包，没有X11依赖 conky-cliAUR
- Lua支持： conky-luaAUR
- Nvidia和Lua支持： conky-lua-nvAUR

一些在conky变量上的建设需要安装额外的应用才能被使用，例如温度控制的 Hddtemp 和音乐控制的 MPD

你可以编辑`~/.conkyrc`文件来定制您的conky或是使用homeproject-screenshot等其他网站上的范例

附加应用:

- **Conky Manager** — Conky小部件的主题管理器. 它提供开启、关闭选项, 浏览和编辑已经安装的Conky主题.

: https://teejeetech.com/conky-manager/ || conky-manager包

配置

- 当您在编辑配置文件时,点击保存命令可立即看到conky界面的变化.您也没有必要重新登录您的X环境.所以您可以尽情尝试每一个设置，保存配置文件并查看conky界面的变化,然后修改不合适的地方.
- 或者，您可以使用默认配置:

```
$ conky -C > ~/.config/conky/conky.conf
```

当然最好还是使用位于当前用户下~/.conkyrc的配置文件. 就像其他的应用一样, conky会先查看当前用户下的.conkyrc文件.如果检测失败,那么它将使用位于/etc/conky的默认配置文件.

如果您保存配置文件在本地,比如在保存在您的home目录中,您将不能查看任何的日志文件除非您更改一些配置. One of the nice features of conky is to pipe to your desktop some /var/log/ files to read all kinds of log messages.这些文件只能在root身份下查看,然后您需要通过sudo来启动conky.用root身份来启动conky是不推荐的,所以您需要进行以下设置:

```
$ usermod -aG log username
```

将 username 加入 log group. 现在 username 便可以读取日志文件了，您将能够在桌面上使用conky来重定向日志文件。

- 如果conky并没有显现应有的效果 -- 比如 minimum_size -- 您需查看是否是因清空了 /etc/conky/conky.conf中的内容，或是因注释相关字段所造成。

双屏幕

当你使用双屏幕配置时, 你需要进行一些设置来将 *conky* 放置到你想让它呆在桌面的某个位置.

通过调整gap_x, 假设你设置的是1680x1050像素的分辨率，你希望窗口位于左侧显示器的中间顶部，你应使用 :

```
alignment = 'top_left',
gap_X = 840,
```

alignment 的作用是显而易见的， gap_X是从屏幕左边框开始的距离（以像素为单位）。.

xinerama_head是一个可替换的选项，下面将在第二个屏幕的右上角放置“conky”窗口:

```
alignment = 'top_right',
xinerama_head = 2,
```

配置文件语法更改

从conky 1.10以来，配置文件都是用新的lua语法编写的，比如：

```
conky.config = {
  -- Comments start with a double dash
  bool_value = true,
  string_value = 'foo',
  int_value = 42,
}
conky.text = [[
$variable
${evaluated variable}
]]
```

下面的一些示例可能仍然使用旧语法，例如：

```
bool_value yes
string_value 'foo'
int_value 42
```

通过Lua脚本可以从旧语法转换为新的Lua语法。 here.

字体

要用conky显示unicode格式图片和emoji，你需要支持此功能的font 然后将conky配置为需显示的unicode字体. 例如:

```
${font Symbola:size=48}☺${font}
```

符号字体

符号字体常用于更复杂的conky配置，其中一些流行的配置包括；

- ttf-pizzadude-bulletsAUR - PizzaDude Bullets font
- otf-font-awesome包
- ttf-weather-iconsAUR - Erik flowers weather icon font with 222 glyphs

自启动

Conky可以通过几种不同的方式自启动, 一如 "Autostarting"所述. 请选择最适合您的窗口管理器/桌面环境的方式.

Conky有一种配置，使它在后台分支运行。这可能对于某些自动启动设置有效。

In `conky.conf`:

```
conky.config = {
    background = true,
}
```

如果你使用图形桌面环境，并希望通过`conky.desktop` 自启动，请使用以下命令：

```
~/.config/autostart/conky.desktop
[Desktop Entry]
Type=Application
Name=conky
Exec=conky --daemonize --pause=5
StartupNotify=false
Terminal=false
```

`pause=5`参数在“conky”启动时会延时5秒钟，以确保桌面有时间加载并启动。

故障排除

这些是人们在conky发现的问题和他们的解决方案。

Conky启动并且在屏幕上不显示任何内容

首先检查配置文件文本变量中的语法错误。然后再次检查你的用户是否有权运行配置文件中的每个命令，以及是否安装了所有需要的包。

不最小化显示桌面

Using Compiz: 如果 'Show Desktop' 选项或键绑定与所有其他窗口一起和condy最小化, 启动compiz配置设置管理器，转到“General Options”并取消选中“Hide Skip Taskbar Windows”选项。.

如果不使用compiz，请尝试编辑 `conky.conf` 并添加/更改如下:

```
own_window_type = 'override',
```

或者

```
own_window_type = 'desktop',
```

请参阅“conky”帮助文档了解具体差异。但是，后一个选项允许您使用调整大小键绑定（例如OpenBox）将窗口捕捉到“conky”的边界，而第一个选项则没有。

在GNOME Shell集成

有人在GNOME内的*conky*经历了错误.

在 `conky.conf`添加:

```
own_window = true,
own_window_type = 'desktop',
```

避免闪烁

*Conky*需要X服务内的双重缓冲扩展名*(DBE)* 支持来避免闪烁，因为没有它，窗口就无法足够快速的更新窗口. 可以通过 在 `/etc/X11/xorg.conf` 里的 Xorg通过在`"Module"`中添加`Load "dbe"` 选项来启动. `xorg.conf` 文件已经被包含特定配置文件的`/etc/X11/xorg.conf.d` 所替代(1.8.x 版本以上)。只要 *DBE* 存在于`/usr/lib/xorg/modules`它就会被自动加载. 加载模块列表可以使用 `grep LoadModule /var/log/Xorg.0.log`查看.

要启用双重缓冲，请将`double_buffer`选项加入`conky.conf`:

```
conky.config = {
    double_buffer = true,
}
```

你还可以看

- 官方网站
- 官方配置的Conky变量
- 官方Conky对象
- Arch论坛中的Conky配置
- 常见问题

13.3 Lm sensors

难度 高级 | **篇幅** 28.0 KB | **所属** 系统监视器

本节将讲到

- 安装
- 配置
- 运行传感器
- 使用传感器数据
- 提示和技巧

翻译状态：

- 本文（或部分内容）译自 Lm sensors，最近一次同步于 2023-8-26，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

lm_sensors（Linux monitoring sensors，Linux 监控传感器）是一个免费的开源应用程序，提供用于监控温度、电压和风扇的工具和驱动程序。本文档介绍如何安装、配置和使用 lm_sensors。

安装

安装 lm_sensors包 包。

注意：更多文档请访问 GitHub 存储库。以后可能会安装这些，见 FS#48354。

配置

在 root 下使用 *sensors-detect* 来检测并生成内核模块列表：

警告：不要使用默认选项以外的任何选项（只需点击 `Enter`），除非您确切地知道自己在做什么。见 #Laptop screen issues after running sensors-detect.。

```
# sensors-detect
```

它将询问探测各种硬件。默认答案是“safe（安全）”，因此只需对所有问题点击 `Enter` 通常不会造成任何问题。这将创建 `/etc/conf.d/lm_sensors` 配置文件，该文件由 `lm_sensors.service` 用于在启动时自动加载内核模块。

检测完成后，将对探测进行总结。

示例：

```
# sensors-detect
This program will help you determine which kernel modules you need
to load to use lm_sensors most effectively. It is generally safe
and recommended to accept the default answers to all questions,
unless you know what you're doing.
```

```
Some south bridges, CPUs or memory controllers contain embedded sensors.
Do you want to scan for them? This is totally safe. (YES/no):
Module cpuid loaded successfully.
Silicon Integrated Systems SIS5595...                       No
VIA VT82C686 Integrated Sensors...                          No
VIA VT8231 Integrated Sensors...                            No
AMD K8 thermal sensors...                                   No
AMD Family 10h thermal sensors...                           No

...

Now follows a summary of the probes I have just done.
Just press ENTER to continue:

Driver `coretemp':
  * Chip `Intel digital thermal sensor' (confidence: 9)

Driver `lm90':
  * Bus `SMBus nForce2 adapter at 4d00'
    Busdriver `i2c_nforce2', I2C address 0x4c
    Chip `Winbond W83L771AWG/ASG' (confidence: 6)

Do you want to overwrite /etc/conf.d/lm_sensors? (YES/no):
ln -s '/usr/lib/systemd/system/lm_sensors.service' '/etc/systemd/system/multi-
user.target.wants/lm_sensors.service'
Unloading i2c-dev... OK
Unloading cpuid... OK
```

注意：当被问及即将生成 `/etc/conf.d/lm_sensors` 时，如果用户回答 **YES**，则会自动启用 systemd 服务。回答 **YES** 也会自动启动该服务。

运行传感器

示例运行 `sensors`：

```
$ sensors
coretemp-isa-0000
Adapter: ISA adapter
Core 0:       +35.0°C  (crit = +105.0°C)
Core 1:       +32.0°C  (crit = +105.0°C)

w83l771-i2c-0-4c
Adapter: SMBus nForce2 adapter at 4d00
temp1:        +28.0°C  (low  = -40.0°C, high = +70.0°C)
                       (crit = +85.0°C, hyst = +75.0°C)
temp2:        +37.4°C  (low  = -40.0°C, high = +70.0°C)
                       (crit = +110.0°C, hyst = +100.0°C)
```

添加双列直插内存模块（DIMM）温度传感器

要查找 DIMM 的温度传感器，请安装 i2c-tools包 包。安装后，加载 `i2c-dev` 内核模块。

```
# modprobe i2c_dev
```

要显示所有列，在 root 下使用 *i2cdetect*：

```
# i2cdetect -l
i2c-2   smbus         SMBus PIIX4 adapter port 2 at 0b00 SMBus adapter
i2c-2   smbus         SMBus PIIX4 adapter port 1 at 0b20 SMBus adapter
i2c-0   smbus         SMBus PIIX4 adapter port 0 at 0b00 SMBus adapter
```

否则，其输出将显示如下：

```
i2c-2   unknown       SMBus PIIX4 adapter port 2 at 0b00 N/A
i2c-2   unknown       SMBus PIIX4 adapter port 1 at 0b20 N/A
i2c-0   unknown       SMBus PIIX4 adapter port 0 at 0b00 N/A
```

在以下示例中，内存棒连接到总线 `SMBus 0`。*i2cdetect* 命令将显示连接到总线的设备。`-y **0**` 参数使用 `i2c-**0**` smbus。如果需要，检查其他总线。

```
# i2cdetect -y 0
0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- 0c -- -- --
10: 10 -- -- -- -- -- -- -- 18 19 -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- 36 -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 4f
50: 50 51 -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- 77
```

RAM SPD（*s*erial *p*resence *d*etect，串行存在检查）从同一总线上的地址 `0x50` 开始，RAM 温度传感器从 `0x18` 开始。在本例中，有2个 DIMM 可用。地址 `0x18` 和 `0x19` 是 DIMM 温度传感器。

为了读取内存棒的温度，我们需要加载 `jc42` 内核模块。您需要告诉模块要使用哪些地址。此过程包括将 *模块名* 和 *地址* 写入 *`smbus_path`*。例如：

```
# modprobe jc42
# echo jc42 0x18 > /sys/bus/i2c/devices/i2c-0/new_device
# echo jc42 0x19 > /sys/bus/i2c/devices/i2c-0/new_device
```

之后，您的内存棒温度将可见：

```
$ sensors
jc42-i2c-0-19
Adapter: SMBus PIIX4 adapter port 0 at 0b00
temp1:        +50.7°C  (low  =  +0.0°C)                  ALARM (HIGH, CRIT)
                       (high =  +0.0°C, hyst =  +0.0°C)
                       (crit =  +0.0°C, hyst =  +0.0°C)

jc42-i2c-0-18
Adapter: SMBus PIIX4 adapter port 0 at 0b00
temp1:        +51.8°C  (low  =  +0.0°C)                  ALARM (HIGH, CRIT)
                       (high =  +0.0°C, hyst =  +0.0°C)
                       (crit =  +0.0°C, hyst =  +0.0°C)
```

从内存模块读取电涌保护器（SPD）值（可选）

要从内存模块中读取 SPD 计时值，请安装 i2c-tools包 包。安装后，加载 `at24` 或 `eeprom` （已弃用）内核模块。

```
# modprobe at24
```

最后，使用 `decode-dimms` 查看内存信息。

这里是一台机器的部分输出：

```
# decode-dimms
Memory Serial Presence Detect Decoder
By Philip Edelbrock, Christian Zuckschwerdt, Burkart Lingner,
Jean Delvare, Trent Piepho and others

Decoding EEPROM: /sys/bus/i2c/drivers/eeprom/0-0050
Guessing DIMM is in                             bank 1

------ SPD EEPROM Information ------
EEPROM CRC of bytes 0-116                       OK (0x583F)
# of bytes written to SDRAM EEPROM              176
```

```
Total number of bytes in EEPROM                 512
Fundamental Memory type                         DDR3 SDRAM
Module Type                                     UDIMM

------ Memory Characteristics ------
Fine time base                                  2.500 ps
Medium time base                                0.125 ns
Maximum module speed                            1066MHz (PC3-8533)
Size                                            2048 MB
Banks x Rows x Columns x Bits                   8 x 14 x 10 x 64
Ranks                                           2
SDRAM Device Width                              8 bits
tCL-tRCD-tRP-tRAS                               7-7-7-33
Supported CAS Latencies (tCL)                   8T, 7T, 6T, 5T

------ Timing Parameters ------
Minimum Write Recovery time (tWR)               15.000 ns
Minimum Row Active to Row Active Delay (tRRD)   7.500 ns
Minimum Active to Auto-Refresh Delay (tRC)      49.500 ns
Minimum Recovery Delay (tRFC)                   110.000 ns
Minimum Write to Read CMD Delay (tWTR)          7.500 ns
Minimum Read to Pre-charge CMD Delay (tRTP)     7.500 ns
Minimum Four Activate Window Delay (tFAW)       30.000 ns

------ Optional Features ------
Operable voltages                               1.5V
RZQ/6 supported?                                Yes
RZQ/7 supported?                                Yes
DLL-Off Mode supported?                         No
Operating temperature range                     0-85C
Refresh Rate in extended temp range             1X
Auto Self-Refresh?                              Yes
On-Die Thermal Sensor readout?                  No
Partial Array Self-Refresh?                     No
Thermal Sensor Accuracy                         Not implemented
SDRAM Device Type                               Standard Monolithic

------ Physical Characteristics ------
Module Height (mm)                              15
Module Thickness (mm)                           1 front, 1 back
Module Width (mm)                               133.5
Module Reference Card                           B

------ Manufacturer Data ------
Module Manufacturer                             Invalid
Manufacturing Location Code                     0x02
Part Number                                     OCZ3G1600LV2G

...
```

使用传感器数据

图形前端

传感器数据有多种前端。

- **psensor** — GTK 应用程序，用于监测硬件传感器，包括温度和风扇速度。监控主板和 CPU（使用 lm-sensors）、Nvidia GPU（使用 XNVCtrl）和硬盘（使用 hddtemp 或 libatasmart）。

: https://wpitchoune.net/psensor/ || psensor包

- **xsensors** — lm-sensors 的 X11 介面。

: https://github.com/Mystro256/xsensors || xsensors包

- **Netdata** — 基于 Web 的系统监视器（netdata）。

: https://github.com/netdata/netdata || netdata包

- **CoolerControl** — 监测和控制冷却设备的程序。

: https://gitlab.com/coolercontrol/coolercontrol || coolercontrolAUR

特定于桌面环境的：

- **Freon (GNOME Shell extension)** — 用于在 GNOME Shell 中显示 CPU 温度、磁盘温度、显卡温度、电压和风扇转速的扩展。

: https://github.com/UshakovVasilii/gnome-shell-extension-freon || gnome-shell-extension-freonAUR

- **GNOME Sensors Applet** — GNOME 面板的小程序，用于显示硬件传感器的读数，包括 CPU 温度、风扇速度和电压读数。

: http://sensors-applet.sourceforge.net/ || sensors-applet包

- **lm-sensors (LXPanel plugin)** — 通过 lm-sensors 监测 LXDE 中的温度/电压/风扇速度。

: https://danamlund.dk/sensors_lxpanel_plugin/ || sensors-lxpanel-pluginAUR

- **MATE Sensors Applet** — 在 MATE 面板中显示硬件传感器的读数。

: https://github.com/mate-desktop/mate-sensors-applet || mate-sensors-applet包

- **Sensors (Xfce4 panel plugin)** — Xfce 面板的硬件传感器插件

: https://goodies.xfce.org/projects/panel-plugins/xfce4-sensors-plugin || xfce4-sensors-plugin包

- **Thermal Monitor (Plasma 5 applet)** — 用于监控 CPU、GPU 和其他可用温度传感器的 KDE Plasma 小程序。

: https://gitlab.com/agurenko/plasma-applet-thermal-monitor || plasma5-applets-thermal-monitor包

sensord

有一个名为 *sensord* 的可选守护程序（包含在 lm_sensors包 包中），它可以将数据记录到轮询数据库（round robin database，rrd）中，然后以图形方式进行可视化。有关详细信息，请参阅 sensord(8) 手册页。

提示和技巧

调整数值

在某些情况下，显示的数据可能不正确，或者用户可能希望重命名输出。用例包括：

- 由于错误的偏移导致温度值不正确（即报告的温度比实际温度高 20°C）。
- 用户希望重命名某些传感器的输出。
- 核心可能以不正确的顺序显示。

以上所有用例（以及更多用例）都可以通过重写包来进行调整。通过创建 `/etc/sensors.d/*foo*`， 包提供了 `/etc/sensors3.conf` 中的设置，其中任何数量的调整都将重写默认值。建议将 'foo' 重命名为主板品牌和型号，但此命名法是可选的。

多个主板的自定义配置文件可以在 lm-sensors 包的 configs 目录中找到，并用作模板。

注意：不要直接编辑 `/etc/sensors3.conf`，因为包更新将覆盖任何更改，从而丢失这些更改。

示例 1. 调整温度偏移

这是 Zotac ION-ITX-A-U 主板上的一个真实例子。将堆芯温度值降低了20°C（过高），并根据英特尔规格进行了调整。

```
$ sensors
coretemp-isa-0000
Adapter: ISA adapter
Core 0:       +57.0°C  (crit = +125.0°C)
Core 1:       +55.0°C  (crit = +125.0°C)
...
```

使用 `-u` 开关来运行 `sensors`，以查看每个物理芯片有哪些可用选项（原始模式）。如果您看到的一些原始标签似乎是不可配置的，请查看 `/sys/class/hwmon` 目录树。这里提到的每个设备都有一个 `name` 文件，可以用来匹配它引用的设备。然后尝试该目录引用的标签。

```
$ sensors -u
coretemp-isa-0000
Adapter: ISA adapter
Core 0:
  temp2_input: 57.000
  temp2_crit: 125.000
  temp2_crit_alarm: 0.000
Core 1:
  temp3_input: 55.000
  temp3_crit: 125.000
  temp3_crit_alarm: 0.000
...
```

创建以下文件以覆盖默认值：

```
/etc/sensors.d/Zotac-IONITX-A-U
chip "coretemp-isa-0000"
  label temp2 "Core 0"
```

```
  compute temp2 @-20,@-20

  label temp3 "Core 1"
  compute temp3 @-20,@-20
```

现在调用 `sensors` 显示调整后的值：

```
$ sensors
coretemp-isa-0000
Adapter: ISA adapter
Core 0:       +37.0°C  (crit = +105.0°C)
Core 1:       +35.0°C  (crit = +105.0°C)
...
```

示例 2. 重命名标签

这是华硕 A7M266 上的一个真实例子。用户希望温度标签`temp1` 和 `temp2`的名称更加详细：

```
$ sensors
as99127f-i2c-0-2d
Adapter: SMBus Via Pro adapter at e800
...
temp1:        +35.0°C  (high =  +0.0°C, hyst = -128.0°C)
temp2:        +47.5°C  (high = +100.0°C, hyst = +75.0°C)
...
```

创建以下文件以覆盖默认值：

```
/etc/sensors.d/Asus_A7M266
chip "as99127f-*"
  label temp1 "Mobo Temp"
  label temp2 "CPU0 Temp"
```

现在调用 `sensors` 显示调整后的值：

```
$ sensors
as99127f-i2c-0-2d
Adapter: SMBus Via Pro adapter at e800
...
Mobo Temp:        +35.0°C  (high =  +0.0°C, hyst = -128.0°C)
CPU0 Temp:        +47.5°C  (high = +100.0°C, hyst = +75.0°C)
...
```

示例 3. 多 CPU 系统的核心重新编号

这是带有双至强 CPU 的惠普 Z600 工作站上的一个真实例子。物理核心的实际编号不正确：编号为 0、1、9、10，重复到第二个 CPU 中。大多数用户希望堆芯温度按顺序报告，即 0,1,2,3,4,5,6,7。

```
$ sensors
coretemp-isa-0000
Adapter: ISA adapter
Core 0:       +65.0°C  (high = +85.0°C, crit = +95.0°C)
Core 1:       +65.0°C  (high = +85.0°C, crit = +95.0°C)
Core 9:       +66.0°C  (high = +85.0°C, crit = +95.0°C)
Core 10:      +66.0°C  (high = +85.0°C, crit = +95.0°C)

coretemp-isa-0004
Adapter: ISA adapter
Core 0:       +54.0°C  (high = +85.0°C, crit = +95.0°C)
Core 1:       +56.0°C  (high = +85.0°C, crit = +95.0°C)
Core 9:       +60.0°C  (high = +85.0°C, crit = +95.0°C)
Core 10:      +61.0°C  (high = +85.0°C, crit = +95.0°C)
...
```

同样，使用 `-u` 开关来运行 `sensors`，以查看每个物理芯片有哪些可用选项：

```
$ sensors -u coretemp-isa-0000
coretemp-isa-0000
Adapter: ISA adapter
Core 0:
  temp2_input: 61.000
  temp2_max: 85.000
  temp2_crit: 95.000
  temp2_crit_alarm: 0.000
Core 1:
  temp3_input: 61.000
  temp3_max: 85.000
  temp3_crit: 95.000
  temp3_crit_alarm: 0.000
Core 9:
  temp11_input: 62.000
  temp11_max: 85.000
  temp11_crit: 95.000
Core 10:
  temp12_input: 63.000
  temp12_max: 85.000
  temp12_crit: 95.000
$ sensors -u coretemp-isa-0004
coretemp-isa-0004
Adapter: ISA adapter
Core 0:
  temp2_input: 53.000
  temp2_max: 85.000
  temp2_crit: 95.000
  temp2_crit_alarm: 0.000
Core 1:
  temp3_input: 54.000
  temp3_max: 85.000
  temp3_crit: 95.000
  temp3_crit_alarm: 0.000
Core 9:
  temp11_input: 59.000
  temp11_max: 85.000
  temp11_crit: 95.000
Core 10:
  temp12_input: 59.000
  temp12_max: 85.000
  temp12_crit: 95.000
...
```

创建以下文件以覆盖默认值：

```
/etc/sensors.d/HP_Z600
chip "coretemp-isa-0000"
  label temp2 "Core 0"
  label temp3 "Core 1"
  label temp11 "Core 2"
  label temp12 "Core 3"

chip "coretemp-isa-0004"
  label temp2 "Core 4"
  label temp3 "Core 5"
  label temp11 "Core 6"
  label temp12 "Core 7"
```

现在调用 `sensors` 显示调整后的值：

```
$ sensors
coretemp-isa-0000
Adapter: ISA adapter
Core0:        +64.0°C  (high = +85.0°C, crit = +95.0°C)
Core1:        +63.0°C  (high = +85.0°C, crit = +95.0°C)
Core2:        +65.0°C  (high = +85.0°C, crit = +95.0°C)
Core3:        +66.0°C  (high = +85.0°C, crit = +95.0°C)

coretemp-isa-0004
Adapter: ISA adapter
```

```
Core4:         +53.0°C  (high = +85.0°C, crit = +95.0°C)
Core5:         +54.0°C  (high = +85.0°C, crit = +95.0°C)
Core6:         +59.0°C  (high = +85.0°C, crit = +95.0°C)
Core7:         +60.0°C  (high = +85.0°C, crit = +95.0°C)
...
```

自动化 lm_sensors 部署

希望在多台机器上部署 lm_sensors 的用户可以使用以下命令接受所有问题的默认设置：

```
# sensors-detect --auto
```

S.M.A.R.T. 驱动器温度

从内核 5.6 开始 `drivetemp` 模块将通过硬件监控系统报告 SATA/SAS 温度，但 `sensors-detect` 无法自动检测到这一点，因此该模块必须手动加载。

```
# modprobe drivetemp
```

现在，您应该在 `sensors` 的输出中看到类似的条目：

```
sensors
drivetemp-scsi-1-0
Adapter: SCSI adapter
temp1:         +33.0°C

drivetemp-scsi-2-0
Adapter: SCSI adapter
temp1:         +32.0°C  (low  =  +0.0°C, high = +70.0°C)
                        (crit low =  +0.0°C, crit = +70.0°C)
                        (lowest = +29.0°C, highest = +41.0°C)
```

您现在可以在启动时加载模块。或者，手动将其添加到 `/etc/conf.d/lm-sensors` 的 `HWMON_MODULES` 行中。请注意，当 `sensors-detect` 被允许再次写入此文件时，它将不会自动添加。

持久化设备名称

许多软件都希望在 `/sys/class/hwmon/hwmonX` 中找到传感器设备，但通常情况下，提供 hwmon 接口的设备超过 1-2 个的系统上就无法
在 `/sys/class/hwmon/hwmonX` 中正常找到传感器设备了。软件可能应该解析 `hwmon?/name` 或使用 lmsensors 库，但遗憾的是，它们通常不会。一些软件（例如 Monitorix（英语：Monitorix） 或它的某些模块，即 amdgpu）期望在其他地方使用持久名称。

因此，以下类型的 udev 规则可能是有用的。并不是所有的软件都能使用它们（例如，KDE 系统监视器 - 不幸的是，这使得这些软件在许多系统上几乎毫无用处）。在许多情况下，只需在 udev 规则中匹配 hwmon 子系统和合适的名称就足够了，但并不总是如此！有关编写规则的更多信息，请参阅 Udev 页面。

不能在 /sys 层次下重命名或符号链接。 SYMLINK+= -语句也将不起作用。因此，我们需要使用 RUN+= 语句（注意，符号链接不必要像本例中那样位于 /dev 之下 - 既没有标准，也没有合适的位置）。

```
/udev/rules.d/99-persistent-hwmon-names
# 我的主板传感器芯片：
ACTION=="add", SUBSYSTEM=="hwmon", ATTRS{name}=="nct6687", RUN+="/bin/sh -c 'ln -
s /sys$devpath /dev/nct6678'"
# 提供传感器的 USB 设备：
ACTION=="add", SUBSYSTEM=="hwmon", ATTRS{name}=="corsaircpro", RUN+="/bin/sh -c 'ln -
s /sys$devpath /dev/corsaircpro'"
# 我的 GPU：
ACTION=="add", SUBSYSTEM=="hwmon", ATTRS{vendor}=="0x1002", ATTRS{device}=="0x73bf",
RUN+="/bin/sh -c 'ln -s /sys$devpath /dev/rx6900xt'"
```

疑难解答

K10Temp 模块

一些 K10 处理器的温度传感器存在问题。有关更多信息，请参见k10temp文档。

在受影响的机器上，模块将报告“不可靠的 CPU 热传感器；监控已禁用（unreliable CPU thermal sensor; monitoring disabled）”。要强制监控，您可以运行以下命令：

```
# rmmod k10temp
# modprobe k10temp force=1
```

确认传感器实际上是有效和可靠的。如果是，可以编辑 /etc/modprobe.d/k10temp.conf 并添加：

```
options k10temp force=1
```

这将允许模块在引导时加载。

华硕 B450M-A/A320M-K/A320M-K-BR 主板

自2020年11月，这些主板使用的是不受 it87 内核驱动支持的 IT8655E 芯片 。不过，内核驱动的上游版本 支持它。DKMS 变体包含在 it87-dkms-gitAUR 中。

使用 AM4 插槽的华硕 B450/X399/X470 主板

某些新近的华硕主板使用 ITE IT8665E 芯片，访问温度，风扇和电压传感器可能需要 asus-wmi-sensors 模块。从 5.17 开始它成为了主线内核的一部分：加载使用了 UEFI 接口的 asus-wmi-sensors 内核模块，可能需要在某些主板上进行 BIOS 更新 。

或者，it87 模块直接从芯片读取值，安装 it87-dkms-gitAUR 并加载 it87 内核模块。

华硕 H97/Z97/Z170/Z370i/X570/B550 主板

对于某些新近的华硕主板，风扇和电压传感器访问可能需要加载 `nct6775` 内核模块。

你还可能需要添加以下内核参数：

```
acpi_enforce_resources=lax
```

更多信息见 https://bugzilla.kernel.org/show_bug.cgi?id=204807。

注意：从内核 5.16 开始 ，对于大多数主板来说，不再需要上述内核参数，应该避免使用。

华擎 Deskmini H470

Deskmini H470 的 STX 主板使用 NCT6683 芯片，为了访问温度、风扇和电压传感器，需要加载 `nct6683` 模块。

为了获得 `nct6683` 模块的正确值，请创建一个模块配置文件：

```
/etc/modprobe.d/nct6683.conf
options nct6683 force=1
```

技嘉 B250/Z370/B450M/B560M/B660M/Z690 主板

自2019年5月，一些技嘉主板使用 ITE IT8686E, ITE8689 (用于 B560 and B660M) 或 ITE8689E (用于 Z690) 芯片，it87 内核驱动不支持这些芯片 。不过，内核驱动的上游版本 支持它。DKMS 变体包含在 it87-dkms-gitAUR 中。与 #华硕 H97/Z97/Z170/Z370i/X570/B550 主板一样，在尝试安装模块之前需要内核参数：

```
acpi_enforce_resources=lax
```

此外，在加载模块时提供芯片的 id，如下所示：

```
# modprobe it87 force_id=0x8686
```

或

```
# modprobe it87 force_id=0x8689  # for B560
# modprobe it87 force_id=0x8628  # for Z690
```

或者你可以在启动过程中通过创建以下两个文件加载模块：

```
/etc/modules-load.d/it87.conf
it87
```

对于 Z690

```
/etc/modprobe.d/it87.conf
options it87 force_id=0x8628
```

对于其它的

```
/etc/modprobe.d/it87.conf
options it87 ignore_resource_conflict=1
```

一旦模块被加载，您就可以使用 *sensors* 工具来探测芯片。

现在，您还可以使用风扇转速控制来控制机箱风扇的转速级数。

可选地,安装 zenpower3-dkmsAUR 可以允许对主板的冷却系统进行更大的微调。不过，它会禁用默认的 k10temp 模块。

技嘉 GA-J1900N-D3V

该主板使用 ITE IT8620E 芯片（也可用于读取电压、主板温度和风扇速度）。自2014年10月，lm_sensors 不支持芯片 ITE IT8620E 的驱动程序 。lm_sensors 的开发人员报告称，该芯片与用于硬件监控的 IT8728F 部分兼容。然而，自2016年8月， 列出了支持的 IT8620E。

你以在运行时使用 modprobe 加载模块：

```
$ modprobe it87 force_id=0x8728
```

或者你可以在启动过程中通过创建以下两个文件加载模块：

```
/etc/modules-load.d/it87.conf
it87
/etc/modprobe.d/it87.conf
options it87 force_id=0x8603
```

一旦模块被加载，您就可以使用 *sensors* 工具来探测芯片。

现在，您还可以使用风扇转速控制来控制机箱风扇的转速级数。

运行 sensors-detect 后笔记本电脑出现屏幕问题

这是由于 lm-sensors 在探测传感器时干扰了屏幕的 Vcom 值造成的。论坛上已经讨论并解决了这个问题： https://bbs.archlinux.org/viewtopic.php?id=193048 。不过，在运行任何建议的命令之前，请确保仔细阅读帖子。

在 AMD Navi 2 GPU 上的 i2c 总线错误

目前，在 AMD Navi 2 GPU 上内核处理 i2c 总线读取的方式中存在一个错误。该总线目前只能与 EEPROM 一起使用，尝试将其与其他设备一起使用会导致其故障。这可能会导致崩溃、黑屏，甚至导致显卡的异常行为，比如无法切换电源状态。如果您有基于 Navi 2 的显卡，目前建议不要扫描 i2c 总线。您可以在此处阅读更多信息：https://gitlab.freedesktop.org/drm/amd/-/issues/1470

同步

误删、覆盖随时会发生——本章要解决的就是把数据可靠复制到另一处，以及改动之后只搬运变化的部分。建议先读《同步和备份程序》，弄清备份、数据同步、增量备份与版本控制的分工，「也可查阅」留作检索；再读《Rsync》，安装与作为备份工具两节必读，「我自己的基于 rsync 的快照式备份过程」可直接照搬，文件系统克隆可跳读。

最容易踩的坑是把同步当备份：rsync 连源端的删除一并同步，一次误删就被复制到备份端，真正兜底的是保留历史的那一层；把增量备份当成版本控制也是误解，前者省的是流量和空间，后者才管追溯。

14.1　同步和备份程序

难度 高级 | **篇幅** 25.4 KB | **所属** 同步

本节将讲到

- 备份概览
- 数据同步
- 增量备份
- 版本控制系统
- 也可查阅

这个页面列出并比较了在两个或多个位置之间同步数据的应用程序，以及在此功能之上建立的以备份为目的的制作重要数据的增量副本的应用程序。因为它们的关系，这两组程序共享许多特征所有在同一篇文章里解释描述它们.

备份概览

备份重要数据是必须采取的措施，因为人和机器的处理错误随着时间推移非常可能产生损坏,并且存储数据的物理媒体也不可避免的注定损坏. 为了选择满足每个人的需求的程序，下面的一些问题要考虑:

- 存储数据的备份媒介, 比如. CD, DVD, 远程服务器, 外部硬盘, 等的种类.
- 计划的备份频率, 比如. 每天, 每周, 每月, 等等.
- 希望从备份获取的功能，比如，压缩，加密，处理重命名等.
- 如果需要的话存储重命名的方式.

数据同步

这些应用做的只是以一种“镜像”的方式在多地点/多机器间保持目录同步。 尽管如此，它们中的大多数仍然运行存储和转换到老版本的修改过的或者删除的文件.

也可查阅:

- List of applications/Utilities#File synchronization and backup
- List of applications/Internet#Cloud synchronization clients
- Wikipedia:Comparison of file synchronization software

说明

名字: 应用的名字，链接到archwiki文章或者官方网站. **包**: 到包的链接. **实现**: 应用程序基于的编程语言，库或者实用程序. **增量传输**: 仅仅文件修改过的*部分*会被传输. **加密传输**: 当通过网络传输时默认加密. **FS元数据**: 文件的权限和属性是同步的 . **可恢复**: 同步能继续，如果被打断的话. **处理重命名**: 移动过/或重命名过的文件会被监测并不会存储或者传输两次. 它通常意味着会计算文件或文件块的校验和.程序如果没有这项功能能通过和 hsyncAUR结合起来来实现, 这个程序*只做*同步命名. **版本控制**: 旧版本的文件也被备份了 (*反向增量备份*). **改变传播**: 指定能传播多少地点.

- *单向*意味着两地点的单向传播,
- *双向*意味着两地点的双向传播
- *多向*意味着多地点的完全同步.

冲突解决方案: 这个程序会要么自动要么交互的处理文件冲突, 即是它不静默的丢弃冲突文件. 这项属性不适应于只支持单方向传播的程序. **FS监听**: 应用程序监听文件系统的变化来触发同步. **CLI**: 应用程序提供命令行界面. **其它界面**: 应用有的特殊用户界面, 比如. GUI, TUI, 或者基于网页. **证书**: 服务器程序和客户端程序的证书. **其它平台**: 不仅仅支持Linux. **维护**: 项目还在被维护. **特性**: 特别是能将应用程序和其它的区分开的特性的说明.

表格

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：明确列出了支持的协议 / 工具. 丢弃 *加密传输* 这一列 因为它取决于协议/工具. 简化并给*改变传输*列上色 并把它向左移. 拆分未维护到子分区? 查阅 User:Larivact/drafts/Synchronization and backup programs#Synchronization.（在Talk:同步和备份程序讨论）

名字	安装包	实现	增量传输	加密传输	FS元数据	可恢复	处理重命名	版本控制	改变传播	冲突解决方案	FS监听	CLI	其它界面	证书	其它平台	维护	特性
FreeFileSync	freefilesyncAUR	C++	?	SFTP	?	?	Yes	Yes	**uni**directional / **multi**directional	是	?	否	是	GPL	Windows,	是	

名字	安装包	实现	增量传输	加密传输	FS元数据	可恢复	处理重命名	版本控制	改变传播	冲突解决方案	FS监听	CLI	其它界面	证书	其它平台	维护	特性
															macOS		
git-annex	git-annex包	Haskell, git	rsync	rsync	?	?	?	是	**multi**directional; with git remotes	重命名冲突文件	?	是	git-annex assistant	GPLv3	macOS, Android	是	用git管理文件
osync.sh	osyncAUR	Bash, based on rsync	rsync	rsync	?	是	否	是	**bi**directional	保存多版本的文件	可选的	是	否	BSD		是	
rclone	rclone包	Go	否	?	?	?	?	?	**uni**directional	?	?	是	Rclone Browser	MIT	*BSD, Plan 9, Solaris, Windows, macOS	是	针对与云存储同步进行了优化,表现因远程位置支持的特性而异.
rdiff-backup	rdiff-backup包	Python 2, libr	rsync	rsync	是	?	否	是	**uni**directional	否	是	否	GPL	Win32	?		

名字	安装包	实现	增量传输	加密传输	FS元数据	可恢复	处理重命名	版本控制	改变传播	冲突解决方案	FS监听	CLI	其它界面	证书	其它平台	维护	特性
		sync															
Resilio Sync	rslsyncAUR	C++	是	是	?	是	?	是	**multi**directional	?	?	否	Web	Proprietary freemium	FreeBSD, Windows, macOS, Android, iOS, Windows Phone, Amazon Kindle Fire	是	P2P同步
Rsync	rsync包	C	是	SSH or native protocol	是	是	否	 --link-dest with hard links --backup									

增量备份

那些能增量备份的程序会记住并考虑上次账户运行期间备份的数据(所谓的 "差异") 并消除重复未更改数据的需要.将数据还原到特定时间点需要定位上次完整备份和所有增量备份到赢应恢复的时刻 . 这种备份方法对经常备份的人很有用.

可查阅:

- Wikipedia:List of backup software
- Wikipedia:Comparison of backup software

- Wikipedia:Comparison of online backup services

说明:

- 名字: 应用名字, 链接到archwiki文章或者官方网站.
- 包: 链接到安装包.
- **实现**: 程序基于的编程语言、库或者实用工具.
- **压缩存储**: 用作存储的压缩方法.
- **加密存储**: 加密被用作存储.
- **增量传输**: 只有文件修改过的*部分*会被传输.
- **加密传输**: 当通过网络传输时数据默认加密.
- **文件系统元数据**: 文件系统权限和属性也备份了.
- **易访问**: 备份明确地存储在文件系统中，或者可以挂载.
- **可恢复**: 如果中断备份可以不重启继续.
- **处理重命名**: 移动/重命名的文件会被监测并不会存储或转移两次; 它通常意味着它会计算文件或文件块的校验和.
- **命令行**: 应用程序是命令行驱动的，即它可用来编写脚本.
- **其它界面**: 这个应用有其它特定的用户界面，比如 GUI, TUI, 或者基于网络的接口.
- **证书**: 服务器或客户端程序的证书.
- **其它平台**: 除了Linux之外支持的操作系统.
- **维护**: 这个项目是否还在维护.
- **特性**: 特别是能将应用程序和其它的区分开的特性的说明.

单机器

这些程序针对把数据从它们安装的机器备份, 尽管备份目的可能在外部机器或者是存储媒体.

基于块的增量

如果一个文件被修改, 下次快照这些程序只存储修改过的*部分* . 与 #基于文件的增量的程序相比, 它们在磁盘方面效率更高, 特别是大文件但是只有小修改; 另一方面,存档的快照必须能用创造它们的备份程序打开 , 因为文件必须从二进制差异中重建.

名字	包	实现	压缩存储	加密存储	增量传输	加密传输	文件元数据	容易访问	可恢复	处理重命名	命令行	其他界面	证书	其它平台	维护	特性
Areca Backup	arecaAUR	Java	Zip,	AES 128,	是	是	是	否	只可	否	是	是	GPLv2	Windows	是	

名字	包	实现	压缩存储	加密存储	增量传输	加密传输	文件元数据	容易访问	可恢复	处理重命名	命令行	其他界面	证书	其它平台	维护	特性
			Zip 64	AES 256					以暂停							
BorgBackup	borg包	Python, C (Cython)	lz4, zlib, lzma, zstd	AES 256	是	SSH	是	是	是	是	是	third party	BSD	*BSD, macOS, Windows (Cygwin / WSL)	是	基于可变长度块的重复数据删除；支持本地和基于ssh的远程备份目的地.
bup	bup包 bup-gitAUR	C, Python, git	是	否	是	是	不成熟	是	在你离开的地方继续	是	是	bupsAUR	GPLv2	NetBSD, Windows, macOS	是	和git一样的存储格式.
Duplicati	duplicati-latestAUR	C#	是	是	是	是	scheduled for 2.0	否	Pausing only	否	是	是	LGPL	Windows	是	

名字	包	实现	压缩存储	加密存储	增量传输	加密传输	文件元数据	容易访问	可恢复	处理重命名	命令行	其他界面	证书	其它平台	维护	特性
							release									
Duplicity	duplicity包	librsync	gzip	gpg	是	是	?	否	是	否	是	Yes	GPL		是	
Kup Backup System	kup包	rsync, bup front-end	是	是	是	是	Immature	是	否	是	bup	Qt	GPLv2		是	
restic	restic包 restic-gitAUR	Go	否	AES-256	是	是	是	是	是	是	是	否	BSD	OpenBSD, Windows, macOS	是	支持多种通过本地或通过rclone包的云服务存储.
ZBackup	zbackupAUR	C++	LZMA, LZO	AES	是	是	?	planned	否	Kinda through tar	是	否	GPLv2		是	库由不可变文件组成.

基于文件的增量

如果文件被修改, 这些文件会在下次快照存储整个版本的文件. 与 #基于块的增量应用相比, 它们对存储空间的利用率不够高, 特别是备份大文件但是修改小时; 另一方面, 通常存档的快照没有备份程序安装的时候也能打开.

特殊说明:

- **硬链接**: 是否将为修改的文件存储为之前版本的硬链接.

名字	包	实现	压缩存储	加密存储	增量传输	加密传输	文件系统元数据	易访问	可恢复	处理重命名	硬链接	命令行	其他界面	证书	其它平台	维护	特性
Back In Time	backintimeAUR	Python, rsync, diff	否	否	rsync	rsync	rsync	是	否	否	Yes \[失效链接 2020-08-06 ⓘ\]	是	Qt	GPLv2		是	
DAR (Disk ARchive)	darAUR	C++	special archive format	是	是	是	?	?	?	?	No	是	dargui AUR	GPL	FreeBSD, NetBSD, Windows, macOS	是	
Link-Backup\[失效链接 2022-09-23 ⓘ\]	link-backupAUR	Python 2	否	否	?	SSH	?	?	是	是	No \[失效链接 2022-09-23 ⓘ\]	是	否	MIT		否	把它自己复制到服务器.
rdup	rdupAUR	C	tar.gz	gpg, blowfish and others	?	?	?	是	?	否	是	是	否	GPLv3		否	一套命令行工具.
rsnapshot	rsnapshot包	rsync	否	否	是	是	?	?	?	?	Yes	是	否	GPLv2	Win32	No	
TimeShift	timeshift AUR	rsync	否	否	rsync	rsync	?	?	?	?	是	否	GTK	GPLv3	专门为完整系统备份到专用设备而设计的.	是	

面向网络

这些程序被设计为旨在集中链接到网络的多台计算机的备份, 通过一种服务器-客户端模型. 通常，它们部署更复杂, 与 #单机器解决方案相比.

特殊说明:

- **控制方向**: 拉取: 服务器登录客户端. 推动: 客户端启动会话.
- **增量类型**: 通过删除重复数据来减少已用空间的策略 (i.e., 除了压缩之外).
- **基于文件**: 如果文件被修改, 整个新版本将存在每个快照中.
- **硬链接**: 未修改的文件是否被存为之前版本的硬链接.
- **基于块**: 只有文件修改过的*部分*被存在每个快照中.

名字	包	实现	控制方向	压缩存储	加密存储	增量传输	加密传输	文件系统元数据	易访问	可恢复	处理重命名	增量种类	命令行	其它界面	证书	其它平台	维护	特性
BackuPPC	backuppc包	Perl	Pull	是	否	是	是	是	否	是	?	基于文件, 硬链接	否	Web	GPLv2	任何平台 (不需要客户端)	是	相同或不同客户端的备份中相同文件仅存储一次.
Bacula	bacula* in AUR	C++	Pull	是	是	?	是	?	?	是	?	基于文件	是	GUI, Web	AGPLv3	Windows, macOS	是	
burp	burp-backup AUR	librsync	Push	是	是	是	是	是	?	是	?	基于块	是	burp-ui	AGPLv3	Windows, macOS	是	
SafeKeep	safekeepAUR	rdiff-backup	Pull	否	否	?	是	?	?	?	?	基于块	是	是	GPL		否	与LVM和数据库集

名字	包	实现	控制方向	压缩存储	加密存储	增量传输	加密传输	文件系统元数据	易访问	可恢复	处理重命名	增量种类	命令行	其它界面	证书	其它平台	维护	特性
																		成以创建一致的备份.带宽限制.
Snebu	snebuAUR	C	Push or Pull	是	否	?	是	?	?	?	?	基于文件	是	否	GPLv3		?	支持任意保留计划.
Synbak	synbak包	Multi tool wrapper	?	是	否	是	是	是	?	?	?	?	否	Web	GPLv3		是	统一许多备份方法.
UrBackup	urbackup* in AUR	C++	Pull	否	否	是	Internet transfers only	是	是	是	是	基于文件,硬链接和符号链接/基于块的CoW-Snapshots	是(client)	GUI,Web	AGPLv3+	Windows,macOS	是	相同或不同客户端的相同文件的备份只

名字	包	实现	控制方向	压缩存储	加密存储	增量传输	加密传输	文件系统元数据	易访问	可恢复	处理重命名	增量种类	命令行	其它界面	证书	其它平台	维护	特性
																		存储一次.在文件系统快照里集成了LVM, dattobd和btrfs .

版本控制系统

虽然版本控制系统大多数时候用于源代码, 但它们可以跟踪目录里的任何文件.

查阅 List of applications/Utilities#Version control systems 和 dotfiles.

也可查阅

- Backing up Linux and other Unix(-like) systems
- Exhaustive list of backup solutions for Linux
- Performance comparison of five remote incremental backup tools: Rsync, Rdiff-backup, Duplicity, Areca and Link-Backup
- Mirroring an Entire Site using Rsync over SSH
- rsync-snapshot.sh — Local and remote snapshot backup using rsync with hard links

14.2 Rsync

难度 高级 | **篇幅** 25.7 KB | **所属** 同步

本节将讲到

- 安装

- 作为 cp/mv 的替代
- 作为备份工具
- 我自己的基于 rsync 的快照式备份过程
- (cc) marcio rps AT gmail.com

翻译状态：

- 本文（或部分内容）译自 rsync，最近一次同步于 2026-10-14，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

rsync 是一个开源工具，可以进行快速的增量文件传输。

安装

安装 rsync包 软件包。

必须在源计算机和目标计算机上都安装 *rsync*。

前端

- **Grsync** — GTK 前端。

: https://www.opbyte.it/grsync/ || grsync包

- **luckyBackup** — 用 C++ 编写的 Qt 前端。

: https://luckybackup.sourceforge.net/index.html || luckybackupAUR

- **nbRsync** — 为rsync开发的JavaFX GUI程序，有集成调度器.

: https://trixon.se/projects/nbrsync/ || 未被打包？在 AUR 里搜索

其他使用 rsync 的工具是 rdiff-backupAUR, osyncAUR 和 yarsyncAUR。

作为 cp/mv 的替代

注意：在不同的文件系统之间使用 rsync 而不是 cp/mv 是高效的，但对于在同一个文件系统上复制或移动文件却不然。参见

rsync 可以作为 `cp` 或 `mv` 命令的高级替代品，特别是对于较大文件的复制：

```
$ rsync -P source destination
```

其中 `-P` 与 `--partial --progress` 选项的作用是相同的，该选项使得文件可以分块传输并显示传输过程中的进度。

您可能需要使用 `-r`/`--recursive` 选项递归到目录中传输。

可以像 cp 命令一样本地复制文件，但 rsync 令人激动的用途在于远程复制文件，例如在两个不同的主机之间。远程位置可以用主机加冒号进行指定：

```
$ rsync source host:destination
```

或者

```
$ rsync host:source destination
```

网络文件传输默认使用SSH协议，`host` 可以是真实的主机名或来自于 `.ssh/config` 的预先定义的配置文件/别名。

无论是本地或远程文件传输，rsync 首先会创建一个文件列表，其中含有接下来会被用于确定各个文件是否需要构建的信息（默认为文件大小和上次修改时间戳）。对于每个需要构建的文件，都会找出其所有块（长度**S**字节，不重叠，偏移量可由**S**整除）的一个弱校验和和一个强校验和。使用这一信息，一个大文件就可以由 rsync 构建，而无需传输整个文件。更详细的实际解释和详细的数学解释见 rsync 如何工作及 rsync 算法。

要快速使用合理默认值，可以使用一些别名：

```
cpr() {
  rsync --archive -hh --partial --info=stats1,progress2 --modify-window=1 "$@"
}
mvr() {
  rsync --archive -hh --partial --info=stats1,progress2 --modify-window=1 --remove-source-files "$@"
}
```

- `-hh`: 以人类可读的格式输出数字
- `--info=stats1,progress2`: `stats1` 使用详细级别1显示 rsync 传输统计信息。`progress2` 打印总的传输进度，而不是每个文件的传输进度 (`progress1`)
- `--modify-window=1`: 当比较两个文件的时间戳时，如果时间戳的差异小于1秒，则将它们的时间戳视为相等
- `--remove-source-files`: 成功同步后从源目录中删除文件

注意：此处对*校验和*这一名词的使用与 `--checksum` 选项的行为**不是**相等价的。`--checksum` 选项影响的是在传输任何文件之前使用的决定是否跳过文件的启发式方法。在基于块的文件构建中一定会使用*校验和*，这是 rsync 传输文件的方法，与 `--checksum` 无关。

注意尾部的斜杠

Arch 默认使用 GNU cp (coreutils包 的一部分)。 然而，rsync 遵循 BSD cp 的约定, 源目录后面带有一个斜杠“/”有着特殊的处理。比如命令

```
$ rsync -r source destination
```

创建一个包含"source"的内容的"destination/source"目录。而命令

```
$ rsync -r source/ destination
```

把"source/"目录下的所有文件全部复制到"destination"目录下，而没有中间的子目录，和以下命令一样：

```
$ rsync -r source/. destination
```

这与 GNU cp 的行为是不同的，在 GNU cp 中"source" 与 "source/" 意义相同("source/."则不然)。并且，一些shell在Tab补全目录名的时候自动追加尾部下划线。由于这些因素，新手或偶尔使用 rsync 的用户可能倾向于忘记 rsync 的不同行为，在命令行上留下了尾部斜杠，从而无意间弄错，甚至覆盖重要文件。

谨慎起见，可以使用包装脚本在调用 rsync 之前自动删除尾部斜杠：

```
#!/bin/bash
new_args=()
for i in "${@}"; do
    case "${i}" in
        /)
            i="/"
        ;;
        */)
            i="${i%/}"
        ;;
        esac
    new_args+=("${i}")
done
exec rsync "${new_args[@]}"
```

该脚本可以放在 path 中的某个位置，并在 shell 的 init 文件中指定别名为 rsync。

作为备份工具

rsync 协议可以很容易地用于备份，只传输自上次备份以来已更改的文件。本节将介绍一些简单的基于 rsync 的计划备份脚本，通常用于复制到可移动介质。

自动备份

以下面的脚本为例，该脚本放置于 `/etc/cron.daily` 目录下，如果 cron 守护程序被正确安装和配置，它将每天运行。配置和使用 cron 是本文的范围之外。

首先，创建一个包含相应命令选项的脚本：

```
/etc/cron.daily/backup
#!/bin/sh
rsync -a --delete --quiet /path/to/backup /location/of/backup
```

`-a` : 表示文件应被存档，这意味着他们的大部分特性被保留 (不包括 ACLs, 硬链接或扩展属性，如 capabilities) **`--delete`** : 指同步源的删除操作。

在这里，`/path/to/backup` 应该改成需要被备份的路径 (比如 `/home`)，`/location/to/backup` 是备份应存放的位置 (比如 `/media/disk`).

最后，脚本必须可执行。

通过 SSH 自动备份

如果是通过 SSH 备份到远程主机，改为使用此脚本：

```
/etc/cron.daily/backup
#!/bin/sh
rsync -a --delete --quiet -e ssh /path/to/backup
remoteuser@remotehost:/location/of/backup
```

`-e ssh`：告诉rsync的使用SSH **`remoteuser`**：远程主机 `remotehost` 上的用户名 **`-a`**：组中的所有这些选项 `-rlptgoD` (recursive, links, perms, times, group, owner, devices)

注意： 在每次增量备份的时候，Rsync 都会试着修改目标主机上所有先前备份过的文件以匹配源主机上的文件状态。这意味着在通过 SSH （并且使用 `-a`选项保留权限和所有者的时候）备份属于 root 的文件，需要目标主机的 root 权限。通常的办法是将 SSH 守护程序设置为允许使用公钥而不是密码登录，并以 root 用户运行 rsync 命令。

使用 NetworkManager 自动备份

该脚本在网络连接后开始备份。

首先，创建一个包含相应命令选项的脚本：

```
/etc/NetworkManager/dispatcher.d/backup
#!/bin/sh

if [ x"$2" = "xup" ] ; then
        rsync --force --ignore-errors -a --delete --bwlimit=2000 --files-
from=files.rsync /path/to/backup /location/of/backup
fi
```

`-a`：组中的所有选项 `-rlptgoD` recursive, links, perms, times, group, owner, devices **`--files-from`**：从文件中读取到备份路径`/path/to/backup`的相对路径 **`--bwlimit`**：限 I/O 带宽；每秒千字节

这个脚本必须属于 root 用户 (参见 NetworkManager#使用 NetworkManager 调度网络服务)。

使用 systemd 和 inotify 自动备份

注意：

- 由于 inotify 和 systemd 的限制， (见 这个问题和回答)，递归的文件系统监测是不可能的。即使你监测了一个目录和它的内容，它也不会递归子目录并检测其中的内容。你必须明确地指出所有需要检测的目录，就算这些目录是你已经监视的目录的子目录。
- 以下设置基于一个 systemd/用户 的实例。

相比于执行一个基于时间计划的定时备份，比如使用 cron，在每次文件变动的时候都执行备份也是可行的。`systemd.path` 单元使用 `inotify` 以监测文件系统，并且可以与 `systemd.service` 文件一同使用来在捕获到文件系统事件的时候启动任何进程（如你的 rsync 备份）。

首先，创建监测备份文件的 `systemd.path` 文件：

```
~/.config/systemd/user/backup.path
[Unit]
Description=Checks if paths that are currently being backed up have changed

[Path]
PathChanged=%h/documents
PathChanged=%h/music

[Install]
WantedBy=default.target
```

然后创建一个 `systemd.service` 文件，它将会在检测到变化的时候启动。默认情况下一个与 `.path` 单元同名(本例中为 `backup.path`)，但以 `.service` 结尾的服务（本例中为 `backup.service`）将会启动。

注意：如果你需要运行多个 rsync 命令，使用 `Type=oneshot`。这将允许你指定多个 `ExecStart=` 参数，每个参数对应一个 rsync 命令。更简单的方法是写一个脚本来启动你所有的备份，就像 cron 脚本一样。

```
~/.config/systemd/user/backup.service
[Unit]
Description=Backs up files

[Service]
ExecStart=/usr/bin/rsync %h/./documents %h/./music -CERrltm --delete ubuntu:
```

现在只需要像普通的 systemd 服务那样，启动/启用 `backup.path` 它就会开始监测文件变化并且自动启动 `backup.service` 了。

每周差异备份

这个 rsync 选项很有用，每次运行时创建一个完整备份，并且每天在一个单独的目录中保留已修改文件的差异备份副本。

首先，创建一个包含相应命令选项的脚本：

```
/etc/cron.daily/backup
#!/bin/sh

DAY=$(date +%A)

if [ -e /location/to/backup/incr/$DAY ] ; then
  rm -fr /location/to/backup/incr/$DAY
fi

rsync -a --delete --quiet --inplace --backup --backup-dir=/location/to/backup/incr/
$DAY /path/to/backup/ /location/to/backup/full/
```

`--inplace` 选项包含 `--partial` ，就地更新目标文件。

快照备份

同样你也可以用这个工具来创建你文件的快照树（即一个有日期顺序的文件副本的目录）。这个快照树是基于硬链接的，这意味着只有修改过的文件才会占用空间。这也是苹果 TimeMachine 的原理。

实现这个功能的脚本很容易实现，使用 `--link-dest` 选项来创建增量快照，硬链接未改变的文件：

```
/usr/local/bin/snapbackup.sh
#!/bin/sh

# 基本的快照式 rsync 备份脚本

# 配置
OPT="-aPh"
LINK="--link-dest=/snapshots/username/last/"
SRC="/home/username/files/"
SNAP="/snapshots/username/"
LAST="/snapshots/username/last"
date=`date "+%Y-%b-%d:_%T"`

# 运行 rsync 以创建快照
rsync $OPT $LINK $SRC ${SNAP}$date

# 删除指向上一个快照的符号链接
rm -f $LAST

# 创建到最新快照的新符号链接，以便下次备份到硬链接
ln -s ${SNAP}$date $LAST
```

这里必须有一个指向完整备份的符号链接，以作为 `--link-dest` 的目标。如果最近的快照被删除，那么就需要重新创建一个指向新的快照的符号链接。如果 `--link-dest` 没有找到有效的符号链接，rsync将继续复制所有文件，而不只复制变化。

一个更加复杂一点的版本每次保留一个最新的完整备份到 `$SNAP/latest` 。若自上次完整备份起有一定数量的文件发生了变化，那么脚本将创建一个新的快照到 `$SNAP/$DATETAG`，并使用 `cp -al` 来硬链接未改变的文件：

```
/usr/local/bin/rsnapshot.sh
#!/bin/sh

## 我自己的基于 rsync 的快照式备份过程
## (cc) marcio rps AT gmail.com

# 配置变量

SRC="/home/username/files/" #dont forget trailing slash!
SNAP="/snapshots/username"
OPTS="-rltgoi --delay-updates --delete --chmod=a-w"
MINCHANGES=20

# 以低优先级运行此进程

ionice -c 3 -p $$
renice +12  -p $$

# 同步

rsync $OPTS $SRC $SNAP/latest >> $SNAP/rsync.log

# 检查是否有足够的变化，如果有
# 则制作一份以日期命名的硬链接副本

COUNT=$( wc -l $SNAP/rsync.log|cut -d" " -f1 )
```

```
if [ $COUNT -gt $MINCHANGES ] ; then
        DATETAG=$(date +%Y-%m-%d)
        if [ ! -e $SNAP/$DATETAG ] ; then
                cp -al $SNAP/latest $SNAP/$DATETAG
                chmod u+w $SNAP/$DATETAG
                mv $SNAP/rsync.log $SNAP/$DATETAG
               chmod u-w $SNAP/$DATETAG
         fi
fi
```

懒得手动运行的话，你也可以让这个脚本作为 systemd/定时器单元运行。

全盘系统备份

本节介绍使用rsync传输整个/树的副本，不包括一些选定的目录。这种方法被认为比使用dd进行磁盘克隆更好，因为它允许使用不同的大小、分区表和文件系统，也比使用cp-a进行复制更好，因为其允许对文件权限、属性、访问控制列表和扩展属性进行更大的控制。 本节是关于使用 *rsync* 来创建一份整个 `/` 树的副本，其中不包含特定的几个文件夹。此方法相较于使用 `dd` 来进行硬盘克隆要更佳，因为它允许你在使用不同大小、分区表和文件系统的存储设备间传输；也比 `cp -a` 更好，因为它允许你对文件权限、属性、访问控制列表和扩展属性有更好的掌握。

rsync 在系统运行时亦可进行备份，但传输期间改变的文件可能不会被备份。这可能会造成使用这些文件的程序的一些未知错误或未定义行为。为缓解这个问题，可以注销所有用户并关闭所有程序和数据库。

这种方法对于将现有的已安装系统迁移到新的硬盘或固态硬盘上非常有效。

在 root 权限下运行此命令，以确保 rsync 能访问到所有系统文件，并且保留权限：

```
 # rsync -aAXHv --exclude='/dev/*' --exclude='/proc/*' --exclude='/sys/*' --
exclude='/tmp/*' --exclude='/run/*' --exclude='/mnt/*' --exclude='/media/*' --
exclude='/lost+found/' / /path/to/backup
```

通过使用 `-aAX` 选项集，文件以归档模式传输，确保符号链接、设备、权限、所有权、修改时间、ACLs和扩展属性得以保留，前提是目标文件系统支持这一功能。选项 `-H` 保留了硬链接，但会使用更多的内存。

`--exclude` 选项使符合给定模式的文件/文件夹被排除。可以结合使用`--exclude-from=*file*`选项，排除与`*file*`中的模式（每条一个）相匹配的文件/目录。这与#筛选规则的高级用法中描述的例子类似，但不需要+/-语法。

在上述命令中，`/dev`、`/proc`、`/sys`、`/tmp`和`/run` 等目录被包括在内，但这些目录的内容被排除在外。这是因为它们在系统启动时才会被填入内容，但这些目录本身不会被创建。 `/lost+found` 是针对文件系统的。为排除模式加上引号可以避免被 shell 误扩展，例如，在通过 SSH 备份时，这是必要的。以 `*` 结尾的排除路径可以确保目录本身在不存在的情况下被创建。

注意：

- 如果计划将系统备份到 `/mnt` 或 `/media` 以外的其他位置，请不要忘记将其添加到排除的模式列表中，以避免无限循环。
- 如果系统中有任何绑定挂载，也应排除它们，以便绑定挂载的内容只复制一次。
- 如果使用交换文件，请确保也将其排除在外。
- 考虑是否要备份 `/home/` 目录。如果它包含您的数据，它可能比系统大得多。另外，请考虑排除不重要的子目录，例如 `/home/*/.thumbnails/*`, `/home/*/.cache/mozilla/*`, `/home/*/.cache/chromium/*`, 和 `/home/*/.local/share/Trash/*`，具体取决于系统上安装的软件。
- 如果安装了 GVFS，则必须排除 `/home/*/.gvfs`，以防止 rsync 错误。
- 如果安装了 Dhcpcd ≥ 9.0.0，请排除 `/var/lib/dhcpcd/*` 目录，因为它将多个系统目录作为子目录挂载在那里。

您可能希望包括其他 rsync 选项，或删除一些选项，例如以下选项。有关完整列表，请见 rsync(1)。

- 如果您在内存非常低的系统上运行，请考虑删除 `-H` 选项；然而，这在大多数现代机器上应该没有问题。文件系统上可能有许多硬链接，具体取决于使用的软件（例如，如果您使用 Flatpak）。许多硬链接位于 `/usr/` 目录下。
- 如果在同一备份目录中多次运行 rsync，则可能需要添加 rsync 的 `--delete` 选项。在这种情况下，请确保源路径不以 `/*` 结尾，否则此选项将仅对源目录子目录中的文件有效，但对直接驻留在源目录内的文件无效。
- 如果您使用任何稀疏文件，如虚拟磁盘、Docker 映像和类似文件，则应添加 `-S` 选项。
- `--numeric-ids` 选项将禁止映射到用户名和组名；相反，数字的组和用户 ID 将被直接传输。这在通过 SSH 进行备份或使用 live 系统备份不同的系统盘时非常有用。
- 选择 `--info=progress2` 选项而不是 `-v` 选项将显示整体进度信息和传输速度，而不是正在传输的文件列表。
- 为了避免在递归时跨越文件系统边界，请添加选项 `-x`/`--one-file-system`。这将防止备份层次结构中的任何挂载点。

还原备份

如果希望还原备份，请使用执行的 rsync 命令，但源和目标相反。（你可能还需要在源路径末尾添加一个斜杠）。如果你使用了 `--exclude` 选项，那么在执行恢复命令时，最好移除这些选项。

筛选规则的高级用法

rsync 可以从单个过滤器文件中读取所有这些规则，而不是单独指定包含和排除规则。rsync 以自顶向下的顺序处理规则；首先匹配的规则获胜。

```
backup.filter
# 排除的模式

- .thumbnails/***
- node_modules/***
- venv/***

# 包含的模式

+ /Documents/***
+ /Books/***
+ /Music/***

# 排除其他所有内容
- /**
```

*** 是一种特殊的 rsync 模式，它递归地匹配文件夹及其所有内容。

查看 rsync(1) § PATTERN MATCHING RULES 和 rsync(1) § FILTER RULES IN DEPTH 获取更多详情。

然后用此命令运行 rsync：

```
$ rsync -aAXHv --filter="merge backup.filter" $SRC $DEST
```

关键字是 `--filter "merge ..."` 参数，它将获取过滤器文件并按顺序解析每个同步文件的规则。

从路径列表复制

除了使用#筛选规则的高级用法外，还可以使用`--files-from` 选项。它可以从包含目录或文件路径列表的文本文件中获取输入，每条路径都以新行分隔。需要注意的是，如果用户想要递归复制目录，即使已经包含 `-a` 选项，也必须手动为该选项指定 `-r` 标志。

例如，可以使用以下命令归档目录列表和所有递归目录：

```
$ rsync -aAXHvr --files-from="dir_list.txt" $SRC $DEST
```

文件系统克隆

rsync 提供了一种复制文件系统中所有数据的方法，同时保留尽可能多的信息，包括文件系统元数据。这是一个文件系统级别的数据克隆过程，其中源文件系统和目标文件系统不需要是同一类型。它可以用于备份、文件系统迁移或数据恢复。

rsync 的*归档*模式接近于适合该工作，但它不备份特殊的文件系统元数据，例如访问控制列表、扩展属性或稀疏文件财产。为了在文件系统级别成功克隆，需要提供一些其他选项：

```
rsync -qaAXHSU SOURCE_DIR DESTINATION_DIR
```

它们的意思是（来自手册页）：

```
--acls, -A              保留 ACLs（包含 --perms）
--xattrs, -X            保留扩展属性
--hard-links, -H        保留硬链接
--sparse, -S            将空序列转换为稀疏块
--atimes, -U            保留访问（使用）时间
```

此外，如果要从副本中排除树下挂载的其他文件系统，请使用 `-x`。

注意：

- 如果您正在使用 rsync 将 Arch 安装迁移到新驱动器，请记住在 `SOURCE_DIR` 末尾添加一个尾随斜杠，原因在#注意尾部的斜杠中提到。
- 文件的创建时间（`crtime`）不会被保留。虽然 rsync 提供了 `-N`/`--crtimes` 选项来保留创建时间，但 Linux 不支持该功能。

提示：如果你使用 btrfs，可以考虑使用 发送/接收（send/receive） 功能。

可以使用 `diff` 的递归选项在文件系统级别简单地重新读取和检查生成的副本（例如，在数据恢复尝试之后）：

```
diff -r SOURCE_DIR DESTINATION_DIR
```

通过使用 rsync（如本文所述）和更新 fstab 和引导加载程序（如迁移到新硬件中所述），可以成功执行文件系统迁移。这本质上提供了一种将任何根文件系统转换为另一个文件系统的方法。

作为守护程序

rsync 可以在侦听TCP端口 `873` 的服务器上作为守护程序运行。

编辑模板 `/etc/rsyncd.conf`，配置共享并启动 `rsyncd.service`。

注意：截至 rsync包 3.2.0-1，该软件包采用了上游 systemd 单元文件 `rsyncd.service` 和 `rsyncd@.service`。`ProtectHome` 的更改已被注释，`[Service]` 节下的安全功能 `ProtectSystem=full` 仍处于活动状态。这将使 `/boot/`, `/etc/` 和 `/usr/` 目录只读。如果您需要 rsyncd 写入系统目录，您可以编辑该单元，并在重写代码段的 `[Service]` 节中设置 `ProtectSystem=off`。

来自客户端使用，例如列出服务器内容：

```
$ rsync rsync://server/share
```

将文件从客户端传输到服务器：

```
$ rsync local-file rsync://server/share/
```

考虑在防火墙中打开 TCP 端口 873，并使用用户身份验证。

注意：包括用户身份验证在内的所有传输数据均未加密。

示例配置

从文件列表共享

```
/etc/rsyncd.conf
...

# 跨越文件系统边界时需要。
#use chroot  = no
read only = yes

...

[sync]
    path          = /
# 要复制的文件列表。
    include from = /backup.list
# 排除其余部分。
    exclude       = *
```

在文件列表中，所有*中间路径*都是必需的，除非使用 `***` 通配符：

```
/backup.list
/etc/
/etc/conf.d/
/etc/conf.d/hwclock
/etc/fonts/***
```

另见

- 更多用法示例在社区贡献和 General Programming 论坛
- 如何 – 使用带硬链接的 rsync 进行本地和远程快照备份包括带硬链接的文件重复数据消除、MD5 完整性签名、'chattr' 保护、过滤规则、磁盘配额、指数分布的保留策略（备份循环，同时保存比旧备份更新的备份）
- 与 SSH 密钥/身份文件一同使用 rsync
- rrsync(1) 是一个通过 SSH 登录来配置受限 rsync 用户的脚本，包含在 rsync包软件包中。有关更多详细信息及示例，请参阅 OpenSSH#限制。

存储虚拟化

存储虚拟化要解决的是物理磁盘与使用需求之间的错位：单块盘容量写死、不能跨盘拼接，也扛不住一块盘坏掉。LVM 把磁盘抽象成可伸缩的逻辑卷，RAID 用多块磁盘拼出容量、速度与冗余。

两节都必读，先《LVM》把物理卷、卷组、逻辑卷的分层与准备规划弄清楚，再《RAID》看各级别在容量与容错上的取舍；暂不做冗余的话，维护部分可以先略过。最容易踩的坑是把两者当成备份——卷组坏一块盘、阵列同时坏两块，数据照样没了；分区与阵列还要在装系统之前定好，装完再改代价极高。

15.1　LVM

难度 高级 | **篇幅** 36.4 KB | **所属** 存储虚拟化

本节将讲到

- LVM基本组成
- 优点
- 缺点
- 准备
- 在LVM上安装Arch Linux

引用 Wikipedia:Logical Volume Manager (Linux)) 的解释：

: LVM 是一种可用在 Linux 内核的逻辑分卷管理器；可用于管理磁盘驱动器或其他类似的大容量存储设备。

LVM基本组成

LVM利用Linux内核的device-mapper功能来实现存储系统的虚拟化（系统分区独立于底层硬件）。 通过LVM，你可以实现存储空间的抽象化并在上面建立虚拟分区（virtual partitions），可以更简便地扩大和缩小分区，可以增删分区时无需担心某个硬盘上没有足够的连续空间，避免为正在使用的磁盘重新分区的麻烦、为调整分区而不得不移动其他分区的不便。

LVM的基本组成部分如下：

物理卷 (PV) : 一个可供存储LVM的块设备. 例如: 一块硬盘, 一个MBR或GPT分区, 一个回环文件, 一个被内核映射的设备 (例如 dm-crypt).它包含一个特殊的LVM头。 **卷组 (VG)** : 物理卷的一个组，作为存放逻辑卷的容器。 PEs are allocated from a VG for a LV. **逻辑卷 (LV)** : "虚拟/逻辑卷"存放在一个卷组中并由物理块组成。是一个类似于物理设备的块设备，例如，你可以直接在它上面创建一个文件系统文件系统。 **物理块**

(PE)：一个卷组中最小的连续区域(默认为4 MiB)，多个物理块将被分配给一个逻辑卷。你可以把它看成物理卷的一部分，这部分可以被分配给一个逻辑卷。

示例:



优点

比起普通的硬盘分区管理方式，LVM更富于灵活性：

- 将多块硬盘看作一块大硬盘
- 使用逻辑卷（LV），可以创建跨越众多硬盘空间的分区。
- 可以创建小的逻辑卷（LV），在空间不足时再动态调整它的大小。
- 在调整逻辑卷（LV）大小时可以不用考虑逻辑卷在硬盘上的位置，不用担心没有可用的连续空间。
- 可以在线（online）对逻辑卷（LV）和卷组（VG）进行创建、删除、调整大小等操作。不过LVM上的文件系统也需要重新调整大小，好在某些文件系统（例如ext4）也支持在线操作。
- 无需重新启动服务，就可以将服务中用到的逻辑卷（LV）在线（online）/动态（live）迁移至别的硬盘上。
- 允许创建快照，可以保存文件系统的备份，同时使服务的下线时间（downtime）降低到最小。
- 支持各种设备映射目标（device-mapper targets），包括透明文件系统加密和缓存常用数据（caching of frequently used data）。这将允许你创建一个包含

一个或多个磁盘、并用LUKS加密的系统，使用LVM on top 可轻松地管理和调整这些独立的加密卷 （例如. /, /home, /backup等) 并免去开机时多次输入密钥的麻烦。

缺点

- 在系统设置时需要更复杂的额外步骤。
- Windows系统并不支持LVM，若使用双系统，你将无法在Windows上访问LVM分区。

准备

确保已安装lvm2包包。

在LVM上安装Arch Linux

你应该在安装过程中的的分区和创建文件系统这一步中创建LVM卷。不要直接格式化一个分区作为你的根文件系统（/），而应将其创建在一个逻辑卷（LV）中。

快速导览：

- 创建物理卷（PV）所在的分区。
- 创建物理卷（PV）。如果你只有一个硬盘，那么你最好只创建一个分区一个物理卷；如果你有多个硬盘，你可以创建多个分区，在每个分区上分别创建一个物理卷。
- 创建卷组（VG），并把所有物理卷加进卷组。
- 在卷组（VG）上创建逻辑卷（LV）。
- 继续安装指南中的格式化分区步骤。
- 当你做到安装指南中的“Initramfs”步骤时，把 `lvm2`加入到 `mkinitcpio.conf` 文件中（请参考下文）。

警告：若你使用不支持LVM的引导程序，`/boot`不能置于LVM中。你必须创建一个独立的`/boot`分区并直接格式化它。已知支持LVM的引导程序只有GRUB。

创建分区

在继续配置LVM前，必须对设备进行分区。

创建分区：

- 若使用MBR,设置分区类型为`8e` (在"fdisk"中为`Linux LVM`).
- 若使用GPT, 设置分区类型为`E6D6D379-F507-44C2-A23C-238F2A3DF928` (在"fdisk"中为`Linux LVM`；在"gdisk"中为`8e00`).

创建物理卷（PV）

可通过以下命令列出可被用作物理卷的设备：

```
# lvmdiskscan
```

警告：请确认你对正确的设备进行操作，否则会导致文件丢失！

在列出的设备上创建物理卷：

```
 # pvcreate DEVICE
```

该命令在各个设备上创建LVM头。如#LVM基本组成所示, *DEVICE*可以是磁盘（如`/dev/sda`），分区（如`/dev/sda2`）或环回设备。例如：

```
# pvcreate /dev/sda2
```

你可以用以下命令查看已创建好的物理卷：

```
# pvdisplay
```

注意：如果你用的是未格式化过且擦除块（erase block）大小小于1M的SSD，请采用以下命令`pvcreate --dataalignment 1m /dev/sda`来设置对齐（alignment），可以参考链接（英文）

创建卷组（VG）

创建完成物理卷（PV）之后，下一步就是在该物理卷创建卷组（VG）了。 首先必须先在其中一个物理卷（PV）创建一个卷组：

```
# vgcreate <volume_group> <physical_volume>
```

可用作字符卷组的名称可在lvm(8)中查到。

例如：

```
# vgcreate VolGroup00 /dev/sda2
```

然后让该卷组扩大到其他所有的物理卷:

```
# vgextend <卷组名> <物理卷>
# vgextend <卷组名> <其它物理卷>
# ...
```

例如：

```
# vgextend VolGroup00 /dev/sdb1
# vgextend VolGroup00 /dev/sdc
```

其中，“VolGroup00”名字换成你自己起的名字即可。 接下来可以用以下命令查看卷组：

```
# vgdisplay
```

注意：你可以创建多个的卷组，但这将使你的硬盘空间分布在不同（逻辑）磁盘上。

一步创建卷组

LVM支持将卷组与物理卷的创建聚合在一个命令中。例如，为了在前文提到的三个设备中创建名为VolGroup00的卷组，可以执行如下命令：

```
# vgcreate VolGroup00 /dev/sda2 /dev/sdb1 /dev/sdc
```

该命令首先会在分区上创建物理卷（如果之前没有创建过），再创建一个包含三个物理卷的卷组。如果设备上已经存在文件系统，命令会提出警告。

创建逻辑卷（LV）

提示：若想在LVM上使用快照, 逻辑卷缓存（logical volume caching）, thin provisioned logical volumes 或 RAID，请参见 #逻辑卷类型.

创建完卷组（VG）之后，就可以开始创建逻辑卷（LV）了。输入下面命令以指定新逻辑卷的名字、大小及其所在的卷组：

```
# lvcreate -L <卷大小> <"卷组名> -n <卷名>
```

例如：

```
# lvcreate -L 10G VolGroup00 -n lvolhome
```

该逻辑卷创建完后，你就可以通过`/dev/mapper/Volgroup00-lvolhome`或`/dev/VolGroup00/lvolhome`来访问它。与卷组命名类似，你可以按你的需要将逻辑卷命名。

你可以指定一个或多个物理卷来限制LVM分配数据空间的位置。比如你希望在较小的SSD硬盘上创建根文件系统，并在较慢的机械硬盘上创建家目录卷，仅需把物理卷设备加入到命令中，例如：

```
# lvcreate -L 10G VolGroup00 -n lvolhome /dev/sdc1
```

如果你想让要创建的逻辑卷拥有卷组（VG）的所有未使用空间，请使用以下命令：

```
# lvcreate -l +100%FREE  <volume_group> -n <logical_volume>
```

可以通过以下命令来查看逻辑卷：

```
# lvdisplay
```

注意：为了使上述命令能正常运行，你可能需要加载*device-mapper*内核模块（请使用命令**modprobe dm-mod**）。

提示：一开始可以创建小一点的逻辑卷，在卷组里留下一部分未使用空间，以后就可以根据需要再作扩展了。

建立文件系统与挂载逻辑卷

现在你的逻辑卷应该已经在`/dev/mapper/`和`/dev/*YourVolumeGroupName*`中了。如果你无法在以上位置找到它，请使用以下命令来加载模块、并扫描与激活卷组：

```
# modprobe dm-mod
# vgscan
# vgchange -ay
```

现在你可以在逻辑卷上创建文件系统并像普通分区一样挂载它了（如果你正在安装Arch linux，需要更详细的信息，请参考挂载分区）：

```
# mkfs.<类型> /dev/mapper/<卷组名>-<卷名>
```

```
# mount /dev/mapper/<卷组名>-<卷名> <挂载点>
```

例如：

```
# mkfs.ext4 /dev/mapper/VolGroup00-lvolhome
# mount /dev/mapper/VolGroup00-lvolhome /home
```

警告：挂载点请选择你所新建的逻辑卷（例如：`/dev/mapper/Volgroup00-lvolhome`），**不要**使用逻辑卷所在的实际分区设备（即不要使用：`/dev/sda2`）。

配置mkinitcpio

如果你的根文件系统基于LVM，你需要启用适当的mkinitcpio钩子，否则你的系统可能无法启动。

- 若使用基于busybox的initramfs，请启用`udev`和`lvm2`。
- 若使用基于systemd的initramfs，请启用`systemd`和`sd-lvm2`。

`udev`默认已经预设好，不必手动启用了。你只需要编辑`/etc/mkinitcpio.conf`文件，在`block`与`filesystem`这两项中间插入`lvm2`：

基于busybox的initramfs

```
/etc/mkinitcpio.conf
HOOKS="base udev ... block lvm2 filesystems"
```

基于systemd的initramfs:

```
/etc/mkinitcpio.conf
HOOKS=(base systemd ... block lvm2 filesystems)
```

之后你就可以继续下一步的创建和启用镜像操作了。

提示：

- `lvm2`和`sd-lvm2`钩子被lvm2包安装，而不是mkinitcpio包。如果你在"arch-chroot"中新安装的Arch Linux中运行"mkinitcpio"，必须在环境中安装lvm2包以使*mkinitcpio*找到`lvm2`或`sd-lvm2`钩子。如果lvm2包未安装, *mkinitcpio*将报错：`Error: Hook 'lvm2' cannot be found.`
- 若根文件系统在LVM + RAID上，请参见#为RAID配置mkinitcpio.

内核参数

如果你的根文件系统位于逻辑分卷，则`root=` 内核参数必须指向一个映射设备，比如`/dev/mapper/*vg-name*-*lv-name*`。

配置

高级选项

可以通过修改`/etc/lvm/lvm.conf`文件中的`auto_activation_volume_list`参数限制自动激活的卷。如果存在问题，可以将此选项注释掉。

调整卷

物理卷

对于存在物理卷的设备，在扩增其容量之后或缩小其容量之前，必须使用`pvresize`命令对应地增加或减少物理卷的大小。

扩增

增大分区`/dev/sda1`的容量之后，需要执行以下命令扩展物理卷的大小

```
# pvresize /dev/sda1
```

命令将自动探测设备当前大小并将物理卷扩展到其最大容量。

注意：该命令可在卷在线（on-line）时运行。

缩小

在减少某个物理卷所在设备大小之前，需要通过指定`--setphysicalvolumesize *大小*`参数缩小物理卷大小，例如：

```
# pvresize --setphysicalvolumesize 40G /dev/sda1
```

该命令可能会提示以下错误：

```
 /dev/sda1: cannot resize to 25599 extents as later ones are allocated.
 0 physical volume(s) resized / 1 physical volume(s) not resized
```

即该物理卷已分配物理区域超过了命令指定的新大小边界，`pvresize`会拒绝将物理卷缩小。若磁盘空间足够，可通过pvmove将物理区域重新分配至别的位置来解决这个问题。

移动物理区域

在移动空闲的物理区域到卷尾部之前，需要运行`# pvdisplay -v -m`命令查看物理分段。如下例所示，仅有一个物理卷`/dev/sdd1`, 一个卷组`vg1`和一个逻辑卷`backup`

```
# pvdisplay -v -m
    Finding all volume groups.
    Using physical volume(s) on command line.
  --- Physical volume ---
  PV Name               /dev/sdd1
  VG Name               vg1
  PV Size               1.52 TiB / not usable 1.97 MiB
  Allocatable           yes
  PE Size               4.00 MiB
  Total PE              399669
  Free PE               153600
  Allocated PE          246069
  PV UUID               MR9J0X-zQB4-wi3k-EnaV-5ksf-hN1P-Jkm5mW

  --- Physical Segments ---
  Physical extent 0 to 153600:
    FREE
  Physical extent 153601 to 307199:
    Logical volume   /dev/vg1/backup
    Logical extents  1 to 153599
  Physical extent 307200 to 307200:
    FREE
  Physical extent 307201 to 399668:
```

```
    Logical volume   /dev/vg1/backup
    Logical extents  153601 to 246068
```

可用空间在卷中段。为了减小物理卷大小，首先必须把所有的已用分段移到前部。

此例中，第一个可用空间在第0至第153600分段共153601个可用区域。我们可以从最后的分段中移动相同数目的物理区域来填补这段空间

```
# pvmove --alloc anywhere /dev/sdd1:307201-399668 /dev/sdd1:0-92466
/dev/sdd1: Moved: 0.1 %
/dev/sdd1: Moved: 0.2 %
...
/dev/sdd1: Moved: 99.9 %
/dev/sdd1: Moved: 100,0%
```

注意：

- 命令将92467 (399668-307201)个物理区域**从**最后一个分段**移动到**第一个分段。由于第一个分段共有153600个空闲的物理区域，可以容纳92467个物理区域，命令可以成功执行。
- 参数`--alloc anywhere`可以用于在同一个分区中移动物理区域的。若要在不同分区中移动，命令形式应该是`# pvmove /dev/sdb1:1000-1999 /dev/sdc1:0-999`
- 当操作的数据较多时，移动操作将持续很久（一到两个小时）。最好在Tmux或GNU Screen会话中执行此过程。任何形式的意外中断都可能会导致致命错误。
- 当操作完成后，可运行Fsck保证文件系统完整性。

调整物理卷大小

当所有空闲分段都移动到最后的物理区域时，运行`# vgdisplay`查看。

之后可以再次运行命令：

```
# pvresize --setphysicalvolumesize size PhysicalVolume
```

结果类似：

```
# pvs
  PV         VG   Fmt  Attr PSize    PFree
  /dev/sdd1  vg1  lvm2 a--     1t     500g
```

调整分区大小

最后，你可以用你喜欢的分区工具来缩小该分区。

逻辑卷

提示：lvresize(8)提供一些与lvextend(8)和lvreduce(8)相同的命令与选项，并同时允许两种类型的操作。然而，这几个命令都提供一个`-r`/`--resizefs`选项，使用fsadm(8)在调整逻辑卷时同时调整其中的文件系统（支持*ext2*, ext3, ext4, *ReiserFS* 和 XFS ）。 因此，对普通使用来说，使用`lvresize`和`--resizefs`将会更容易, 除非您有特定的需求或希望完全控制流程。

警告：尽管扩大一个文件系统可以“在线”(on-line)，也就是当它已挂载时)完成，甚至对根分区。缩小一个文件系统却往往要求先卸载（umount）它，以避免丢失数据。请先确保你的文件系统支持相关操作。

同时缩小逻辑卷和其文件系统

注意：只有*ext2*，ext3，ext4，*ReiserFS*和 XFS 文件系统支持以下操作。对于其他文件系统，请参见#单独设置文件系统大小。

将MyVolGroup组中的逻辑卷mediavol扩大10GiB，并同时扩大其文件系统：

```
# lvresize -L +10G --resizefs MyVolGroup/mediavol
```

将MyVolGroup组中的逻辑卷mediavol大小调整为15GiB，并同时调整其文件系统：

```
# lvresize -L 15G --resizefs MyVolGroup/mediavol
```

将卷组中的所有剩余空间分配给mediavol：

```
# lvresize -l +100%FREE --resizefs MyVolGroup/mediavol
```

更多选项请参见lvresize(8)。

单独设置文件系统大小

对于不支持fsadm(8)的文件系统，请在缩小逻辑卷前或扩大逻辑卷后，使用适当的工具来调整文件系统的大小。

先将MyVolGroup组中的逻辑卷mediavol扩大2 GiB，但不调整其文件系统：

```
# lvresize -L +2G MyVolGroup/mediavol
```

然后在调整其文件系统，是其达到逻辑卷的大小：（以ext4为例）

```
# resize2fs /dev/MyVolGroup/mediavol
```

要将逻辑卷mediavol缩小500 MiB，先计算调整后文件系统的大小并调整文件系统(以ext4为例)：

```
# resize2fs /dev/MyVolGroup/mediavol 调整后的大小
```

然后再缩小逻辑卷的大小：

```
# lvresize -L -500M MyVolGroup/mediavol
```

更多选项请参见lvresize(8)。

重命名卷

重命名卷组

要重命名一个卷组，请使用vgrename(8)命令。

可使用下面的任意一条命令将卷组vg02重命名为my_volume_group

```
# vgrename /dev/vg02 /dev/my_volume_group
# vgrename vg02 my_volume_group
```

重命名逻辑卷

要重命名一个逻辑卷，请使用lvrename(8)命令。

可使用下面的任意一条命令将vg02组中的逻辑卷lvold重命名为lvnew.

```
# lvrename /dev/vg02/lvold /dev/vg02/lvnew
# lvrename vg02 lvold lvnew
```

移除逻辑卷

警告：在移除逻辑卷之前，请先备份好数据以免丢失！

首先，找到你所要移除的逻辑卷的名称。你可以使用以下命令来查看系统的所有逻辑卷：

```
# lvs
```

接下来，找到你所要移除的逻辑卷的挂载点

```
$ lsblk
```

并卸载它：

```
# umount /<mountpoint>
```

最后，使用以下命令来移除逻辑卷：

```
# lvremove <volume_group>/<logical_volume>
```

例如：

```
# lvremove VolGroup00/lvolhome
```

请输入y来确定你要执行移除逻辑卷操作。

此外，请不要忘了更新/etc/fstab。

你可以再次使用lvs命令来确认你的逻辑卷已被移除。

添加物理卷到卷组中

首先创建一个新的物理卷，再把卷组扩充到该物理卷上：

```
# pvcreate /dev/sdb1
# vgextend VolGroup00 /dev/sdb1
```

这将增加你卷组中的物理区域总数，你可以按需要将它们分配到逻辑卷中。

注意：将分区表保存在LVM所在媒体设备是个值得借鉴的方式。对于MBR可以使用类型8e，或GPT类型8e00。

从卷组中移除（物理）分区

如果在这个物理分区上创建了一个逻辑卷，请先移除它。

首先，分区中的所有数据需要被转移到别的分区，幸而LVM提供了以下的简便方式：

```
# pvmove /dev/sdb1
```

如果你想指定所要转移的目标分区，那么可以把该分区作为pvmove的第二个参数：

```
# pvmove /dev/sdb1 /dev/sdf1
```

接着，从卷组（VG）中移除物理卷（PV）：

```
# vgreduce myVg /dev/sdb1
```

或者把所有的空物理卷（PV）都移除掉：

```
# vgreduce --all vg0
```

最后，如果你仍然想要使用该分区，而且不想让LVM以为它是一个物理卷，那么你可以执行以下命令：

```
# pvremove /dev/sdb1
```

停用卷组

只需执行：

```
# vgchange -a n my_volume_group
```

这将停用此卷组，以便你卸载存储它的容器。

逻辑卷类型

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 是否需要为thin-provisioned卷和RAID卷的创建提供说明？（在 Talk:LVM# 中讨论）

除了普通的逻辑卷, LVM还支持：快照, logical volume caching, thin provisioned逻辑卷，以及RAID.

快照功能

介绍

LVM可以给系统创建一个快照，由于使用了写入时复制(copy-on-write) 策略，相比传统的备份更有效率。 初始的快照只有关联到实际数据的inode的实体链接(hark-link)而已。只要实际的数据没有改变，快照就只会包含指向数据的inode的指针，而非数据本身。一旦你更改了快照对应的文件或目录，LVM就会自动拷贝相应的数据，包括快照所对应的旧数据的拷贝和你当前系统所对应的新数据的拷贝。这样的话，只要你修改的数据（包括原始的和快照的）不超过2G，你就可以只使用2G的空间对一个有35G数据的系统创建快照。要创建快照，在卷组中必须有未被分配的空间。和其他逻辑卷一样，快照也会占用卷组中的空间。所以，如果你计划使用快照来备份你的根（root）分区，不要将整个卷组的空间都分配给根（root）逻辑卷。

配置

你可以像创建普通逻辑卷一样创建快照逻辑卷。

```
# lvcreate --size 100M --snapshot --name snap01 /dev/vg0/lv
```

你可以修改少于100M的数据，直到该快照逻辑卷空间不足为止。

要将逻辑卷卷'lv' 恢复到创建快照'snap01'时的状态，请使用：

```
# lvconvert --merge /dev/vg0/snap01
```

如果逻辑卷处于活动状态，则在下次重新启动时将进行合并（merging）(合并（merging）甚至可在LiveCD中进行)。

注意：合并后快照将被删除。

也以拍摄多个快照，每个快照都可以任意与对应的逻辑卷合并。

快照也可以被挂载，并可用**dd**或者**tar**备份。使用**dd**备份的快照的大小为拍摄快照后对应逻辑卷中变更过文件的大小。 要使用备份，只需创建并挂载一个快照，并将备份写入或解压到其中。再将快照合并到对应逻辑卷即可。

本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。

附注： 是否需要单独的文章来介绍：创建在系统更新前自动备份根目录的脚本，用于回滚。并修改`menu.lst`以引导备份？（在 Talk:LVM# 中讨论）

快照主要用于提供一个文件系统的拷贝，以用来备份; 比起直接备份分区，使用快照备份可以提供一个更符合原文件系统的镜像。

LVM 缓存

来自lvmcache(7):

: Cache逻辑卷将使用一个较小而快速的逻辑卷，来提高较大但慢速的逻辑卷的性能。它将大型逻辑卷中经常使用的块存储到较快的缓存卷中。 LVM将这个小而快速的逻辑卷称为缓冲池逻辑卷（cache pool LV）。 大而慢的逻辑卷则被称为源逻辑卷（origin LV）。由于dm-cache（内核驱动）的要求，LVM进一步将缓存池逻辑卷分为两个设备 - 缓存数据逻辑卷和缓存元数据逻辑卷。缓存数据逻辑卷存放源逻辑卷中常用块的拷贝，以提升源逻辑卷的速度。 缓存元数据逻辑卷则存储记录信息，这些信息指定了源逻辑卷中的数据被存放到缓存逻辑卷中的位置。 (即，确定源逻辑卷中的一个块是存储到自身空间还是缓存逻辑卷中？)。要想创建最好、最稳定的缓存逻辑卷，你必须熟悉这些信息。所有这些被提到的逻辑卷都必须存放在一个卷组中。

创建缓存

最快速的方法是直接在快速的设备上创建一个物理卷，并把它添加到一个卷组中：

```
# vgextend dataVG /dev/sdx
```

只需一步，即可在sdb上创建一个缓存池，自动生成元数据，将逻辑卷dataLV缓存到sdb上：

```
# lvcreate --type cache --cachemode writethrough -L 20G -n dataLV_cachepool dataVG/dataLV /dev/sdx
```

显然，如果你想让缓存变得更大，可更改 `-L`参数。

注意：Cachemode有两个可能的参数：

- `writethrough` 确保任何数据写入都会被同时存储到缓存池逻辑卷和源逻辑卷中。 在这种情况下，丢失与缓存池逻辑卷关联的设备不会丢失任何数据；
- `writeback` 可提供更好的性能, 如果用于缓存的设备发生故障，数据丢失的风险会更高。

如果未指定`--cachemode`，将会自动选择`writethrough`。

删除缓存

如果需要撤消上面的创建操作：

```
# lvconvert --uncache dataVG/dataLV
```

这会将缓存中挂起的写入操作提交到源逻辑卷， 然后删除缓存逻辑卷。 其它可用的参数和介绍，请参见：lvmcache(7).

RAID

来自lvmraid(7):

: lvm(8) RAID是一种创建逻辑卷的方法，它使用多个物理设备来提高性能或容错能力。在LVM中，这些物理设备是单个卷组中的物理卷。

LVM RAID支持RAID 0，RAID 1，RAID 4，RAID 5，RAID 6和RAID 10。每个RAID等级的细节请见：Wikipedia:Standard RAID levels

配置RAID

创建物理卷：

```
# pvcreate /dev/sda2 /dev/sdb2
```

创建卷组：

```
# vgcreate VolGroup00 /dev/sda2 /dev/sdb2
```

使用`lvcreate --type *raidlevel*`参数创建逻辑卷。更多选项，请参见lvmraid(7)和lvcreate(8)。

```
# lvcreate --type RaidLevel [OPTIONS] -n Name -L Size VG [PVs]
```

例如：

```
# lvcreate --type raid1 --mirrors 1 -L 20G -n myraid1vol VolGroup00 /dev/sda2 /dev/sdb2
```

这将会在设备`/dev/sda2`、`/dev/sdb2`上和"VolGroup00"卷组中创建一个20GiB的镜像（mirrored）逻辑卷"myraid1vol"。

为RAID配置mkinitcpio

如果你的根文件系统在LVM RAID上，除了`lvm2`或`sd-lvm2`钩子，你还需添加`dm-raid`以及恰当的RAID模块(例如`raid0`, `raid1`, `raid10`或`raid456`)到`mkinitcpio.conf`中的MODULES数组中。

对于基于busybox的initramfs：

```
/etc/mkinitcpio.conf
MODULES=(dm-raid raid0 raid1 raid10 raid456)
HOOKS=(base udev ... block lvm2 filesystems)
```

对于基于systemd的initramfs：

```
/etc/mkinitcpio.conf
MODULES=(dm-raid raid0 raid1 raid10 raid456)
HOOKS=(base systemd ... block sd-lvm2 filesystems)
```

图形化配置

目前没有“官方的”管理LVM卷的GUI工具，不过system-config-lvmAUR\[损坏的链接：package not found\]涵盖了大多数常见操作，并提供卷状态的简单可视化。它还可以在调整逻辑卷大小时自动调整许多文件系统的大小。

常见问题

由于禁用lvmetad带来的开/关机问题

”必须“在`/etc/lvm/lvm.conf`中设置`use_lvmetad = 1`。 现在这已时默认设置 - 如果存在 `lvm.conf.pacnew`文件，你必须合并这个更改。

LVM 命令不起作用

- 加载以下模块:

```
# modprobe dm_mod
```

正常情况下，`dm_mod`模块应当被自动加载。假如该模块无法被自动加载，你可以试着修改`/etc/mkinitcpio.conf`：

!\[](https://wiki.archlinuxcn.org/wzh/File:Tango-preferences-desktop-locale.png)**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 英文章不正。是否应当考虑加入到"/etc/modules-load.d"？ （在 Talk:LVM# 中讨论）

```
/etc/mkinitcpio.conf:
MODULES="dm_mod ..."
```

你需要重建initramfs来提交你对`/etc/mkinitcpio.conf`的更改。

- 测试以*lvm*开头的命令是否可以被正确执行，例如：

```
# lvm pvdisplay
```

逻辑卷无法显示

如果你在挂载某个已创建好的逻辑卷时，发现它没有出现在`lvscan`命令的结果列表里，但是使用`lvdisplay -a`可找到此卷且状态为Not Available，那么你可以用以下命令去激活它：

```
# vgscan
# vgchange -ay
```

在可移除设备上的LVM问题

症状：

```
# vgscan
 Reading all physical volumes.  This may take a while...
 /dev/backupdrive1/backup: read failed after 0 of 4096 at 319836585984: Input/output
error
 /dev/backupdrive1/backup: read failed after 0 of 4096 at 319836643328: Input/output
error
 /dev/backupdrive1/backup: read failed after 0 of 4096 at 0: Input/output error
 /dev/backupdrive1/backup: read failed after 0 of 4096 at 4096: Input/output error
```

```
 Found volume group "backupdrive1" using metadata type lvm2
 Found volume group "networkdrive" using metadata type lvm2
```

产生原因：

: 在停用卷组（VG）之前就移除掉外部的LVM设备。在你断开连接之前，请确保以下命令被执行：

```
# vgchange -an volume group name
```

解决方案：如果你已经用`# vgchange -ay *vg*`命令来激活卷组，但仍有*Input/output error*的错误信息。执行以下命令：

```
# vgchange -an volume group name
```

移除外部设备，稍候几分钟后再执行以下命令：

```
# vgscan
# vgchange -ay volume group name
```

无法调整相邻逻辑卷的大小

在尝试扩大一个逻辑卷时出现以下错误：

```
" Insufficient suitable contiguous allocatable extents for logical volume "
```

其原因是此逻辑卷在创建时使用了显式连续分配策略（an explicit contiguous allocation policy）(`-C y`选项或`--alloc contiguous`)，并且没有其他相邻的连续数据块可用(另见reference)。

要修复此错误，在扩展逻辑卷之前，使用`lvchange --alloc inherit <logical_volume>`更改其分配策略。 如果需要保持连续分配策略，另一种方法是将卷移动到具有足够可用区域的磁盘区域(见)。

"grub-mkconfig"命令报告"unknown filesystem"错误

请确保在生成grub.cfg前移除了快照卷。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 内容未翻译（在 Talk:LVM# 中讨论）

Thinly-provisioned root volume device times out

With a large number of snapshots, `thin_check` runs for a long enough time so that waiting for the root device times out. To compensate, add the `rootdelay=60` kernel boot parameter to your boot loader configuration. Or, make `thin_check` skip checking block mappings (see) and regenerate the initramfs:

```
/etc/lvm/lvm.conf
thin_check_options = [ "-q", "--clear-needs-check-flag", "--skip-mappings" ]
```

Delay on shutdown

If you use RAID, snapshots or thin provisioning and experience a delay on shutdown, make sure `lvm2-monitor.service` is started. See FS#50420.

更多资源

- LVM2 Resource Page on SourceWare.org
- LVM HOWTO article at The Linux Documentation project
- LVM article at Gentoo wiki
- LVM2 Mirrors vs. MD Raid 1 post by Josh Bryan
- Ubuntu LVM Guide Part 1Part 2 detals snapshots

15.2 RAID

难度 高级 | **篇幅** 46.1 KB | **所属** 存储虚拟化

本节将讲到

- RAID 级别
- 实现方式
- 安装
- 在 Live CD 中挂载 RAID
- 在 RAID 上安装 Arch Linux

独立磁盘冗余阵列 (**R**edundant **A**rray of **I**ndependent **D**isks，RAID) 是一种将多个磁盘驱动器组件（通常是多块硬盘或多个分区）组合为一个逻辑单元的存储技术。根据 RAID 的部署情况，这个逻辑单元可以是单个的文件系统，也可以是一个能在其上建立多个分区的透明中间层。

根据所需的冗余量和性能要求，数据按照 #RAID 级别中的某一种方式分布在驱动器中。所选的 RAID 级别决定了是否可以防止数据丢失（硬盘故障时）、是否提高性能或结合两者优势。

本文介绍如何使用 mdadm(8) 创建并管理一个软件磁盘阵列。

警告：确保在操作前备份所有数据。

RAID 级别

尽管大部分 RAID 级别都或多或少地包含了数据冗余，RAID 并不能完全保证数据安全。如果遇到火灾、计算机被盗或者多块硬盘同时损坏，RAID 将无法保护数据。此外，配置一个带有 RAID 的系统是一个复杂的过程，可能会破坏现有数据。

基本 RAID 级别

有多种不同的基本 RAID 级别，下面列出了最常用的几种。

RAID 0：将多块硬盘组合为一个条带卷'并不提供数据冗余（*实际上会降低可靠性），但它仍可以被当做是 RAID，而且它确实提供了*巨幅的速度提升。*如果提高速度比数据安全更重要（比如作为 swap 分区），可以选择这种 RAID 级别。在服务器上，*

RAID 1 和 RAID 5 阵列更加合适。在 RAID 0 阵列中，块设备的大小是最小组成分区的大小乘以组成分区的数量。

RAID 1 : 这是最直接的 RAID 级别：完全镜像。与其他 RAID 级别一样，它只在分区位于不同物理硬盘上才有效。如果某一块硬盘损坏，由 RAID 阵列提供的块设备将不受影响。可以使用 RAID 1 的情境包括了除 swap 和临时文件外的其他所有情境。请注意，如果使用由软件实现的 RAID，引导分区只能选择 RAID 1，因为读取引导分区的引导器通常无法辨识 RAID，但一个 RAID 1 的组成分区可以像常规分区一样读取。RAID 1 阵列块设备的大小是最小组成分区的大小。

RAID 5 : 需要至少 3 块物理硬盘，并结合了 RAID 1 的数据冗余和 RAID 0 的速度与可用空间上的优势。RAID 5 使用了类似 RAID 0 的条带化技术，同时也将奇偶校验块*分布式地存储在每一块磁盘上*。如果遭遇硬盘损坏，这些奇偶校验块就可以用来在替代的新磁盘上重建损坏的数据。RAID 5 仅可弥补一个组成磁盘损坏带来的损失。 : > **注意**：RAID 5 是结合了速度与数据冗余优势的常用选择。但值得注意的是，当一块硬盘损坏而没有及时更换，此时若再有硬盘损坏，则所有数据都将丢失。此外，考虑到现代磁盘的超大容量和消费级硬盘无法恢复的读取错误率 (Unrecoverable read error, URE)，超过 4TiB 的阵列在重建数据时出现至少一处读取错误 (URE) 的概率几乎在**预料之中**（概率大于 50%）。因此，存储行业不再推荐使用 RAID 5。

RAID 6 : 需要至少 4 块物理硬盘，提供了和 RAID 5 一样的优势并且在两块硬盘损坏时仍能保证数据安全。RAID 6 使用了和 RAID 5 类似的条带化技术，但是把两个不同的奇偶校验块*分布式地存储在每一块磁盘上*。如果磁盘发生故障，这些奇偶校验块将用于重建替换磁盘上的数据。RAID 6 可以承担两个组成磁盘的损失。在抵御无法恢复的读取错误 (Unrecoverable read error, URE) 时也某种程度上更加可靠，因为磁盘阵列在重建某一块损坏硬盘的数据时仍然有奇偶校验块可以校验数据。但是，总体而言，RAID 6 开销较大，大多数时候 far2 布局的 RAID 10（参见下文）提供了更快的速度和更强的可靠性，因此更倾向于采用 RAID 10。

嵌套 RAID 级别

RAID 1+0 : RAID1+0 是一种结合了两种基本 RAID 级别的嵌套级别，它相对基本级别提高了性能且增加了冗余量。它通常被称为 *RAID10*，但是，Linux MD（内核自带的 RAID 实现）支持的 RAID10 不是简单的两层 RAID 重叠，请看下文。

RAID 10 : Linux 下的 RAID10 建立在 RAID1+0 的概念上，但它将其实现为单一的一层，这一层可以有多种不同的布局。可参考创建复杂 RAID 10。 : 在 Y 块硬盘上的*近 X 布局*在不同硬盘上重复储存每个数据块 X 次，但不需要 Y 可以被 X 整除。数据块放

在所镜像的磁盘上几乎相同的位置，这就是*近布局*名字的来源。它可以工作在任意数量的磁盘上，最少是 2 块。在 2 块硬盘上的近 2 布局相当于 RAID1，4 块硬盘上的近 2 布局相当于 RAID1+0。 : 在 Y 块硬盘上的*远 X 布局*设计用于在镜像阵列中提供与条带化技术一样快的读取速度。它通过把每块硬盘分成前后两部分来实现这一点，写入第一块硬盘前半部分数据也会写入第二块硬盘的后半部分，反之亦然。这样可以达到一个效果，那就是把对连续数据的读取条带化，而这正是 RAID0 和 RAID5 读取性能高的原因。它的缺点在于写入连续数据时有轻微性能损失，因为硬盘磁头要运动到另一片区域来写入镜像。当数据读取性能和可用性/冗余性一样重要时，比起 RAID1+0 和 RAID5，更应该优先考虑远 2 布局的 RAID10。需注意这种方式仍无法代替备份。详情请阅读维基百科相关页面。

警告：mdadm cannot reshape arrays in *far X* layouts which means once the array is created, you will not be able to `mdadm --grow` it. For example, if you have a 4x1TB RAID10 array and you want to switch to 2TB disks, your usable capacity will remain 2TB. For such use cases, stick to *near X* layouts.

RAID 级别对比

RAID 级别	数据冗余	物理设备利用率	读取性能	写入性能	最少磁盘数量
0	否	100%	n 倍 **最优**		

最优

较优

较优

最优; 与 RAID0 相当但加入了数据冗余

\ *其中* n* 表示用于组成阵列的磁盘数量。

LINEAR

LINEAR allows to map two or more devices into a single device, without parallel accesses like **RAID0** but allowing to fully use disks from different sizes. To create a pseudo RAID using this mode without *mdadm*, one can either use the low-level dmsetup(8) utility, the high-level LVM framework or the Btrfs filesystem.

实现方式

RAID 设备可以用不同方式来管理：

软件 RAID : 这是最简单的实现方式，因为它不依赖于专用固件或专有软件。这种阵列由操作系统通过以下方式进行管理：

- 通过抽象层管理（比如 mdadm）；

> **注意**：这是在本指南下文将要使用的方法。

- 通过逻辑卷管理器来管理（比如 LVM）；
- 通过文件系统的某个组件来管理（比如 ZFS，Btrfs）。

硬件 RAID：这种阵列由安装在计算机上的专用硬件卡直接管理，硬盘直接连接在该计算机上。RAID 的处理逻辑由板载的处理器完成，它独立于主处理器 (CPU)。尽管这种方案独立于任何操作系统，但却需要驱动程序来使硬件 RAID 控制器正常工作。取决于不同的制造商，硬件 RAID 阵列可以在 option ROM 里设置或者操作系统安装完成后另行安装配套软件来设置。这种设置是独立于 Linux 内核的：内核并不能看到单独的每块硬盘。

FakeRAID：这种类型的 RAID 应当称为 BIOS 或板载 RAID，却常被错误地当做硬件 RAID 来宣传。这种阵列由伪 RAID 控制器来管理，RAID 逻辑由 option ROM 或在 UEFI 情况下由安装了 EFI Sata 驱动程序的固件本身来完成，但并不是实现了*所有* RAID 功能的完整硬件 RAID 控制器。因此，这种 RAID 有时被称为 FakeRAID。dmraid包 会被用于处理这种控制器。这里列出一些 FakeRAID 控制器：Intel Rapid Storage，JMicron JMB36x RAID ROM，AMD RAID，ASMedia 106x 和 NVIDIA MediaShield。

我正在使用哪一种 RAID？

由于软件 RAID 是由用户部署的，因此用户很容易就知道 RAID 的类型。

但是，辨别 FakeRAID 和真正的硬件 RAID 是很困难的。如上所述，制造商经常错误地混淆这两种类型的 RAID，还可能进行虚假宣传。这种情况下应该采取的最好方式是运行 `lspci` 命令并在输出信息中找到你的 RAID 控制器，然后根据这些信息进行更进一步的搜索。硬件 RAID 控制器会在这一列表中出现，但 FakeRAID 不会。同时，真正的硬件 RAID 控制器通常价格很高，如果这个系统是自行组装的，那么很可能安装一个硬件 RAID 会使电脑的价格显著提高。

安装

安装 mdadm包。*mdadm* 用于管理在纯块设备上建立起来的纯软件 RAID：底层硬件不提供任何 RAID 逻辑，只是一些磁盘而已。*mdadm* 可以在任何块设备集合上工作，甚至是那些非常规的设备。比如，你可以用一堆 U 盘来建立 RAID 阵列。

准备设备

警告：这些步骤会擦除指定设备上的所有数据，输入命令请小心！

如果设备是旧设备重用或刚从一个现有的阵列上拆下来，请擦除所有旧的 RAID 配置信息：

```
# mdadm --misc --zero-superblock /dev/drive
```

或者是删除设备上的某个特定的分区：

```
# mdadm --misc --zero-superblock /dev/partition
```

注意：

- 清除一个分区的 superblock 不会影响到磁盘上的其他分区。
- 考虑到 RAID 本身的功能特点，在运行中的磁盘阵列中完全地 安全擦除磁盘 是很困难的。请在创建阵列前考虑要不要安全擦除磁盘。
- You can do the whole disk preparation procedure from a GUI with blivet-guiAUR.

对磁盘进行分区

强烈推荐对用于阵列的硬盘进行分区。考虑到大多数 RAID 用户会用到超过 2 TiB 的硬盘，因此推荐并要求使用 GPT。参阅分区获取关于磁盘分区的更多信息以及可供使用的分区工具。

注意：也可以在裸磁盘（没有分区的磁盘）上直接创建 RAID，但不推荐这么做，因为这样可能会导致更换损坏硬盘时出现问题

注意：当更换 RAID 中的某块损坏的硬盘时，新硬盘的大小必须恰好等于或大于损坏的硬盘，否则无法完成阵列重建过程。即使是同一厂商相同型号的硬盘也可能有容量上的细微差别。通过在磁盘末尾保留一些未分配的空间可以消除磁盘容量上的细微差异，这样可以使替代磁盘的型号选择更加容易。因此，最好在磁盘末尾留出大约 100 MiB 的未分配空间。

GUID 分区表

- 创建分区以后，这些分区的类型标识符 (GUID) 应该是 `A19D880F-05FC-4D3B-A006-743F0F84911E`（在 *fdisk* 里将分区类型改为 `Linux RAID` 或在 *gdisk* 里改为 `FD00` 可以给所选分区分配这个标识符）。
- 如果使用了一个更大的磁盘阵列，可以考虑分配文件系统标签\[损坏的链接：无效的章节\] 或分区标签\[损坏的链接：无效的章节\] 用于以后区分每块单独的硬盘。
- 建议在每个设备上创建大小相同的分区。

主引导记录 (MBR)

对于在使用 MBR 的硬盘上创建分区的用户，可用的分区类型 ID 包括：

- `0xDA`：无文件系统（*fdisk* 中称为 `Non-FS data`）。这是**推荐**用于 Arch Linux 上 RAID 阵列的 mdadm 分区类型。
- `0xFD`：自动检测的 RAID（*fdisk* 中称为 `Linux RAID autodetect`）。This partition type should only be used if RAID autodetection is desireable (non-initramfs system, old mdadm metadata format).

更多信息请参阅 Linux Raid Wiki:Partition Types。

创建阵列

使用 `mdadm` 来创建阵列。参阅 mdadm(8) 获取支持的选项。下面列出部分使用范例。

警告：不要简单地复制/粘贴下面的示例，确保你已经使用正确的选项和设备名称替代了示例中相应的内容。

注意：

- 如果有一个从 Syslinux 启动的 RAID1 阵列，在 syslinux v4.07 中要求元数据值为 1.0，而不是默认的 1.2。
- 当用 Arch 安装镜像 创建磁盘阵列时，请使用 `--homehost=*yourhostname*` 选项设置 主机名（或者 `--homehost=any` 选项无论在什么主机上都用相同的主机名），否则主机名称 `archiso` 会被写入阵列的元数据中。

提示：要为 RAID 设备指定一个自定义的名称，可以使用 `--name=*MyRAIDName*` 选项或将 RAID 设备路径改为 `/dev/md/*MyRAIDName*`。Udev 会使用该名称创建指向 `/dev/md/` 内 RAID 阵列的符号链接。如果 `homehost` 与当前 主机名 匹配（或者 homehost 设为了 `any`）则链接将会是 `/dev/md/*name*`，如果 homehost 不匹配那么链接将会是 `/dev/md/*homehost*:*name*`。

下面这个例子展示了在 2 个设备上建立 RAID1 阵列：

```
 # mdadm --create --verbose --level=1 --raid-devices=2 /dev/md/MyRAID1Array /dev/sdb1 /dev/sdc1
```

下面这个例子展示了使用 4 块磁盘作为工作 (active) 磁盘，1 块作为备用 (spare) 磁盘建立 RAID5 阵列：

```
 # mdadm --create --verbose --level=5 --raid-devices=4 /dev/md/MyRAID5Array /dev/sdb1 /dev/sdc1 /dev/sdd1 /dev/sde1 --spare-devices=1 /dev/sdf1
```

提示：`--chunk` 可用于修改默认的区块大小。更多关于优化区块大小的内容请参阅 Chunks: the hidden key to RAID performance。

下面这个例子展示了在 2 个设备上建立远 2 布局的 RAID10 阵列 (RAID10, far2)：

```
 # mdadm --create --verbose --level=10 --raid-devices=2 --layout=f2 /dev/md/MyRAID10Array /dev/sdb1 /dev/sdc1
```

这样，阵列就会在虚拟设备 `/dev/mdX` 下建立起来，容量已经合并且可以使用（但处于降级模式）。mdadm 在后台同步数据时你已经可以直接开始使用这个阵列了。储存奇偶校验位可能要花很长时间，可以用这个命令查看进度：

```
$ cat /proc/mdstat
```

更新配置文件

默认情况下，`mdadm.conf` 中的大部分内容都被注释掉了，它应该只包含如下内容：

```
/etc/mdadm.conf
...
DEVICE partitions
...
```

这一指令告诉 mdadm 检查由 `/proc/partitions` 引用的所有设备并尽可能将其中的阵列组合起来。如果你确实想启动所有可用的阵列并且确信不存在意料之外的超级块（比如安装了新的存储设备），那么这样的配置就很好。当然有一种更精准的控制方法，就是显式地将阵列添加到 `/etc/mdadm.conf`：

```
# mdadm --detail --scan >> /etc/mdadm.conf
```

这将会在配置中添加类似这样的内容：

```
/etc/mdadm.conf
...
DEVICE partitions
...
ARRAY /dev/md/MyRAID1Array metadata=1.2 name=pine:MyRAID1Array
UUID=27664f0d:111e493d:4d810213:9f291abe
```

这也会使 mdadm 检查由 `/proc/partitions` 引用的设备。但是，只有超级块的 UUID 是 `27664…` 的设备才会被组合成激活的阵列。

更多信息请参阅 mdadm.conf(5)。

组合成磁盘阵列

更新配置文件后即可用 mdadm 组合磁盘阵列：

```
# mdadm --assemble --scan
```

格式化 RAID 上的文件系统

提示：To create multiple volumes inside a RAID array, follow the LVM on software RAID article.

现在磁盘阵列已经可以像普通分区一样被格式化成某个文件系统，但要记住：

- 不是所有文件系统都支持超大分区（参阅 Wikipedia:Comparison of file systems#Limits）。
- 文件系统需要支持在线增大和收缩（参阅 Wikipedia:Comparison of file systems#Features）。
- 应合理计算跨度大小和带区宽度来获得最佳性能。

计算 stride（跨度大小）和 stripe width（带区宽度）

优化文件系统结构以适应底层 RAID 结构需要 2 个参数：*stride* 和 *stripe width*。它们对应于 RAID 的 *chunk size（区块大小）*、文件系统的 *block size（块大小）*以及 *"data disks"（数据盘）的数量*。

Chunk size（RAID 区块大小）是 RAID 阵列的一个属性，在阵列创建时就已经定好了。目前 `mdadm` 默认该值为 512 KiB。这个参数可以用 `mdadm` 读取：

```
# mdadm --detail /dev/mdX | grep 'Chunk Size'
```

Block size（块大小）是文件系统的一个属性，在文件系统创建时就已经定好了。在大部分文件系统上（包括 ext4）默认是 4 KiB。更多关于当前系统 ext4 的信息可以查看 `/etc/mke2fs.conf` 文件。

"Data disks"（数据盘）的数量指的是阵列能够完全重建数据所要求的最少可用设备。例如，对于 N 个设备的 raid0 来说这个数量是 N，对于 raid5 来说是 N-1。

当你获得了这三个参数时，stride 和 stripe width 可以用下列公式来计算：

```
stride = chunk size / block size
stripe width = number of data disks * stride
```

范例 1. RAID0

本范例使用了正确的 stride 和 stripe width 将合并后的分区格式化成了 ext4：

- 假设这一 RAID0 阵列是由 2 块物理硬盘组成的。
- Chunk size（RAID 区块大小）是 512 KiB。
- Block size（文件系统块大小）是 4 KiB。

因为 stride = chunk size / block size。在这个例子中，stride 大小是 512/4 = 128。

因为 stripe width = # of physical **data** disks \ *stride*。*在这个例子中，stripe width 的大小是 2*\128 = 256。

```
 # mkfs.ext4 -v -L myarray -b 4096 -E stride=128,stripe-width=256 /dev/md0
```

范例 2. RAID5

本范例使用了正确的 stride 和 stripe width 将合并后的分区格式化成了 ext4：

- 假设这一 RAID5 阵列由 4 块物理硬盘组成，3 块是数据盘，1 块是奇偶校验盘。
- Chunk size（RAID 区块大小）是 512 KiB。
- Block size（文件系统块大小）是 4 KiB。

因为 stride = chunk size / block size。在这个例子中，stride 大小是 512/4 = 128。

因为 stripe width = # of physical **data** disks \ *stride*。*在这个例子中，stripe width 的大小是 3*\128 = 384.

```
 # mkfs.ext4 -v -L myarray -b 4096 -E stride=128,stripe-width=384 /dev/md0
```

关于 stride 和 stripe width 的更多信息，请参阅：RAID Math。

范例 3. RAID10,far2

本范例使用了正确的 stride 和 stripe width 将合并后的分区格式化成了 ext4：

- 假设这一 RAID10 阵列由 2 块物理硬盘组成。考虑到远 2 布局的 RAID10 的自身特性，两块硬盘都是数据盘。
- Chunk size（RAID 区块大小）是 512 KiB。
- Block size（文件系统块大小）是 4 KiB。

因为 stride = chunk size / block size。 在这个例子中，stride 大小是 512/4 = 128。

因为 stripe width = # of physical **data** disks \ *stride*。 *在这个例子中，stripe width 的大小是* 2\128 = 256。

```
# mkfs.ext4 -v -L myarray -b 4096 -E stride=128,stripe-width=256 /dev/md0
```

在 Live CD 中挂载 RAID

如果你需要在 Live CD 中挂载 RAID 分区，用这个命令：

```
# mdadm --assemble /dev/mdnumber /dev/disk1 /dev/disk2 /dev/disk3 /dev/disk4
```

如果缺一块盘的 RAID 1 被错误地识别为了 RAID 1（参照 `mdadm --detail /dev/md*number*`）并报告为非活动状态（参照 `cat /proc/mdstat`），需要先停止磁盘阵列：

```
# mdadm --stop /dev/mdnumber
```

在 RAID 上安装 Arch Linux

注意：本节仅适用于根文件系统在磁盘阵列上的情况。如果你的磁盘阵列上只是一个数据分区，那么可以跳过本节。

你应该在安装过程中的分区和格式化步骤之间创建 RAID 阵列。这将会把一个位于 RAID 阵列上的分区格式化成根文件系统，而不是直接格式化一个分区。 按照 #安装 一节的步骤创建 RAID 阵列。，然后继续安装过程，直到 pacstrap 步骤完成。 当使用 UEFI 启动时，还需要阅读 EFI 系统分区#ESP在软RAID1上。

更新配置文件

注意：这些操作应该在 chroot 以外完成，因此要在文件路径前加上 `/mnt`。

在基本系统安装完成以后，RAID 的默认配置文件 `mdadm.conf` 需要这样来更新：

```
# mdadm --detail --scan >> /mnt/etc/mdadm.conf
```

在运行这个命令以后一定要用文本编辑器检查 `mdadm.conf` 配置文件来确保它的内容看起来是合理的。

注意：为防止系统启动时 `mdmonitor.service` 启动失败（由 udev 启动），你需要取消 `MAILADDR` 的注释，并且在 `mdadm.conf` 结尾留下可处理磁盘阵列出错通知的邮件地址和/或应用程序。参阅 #启用事件邮件通知。

现在继续安装过程直到 安装指南#关于 initramfs 步骤之前为止，然后按照下一节的步骤做。

配置 mkinitcpio

注意：这些操作应该在 chroot 时完成。

安装 mdadm包，并向 `mkinitcpio.conf` 中的钩子)数组添加 `mdadm_udev` 来为初始化内存盘添加 mdadm 支持：

```
/etc/mkinitcpio.conf
...
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block mdadm_udev filesystems fsck)
...
```

然后重新生成初始化内存盘。

注意：每次修改 `/etc/mdadm.conf` 都需要重新生成 initramfs。

配置引导加载器

根设备

将 `root` 参数指向映射的磁盘，例如：

```
root=/dev/md/MyRAIDArray
```

如果按上述用内核设备节点来指定映射磁盘的方法之后，从软件 RAID 分区启动失败了，那就用块设备持久化命名中的某种方法来指定映射的磁盘，例如：

```
root=LABEL=Root_Label
```

参考 GRUB#RAID。

RAID0 layout

注意：This also affects existing mdraid RAID0 users that upgrade from an older version of the Linux kernel to 5.3.4 or newer.

Since version 5.3.4 of the Linux kernel, you need to explicitly tell the kernel which RAID0 layout should be used: RAID0_ORIG_LAYOUT (`1`) or RAID0_ALT_MULTIZONE_LAYOUT (`2`). You can do this by providing the kernel parameter as follows:

```
raid0.default_layout=2
```

The correct value depends upon the kernel version that was used to create the raid array: use `1` if created using kernel 3.14 or earlier, use `2` if using a more recent version of the kernel. One way to check this is to look at the creation time of the raid array:

```
mdadm --detail /dev/md1
/dev/md1:
           Version : 1.2
     Creation Time : Thu Sep 24 10:17:41 2015
        Raid Level : raid0
        Array Size : 975859712 (930.65 GiB 999.28 GB)
      Raid Devices : 3
```

```
     Total Devices : 3
       Persistence : Superblock is persistent

       Update Time : Thu Sep 24 10:17:41 2015
             State : clean
    Active Devices : 3
   Working Devices : 3
    Failed Devices : 0
     Spare Devices : 0

        Chunk Size : 512K

Consistency Policy : none

              Name : archiso:root
              UUID : 028de718:20a81234:4db79a2c:e94fd560
            Events : 0

    Number   Major   Minor   RaidDevice State
       0     259        2        0      active sync   /dev/nvme0n1p1
       1     259        6        1      active sync   /dev/nvme2n1p1
       2     259        5        2      active sync   /dev/nvme1n1p2
```

Here we can see that this raid array was created on September 24, 2015. The release date of Linux Kernel 3.14 was March 30, 2014, and as such this raid array is most likely created using a multizone layout (`2`).

维护 RAID

数据清扫 (Scrubbing)

定期运行数据清扫 (Scrubbing) 来检查并修复错误是一种很好的做法。一次完整数据清扫可能会花费数个小时，具体取决于磁盘阵列的大小和配置。

启动数据清扫：

```
# echo check > /sys/block/mdX/md/sync_action
```

数据检查操作会扫描驱动器来检查坏扇区并自动修复它们。如果找到了包含损坏数据的好扇区（本扇区中的数据与另一块硬盘中记录本扇区应有的数据不符，例如奇偶校验块和另一块数据块相结合可以证明本数据块是错误的），那就不动作，但记录下这一事件（见下文）。这种“不动作”允许管理员自行检查坏扇区中原本的数据和从冗余数据中重建的数据，然后决定保留哪个。

与许多 mdadm 相关的任务/事项一样，数据清扫的进度也可以通过查看 `/proc/mdstat` 文件来查询。

例如：

```
$ cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4] [raid1]
md0 : active raid1 sdb1 sdc1
      3906778112 blocks super 1.2 [2/2] [UU]
      [>.....................]  check =  4.0% (158288320/3906778112) finish=386.5min
speed=161604K/sec
      bitmap: 0/30 pages [0KB], 65536KB chunk
```

安全地停止当前的数据清扫操作：

```
# echo idle > /sys/block/md0/md/sync_action
```

注意：如果在数据清扫暂停时重启了系统，将继续进行清扫。

数据清扫完成后，管理员可以检查有多少数据块（如果有的话）被标记为坏块：

```
# cat /sys/block/md0/md/mismatch_cnt
```

关于数据清扫的一般说明

注意：用户也可以 echo **repair** 到 `/sys/block/md0/md/sync_action` 里面，但是这是不明智的，因为一旦遇到数据不一致就被自动改写为一致。危险之处在于我们实际并不知道奇偶校验块和数据块哪个是对的（在 RAID1 中是不知道哪个数据块是对的）。因此自动清扫操作能不能用正确数据替换错误数据取决于运气。

以 root 身份设定一个 cron 任务来定期执行清扫是很好的。raid-checkAUR 会对此有所帮助。如果要用 systemd 定时器而不是 cron 来执行定期清扫，raid-check-systemdAUR 中包含了相同的脚本和配套的 systemd 定时器单元文件。

注意：对于常规的大容量驱动器，清扫工作耗时大约 **6 秒每 GB**（即每 TB 大约需要 1 小时 45 分钟），因此应合理确定 cron 任务或定时器的开始时间。

对清扫 RAID1 和 RAID10 的说明

由于 RAID1 和 RAID10 本质上就是没有数据缓冲的，所以即使阵列工作正常，它的不匹配计数仍然可能是个非零值。这些不匹配计数只存在于正在写入数据的区域，且它们不反映任何问题。但是，我们无法区分非零的不匹配计数到底代表正在写入数据还是确实存在问题。这是造成 RAID1 和 RAID10 阵列误报错误的根源。即使如此，仍然建议定期清扫以发现并纠正驱动器上可能出现的坏扇区。

从阵列中移除设备

要从阵列中移除一个块设备，先将这个设备标记为 faulty（故障）：

```
 # mdadm --fail /dev/md0 /dev/failing_array_member
```

现在从阵列中移除这个设备：

```
 # mdadm --remove /dev/md0 /dev/failing_array_member
```

If the device has not failed entirely, but you would like to replace it, e.g. because it looks like it is dying, you can actually handle replacement more gracefully by first adding a new drive and then telling mdadm to replace it.

For example, with `/dev/sdc1` as the new one and `/dev/sdb1` as the failing one:

```
# mdadm /dev/md0 --add /dev/sdc1
# mdadm /dev/md0 --replace /dev/sdb1 --with /dev/sdc1
```

The `--with /dev/sdc1` part is optional, but more explicit. See for more details.

永久移除设备（比如想把一个设备拿出来单独使用）： 先使用上述两个命令，然后：

```
# mdadm --zero-superblock /dev/failing_array_member
```

警告：

- 不要在 RAID0 阵列或者其他线性存放数据的阵列中进行这个操作！否则数据会丢失！
- 重新使用已经移除的硬盘却不清除它的超级块将会导致下次启动时丢失所有数据。（因为 mdadm 将会把它当做磁盘阵列的一部分来使用）。

停止使用某个阵列：

1. 卸载 (umount) 目标阵列
2. 用这个命令停止磁盘阵列运行：`mdadm --stop /dev/md0`
3. 将本节开头的三个命令在每块硬盘上都运行一遍。
4. 将 `/etc/mdadm.conf` 中的相关行移除。

向阵列中添加设备

可以在系统正在运行且设备已经挂载的情况下使用 mdadm 添加新设备。 按照前文所述的方法，使用现有阵列中相同的布局对新设备进行分区。

如果 RAID 阵列尚未组合，先组合它们：

```
# mdadm --assemble /dev/md0 /dev/sda1 /dev/sdb1
```

向阵列中添加新设备：

```
# mdadm --add /dev/md0 /dev/sdc1
```

这一步 mdadm 不会花费很长时间。

根据不同的 RAID 类型（比如 RAID1），mdadm 可能会把部分设备添加为备用设备，而不在这些设备上存放数据。可以使用 `--grow` 加上 `--raid-devices` 选项来增加 RAID 利用的磁盘数量。例如，将一个阵列增加至四块硬盘：

```
# mdadm --grow /dev/md0 --raid-devices=4
```

可以这样检查进度：

```
# cat /proc/mdstat
```

用这个命令检查已经添加的设备：

```
# mdadm --misc --detail /dev/md0
```

注意：对于 RAID0 设备可能会收到以下错误信息：

```
mdadm: add new device failed for /dev/sdc1 as 2: Invalid argument
```

这是由于上述命令会将新磁盘添加为 "spare"（备用）盘，但是 RAID0 中不存在备用盘。如果想往 RAID0 阵列中添加磁盘，你需要同时使用 "grow" 和 "add" 参数。如下所示：

```
# mdadm --grow /dev/md0 --raid-devices=3 --add /dev/sdc1
```

增大 RAID 卷的大小

如果给阵列安装了更大的磁盘，或者增大了分区大小，可能就需要增大 RAID 卷的大小以适应更大的可用空间。这一过程可用首先按照上文中关于更换磁盘的步骤来做。当 RAID 卷在更大的磁盘上重建完成后，这个卷需要 "grow" 来填充多出来的空间。

```
# mdadm --grow /dev/md0 --size=max
```

接着，在 RAID 卷 `/dev/md0` 上的现有分区可能需要调整大小。参阅分区获取更多信息。最后，上述分区上的文件系统也需要重新调整大小。如果用 `gparted` 来完成分区操作，这些都会自动完成。如果用的是其他工具，请手动卸载 (unmount) 文件系统并调整其大小。

```
# umount /storage
# fsck.ext4 -f /dev/md0p1
# resize2fs /dev/md0p1
```

修改同步速度限制

同步工作需要一定的时间。如果本机不需要完成其他任务，可以提高速度限制值。

```
# cat /proc/mdstat
 Personalities : [raid10]
 md127 : active raid10 sdd1 sdc1 sdb1 sda1
      31251490816 blocks super 1.2 512K chunks 2 far-copies [4/4] [UUUU]
      [=>...................]  resync =  5.2% (1629533760/31251490816) finish=2071.7min
speed=238293K/sec
      bitmap: 221/233 pages [884KB], 65536KB chunk
```

In the above example, it would seem the max speed is limited to approximately 238 M/sec.

查看当前速度限制（单位为 KiB/s）：

```
# sysctl dev.raid.speed_limit_min
dev.raid.speed_limit_min = 1000
# sysctl dev.raid.speed_limit_max
dev.raid.speed_limit_max = 200000
```

Set a new maximum speed of raid resyncing operations using sysctl:

```
# sysctl -w dev.raid.speed_limit_min=600000
# sysctl -w dev.raid.speed_limit_max=600000
```

然后查看同步速度和预计完成时间：

```
# cat /proc/mdstat
  Personalities : [raid10]
 md127 : active raid10 sdd1 sdc1 sdb1 sda1
      31251490816 blocks super 1.2 512K chunks 2 far-copies [4/4] [UUUU]
      [=>...................]  resync =  5.3% (1657016448/31251490816) finish=1234.9min
speed=399407K/sec
      bitmap: 221/233 pages [884KB], 65536KB chunk
```

RAID5 performance

To improve RAID5 performance for fast storage (e.g. NVMe), increase `/sys/block/mdx/md/group_thread_cnt` to more threads. For example, to use 8 threads to operate on a RAID5 device:

```
# echo 8 > /sys/block/md0/md/group_thread_cnt
```

See git kernel commit 851c30c9badf.

Update RAID superblock

To update the RAID superblock, you need to first unmount the array and then stop the array with the following command:

```
# mdadm --stop /dev/md0
```

Then you can update certain parameters by reassembling the array. For example, to update the `homehost`:

```
# mdadm --assemble --update=homehost --homehost=NAS /dev/md0 /dev/sda1 /dev/sdb1
```

See the arguments of `--update` for details.

监视运行情况

可以显示当前 RAID 设备状态的一行简单命令：

```
# awk '/^md/ {printf "%s: ", $1}; /blocks/ {print $NF}' </proc/mdstat
md1: [UU]
md0: [UU]
```

用 watch 监视 mdstat

```
# watch -t 'cat /proc/mdstat'
```

或者最好用 tmux包

```
# tmux split-window -l 12 "watch -t 'cat /proc/mdstat'"
```

用 iotop 追踪 IO

iotop包 可以显示各个进程的输入输出状态。请用这个命令来查看 RAID 线程的输入输出。

```
 # iotop -a $(sed 's/^/-p /g' <<<`pgrep "_raid|_resync|jbd2"`)
```

用 iostat 追踪 IO

sysstat包 包中的 *iostat* 实用程序可以显示各个设备和分区的输入输出统计。

```
# iostat -dmy 1 /dev/md0
# iostat -dmy 1 # all
```

mdadm via systemd

mdadm provides the systemd service `mdmonitor.service` which can be useful for monitoring the health of your raid arrays and notifying you if anything goes wrong.

This service is special in that it cannot be manually activated like a regular service; *mdadm* will take care of activating it via udev upon assembling your arrays on system startup, but it will **only** do so if an email address and/or program has been configured for its notifications (see below).

启用事件邮件通知

mdadm 提供了名为 `mdmonitor.service` 的 systemd 服务，可帮助监控 RAID 阵列的健康度，并在出现问题时通过邮件通知你。

这个服务较为特殊，它无法用通常的方法来激活；**仅**在 *mdadm* 按下面的步骤为通知配置邮件地址后，它才会在系统启动阶段组装阵列时，通过 udev 激活该服务。

警告：邮件地址配置错误会导致监控服务启动静默失败。

注意：在发送邮件前，需要有一个配置好的邮件传输代理。

通过编辑 `/etc/mdadm.conf` 并配置邮件地址来启用该功能：MAILADDR *user@domain*

接下来，通过如下命令来验证功能是否正常：

```
# mdadm --monitor --scan --oneshot --test
```

如果测试成功并邮件正常送达，则配置完成；在下次阵列完成组装时，`mdmonitor.service` 将开始监控阵列错误。

Program notifications

Like Email notification above, edit `/etc/mdadm.conf` and edit the line:

```
PROGRAM /usr/sbin/handle-mdadm-events
```

The argument for PROGRAM is the script you want to run for any event. Which then interact with proper network monitoring agents. Or even IM clients or push notification services like ntfy.sh for home users.

Test in the same way as for email notification above.

故障排除

如果你在重启计算机时遇到类似 "invalid raid superblock magic" 的错误，并且除了已经配置好的硬盘外又接了其他硬盘，请检查硬盘顺序是不是正确的。在安装 RAID 时，硬盘编号可能是 hdd、hde 和 hdf，但是重启后它们的编号可能变成了 hda、hdb 和 hdc。请相应地调整你的内核参数。这种情况经常发生。

Error: "kernel: ataX.00: revalidation failed"

如果你突然（在重启、修改 BIOS 设置后）遇到类似错误信息：

```
Feb  9 08:15:46 hostserver kernel: ata8.00: revalidation failed (errno=-5)
```

这并不意味着设备已经损坏。尽管你可能会在网上看到一些讲内核崩溃的链接指向最坏的结果，但总而言之，并不是内核崩溃。可能你不知怎么的在 BIOS 里或内核参数里修改了 APIC 或 ACPI 设置。把它改回来就好了。通常关闭 APIC 和/或 ACPI 就好了。

磁盘阵列以只读模式启动

当由 md 启动磁盘阵列时，超级块将被改写，数据同步可能已经开始了。要以只读模式启动，可以向内核模块 `md_mod` 传递参数 `start_ro`。设置了这个参数以后，新的磁盘阵列进入 'auto-ro' 模式，该模式停止了所有内部读写操作（更新超级块、再同步、数据恢复），并且会在第一个写入请求到来时自动切换至 'rw' （读写）模式。

注意：在第一个写入请求到来前使用 `mdadm --readonly` 命令可以将阵列设置为真正的 'ro' （只读）模式，还可以用 `mdadm --readwrite` 命令直接开始再同步而不用等待写入请求。

要在启动时设置该参数，在内核启动参数中添加 `md_mod.start_ro=1`。

或者通过内核模块#使用 /etc/modprobe.d/中的文件在模块加载时传递该参数，也可以直接从 `/sys/` 传递：

```
# echo 1 > /sys/module/md_mod/parameters/start_ro
```

在损坏或缺失磁盘的情况下恢复 RAID

当磁盘阵列中的一个驱动器由于任何原因损坏时，也可能会发生上述错误。如果在缺失一块硬盘的情况下你仍需要强制启动磁盘阵列，输入以下命令（根据实际修改）：

```
# mdadm --manage /dev/md0 --run
```

现在你应该可以用类似下面的命令来挂载它（如果在 fstab 里有它的话）：

```
# mount /dev/md0
```

现在磁盘阵列应该已经工作并且可以使用，但仍旧是缺一块盘的状态。因此需要按照上文 #准备设备所述再添加一个磁盘分区，然后就可以将新磁盘分区加入到磁盘阵列中：

```
# mdadm --manage --add /dev/md0 /dev/sdd1
```

如果输入：

```
# cat /proc/mdstat
```

你就能看到磁盘阵列已经激活并正在重建。

你可能还需要更新你的配置文件（参阅：#更新配置文件）。

基准测试

用于 RAID 基准测试的工具有很多种。不同 RAID 最显著的不同是多个线程在读取 RAID 卷时的速度提升程度。

bonnie++包 测试对一个或多个数据库类型文件的读写，以及对小文件的创建、读取、删除，这样可以模拟 Squid、INN 和 Maildir format e-mail 等程序的文件读写。其附带的 ZCAV 程序可以测试硬盘不同区域的性能，且不用向硬盘写入任何数据。

不应该使用 hdparm 来对 RAID 进行基准测试，因为它多次返回的结果会非常不一致。

参考资料

- Linux kernel RAID documentation
- Linux Software RAID (thomas-krenn.com)
- Linux RAID wiki entry in The Linux kernel archives
- How Bitmaps Work
- Chapter 19. Managing RAID of Red Hat Enterprise Linux 9 documentation
- Linux-RAID FAQ on the Linux documentation project
- BAARF(Archive.org) including *Why should I not use RAID 5?*(Archive.org) by Art S. Kagel

- Introduction to RAID, Nested-RAID: RAID-5 and RAID-6 Based Configurations, Intro to Nested-RAID: RAID-01 and RAID-10, and Nested-RAID: The Triple Lindy in Linux Magazine
- HowTo: Speed Up Linux Software Raid Building And Re-syncing
- Wikipedia:Non-RAID drive architectures

邮件列表

- Kernel Linux-Raid mailing list

mdadm

- mdadm source code
- Software RAID on Linux with mdadm in Linux Magazine
- Wikipedia - mdadm

论坛帖子

- Raid Performance Improvements with bitmaps
- GRUB and GRUB2
- Can't install grub2 on software RAID
- Use RAID metadata 1.2 in boot and root partition

集群文件系统

单机的一块磁盘或一层文件系统撑不住容量与可用性时，需要把多台机器的存储组织成统一访问的整体，这是本章要解决的问题。两节是两条互不相通的路线：Glusterfs 篇幅短，安装、配置之后重点落在启动时自动挂载卷，必读；Ceph 篇幅长，从术语讲到安装，再讲到启动一个存储集群，真要搭起来就按顺序读完，只求概念可只读术语。

最容易踩的坑是把两者当成同一类东西：术语体系和搭建流程并不通用，Ceph 的术语没读懂就急着起集群，会处处卡住。另一处是卷已配好却漏掉开机自动挂载，重启后挂载点空着，看上去像数据丢了。

16.1　Glusterfs

难度 入门 | **篇幅** 2.8 KB | **所属** 集群文件系统

本节将讲到

- 安装
- 配置
- 启动时自动挂载 gluster 卷
- 参考

翻译状态：

- 本文（或部分内容）译自 Glusterfs，最近一次同步于 2023-10-10，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Glusterfs 是一个可扩展的网络文件系统。

安装

安装 glusterfs包 包。

配置

根据不同需求，可按照多种不同方式配置 Glusterfs，包括分布式以及完全复制。下列示例创建了两个完全复制的服务器节点 gluster1 及 gluster2，各包含两个磁盘，其中 `sda` 用于操作系统，另一个磁盘 `sdb` 用于 Glusterfs。除非有特别说明，所有操作都在 glusterfs1 上执行：

- 在各台服务器上启动/启用 `glusterd.service` 服务。
- 链接两台服务器

```
# gluster peer probe gluster2
```

- 在各服务器上分区并格式化 glusterfs 磁盘
- 上游建议在磁盘上创建单一的 XFS 分区
- 在各服务器的 `/etc/fstab` 中添加如下内容以自动挂载磁盘，其中 `/dev/sd*XY*` 为对应的磁盘分区（例如 `/dev/sdb1`）。

```
/etc/fstab
/dev/sdXY /export/sdXY xfs defaults 0 0
```

- 在各服务器上挂载磁盘，并创建 *brick* 文件夹：

```
 # mkdir -p /export/sdXY/brick
```

- 在主服务器上启用复制

```
 # gluster volume create gv0 replica 2 gluster1.mydomain.net:/export/sdb1/brick
gluster2.mydomain.net:/export/sdb1/brick
```

- 确认卷是否正确创建

```
 # gluster volume info
```

- 启动卷

```
 # gluster volume start gv0
```

- 挂载卷

```
 # mkdir -p /mnt/glusterClientMount
 # mount -t glusterfs gluster1:/gv0 /mnt/glusterClientMount
```

启动时自动挂载 gluster 卷

要在启动时挂载 gluster 卷，systemd 需要等待网络正常及 `glusterd` 服务启动。可以在 fstab 中添加如下选项：

```
/etc/fstab
localhost:/gv0 /mnt/glusterClientMount glusterfs defaults,_netdev,x-
systemd.requires=glusterd.service,x-systemd.automount 0 0
```

参考

- glusterfs 官方安装手册
- 关于如何在 Arch Linux 安装 Glusterfs 的博客文章

16.2 Ceph

难度 入门 | **篇幅** 5.7 KB | **所属** 集群文件系统

本节将讲到

- 术语
- 安装
- 启动一个存储集群
- 参阅

翻译状态：

- 本文（或部分内容）译自 Ceph，最近一次同步于 2023-10-10，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Ceph 是一个专注于分布式的、弹性可扩展的、高可靠的、性能优异的存储系统平台，可用于为虚拟机提供块存储方案或通过 FUSE 提供常规的文件系统。Ceph 是个高度可配置的系统，管理者可以控制系统的各个方面。它提供了一个命令行界面用于监视和控制其存储集群。Ceph 也包含鉴证和授权功能，可兼容多种存储网关接口，如 OpenStack Swift 和 Amazon S3。

引自 Wikipedia: Ceph (software)):

: Ceph 是一个自由的存储平台软件，设计用于使用单个分布式计算机集群提供对象/块及文件存储服务。Ceph 的主要目标为：无单点故障的完全分布式结构，艾子节级的可扩展性以及获取自由。The data is replicated, making it fault tolerant.

引自 Ceph.com:

: Ceph 是一个针对高性能、高可靠性及可扩展性设计的分布式对象存储及文件存储系统。

警告：

术语

提示：官方文档提供了完整的术语表

- **Client** : 连接到 Ceph 集群获取数据，但不属于 Ceph 集群中一部分的模块。
- **MONs** : 即监视器，存放了集群状态及例如服务和数据位置等集群信息。
- **MDSs** : 即元数据服务器，为 Ceph 文件系统存储元数据，以减轻存储集群的负载（例如`ls` 等命令所需的信息）。
- **Node** : 运行如 OSDs 及 MONs 等 Ceph 服务的单台设备。
- **OSDs** : 即 OSD daemons，负责集群的数据存储工作，也同时负责多项其它任务，例如数据复制，恢复及重平衡。
- **Storage cluster** : 负载数据存储的核心软件（OSDs+MONs）。

安装

软件包

可以安装 cephAUR。如果愿意冒险，也可以安装开发版的 ceph-gitAUR。

存储集群的所有节点都要安装 cephAUR。

NTP 客户端

警告：应当同步监视器节点的时钟以避免时钟偏移，否则将导致集群性能下降甚至停止工作。官方文档建议所有节点都应采取某种方式同步时钟。

在节点上安装并运行时钟同步客户端，可参阅时钟同步。

启动一个存储集群

在使用存储集群前，需要先使用标识符和密钥初始化集群的监视器。

Ceph 的上游文档非常完善，且适用于最新版本。

要初始化存储集群，请参考官方手动部署指南中的步骤。

启动一个监视器

鉴于你所用的系统很可能使用了 systemd，你可以通过 systemd 单元启用监视器。

举个例子，对于名称为 `node1` 的监视器，可以按Systemd#使用单元中的描述启动并启用 `ceph-mon@node1.service`。

参阅

- 官方网站
- 主页
- 文档
- 官方源码下载
- GitHub organization
- Ceph
- Ceph中国社区

文件管理

图形界面里搬文件，装完系统后手上未必有顺手的工具：本章要做的是把 Dolphin 装起来、按习惯配置好，出状况时知道去哪里找答案。只有《Dolphin》一节，安装与配置必读且不宜颠倒——安装只让它能启动，配置才决定好不好用；Tips and Tricks 可慢读，日常效率多从这里来；故障排除和参见留到需要时再翻，跳过不影响主线。

最容易踩的坑是把装好当成用好：装完只算能打开文件，默认设置一概不改，视图与行为顺着别人的习惯走；另一个是遇到异常就换软件，其实故障排除里多半已有对应条目。

17.1　Dolphin

难度 进阶 | **篇幅** 14.7 KB | **所属** 文件管理

本节将讲到

- 安装
- 配置
- Tips and Tricks
- 故障排除
- 参见

翻译状态：

- 本文（或部分内容）译自 Dolphin，最近一次同步于 2025-07-19，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Dolphin 是 KDE 的默认文件管理器。请参阅 Dolphin 模拟器来了解与之重名的游戏机模拟器。

安装

安装 dolphin包 软件包.

插件

- audiocd-kio包: 添加音频CD支持
- baloo包: 添加标签支持 (查看 #文件标签)
- dolphin-plugins包: 添加 Git, Mercurial 和 Dropbox 支持
- kio-admin包 作为管理员来管理文件
- kompare包: 对比文件(或者选择两个文件，*{右键} > 使用……打开 > {你的对比工具}*.)

- konsole包: 聚合的终端面板

注意：一些插件可能仍需要通过以下步骤启用： *配置* > *配置 Dolphin...* > *右键菜单*

文件预览

- ffmpegthumbs包: 预览视频文件 (基于ffmpeg)
- icoutils包 : 预览 \.ico, \.*cur* 文件和嵌入了图标的 \.exe*
- kde-thumbnailer-apkAUR: 预览 \.apk* 文件
- kdegraphics-thumbnailers包: 图像文件、PDF 与 Blender \.blend* 文件
- kimageformats包: Gimp *.xcf*、*.heic*文件（用libheif包）
- qt6-imageformats包 : *.webp*、*.tiff*、*.tga*、*.jp2* 文件
- libappimage包 : 预览嵌入了图标的 \.AppImage* 中的图标
- resvgAUR: 快速准确地预览SVG图像缩略图
- kdesdk-thumbnailers包: 缩略图系统的插件
- raw-thumbnailerAUR: 预览 \.raw* 文件
- taglib包 : 音频文件

注意：预览一些特殊的文件类型必须在 Previews for specific file-types must be enabled in *配置* > *配置 Dolphin...* > *界面* > *预览*.

注意：要启用 resvgAUR 代替内置的 SVG 缩略图编辑器，请关闭 SVG 图像。

配置

单击打开文件夹/文件

配置单击打开文件，你需要安装qt6ct包并在终端运行它，然后它会在‘界面’标签处为你提供单击打开文件、更改主题等的选项。

如果你使用Kvantum主题引擎——打开*kvantum manager* > *配置当前主题* > *杂项* > *点击行为*。

更改默认终端模拟器

Dolphin和其他KDE软件默认使用konsole包。你可以运行`kcmshell6 componentchooser`并且在*其它...*弹窗中选择你的终端模拟器或者在弹窗选择区域输入运行命令以更改默认终端模拟器。（第二个选项将会为这个命令创建一个新的本地桌面快捷方式）

例如，为了能让Dolphin在 alacritty 中打开 tmux ，在选择`Other...`后输入 `alacritty -e tmux`。

这个设置也可以通过修改~/.config/kdeglobals的配置文件来更改。例如，在[General]区域添加 alacritty 来使用alacritty：

```
TerminalApplication=alacritty
TerminalService=Alacritty.desktop
```

在 TerminalApplication 输入一个命令，在 TerminalService 输入一个桌面快捷方式（TerminalService应该是可选的）

注意：这不会影响在Dolphin窗口中的终端（使用F4打开的）

**本文或本章节的事实准确性存在争议。**

原因： This does (only) work in KDE and might not be dolphin related and might not work in many other WMs.（在 Talk:Dolphin 中讨论）

你可以通过*键盘* > *快捷键*来选择你喜欢的终端并且为其设置一个*运行*快捷键为 Ctrl+Alt+T 以覆写Konsole的快捷键。请注意，以这种方式打开的终端可能不会遵循你在终端配置文件中设置的终端背景颜色，但否则它应该与在窗口中运行的实例一样。

KIO slaves

Dolphin 使用 *KIO slaves* 进行网络访问、垃圾箱和其他功能，这与使用 GVFS 的 GTK 文件管理器不同。可用协议显示在位置栏（可编辑模式）。要快速将它们添加为书签，请在工作区中单击鼠标右键，然后选择“添加到常用位置”。

您可以手动安装 KIO slaves。例如，要在 Dolphin 中浏览您的 Google 云端硬盘，请安装 kio-gdrive包。

Tips and Tricks

文件标签

Dolphin 为文件标签提供了广泛的支持。您可以通过右键单击文件并选择*分配标签*来向文件添加标签。您可以在“属性”菜单或“信息”面板查看文件上的标签。

Dolphin 使用 user.xdg.tags extended attribute直接将标签与每个文件一起存储。Baloo 将这些标签索引到自己的数据库中，以便快速搜索，并维护所有已知标签的列表。

激活Baloo 来在位置面板中展示所有被索引的标签并且可以通过搜索文件标签来搜索文件。

隐藏自定义文件/目录

文件/目录 可以通过创建`.hidden`文件，并在里面写入文件/目录的名字（一行一个）来隐藏

创建自定义服务菜单

你可以通过在以下路径中添加`*.desktop`来为dolphin添加自定义服务菜单入口（见 ）

- `~/.local/share/kio/servicemenus/`
- `/usr/share/kio/servicemenus/`

注意：Desktop 入口 **必须** 是可执行的以在dolphin中展出

```
gamemode.desktop
[Desktop Entry]
Type=Service
MimeType=application/*;
Actions=runWithGamemode

[Desktop Action runWithGamemode]
Name=Run with Gamemode
Name[de]=Mit Gamemode ausführen
Icon=input-gaming
Exec=gamemoderun %u
```

这个添加`Run with Gamemode`菜单项在所有*应用程序* mime types

提示：To create a servicemenu for all file-types, use the `application/octet-stream` mimetype.

故障排除

设备名称显示为"X GiB Harddrive"

创建一个文件系统标签或分区标签，Dolphin 会在设备列表中显示此标签，而不是大小。请参见块设备持久化命名#by-label.

在外部驱动器上将文件移动到废纸篓需要很长时间

若要将文件移入废纸篓，用户必须具有对废纸篓的独占访问权限。理由是您不希望其他人看到您删除的内容。为此，在外部驱动器上创建了一个文件夹 `.Trash-1000/`，具有权限模式`700`。

如果无法设置正确的访问权限，dolphin 会（与 GNOME 不同）将文件移动到主目录中的垃圾箱中，这需要时间。

要安装 U 盘/外部 HDD，Dolphin 使用 Udisks。FAT32 / EXFAT / NTFS 不支持 UNIX 文件权限，udisk 默认以模式 755 挂载它们。要配置 udisk 以使用模式 700 挂载这些驱动器，请查看文件 `/etc/udisks2/mount_options.conf.example` 。复制文件（名称应以 .conf 结尾），取消对相关部分的注释，并将三个文件系统添加到带有 xyz_defaults 选项 `fmask=177,dmask=077` 的行中。

(背景信息：,)

透明字体

使用 GTK Qt 样式时，选择框中的字体可能会变得透明。原生 Qt 样式（如 *Cleanlooks* 和 *Oxygen*）不受影响。

已安装SMB共享上的崩溃

参见 Samba#无法覆盖文件。.

图标不显示

如果 Dolphin 没有显示图标，安装并运行 qt6ct包，在图标主题选项卡中选择一个*图标主题*，然后*应用*。

如果图标仍未在 Dolphin 中显示，请在你的xprofile中将 `QT_QPA_PLATFORMTHEME` 环境变量设置为 `qt6ct` 。或者，使用`platformtheme`标志启动 Dolphin：

```
$ dolphin --platformtheme qt6ct
```

此外，当您使用不常见、不完整的图标主题时，请确保安装并继承备份图标主题，例如 *hicolor* 或 *Adwaita*。

图标太大

如果在 KDE 环境之外的 Dolphin 上图标过大，请用以下方法启动它：

```
XDG_CURRENT_DESKTOP=KDE KDE_SESSION_VERSION=6 QT_AUTO_SCREEN_SCALE_FACTOR=0 dolphin
```

不匹配的文件夹视图背景颜色

在 Plasma 以外的其他位置运行 Dolphin 时，文件夹视图窗格中的背景颜色可能与系统 Qt 主题不匹配。这是因为 Dolphin 从 `[Colors:View]` 中 `~/.config/kdeglobals`

```
~/.config/kdeglobals
...
[Colors:View]
BackgroundNormal=#2E2E2E
...
```

如果您在文件夹视图窗格周围看到蓝色边框（如果您处于拆分视图中，则只会在焦点窗格周围），您可以通过 qt6ct 应用程序应用 `fusion-fixes.qss` 样式表来摆脱它。这个答案描述了如何让 adwaita 黑暗主题在 Gnome 下为 dolphin 工作。

或者，使用 kvantum包 来管理您的 Qt6 主题。有关使用说明，请参阅Kvantum项目主页。

Zsh 配置文件未加载集成终端（Zsh profile not loading in integrated terminal）

**本文或本章节的事实准确性存在争议。**

原因： 这个*issue*不仅限于 Zsh;默认情况下，Konsole （以及 Konsolepart） 不会启动登录 shell。`zprofile` 应该在启动会话时由显示管理器加载（就像由 SDDM 完成的那样），而不是由终端模拟器加载。（在 Talk:Dolphin 中讨论）

如果您的 zsh 配置文件未加载，请尝试编辑您当前的配置文件。右键单击集成终端，然后单击*编辑当前配置文件...*并将启动命令编辑为 `/bin/zsh --login`

GTK 应用程序未使用Dolphin

参见统一 Qt 和 GTK 应用程序的外观#一致的文件对话框.

无法安装任何右键菜单插件

当您转到Dolphin菜单*配置>配置 Dolphin>右键菜单>下载新服务*并尝试安装任何服务时，您会收到以下错误消息：

```
/usr/bin/servicemenuinstaller: error while loading shared libraries:
libpackagekitqt6.so.1: cannot open shared object file: No such file or directory
```

这可以通过安装 packagekit-qt6包 来解决。然后重新启动 Dolphin。

在其他Window manager运行tong chang de，Dolphin无法找到应用

你可以通过安装 archlinux-xdg-menu包 软件包，并且运行：

```
$ XDG_MENU_PREFIX=arch- kbuildsycoca6 --noincremental
```

这更新KService 桌面文件系统配置文件缓存（见kbuildsycoca6(8)），很多KDE软件依赖它来选择桌面入口。`--noincremental`参数是可选的。由于archlinux-xdg-menu包会创建一个带有`arch-`的 XDG桌面菜单（见xdg-menu）， 因此`XDG_MENU_PREFIX`是必需的。

这个XDG 桌面菜单文件可以在`/etc/xdg/menus/*-applications.menu` 找到。

通常你不需要安装`kbuildsycoca6`，因为这是dolphin依赖的包kservice包的一部分

注意：这个解决方案仅仅是重新构建缓存并且不会使它更新

参见

- Wikipedia:Dolphin (file manager))
- KDE 用户群: Dolphin
- Dolphin 手册

日志

服务起不来、开机后行为莫名改变，最后往往都得靠日志说话。这一章要解决的是：把 journald 收来的日志按优先级筛出真正有用的那几行，并让占用可控。

只有 Systemd/Journal 一节，建议整节读完：优先级与功能、过滤输出是主线，须按顺序看透，跳过筛法直接翻大小限制，多半查不出问题。日志大小限制、配合 syslog 使用、手动清理日志可留到需要时回查。常见的误解是以为 journal 上线就该卸掉 syslog，其实两者可以并行；另一个是把日志占满磁盘当常态，等出事才手工清理，不如先把大小上限定好。

18.1　Systemd/Journal

难度 进阶 | **篇幅** 14.1 KB | **所属** 日志

本节将讲到

- 优先级
- 功能
- 过滤输出
- 日志大小限制
- 配合 syslog 使用

主文档请参考 systemd。

systemd 提供了自己的日志系统（logging system），称为 journal。使用 systemd 日志，无需额外安装日志服务（syslog）。使用 journalctl(1) 命令读取日志。

Arch Linux 中，`/var/log/journal/` 目录是 systemd包 软件包的一部分。默认情况下 `/etc/systemd/journald.conf` 中的`Storage=` 为 `auto`，systemd 会将日志记录写入 `/var/log/journal`。若被删除，systemd 不会自动创建此目录，而是将日志写入 `/run/log/journal`，重启时内容会消失。如果 `journald.conf` 中的 `Storage=persistent`，`systemd-journald.service` 重启或系统重启时会重新创建 `/var/log/journal/`。

Systemd 日志可以按照#优先级和#功能标记日志，符合 syslog 协议 (RFC 5424) 标准。

提示：如果 `/var/log/journal/` 位于 btrfs 文件系统，应该考虑对这个目录禁用写入时复制，方法参阅 Btrfs#Copy-on-Write (CoW))。

优先级

日志会带 syslog 优先级代码，标记消息的重要性，参考 RFC 5424 6.2.1。

数值	优先级	关键字	描述	示例
0	Emergency	emerg	System is unusable	Severe Kernel BUG, systemd dumped core. This level should not be used by applications.
1	Alert	alert	Should be corrected immediately	Vital subsystem goes out of work. Data loss. `kernel: BUG: unable to handle kernel paging request at ffffc90403238ffc`.
2	Critical	crit	Critical conditions	Crashes, coredumps. Like familiar flash: `systemd-coredump: Process 25310 (plugin-containe) of user 1000 dumped core` Failure in the system primary application, like X11.
3	Error	err	Error conditions	Not fatal error reported: `kernel: usb 1-3: 3:1: cannot get freq at ep 0x84`, `systemd: Failed unmounting /var.`, `libvirtd: internal error: Failed to initialize a valid firewall backend`).
4	Warning	warning	May indicate that an error will occur if action is not taken.	A non-root file system has only 1GB free. `org.freedesktop.Notifications: (process:5999): Gtk-WARNING **: Locale not supported by C library. Using the fallback 'C' locale`.
5	Notice	notice	Events that are unusual, but not error conditions.	`systemd: var.mount: Directory /var to mount over is not empty, mounting anyway. gcr-prompter: Gtk: GtkDialog mapped without a transient parent. This is discouraged`.
6	Informational	info	Normal operational messages that require no action.	`lvm: 7 logical volume(s) in volume group "archvg" now active`.
7	Debug	debug	Messages which may need to be enabled first, only useful for debugging	`kdeinit5: powerdevil: Scheduling inhibition from ":1.14" "firefox" with cookie 13 and reason "screen"`

These rules are recommendations, and the priority level of a given error is at the application developer's discretion. It is always possible that the error will be at a higher or lower level than expected.

Examples:

- Info message:

```
    pulseaudio: W: [pulseaudio] alsa-mixer.c: Volume element Master has 8 channels.
That's too much! I can't handle that!
```
It is an warning or error by definition.

- Plaguing alert message:

```
    sudo:     user : a password is required ; TTY=pts/0 ; PWD=/home/user ; USER=root ;
COMMAND=list /usr/bin/pacman --color auto -Sy
```
The reason - user was manually added to sudoers file, not to wheel group, which is arguably normal action, but sudo produced an alert on every occasion.

功能

日志文件会带上 syslog 功能码，标记发出日志的程序类型 RFC 5424 6.2.1。

功能码	关键字	描述	信息
0	kern	kernel messages	
1	user	user-level messages	
2	mail	mail system	Archaic POSIX still supported and sometimes used system, for more mail(1))
3	daemon	system daemons	All daemons, including systemd and its subsystems
4	auth	security/authorization messages	Also watch for different facility 10
5	syslog	messages generated internally by syslogd	As it standartized for syslogd, not used by systemd (see facility 3)
6	lpr	line printer subsystem (archaic subsystem)	
7	news	network news subsystem (archaic subsystem)	
8	uucp	UUCP subsystem (archaic subsystem)	
9		clock daemon	systemd-timesyncd
10	authpriv	security/authorization messages	Also watch for different facility 4
11	ftp	FTP daemon	
12	-	NTP subsystem	
13	-	log audit	
14	-	log alert	
15	cron	scheduling daemon	
16	local0	local use 0 (local0)	
17	local1	local use 1 (local1)	
18	local2	local use 2 (local2)	
19	local3	local use 3 (local3)	
20	local4	local use 4 (local4)	
21	local5	local use 5 (local5)	
22	local6	local use 6 (local6)	
23	local7	local use 7 (local7)	

So, useful facilities to watch: 0,1,3,4,9,10,15.

过滤输出

`journalctl`可以根据特定字段过滤输出。如果过滤的字段比较多，需要较长时间才能显示出来。

示例：

- 显示 `*PATTERN*` 模式的日志:

```
# journalctl --grep=PATTERN
```

- 显示本次启动后的所有日志：

```
# journalctl -b
```

- `journalctl -b -0` 显示本次启动的信息
- `journalctl -b -1` 显示上次启动的信息
- `journalctl -b -2` 显示上上次启动的信息 `journalctl -b -2`
- 只显示错误、冲突和重要告警信息

```
# journalctl -p err..alert
```

也可以使用数字， `journalctl -p 3..1`。If single number/keyword used, `journalctl -p 3` - all higher priority levels also included.

- 包含日志消息类型的描述：

```
# journalctl -x
```

注意在报告 bug 和寻求帮助时不要使用此命令，因为这个命令会产生大量的输出。用 `journalctl --list-catalog` 可以查看所有类型的描述。

- 显示从某个日期 (或时间) 开始的消息:

```
# journalctl --since="2012-10-30 18:17:16"
```

- 显示从某个时间 (例如 20分钟前) 的消息:

```
# journalctl --since "20 min ago"
```

- 显示最新信息

```
# journalctl -f
```

- 显示特定程序的所有消息:

```
# journalctl /usr/lib/systemd/systemd
```

- 显示特定进程的所有消息:

```
# journalctl _PID=1
```

- 显示指定单元的所有消息：

```
# journalctl -u man-db.service
```

- Show all messages from user services by a specific unit:

```
$ journalctl --user -u dbus
```

- 显示内核环缓存消息r:

```
# journalctl -k
```

- Show auth.log equivalent by filtering on syslog facility:

```
# journalctl -f -l SYSLOG_FACILITY=10
```

- If your journal directory (by default located under `/var/log/journal`) contains huge amount of log data then `journalctl` can take several minutes in filtering output. You can speed it up significantly by using `--file` option to force `journalctl` to look only into most recent journal:

```
    # journalctl --file /var/log/journal/*/system.journal -f
```

详情参阅journalctl(1)、systemd.journal-fields(7)，以及 Lennert 的这篇博文。

- By default, *journalctl* truncates lines longer than screen width, but in some cases, it may be better to enable wrapping instead of truncating. This can be controlled by the `SYSTEMD_LESS` environment variable, which contains options passed to less (the default pager) and defaults to `FRSXMK` (see less(1) and journalctl(1) for details).

: By omitting the `S` option, the output will be wrapped instead of truncated. For example, start *journalctl* as follows:

```
$ SYSTEMD_LESS=FRXMK journalctl
```

: To set this behaviour as default, export the variable from `~/.bashrc` or `~/.zshrc`.

- While the journal is stored in a binary format, the content of stored messages is not modified. This means it is viewable with *strings*, for example for recovery in an environment which does not have *systemd* installed, e.g.:{{bc|$ strings /mnt/arch/var/log/journal/af4967d77fba44c6b093d0e9862f6ddd/system.journal | grep -i *message*

日志大小限制

如果按上面的操作保留日志的话，默认日志最大限制为所在文件系统容量的 10%，即：如果 `/var/log/journal` 储存在 50GiB 的根分区中，那么日志最多存储 5GiB 数据。用 `systemd-journald` 日志查看当前设置:

```
# journalctl -b -u systemd-journald
```

可以修改配置文件指定最大限制。如限制日志最大 50MiB：

```
/etc/systemd/journald.conf
SystemMaxUse=50M
```

还可以通过配置片段而不是全局配置文件进行设置：

```
/etc/systemd/journald.conf.d/00-journal-size.conf
[Journal]
SystemMaxUse=50M
```

修改配置后要立即生效，请重启 `systemd-journald.service` 服务。

详情参见 journald.conf(5).

配合 syslog 使用

systemd 提供了 socket `/run/systemd/journal/syslog`，以兼容传统日志服务。所有系统信息都会被传入。要使传统日志服务工作，需要让服务链接该 socket，而非 `/dev/log`（官方说明）。Arch 软件仓库中的 syslog-ng包 已经包含了需要的配置。

`journald.conf` 使用 `no` 转发socket . 为了使 *syslog-ng* 配合 *journald* , 你需要在 `/etc/systemd/journald.conf` 中设置 `ForwardToSyslog=yes` . 参阅 Syslog-ng#Overview 了解更多细节.

如果你选择使用 rsyslogAUR , 因为 rsyslog 从日志中直接传出消息,所以不再必要改变那个选项..

设置开机启动 syslog-ng：

```
# systemctl enable syslog-ng
```

这里有一份很不错的 `journalctl` 指南。

Per unit size limit by a journal namespace

Edit the unit file for the service you wish to configure (for example sshd) and add `LogNamespace=ssh` in the `[Service]` section.

Then create `/etc/systemd/journald@ssh.conf` by copying `/etc/systemd/journald.conf`. After that, edit `journald@ssh.conf` and adjust `SystemMaxUse` to your liking.

Restarting the service should automatically start the new journal service `systemd-journald@ssh.service`. The logs from the namespaced service can be viewed with `journalctl --namespace ssh`.

See systemd-journald.service(8) § JOURNAL NAMESPACES for details about journal namespaces.

手动清理日志

`/var/log/journal` 存放着日志, `rm` 应该能工作. 或者使用`journalctl`,

例如:

- 清理日志使总大小小于 100M:

```
# journalctl --vacuum-size=100M
```

- 清理最早两周前的日志.

```
# journalctl --vacuum-time=2weeks
```

Journal files must have been rotated out and made inactive before they can be trimmed by vacuum commands. Rotation of journal files can be done by running `journalctl --rotate`. The `--rotate` argument can also be provided alongside

one or more vacuum criteria arguments to perform rotation and then trim files in a single command.

参阅 journalctl(1) 获得更多信息.

转发 journald 到 /dev/tty12

建立一个 Systemd#替换单元文件|drop-in directory\]\] `/etc/systemd/journald.conf.d` 然后在其中建立 `fw-tty12.conf`：

```
/etc/systemd/journald.conf.d/fw-tty12.conf
[Journal]
ForwardToConsole=yes
TTYPath=/dev/tty12
MaxLevelConsole=info
```

然后重新启动 systemd-journald.

查看特定位置的日志

有时你希望查看另一个系统上的日志.例如从 Live 环境修复现存的系统.

这种情况下你可以挂载目标系统 (例如挂载到 `/mnt`),然后用 `-D`/`--directory` 参数指定目录,例如:

```
# journalctl -D /mnt/var/log/journal -n 1000
```

普通用户访问日志

在默认的配置中，普通用户仅能访问自己的日志，要让普通用户访问系统日志，可以将用户加入 `systemd-journal` 用户组。`adm` 和 `wheel` 组中的用户也可以读取日志。

更多信息请参考 journalctl(1) § DESCRIPTION 和用户和用户组#用户组管理。

模块

本章只处理一件事：Capi4hylafax 的安装与配置。安装环节决定这个软件在系统里怎样落地，配置环节决定它能否按预期运转，两段前后相扣，前一环留下的文件位置与默认值正是后一环的起点。

本章仅此一节，没有可跳的部分，从头读到尾即可。最容易踩的坑是把顺序做反：没装好就先去改配置，对着并不存在的路径和参数较劲；或者把装完当成配完，包一装上就以为通了。另一处误解是照抄别人的配置值而不核对本机环境，每处看着都对，整体却不工作。建议先顺安装流程走一遍，再回到配置逐项核对，每改一处都清楚它对应什么。

19.1　Capi4hylafax

难度 入门 | **篇幅** 1.4 KB | **所属** 模块

本节将讲到

- 安装
- 配置

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： 暂无替代方案 (在Talk:Capi4hylafax 讨论)

Capi4hylafax 是一个用于 Hylafax 的 CAPI 插件，用于在 CAPI 2.0 设备上启用 ISDN 传真。

安装

安装 capi4hylafax包\[损坏的链接：package not found\] 包。

配置

以 root 用户身份运行 *faxsetup* 以根据需要调整 hylafax。根据需要进一步调整 `/var/spool/hylafax/etc/config/config.faxCAPI`。

警告：甚至不要尝试为 CAPI 2.0 设备运行 *faxaddmodem*！

c2faxaddmodem 是配置 ISDN 卡的不错的工具。

启动 `hylafax.service` 和 `c2faxrecv.service`。

确保您的设备对 `uucp` 用户组具有正确的权限。

将以下行添加到 `/var/spool/hylafax/etc/config`：

```
SendFaxCmd: /usr/bin/c2faxsend
```

注意：如果您需要多个 ISDN 控制器，请阅读 capi4hylafax 手册。

有关提示和技巧，请参见 Hylafax。

监控

监控这一章解决的是让一台 Arch 机器替人盯着画面：把摄像头接进来，交给 Motion 按你的规则判断并作出反应，而不是靠人守在屏幕前。本章只有一节，安装与配置都必读，没有可跳过的地方——安装决定 Motion 能否跑起来，配置决定它跑起来后是否照你的意图工作，按这个顺序读完即可。最容易踩的坑是把 Motion 当成打开就能用的图形程序：它的行为完全由配置决定，装好只说明程序在位，不等于已按你的方式工作；配置里写了并不存在的路径或设备，也会被误当成装错。遇到没反应，先核对配置，再怀疑安装。

20.1　Motion

难度 入门 | **篇幅** 1.7 KB | **所属** 监控

本节将讲到

- 安装
- 配置

Motion 是监视来自摄像机的视频信号的程序。它能够检测图像的重要部分是否已更改；换句话说，它可以检测运动。

安装

安装 motion包 软件包。

配置

可以使用 `-c` 选项调用 Motion 来定义特定的配置文件（与 motionEye 一样）。

如果不指定 `-c` 或给 Motion 的文件名不存在，Motion 将按以下顺序搜索名为 `motion.conf` 的配置文件：

1. 调用 motion 的当前目录
2. 当前用户主目录（shell 环境变量 `$HOME`）中的目录 `.motion`。例如 `~/.motion/motion.conf`
3. `/etc/motion/`

默认配置文件（`/etc/motion/motion.conf`）带有注释，因此很容易找到所需的选项以及如何设置（例如 `rotate 90`）。

默认情况下，Motion 将使用端口 `8080`（每个连接的摄像机则 +1），对 Web 配置服务器（端口 `8080`）和摄像机视频流的访问仅限于本地连接。